

Rust 程式設計語言

由 Steve Klabnik 與 Carol Nichols，以及 Rust 社群的貢獻撰寫而成

此版本假設你使用的是 Rust 1.65（於 2022-11-03 發布）或更高的版本，並在所有專案中的 `Cargo.toml` 都有 `edition="2021"` 來使用 Rust 2021 版號。請查看第一章的「安裝」段落來安裝或更新 Rust。

本書的 HTML 格式可以在線上閱讀：<https://doc.rust-lang.org/stable/book/>（正體中文版）。而離線版則包含在 `rustup` 安裝的 Rust 中，輸入 `rustup docs --book` 就能開啟。

社群中也有提供本書的各種譯本。

本書也有由 [No Starch Press](#) 出版平裝與電子版格式。

 想要更有互動的學習體驗？來嘗試不同的 Rust Book，賣點有：隨堂測驗、重點提示、視覺化呈現，更多都在 <https://rust-book.cs.brown.edu>

-
- commit: [3f64052](#)
-

前言

雖然不是那麼明確，但 Rust 程式設計語言旨在**賦權 (empowerment)**：無論你現在寫的是何種程式碼，Rust 都能賦予你更多能力，在更廣泛的領域中帶有自信地向前邁進。

比方說，「系統層級」的工作會需要處理低階的記憶體管理、資料呈現與程序並行的細節。對以往來說，這塊程式領域被視為巫術般的存在，只有投入好幾年時間學習的選中之人才有辦法駕馭，也才能懂得如何避開其惡名昭彰的陷阱。而且就算是這個領域的實作者也謹慎行事，也生怕他們的程式碼會出現漏洞、崩潰或損壞。

Rust 破除了這些障礙，消除了以往的陷阱並提供友善全面的工具來協助你。想要「深入」底層控制的程式設計師可以使用 Rust，無需冒著常見的崩潰或安全漏洞風險，也無需學習得經常改變的工具鏈其中的細節。更好地是，這個語言本身就是設計成能引導你自然而然地寫出可靠且在速度與記憶體使用都十分高效的程式碼。

已經在處理底層程式碼的程式設計師可以使用 Rust 來擴展他們的野心。舉例來說，在 Rust 中進行平行化是相對低風險的動作，編譯器會幫你抓到典型的錯誤。而且你也能更有自信且更積極地進行最佳化，不必擔心會意外造成崩潰或漏洞。

但 Rust 並不只限於底層的系統程式設計。其表現力與易用易讀的程度能令人愉快地寫出 CLI 應用程式、網頁伺服器與許多其他種程式碼。你會在本書中看到這兩種簡單的範例。使用 Rust 能這讓你將一個領域所學到的技能延伸到另一個領域，你可以透過寫網頁應用程式來學習 Rust，然後應用同樣的技能到你的樹莓派 (Raspberry Pi) 上。

本書充分體現了 Rust 賦予其使用者更多能力的潛力。其內容平易近人，不止致力於協助你提升 Rust 的知識，還能提升讓你身為程式設計師的整體信心。所以讓我們準備開始學習吧，歡迎加入 Rust 的社群！

— Nicholas Matsakis 與 Aaron Turon

介紹

注意：本書的（英文）版本與出版的 [The Rust Programming Language](#) 以及電子書版本的 [No Starch Press](#) 一致。

歡迎閱讀 **Rust 程式設計語言**，這是一本 Rust 的入門書籍。Rust 程式設計語言能幫助你寫出更快更可靠的軟體。在設計程式語言時，「上層的易讀易用性」與「底層的掌控性」經常難以取捨。Rust 直接挑戰這個矛盾。Rust 在強大的技術能力與良好的開發者體驗之間取得平衡，讓你能控制底層的實作細節（比如記憶體使用），但免於以往這樣的控制所帶來的相關麻煩。

Rust 適用於誰

Rust 的各種特長讓它適用於很多人，我們來討論一些最重要的客群。

開發團隊

Rust 被認定是一個有生產力的工具，讓能力不均的大型系統程式設計團隊能夠協同開發。底層程式碼容易產生難以察覺的錯誤，在多數其他語言中，只能靠大量的測試、以及經驗豐富的開發者小心翼翼地審核程式碼，才能找出它們。而在 Rust 中，編譯器扮演著守門員的角色阻擋這些難以捉摸的程式錯誤，包含並行（concurrency）的錯誤。透過與編譯器一同合作，開發團隊可以將他們的時間專注在程式邏輯，而不是成天追著錯誤跑。

Rust 也將一些現代化的開發工具帶入系統程式設計的世界中：

- Cargo 是個管理依賴函式庫與建構的工具，讓新增、編譯與管理依賴函式庫變得十分輕鬆，並在 Rust 生態系統維持一致性。
- Rustfmt 工具確保開發者遵循統一的程式碼風格。
- Rust Language Server 為整合開發環境（IDE）提供了程式碼補全與行內錯誤訊息。

透過使用這些工具、以及其他 Rust 生態系統中的工具，開發者在寫系統層級的程式時更有生產力。

學生

Rust 適合學生以及那些對學習系統概念有興趣的人。許多人透過 Rust 學會相關主題，例如作業系統開發。社群的人都非常友善，且樂於解答學習者的問題。如同對本書的投入，Rust 團隊致力於讓系統概念深入人心，尤其是剛開始學習程式設計的人。

公司

已有大大小小數以百計的公司，在正式生產環境中使用 Rust 來處理各種任務，包含命令列工具、網路服務、DevOps 工具、嵌入式裝置、影音分析與轉碼、加密貨幣、生物資訊、搜尋引擎、物聯網應用、機器學習，甚至是 Firefox 瀏覽器的主要部分。

開源開發者

Rust 適用於想要建構 Rust 程式設計語言、社群、開發工具與函式庫的開發者。我們很樂於看到你願意對 Rust 語言貢獻。

重視速度與穩定性的開發者

Rust 適用於追求語言速度與穩定性的開發者。所謂的速度，我們指的是 Rust 程式碼的執行速度以及 Rust 讓你能開始撰寫程式碼的速度。Rust 編譯器的檢查能確保新增功能與重構時的穩定性。這與沒有這些檢查的語言形成對比，開發者通常不敢修改其脆弱的遺留程式碼。Rust 還力求無成本抽象化 (zero-cost abstractions)，高階的特性編譯成底層程式碼後，執行的速度能像手刻一樣快。Rust 致力於讓安全的程式碼同時也能是執行迅速的程式碼。

這裡提及的僅是一部分的最大受益者，Rust 語言期許能支援更多其他使用者。總體來說，Rust 最大的野心是消除數十年來開發者不得不作出的取捨，像是提供安全性與生產力、具有速度又易讀易用。歡迎嘗試 Rust，看看這門語言適不適合你。

本書寫給誰看

本書假設你已經使用其他程式語言寫過程式碼，無論哪種語言。我們會試著讓不同背景的開發者能理解內容，但我們不會花費很多篇幅談論什麼是程式設計或教你怎麼寫程式。如果你剛開始學習程式語言，一本專門介紹程式設計的書籍會更適合你。

如何閱讀本書

一般來說，本書預設你會從前往後依序閱讀。後面的章節建立在前面提到的概念上，並且前面的章節不會深入某特定主題，而於後面的章節再議。

你會發現本書有兩種類型的章節：概念章節與專案章節。在概念章節中，你會學到 Rust 的某些概念。在專案章節中，我們會一起應用當前所學來做小專案。第二、十二和二十章是專案章節，其餘是概念章節。

第一章會解釋如何安裝 Rust、如何寫支「Hello, world!」程式以及如何使用 Cargo —— Rust 的套件管理與建構工具。第二章透過實作一款猜數字遊戲的程式來介紹 Rust。我們在此提及大概的觀

念，並在之後的章節提供更詳細的介紹。如果你想馬上動手實作看看的話，第二章會很適合你。第三章會涵蓋 Rust 與其他程式設計語言類似的功能。第四章則會學習 Rust 的所有權系統。如果你是嚴謹派的讀者、傾向先學習所有細節再進入實作，你可能會想跳過第二章直接前往第三章。當你想要應用學到的細節時，再回到第二章練習。

第五章討論結構體與方法，而第六章涵蓋列舉、`match` 表達式與 `if let` 控制流程的語法。你會在 Rust 中用結構體與列舉來自訂型別。

在第七章中，你會學到 Rust 的模組系統與隱私規則，讓你可以組織程式碼以及其公開應用程式介面 (Application Programming Interface, API)。第八章會討論標準函式庫提供的一些常見集合資料結構，像是向量、字串與雜湊映射。第九章會探討 Rust 的錯誤處理哲學與技巧。

第十章將深入探討泛型、特徵 (traits) 與生命週期，讓你能定義出能套用多種型別的程式碼。第十一章都在討論測試，就算有 Rust 的安全性保障，還是必須透過測試來確保你的程式邏輯正確。在第十二章中，我們會動手實作 `grep` 命令列工具的部分功能，可以搜尋檔案中的文字。我們將會應用前幾章討論過的許多概念。

第十三章會探索閉包與疊代器，這是 Rust 借鑒函式程式設計語言的功能。在第十四章中，我們要更深入研究 Cargo 並討論分享函式庫給其他人的最佳方式。第十五章會討論標準函式庫提供的智慧指標以及能啟用它們功能的特徵 (traits)。

在第十六章中，我們會介紹各種不同的並行程式設計模型，並談論 Rust 如何幫助你無懼地開發多執行緒的程式。第十七章會拿 Rust 的慣用風格與你可能較熟悉的物件導向程式設計原則作比較。

第十八章涉及模式與模式配對，它們的強大力量讓你能用 Rust 表達更多概念。第十九章是進階主題的大雜燴，其中包含：不安全 (unsafe) 的 Rust、巨集、以及更多關於生命週期、特徵、型別、函式與閉包的介紹。

在第二十章中，我們會完整實作一個底層跑多執行緒的網頁伺服器！

最後，以參照的方式收錄本語言的一些實用資訊。附錄 A 涵蓋 Rust 的關鍵字、附錄 B 涵蓋 Rust 的運算子與符號、附錄 C 涵蓋標準函式庫提供的可推導的特徵 (derivable traits)、附錄 D 涵蓋一些實用開發工具，然後附錄 E 會解釋 Rust 的版號。在附錄 F 中你可以找到本書籍的各種翻譯版本，然後在附錄 G 我們會講解 Rust 的開發流程以及什麼是每夜版 (Nightly) Rust。

本書沒有錯誤的閱讀方式——如果你想要跳過一些章節，儘管跳過吧！後面也許會遇到不懂的地方而需要回頭去看。總之用最適合自己的方式閱讀。

學習 Rust 的過程中有個重要的部分——學習如何閱讀編譯器顯示的錯誤訊息，讓訊息引導你寫出正確的程式碼。因此，我們特意提供很多無法編譯的範例，以及編譯器對應顯示的錯誤訊息。如果你隨意挑一則範例執行的話，它可能無法編譯喔！請務必看看範例旁的文字來瞭解該範例是不是故意出錯。可愛的吉祥物 Ferris 也能幫助你分辨哪些程式碼本來就無法運作：

Ferris	意思
	此程式碼無法編譯！

Ferris	意思
	此程式碼會恐慌！
	此程式碼不會產生預期的行為。

在大多數的情況下，我們會引導你將無法編譯的程式碼寫成正確的版本。

原始碼

產生本書的原始檔案可以在 [GitHub](#) 上找到。

開始入門

讓我們開始你的 Rust 旅途吧！千里之行始於足下，在此章節我們將討論：

- 在 Linux、macOS 和 Windows 上安裝 Rust
- 寫一支印出 `Hello, world!` 的程式
- 使用 Rust 的套件管理工具暨建構系統 `cargo`

安裝教學

第一步是安裝 Rust，我們將會透過 `rustup` 安裝 Rust，這是個管理 Rust 版本及相關工具的命令列工具。你將會需要網路連線才能下載。

注意：如果你基於某些原因不想使用 `rustup` 的話，請前往 [Rust 其他安裝方法的頁面](#) 尋求其他選項。

以下步驟將會安裝最新的穩定版 Rust 編譯器。Rust 的穩定性能確保本書的所有範例在更新的 Rust 版本仍然能繼續編譯出來。輸出的結果可能會在不同版本間而有些微的差異，因為 Rust 時常會改善錯誤與警告訊息。換句話說，任何你所安裝的最新穩定版 Rust 都應該能夠正常運行本書的內容。

命令列標記

在本章節到整本書為止，我們將會顯示一些終端機會用到的命令。任一你會用到的命令都會始於 `$`。但你不需要去輸入 `$`，因為這通常代表每一命令列的起始位置。而沒有出現 `$` 的行數，通常則代表前一行命令列輸出的結果。除此之外，針對 PowerShell 的範例則將會使用 `>` 而不是 `$`。

在 Linux 或 macOS 上安裝 `rustup`

如果你使用的是 Linux 或 macOS，請開啟終端機然後輸入以下命令：

```
$ curl --proto '=https' --tlsv1.3 https://sh.rustup.rs -sSf | sh
```

這道命令會下載一支腳本然後開始安裝 `rustup` 工具，接著安裝最新的穩定版 Rust。下載過程中可能會要求你輸入你的密碼。如果下載成功的話，將會出現以下內容：

```
Rust is installed now. Great!
```

你還會需要一個**連結器 (linker)** 來讓 Rust 將編譯好的輸出資料整理到一個檔案內。通常你很可能已經有安裝了，但如果你遇到連結器相關的錯誤時，這代表你需要安裝一個 C 編譯器，因為它通常都會帶有一個的連結器。有個 C 編譯器通常也很實用，因為一些常見的 Rust 套件也會依賴於 C 而需要一個 C 編譯器。

在 macOS 上，你可以輸入以下命令來安裝 C 編譯器：

```
$ xcode-select --install
```

Linux 使用者的話則需要依據他們的發行版文件來安裝 GCC 或 Clang。舉例來說，如果你使用 Ubuntu 的話，你可以安裝 `build-essential` 套件。

在 Windows 上安裝 `rustup`

在 Windows 上請前往[下載頁面](#)並依照指示安裝 Rust。在安裝的某個過程中，你將會看到一個訊息要求說你還需要 C++ build tools for Visual Studio 2013 或更新的版本。

要取得 build tools 的話，你需要安裝 [Visual Studio 2022](#)。當你被問到要安裝哪些時，請記得包含：

- “Desktop Development with C++”
- The Windows 10 or 11 SDK
- The English language pack component（以及其他你想選擇的語言包）

本書接下來使用的命令都相容於 `cmd.exe` 和 PowerShell。如果有特別不同的地方，我們會解釋該怎麼使用。

疑難排除

想簡單確認你是否有正確安裝 Rust 的話，請開啟 shell 然後輸入此命令：

```
$ rustc --version
```

你應該會看到已發佈的最新穩定版本號、提交雜湊（hash）以及提交日期如以下格式所示：

```
rustc x.y.z (abcabcabc yyyy-mm-dd)
```

如果你看到這則訊息代表你成功安裝 Rust 了！如果你沒有看到的話，請如下檢查 Rust 是否在你的 `%PATH%` 系統變數裡。

在 Windows CMD 中請使用：

```
> echo %PATH%
```

在 PowerShell 中請使用：

```
> echo $env:Path
```

在 Linux 和 macOS 的話請使用：

```
$ echo $PATH
```

如果以上步驟皆正確無誤，但還是無法執行 Rust 的話，你可以前往一些地方尋求協助。例如您可以前往[社群頁面](#)聯絡其他 Rustaceans（這是我們常用稱呼自己取的暱稱）交談並取得協助。

更新與解除安裝

當你透過 `rustup` 安裝完 Rust 後，要更新到最新版本的方法非常簡單。在你的 shell 中執行以下更新腳本即可：

```
$ rustup update
```

要解除安裝 Rust 與 `rustup` 的話，則在 shell 輸入以下解除安裝腳本：

```
$ rustup self uninstall
```

本地端技術文件

安裝 Rust 的同時也會包含一份本地的技術文件副本，讓你可以離線閱讀。執行 `rustup doc` 就可以用你的瀏覽器開啟本地文件。

每當有任何型別或函式出現而你卻不清楚如何使用時，你就可以閱讀應用程式介面（API）技術文件來理解！

Hello, World!

現在你已經安裝好 Rust，是時候開始寫你的第一支 Rust 程式。當我們學習一門新的語言時，有一個習慣是寫一支印出「Hello, world!」到螢幕上的小程序，此章節將教你做一樣的事！

注意：本書將假設你已經知道命令列最基本的使用方法。Rust 對於你的編輯器、工具以及程式碼位於何處沒有特殊的要求，所以如果你更傾向於使用整合開發環境（IDE）的話，請儘管使用你最愛的 IDE。許多 IDE 都已經針對 Rust 提供某種程度的支援，請查看你所使用的 IDE 技術文件以瞭解詳情。Rust 團隊正透過 `rust-analyzer` 積極提升 IDE 的支援，請查 [附錄 D](#) 來了解更多細節。

建立專案目錄

你將先建立一個目錄來儲存你的 Rust 程式碼。程式碼位於何處並不重要，但為了能好好練習書中的範例和專案，我們建議你可以在你的 home 目錄建立一個 *projects* 目錄然後將你所有的專案保存在此。

請開啟終端機然後輸入以下命令來建立 *projects* 目錄和另一個在 *projects* 目錄底下的真正要寫「Hello, world!」專案的目錄。

對於 Linux、macOS 和 Windows 的 PowerShell，請輸入：

```
$ mkdir ~/projects
$ cd ~/projects
$ mkdir hello_world
$ cd hello_world
```

對於 Windows CMD，請輸入：

```
> mkdir "%USERPROFILE%\projects"
> cd /d "%USERPROFILE%\projects"
> mkdir hello_world
> cd hello_world
```

編寫並執行 Rust 程式

接著，請產生一個全新原始碼檔案並命名為 *main.rs*。Rust 的文件檔案都會以 *.rs* 副檔名稱作為結尾。如果你用到不只一個單字的話，慣例上是用底線區隔開來。比方說，請使用 *hello_world.rs* 而不是 *helloworld.rs*。

現在請開啟 `main.rs` 檔案然後輸入範例 1-1 中的程式碼。

檔案名稱：`main.rs`

```
fn main() {  
    println!("Hello, world!");  
}
```

範例 1-1：印出「Hello, world!」的程式

儲存檔案然後回到你的專案目錄底下 `~/projects/hello_world`。在 Linux 或 macOS 上，請輸入以下命令來編譯並執行檔案：

```
$ rustc main.rs  
$ ./main  
Hello, world!
```

在 Windows 上則輸入 `.\main.exe` 而非 `./main`：

```
> rustc main.rs  
> .\main.exe  
Hello, world!
```

不管你的作業系統為何，終端機上應該都會出現 `Hello, world!`。如果你沒有看到，可以回到安裝章節中的「[疑難排除](#)」尋求協助。

如果 `Hello, world!` 有印出來，那麼恭喜你！你正式寫了一支 Rust 程式，所以你也正式成為 Rust 開發者了——歡迎加入！

分析這支 Rust 程式

讓我們來仔細瞧瞧你的「Hello, world!」程式。這是第一塊拼圖：

```
fn main() {  
  
}
```

這幾行定義了一個 `main` 函式。`main` 是一個特別的函式：它是每個可執行的 Rust 程式永遠第一個執行的程式碼。第一行宣告了一個函式 `main`，它沒有參數也不回傳任何東西。如果有參數的話，它們會被加進括號 `()` 內。

函式本體被囊括在大括號 `{ }` 內，Rust 要求所有函式都用大括號包起來。一般來說，良好的程式碼風格會要求將前大括號置於宣告函式的同一行，並用一個空格區隔開來。

如果你想要在不同 Rust 專案之間統一標準風格的話，`rustfmt` 可以格式化你的程式成特定的風格（更多 `rustfmt` 資訊請詳見[附錄 D](#)）。Rust 團隊已經將此工具納入標準 Rust 發行版中，就像 `rustc` 一樣，它應該已經安裝到你的電腦上了！

在 `main` 函式本體中有以下程式碼：

```
println!("Hello, world!");
```

此行負責了整支程式要做的事：它將文字顯示在螢幕上。這邊有四個細節要注意。

首先，Rust 的排版風格是 4 個空格而非一個 tab。

第二，`println!` 會呼叫一支 Rust 巨集（macro）。如果是呼叫函式的話，那則會是 `println`（去掉 `!`）。我們會在第十九章討論更多巨集的細節。現在你只需要知道使用 `!` 代表呼叫一支巨集而非一個正常的函式，且該巨集遵守的規則不全都和函式一樣。

第三，`"Hello, world!"` 是一個字串，我們將此字串作為引數傳遞給 `println!`，然後該字串就會被顯示到螢幕上。

第四，我們用分號（`;`）作為該行結尾，代表此表達式的結束和下一個表達式的開始。多數的 Rust 程式碼都以分號做結尾。

編譯和執行是不同的步驟

你剛剛執行了一個新建立的程式，讓我們來檢查過程中的每一個步驟吧。

在你執行一支 Rust 程式前，你必須用 Rust 編譯器來編譯它，也就是輸入 `rustc` 命令然後加上你的原始檔案，像這樣子：

```
$ rustc main.rs
```

如果你已經有 C 或 C++ 的背景，你應該就會發現這和 `gcc` 或 `clang` 非常相似。編譯成功後，Rust 編譯器會輸出一個二進制執行檔（binary executable）。

在 Linux、macOS 和 Windows 上的 PowerShell，你可以在你的 shell 輸入 `ls` 來查看你的執行檔：

```
$ ls
main  main.rs
```

在 Linux 和 macOS，你會看到兩個檔案。而在 Windows 上的 PowerShell，你會和使用 CMD 一樣看到三個檔案。在 Windows 上的 CMD，你需要輸入：

```
> dir /B %= /B 選項代表只顯示檔案名稱 %=
main.exe
main.pdb
main.rs
```

這顯示了副檔名為 `.rs` 的原始碼檔案、執行檔（在 Windows 上為 `main.exe`；其他則為 `main`），然後在 Windows 上會再出現一個副檔名為 `.pdb` 的除錯資訊文件。在這裡，你就可以像這樣執行 `main` 或 `main.exe` 檔案：

```
$ ./main # 在 Windows 上則是 .\main.exe
```

如果你的 `main.rs` 正是你的「Hello, world!」程式，這命令就會顯示 `Hello, world!` 到你的終端機。

如果你比較熟悉動態語言，像是 Ruby、Python 或 JavaScript，你可能會比較不習慣將編譯與執行程式分為兩個不同的步驟。Rust 是一門**預先編譯**（ahead-of-time compiled）的語言，代表你可以編譯完成後將執行檔送到其他地方，然後他們就算沒有安裝 Rust 一樣可以執行起來。但如果你給某個人 `.rb`、`.py` 或 `.js` 檔案，他們就需要 Ruby、Python 或 Javascript 分別都有安裝好。當然你在這些語言只需要一行命令就可以執行，在語言設計中這一切都只是取捨。

在簡單的程式使用 `rustc` 來編譯不會有什麼問題，但當你的專案成長時，你將會需要管理所有選擇並讓程式碼易於分享。接下來我們將介紹 Cargo 這項工具給你，它將協助你寫出真正的 Rust 程式。

Hello, Cargo!

Cargo 是 Rust 的建構系統與套件管理工具。大部分的 Rustaceans 都會用此工具來管理他們的專案，因為 Cargo 能幫你處理很多任務，像是建構你的程式碼、下載你程式碼所需要的依賴函式庫並建構它們。我們常簡稱程式碼所需要用到的函式庫為**依賴 (dependencies)**。

簡單的 Rust 程式像是我們目前所寫的不會有任何依賴。當我們用 Cargo 建構「Hello, world!」專案時，Cargo 只會用到建構程式碼的那部分。隨著你寫的 Rust 程式越來越複雜，你將會加入一些依賴函式庫來幫助你。而如果你使用 Cargo 的話，加入這些依賴就會簡單很多。

既然大多數的 Rust 專案都是用 Cargo，所以接下來本書也將假設你也使用 Cargo。Cargo 在你使用「[安裝教學](#)」的官方安裝連結來安裝 Rust 時就已經連同安裝好了。如果你是用其他方式下載 Rust 的話，想要檢查 Cargo 有沒有下載好可以透過你的終端機輸入：

```
$ cargo --version
```

如果你有看到版本號，那就代表你有安裝了！如果你看到錯誤訊息，像是 `command not found`，請查看你的安裝辦法的技術文件，尋找如何額外下載 Cargo。

使用 Cargo 建立專案

讓我們來用 Cargo 建立一個專案，並來比較它和我們原本的「Hello, world!」專案有什麼差別。請回到你的 *projects* 目錄（或者任何你決定存放程式碼的地方），然後在任何作業系統上輸入：

```
$ cargo new hello_cargo
$ cd hello_cargo
```

第一道命令會建立一個新的目錄與專案叫做 *hello_cargo*。我們將我們的專案命名為 *hello_cargo*，然後 Cargo 就會產生相同名稱的目錄並產生所需的檔案。

進入 *hello_cargo* 然後顯示檔案的話，你會看到 Cargo 產生了兩個檔案和一個目錄：*Cargo.toml* 檔案以及一個 *src* 目錄，其內包含一個 *main.rs* 檔案。

它還會初始化成一個新的 Git repository 並附上 *.gitignore* 檔案。如果已經在 Git repository 內的話，執行 `cargo new` 則不會產生 Git 的檔案。你可以用 `cargo new --vcs=git` 覆寫這項行為。

注意：Git 是一個常見的版本控制系統。你可以加上 `--vcs` 來變更 `cargo new` 去使用不同的版本控制系統，或是不用任何版本控制系統。請執行 `cargo new --help` 來查看更多可使用的選項。

請用任何你喜歡的編輯器開啟 *Cargo.toml*，它應該會看起來和範例 1-2 差不多。

檔案名稱：Cargo.toml

```
[package]
name = "hello_cargo"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html

[dependencies]
```

範例 1-2：用 `cargo new` 產生的 *Cargo.toml*

此檔案用的是 *TOML* (*Tom's Obvious, Minimal Language*) 格式，這是 Cargo 配置文件的格式。

第一行的 `[package]` 是一個段落 (section) 標題，說明以下的陳述式 (statement) 會配置這個套件。隨著我們加入更多資訊到此文件，我們也會加上更多段落。

接下來三行就是 Cargo 編譯你的程式所需的配置資訊：名稱、版本、誰寫的以及哪個 Rust `edition` 會用到。我們會在附錄 E 介紹什麼是 `edition`。

最後一行 `[dependencies]` 是用來列出你的專案會用到哪些依賴的段落。在 Rust 中，程式碼套件會被稱為 *crates*。我們在此專案還不需要任何其他 crate。但是我們會在第二章開始用到，屆時我們會再來介紹。

現在請開啟 *src/main.rs* 來看看：

檔案名稱：src/main.rs

```
fn main() {
    println!("Hello, world!");
}
```

Cargo 預設會為你產生一個「Hello, world!」程式，就像我們範例 1-1 寫的一樣！目前我們寫的專案與 Cargo 產生的程式碼不同的地方在於 Cargo 將程式碼放在 *src* 目錄底下，而且我們還有一個 *Cargo.toml* 配置文件在根目錄。

Cargo 預期你的原始檔案都會放在 *src* 目錄底下。專案的根目錄是用來放 README 檔案、授權條款、配置檔案以及其他與你的程式碼不相關的檔案。使用 Cargo 能夠幫助你組織你的專案，讓一切井然有序。

如果你的專案還沒開始使用 Cargo 的話，像是我們剛剛寫的「Hello, world!」專案，你只要將程式碼移入 *src* 然後產生正確的 *Cargo.toml* 檔案，就可以將它轉換成能夠使用 Cargo 的專案。

建構並執行 Cargo 專案

現在讓我們看看用 Cargo 產生的「Hello, world!」程式在建構和執行時有什麼差別！請在你的 `hello_cargo` 目錄下輸入以下命令來建構專案：

```
$ cargo build
Compiling hello_cargo v0.1.0 (file:///projects/hello_cargo)
Finished dev [unoptimized + debuginfo] target(s) in 2.85 secs
```

此命令會產生一個執行檔 `target/debug/hello_cargo`（在 Windows 上則是 `target\debug\hello_cargo.exe`），而不是在你目前的目錄。因為預設的建構會是 debug build，Cargo 會將執行檔放進名為 `debug` 的目錄。你可以用以下命令運行執行檔：

```
$ ./target/debug/hello_cargo # 在 Windows 上的話則是
.\target\debug\hello_cargo.exe
Hello, world!
```

如果一切順利，`Hello, world!` 就會顯示在終端機上。第一次執行 `cargo build` 的話，還會在根目錄產生另一個新檔案：`Cargo.lock`。此檔案是用來追蹤依賴函式庫的確切版本。不過此專案沒有任何依賴，所以目前這個檔案看起來內容會有點少。你不會需要去手動更改此檔案，Cargo 會幫你管理這個檔案的內容。

我們剛用 `cargo build` 建構完專案並用 `./target/debug/hello_cargo` 執行它。不過我們其實也可以只用一道命令 `cargo run` 來編譯程式碼並接著運行產生的執行檔：

```
$ cargo run
Finished dev [unoptimized + debuginfo] target(s) in 0.0 secs
Running `target/debug/hello_cargo`
Hello, world!
```

使用 `cargo run` 通常比執行 `cargo build` 然後使用執行檔的完整路徑還要方便，所以多數開發者通常都直接使用 `cargo run`。

請注意到這次輸出的結果我們沒有看到 Cargo 有在編譯 `hello_cargo` 的跡象，這是因為 Cargo 可以知道檔案完全沒被更改過，所以它不用重新建構可以選擇直接執行執行檔。如果你有變更你的原始碼的話，Cargo 才會在執行前重新建構專案，你才會看到這樣的輸出結果：

```
$ cargo run
Compiling hello_cargo v0.1.0 (file:///projects/hello_cargo)
Finished dev [unoptimized + debuginfo] target(s) in 0.33 secs
Running `target/debug/hello_cargo`
Hello, world!
```

Cargo 還提供一道命令 `cargo check`，此命令會快速檢查你的程式碼，確保它能編譯通過但不會產生執行檔：

```
$ cargo check
Checking hello_cargo v0.1.0 (file:///projects/hello_cargo)
Finished dev [unoptimized + debuginfo] target(s) in 0.32 secs
```

為何你會不想要產生執行檔呢？這是因為 `cargo check` 省略了產生執行檔的步驟，所以它執行的速度比 `cargo build` 還來的快。如果你在寫程式時需要持續檢查的話，使用 `cargo check` 可以加快整體過程，讓你知道你的專案是否能編譯！所以許多 Rustaceans 都會在寫程式的過程中時不時執行 `cargo check` 來確保它能編譯。最後當他們準備好要使用執行檔時，才會用 `cargo build`。

讓我們來回顧我們目前學到的 Cargo 內容：

- 我們可以用 `cargo new` 產生專案。
- 我們可以用 `cargo build` 建構專案。
- 我們可以用 `cargo run` 同時建構並執行專案。
- 我們可以用 `cargo check` 建構專案來檢查錯誤，但不會產生執行檔。
- Cargo 會儲存建構結果在 `target/debug` 目錄底下，而不是放在與我們程式碼相同的目錄。

使用 Cargo 還有一項好處是在任何作業系統所使用的命令都是相同的，所以到這邊開始我們不再需要特別提供 Linux 和 macOS 相對於 Windows 不同的特殊命令。

建構發佈版本 (Release)

當你的專案正式準備好要發佈的話，你可以使用 `cargo build --release` 來最佳化編譯結果。此命令會產生執行檔到 `target/release` 而不是 `target/debug`。最佳化可以讓你的 Rust 程式碼跑得更快，不過也會讓編譯的時間變得更久。這也是為何 Cargo 提供兩種不同的設定檔 (profile)：一個用來作為開發使用，讓你可以快速並經常重新建構；另一個用來最終產生你要給使用者運行的程式用，它通常不會需要重新建構且能盡所能地跑得越快越好。如果你要做基準化分析 (benchmarking) 來檢測程式運行時間的話，請確認執行的是 `cargo build --release` 並使用 `target/release` 底下的執行檔做檢測。

將 Cargo 視為常規

雖然在簡單的專案下，Cargo 比起只使用 `rustc` 的確沒辦法突顯出什麼價值。但是當你的程式變得越來越複雜時，它將證明它的用途。當程式成長到好幾個檔案或需要依賴項目時，讓 Cargo 來協調你的專案會來的簡單許多。

儘管 `hello_cargo` 是個小專案，但它使用了你未來的 Rust 生涯中真實情況下會用到的工具。事實上，所有存在的專案，你幾乎都可以用以下命令完成：使用 Git 下載專案、移至專案目錄然後建構完成。

```
$ git clone example.org/someproject
$ cd someproject
$ cargo build
```

有關 Cargo 的更多資訊，請查看它的[技術文件](#)。

總結

你已經完成你的 Rust 旅途的第一步了！在本章節你學到了：

- 使用 `rustup` 安裝最新穩定版 Rust
- 更新到最新 Rust 版本
- 開啟本地端安裝的技術文件
- 直接使用 `rustup` 編寫並執行一支「Hello, world!」程式
- 使用 Cargo 建立並執行一個新專案

接下來是時候來建立一個更實際的程式來熟悉 Rust 程式碼的讀寫了。所以在第二章我們將寫出一支猜謎遊戲的程式。如果你想直接學習 Rust 的常見程式設計概念的話，你可直接閱讀第三章，之後再回來看第二章。

設計猜謎遊戲程式

讓我們親自動手一同完成一項專案來開始上手 Rust 吧！本章節會介紹一些常見 Rust 概念，展示如何在實際程式中使用它們。你會學到 `let`、`match`、方法、關聯函式、外部 crate 以及更多等等！我們會在之後的章節更詳細地探討這些概念。在本章中，你會練習到基礎概念。

我們會實作個經典新手程式問題：猜謎遊戲。它的運作方式如下：程式會產生 1 到 100 之間的隨機整數。接著它會通知玩家猜一個數字。在輸入猜測數字之後，程式會回應猜測的數字太低或太高。如果猜對的話，遊戲就會顯示祝賀訊息並關閉。

設置新專案

要設置新專案的話，前往你在第一章建立的 *projects* 目錄並使用 Cargo 建立一個新的專案，如下所示：

```
$ cargo new guessing_game
$ cd guessing_game
```

第一道命令 `cargo new` 會接收專案名稱（`guessing_game`）作為引數（argument）。第二道命令會將目錄移至新專案中。

檢查看看產生的 *Cargo.toml* 檔案：

檔案名稱：Cargo.toml

```
[package]
name = "guessing_game"
version = "0.1.0"
edition = "2021"

# See more keys and their definitions at https://doc.rust-
lang.org/cargo/reference/manifest.html

[dependencies]
```

如同你在第一章看到的，`cargo new` 會產生一支「Hello, world!」程式。請檢查 *src/main.rs* 檔案：

檔案名稱：src/main.rs


```
fn main() {  
    println!("Hello, world!");  
}
```

現在讓我們用 `cargo run` 命令同時完成編譯與執行「Hello, world!」程式：

```
$ cargo run  
Compiling guessing_game v0.1.0 (file:///projects/guessing_game)  
Finished dev [unoptimized + debuginfo] target(s) in 1.50s  
Running `target/debug/guessing_game`  
Hello, world!
```

`run` 命令在你需要對專案快速疊代時會很有用，我們要寫的遊戲也是如此，在繼續下一步之前可以快速測試每一步。

請重新開啟 `src/main.rs` 檔案。你要寫的程式碼全都會位於此檔案中。

處理猜測

猜謎遊戲的第一個部分會要求使用者輸入數字、處理該輸入，並檢查該輸入是否符合格式。所以我們要先讓玩家能夠輸入猜測數字，請輸入範例 2-1 的程式碼至 `src/main.rs`。

檔案名稱：`src/main.rs`

```
use std::io;  
  
fn main() {  
    println!("請猜測一個數字！");  
  
    println!("請輸入你的猜測數字。");  
  
    let mut guess = String::new();  
  
    io::stdin()  
        .read_line(&mut guess)  
        .expect("讀取該行失敗");  
  
    println!("你的猜測數字：{guess}");  
}
```

範例 2-1：取得使用者的猜測數字並顯示出來的程式

這段程式碼包含大量的資訊，所以讓我們一行一行來慢慢看吧。要取得使用者輸入並印出為輸出結果，我們需要將 `io` 輸入 / 輸出 (input/output) 函式庫引入作用域中。`io` 函式庫來自標準函式庫（常稱為 `std`）：

```
use std::io;
```

在預設情況下，Rust 會將一些在標準函式庫定義的型別引入每個程式的作用域中。這樣的集合稱為 **prelude**，你可以在[標準函式庫的技術文件中](#)看到這包含了那些型別。

如果你想使用的型別不在 prelude 的話，你需要顯式（explicit）地使用 `use` 陳述式（statement）將該型別引入作用域。`std::io` 函式庫能提供一系列實用的功能，這包含接收使用者輸入的能力。

如同你在第一章所見的，`main` 函式是程式的入口點（entry point）：

```
fn main() {
```

`fn` 語法用來宣告新的函式（function），其中括號 `()` 說明此函式沒有任何參數，然後大括號 `{` 會作為函式本體的開頭。

同樣如第一章所學的，`println!` 是個能將字串顯示到螢幕上的巨集：

```
println!("請猜測一個數字!");  
  
println!("請輸入你的猜測數字。");
```

此程式碼會顯式提示訊息向使用者說明此遊戲該輸入什麼。

透過變數儲存數值

接著我們要建立一個變數來儲存使用者輸入，如以下所示：

```
let mut guess = String::new();
```

現在程式變得越來越有趣了！在短短的這行當中有許多事情發生。先注意到我們使用了 `let` 陳述式建立了一個變數（variable）。以下是另一個例子：

```
let apples = 5;
```

這行建立了一個新的變數叫做 `apple` 並將數值 5 綁定給它。在 Rust 中，變數預設是不可變的（immutable），也就是一旦我們給予變數一個數值，該數值就不會被改變。我們會在第三章的「[變數與可變性](#)」段落討論此概念。要讓變數成為可變的話，我們可以在變數名稱前面加上 `mut`：

```
let apple = 5; // 不可變的  
let mut banana = 5; // 可變的
```

注意：`//` 語法用來產生註解（comment）直到該行結束。Rust 會忽略註解中所有內容，我們會在第三章進一步討論到。

讓我們回到猜謎遊戲程式，你現在就知道 `let mut guess` 會產生一個可變變數叫做 `guess`。等號（`=`）告訴 Rust 我們現在想綁定某個值給變數，而等號的另一邊就是要綁定給 `guess` 的數值，也就是呼叫 `String::new` 的結果，這是一個回傳新的 `String` 實例（instance）的函式。`String` 是個標準函式庫提供的字串型別，這是可增長的 UTF-8 編碼文字。

`::new` 中的 `::` 語法代表 `new` 是 `String` 型別的關聯函式。**關聯函式（associated function）** 是針對型別實作的函式，在此例中就是 `String`。此 `new` 函式建立一個新的空字串。你會在許多型別中找到 `new` 函式，因為這是函式建立某種新數值的常見名稱。

總結來說，`let mut guess = String::new();` 這行會建立一個可變變數，且目前會得到一個新的空 `String` 實例。

取得使用者輸入

回想一下我們在程式第一行透過 `use std::io;` 來包含標準函式庫中的輸入 / 輸出功能。現在我們要從 `io` 模組（module）呼叫 `stdin` 函式，讓我們能處理使用者的輸入：

```
io::stdin()
    .read_line(&mut guess)
```

如果我們沒有匯入 `io` 函式庫，也就是將 `use std::io` 這行置於程式最一開始的位置的話，我們還是能直接寫出 `std::io::stdin` 來呼叫函式。`stdin` 函式會回傳一個 `std::io::Stdin` 實例，這是代表終端機標準輸入控制代碼（handle）的型別。

接下來 `.read_line(&mut guess)` 這行會對標準輸入控制代碼呼叫 `read_line` 方法（method）來取得使用者的輸入。我們還傳遞了 `&mut guess` 作為引數（argument）給 `read_line`，來告訴它使用者輸入時該儲存什麼字串。整個 `read_line` 的任務就是取得使用者在標準輸入寫入的任何內容，並加入到字串中（不會覆寫原有內容），使得我們可以傳遞該字串作為引數。字串引數需要是可變的，這樣該方法才能變更字串的內容。

& 說明此引數是個**參考（reference）**，這讓程式中的多個部分可以取得此資料內容，但不需要每次都複製資料到記憶體中。參考是個複雜的概念，而 Rust 其中一項主要優勢就是能夠輕鬆又安全地使用參考。你現在還不用知道一堆細節才能完成程式。現在你只需要知道參考和變數一樣，預設都是不可變的。因此你必須寫 `&mut guess` 而不是 `&guess` 才能讓它成為可變的。（第四章會再全面詳細解釋參考。）

使用 `Result` 處理可能的錯誤

我們要繼續處理這段程式碼。我們已經討論到第三行了，不過這仍然是這段單一邏輯程式碼中的一部分。接下來的部分是此方法：

```
.expect("讀取該行失敗");
```

我們可以將程式碼寫成這樣：

```
io::stdin().read_line(&mut guess).expect("讀取行數失敗");
```

但是這麼長通常會很難閱讀，最好還是能夠分段。當你透過 `.method_name()` 語法呼叫方法時，通常換行來寫並加上縮排，來拆開一串很長的程式碼會比較好閱讀。現在讓我們來討論這行在做什麼。

如稍早提過的，`read_line` 會將使用者任何輸入轉換至我們傳入的字串，但它還回傳了一個 `Result` 數值。`Result` 是種**列舉 (enumerations)**，常稱為 *enums*。列舉是種可能有數種狀態其中之一的型別，而每種可能的狀態我們稱之為列舉的**變體 (variants)**。

第六章會更詳細地介紹列舉，這些 `Result` 型別的目的是要編碼錯誤處理資訊。

`Result` 的變體有 `Ok` 和 `Err`。`Ok` 變體指的是該動作成功完成，且 `Ok` 內部會包含成功產生的數值。而 `Err` 變體代表動作失敗，且 `Err` 會包含該動作如何與為何會失敗的資訊。

`Result` 型別的數值與任何型別的數值一樣，它們都有定義些方法。`Result` 的實例有 `expect` 方法讓你呼叫。如果此 `Result` 實例數值為 `Err` 的話，`expect` 會讓程式崩潰並顯示作為引數傳給 `expect` 的訊息。如果 `read_line` 回傳 `Err` 的話，這可能就是從底層作業系統傳來的錯誤結果。如果此 `io::Result` 實例數值為 `Ok` 的話，`expect` 會接收 `Ok` 的回傳值並只回傳該數值，讓你可以使用。在此例中，數值將為使用者輸入進標準輸入介面的位元組數字。

如果你沒有呼叫 `expect`，程式仍能編譯，但你會收到一個警告：

```
$ cargo build
   Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
warning: unused `Result` that must be used
--> src/main.rs:10:5
   |
10 |     io::stdin().read_line(&mut guess);
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
= note: `[warn(unused_must_use)]` on by default
= note: this `Result` may be an `Err` variant, which should be handled

warning: `guessing_game` (bin "guessing_game") generated 1 warning
    Finished dev [unoptimized + debuginfo] target(s) in 0.59s
```

Rust 警告你沒有使用 `read_line` 回傳的 `Result` 數值，這意味著程式沒有處理可能發生的錯誤。

要解決此警告的正確方式是實際進行錯誤處理，但因為我們只想要當問題發生時直接讓程式當掉，所以你可以先使用 `expect` 就好。你會在[第九章](#)學到如何從錯誤中恢復。

透過 `println!` 佔位符印出數值

在結束大括號之前，目前程式碼中還有一行要來討論：

```
println!("你的猜測數字：{guess}");
```

此行會印出存有使用者輸入的字串。其中的大括號 `{}` 是個佔位符（placeholder）：將 `{}` 想成是個小蟹鉗會夾住某個數值。當要印出變數的數值時，變數名稱可以放進括號內。當要印出表達式運算出的結果時，則先將空括號放進要格式化的字串，然後在字串後用逗號以相同的順序列出要印出的表達式列表。用 `println!` 同時印出變數與表達式結果的話會如以下所示：

```
let x = 5;
let y = 10;

println!("x = {x} 而且 y + 2 = {}", y + 2);
```

此程式碼會印出 `x = 5 而且 y + 2 = 12`。

測試第一個部分

讓我們來測試猜謎遊戲中的第一個部分。請使用 `cargo run` 來執行它：

```
$ cargo run
   Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
   Finished dev [unoptimized + debuginfo] target(s) in 6.44s
   Running `target/debug/guessing_game`
請猜測一個數字！
請輸入你的猜測數字。
6
你的猜測數字：6
```

到目前為止，遊戲的第一個部分就完成了：我們取得了鍵盤的輸入然後顯示出來。

產生祕密數字

接下來，我們要產生一個能讓使用者猜看看的祕密數字。祕密數字每次都要不同，這樣遊戲才值得多玩幾次。讓我們使用 1 到 100 之間的隨機數字，這樣遊戲才不會太困難。Rust 的標準函式庫並不包含產生隨機數字的功能。然而，Rust 團隊有提供個 `rand crate`。

使用 Crate 來取得更多功能

所謂的 crate 是一個 Rust 原始碼檔案的集合。我們正在寫的專案屬於**執行檔 (binary) crate**，也就會是個執行檔。而 `rand crate` 屬於**函式庫 (library) crate**，這會包含讓其他程式能夠使用的程式碼。

Cargo 協調外部 crate 的功能正是它的亮點。在我們可以使用 `rand` 來寫程式碼前，我們需要修改 `Cargo.toml` 檔案來包含 `rand crate` 作為依賴函式庫 (dependency)。開啟該檔案然後將以下行數加到 Cargo 自動產生的 `[dependencies]` 標頭 (header) 段落中最後一行下面。記得確認 `rand` 指定的版本數字與我們相同，不然此教學的範例程式碼可能不會運行成功：

檔案名稱：Cargo.toml

```
[dependencies]
rand = "0.8.5"
```

在 `Cargo.toml` 檔案中，標頭以下的所有內容都是該段落的一部分，一直到下個段落出現為止。

`[dependencies]` 段落是告訴 Cargo 此專案要依賴哪些 crate，以及那些 crate 的版本為何。在此例中，我們透過語意化版本 `0.8.5` 來指定 `rand crate`。Cargo 能夠理解**語意化版本 (Semantic Versioning)**，有時也被稱之為 *SemVer*，這是一種定義版本數字的標準。數字 `0.8.5` 其實是 `^0.8.5` 的縮寫，這代表任何至少為 `0.8.5` 且低於 `0.9.0` 版本。

Cargo 將這些版本提供的公開 API 視為是與版本 `0.8.5` 相容的，這樣的規格讓你能在本章節取得最新的 patch 發佈版本程式碼。任何 `0.9.0` 以上的版本就不會保證提供以下範例所使用的相同 API。

現在，在不改變任何程式碼的情況下，讓我們建構 (build) 專案吧，如範例 2-2 所示。

```
$ cargo build
  Updating crates.io index
Downloaded rand v0.8.5
Downloaded libc v0.2.127
Downloaded getrandom v0.2.7
Downloaded cfg-if v1.0.0
Downloaded ppv-lite86 v0.2.16
Downloaded rand_chacha v0.3.1
Downloaded rand_core v0.6.3
  Compiling libc v0.2.127
  Compiling getrandom v0.2.7
  Compiling cfg-if v1.0.0
  Compiling ppv-lite86 v0.2.16
  Compiling rand_core v0.6.3
  Compiling rand_chacha v0.3.1
  Compiling rand v0.8.5
  Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
  Finished dev [unoptimized + debuginfo] target(s) in 2.53s
```

範例 2-2：在新增 `rand` crate 作為依賴後，執行 `cargo build` 的輸出

你可能會看到不同的版本數字（但多虧有 SemVer，它們都會與程式碼相容！）和不同的行數（依照作業系統可能會不同）以及每行順序可能會不相同。

當我們匯入了外部依賴，Cargo 會從 *registry* 取得所有 crate 的最新版本訊息，這是份 [Crates.io](https://crates.io) 的資料副本。Crates.io 是個讓 Rust 生態系統中的每個人都能發佈它們的開源 Rust 專案並讓其他人使用的地方。

在更新 registry 之後，Cargo 會檢查 `[dependencies]` 段落並下載你還沒有的 crate。在此例中，雖然我們只有列出 `rand` 作為依賴，但 Cargo 還得下載 `rand` 所依賴的其他 crate 才能運作。在下載完 crates 之後，Rust 會編譯依賴函式庫以及使用到它們的專案。

如果你立即再次執行 `cargo build` 且沒有作出任何改變的話，你除了 `Finished` 這行以外不會在收到任何輸出。Cargo 知道它已經下載並編譯依賴函式庫了，而且你沒有在 `Cargo.toml` 檔案中再做任何改變。Cargo 也知道你沒有修改任何程式碼，所以也不會再重新編譯它。既然沒事可做，它就只好馬上結束。

如果你開啟 `src/main.rs` 檔案，加些瑣碎的修改，然後儲存並再次建構的話，你會只看到兩行輸出：

```
$ cargo build
  Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
  Finished dev [unoptimized + debuginfo] target(s) in 2.53 secs
```

這幾行表示 Cargo 只更新你對 `src/main.rs` 檔案的瑣碎修改進行建構。你的依賴沒變，所以 Cargo 知道它可以重複使用已經下載並編譯過的程式碼。

透過 *Cargo.lock* 檔案確保建構可以重現

Cargo 有個機制能確保任何人或你在任何時候重新建構程式碼時，都能產生相同結果。舉例來說，要是下一週 `rand` crate 發佈了版本 0.8.6 且該版本包含重大程式錯誤更新，卻也有個會破壞你的程式碼的迴歸錯誤 (regression)，這時會發生什麼事呢？為了處理這樣的狀況，Rust 會在你第一次執行 `cargo build` 時建立個 *Cargo.lock* 檔案，它會位於 *guessing_game* 目錄中。

當你第一次建構專案時，Cargo 會決定出符合情境的依賴函式庫版本，然後將它們寫入 *Cargo.lock* 檔案中。當你在未來建構專案時，Cargo 會看到 *Cargo.lock* 的存在並使用其指定的版本，而非重新再次決定該用哪些版本。這讓你有個能自動重現的建構方案。換句話說，你的專案仍會繼續使用 0.8.5 直到你顯式升級為止，這都多虧了 *Cargo.lock* 檔案。由於 *Cargo.lock* 對於重現建構非常重要，所以通常它會和其他程式碼一同上傳到專案的版本控制源頭。

升級 Crate 來取得新版本

當你真的想升級 crate 時，Cargo 有提供個命令 `update`，這會忽略 *Cargo.lock* 檔案並依據 *Cargo.toml* 指定的規格決定所有合適的最新版本。如果成功的話，Cargo 會將這些版本寫入 *Cargo.lock* 檔案中。不然的話，Cargo 預設只會尋找大於 0.8.5 且小於 0.9.0 的版本。如果 `rand` 有發佈兩個新版本 0.8.6 和 0.9.0，當你輸入 `cargo update` 時，你會看到以下結果：

```
$ cargo update
  Updating crates.io index
  Updating rand v0.8.5 -> v0.8.6
```

Cargo 會忽略 0.9.0 的發布版本。此時你也會注意到 *Cargo.lock* 檔案中的變更，指出你現在使用的 `rand` crate 版本為 0.8.6。如果你想使用 `rand` 版本 0.9.0 或任何版本 0.9.x 系列更新 *Cargo.toml* 檔案，如下所示：

```
[dependencies]
rand = "0.9.0"
```

下次你執行 `cargo build` 時，Cargo 將會更新 crate registry，並依據你指定的新版本來重新評估 `rand` 的確切版本。

Cargo 與其生態系統還有很多內容可以介紹，我們會在第十四章討論它們。但現在你只需要知道這些就好。Cargo 讓重複使用函式庫變得非常容易，讓 Rustaceans 可以組合許多套件寫出簡潔的專案。

產生隨機數字

讓我們開始使用 `rand` 產生數字來猜吧！下一步是更新 *src/main.rs*，如範例 2-3 所示。

檔案名稱：*src/main.rs*


```

use std::io;
use rand::Rng;

fn main() {
    println!("請猜測一個數字!");

    let secret_number = rand::thread_rng().gen_range(1..=100);

    println!("祕密數字為：{secret_number}");

    println!("請輸入你的猜測數字。");

    let mut guess = String::new();

    io::stdin()
        .read_line(&mut guess)
        .expect("讀取該行失敗");

    println!("你的猜測數字：{guess}");
}

```

範例 2-3：新增程式碼來產生隨機數字

首先我們加上 `use` 這行：`use rand::Rng;`。 `Rng` 特徵 (trait) 定義了隨機數字產生器實作的方法，所以此特徵必須引入作用域，我們才能使用這些方法。第十章會詳細解釋特徵。

接著，我們在中間加上兩行。我們在第一行呼叫的 `rand::thread_rng` 函式會回傳我們要使用的特定隨機數字產生器：這會位於目前執行緒 (thread) 並由作業系統提供種子 (seed)。然後我們對隨機數字產生器呼叫 `gen_range` 方法。此方法由 `Rng` 特徵所定義，而我們則是用 `use rand::Rng;` 陳述式將此特徵引入作用域中。`gen_range` 方法接收一個範圍表達式作為引數並產生一個在此範圍之間的隨機數字。我們所使用的範圍表達式的格式為 `start..=end`。這個範圍會包含下限和上限，所以我們需要指定 `1..=100` 來索取 1 到 100 之間的數字。

注意：你不可能憑空就知道該使用 `crate` 中的哪些特徵或是呼叫哪些方法與函式，所以每個 `crate` 都會提供技術文件解釋如何使用它。`Cargo` 另一大亮點就是執行 `cargo doc --open` 命令就能建構所有本地端依賴函式庫的技術文件，並在你的瀏覽器中開啟。舉例來說，如果你對 `rand` `crate` 的其他功能有興趣的話，你可以執行 `cargo doc --open` 然後點擊左側邊欄的 `rand`。

第二行會印出祕密數字，這在開發程式時能用來作測試，不過在最終版本我們會刪除它。如果在遊戲一開始程式就印出答案的話跟本就沒有玩的必要了！

請嘗試執行程式幾次：

```
$ cargo run
  Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
  Finished dev [unoptimized + debuginfo] target(s) in 2.53s
  Running `target/debug/guessing_game`
請猜測一個數字！
祕密數字為：7
請輸入你的猜測數字。
4
你的猜測數字：4

$ cargo run
  Finished dev [unoptimized + debuginfo] target(s) in 0.02s
  Running `target/debug/guessing_game`
請猜測一個數字！
祕密數字為：83
請輸入你的猜測數字。
5
你的猜測數字：5
```

你應該會得到不同的隨機數字，而且它們都應該要在 1 到 100 的範圍內。做得好！

將猜測的數字與祕密數字做比較

現在我們有使用者的輸入與隨機數字，我們可以來比較它們了。這步驟顯示在範例 2-4。注意此程式碼還無法編譯，我們會解釋為什麼。

檔案名稱：src/main.rs

```
use rand::Rng;
use std::cmp::Ordering;
use std::io;

fn main() {
    // --省略--

    println!("你的猜測數字：{guess}");

    match guess.cmp(&secret_number) {
        Ordering::Less => println!("太小了！"),
        Ordering::Greater => println!("太大了！"),
        Ordering::Equal => println!("獲勝！"),
    }
}
```



範例 2-4：處理比較兩個數字後的可能數值

首先我們加上另一個 `use` 陳述式，這將 `std::cmp::Ordering` 型別從標準函式庫引入作用域中。`Ordering` 是另一個列舉，擁有的變體為 `Less`、`Greater` 與 `Equal`。這些是當你比較兩個數值時的三種可能結果。

然後我們在底下加上五行程式碼來使用 `Ordering` 型別。`cmp` 方法會比較兩個數值，並能在任何可以比較的數值中進行呼叫。其參考一個任何你想做比較的數值，在此例中就是將 `guess` 與 `secret_number` 做比較。然後它會回傳我們透過 `use` 陳述式引入作用域的 `Ordering` 列舉其中一個變體。我們使用 `match` 表達式來依據透過 `guess` 與 `secret_number` 呼叫 `cmp` 回傳的 `Ordering` 變體來決定下一步要做什麼。

`match` 表達式由分支 (arms) 所組成。分支包含一個能被配對的模式 (pattern) 以及對應的程式碼，這在當 `match` 的數值能與該分支的模式配對時就能執行。Rust 會用 `match` 得到的數值依序遍歷每個分支中的模式。`match` 結構與模式是 Rust 中非常強大的特色，能讓你表達各種程式碼可能會遇上的情形，並確保你有將它們全部處理完。這些特色功能會在第六章與第十八章分別討論其細節。

讓我們看看在此例中使用的 `match` 表達式。假設使用者猜測的數字是 50 而這次隨機產生的祕密數字是 38。

當程式碼比較 50 與 38 時，`cmp` 方法會回傳 `Ordering::Greater`，因為 50 大於 38。`match` 表達式會取得 `Ordering::Greater` 數值並開始檢查每個分支的模式。它會先查看第一個分支的模式 `Ordering::Less` 並看出數值 `Ordering::Greater` 無法與 `Ordering::Less` 配對，所以它忽略該分支的程式碼，並移到下一個分支。而下個分支的模式 `Ordering::Greater` 能配對到 `Ordering::Greater`！所以該分支對應的程式碼就會執行並印出 太大了！ 到螢幕上。最後 `match` 表達式就會在第一次成功配對就結束，所以在此情境中它不需要再查看最後一個分支。

然而範例 2-4 的程式碼還無法編譯，讓我們嘗試看看：

```
$ cargo build
   Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
error[E0308]: mismatched types
  --> src/main.rs:26:21
   |
26 |         match guess.cmp(&secret_number) {
   |                       --- ^^^^^^^^^^^^^^^^^ expected struct `String`, found
integer
   |
   |             arguments to this function are incorrect
   |
   = note: expected reference `&String`
             found reference `&{integer}`
note: associated function defined here

For more information about this error, try `rustc --explain E0308`.
error: could not compile `guessing_game` due to previous error
```

錯誤的關鍵表示**型別無法配對 (mismatched types)**。Rust 有個強力的靜態型別系統，但它也提供了型別推斷。當我們寫 `let mut guess = String::new()` 時，Rust 能夠推斷出 `guess` 應該是 `String` 讓我們不必親自寫出型別。另一方面，`secret_number` 則是個數字型別。以下是一些在 Rust 中可以包含數字 1 到 100 的數字型別：32 位元數字 `i32`、非帶號 (unsigned) 32 位元數字 `u32`、64 位元數字 `i64`，以及更多等等。Rust 預設的數字型別為 `i32`，這就是 `secret_number` 的型別，除非你特地加上型別詮釋，Rust 才會推斷成不同的數字型別。此錯誤原因是因為 Rust 無法比較將字串與數字型別做比較。

所以我們要將程式從輸入讀取的 `String` 轉換成真正的數字型別，讓我們可以將其與祕密數字做比較。我們可以在 `main` 函式本體加上另一行程式碼：

檔案名稱：src/main.rs

```
// --省略--

let mut guess = String::new();

io::stdin()
    .read_line(&mut guess)
    .expect("讀取該行失敗");

let guess: u32 = guess.trim().parse().expect("請輸入一個數字!");

println!("你的猜測數字: {guess}");

match guess.cmp(&secret_number) {
    Ordering::Less => println!("太小了!"),
    Ordering::Greater => println!("太大了!"),
    Ordering::Equal => println!("獲勝!"),
}
```

這行程式碼就是：

```
let guess: u32 = guess.trim().parse().expect("請輸入一個數字!");
```

我們建立了一個變數叫做 `guess`。小等一下，程式不是已經有個變數叫做 `guess` 了嗎？的確是的，但 Rust 允許我們遮蔽之前的 `guess` 數值成新的數值。**遮蔽 (Shadowing)** 讓我們可以重複使用 `guess` 變數名稱，而不必強迫我們得建立兩個不同的變數，舉例來說像是 `guess_str` 和 `guess`。我們會在第三章更詳細地解釋此概念，現在這邊只需要知道這常拿來將一個數值的型別轉換成另一個型別。

我們將此新的變數綁定給 `guess.trim().parse()` 表達式。表達式中的 `guess` 指的是原本儲存字串輸入的 `guess`。`String` 中的 `trim` 方法會去除開頭與結尾的任何空白字元，我們一定要這樣做才能將字串與 `u32` 作比較，因為它只會包含數字字元。使用者一定得按下 `enter` 才能滿足 `read_line` 並輸入他們的猜測數字，這樣會加上一個換行字元。當使用者按下 `enter` 時，字串結尾就會加上換行字元。舉例來說，如果使用者輸入 5 並按下 `enter` 的話，`guess` 看起來會像這

樣：5\n。 \n 指的是「換行 (newline)」，這是按下 enter 的結果（在 Windows 按下 enter 的結果會是輸入和換行 \r\n）。 trim 方法能去除 \n 或 \r\n，讓結果只會是 5。

而字串中的 parse 方法會轉換字串成其他型別。我們在此用它將字串轉換成數字，我們需要使用 let guess: u32 來告訴 Rust 我們想使用的確切數字型別。guess 後面的冒號（:）告訴 Rust 我們會詮釋此變數的型別。Rust 有些內建的數字型別，這裡的 u32 是個非帶號 (unsigned) 的 32 位元整數。對於不大的正整數來說，這是不錯的預設選擇。你會在第三章學到其他數字型別。

除此之外，在此範例程式中的 u32 詮釋與 secret_number 的比較意味著 Rust 也會將 secret_number 推斷成 u32。所以現在會有兩個相同型別的數值能做比較了！

parse 的呼叫很容易造成錯誤，因為它只適用於邏輯上能轉換成數字的字元。舉例來說，如果字串包含 A👉% 的話，就不可能轉換成數字。因為它可能會失敗，parse 方法回傳的是 Result 型別，就和 read_line 方法一樣（在之前的「使用 Result 處理可能的錯誤」段落提及）。我們也會用相同的方式來處理此 Result，也就是呼叫 expect 方法。如果 parse 回傳 Result 的 Err 變體的話，由於它無法從字串建立數字，expect 的呼叫會讓遊戲當掉並顯示我們給予的訊息。如果 parse 能成功將字串轉成數字，它將會回傳 Result 的 Ok 變體，而 expect 將會回傳 Ok 的內部數值。

現在讓我們執行程式：

```
$ cargo run
  Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
  Finished dev [unoptimized + debuginfo] target(s) in 0.43s
  Running `target/debug/guessing_game`
請猜測一個數字！
祕密數字為：58
請輸入你的猜測數字。
76
你的猜測數字：76
太大了！
```

不錯！儘管我們在猜測數字前加了一些空格，但程式仍能推斷出使用者猜測的是 76。多執行程式幾次來驗證不同種輸入產生的不同行為：像是正確猜出數字、猜測的數字太高或猜測的數字太低。

我們已經大致上將遊戲完成了，但使用者只能猜測一次。讓我們用迴圈來修改吧！

透過迴圈來允許多次猜測

loop 關鍵字會產生無限迴圈。我們加入此迴圈讓使用者可能有更多機會可以猜測：

檔案名稱：src/main.rs

```
// --省略--

println!("祕密數字為：{secret_number}");

loop {
    println!("請輸入你的猜測數字。");

    // --省略--

    match guess.cmp(&secret_number) {
        Ordering::Less => println!("太小了！"),
        Ordering::Greater => println!("太大了！"),
        Ordering::Equal => println!("獲勝！"),
    }
}
}
```

如同你所見，我們將輸入猜測提示以下的程式碼都移入迴圈中。請確保迴圈中的每一行有用四個空格來做縮排，然後再次執行程式。現在程式會不停地尋問要猜測的數字了！但這樣帶來了新的問題，看來使用者無法離開遊戲！

使用者的確永遠可以使用快捷鍵 `ctrl-c` 來中斷程式。但還有其他辦法能逃離這個無限循環，如同在「[將猜測的數字與祕密數字做比較](#)」中討論 `parse` 時提到的，如果使用者輸入非數字答案的話，程式就會當掉。我們可以利用此特性來讓使用者離開，如以下所示：

```
$ cargo run
   Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
   Finished dev [unoptimized + debuginfo] target(s) in 1.50s
   Running `target/debug/guessing_game`
請猜測一個數字！
祕密數字為：59
請輸入你的猜測數字。
45
你的猜測數字：45
太小了！
請輸入你的猜測數字。
60
你的猜測數字：60
太大了！
請輸入你的猜測數字。
59
你的猜測數字：59
獲勝！
請輸入你的猜測數字。
quit
thread 'main' panicked at '請輸入一個數字!: ParseIntError { kind: InvalidDigit
}', src/main.rs:28:47
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

輸入 `quit` 就能離開遊戲，但是你會看到其他非數字輸入也是如此。這並不是最理想的方案，我們想要在猜對數字時自動停止。

猜對後離開

讓我們加上 `break` 陳述式來在使用者獲勝時離開遊戲：

檔案名稱：src/main.rs

```
// --省略--

match guess.cmp(&secret_number) {
    Ordering::Less => println!("太小了！"),
    Ordering::Greater => println!("太大了！"),
    Ordering::Equal => {
        println!("獲勝！");
        break;
    }
}
```

在 `獲勝！` 之後加上 `break` 這行讓程式在使用者猜對祕密數字時可以離開迴圈。離開迴圈也意味著離開程式，因為此迴圈是 `main` 中的最後一個部分。

處理無效輸入

為了進一步改善遊戲體驗，當使用者的輸入不是數字時，我們不該讓程式直接當掉。遊戲程式可以忽略非數字來讓使用者繼續猜測。我們可以修改 `guess` 這段將 `String` 轉換成 `u32` 的程式碼，如範例 2-5 所示。

檔案名稱：src/main.rs

```
// --省略--

io::stdin()
    .read_line(&mut guess)
    .expect("讀取該行失敗");

let guess: u32 = match guess.trim().parse() {
    Ok(num) => num,
    Err(_) => continue,
};

println!("你的猜測數字：{guess}");

// --省略--
```

範例 2-5：忽略非數字的猜測並要求下一個猜測數字，而不是讓程式當掉

我們將 `expect` 的呼叫換成 `match` 表達式，從錯誤中當掉改成實際處理錯誤。你應該還記得 `parse` 回傳的是 `Result` 型別，且 `Result` 是個列舉，其變體為 `Ok` 與 `Err`。我們在此使用 `match` 表達式，如同我們對 `cmp` 方法回傳的 `Ordering` 處理方式一樣。

如果 `parse` 能成功將字串轉換成數字，它會回傳 `Ok` 數值內包含的結果數字。該 `Ok` 數值就會配對到第一個分支的模式，然後 `match` 表達式就會回傳 `parse` 產生並填入 `Ok` 內的 `num` 數值。該數字最後就會如我們所願變成我們建立的 `guess` 變數。

如果 `parse` 無法將字串轉換成數值的話，它會回傳包含與錯誤相關資訊的 `Err` 數值。該 `Err` 數值並不符合 `match` 的第一個分支模式 `Ok(num)`，但它能配對到第二個分支。底線 `_` 是個捕獲數值，在此例中，我們說我們想要配對到所有的 `Err` 數值，無論其中有什麼資訊在裡面。所以程式會執行第二條分支 `continue`，這告訴程式繼續 `loop` 下一個疊代並要求其他猜測數字。如此一來程式就能忽略所有 `parse` 可能會遇到的所有錯誤！

現在程式的每個部分都如我們所預期的了，讓我們試試看：


```
$ cargo run
  Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
  Finished dev [unoptimized + debuginfo] target(s) in 4.45s
  Running `target/debug/guessing_game`
請猜測一個數字！
祕密數字為：61
請輸入你的猜測數字。
10
你的猜測數字：10
太小了！
請輸入你的猜測數字。
99
你的猜測數字：99
太大了！
請輸入你的猜測數字。
foo
請輸入你的猜測數字。
61
你的猜測數字：61
獲勝！
```

太棒了！有了最後一項小修改，我們終於完成了猜謎遊戲。回想一下程式仍然會印出祕密數字。這在測試很有用，但在實際遊戲時就毀了樂趣了。讓我們刪除會印出祕密數字的 `println!`。範例 2-6 就是最終的程式碼。

檔案名稱：src/main.rs

```

use rand::Rng;
use std::cmp::Ordering;
use std::io;

fn main() {
    println!("請猜測一個數字！");

    let secret_number = rand::thread_rng().gen_range(1..=100);

    loop {
        println!("請輸入你的猜測數字。");

        let mut guess = String::new();

        io::stdin()
            .read_line(&mut guess)
            .expect("讀取該行失敗");

        let guess: u32 = match guess.trim().parse() {
            Ok(num) => num,
            Err(_) => continue,
        };

        println!("你的猜測數字：{guess}");

        match guess.cmp(&secret_number) {
            Ordering::Less => println!("太小了！"),
            Ordering::Greater => println!("太大了！"),
            Ordering::Equal => {
                println!("獲勝！");
                break;
            }
        }
    }
}

```

範例 2-6：完整的猜謎遊戲程式碼

此時此刻，你已經完成了猜謎遊戲。恭喜你！

總結

此專案讓你能動手實踐並親自體驗許多 Rust 的新概念：`let`、`match`、函式、外部 crate 的使用以及更多等等。在接下來陸續的章節，你將深入學習這些概念。第三章會涵蓋多數程式設計語言都有的概念，像是變數、資料型別與函式，以及如何在 Rust 中使用它們。第四章會探索所有權

(ownership)，這是 Rust 與其他語言最不同的特色。第五章會討論結構體 (structs) 與方法語法，而第六章會解釋列舉。

常見程式設計概念

此章節涵蓋了幾乎所有程式語言都會出現的概念，以及如何在 Rust 使用。許多程式語言的核心概念都是相同的，所以此章節所介紹的概念也不是 Rust 所特有的。不過我們會用 Rust 的程式碼來討論它們，並解釋使用這些概念的慣例。

更明確來說，你將會學習到變數、基本型別、註解以及控制流程。這些基本概念會出現在每個 Rust 程式中，所以提早學習這些概念可以為你打下穩健的基礎。

關鍵字 (Keywords)

Rust 程式語言和其他語言一樣會保留一系列的**關鍵字 (keywords)**。請注意你將無法用這些字作為變數或函式名稱。大多數關鍵字都有特別意義，而你將會在你的 Rust 程式使用它們來處理不同的任務；而有一些目前則還沒有任何用途，但是在未來可能會被 Rust 加入所以作為保留。你可以在[附錄 A](#) 查看關鍵字列表。

變數與可變性

如同「[透過變數儲存數值](#)」提到的，變數預設是不可變的。這是 Rust 推動你能充分利用 Rust 提供的安全性和簡易並行性來寫程式的許多方法之一。不過，你還是有辦法能讓你的變數成為可變的。讓我們來探討為何 Rust 鼓勵你多多使用不可變，以及何時你會想要改為可變的。

當一個變數是不可變的，只要有數值綁定在一個名字上，你就無法改變其值。為了方便說明，讓我們使用 `cargo new variables` 在 `projects` 目錄下產生一個新專案叫做 `variables`。

再來在你的 `variables` 目錄下開啟 `src/main.rs` 然後覆蓋程式碼為以下內容。這是段還無法編譯的程式碼：

檔案名稱：`src/main.rs`

```
fn main() {  
    let x = 5;  
    println!("x 的數值為：{x}");  
    x = 6;  
    println!("x 的數值為：{x}");  
}
```



儲存然後使用 `cargo run` 執行程式。你應該會收到一則有關於不可變的錯誤訊息，如下所示：

```
$ cargo run  
    Compiling variables v0.1.0 (file:///projects/variables)  
error[E0384]: cannot assign twice to immutable variable `x`  
--> src/main.rs:4:5  
  |  
2 |     let x = 5;  
  |     -  
  |     |  
  |     first assignment to `x`  
  |     help: consider making this binding mutable: `mut x`  
3 |     println!("x 的數值為：{x}");  
4 |     x = 6;  
  |     ^^^^^ cannot assign twice to immutable variable  
  
For more information about this error, try `rustc --explain E0384`.  
error: could not compile `variables` due to previous error
```

此範例顯示了編譯器如何協助你找到你程式碼的錯誤。雖然看到編譯器錯誤訊息總是令人感到沮喪，但這通常是為了讓你**知道**你的程式無法安全地完成你想讓它完成的任務。它們**不代表**你不是個優秀的程式設計師！有經驗的 Rustaceans 時常會與編譯器錯誤訊息打交道。

你得到的錯誤訊息說明「cannot assign twice to immutable variable `x`」，因為你嘗試第二次賦值給 `x` 變數。

當我們嘗試改變一個原先設計為不可變的變數時，能夠產生編譯時錯誤是很重要的。因為這樣的情況很容易導致程式錯誤。如果我們有一部分的程式碼在執行時認為某個數值絕對不會改變，但另一部分的程式碼卻更改了其值，那麼這就有可能讓前一部分的程式碼就可能以無法預測的方式運行。這樣的程式錯誤的起因是很難追蹤的，尤其是當第二部分的程式碼**偶而**才會改變其值。Rust 編譯器會保證當你宣告一個數值不會被改變時，它就絕對不會被改變。這樣你就不需要去追蹤該值可能會被改變，讓你的程式碼更容易推導。

但同時可變性也是非常有用的，能讓程式碼變得更好寫。雖然變數預設是不可變的，但就如同[第二章](#)一樣你可以在變數名稱前面加上 `mut` 讓它們可以成為可變的。加上 `mut` 也向未來的讀取者表明了其他部分的程式碼將會改變此變數的數值。

舉例來說，讓我們改變 `src/main.rs` 成以下程式碼：

檔案內容：`src/main.rs`

```
fn main() {  
    let mut x = 5;  
    println!("x 的數值為：{x}");  
    x = 6;  
    println!("x 的數值為：{x}");  
}
```

當你執行程式的話，我們會得到：

```
$ cargo run  
    Compiling variables v0.1.0 (file:///projects/variables)  
    Finished dev [unoptimized + debuginfo] target(s) in 0.30s  
    Running `target/debug/variables`  
x 的數值為：5  
x 的數值為：6
```

當使用 `mut` 時，我們可以將 `x` 的數值從 5 改變為 6。何時使用可變性的決定權在你手上，你可以依照特定場合做出你認為最佳的選擇。

常數

常數 (constants)。和不可變變數一樣，常數會讓數值與名稱綁定且不允許被改變，但是不可變變數與常數還是有些差異。

首先，你無法在常數使用 `mut`，常數不是預設不可變，它們永遠都不可變。如果你使用 `const` 宣告而非 `let` 的話，你**必須**指明型別。我們會在下一章「[資料型別](#)」詳細解釋型別與型別詮釋，所以現在先別擔心細節。你只需要先知道你永遠必須先詮釋常數的型別。

常數可以被定義在任一有效範圍，包含全域有效範圍。這讓它們非常有用，讓許多部分的程式碼都能夠知道它們。

最後一個差別是常數只能被常數表達式設置，而不能用任一在運行時產生的其他數值設置。

以下為一個常數名稱的範例：

```
const THREE_HOURS_IN_SECONDS: u32 = 60 * 60 * 3;
```

變數的名稱為 `THREE_HOURS_IN_SECONDS`，且它的數值被設為 60（一分鐘有多少秒）乘上 60（一小時有多少分鐘）乘上 3（此程式想要計算的小時數量）。Rust 的常數命名規則為使用全部英文大寫並用底線區隔每個單字。編譯器能夠在編譯時用特定限制集合內的操作進行運算，讓我們能用易於理解且驗證的方式寫出此數值，而不用將常數設為 10,800。你可以查閱 Rust Reference 的 [constant evaluation 段落](#)來瞭解哪些操作可以在宣告常數時使用。

在整支程式運行時，常數在它們的範圍內都是有效的。這樣的性質讓常數在處理應用程式中需要被許多程式碼部份所知道的數值的情況下是非常好的選擇，像是一款遊戲中玩家能夠得到的最高分數或者光速的數值。

將會擴散到所有程式碼的數值定義為常數，對於幫助未來程式碼的維護者理解是非常好的選擇。這也讓未來需要更新數值的話，你知道需要修改寫死的地方就好。

遮蔽 (Shadowing)

如同你在猜謎遊戲教學所看到的，在[第二章](#)你可以用之前的變數再次宣告新的變數。Rustaceans 會說第一個變數被第二個變數所遮蔽了，這代表當你使用該變數名稱時，編譯器會看到的是第二個變數的數值。第二個變數會遮蔽第一個變數，佔據變數名稱的使用權，直到它自己也被遮蔽或是離開作用域。我們可以用 `let` 關鍵字來重複宣告相同的變數名稱來遮蔽一個變數：

檔案名稱：src/main.rs

```
fn main() {
    let x = 5;

    let x = x + 1;

    {
        let x = x * 2;
        println!("x 在內部範圍的數值為：{x}");
    }

    println!("x 的數值為：{x}");
}
```

此程式首先將 `x` 給予 5，然後它重複用 `let x =` 建立一個新變數 `x`，取代了原本的數值並加上 1，所以 `x` 的數值變為 6。然後在接下來括號的內部範圍內，第三次的 `let` 陳述式一樣遮蔽了

`x` 讓它將原本的值乘與 `2`，讓 `x` 數值為 `12`。當該範圍結束時，內部的遮蔽也結束，所以 `x` 就回到原本的 `6`。當我們運行此程式時，就會輸出以下結果：

```
$ cargo run
  Compiling variables v0.1.0 (file:///projects/variables)
    Finished dev [unoptimized + debuginfo] target(s) in 0.31s
    Running `target/debug/variables`
x 在內部範圍的數值為：12
x 的數值為：6
```

遮蔽與標記變數為 `mut` 是不一樣的，因為如果我們不小心重新賦值而沒有加上 `let` 關鍵字的話，是會產生編譯期錯誤的。使用 `let` 的話，我們可以作出一些改變，然後在這之後該變數仍然是不可變的。

另一個 `mut` 與遮蔽不同的地方是，我們能有效地再次運用 `let` 產生新的變數，可以在重新運用相同名稱時改變它的型別。舉例來說，當我們希望程式要求使用者顯示出字串間應該顯示多少空格，同時我們又希望它被存為一個數字時，我們可以這樣做：

```
let spaces = "  ";
let spaces = spaces.len();
```

第一次宣告 `spaces` 的變數是一個字串型別，而第二次宣告 `spaces` 則成了數字型別。遮蔽這項功能讓我們不必去宣告像是 `spaces_str` 與 `spaces_num`，我們可以重複使用 `spaces` 這個變數名稱。不過，可變變數仍然是無法變更變數型別的，如果這樣做的話我們就會拿到編譯期錯誤：

```
let mut spaces = "  ";
spaces = spaces.len();
```

此錯誤訊息告訴我們我們不允許改變變數的型別：

```
$ cargo run
  Compiling variables v0.1.0 (file:///projects/variables)
error[E0308]: mismatched types
--> src/main.rs:3:14
   |
2  |     let mut spaces = "  ";
   |                      ----- expected due to this value
3  |     spaces = spaces.len();
   |                ^^^^^^^^^^^^^^^^^ expected `&str`, found `usize`

For more information about this error, try `rustc --explain E0308`.
error: could not compile `variables` due to previous error
```

現在我們講完變數了，讓我們看看它們可以擁有的資料型別吧。

資料型別

每個數值在 Rust 中都屬於某種**資料型別**，這告訴 Rust 何種資料被指定，好讓它能妥善處理資料。我們將討論兩種資料型別子集：純量（scalar）與複合（compound）。

請記住 Rust 是一門**靜態型別**語言，這代表它必須在編譯時知道所有變數的型別。編譯器通常能依據數值與我們使用的方式推導出我們想使用的型別。但有時候如果多種型別都有可能時，像是第二章的「將猜測的數字與祕密數字做比較」用到的 `parse` 將 `String` 轉換成數字時，我們就需要像這樣加上型別詮釋：

```
let guess: u32 = "42".parse().expect("這不是數字！");
```

如果我們沒有像上列程式碼這樣加上型別詮釋：`u32` 的話，Rust 將會顯示以下錯誤訊息。這表示編譯器需要我們給予更多資訊才能夠知道我們想用何種型別：

```
$ cargo build
   Compiling no_type_annotations v0.1.0 (file:///projects/no_type_annotations)
error[E0282]: type annotations needed
  --> src/main.rs:2:9
   |
2  |     let guess = "42".parse().expect("這不是數字！");
   |           ^^^^^
   |
help: consider giving `guess` an explicit type
   |
2  |     let guess: _ = "42".parse().expect("這不是數字！");
   |               +++

For more information about this error, try `rustc --explain E0282`.
error: could not compile `no_type_annotations` due to previous error
```

你將會看到其他資料型別的各种型別詮釋。

純量型別

純量型別代表單一數值。Rust 有四種主要純量型別：整數、浮點數、布林以及字元。你應該在其他程式語言就看過它們了，讓我們來看看它們在 Rust 是怎麼使用的：

整數型別

整數是沒有小數點的數字。我們在第二章用到了一個整數型別 `u32`，此型別表示其擁有的數值應該是一個佔 32 位元大小的非帶號整數（帶號整數的話則是用 `i` 起頭而非 `u`）。表格 3-1 展示了

Rust 中內建的整數型別。我們可以使用以下任何一種型別來宣告一個整數數值。

表格 3-1：Rust 中的整數型別

長度	帶號	非帶號
8 位元	<code>i8</code>	<code>u8</code>
16 位元	<code>i16</code>	<code>u16</code>
32 位元	<code>i32</code>	<code>u32</code>
64 位元	<code>i64</code>	<code>u64</code>
128 位元	<code>i128</code>	<code>u128</code>
系統架構	<code>isize</code>	<code>usize</code>

每個變體都可以是帶號或非帶號的，並且都有明確的大小。帶號與非帶號的區別是數字能不能有負數，換句話說就是數字能否帶有正負符號，如果沒有的話那就只會出現正整數而已。就像在紙上寫數字一樣：當我們需要考慮符號時，我們就會在數字前面加上正負號；但如果我們只在意正整數的話，那它可以不帶符號。帶號數字是以二補數的方式儲存。

每一帶號變體可以儲存的數字範圍包含從 $-(2^{n-1})$ 到 $2^{n-1} - 1$ 以內的數字， n 就是該變體佔用的位元大小。所以一個 `i8` 可以儲存的數字範圍就是從 $-(2^7)$ 到 $2^7 - 1$ ，也就是 -128 到 127。而非帶號可以儲存的數字範圍則是從 0 到 $2^n - 1$ ，所以 `u8` 可以儲存的範圍是從 0 到 $2^8 - 1$ ，也就是 0 到 255。

另外，`isize` 與 `usize` 型別則是依據你程式運行的電腦架構來決定大小，所以上方表格才用「系統架構」來表示長度：如果你在 64 位元架構上的話就是 64 位元；如果你是 32 位元架構的話就是 32 位元。

你可以用表格 3-2 列的格式來寫出整數字面值（literals）。能適用於數種數字型別的數字字面值都允許在最後面加上型別，比如說用 `57u8` 來指定型別。數字字面值也可以加上底線 `_` 分隔方便閱讀，比如說 `1_000` 其實就和指定 `1000` 的數值一樣。

表格 3-2：Rust 中的整數字面值

數字字面值	範例
十進制	<code>98_222</code>
十六進制	<code>0xff</code>
八進制	<code>0o77</code>
二進制	<code>0b1111_0000</code>
位元組（僅限 <code>u8</code> ）	<code>b'A'</code>

所以你該用哪些整數型別呢？如果你不確定的話，Rust 預設的型別是很好的起始點：整數型別預設是 `i32`。而你會用到 `isize` 或 `usize` 的主要時機是作為某些集合的索引。

整數溢位

假設你有個變數型別是 `u8` 可以儲存 0 到 255 的數值。如果你想要改變變數的值超出這個範圍的話，比方說像是 256，那麼就會發生**整數溢位**，這會產生兩種不同的結果。如果你是在除錯模式編譯的話，Rust 會包含整數溢位的檢查，造成你的程式在執行時**恐慌 (panic)**。Rust 使用恐慌來表示程式因錯誤而結束，我們會在第九章的「[對無法復原的錯誤使用 panic!](#)」段落討論更多造成恐慌的細節。

當你是在發佈模式下用 `--release` 來編譯的話，Rust 則**不會**加上整數溢位的檢查而造成恐慌。相反地，如果發生整數溢位的話，Rust 會作出**二補數包裝**的動作。簡單來說，超出最大值的數值可以被**包裝**成該型別的最低數值。以 `u8` 為例的話，256 會變成 0、257 會變成 1，以此類推。程式不會恐慌，但是該變數可能會得到一個不是你原本預期的數值。通常依靠整數溢位的行為仍然會被視為邏輯錯誤。

要顯式處理可能的溢位的話，你可以使用以下標準函式庫中基本型別提供的一系列方法：

- 將所有操作用 `wrapping_*` 方法包裝，像是 `wrapping_add`。
- 使用 `checked_*` 方法，如果有溢位的話其會回傳 `None` 數值。
- 使用 `overflowing_*` 方法，其會回傳數值與一個布林值來顯示是否有溢位發生。
- 屬於 `saturating_*`，讓數值溢位時保持在最小或最大值。

浮點數型別

Rust 還有針對有小數點的**浮點數**提供兩種基本型別：`f32` 和 `f64`，分別佔有 32 位元與 64 位元的大小。而預設的型別為 `f64`，因為現代的電腦處理的速度幾乎和 `f32` 一樣卻還能擁有更高的精準度。所有的浮點數型別都是帶號的 (signed)。

以下為展示浮點數的範例：

檔案名稱：src/main.rs

```
fn main() {  
    let x = 2.0; // f64  
  
    let y: f32 = 3.0; // f32  
}
```

浮點數是依照 IEEE-754 所定義的，`f32` 型別是單精度浮點數，而 `f64` 是倍精度浮點數。

數值運算

Rust 支援你所有想得到的數值型別基本運算：加法、減法、乘法、除法和取餘。整數除法會取最接近零的下界數值。以下程式碼展示出如何在 `let` 陳述式使用這些運算：

檔案名稱：src/main.rs

```
fn main() {  
    // 加法  
    let sum = 5 + 10;  
  
    // 減法  
    let difference = 95.5 - 4.3;  
  
    // 乘法  
    let product = 4 * 30;  
  
    // 除法  
    let quotient = 56.7 / 32.2;  
    let truncated = -5 / 3; // 結果為 -1  
  
    // 取餘  
    let remainder = 43 % 5;  
}
```

每一個陳述式中的表達式都使用了一個數學運算符號並計算出一個數值出來，賦值給該變數。[附錄 B](#) 有提供列表列出 Rust 所提供的所有運算子。

布林型別

如同其他多數程式語言一樣，Rust 中的布林型別有兩個可能的值：`true` 和 `false`。布林值的大小為一個位元組。要在 Rust 中定義布林型別的話用 `bool`，如範例所示：

檔案名稱：src/main.rs

```
fn main() {  
    let t = true;  
  
    let f: bool = false; // 型別詮釋的方式  
}
```

布林值最常使用的方式之一是作為條件判斷，像是在 `if` 表達式中使用。我們將會在「[控制流程](#)」段落介紹如何在 Rust 使用 `if` 表達式。

字元型別

Rust 的 `char` 型別是最基本的字母型別，以下程式碼顯示了使用它的方法：

檔案名稱：src/main.rs

```
fn main() {  
    let c = 'z';  
    let z: char = 'Z'; // 明確標註型別的寫法  
    let heart_eyed_cat = '😻';  
}
```

注意到 `char` 字面值是用單引號賦值，宣告字串字面值時才是用雙引號。Rust 的 `char` 型別大小為四個位元組並表示為一個 Unicode 純量數值，這代表它能擁有的字元比 ASCII 還來的多。舉凡標音字母 (Accented letters)、中文、日文、韓文、表情符號以及零長度空格都是 Rust `char` 的有效字元。Unicode 純量數值的範圍包含從 `U+0000` 到 `U+D7FF` 以及 `U+E000` 到 `U+10FFFF`。但是一個「字元」並不是真正的 Unicode 概念，所以你對於什麼是一個「字元」的看法可能會和 Rust 的 `char` 不一樣。我們將會在第八章的「[透過字串儲存 UTF-8 編碼的文字](#)」來討論此議題。

複合型別

複合型別可以組合數個數值為一個型別，Rust 有兩個基本複合型別：元組 (tuples) 和陣列 (arrays)。

元組 (Tuple) 型別

元組是個將許多不同型別的數值合成一個複合型別的常見方法。元組擁有固定長度：一旦宣告好後，它們就無法增長或縮減。

我們建立一個元組的方法是寫一個用括號囊括起來的數值列表，每個值再用逗號分隔開來。元組的每一格都是一個獨立型別，不同數值不必是相同型別。以下範例我們也加上了型別詮釋，平時不一定要加上：

檔案名稱：src/main.rs

```
fn main() {  
    let tup: (i32, f64, u8) = (500, 6.4, 1);  
}
```

此變數 `tup` 就是整個元組，因為一個元組就被視為單一複合元素。要拿到元組中的每個獨立數值的話，我們可以用模式配對 (pattern matching) 來解構一個元組的數值，如以下所示：

檔案名稱：src/main.rs

```
fn main() {
    let tup = (500, 6.4, 1);

    let (x, y, z) = tup;

    println!("y 的數值為：{y}");
}
```

此程式先是建立了一個元組然後賦值給 `tup`，接著它用模式配對和 `let` 將 `tup` 拆成三個個別的變數 `x`、`y` 和 `z`。這就叫做**解構 (destructuring)**，因為它將單一元組拆成了三個部分。最後程式將 `y` 的值印出來，也就是 `6.4`。

我們也可以直接用句號 (`.`) 再加上數值的索引來取得元組內的元素。舉例來說：

檔案名稱：src/main.rs

```
fn main() {
    let x: (i32, f64, u8) = (500, 6.4, 1);

    let five_hundred = x.0;

    let six_point_four = x.1;

    let one = x.2;
}
```

此程式建立了元組 `x`，然後用它們個別的索引來存取元組的元素。和多數程式語言一樣，元組的第一個索引是 0。

沒有任何數值的元組有一種特殊的名稱叫做**單元型別 (Unit)**，其數值與型別都寫作 `()`，通常代表一個空的數值或空的回傳型別。表達式要是沒有回傳任何數值的話，它們就會隱式回傳單元型別。

陣列型別

另一種取得數個數值集合的方法是使用**陣列**。和元組不一樣的是，陣列中的每個型別必須是一樣的。和其他語言的陣列不同，Rust 的陣列是固定長度的。

我們將數值寫在陣列中的括號內，每個數值再用逗號區隔開來：

檔案名稱：src/main.rs

```
fn main() {
    let a = [1, 2, 3, 4, 5];
}
```

當你想要你的資料被配置在堆疊 (stack) 而不是堆積 (heap) 的話，使用陣列是很好的選擇（我們會在第四章討論堆疊與堆積的內容）。或者當你想確定你永遠會取得固定長度的元素時也是。所以陣列不像向量 (vector) 型別那麼有彈性，**向量**是標準函式庫提供的集合型別，類似於陣列但**允許**變更長度大小。如果你不確定該用陣列或向量的話，通常你應該用向量就好。第八章將會討論更多向量的細節。

不過如果你知道元素的多寡不會變的話，陣列就是個不錯的選擇。舉例來說，如果你想在程式中使用月份的話，你可能就會選擇用陣列宣告，因為永遠只會有 12 個月份：

```
let months = ["一月", "二月", "三月", "四月", "五月", "六月", "七月",  
             "八月", "九月", "十月", "十一月", "十二月"];
```

要詮釋陣列型別的話，你可以在中括號寫出型別和元素個數，並用分號區隔開來，如以下所示：

```
let a: [i32; 5] = [1, 2, 3, 4, 5];
```

`i32` 在此是每個元素的型別，在分號後面的數字 `5` 指的是此陣列有五個元素。

如果你想建立的陣列中每個元素數值都一樣的話，你可以指定一個數值後加上分號，最後寫出元素個數。如以下所示：

```
let a = [3; 5];
```

陣列 `a` 會包含 `5` 個元素，然後每個元素的初始化數值均為 `3`。這樣寫與 `let a = [3, 3, 3, 3, 3];` 的寫法一樣，但比較簡潔。

獲取陣列元素

一個陣列是被配置在堆疊上且已知固定大小的一整塊記憶體，你可以用索引來取得陣列的元素，比如：

檔案名稱：src/main.rs

```
fn main() {  
    let a = [1, 2, 3, 4, 5];  
  
    let first = a[0];  
    let second = a[1];  
}
```

在此範例中，變數 `first` 會得到數值 `1`，因為這是陣列索引 `[0]` 的數值。變數 `second` 則會從陣列索引 `[1]` 得到數值 `2`。

無效的陣列元素存取

讓我們看看如果我們存取陣列之後的元素會發生什麼事呢？假設你修改成以下範例，這是改寫自第二章猜謎遊戲要從使用者取得陣列索引的程式碼：

檔案名稱：src/main.rs

```
use std::io;

fn main() {
    let a = [1, 2, 3, 4, 5];

    println!("請輸入陣列索引");

    let mut index = String::new();

    io::stdin()
        .read_line(&mut index)
        .expect("讀行失敗");

    let index: usize = index
        .trim()
        .parse()
        .expect("輸入的索引並非數字");

    let element = a[index];

    println!(
        "索引 {index} 元素的數值為：{element}"
    );
}
```



此程式碼能編譯成功。如果你透過 `cargo run` 執行此程式碼並輸入 `0`、`1`、`2`、`3` 或 `4` 的話，程式將會印出陣列索引對應的數值。但如果你輸入超出陣列長度的數值，像是 `10` 的話，你會看到像是這樣的輸出結果：

```
thread 'main' panicked at 'index out of bounds: the len is 5 but the index is 10', src/main.rs:19:19
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

此程式會在使用無效數值進行索引操作時產生**執行時**（runtime）錯誤。程式會退出並回傳錯誤訊息，且不會執行最後的 `println!`。當你嘗試使用索引存取元素時，Rust 會檢查你的索引是否小於陣列長度，如果索引大於或等於陣列長度的話，Rust 就會恐慌。這樣的檢查必須發生在執行時，尤其是在此例，因為編譯器無法知道之後的使用者將會輸入哪些數值。

這是 Rust 記憶體安全原則給予的保障。在許多低階語言並不會提供這樣的檢查，所以當你提供不正確的索引時，無效的記憶體可能會被存取。Rust 會保護你免於這樣的錯誤風險，並立即離開程

式，而不是允許記憶體存取並繼續。第九章將會討論更多有關 Rust 的錯誤處理方式以及如何讓你寫出易讀且安全的程式碼，而不會恐慌或造成無效的記憶體存取。

函式

函式在 Rust 程式碼中無所不在。你已經見過一個語言最重要的函式了：`main` 函式是許多程式的入口點。此外你也看到了 `fn` 關鍵字能讓你宣告新的函式。

Rust 程式碼使用 *snake case* 式作為函式與變數名稱的慣例風格。所有的字母都是小寫，並用底線區隔單字。以下是一支包含函式定義範例的程式：

檔案名稱：src/main.rs

```
fn main() {  
    println!("Hello, world!");  
  
    another_function();  
}  
  
fn another_function() {  
    println!("另一支函式。");  
}
```

我們在 Rust 中定義函式是先從 `fn` 開始，再加上函式名稱和一組括號，大括號告訴編譯器函式本體的開始與結束位置。

我們可以輸入函式的名稱並加上括號來呼叫任何我們定義過的函式。因為 `another_function` 已經在程式中定義了，他就可以在 `main` 函式中呼叫。注意到我們是在原始碼中的 `main` 函式之後定義 `another_function` 的，我們當然也可以把它定義在前面。Rust 不在乎你的函式是在哪裡定義的，只需要知道它定義在作用域的某處，且能被呼叫者看到就好。

讓我們開啟一個新的專案叫做 *functions* 來進一步探索。請將 `another_function` 範例放入 `src/main.rs` 然後執行它。你應該會看到以下輸出：

```
$ cargo run  
Compiling functions v0.1.0 (file:///projects/functions)  
Finished dev [unoptimized + debuginfo] target(s) in 0.28s  
Running `target/debug/functions`  
Hello, world!  
另一支函式。
```

程式碼會按照 `main` 函式中的順序執行。首先，「Hello, world!」的訊息會先顯示出來，再來才會呼叫 `another_function` 並印出它的訊息。

參數

我們也可以定義函式成擁有參數（**parameters**）的，這是函式簽名（**signatures**）中特殊的變數。當函式有參數時，你可以提供那些參數的確切數值。嚴格上來說，我們傳遞的數值會叫做引數（**arguments**）。但為了方便起見，通常大家不太會去在意兩者的區別。雖然函式定義時才叫參數，傳遞數值時叫做引數，但很多情況下都能被交換使用。

以下的 `another_function` 是加上參數後的版本：

檔案名稱：src/main.rs

```
fn main() {
    another_function(5);
}

fn another_function(x: i32) {
    println!("x 的數值為：{x}");
}
```

嘗試執行程式的話，你應該會看到以下輸出結果：

```
$ cargo run
   Compiling functions v0.1.0 (file:///projects/functions)
   Finished dev [unoptimized + debuginfo] target(s) in 1.21s
   Running `target/debug/functions`
x 的數值為：5
```

宣告 `another_function` 時有一個參數叫做 `x`，而 `x` 的型別被指定為 `i32`。當我們傳遞 `5` 給 `another_function` 時，`println!` 巨集會將 `5` 置於格式化字串中的大括號內 `x` 的位置。

在函式簽名中，你必須宣告每個參數的型別，這是 Rust 刻意做下的設計決定：在函式定義中要求型別詮釋，代表編譯器幾乎不需要你在其他地方再提供資訊才能知道你要使用什麼型別。而且如果編譯器能知道函式預期的型別的話，它還能夠給予更有幫助的錯誤訊息。

如果要定義函式擁有數個參數時，會用逗號區隔開來，像這樣：

檔案名稱：src/main.rs

```
fn main() {
    print_labeled_measurement(5, 'h');
}

fn print_labeled_measurement(value: i32, unit_label: char) {
    println!("測量值為：{value}{unit_label}");
}
```

此範例建立了一個有兩個參數的函式 `print_labeled_measurement`，第一個參數叫做 `value` 而型別為 `i32`，第二個參數叫做 `unit_label` 而型別為 `char`。接著函式會印出包含 `value` 與 `unit_label` 的文字。

讓我們試著執行此程式碼，請覆蓋你的專案 `functions` 內的 `src/main.rs` 檔案內容為以上範例，然後用 `cargo run` 執行程式：

```
$ cargo run
Compiling functions v0.1.0 (file:///projects/functions)
Finished dev [unoptimized + debuginfo] target(s) in 0.31s
Running `target/debug/functions`
測量值為：5h
```

因為我們呼叫函式時，將 `5` 給了 `value` 且將 `'h'` 給了 `unit_label`，程式輸出就會包含這些數值。

陳述式與表達式

函式本體是由一系列的陳述式（statements）並在最後可以選擇加上表達式（expression）來組成。目前我們只講了沒有用到表達式做結尾的函式。由於 Rust 是門基於表達式（expression-based）的語言，知道這樣的區別是很重要的。其他語言通常沒有這樣的區別，所以現在讓我們來看看陳述式和表達式有什麼不同，以及它們怎麼影響函式本體。

- **陳述式**（Statements）是進行一些動作的指令，且不回傳任何數值。
- **表達式**（Expressions）則是計算並產生數值。讓我們來看一些範例：

我們其實已經使用了很多次陳述式與表達式。建立一個變數然後用 `let` 關鍵字賦值給它就是一道陳述式。在範例 3-1 中的 `let y = 6;` 就是個陳述式。

檔案名稱：`src/main.rs`

```
fn main() {
    let y = 6;
}
```

範例 3-1：包含一道陳述式的 `main` 函式宣告

此函式定義也是陳述式，整個範例本身就是一個陳述式。

陳述式不會回傳數值，因此你無法將 `let` 陳述式賦值給其他變數。如同以下程式碼所做的，你將會得到一個錯誤：

檔案名稱：`src/main.rs`

```
fn main() {  
    let x = (let y = 6);  
}
```

當你執行此程式時，你就會看到這樣的錯誤訊息：

```

$ cargo run
   Compiling functions v0.1.0 (file:///projects/functions)
error: expected expression, found `let` statement
--> src/main.rs:2:14
  |
2 |     let x = (let y = 6);
  |               ^^^

error: expected expression, found statement (`let`)
--> src/main.rs:2:14
  |
2 |     let x = (let y = 6);
  |               ^^^^^^^^^^^
  |
= note: variable declaration using `let` is a statement

error[E0658]: `let` expressions in this position are unstable
--> src/main.rs:2:14
  |
2 |     let x = (let y = 6);
  |               ^^^^^^^^^^^
  |
= note: see issue #53667 <https://github.com/rust-lang/rust/issues/53667> for
more information

warning: unnecessary parentheses around assigned value
--> src/main.rs:2:13
  |
2 |     let x = (let y = 6);
  |               ^       ^
  |
= note: `#[warn(unused_parens)]` on by default
help: remove these parentheses
  |
2 -     let x = (let y = 6);
2 +     let x = let y = 6;
  |

For more information about this error, try `rustc --explain E0658`.
warning: `functions` (bin "functions") generated 1 warning
error: could not compile `functions` due to 3 previous errors; 1 warning
emitted

```

`let y = 6` 陳述式不回傳數值，所以 `x` 得不到任何數值。這就和其他語言有所不同，像是 C 或 Ruby，通常它們的賦值仍能回傳所得到的值。在那些語言，你可以寫 `x = y = 6` 同時讓 `x` 與 `y` 都取得 6，但在 Rust 就不行。

表達式則會運算出一個數值，並組合成你大部分所寫的 Rust 程式。先想想看一個數學運算比如 `5 + 6`，這就是個會算出 `11` 的表達式。表達式可以是陳述式的一部分：在範例 3-1 中 `let y = 6;` 的 `6` 其實就是個算出 `6` 的表達式。呼叫函式也可以是表達式、呼叫巨集也是表達式、我們用 `{}` 產生的作用域也是表達式。舉例來說：

檔案名稱：src/main.rs

```
fn main() {  
    let x = 5;  
  
    let y = {  
        let x = 3;  
        x + 1  
    };  
  
    println!("y 的數值為：{y}");  
}
```

此表達式：

```
{  
    let x = 3;  
    x + 1  
}
```

就是一個會回傳 `4` 的區塊，此值再用 `let` 陳述式賦值給 `y`。請注意到 `x + 1` 這行沒有加上分號，它和你目前看到的寫法有點不同，因為表達式結尾不會加上分號。如果你在此表達式加上分號的話，它就不會回傳數值。在我們繼續探討函式回傳值與表達式的同時請記住這一點。

函式回傳值

函式可以回傳數值給呼叫它們的程式碼，我們不會為回傳值命名，但我們必須用箭頭（`->`）來宣告它們的型別。在 Rust 中，回傳值其實就是函式本體最後一行的表達式。你可以用 `return` 關鍵字加上一個數值來提早回傳函式，但多數函式都能用最後一行的表達式作為數值回傳。以下是一個有回傳數值的函式範例：

檔案名稱：src/main.rs

```
fn five() -> i32 {
    5
}

fn main() {
    let x = five();

    println!("x 的數值為：{x}");
}
```

在 `five` 函式中沒有任何函式呼叫、巨集甚至是 `let` 陳述式，只有一個 `5`。這在 Rust 中完全是合理的函式。請注意到函式的回傳型別也有指明，就是 `-> i32`。嘗試執行此程式的話，輸出結果就會像是這樣：

```
$ cargo run
   Compiling functions v0.1.0 (file:///projects/functions)
   Finished dev [unoptimized + debuginfo] target(s) in 0.30s
   Running `target/debug/functions`
x 的數值為：5
```

`five` 中的 `5` 就是函式的回傳值，這就是為何回傳型別是 `i32`。讓我們進一步研究細節，這邊有兩個重要的地方：首先這行 `let x = five();` 顯示了我們用函式的回傳值作為變數的初始值。因為函式 `five` 回傳 `5`，所以這行和以下程式碼相同：

```
let x = 5;
```

再來，`five` 函式沒有參數但有定義回傳值的型別。所以函式本體只需有一個 `5` 就好，不需加上分號，這樣就能當做表達式回傳我們想要的數值。

讓我們再看另一個例子：

檔案名稱：src/main.rs

```
fn main() {
    let x = plus_one(5);

    println!("x 的數值為：{x}");
}

fn plus_one(x: i32) -> i32 {
    x + 1
}
```

執行此程式會顯示 `x 的數值為：6`，但如果我們的最後一行 `x + 1` 加上分號的話，就會將它從表達式變為陳述式。我們就會得到錯誤：

檔案名稱：src/main.rs

```
fn main() {  
    let x = plus_one(5);  
  
    println!("x 的數值為：{x}");  
}  
  
fn plus_one(x: i32) -> i32 {  
    x + 1;  
}
```



編譯此程式就會產生以下錯誤：

```
$ cargo run  
   Compiling functions v0.1.0 (file:///projects/functions)  
error[E0308]: mismatched types  
  --> src/main.rs:7:24  
   |  
7 | fn plus_one(x: i32) -> i32 {  
   |     -----          ^^^ expected `i32`, found `()`  
   |     |  
   |     implicitly returns `()` as its body has no tail or `return` expression  
8 |     x + 1;  
   |         - help: removing this semicolon  
  
For more information about this error, try `rustc --explain E0308`.  
error: could not compile `functions` due to previous error
```

錯誤訊息 `mismatched types` 就告訴了我們此程式碼的核心問題。`plus_one` 的函式定義說它會回傳 `i32` 但是陳述式不會回傳任何數值。我們用單元型別 `()` 表示不會回傳任何值。因此沒有任何值被回傳，這和函式定義相牴觸，最後產生錯誤。在此輸出結果，Rust 提供了一道訊息來協助解決問題：它建議移除分號，這樣就能修正錯誤。

註解

所有程式設計師均致力於讓他們的程式碼易於閱讀，不過有時候額外的解釋還是需要的。這種情況下，開發者會在他們的程式碼留下一些**註解 (comments)**，編譯器會忽略這些字，但其他人在閱讀程式碼時可能就會覺得很有幫助。

這是一個簡單地註解：

```
// 安安，你好
```

在 Rust 中，慣用的註解風格是用兩行斜線再加上一個空格起頭，然後註解就能一直寫到該行結束為止。如果註解會超過一行的話，你需要在每一行都加上 `//`，如下所示：

```
// 這邊處理的事情很複雜，長到  
// 我們需要多行註解來解釋！  
// 希望此註解能幫助你理解。
```

註解也可以加在程式碼之後：

檔案名稱：src/main.rs

```
fn main() {  
    let lucky_number = 7; // 幸運 777 !  
}
```

不過你會更常看到它們用以下格式，註解會位於要說明的程式碼上一行：

檔案名稱：src/main.rs

```
fn main() {  
    // 幸運 777 !  
    let lucky_number = 7;  
}
```

Rust 還有另一種註解：技術文件註解。我們會在第十四章的「[發佈 Crate 到 Crates.io](#)」段落提到它。

控制流程

在大多數程式語言中，能夠決定依據某項條件是否為 `true` 來執行些程式碼，以及依據某項條件是否為 `true` 來重複執行些程式碼是非常基本的組成元件。在 Rust 程式碼中能讓你控制執行流程的常見方法有 `if` 表達式以及迴圈。

if 表達式

`if` 能讓你依照條件判斷對你的程式碼產生分支。基本上你提供一個條件然後就像是在說：「如果此條件符合的話，就執行這個程式碼區塊；如果沒有的話，就不要執行這段程式碼。」

請在你的 `projects` 目錄下建立一個新的專案叫做 `branches` 好讓我們來探討 `if` 表達式。接著請在 `src/main.rs` 檔案內輸入以下內容：

檔案名稱：`src/main.rs`

```
fn main() {  
    let number = 3;  
  
    if number < 5 {  
        println!("條件為真");  
    } else {  
        println!("條件為否");  
    }  
}
```

所有的 `if` 表達式都由 `if` 關鍵字開始再加上一個條件。在此例中的條件是判斷變數 `number` 是否小於 5。條件符合時所要執行的程式碼區塊被放在條件之後的大括號裡。與 `if` 表達式條件相關的程式碼段落有時也被稱為分支 (**arms**)，就像我們在第二章「將猜測的數字與祕密數字做比較」段落提到的 `match` 表達式的分支一樣。

另外，我們還可以選擇性地加上 `else` 表達式（就像範例寫的），讓條件不符時可以去執行另外一段程式碼。如果你沒有提供 `else` 表達式且條件為否的話，程式會直接略過 `if` 的程式碼區塊，接著執行後續的程式碼。

請嘗試執行此程式碼，你應該會看到以下輸出結果：

```
$ cargo run  
Compiling branches v0.1.0 (file:///projects/branches)  
Finished dev [unoptimized + debuginfo] target(s) in 0.31s  
Running `target/debug/branches`  
條件為真
```

讓我們來變更 `number` 的值使條件變成 `false`，再來看看會發生什麼事：

```
let number = 7;
```

再跑一次程式，然後看看輸出：

```
$ cargo run
  Compiling branches v0.1.0 (file:///projects/branches)
    Finished dev [unoptimized + debuginfo] target(s) in 0.31s
    Running `target/debug/branches`
條件為否
```

還有一點值得注意的是程式碼的條件判斷**必須是** `bool`。如果條件不是 `bool` 的話，我們就會遇到錯誤。比方說，試試以下程式碼：

檔案名稱：src/main.rs

```
fn main() {
    let number = 3;

    if number {
        println!("數字為三");
    }
}
```



這次 `if` 條件計算出數值 `3`，然後 Rust 丟出錯誤給我們：

```
$ cargo run
  Compiling branches v0.1.0 (file:///projects/branches)
error[E0308]: mismatched types
--> src/main.rs:4:8
   |
4 |     if number {
   |           ^^^^^ expected `bool`, found integer

For more information about this error, try `rustc --explain E0308`.
error: could not compile `branches` due to previous error
```

錯誤訊息告訴我們 Rust 預期收到 `bool` 但是卻拿到整數。這和 Ruby 和 JavaScript 就不同，Rust 不會自動將非布林值型別轉換成布林值。你永遠必須顯式提供布林值給 `if` 作為它的條件判斷。舉例來說，如果我們希望 `if` 只會在數值不為 `0` 才執行，我們可以將 `if` 表達式改成以下範例：

檔案名稱：src/main.rs

```
fn main() {
    let number = 3;

    if number != 0 {
        println!("數字不為零");
    }
}
```

執行此程式碼就會印出「數字不為零」。

使用 `else if` 處理多重條件

想要實現多重條件的話，你可以將 `if` 和 `else` 組合成 `else if` 表達式。舉例來說：

檔案名稱：src/main.rs

```
fn main() {
    let number = 6;

    if number % 4 == 0 {
        println!("數字可以被 4 整除");
    } else if number % 3 == 0 {
        println!("數字可以被 3 整除");
    } else if number % 2 == 0 {
        println!("數字可以被 2 整除");
    } else {
        println!("數字無法被 4、3、2 整除");
    }
}
```

程式有四種可能的分支，當你執行它時你應該會看到以下輸出結果：

```
$ cargo run
Compiling branches v0.1.0 (file:///projects/branches)
Finished dev [unoptimized + debuginfo] target(s) in 0.31s
Running `target/debug/branches`
數字可以被 3 整除
```

當此程式執行時，他會依序檢查每一個 `if` 表達式，並執行第一個條件為 `true` 的程式碼段落。注意到雖然 6 的確可以除以 2，但我們沒有看到「數字可以被 2 整除」，也沒有看到來自 `else` 那段「數字無法被 4、3、2 整除」。這是因為 Rust 只會執行第一個條件為 `true` 的區塊，而當它遇到時它就不會再檢查其他條件。

使用太多的 `else if` 表達式很容易讓你的程式碼變得凌亂，所以當你需要用到一個以上時，你可能會想要先重構程式碼看看。為此我們在第六章會介紹一個功能強大的 Rust 條件判斷結構叫做 `match`。

在 `let` 陳述式中使用 `if`

因為 `if` 是表達式，所以我們可以像範例 3-2 這樣放在 `let` 陳述式的右邊，將結果賦值給變數。

檔案名稱：src/main.rs

```
fn main() {  
    let condition = true;  
    let number = if condition { 5 } else { 6 };  
  
    println!("數字結果為：{number}");  
}
```

範例 3-2：將 `if` 表達式的結果賦值給變數

變數 `number` 會得到 `if` 表達式運算出的數值。執行此程式看看會發生什麼事：

```
$ cargo run  
Compiling branches v0.1.0 (file:///projects/branches)  
Finished dev [unoptimized + debuginfo] target(s) in 0.30s  
Running `target/debug/branches`  
數字結果為：5
```

你應該還記得程式碼區塊也可以是表達式且會回傳最後一行的數值，而且數字本身也是表達式。在此例中，`if` 表達式的值取決於哪段程式碼被執行。這代表可能成為最終結果的每一個 `if` 分支必須要是相同型別。在範例 3-2 中，各分支的型別都是 `i32`。如果型別不一致的話，如以下範例所示，我們會得到錯誤：

檔案名稱：src/main.rs

```
fn main() {  
    let condition = true;  
  
    let number = if condition { 5 } else { "六" };  
  
    println!("數字結果為：{number}");  
}
```



當我們嘗試編譯程式碼時，我們會得到錯誤。`if` 和 `else` 分支的型別並不一致，而且 Rust 還確切指出程式出錯的地方在哪：

```
$ cargo run
   Compiling branches v0.1.0 (file:///projects/branches)
error[E0308]: `if` and `else` have incompatible types
  --> src/main.rs:4:44
   |
4 |         let number = if condition { 5 } else { "六" };
   |                                     -          ^^^^^ expected integer, found
   |                                     `&str`
   |                                     |
   |                                     expected because of this

For more information about this error, try `rustc --explain E0308`.
error: could not compile `branches` due to previous error
```

使用迴圈重複執行

Rust 提供三種迴圈：`loop`、`while` 和 `for`。讓我們每個都嘗試看看吧。

loop 關鍵字告訴 Rust 去反覆不停地執行一段程式碼直到你親自告訴它要停下來。

檔案名稱：src/main.rs

當我們執行此程式時，我們會看到 再一次！ 一直不停地重複顯示出來，直到我們手動停下程式為止。大多數的終端機都支援 `ctrl-c` 這個快捷鍵來中斷一支卡在無限迴圈的程式，你可以自己試試看：

```
$ cargo run
Compiling loops v0.1.0 (file:///projects/loops)
Finished dev [unoptimized + debuginfo] target(s) in 0.29s
Running `target/debug/loops`
再一次！
再一次！
再一次！
再一次！
^C再一次！
```

`^C` 這個符號表示你按下了 `ctrl-c`。按照程式收到中斷訊號的時間點，你可能不會看到 `再一次！` 出現在 `^C` 之後。

幸運的是 Rust 有提供一個從程式中打破迴圈的方法。你可以在迴圈內加上 `break` 關鍵字告訴程式何時停止執行迴圈。回想一下我們在第二章「猜對後離開」段落就做過這樣的事，當使用者猜對正確數字而獲勝時就會離開程式。

我們在猜謎遊戲中也用到了 `continue`。這在迴圈中會告訴程式跳過這次疊代中剩餘的程式碼，然後進行下一次疊代。

從迴圈回傳數值

其中一種使用 `loop` 的用途是重試某些你覺得會失敗的動作，像是檢查一個執行緒是否已經完成其任務。這樣你可能就會想傳遞任務結果給之後的程式碼。要做到這樣的事，你可以在你要用來停下迴圈的 `break` 表達式內加上一個你想回傳數值，該值就會被停止的迴圈回傳，如以下所示：

```
fn main() {
    let mut counter = 0;

    let result = loop {
        counter += 1;

        if counter == 10 {
            break counter * 2;
        }
    };

    println!("結果為：{result}");
}
```

在迴圈之前，我們宣告了一個變數 `counter` 並初始化為 `0`，然後我們宣告了另一個變數 `result` 來取得迴圈回傳的值。在迴圈每一次的疊代中，我們將變數 `counter` 加上 `1` 並檢查它是否等於 `10`。如果是的話就用 `break` 關鍵字回傳 `counter * 2`。在迴圈結束後，我們用分號才結束這個賦值給 `result` 的陳述式。最後我們印出 `result`，而結果為 `20`。

用迴圈標籤辨別多重迴圈

如果你有迴圈在迴圈之內的話，`break` 和 `continue` 會用在該位置最內層的迴圈中。你可以選擇在迴圈使用迴圈標籤 (**loop label**)，然後使用 `break` 和 `continue` 加上那些迴圈標籤定義的關鍵字，而不是作用在最內層迴圈而已。以下是使用雙層巢狀迴圈的範例：

```
fn main() {
    let mut count = 0;
    'counting_up: loop {
        println!("count = {count}");
        let mut remaining = 10;

        loop {
            println!("remaining = {remaining}");
            if remaining == 9 {
                break;
            }
            if count == 2 {
                break 'counting_up;
            }
            remaining -= 1;
        }

        count += 1;
    }
    println!("End count = {count}");
}
```

外層迴圈有個 `'counting_up` 的標籤，而且其會從 0 數到 2。而內層沒有迴圈標籤的迴圈則會從 10 數到 9。第一個 `break` 沒有指定任何標籤只會離開內層迴圈。而陳述式 `break 'counting_up;` 則會離開外層迴圈。此程式碼會印出：

```
$ cargo run
Compiling loops v0.1.0 (file:///projects/loops)
Finished dev [unoptimized + debuginfo] target(s) in 0.58s
Running `target/debug/loops`
count = 0
remaining = 10
remaining = 9
count = 1
remaining = 10
remaining = 9
count = 2
remaining = 10
End count = 2
```

使用 `while` 做條件迴圈

在程式中用條件判斷迴圈的執行通常是很有用的。當條件為 `true` 時，迴圈就繼續執行。當條件不再是 `true` 時，程式就用 `break` 停止迴圈。這樣的循環行為可以用 `loop`、`if`、`else` 和 `break` 組合出來。如果你想嘗試的話，你現在就可以自己寫寫看看。但是這種模式非常常見，所以 Rust 有提供內建的結構稱為 `while` 迴圈。在範例 3-3 我們就是使用 `while` 讓該程式會循環三次，每次計數都減一，然後在迴圈之後印出訊息並離開。

檔案名稱：src/main.rs

```
fn main() {
    let mut number = 3;

    while number != 0 {
        println!("{number}!");

        number -= 1;
    }

    println!("升空!!!");
}
```

範例 3-3：使用 `while` 迴圈，當條件符合就持續執行程式碼

這樣消除了很多使用 `loop`、`if`、`else` 與 `break` 會有的巢狀結構，這樣可以更易閱讀。當條件為 `true` 的，程式碼就執行；不然的話，它就離開迴圈。

使用 `for` 遍歷集合

你可以用 `while` 來遍歷一個集合的元素，像是陣列等等。舉例來說，範例 3-4 的迴圈就會印出陣列 `a` 的每個元素。

檔案名稱：src/main.rs

```
fn main() {
    let a = [10, 20, 30, 40, 50];
    let mut index = 0;

    while index < 5 {
        println!("數值為：{}", a[index]);

        index += 1;
    }
}
```

範例 3-4：使用 `while` 遍歷集合的每個元素

程式在此對陣列的每個元素計數，它先從索引 `0` 開始，然後持續循環直到它抵達最後一個陣列索引為止（也就是 `index < 5` 不再為 `true`）。執行此程式會印出陣列裡的每個元素：

```
$ cargo run
Compiling loops v0.1.0 (file:///projects/loops)
Finished dev [unoptimized + debuginfo] target(s) in 0.32s
Running `target/debug/loops`
數值為：10
數值為：20
數值為：30
數值為：40
數值為：50
```

所有五個元素都如預期顯示在終端機上。儘管 `index` 會在某一刻達到 `5`，但是迴圈會在嘗試取得陣列第六個元素前就停止執行。

但這樣的方式是容易出錯的，我們可能取得錯誤的索引數值或測試條件而造成程式恐慌。這同時也使程式變慢，因為編譯器得在執行時的程式碼對迴圈中每次疊代中進行索引是否在陣列範圍內的條件檢查。

所以更簡潔的替代方案是，你可以使用 `for` 迴圈來對集合的每個元素執行一些程式碼。`for` 迴圈的樣子就像範例 3-5 寫的這一樣。

檔案名稱：src/main.rs

```
fn main() {
    let a = [10, 20, 30, 40, 50];

    for element in a {
        println!("數值為：{element}");
    }
}
```

範例 3-5：使用 `for` 迴圈遍歷集合的每個元素

當我們執行此程式時，我們會看到和範例 3-4 一樣的結果。最重要的是，我們增加了程式的安全性，去除了造成程式錯誤的可能性。不會出現超出陣列大小或是讀取長度不足的風險。

比方說在範例 3-4 的程式碼，如果你變更陣列 `a` 的元素為只有 4 個，但忘記更新條件判斷為 `while index < 4` 的話，程式就會恐慌。使用 `for` 迴圈的話，我們變更陣列長度時，就不需要去記得更新其他程式碼。

`for` 迴圈的安全性與簡潔程度讓它成為 Rust 最常被使用的迴圈結構。就算你想執行的是依照次數循環的程式碼，像是範例 3-3 的 `while` 迴圈範例，多數 Rustaceans 還是會選擇 `for` 迴圈。要這麼做的方法是使用 `Range`，這是標準函式庫提供的型別，用來產生一連串的數字序列，從指定一個數字開始一直到另一個數字之前結束。

以下是我們用 `for` 迴圈來計數的另一種方式，它用了一個我們還沒講過的方法 `rev`，這可以用來反轉這個範圍：

檔案名稱：src/main.rs

```
fn main() {  
    for number in (1..4).rev() {  
        println!("{number}!");  
    }  
    println!("升空!!!");  
}
```

這樣是不是看起來好讀許多？

總結

你做到了！這的確是篇大章節：你學到了變數、純量與複合資料型別、函式、註解、`if` 表達式以及迴圈！如果你想練習此章的概念，你可以試著打造以下程式：

- 轉換攝氏與華氏溫度。
- 產生第 n 個斐波那契數字。
- 試著用重複的歌詞印出 Christmas carol 的 The Twelve Days of Christmas。

當你準備好後，我們就來探討一個其他語言**不常見**的概念：所有權。

理解所有權

所有權可以說是 Rust 最與眾不同的特色，並深深影響著整個語言。這讓 Rust 不需要垃圾回收（garbage collector）就可以保障記憶體安全，因此理解 Rust 中的所有權如何運作至關重要。在本章節，我們將討論所有權以及一些相關的功能：借用、切片與 Rust 如何在記憶體配置資料。

什麼是所有權？

所有權在 Rust 中用來管理程式記憶體的一系列規則。

所有程式都需要在執行時管理它們使用記憶體的方式。有些語言會用垃圾回收機制，在程式執行時不斷尋找不再使用的記憶體；而有些程式，開發者必須親自配置和釋放記憶體。Rust 選擇了第三種方式：記憶體由所有權系統管理，且編譯器會在編譯時加上一些規則檢查。如果有地方違規的話，程式就無法編譯。這些所有權的規則完全不會降低執行程式的速度。

因為所有權對許多程式設計師來說是個全新的觀念，所以的確需要花一點時間消化。好消息是隨著你越熟悉 Rust 與所有權系統的規則，你越能輕鬆地開發出安全又高效的程式碼。加油，堅持下去！

當你理解所有權時，你就有一個穩健的基礎能夠理解那些使 Rust 獨特的功能。在本章節中，你將透過一些範例來學習所有權，我們會專注在一個非常常見的資料結構：字串。

堆疊（Stack）與堆積（Heap）

在許多程式語言中，你通常不需要去想到堆疊與堆積。但在像是 Rust 這樣的系統程式語言，資料是位於堆疊還是堆積就會有差，這會影響語言的行為也是你得作出某些特定決策的理由。在本章稍後討論所有權時，都會談到堆疊與堆積的關聯，所以這裡預先稍作解釋。

堆疊與堆積都是提供程式碼在執行時能夠使用的記憶體部分，但他們組成的方式卻不一樣。堆疊會按照它取得數值的順序依序存放它們，並以相反的順序移除數值。這通常稱為**後進先出（last in, first out）**。你可以把堆疊想成是盤子，當你要加入更多盤子，你會將它們疊在最上面。如果你要取走盤子的話，你也是從最上方拿走。想要從底部或中間，插入或拿走盤子都是不可行的！當我們要新增資料時，我們會稱呼為**推入堆疊（pushing onto the stack）**，而移除資料則是叫做**彈出堆疊（popping off the stack）**。所有在堆疊上的資料都必須是已知固定大小。在編譯時屬於未知或可能變更大小的資料必須儲存在堆積。

堆積就比較沒有組織，當你要將資料放入堆積，你得要求一定大小的空間。記憶體配置器（memory allocator）會找到一塊夠大的空位，標記為已佔用，然後回傳一個**指標（pointer）**，指著該位置的位址。這樣的過程稱為**在堆積上配置（allocating on the heap）**，或者有時直接簡稱為**配置（allocating）**就好（將數值放入堆疊不會被視為是在配置）。因為指標是固定已知的大小，所以你可以存在堆疊上。但當你要存取實際資料時，你就得去透過指標取得資料。你可以想像成是一個餐廳。當你進入餐廳時，你會告訴服務員你的團體有多少人，他就會將你們帶到足夠人數的餐桌。如果你的團體有人晚到的話，他們可以直接詢問你坐在哪而找到你。

將資料推入堆疊會比在堆積上配置還來的快，因為配置器不需要去搜尋哪邊才能存入新資料，其位置永遠在堆疊最上方。相對的，堆積就需要比較多步驟，配置器必須先找到一個夠大的空位來儲存資料，然後作下紀錄為下次配置做準備。

在堆積上取得資料也比在堆疊上取得來得慢，因為你需要用追蹤指標才找得到。現代的處理器如果在記憶體間跳轉越少的話速度就越快。讓我們繼續用餐廳做比喻，想像伺服器就是在餐廳為數個餐桌點餐。最有效率的點餐方式就是依照餐桌順序輪流點餐。如果幫餐桌 A 點了餐之後跑到餐桌 B 點，又跑回到 A 然後又跑到 B 的話，可以想像這是個浪費時間的過程。同樣的道理，處理器在處理任務時，如果處理的資料相鄰很近（就如同存在堆疊）的話，當然比相鄰很遠（如同存在堆積）來得快。

當你的程式碼呼叫函式時，傳遞給函式的數值（可能包含指向堆積上資料的指標）與函式區域變數會被推入堆疊。當函式結束時，這些數值就會被彈出。

追蹤哪部分的程式碼用到了堆積上的哪些資料、最小化堆積上的重複資料、以及清除堆積上沒在使用的資料確保你不會耗盡空間，這些問題都是所有權系統要處理的。一旦你理解所有權後，你通常就不再需要經常考慮堆疊與堆積的問題，不過能理解所有權主要就是為了管理堆積有助於解釋為何它要這樣運作。

所有權規則

首先，讓我們先看看所有權規則。當我們在解釋說明時，請記得這些規則：

- Rust 中每個數值都有個**擁有者 (owner)**。
- 同時間只能有一個擁有者。
- 當擁有者離開作用域時，數值就會被丟棄。

變數作用域

現在既然我們已經知道了基本語法，我們接下來就不再將 `fn main() {` 寫進程式碼範例範例中。所以你在參考時，請記得親自寫在 `main` 函式內。這樣一來，我們的範例可以更加簡潔，讓我們更加專注在細節而非樣板程式。

作為所有權的第一個範例，我們先來看變數的**作用域 (scope)**。作用域是一些項目在程式內的有效範圍。假設我們有以下變數：

```
let s = "hello";
```

變數 `s` 是一個字串字面值 (string literal)，而字串數值是寫死在我們程式內。此變數的有效範圍是從它宣告開始一直到當前**作用域**結束為止。範例 4-1 註解了 `s` 在哪裡是有效的。

```
{                                // s 在此處無效，因為它還沒宣告
    let s = "hello";           // s 在此開始視為有效

    // 使用 s

}
```

範例 4-1：變數與它在作用域的有效範圍

換句話說，這裡有兩個重要的時間點：

- 當 `s` 進入作用域時，它是有效的。
- 它持續被視為有效直到它離開作用域為止。

目前為止，變數何時有效與作用域的關係都還跟其他程式語言相似。現在我們要以此基礎來介紹

`String` 型別

String 型別

要能夠解釋所有權規則，我們需要使用比第三章的「資料型別」介紹過的還複雜的型別才行。之前我們提到的型別都是已知固定大小且儲存在堆疊上的，在作用域結束時就會從堆疊中彈出。而且如果其它部分的程式碼需要在不同作用域使用相同數值的話，它們都能迅速簡單地透過複製產生新的單獨實例。但是我們想要觀察的是儲存在堆積上的資料，並研究 Rust 是如何知道要清理資料的。而 `String` 型別正是個絕佳範例。

我們專注在 `String` 與所有權有關的部分。這些部分也適用於其他基本函式庫或你自己定義的複雜資料型別。我們會在第八章更深入探討 `String`。

我們已經看過字串字面值（string literals），字串的數值是寫死在我們的程式內的。字串字面值的確很方便，但它不可能完全適用於我們使用文字時的所有狀況。其中一個原因是因為它是不可變的，另一個原因是並非所有字串值在我們編寫程式時就會知道。舉例來說，要是我們想要收集使用者的輸入並儲存它呢？對於這些情形，Rust 提供第二種字串型別 `String`。此型別管理配置在堆積上的資料，所以可以儲存我們在編譯期間未知的一些文字。你可以從字串字面值使用 `from` 函式來建立一個 `String`，如以下所示：

```
let s = String::from("hello");
```

雙冒號 `::` 讓我們可以將 `from` 函式置於 `String` 型別的命名空間（namespace）底下，而不是取像是 `string_from` 這樣的名稱。我們將會在第五章的「方法語法」討論這個語法，並在第七章的「參考模組項目的路徑」討論模組（modules）與命名空間。

這種類型的字串是可以被改變的：

```
let mut s = String::from("hello");

s.push_str(", world!"); // push_str() 將字面值加到字串後面

println!("{}", s); // 這會印出 `hello, world!`
```

所以這邊有何差別呢？為何 `String` 是可變的，但字面值卻不行？兩者最主要的差別在於它們對待記憶體的方式。

記憶體與配置

以字串字面值來說，我們在編譯時就知道它的內容，所以可以寫死在最終執行檔內。這就是為何字串字面值非常迅速且高效。但這些特性均來自於字串字面值的不可變性。不幸的是我們無法將編譯時未知大小的文字，或是執行程式時大小可能會改變的文字等對應記憶體塞進執行檔中。

而對於 `String` 型別來說，為了要能夠支援可變性、改變文字長度大小，我們需要在堆積上配置一塊編譯時未知大小的記憶體來儲存這樣的內容，這代表：

- 記憶體配置器必須在執行時請求記憶體。
- 我們不再需要這個 `String` 時，我們需要以某種方法將此記憶體還給配置器。

當我們呼叫 `String::from` 時就等於完成第一個部分，它的實作會請求配置一塊它需要的記憶體。這邊大概和其他程式語言都一樣。

不過第二部分就不同了。在擁有**垃圾回收機制** (garbage collector, GC) 的語言中，GC 會追蹤並清理不再使用的記憶體，所以我們不用去擔心這件事。沒有 GC 的話，識別哪些記憶體不再使用並明確的呼叫程式碼釋放它們就是我們的責任了，就像我們請求取得它一樣。在以往的歷史我們可以看到要完成這件事是一項艱鉅的任務，如果我們忘了，那麼就等於在浪費記憶體。如果我們釋放的太早的話，我們則有可能會拿到無效的變數。要是我們釋放了兩次，那也會造成程式錯誤。我們必須準確無誤地配對一個 `allocate` 給剛好一個 `free`。

Rust 選擇了一條不同的道路：當記憶體在擁有它的變數離開作用域時就會自動釋放。以下是我們解釋作用域的範例 4-1，但使用的是 `String` 而不是原本的字串字面值：

```
{
    let s = String::from("hello"); // s 在此開始視為有效

    // 使用 s

}                                // 此作用域結束
                                // s 不再有效
```

當 `s` 離開作用域時，我們就可以很自然地將 `String` 所需要的記憶體釋放回配置器。當變數離開作用域時，Rust 會幫我們呼叫一個特殊函式。此函式叫做 `drop`，在這裡當時 `String` 的作者就可以寫入釋放記憶體的程式碼。Rust 會在大括號結束時自動呼叫 `drop`。

注意：在 C++，這樣在項目生命週期結束時釋放資源的模式，有時被稱為**資源取得即初始化 (Resource Acquisition Is Initialization, RAII)**。如果你已經用過 RAII 的模式，那麼你應該就會很熟悉 Rust 的 `drop` 函式。

這樣的模式對於 Rust 程式碼的編寫有很深遠的影響。雖然現在這樣看起來很簡單，但在更多複雜的情況下程式碼的行為可能會變得很難預測。像是當我們需要許多變數，所以得在堆積上配置它們的情況。現在就讓我們開始來探討這些情形。

變數與資料互動的方式：移動 (Move)

數個變數在 Rust 中可以有許多不同方式來與相同資料進行互動。讓我們看看使用整數的範例 4-2。

```
let x = 5;  
let y = x;
```

範例 4-2：將變數 `x` 的數值賦值給 `y`

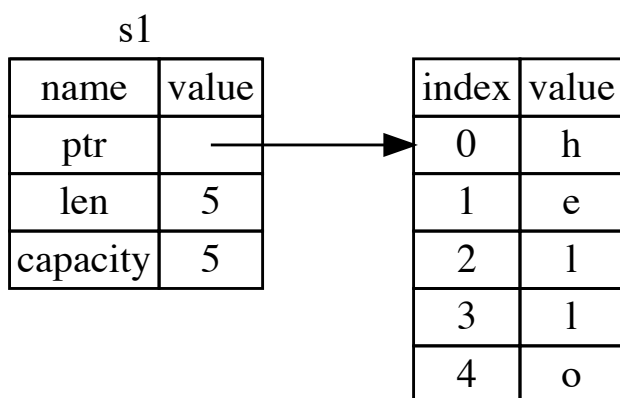
我們大概可以猜到這做了啥：「`x` 取得數值 5，然後拷貝 (copy) 了一份 `x` 的值給 `y`。」所以我們有兩個變數 `x` 與 `y`，而且都等於 5。這的確是我們所想的這樣，因為整數是已知且固定大小的簡單數值，所以這兩個數值 5 都會推入堆疊中。

現在讓我們看看 `String` 的版本：

```
let s1 = String::from("hello");  
let s2 = s1;
```

這和之前的程式碼非常相近，所以我們可能會認為它做的事也是一樣的：也就是第二行也會拿到一份 `s1` 拷貝的值給 `s2`。但事實上卻不是這樣。

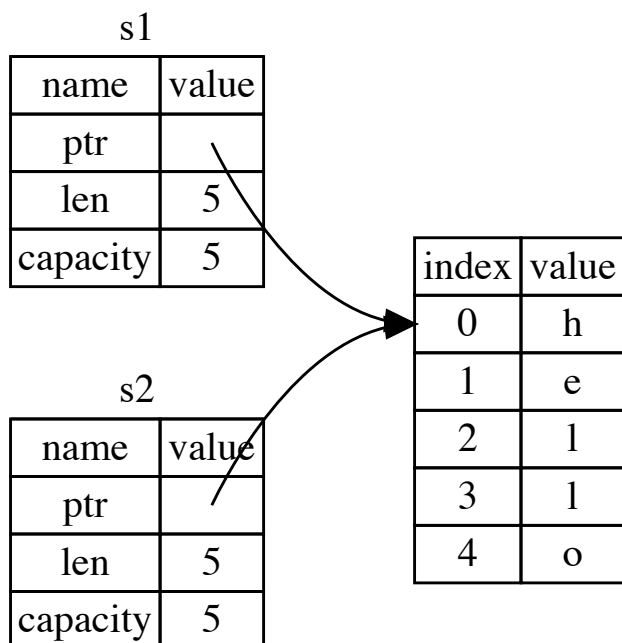
請看看圖示 4-1 來瞭解 `String` 底下的架構到底長什麼樣子。一個 `String` 由三個部分組成，如圖中左側所示：一個指向儲存字串內容記憶體指標、它的長度和它的容量。這些資料是儲存在堆疊上的，但圖右的內容則是儲存在堆積上。



圖示 4-1：將數值 "hello" 賦值給 `s1` 的 `String` 記憶體結構

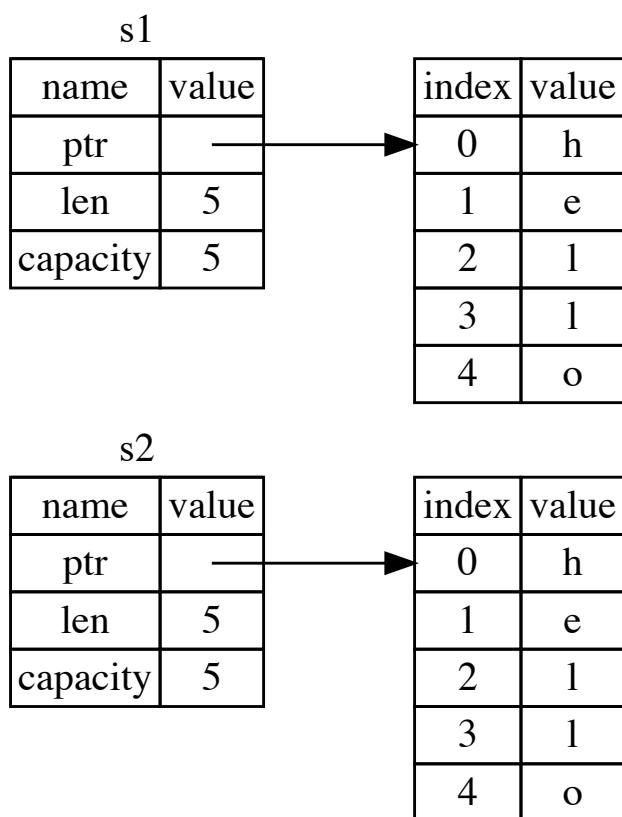
長度指的是目前所使用的 `String` 內容在記憶體以位元組為單位所佔用的大小。而容量則是 `String` 從配置器以位元組為單位取得的總記憶體量。長度和容量的確是有差別的，但現在對我們來說還不太重要，你現在可以先忽略容量的問題。

當我們將 `s1` 賦值給 `s2`，`String` 的資料會被拷貝，不過我們拷貝的是堆疊上的指標、長度和容量。我們不會拷貝指標指向的堆積資料。資料以記憶體結構表示的方式會如圖示 4-2 表示。



圖示 4-2： `s2` 擁有一份 `s1` 的指標、長度和容量的記憶體結構

所以實際上的結構不會長的像圖示 4-3 這樣，如果 Rust 也會拷貝堆積資料的話，才會看起來像這樣。如果 Rust 這麼做的話，`s2 = s1` 的動作花費會變得非常昂貴。當堆積上的資料非常龐大時，對執行時的性能影響是非常顯著的。



圖示 4-3：如果 Rust 也會拷貝堆積資料，`s2 = s1` 可能會長得樣子

稍早我們提到當變數離開作用域時，Rust 會自動呼叫 `drop` 函式並清理該變數在堆積上的資料。但圖示 4-2 顯示兩個資料指標都指向相同位置，這會造成一個問題。當 `s2` 與 `s1` 都離開作用域時，它們都會嘗試釋放相同的記憶體。這被稱呼為**雙重釋放**（double free）錯誤，也是我們之前提過的錯誤之一。釋放記憶體兩次可能會導致記憶體損壞，進而造成安全漏洞。

為了保障記憶體安全，在此情況中 Rust 還會再做一件重要的事。在 `let s2 = s1;` 之後，Rust 就不再將 `s1` 視為有效。因此當 `s1` 離開作用域時，Rust 不需要釋放任何東西。請看看如果在 `s2` 建立之後繼續使用 `s1` 會發生什麼事，以下程式就執行不了：

```
let s1 = String::from("hello");
let s2 = s1;

println!("{}", world!", s1);
```

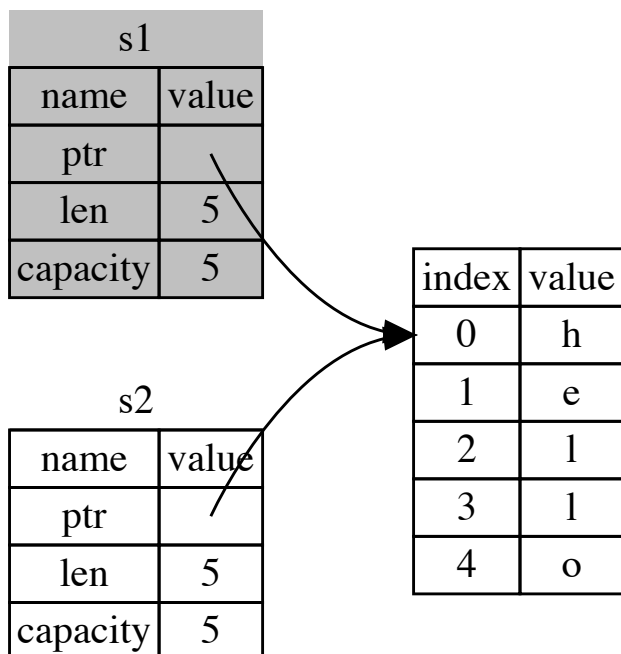


你會得到像這樣的錯誤，因為 Rust 會防止你使用無效的參考：

```
$ cargo run
   Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0382]: borrow of moved value: `s1`
  --> src/main.rs:5:28
   |
2  |     let s1 = String::from("hello");
   |         -- move occurs because `s1` has type `String`, which does not
   |         implement the `Copy` trait
3  |     let s2 = s1;
   |               -- value moved here
4  |
5  |     println!("{}", world!", s1);
   |                               ^^ value borrowed here after move
   |
   = note: this error originates in the macro `$crate::format_args_nl` which
   comes from the expansion of the macro `println` (in Nightly builds, run with -Z
   macro-backtrace for more info)

For more information about this error, try `rustc --explain E0382`.
error: could not compile `ownership` due to previous error
```

如果你在其他語言聽過**淺拷貝**（shallow copy）和**深拷貝**（deep copy）這樣的詞，拷貝指標、長度和容量而沒有拷貝實際內容這樣的概念應該就相近於淺拷貝。但因為 **Rust** 同時又無效化第一個變數，我們不會叫此為淺拷貝，而是稱此動作為**移動**（move）。在此範例我們會稱 `s1` 被移動到 `s2`，所以實際上發生的事長得像圖示 4-4 這樣。



圖示 4-4： `s1` 無效後的記憶體結構

這樣就解決了問題！只有 `s2` 有效的話，當它離開作用域，就只有它會釋放記憶體，我們就完成所有動作了。

除此之外，這邊還表達了另一個設計決策：Rust 永遠不會自動將你的資料建立「深拷貝」。因此任何自動的拷貝動作都可以被視為是對執行效能影響很小的。

變數與資料互動的方式：克隆（Clone）

要是我們真的想深拷貝 `String` 在堆積上的資料而非僅是堆疊資料的話，我們可以使用一個常見的方法（method）叫做 `clone`。我們會在第五章講解方法語法，不過既然方法是很常見的程式語言功能，你很可能已經有些概念了。

以下是 `clone` 方法運作的範例：

```
let s1 = String::from("hello");
let s2 = s1.clone();

println!("s1 = {}, s2 = {}", s1, s2);
```

此程式碼能執行無誤，並明確作出了像圖示 4-3 這樣的行為，也就是堆積資料的確被複製了一份。

當你看到 `clone` 的呼叫，你就會知道有一些特定的程式碼被執行且消費可能是相對昂貴的。你可以很清楚地知道有些不同的行為正在發生。

只在堆疊上的資料：拷貝 (Copy)

還有一個小細節我們還沒提到，也就是我們在使用整數時的程式碼。回想一下範例 4-2 是這樣寫的，它能執行而且是有效的：

```
let x = 5;
let y = x;

println!("x = {}, y = {}", x, y);
```

但這段程式碼似乎和我們剛學的互相矛盾：我們沒有呼叫 `clone`，但 `x` 卻仍是有效的，沒有移動到 `y`。

原因是因為像整數這樣的型別在編譯時是已知大小，所以只會存在在堆疊上。所以要拷貝一份實際數值的話是很快的。這也讓我們沒有任何理由要讓 `x` 在 `y` 建立後被無效化。換句話說，這邊沒有所謂淺拷貝與深拷貝的差別。所以這邊呼叫 `clone` 的話不會與平常的淺拷貝有啥不一樣，我們可以保持這樣就好。

Rust 有個特別的標記叫做 `Copy` 特徵 (trait) 可以用在標記像整數這樣存在堆疊上的型別（我們會在第十章討論什麼是特徵）。如果一個型別有 `Copy` 特徵的話，一個變數在賦值給其他變數後仍然會是有效的。

如果一個型別有實作 (implement) `Drop` 特徵的話，Rust 不會允許我們讓此型別擁有 `Copy` 特徵。如果我們對某個型別在數值離開作用域時，需要再做特別處理的話，我們對此型別標註 `Copy` 特徵會在編譯時期產生錯誤。想要瞭解如何為你的型別實作 `Copy` 特徵的話，請參考附錄 C「[可推導的特徵](#)」。

所以哪些型別有實作 `Copy` 特徵呢？你可以閱讀技術文件來知道哪些型別有，但基本原則是任何簡單地純量數值都可以實作 `Copy`，且不需要配置記憶體或任何形式資源的型別也有實作 `Copy`。以下是一些有實作 `Copy` 的型別：

- 所有整數型別像是 `u32`。
- 布林型別 `bool`，它只有數值 `true` 與 `false`。
- 所有浮點數型別像是 `f64`。
- 字元型別 `char`。
- 元組，不過包含的型別也都要有實作 `Copy` 才行。比如 `(i32, i32)` 就有實作 `Copy`，但 `(i32, String)` 則無。

所有權與函式

傳遞數值給函式的方式和賦值給變數是類似的。傳遞變數給函式會是移動或拷貝，就像賦值一樣。範例 4-3 說明了變數如何進入且離開作用域。

檔案名稱：src/main.rs

```

fn main() {
    let s = String::from("hello"); // s 進入作用域

    takes_ownership(s);           // s 的值進入函式
                                   // 所以 s 也在此無效

    let x = 5;                     // x 進入作用域

    makes_copy(x);                 // x 本該移動進函式裡
                                   // 但 i32 有 Copy，所以 x 可繼續使用
} // x 在此離開作用域，接著是 s。但因為 s 的值已經被移動了
   // 它不會有任何動作

fn takes_ownership(some_string: String) { // some_string 進入作用域
    println!("{}", some_string);
} // some_string 在此離開作用域並呼叫 `drop`
   // 佔用的記憶體被釋放

fn makes_copy(some_integer: i32) { // some_integer 進入作用域
    println!("{}", some_integer);
} // some_integer 在此離開作用域，沒有任何動作發生

```

範例 4-3：具有所有權的函式

如果我們嘗試在呼叫 `takes_ownership` 後使用 `s`，Rust 會拋出編譯時期錯誤。這樣的靜態檢查可以免於我們犯錯。你可以試試看在 `main` 裡哪裡可以使用 `s` 和 `x`，以及所有權規則如何防止你寫錯。

回傳值與作用域

回傳值一樣能轉移所有權，範例 4-4 和範例 4-3 一樣加上了註解說明一個函式如何回傳些數值。

檔案名稱：src/main.rs

```

fn main() {
    let s1 = gives_ownership();           // gives_ownership 移動它的回傳值給 s1

    let s2 = String::from("哈囉");       // s2 進入作用域

    let s3 = takes_and_gives_back(s2);   // s2 移入 takes_and_gives_back
                                           // 該函式又將其回傳值移到 s3
} // s3 在此離開作用域並釋放
   // s2 已被移走，所以沒有任何動作發生
   // s1 離開作用域並釋放

fn gives_ownership() -> String {         // gives_ownership 會將他的回傳值
                                           // 移動給呼叫它的函式

    let some_string = String::from("你的字串"); // some_string 進入作用域

    some_string                             // 回傳 some_string 並移動給
                                           // 呼叫它的函式
}

// 此函式會取得一個 String 然後回傳它
fn takes_and_gives_back(a_string: String) -> String { // a_string 進入作用域

    a_string // 回傳 a_string 並移動給呼叫的函式
}

```

範例 4-4：轉移回傳值的所有權

變數的所有權每次都會遵從相同的模式：賦值給其他變數就會移動。當擁有堆積資料的變數離開作用域時，該數值就會被 `drop` 清除，除非該資料的所有權被移動到其他變數所擁有。

雖然這樣是正確的，但在每個函式取得所有權再回傳所有權的確有點囉唆。要是我們可以讓函式使用一個數值卻不取得所有權呢？要是我們想重複使用同個值，但每次都要傳入再傳出實在是很麻煩。而且我們有時也會想要讓函式回傳一些它們自己產生的值。

Rust 能讓我們使用元組回傳多個數值，如範例 4-5 所示。

檔案名稱：src/main.rs


```
fn main() {  
    let s1 = String::from("hello");  
  
    let (s2, len) = calculate_length(s1);  
  
    println!("{}", s2, len);  
}  
  
fn calculate_length(s: String) -> (String, usize) {  
    let length = s.len(); // len() 回傳 String 的長度  
  
    (s, length)  
}
```

範例 4-5：回傳參數的所有權

但這實在太繁瑣，而且這樣的情況是很常見的。幸運的是 Rust 有提供一個概念能在不轉移所有權的狀況下使用數值，這叫做**參考 (references)**。

參考與借用

我們在範例 4-5 使用元組的問題在於，我們必須回傳 `String` 給呼叫的函式，我們才能繼續在呼叫 `calculate_length` 之後繼續使用 `String`，因為 `String` 會被傳入 `calculate_length`。不過我們其實可以提供個 `String` 數值的參考。**參考 (references)** 就像是指向某個地址的指標，我們可以追蹤存取到該處儲存的資訊，而該地址仍被其他變數所擁有。和指標不一樣的是，參考保證所指向的特定型別的數值一定是有效的。

以下是我們定義並使用 `calculate_length` 時，在參數改用參考物件而非取得所有權的程式碼：

檔案名稱：src/main.rs

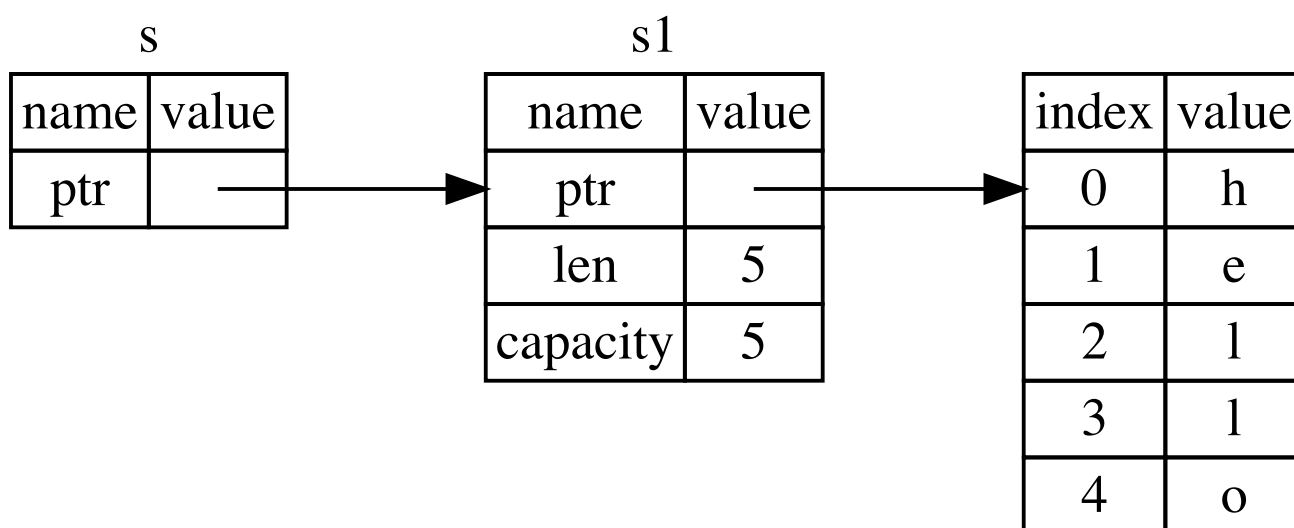
```
fn main() {
    let s1 = String::from("hello");

    let len = calculate_length(&s1);

    println!("{}", s1, len);
}

fn calculate_length(s: &String) -> usize {
    s.len()
}
```

首先你會注意到原先變數宣告與函式回傳值會用到元組的地方都被更改了。再來注意到我們傳遞的是 `&s1` 給 `calculate_length`，然後在定義時我們是取 `&String` 而非 `String`。這些「&」符號就是**參考**，它們允許你不必獲取所有權來參考它。以下用圖示 4-5 示意。



圖示 4-5：顯示 `&String s` 指向 `String s1` 的示意圖

注意：使用 `&` 參考的反向動作是**解參考 (dereferencing)**，使用的是解參考運算符號 `*`。我們會在第八章看到一些解參考的範例並在第 15 章詳細解釋解參考。

讓我們進一步看看函式的呼叫：

```
let s1 = String::from("hello");

let len = calculate_length(&s1);
```

`&s1` 語法讓我們可以建立一個指向 `s1` 數值的參考，但不會擁有它。因為它並沒有所有權，它所指向的資料在不再使用參考後並不會被丟棄。

同樣地，函式簽名也是用 `&` 說明參數 `s` 是個參考。讓我們加一些註解在範例上：

```
fn calculate_length(s: &String) -> usize { // s 是個 String 的參考
    s.len()
} // s 在此離開作用域，但因為它沒有它所指向的資料的所有權
// 所以不會被釋放掉
```

變數 `s` 有效的作用域和任何函式參數的作用域一樣，但當不再使用參考時，參考所指向的數值不會被丟棄，因為我們沒有所有權。當函式使用參考作為參數而非實際數值時，我們不需要回傳數值來還所有權，因為我們不曾擁有過。

我們會稱呼建立參考這樣的動作叫做**借用 (borrowing)**。就像現實世界一樣，如果有人擁有某項東西，他可以借用給你。當你使用完後，你就還給他。你並不擁有它。

所以要是我們嘗試修改我們借用的東西會如何呢？請試試範例 4-6 的程式碼。直接劇透你：它執行不了的！

檔案名稱：src/main.rs

```
fn main() {
    let s = String::from("hello");

    change(&s);
}

fn change(some_string: &String) {
    some_string.push_str(", world");
}
```



範例 4-6：嘗試修改借用的值

以下是錯誤訊息：

```
$ cargo run
   Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0596]: cannot borrow `*some_string` as mutable, as it is behind a `&`
reference
  --> src/main.rs:8:5
   |
7 | fn change(some_string: &String) {
   |                               ----- help: consider changing this to be a mutable
reference: `&mut String`
8 |     some_string.push_str(", world");
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ `some_string` is a `&` reference, so the data
it refers to cannot be borrowed as mutable

For more information about this error, try `rustc --explain E0596`.
error: could not compile `ownership` due to previous error
```

如同變數預設是不可變，參考也是一樣的。我們不被允許修改我們參考的值。

可變參考

我們可以修正範例 4-6 的程式碼，讓我們可以變更借用的數值。我們可以加一點小修改，改用可變參考就好：

檔案名稱：src/main.rs

```
fn main() {
    let mut s = String::from("hello");

    change(&mut s);
}

fn change(some_string: &mut String) {
    some_string.push_str(", world");
}
```

首先我們將 `s` 加上了 `mut`，然後我們在呼叫 `change` 函式的地方建立了一個可變參考 `&mut s`，然後更新函式的簽章成 `some_string: &mut String` 來接收這個可變參考。這樣能清楚表達 `change` 函式會改變它借用的參考。

可變參考有個很大的限制：如果你有一個數值的可變參考，你就無法再對該數值有其他任何參考。所以嘗試建立兩個 `s` 的可變參考的話就會失敗，如以下範例所示：

檔案名稱：src/main.rs

```
let mut s = String::from("hello");

let r1 = &mut s;
let r2 = &mut s;

println!("{}", {}, r1, r2);
```



以下是錯誤資訊：

```
$ cargo run
   Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0499]: cannot borrow `s` as mutable more than once at a time
--> src/main.rs:5:14
   |
4 |     let r1 = &mut s;
   |             ----- first mutable borrow occurs here
5 |     let r2 = &mut s;
   |             ^^^^^^^ second mutable borrow occurs here
6 |
7 |     println!("{}", {}, r1, r2);
   |                               -- first borrow later used here

For more information about this error, try `rustc --explain E0499`.
error: could not compile `ownership` due to previous error
```

此錯誤表示此程式碼是無效的，因為我們無法同時可變借用 `s` 超過一次。第一次可變借用在 `r1` 且必須持續到它在 `println!` 用完為止，但在其產生到使用之間，我們嘗試建立了另一個借用了與 `r1` 相同資料的可變借用 `r2`。

這項防止同時間對相同資料進行多重可變參考的限制允許了可變行為，但是同時也受到一定程度的約束。這通常是新 Rustaceans 遭受挫折的地方，因為多數語言都會任你去改變其值。這項限制的好處是 Rust 可以在編譯時期就防止資料競爭（data races）。**資料競爭**和競爭條件（race condition）類似，它會由以下三種行為引發：

- 同時有兩個以上的指標存取同個資料。
- 至少有一個指標在寫入資料。
- 沒有針對資料的同步存取機制。

資料競爭會造成未定義行為（undefined behavior），而且在執行時你通常是很難診斷並修正的。Rust 能夠阻止這樣的問題發生，不讓有資料競爭的程式碼編譯通過！

如往常一樣，我們可以用大括號來建立一個新的作用域來允許多個可變參考，只要不是同時擁有就好：

```
let mut s = String::from("hello");

{
    let r1 = &mut s;
} // r1 離開作用域，所以建立新的參考也不會有問題

let r2 = &mut s;
```

Rust 對於可變參考和不可變參考的組合中也實施著類似的規則，以下程式碼就會產生錯誤：

```
let mut s = String::from("hello");

let r1 = &s; // 沒問題
let r2 = &s; // 沒問題
let r3 = &mut s; // 很有問題！

println!("{}", {}, and {}", r1, r2, r3);
```



以下是錯誤訊息：

```
$ cargo run
   Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0502]: cannot borrow `s` as mutable because it is also borrowed as
immutable
--> src/main.rs:6:14
   |
4  |     let r1 = &s; // no problem
   |               -- immutable borrow occurs here
5  |     let r2 = &s; // no problem
6  |     let r3 = &mut s; // BIG PROBLEM
   |               ^^^^^^^ mutable borrow occurs here
7  |
8  |     println!("{}", {}, and {}", r1, r2, r3);
   |                                   -- immutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `ownership` due to previous error
```

哇！看來我們也不可以擁有不可變參考的同時擁有可變參考。

擁有不可變參考的使用者可不希望有人暗地裡突然改變了值！不過數個不可變參考是沒問題的，因為所有在讀取資料的人都無法影響其他人閱讀資料。

請注意參考的作用域始於它被宣告的地方，一直到它最後一次參考被使用為止。舉例來說以下程式就可以編譯，因為不可變參考最後一次的使用（`println!`）在可變參考宣告之前：

```
let mut s = String::from("hello");

let r1 = &s; // 沒問題
let r2 = &s; // 沒問題
println!("{}", r1, r2);
// 變數 r1 和 r2 將不再使用

let r3 = &mut s; // 沒問題
println!("{}", r3);
```

不可變參考 `r1` 和 `r2` 的作用域在 `println!` 之後結束。這是它們最後一次使用到的地方，也就是在宣告可變參考 `r3` 之前。它們的作用域沒有重疊，所以程式碼是允許的：編譯器能辨別出參考何時在作用域之前不再被使用。

雖然借用錯誤有時是令人沮喪的，但請記得這是 Rust 編譯器希望提前（在編譯時而非執行時）指出潛在程式錯誤並告訴你問題的源頭在哪。這樣你就不必親自追蹤為何你的資料跟你預期的不一樣。

迷途參考

在有指標的語言中，通常都很容易不小心產生**迷途指標**（**dangling pointer**）。當資源已經被釋放但指標卻還留著，這樣的指標指向的地方很可能就已經被別人所有了。相反地，在 Rust 中編譯器會保證參考絕不會是迷途參考：如果你有某些資料的參考，編譯器會確保資料不會在參考結束前離開作用域。

讓我們來嘗試產生迷途指標，看看 Rust 怎麼產生編譯期錯誤：

檔案名稱：src/main.rs

```
fn main() {
    let reference_to_nothing = dangle();
}

fn dangle() -> &String {
    let s = String::from("hello");

    &s
}
```



以下是錯誤訊息：

```
$ cargo run
Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0106]: missing lifetime specifier
--> src/main.rs:5:16
|
5 | fn dangle() -> &String {
|               ^ expected named lifetime parameter
|
= help: this function's return type contains a borrowed value, but there is
no value for it to be borrowed from
help: consider using the `static` lifetime
|
5 | fn dangle() -> &'static String {
|               ++++++

For more information about this error, try `rustc --explain E0106`.
error: could not compile `ownership` due to previous error
```

此錯誤訊息包含了一個我們還沒介紹的功能：生命週期 (lifetimes)。我們會在第十章詳細討論生命週期。就算我們先不管生命週期的部分，錯誤訊息仍然告訴了我們程式出錯的關鍵點：

```
this function's return type contains a borrowed value, but there is no value
for it to be borrowed from
```

讓我們進一步看看我們的 `dangle` 程式碼每一步發生了什麼：

檔案名稱：src/main.rs

```
fn dangle() -> &String { // 回傳 String 的迷途參考

    let s = String::from("hello"); // s 是個新 String

    &s // 我們回傳 String s 的參考
} // s 在此會離開作用域並釋放，它的記憶體就不見了。
// 危險！
```



因為 `s` 是在 `dangle` 內產生的，當 `dangle` 程式碼結束時，`s` 會被釋放。但我們卻嘗試回傳參考。此參考會指向一個已經無效的 `String`。這看起來不太優！Rust 不允許我們這麼做。

解決的辦法是直接回傳 `String` 就好：

```
fn no_dangle() -> String {
    let s = String::from("hello");

    s
}
```


這樣就沒問題了。所有權轉移了出去，沒有任何值被釋放。

參考規則

讓我們來回顧我們討論到的參考規則：

- 在任何時候，我們要嘛**只能有一個可變參考**，要嘛可以有**任意數量**的不可變參考。
- 參考必須永遠有效。

接下來我們要來看看一個不太一樣的參考型別：切片（slices）。

切片型別

切片 (slice) 讓你可以參考一串集合中的元素序列，而並非參考整個集合。切片也算是某種類型的參考，所以它沒有所有權。

以下是個小小的程式問題：寫一支函式接收一串用空格分開單字的字串，並回傳第一個找到的單字，如果函式沒有在字串找到空格的話，就代表整個字串就是一個單字，所以就回傳整個字串。

我們先來想看看不使用切片的話，以下函式的簽名會長怎樣。這有助於我們理解切片想解決什麼問題：

```
fn first_word(s: &String) -> ?
```

此函式 `first_word` 有一個參數 `&String`。我們不需要取得所有權，所以這是合理的。但我們該回傳啥呢？我們目前還沒有方法能夠描述一個字串的**其中一部分**。不過我們可以回傳單字的最後一個索引，也就是和空格作比較。讓我們像範例 4-7 這樣試試看。

檔案名稱：src/main.rs

```
fn first_word(s: &String) -> usize {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return i;
        }
    }

    s.len()
}
```

範例 4-7：函式 `first_word` 回傳參數 `String` 第一個單字最後的索引

因為我們需要遍歷 `String` 的每個元素並檢查該值是否為空格，我們要用 `as_bytes` 方法將 `String` 轉換成一個位元組陣列。

```
let bytes = s.as_bytes();
```

接下來我們使用 `iter` 方法對位元組陣列建立一個疊代器 (iterator)：

```
for (i, &item) in bytes.iter().enumerate() {
```

我們會在[第十三章](#)討論疊代器的細節。現在我們只需要知道 `iter` 是個能夠回傳集合中每個元素的方法，然後 `enumerate` 會將 `iter` 的結果包裝起來回傳成元組 (tuple)。 `enumerate` 回傳的元組中的第一個元素是索引，第二個才是元素的參考。這樣比我們自己計算索引還來的方便。

既然 `enumerate` 回傳的是元組，我們可以用模式配對來解構元組。我們會在[第六章](#)進一步解釋模式配對。所以在 `for` 迴圈中，我們指定了一個模式讓 `i` 取得索引然後 `&item` 取得元組中的位元組。因為我們從用 `.iter().enumerate()` 取得參考的，所以在模式中我們用的是 `&` 來獲取。

在 `for` 迴圈裡面我們使用字串字面值的語法搜尋位元組是不是空格。如果我們找到空格的話，我們就回傳該位置。不然我們就用 `s.len()` 回傳整個字串的長度。

```
        if item == b' ' {
            return i;
        }
    }

    s.len()
```

我們現在有了一個能夠找到字串第一個單字結尾索引的辦法，但還有一個問題。我們回傳的是一個獨立的 `usize`，它套用在 `&String` 身上才有意義。換句話說，因為它是個與 `String` 沒有直接關係的數值，我們無法保證它在未來還是有效的。參考一下使用了範例 4-7 中函式 `first_word` 的範例 4-8：

檔案名稱：src/main.rs

```
fn main() {
    let mut s = String::from("hello world");

    let word = first_word(&s); // word 取得數值 5

    s.clear(); // 這會清空 String，這就等於 ""

    // word 仍然是數值 5，但是我們已經沒有相等意義的字串了
    // 擁有 5 的變數 word 現在完全沒意義！
}
```

範例 4-8：先儲存呼叫函式 `first_word` 的結果再變更 `String` 的內容

此程式可以成功編譯沒有任何錯誤，而且我們在呼叫 `s.clear()` 後仍然能使用 `word`。因為 `word` 和 `s` 並沒有直接的關係，`word` 在之後仍能繼續保留 5。我們可以用 `s` 取得 5 並嘗試取得第一個單字。但這樣就會是程式錯誤了，因為 `s` 的內容自從我們賦值 5 給 `word` 之後的內容已經被改變了。

要隨時留意 `word` 會不會與 `s` 的資料脫鉤是很煩瑣的且容易出錯！要是我們又寫了個函式 `second_word`，管理這些索引會變得非常難以管控！我們會不得不將函式簽名改成這樣：

```
fn second_word(s: &String) -> (usize, usize) {
```

現在我們得同時紀錄起始與結束的索引，而且我們還產生了更多與原本數值沒辦法直接相關的計算結果。我們現在有三個非直接相關的變數需要保持同步。

幸運的是 Rust 為此提供了一個解決辦法：字串切片（String slice）。

字串切片

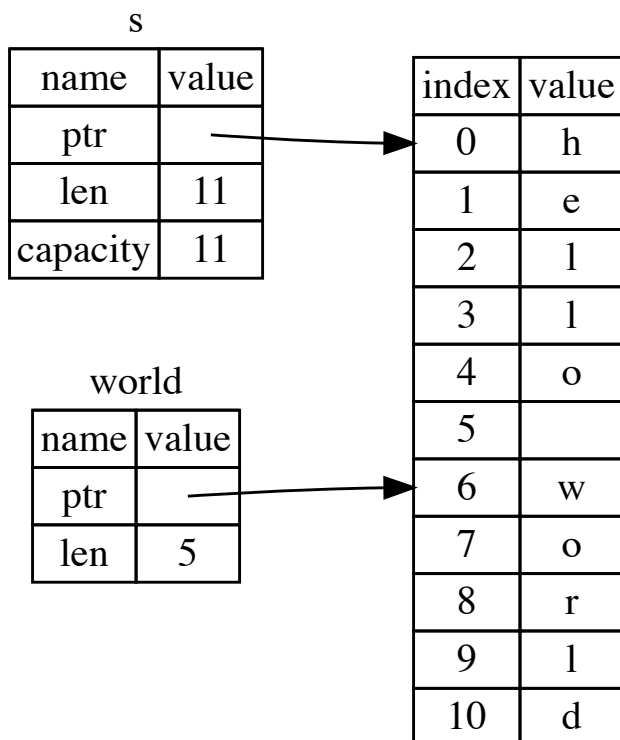
字串切片是 `String` 其中一部分的參考，它長得像這樣：

```
let s = String::from("hello world");

let hello = &s[0..5];
let world = &s[6..11];
```

與其參考整個 `String`，`hello` 只參考了一部分的 `String`，透過 `[0..5]` 來指示。我們可以像這樣 `[起始索引..結束索引]` 用中括號加上一個範圍來建立切片。起始索引 是切片的第一個位置，而 結束索引 在索引結尾之後的位置（所以不包含此值）。在內部的切片資料結構會儲存起始位置，以及 結束索引 與 起始索引 相減後的長度。所以用 `let world = &s[6..11];` 作為例子的話，`world` 就會是個切片，包含一個指標指向索引為 6 的位元組 `s` 和一個長度數值 5。

圖示 4-6 就是此例的示意圖。



圖示 4-6：指向部分 `String` 的字串切片

要是你想用 Rust 指定範圍的語法 `..` 從索引 0 開始的話，你可以省略兩個句點之前的值。換句話說，以下兩個是相等的：

```
let s = String::from("hello");

let slice = &s[0..2];
let slice = &s[..2];
```

同樣地，如果你的切片包含 `String` 的最後一個位元組的話，你同樣能省略最後一個數值。這代表以下都是相等的：

```
let s = String::from("hello");

let len = s.len();

let slice = &s[3..len];
let slice = &s[3..];
```

如果你要獲取整個字串的切片，你甚至能省略兩者的數值，以下都是相等的：

```
let s = String::from("hello");

let len = s.len();

let slice = &s[0..len];
let slice = &s[..];
```

注意：字串切片的索引範圍必須是有效的 UTF-8 字元界限。如果你嘗試從一個多位元組字元（multibyte character）中產生字串切片，你的程式就會回傳錯誤。為了方便介紹字串切片，本章只使用了 ASCII 字元而已。我們會在第八章的「[使用 String 儲存 UTF-8 編碼的文字](#)」做更詳盡的討論。

有了這些資訊，讓我們用切片來重寫 `first_word` 吧。對於「字串切片」的回傳型別我們會寫 `&str`：

檔案名稱：src/main.rs

```
fn first_word(s: &String) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}
```

我們如同範例 4-7 一樣用判斷第一個空格取得了單字結尾的索引。當我們找到第一個空格，我們用字串的初始索引與當前空格的索引作為初始與結束索引來回傳字串切片。

現在當我們呼叫 `first_word`，我們就會取得一個與原本資料有直接相關的數值。此數值是由切片的起始位置即切片中的元素個數組成。

這樣函式 `second_word` 一樣也可以回傳切片：

```
fn second_word(s: &String) -> &str {
```

我們現在有個不可能出錯且更直觀的 API，因為編譯器會確保 `String` 的參考會是有有效的。還記得我們在範例 4-8 的錯誤嗎？就是那個當我們取得單字結尾索引，但字串卻已清空變成無效的錯誤。那段程式碼邏輯是錯誤的，卻不會馬上顯示錯誤。要是我們持續嘗試用該索引存取空字串的話，問題才會浮現。切片可以讓這樣的程式錯誤無所遁形，並及早讓我們知道我們程式碼有問題。使用切片版本 `first_word` 的程式碼的話就會出現編譯期錯誤：

檔案名稱：src/main.rs

```
fn main() {
    let mut s = String::from("hello world");

    let word = first_word(&s);

    s.clear(); // 錯誤！

    println!("第一個單字為：{}", word);
}
```



以下是錯誤訊息：

```
$ cargo run
    Compiling ownership v0.1.0 (file:///projects/ownership)
error[E0502]: cannot borrow `s` as mutable because it is also borrowed as
immutable
    --> src/main.rs:18:5
   |
16 |     let word = first_word(&s);
   |                               -- immutable borrow occurs here
17 |
18 |     s.clear(); // 錯誤！
   |     ^^^^^^^^^ mutable borrow occurs here
19 |
20 |     println!("第一個單字為：{}", word);
   |                               ---- immutable borrow later used here

For more information about this error, try `rustc --explain E0502`.
error: could not compile `ownership` due to previous error
```

回憶一下借用規則，要是我們有不可變參考的話，我們就不能取得可變參考。因為 `clear` 會縮減 `String`，它必須是可變參考。在呼叫 `clear` 之後的 `println!` 用到了 `word` 的參考，所以不可變參考在該處仍必須保持有效。Rust 不允許同時存在 `clear` 的可變參考與 `word` 的不可變參考，所以編譯會失敗。Rust 不僅讓我們的 API 更容易使用，還想辦法讓所有錯誤在編譯期就消除！

字串字面值作為切片

回想一下我們講說字串字面值是怎麼存在執行檔的。現在既然我們已經知道切片，我們就能知道更清楚理解字串字面值：

```
let s = "Hello, world!";
```

此處 `s` 的型別是 `&str`：它是指向執行檔某部份的切片。這也是為何字串字面值是不可變的，`&str` 是個不可變參考。

字串切片作為參數

知道你可以取得字面值的切片與 `String` 數值後，我們可以再改善一次 `first_word`。也就是它的簽名表現：

```
fn first_word(s: &String) -> &str {
```

較富有經驗的 Rustacean 會用範例 4-9 的方式編寫函式簽名，因為這讓該函式可以同時接受 `&String` 和 `&str` 的數值。

```
fn first_word(s: &str) -> &str {
```

範例 4-9：使用字串切片作為參數 `s` 來改善函式 `first_word`

如果我們有字串切片的話，我們可以直接傳遞。如果我們有 `String` 的話，我們可以傳遞此 `String` 的切片或參考。這樣的彈性用到了**強制解參考**（deref coercion），這個功能我們會在第十五章的「[函式與方法的隱式強制解參考](#)」段落做介紹。

定義函式的參數為字串切片而非 `String` 可以讓我們的 API 更通用且不會失去任何功能：

檔案名稱：src/main.rs

```
fn main() {
    let my_string = String::from("hello world");

    // first_word 適用於 `String` 的切片，無論是部分或整體
    let word = first_word(&my_string[0..6]);
    let word = first_word(&my_string[..]);
    // first_word 也適用於 `String` 的參考，這等同於對整個 `String` 切片的操作。
    let word = first_word(&my_string);

    let my_string_literal = "hello world";

    // first_word 適用於字串字面值，無論是部分或整體
    let word = first_word(&my_string_literal[0..6]);
    let word = first_word(&my_string_literal[..]);

    // 因為字串字面值本來就是切片
    // 沒有切片語法也是可行的！
    let word = first_word(my_string_literal);
}
```

其他切片

字串切片如你所想的一樣是特別針對字串的。但是我們還有更通用的切片型別。請考慮以下陣列：

```
let a = [1, 2, 3, 4, 5];
```

就像我們參考一部分的字串一樣，我們可以這樣參考一部分的陣列：

```
let a = [1, 2, 3, 4, 5];

let slice = &a[1..3];

assert_eq!(slice, &[2, 3]);
```


此切片的型別為 `&[i32]`，它和字串運作的方式一樣，儲存了切片的第一個元素以及總長度。你以後會對其他集合也使用這樣的切片。我們會在第八章討論這些集合的更多細節。

總結

所有權、借用與切片的概念讓 Rust 可以在編譯時期就確保記憶體安全。Rust 程式語言讓你和其他程式語言一樣控制你的記憶體使用方式，但是會在擁有者離開作用域時自動清除擁有的資料，讓你不必再編寫或除錯額外的程式碼。

所有權影響了 Rust 很多其它部分執行的方式，所以我們在書中之後討論這些概念。讓我們繼續到第五章，看看如何用 `struct` 將資料組合在一起。

透過結構體組織相關資料

struct 或結構體（structure）是個讓你封裝並命名數個相關數值為單一組合的自定型別。如果你熟悉物件導向語言的話，*struct* 就像是物件的資料屬性。在本章節，我們會比較元組與結構體的差別，介紹如何使用結構體，並討論何時使用結構體組織資料是比較好的選擇。

我們將會解釋如何定義並產生結構體實例，我們也會討論如何定義關聯函式，尤其是叫做**方法（method）**的關聯函式，這能指定結構體型別特定的相關型別。結構體與將會在第六章提到的列舉（enum）是 Rust 產生新型別的基本元件，它們能充分利用 Rust 的編譯時型別檢查。

定義與實例化結構體

結構體 (Structs) 和我們在「[元組型別](#)」段落討論過的元組類似，兩者都能持有多種相關數值。和元組一樣，結構體的每個部分可以是不同的型別。但與元組不同的地方是，在結構體中你必須為每個資料部分命名以便表達每個數值的意義。因為有了這些名稱，結構體通常比元組還來的有彈性：你不需要依賴資料的順序來指定或存取實例中的值。

欲定義結構體，我們輸入關鍵字 `struct` 並為整個結構體命名。結構體的名稱需要能夠描述其所組合出的資料意義。然後在大括號內，我們對每個資料部分定義名稱與型別，我們會稱為欄位 (**fields**)。舉例來說，範例 5-1 定義了一個儲存使用者帳號的結構體。

檔案名稱：src/main.rs

```
struct User {  
    active: bool,  
    username: String,  
    email: String,  
    sign_in_count: u64,  
}
```

範例 5-1：User 結構體定義

要在我們定義後使用該結構體，我們可以指定每個欄位的實際數值來建立結構體的實例 (**instance**)。要建立實例的話，我們先寫出結構體的名稱再加上大括號，裡面會包含數個「key: value」的配對。key 是每個欄位的名稱，而 value 就是你想給予欄位的數值。欄位的順序可以不用和定義結構體時的順序一樣。換句話說，結構體的定義比較像是型別的通用樣板，然後實例會依據此樣板插入特定資料來將產生型別的數值。比如說，我們可以像範例 5-2 這樣宣告一個特定使用者。

檔案名稱：src/main.rs

```
fn main() {  
    let user1 = User {  
        active: true,  
        email: String::from("someone@example.com"),  
        username: String::from("someusername123"),  
        sign_in_count: 1,  
    };  
}
```

範例 5-2：產生一個 User 結構體的實例

要取得結構體中特定數值的話，我們使用句點。如果我們只是想要此使用者的電子郵件信箱，我們使用 `user1.email`。如果該實例可變的話，我們可以使用句點並賦值給該欄位來改變其值。範例

5-3 顯示了如何改變一個可變 `User` 實例中 `email` 欄位的值。

檔案名稱：src/main.rs

```
fn main() {  
    let mut user1 = User {  
        active: true,  
        email: String::from("someone@example.com"),  
        username: String::from("someusername123"),  
        sign_in_count: 1,  
    };  
  
    user1.email = String::from("anotheremail@example.com");  
}
```

範例 5-3：改變 `User` 中 `email` 欄位的值

請注意整個實例必須是可變的，Rust 不允許我們只標記特定欄位是可變的。再來，就像任何表達式一樣，我們可以在函式本體最後的表達式中，建立一個新的結構體實例作為回傳值。

範例 5-4 展示了 `build_user` 函式會依據給予的電子郵件和使用者名稱來回傳 `User` 實例。而 `active` 欄位取得數值 `true` 且 `sign_in_count` 取得數值 `1`。

檔案名稱：src/main.rs

```
fn build_user(email: String, username: String) -> User {  
    User {  
        active: true,  
        email: email,  
        username: username,  
        sign_in_count: 1,  
    }  
}
```

範例 5-4： `build_user` 函式取得電子郵件與使用者名稱並回傳 `User` 實例

函式參數名稱與結構體欄位名稱相同是非常合理的，但是要重複寫 `email` 和 `username` 的欄位名稱與變數就有點冗長了。如果結構體有更多欄位的話，重複寫這些名稱就顯得有些煩人了。幸運的是，我們的確有更方便的語法！

用欄位初始化簡寫語法

由於範例 5-4 的參數名稱與結構體欄位名稱相同，我們可以使用欄位初始化簡寫（field init shorthand）語法來重寫 `build_user`，讓它的結果相同但不必重複寫出 `email` 和 `username`，如範例 5-5 所示。

檔案名稱：src/main.rs

```
fn build_user(email: String, username: String) -> User {
    User {
        active: true,
        email,
        username,
        sign_in_count: 1,
    }
}
```

範例 5-5：build_user 函式使用欄位初始化簡寫，因為參數 username 與 email 結構體欄位相同

在此我們建立了 User 結構體的實例，它有一個欄位叫做 email。我們希望用 build_user 函式中的參數 email 賦值給 email 欄位。然後因為 email 欄位與 email 參數有相同的名稱，我們只要寫 email 就好，不必寫 email: email。

使用結構體更新語法從其他結構體建立實例

通常我們也會從其他的實例來產生新的實例，保留大部分欄位，不過修改一些欄位數值，這時你可以使用結構體更新語法（struct update syntax）。

首先範例 5-6 顯示了我們沒有使用更新語法來建立新的 User 實例 user2。我們設置了新的數值給 email，但其他欄位就使用我們在範例 5-2 建立的 user1 相同的值。

檔案名稱：src/main.rs

```
fn main() {
    // --省略--

    let user2 = User {
        active: user1.active,
        username: user1.username,
        email: String::from("another@example.com"),
        sign_in_count: user1.sign_in_count,
    };
}
```

範例 5-6：從 user1 中建立新的 User 實例

使用結構體更新語法，我們可以用較少的程式碼達到相同的效果，如範例 5-7 所示。.. 語法表示剩下沒指明的欄位都會取得與所提供的實例相同的值。

檔案名稱：src/main.rs

```
fn main() {
    // --省略--

    let user2 = User {
        email: String::from("another@example.com"),
        ..user1
    };
}
```

範例 5-7：對新的 `User` 實例設置新的 `email` 數值，但剩下就都使用 `user1`

範例 5-7 的程式碼產生的 `user2` 實例有不同 `email`，但是有與 `user1` 相同的 `username`、`active` 和 `sign_in_count`。 `..user1` 加在最後面表示任何剩餘的欄位都會與 `user1` 對應欄位的數值相同，不過我們可以用任意順序指定多少想指定的欄位，不需要與結構體定義欄位的順序一樣。

注意到結構體更新語法和賦值一樣使用 `=`，這是因為它也會轉移資料，就如同我們在「[變數與資料互動的方式：移動](#)」段落看到的一樣。在此範例中，我們在建立 `user2` 之後就無法再使用 `user1`，因為 `user1` 的 `username` 欄位的 `String` 被移到 `user2` 了。如果我們同時給 `user2` 的 `email` 與 `username` 新的 `String`，這樣 `user1` 會用到的數值只會有 `active` 和 `sign_in_count`，這樣 `user1` 在 `user2` 就仍會有效。因為 `active` 和 `sign_in_count` 都是有實作 `Copy` 特徵的型別，所以我們在「[變數與資料互動的方式：克隆](#)」段落討論到的行為會造成影響。

使用無名稱欄位的元組結構體來建立不同型別

Rust 還支援定義結構體讓它長得像是元組那樣，我們稱作**元組結構體 (tuple structs)**。元組結構體仍然有定義整個結構的名稱，但是它們的欄位不會有名稱，它們只會有欄位型別而已。元組結構體的用途在於當你想要為元組命名，好讓它跟其他不同型別的元組作出區別，以及對常規結構體每個欄位命名是冗長且不必要的時候。

要定義一個元組結構體，一樣先從 `struct` 關鍵字開始，其後再接著要定義的元組。舉例來說，以下是兩個使用元組結構體定義的 `Color` 和 `Point`：

檔案名稱：src/main.rs

```
struct Color(i32, i32, i32);
struct Point(i32, i32, i32);

fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0, 0, 0);
}
```

注意 `black` 與 `origin` 屬於不同型別，因為它們是不同的元組結構體實例。每個你定義的結構體都是專屬於自己的型別，就算它們的欄位型別可能一模一樣。舉例來說，一個參數為 `Color` 的函式就無法接受 `Point` 引數，就算它們的型別都是三個 `i32` 的組合。除此之外，元組結構體實例和元組類似，你可以將它們解構為獨立部分，你也可以使用 `.` 加上索引來取得每個數值。

無任何欄位的類單元結構體

你也可以定義沒有任何欄位的結構體！這些叫做類單元結構體（**unit-like structs**），因為它們的行為就很像我們在「元組型別」段落討論過的單元型別（unit type）`()` 類似。類單元結構體很適合用在當你要實作一個特徵（trait）或某種型別，但你沒有任何需要儲存在型別中的資料。我們會在第十章討論特徵。以下的範例宣告並實例化一個類單元結構體叫做 `AlwaysEqual`：

```
struct AlwaysEqual;

fn main() {
    let subject = AlwaysEqual;
}
```

我們使用 `struct` 關鍵字定義我們想要的名稱 `AlwaysEqual`，然後加上分號就好，不必再加任何括號！這樣我們就能一樣用 `subject` 變數取得一個 `AlwaysEqual` 的實例：直接使用我們定義的名稱，不用加任何括號。想像一下之後我們可以針對 `AlwaysEqual` 的實例實作與其他型別實例相同的行為，像是為了測試回傳已知的結果。我們不需要任何資料就能實作該行為！你能在第十章看到如何定義特徵（trait）並對任何型別實作它們，這也包含類單元結構體。

結構體資料的所有權

在範例 5-1 的 `User` 結構體定義中，我們使用了擁有所有權的 `String` 型別，而不是 `&str` 字串切片型別。這邊是故意這樣選擇的，因為我們希望每個結構體的實例可以擁有它所有的資料，並在整個結構體都有效時資料也是有效的。

要在結構體中儲存別人擁有的資料參考是可行的，但這會用到**生命週期（lifetimes）**，我們會在第十章才會談到。生命週期能確保資料參考在結構體存在期間都是有效的。要是你沒有使用生命週期來用結構體儲存參考的話，會如以下出錯：

檔案名稱：src/main.rs

```
struct User {  
    active: bool,  
    username: &str,  
    email: &str,  
    sign_in_count: u64,  
}  
  
fn main() {  
    let user1 = User {  
        active: true,  
        username: "someusername123",  
        email: "someone@example.com",  
        sign_in_count: 1,  
    };  
}
```



編譯器會抱怨它需要生命週期標記：


```

$ cargo run
   Compiling structs v0.1.0 (file:///projects/structs)
error[E0106]: missing lifetime specifier
  --> src/main.rs:3:15
   |
3  |     username: &str,
   |               ^ expected named lifetime parameter
   |
help: consider introducing a named lifetime parameter
   |
1  ~ struct User<'a> {
2  |     active: bool,
3  ~     username: &'a str,
   |

error[E0106]: missing lifetime specifier
  --> src/main.rs:4:12
   |
4  |     email: &str,
   |           ^ expected named lifetime parameter
   |
help: consider introducing a named lifetime parameter
   |
1  ~ struct User<'a> {
2  |     active: bool,
3  |     username: &str,
4  ~     email: &'a str,
   |

For more information about this error, try `rustc --explain E0106`.
error: could not compile `structs` due to 2 previous errors

```

在第十章，我們將會討論如何修正這樣的錯誤，好讓你可以在結構體中儲存參考。但現在的話，我們先用有所有權的 `String` 而非 `&str` 參考來避免錯誤。

使用結構體的程式範例

為了瞭解我們何時會想要使用結構體，讓我們來寫一支計算長方形面積的程式。我們會先從單一變數開始，再慢慢重構成使用結構體。

讓我們用 Cargo 建立一個新的專案 *rectangles*，它將接收長方形的長度與寬度，然後計算出長方形的面積。範例 5-8 展示了在我們專案底下 *src/main.rs* 用其中一種方式寫出來的小程式。

檔案名稱：*src/main.rs*

```
fn main() {
    let width1 = 30;
    let height1 = 50;

    println!(
        "長方形的面積為 {} 平方像素。",
        area(width1, height1)
    );
}

fn area(width: u32, height: u32) -> u32 {
    width * height
}
```

範例 5-8：使用變數 *width* 和 *height* 計算長方形面積

現在使用 `cargo run` 執行程式：

```
$ cargo run
Compiling rectangles v0.1.0 (file:///projects/rectangles)
Finished dev [unoptimized + debuginfo] target(s) in 0.42s
Running `target/debug/rectangles`
長方形的面積為 1500 平方像素。
```

雖然此程式碼成功呼叫 `area` 函式計算出長方形的面積，但我們可以做得更好，讓程式碼更簡潔且更易閱讀。

此程式碼的問題在 `area` 的函式簽名就能看出來：

```
fn area(width: u32, height: u32) -> u32 {
```

`area` 函式有寬度與長度兩個參數，可用以計算長方形的面積。但在我們的程式中，其參數相關性卻沒有表達出來。要是能將寬度與長度組合起來的話，會更容易閱讀與管理。我們可以使用我們在第三章提到的「[元組型別](#)」。

使用元組重構

範例 5-9 展示了我們的程式用元組的另一種寫法。

檔案名稱：src/main.rs

```
fn main() {
    let rect1 = (30, 50);

    println!(
        "長方形的面積為 {} 平方像素。",
        area(rect1)
    );
}

fn area(dimensions: (u32, u32)) -> u32 {
    dimensions.0 * dimensions.1
}
```

範例 5-9：使用元組指定長方形的寬度與長度

一方面來說，此程式的確比較好。元組讓我們增加了一些結構，而我們現在只需要傳遞一個引數。但另一方面來說，此版本的閱讀性反而更差。元組無法命名它的元素，所以我們需要索引部分元組，讓我們的計算變得比較不清晰。

我們在計算面積時，哪個值是寬度還是長度的確不重要。但如果我們要顯示出來的話，這就很重要了！我們會需要記住元組索引 0 是 `width` 然後元組索引 1 是 `height`。如果有其他人要維護這段程式碼的話，他就也記住這件事才能使用我們的程式碼。由於我們無法從程式碼表達出資料的意義，它就很容易產生錯誤。

使用結構體重構：賦予更多意義

我們可以用結構體來為資料命名以賦予其意義。我們可以將元組轉換成一個有整體名稱且內部資料也都有名稱的結構體，如範例 5-10 所示。

檔案名稱：src/main.rs

```

struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!(
        "長方形的面積為 {} 平方像素。",
        area(&rect1)
    );
}

fn area(rectangle: &Rectangle) -> u32 {
    rectangle.width * rectangle.height
}

```

範例 5-10：定義 `Rectangle` 結構體

我們在此定義了一個結構體叫做 `Rectangle`。在大括號內，我們定義了 `width` 與 `height` 的欄位，兩者型別皆為 `u32`。然後在 `main` 中，我們建立了一個寬度為 30 長度為 50 的 `Rectangle` 實例。

現在我們的 `area` 函式需要一個參數 `rectangle`，其型別為 `Rectangle` 結構體實例的不可變借用。如同第四章提到的，我們希望借用結構體而非取走其所有權。這樣一來，`main` 能保留它的所有權並讓 `rect1` 繼續使用，這也是為何我們要在要呼叫函式的簽名中使用 `&`。

`area` 函式能夠存取 `Rectangle` 中的 `width` 與 `height` 欄位（存取借用結構體實例的欄位不會移動欄位數值，這就是為何你常看到結構體的借用）。我們的 `area` 函式簽名可以表達出我們想要做的事情了：使用 `width` 與 `height` 欄位來計算 `Rectangle` 的面積。這能表達出寬度與長度之間的關係，並且給了它們容易讀懂的名稱，而不是像元組那樣用索引 0 和 1。這樣清楚多了。

使用推導特徵實現更多功能

現在要是能夠在我們除錯程式時能夠印出 `Rectangle` 的實例並看到它所有的欄位數值就更好了。範例 5-11 嘗試使用我們之前章節提到的 `println!` 巨集，但是卻無法執行。

檔案名稱：src/main.rs

```
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!("rect1 is {}", rect1);
}
```



範例 5-11：嘗試印出 `Rectangle` 實例

當我們編譯此程式碼時，我們會得到以下錯誤訊息：

```
error[E0277]: `Rectangle` doesn't implement `std::fmt::Display`
```

`println!` 巨集預設可以做各式各樣的格式化，大括號告訴 `println!` 要使用 `Display` 特徵的格式化方式：其輸出結果是用來給最終使用者使用的。我們目前遇過的基本型別預設都會實作 `Display`，因為它們也只有一種顯示方式（像是 `1`）能夠給使用者。但是對結構體來說 `println!` 要怎麼格式化輸出結果就會有點不明確了，因為顯示的方式就很有多種。是要加上頓號嗎？是要印出大括號嗎？所有的欄位都要顯示出來嗎？基於這些不確定因素，Rust 不會去猜我們要的是什麼，所以結構體預設並沒有 `Display` 的實作，也就無法使用 `println!` 與 `{}` 佔位符。

如果我們繼續閱讀錯誤訊息，我們會得到一些有幫助的資訊：

```
= help: the trait `std::fmt::Display` is not implemented for `Rectangle`
= note: in format strings you may be able to use `{:?}` (or `{:#?}` for
pretty-print) instead
```

讓我們來試試看吧！`println!` 巨集的呼叫方式現在看起來應該會像這樣 `println!("{:?}", rect1);`。在 `println!` 內加上 `?:` 這樣的標記指的是我們想要使用 `Debug` 特徵來作為輸出格式方式。`Debug` 特徵讓我們能印出對開發者有幫助的資訊，好讓我們在除錯程式時可以看到它的數值。

但是要是編譯這樣的程式的話，哎呀！我們卻還是會得到錯誤：

```
error[E0277]: `Rectangle` doesn't implement `Debug`
```

不過同樣地，編譯器又給了我們有用的資訊：

```
= help: the trait `Debug` is not implemented for `Rectangle`
= note: add `#[derive(Debug)]` to `Rectangle` or manually `impl Debug for Rectangle`
```

Rust 的確有印出除錯資訊的功能，但是我們要針對我們的結構體顯式實作出來才会有對應的功能。為此我們可以在結構體前加上屬性（attribute） `#[derive(Debug)]`，如範例 5-12 所示。

檔案名稱：src/main.rs

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!("rect1 is {:?}", rect1);
}
```

範例 5-12：加上屬性（attribute）來推導（derive） `Debug` 特徵的並印出 `Rectangle` 實例的格式化資訊

現在當我們執行程式，我們不會再得到錯誤了，而且我們可以看到格式化後的輸出結果：

```
$ cargo run
Compiling rectangles v0.1.0 (file:///projects/rectangles)
Finished dev [unoptimized + debuginfo] target(s) in 0.48s
Running `target/debug/rectangles`
rect1 is Rectangle { width: 30, height: 50 }
```

漂亮！雖然這不是非常好看的輸出格式，但是它的確顯示了實例中所有的欄位數值，這對我們除錯時會非常有用。不過如果我們的結構體非常龐大的話，我們會希望輸出格式可以比較好閱讀。為此我們可以在 `println!` 的字串使用 `{:#?}` 而非 `{:?}`。在此例中使用 `{:#?}` 風格的話，輸出結果就會如下：

```
$ cargo run
Compiling rectangles v0.1.0 (file:///projects/rectangles)
Finished dev [unoptimized + debuginfo] target(s) in 0.48s
Running `target/debug/rectangles`
rect1 is Rectangle {
  width: 30,
  height: 50,
}
```

另一種使用 `Debug` 格式印出數值的方式是使用 `dbg!` 巨集。這會拿走一個表達式的所有權（相較於 `println!` 只會拿參考），印出該 `dbg!` 巨集在程式碼中呼叫的檔案與行數，以及該表達式的數值結果，最後回傳該數值的所有權。

呼叫 `dbg!` 巨集會顯示到標準錯誤終端串流（`stderr`），而不像 `println!` 是印到標準輸出終端串流（`stdout`）。我們會在第十二章的「將錯誤訊息寫入標準錯誤而非標準輸出」段落進一步討論 `stderr` 與 `stdout`。

以下的範例我們印出賦值給 `width` 的數值，以及整個 `rect1` 結構體的數值：

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let scale = 2;
    let rect1 = Rectangle {
        width: dbg!(30 * scale),
        height: 50,
    };

    dbg!(&rect1);
}
```

我們在表達式 `30 * scale` 加上 `dbg!`，因為 `dbg!` 會回傳表達式的數值所有權，`width` 將能取得和不加上 `dbg!` 時相同的數值。而我們不希望 `dbg!` 取走 `rect1` 的所有權，所以我們在下一個 `rect1` 的呼叫使用參考。以下是此範例得到的輸出結果：

```
$ cargo run
Compiling rectangles v0.1.0 (file:///projects/rectangles)
Finished dev [unoptimized + debuginfo] target(s) in 0.61s
Running `target/debug/rectangles`
[src/main.rs:10] 30 * scale = 60
[src/main.rs:14] &rect1 = Rectangle {
    width: 60,
    height: 50,
}
```

我們可以看見第一個輸出結果來自 `src/main.rs` 第十行，也就是我們除錯表達式 `30 * scale` 的地方，其結果數值為 `60`（整數實作的 `Debug` 格式只會印出它們的數值）。而在 `src/main.rs` 第十四行所呼叫的 `dbg!` 則輸出 `&rect1` 的數值，也就是 `Rectangle` 的結構體。此輸出就會使用 `Rectangle` 實作的 `Debug` 漂亮格式。當你需要嘗試理解程式碼怎麼運作時，`dbg!` 巨集可以變得相當實用！

除了 `Debug` 特徵之外，Rust 還提供了一些特徵能讓我們透過 `derive` 屬性來使用並為我們的自訂型別擴增實用的行為。這些特徵與它們的行為有列在[附錄 C](#)。我們會在第十章介紹如何實作這些特徵的自訂行為，以及如何建立你自己的特徵。除了 `derive` 以外也有其他很多屬性，想了解更多資訊的話，請參考 Rust Reference 的「[Attributes](#)」段落。

我們的函式 `area` 最後就非常清楚明白了，它只會計算長方形的面積。這樣的行為要是能夠緊貼著我們的 `Rectangle` 結構體，因為這樣一來它就不會相容於其他型別。讓我們看看我們如何繼續重構我們的程式碼，接下來我們可以將函式 `area` 轉換為 `Rectangle` 型別的方法（**method**）。

方法語法

方法 (Methods) 和函式類似，我們用 `fn` 關鍵字並加上它們名稱來宣告，它們都有參數與回傳值，然後它們包含一些程式碼能夠在其他地方呼叫方法。和函式不同的是，方法是針對結構體定義的（或是列舉和特徵物件，我們會在[第六章](#)與[第十七章](#)分別介紹它們），且它們第一個參數永遠是 `self`，這代表的是呼叫該方法的結構體實例。

定義方法

讓我們把 `Rectangle` 作為參數的 `area` 函式轉換成定義在 `Rectangle` 內的 `area` 方法，如範例 5-13 所示。

檔案名稱：src/main.rs

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    println!(
        "長方形的面積為 {} 平方像素。",
        rect1.area()
    );
}
```

範例 5-13：在 `Rectangle` 中定義 `area` 方法

要定義 `Rectangle` 中的方法，我們先為 `Rectangle` 加個 `impl` (implementation) 區塊來開始。所有在此區塊的內容都跟 `Rectangle` 型別有關。再來將 `area` 移入 `impl` 的大括號中，並將簽名中的第一個參數（在此例中是唯一一個）與其本體中用到的地方改成 `self`。在 `main` 中我們

原先使用 `rect1` 作為引數呼叫的 `area`，可以改成使用方法語法（method syntax）來呼叫 `Rectangle` 的 `area` 方法。方法語法在實例後面呼叫，我們在其之後加上句點、方法名稱、括號然後任何所需的引數。

在 `area` 的簽名中，我們使用 `&self` 而非 `rectangle: &Rectangle`。`&self` 是 `self: &Self` 的簡寫。在一個 `impl` 區塊內，`Self` 型別是該 `impl` 區塊要實作型別的別名。方法必須有個叫做 `self` 的 `Self` 型別作為它們的第一個參數，所以 Rust 讓你在寫第一個參數時能直接簡寫成 `self`。注意到我們在 `self` 縮寫的前面仍使用 `&`，已表示此方法是借用 `Self` 的實例，就像我們在 `rectangle: &Rectangle` 做的一樣。就和其他參數一樣，方法可以選擇拿走 `self` 的所有權、像我們這裡借用不可變的 `self` 或是借用可變的 `self`。

我們之所以選擇 `&self` 的原因和我們在之前函式版本的 `&Rectangle` 一樣，我們不想取得所有權，只想讀取結構體的資料，而非寫入它。如果我們想要透過方法改變實例的數值的話，我們會使用 `&mut self` 作為第一個參數。而只使用 `self` 取得所有權的方法更是非常少見，這種使用技巧通常是為了想改變 `self` 成你想要的樣子，並且希望能避免原本被改變的實例繼續被呼叫。

使用方法而非函式最大的原因是，除了可以使用方法語法而不必在方法簽名重複 `self` 的型別之外，其更具組織性。我們將所有一個型別所能做的事都放入 `impl` 區塊中了，而不必讓未來的使用者在茫茫函式庫中尋找 `Rectangle` 的功能。

另外我們還可以選擇將方法的名稱取作其結構體的其中一個欄位。舉例來說，我們也可以在 `Rectangle` 定義一個 `width` 方法：

Filename: src/main.rs

```
impl Rectangle {
    fn width(&self) -> bool {
        self.width > 0
    }
}

fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };

    if rect1.width() {
        println!("長方形的寬度不為零，而是 {}", rect1.width);
    }
}
```

這裡我們選擇讓 `width` 方法判斷實例的 `width` 是否大於 0：如果是的話回傳 `true`；如果為 0 的話就回傳 `false`。我們可以讓欄位與方法擁有相同的名稱，並作為任何用途使用。在 `main` 中，當我們在 `rect1.width` 後方加上括號，Rust 就會知道我們指的是 `width` 方法。當我們沒有使用括號時，Rust 會知道我們指的是 `width` 欄位。

雖然不是必定的做法，但通常我們將方法名稱與欄位設為一樣時，我們希望它只回傳該欄位的數值而已。像這樣的方法稱為 **getter**，Rust 並不會像其他語言那樣自動為結構體欄位實作它們。Getter 常用於將欄位隱藏起來，但提供個公開方法並只限讀取該欄位，來做為該型別的公開 API。我們會在[第七章](#)討論什麼是公開與私有，以及如何設計方法或欄位為公開或私有的。

➔ 運算子跑去哪了？

在 C 與 C++ 中，我們有兩種呼叫方式的運算元：我們會用 `.` 來直接呼叫物件的方法；用 `->` 來呼叫需要先解參考的物件。換句話說，如果 `object` 是指標的話，`object->something()` 就會像是 `(*object).something()`。

Rust 沒有提供 `->` 這樣的運算子。相反地 Rust 有個功能叫做**自動參考與解參考 (automatic referencing and dereferencing)**。呼叫方法是 Rust 少數會有這樣行為的地方。

運作方式如下：當你呼叫方法像是 `object.something()` 時，Rust 會自動加上 `&`、`&mut` 或 `*`，以便符合方法簽名。換句話說，以下範例是相同的：

```
p1.distance(&p2);
(&p1).distance(&p2);
```

第一個呼叫簡潔多了，這種自動參考的行為之所以可行是因為方法有明確的 `self` 參考型別。依據接收者的方法名稱，Rust 可以知道該方法是在讀取（`&self`）、可變的（`&mut self`）或是會消耗的（`self`）。而 Rust 之所以允許借用方法接收者成隱式的原因，是因為這可以讓所有權更易讀懂。

擁有更多參數的方法

讓我們來練習再實作另一個 `Rectangle` 的方法。這次我們要 `Rectangle` 的實例可以接收另一個 `Rectangle` 實例，要是 `self` 本身（第一個 `Rectangle`）可以包含另一個 `Rectangle` 的話我們就回傳 `true`，不然的話就回傳 `false`。也就是我們希望定一個方法 `can_hold`，如範例 5-14 所示。

檔案名稱：src/main.rs

```
fn main() {
    let rect1 = Rectangle {
        width: 30,
        height: 50,
    };
    let rect2 = Rectangle {
        width: 10,
        height: 40,
    };
    let rect3 = Rectangle {
        width: 60,
        height: 45,
    };

    println!("rect1 能容納 rect2 嗎?{}", rect1.can_hold(&rect2));
    println!("rect1 能容納 rect3 嗎?{}", rect1.can_hold(&rect3));
}
```

範例 5-14：使用一個還沒定義完的方法 `can_hold`

然後我們預期的輸出結果會如以下所示，因為 `rect2` 的兩個維度都比 `rect1` 小，但 `rect3` 比 `rect1` 寬：

```
rect1 能容納 rect2 嗎?true
rect1 能容納 rect3 嗎?false
```

我們知道我們要定義方法的話，它一定得在 `impl Rectangle` 區塊底下。方法的名稱會叫做 `can_hold`。它會取得另一個 `Rectangle` 的不可變參考作為參數。我們可以從程式碼呼叫方法的地方來知道參數的可能的型別：`rect1.can_hold(&rect2)` 傳遞了 `&rect2`，這是一個 `rect2` 的不可變參考，同時也是 `Rectangle` 的實例。這是合理的，因為我們只需要讀取 `rect2`（而不是寫入，寫入代表我們需要可變參考），且我們希望 `main` 能夠保持 `rect2` 的所有權，好讓我們之後能在繼續使用它來呼叫 `can_hold` 方法。`can_hold` 的回傳值會是布林值，然後實作細節會是檢查 `self` 的寬度與長度是否都大於其他 `Rectangle` 的寬度與長度。讓我們加入範例 5-13 的 `can_hold` 方法到 `impl` 區塊中，如範例 5-15 所示。

檔案名稱：src/main.rs

```
impl Rectangle {
    fn area(&self) -> u32 {
        self.width * self.height
    }

    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}
```

範例 5-15：在 `Rectangle` 中實作了取得其他 `Rectangle` 作為參數的 `can_hold` 方法

當我們用範例 5-14 的 `main` 函式執行此程式碼的話，我們會得到預期的輸出結果。方法可以在參數 `self` 之後接收更多參數，而那些參數就和函式中的參數用法一樣。

關聯函式

所有在 `impl` 區塊內的方法都屬於**關聯函式 (associated functions)**，因為它們都與 `impl` 實作的型別相關。要是方法不需要自己的型別實例的話，我們可以定義個沒有 `self` 作為它們第一個參數的關聯函式（因此不會被稱作方法）。我們已經在 `String` 型別使用過 `String::from` 這種關聯函式了。

不屬於方法的關聯函式很常用作建構子，來產生新的結構體實例。這通常會叫做 `new`，但是 `new` 其實不是特殊名稱，也沒有內建在語言內。舉例來說，我們可以提供一個只接收一個維度作為參數的關聯函式，讓它賦值給寬度與長度，讓我們可以用 `Rectangle` 來產生正方形，而不必提供兩次相同的值：

檔案名稱：src/main.rs

```
impl Rectangle {
    fn square(size: u32) -> Self {
        Self {
            width: size,
            height: size,
        }
    }
}
```

回傳型別中與函式本體中的 `Self` 關鍵字是 `impl` 關鍵字接著出現的型別別名，在此例中就是 `Rectangle`。

要呼叫關聯函式的话，我們使用 `::` 語法並加上結構體的名稱。比方說 `let sq = Rectangle::square(3);`。此函式用結構體名稱作為命名空間，`::` 語法可以用在關聯函式以及模組的命名空間，我們會在第七章介紹模組。

多重 `impl` 區塊

每個結構體都允許有數個 `impl` 區塊。舉例來說，範例 5-15 與範例 5-16 展示的程式碼是一樣的，它讓每個方法都有自己的 `impl` 區塊。

```
impl Rectangle {  
    fn area(&self) -> u32 {  
        self.width * self.height  
    }  
}  
  
impl Rectangle {  
    fn can_hold(&self, other: &Rectangle) -> bool {  
        self.width > other.width && self.height > other.height  
    }  
}
```

範例 5-16：使用多重 `impl` 來重寫範例 5-15

這邊我們的確沒有將方法拆為 `impl` 區塊的理由，不過這樣的語法是合理的。我們會在第十章介紹泛型型別與特徵，看到多重 `impl` 區塊是非常實用的案例。

總結

結構體讓你可以自訂對你的領域有意義的型別。使用結構體的話，你可以讓每個資料部分與其他部分具有相關性，並為每個部分讓程式更好讀懂。在 `impl` 區塊中，你可以定義與你的型別有關的函式，而方法就是其中一種關聯函式，能讓你指定你的結構體能有何種行為。

但是結構體並不是自訂型別的唯一方法：讓我們看下去 Rust 的列舉功能，讓你的工具箱可以再多一項可以使用的工具。

列舉與模式配對

在本章節中，我們將討論**列舉** (enumerations)，有時也被簡寫為 *enums*。列舉讓你定義一個能夠列舉其可能變體 (variants) 的型別。首先，我們會定義並使用列舉來展示列舉如何將其數據組織起來。再來，我們會來探討一個特定的實用列舉：`Option`，其代表該值為某些東西不然就是什麼都沒有。然後我們會看看 `match` 表達式的模式配對是怎麼運作的，讓它能夠針對列舉中不同數值執行不同的程式碼。最後，我們會介紹 `if let` 這個結構，來用簡潔又方便的方式處理列舉。

定義列舉

結構體讓你能將相關的欄位與資訊組織在一起，像是 `Rectangle` 中的 `width` 與 `height`，而列舉則是讓你能表達一個數值屬於一組特定數值的其中一種。舉例來說，我們可能想要表達 `Rectangle` 是其中一種可能的形狀，而這些形狀可能還包括 `Circle` 與 `Triangle`。Rust 讓我們能以列舉的形式表現這樣的可能性。

讓我們看一個程式碼表達的例子，來看看為何此時用列舉會比結構體更恰當且實用。假設我們要使用 IP 位址，而且現在有兩個主要的標準能使用 IP 位址：IPv4 與 IPv6。這些是我們的程式碼可能會遇到的 IP 位址，我們可以**列舉** (enumerate) 出所有可能的變體，這正是列舉的由來。

任何 IP 位址可以是第四版或第六版，但不是同時存在。IP 位址這樣的特性非常適合使用列舉資料結構，因為列舉的值只能是其中一個變體。第四版與第六版同時都屬於 IP 位址，所以當有程式碼要處理任何類型的 IP 位址時，它們都應該被視為相同型別。

要表達這樣的概念，我們可以定義 `IpAddrKind` 列舉和列出 IP 位址可能的類型 `v4` 和 `v6`。這些稱為列舉的變體 (variants)：

```
enum IpAddrKind {  
    V4,  
    V6,  
}
```

`IpAddrKind` 現在成了能在我們程式碼任何地方使用的自訂資料型別。

列舉數值

我們可以像這樣建立兩個不同變體的 `IpAddrKind` 實例：

```
let four = IpAddrKind::V4;  
let six = IpAddrKind::V6;
```

注意變體會位於列舉命名空間底下，所以我們用兩個冒號來標示。這樣的好處在於 `IpAddrKind::V4` 和 `IpAddrKind::V6` 都是同型別 `IpAddrKind`。比方說，我們就可以定義一個接收任 `IpAddrKind` 的函式：

```
fn route(ip_kind: IpAddrKind) {}
```

然後我們可以用任意變體呼叫此函式：


```
route(IPAddrKind::V4);
route(IPAddrKind::V6);
```

使用列舉還有更多好處。我們再進一步想一下我們的 IP 位址型別還沒有辦法儲存實際的 IP 位址資料，我們現在只知道它是哪種類型。考慮到你已經學會第五章的結構體，你應該會像範例 6-1 這樣嘗試用結構體解決問題。

```
enum IPAddrKind {
    V4,
    V6,
}

struct IPAddr {
    kind: IPAddrKind,
    address: String,
}

let home = IPAddr {
    kind: IPAddrKind::V4,
    address: String::from("127.0.0.1"),
};

let loopback = IPAddr {
    kind: IPAddrKind::V6,
    address: String::from("::1"),
};
```

範例 6-1：使用 `struct` 儲存 IP 位址的資料與 `IPAddrKind` 的變體

我們在這裡定義了一個有兩個欄位的結構體 `IPAddr`：欄位 `kind` 擁有 `IPAddrKind`（我們上面定義過的列舉）型別，`address` 欄位則是 `String` 型別。再來我們有兩個此結構體的實例。第一個 `home` 擁有 `IPAddrKind::V4` 作為 `kind` 的值，然後位址資料是 `127.0.0.1`。第二個實例 `loopback` 擁有 `IPAddrKind` 另一個變體 `V6` 作為 `kind` 的值，且有 `::1` 作為位址資料。我們用結構體來組織 `kind` 和 `address` 的值在一起，讓變體可以與數值相關。

但是我們可以用另一種更簡潔的方式來定義列舉就好，而不必使用結構體加上列舉。列舉內的每個變體其實都能擁有數值。以下這樣新的定義方式讓 `IPAddr` 的 `V4` 與 `V6` 都能擁有與其相關的 `String` 數值：

```
enum IpAddr {
    V4(String),
    V6(String),
}

let home = IpAddr::V4(String::from("127.0.0.1"));

let loopback = IpAddr::V6(String::from("::1"));
```

我們將資料直接附加到列舉的每個變體上，這樣就不再用結構體。這裏我們還能看到另一項列舉的細節：我們定義的每一個列舉變體也會變成建構該列舉的函式。也就是說 `IpAddr::V4()` 是個函式，且接收 `String` 引數並回傳 `IpAddr` 的實例。我們在定義列舉時就會自動拿到這樣的建構函式。

改使用列舉而非結構體的話還有另一項好處：每個變體可以擁有不同型別與資料的數量。第四版的 IP 位址永遠只會有四個 0 到 255 的數字部分，如果我們想要讓 `V4` 儲存四個 `u8`，但 `V6` 位址仍保持 `String` 不變的話，我們在結構體是無法做到的。列舉可以輕鬆勝任：

```
enum IpAddr {
    V4(u8, u8, u8, u8),
    V6(String),
}

let home = IpAddr::V4(127, 0, 0, 1);

let loopback = IpAddr::V6(String::from("::1"));
```

我們展示了許多種定義儲存第四版與第六版 IP 位址資料結構的方式，不過需要儲存 IP 位址並編碼成不同類型的案例實在太常見了，所以[標準函式庫已經幫我們定義好了！](#)讓我們看看標準函式庫是怎麼定義 `IpAddr` 的：它有和我們一模一樣的列舉變體，不過變體各自儲存的資料是另外兩個不同的結構體，兩個定義的內容均不相同：

```
struct Ipv4Addr {
    // --省略--
}

struct Ipv6Addr {
    // --省略--
}

enum IpAddr {
    V4(Ipv4Addr),
    V6(Ipv6Addr),
}
```

此程式碼展示了你可以將任何資料類型放入列舉的變體中：字串、數字型別、結構體都可以。你甚至可以再包含另一個列舉！另外標準函式庫內的型別常常沒有你想得那麼複雜。

請注意雖然標準函式庫已經有定義 `IpAddr`，但我們還是可以使用並建立我們自己定義的型別，而且不會產生衝突，因為我們還沒有將標準函式庫的定義匯入到我們的作用域中。我們會在第七章討論如何將型別匯入作用域內。

讓我們再看看範例 6-2 的另一個列舉範例，這次的變體有各式各樣的型別。

```
enum Message {  
    Quit,  
    Move { x: i32, y: i32 },  
    Write(String),  
    ChangeColor(i32, i32, i32),  
}
```

範例 6-2： `Message` 列舉的變體各自擁有不同的型別與數值數量

此列舉有四個不同型別的變體：

- `Quit` 沒有包含任何資料。
- `Move` 包含了和結構體一樣的名稱欄位。
- `Write` 包含了一個 `String`。
- `ChangeColor` 包含了三個 `i32`。

如同範例 6-2 這樣定義列舉變體和定義不同類型的結構體很像，只不過列舉不使用 `struct` 關鍵字，而且所有的變體都會在 `Message` 型別底下。以下的結構體可以包含與上方列舉變體定義過的資料：

```
struct QuitMessage; // 類單元結構體  
struct MoveMessage {  
    x: i32,  
    y: i32,  
}  
struct WriteMessage(String); // 元組結構體  
struct ChangeColorMessage(i32, i32, i32); // 元組結構體
```

但是如果我們使用不同結構體且各自都有自己的型別的話，我們就無法像範例 6-2 那樣將 `Message` 視為單一型別，輕鬆在定義函式時接收訊息所有可能的類型。

列舉和結構體還有一個地方很像：如同我們可以對結構體使用 `impl` 定義方法，我們也可以對列舉定義方法。以下範例顯示我們可以對 `Message` 列舉定義一個 `call` 方法：

```
impl Message {
    fn call(&self) {
        // 在此定義方法本體
    }
}

let m = Message::Write(String::from("hello"));
m.call();
```

方法本體使用 `self` 來取得我們呼叫方法的值。在此例中，我們建立了一個變數 `m` 並取得 `Message::Write(String::from("hello"))`，而這就會是當我們執行 `m.call()` 時 `call` 方法內會用到的 `self`。

讓我們再看看另一個標準函式庫內非常常見且實用的列舉：`Option`。

Option 列舉相對於空值的優勢

在此段落我們將來研究 `Option`，這是在標準函式庫中定義的另一種列舉。`Option` 廣泛運用在許多場合，它能表示一個數值可能有某個東西，或者什麼都沒有。

舉例來說，如果你向一串包含元素的列表索取第一個值，你會拿到數值，但如果你向空列表索取的話，你就什麼都拿不到。在型別系統中表達這樣的概念可以讓編譯器檢查我們是否都處理完我們該處理的情況了。這樣的功能可以防止其他程式語言中極度常見的程式錯誤。

程式語言設計通常要考慮哪些功能是你要的，但同時哪些功能是你不要的也很重要。Rust 沒有像其他許多語言都有空值。**空值**（Null）代表的是沒有任何數值。在有空值的語言，所有變數都有兩種可能：空值或非空值。

而其發明者 Tony Hoare 在他 2009 的演講「空參考：造成數十億損失的錯誤」（“Null References: The Billion Dollar Mistake”）中提到：

我稱它為我十億美元級的錯誤。當時我正在為一門物件導向語言設計第一個全方位的參考型別系統。我當時的目標是透過編譯器自動檢查來確保所有的參考都是安全的。但我無法抗拒去加入空參考的誘惑，因為實作的方式實在太簡單了。這導致了無數的錯誤、漏洞與系統崩潰，在接下來的四十年中造成了大概數十億美金的痛苦與傷害。

空值的問題在於，如果你想在非空值使用空值的話，你會得到某種錯誤。由於空值與非空值的特性無所不在，你會很容易犯下這類型的錯誤。

但有時候能夠表達「空（null）」的概念還是很有用的：空值代表目前的數值因為某些原因而無效或缺少。

所以問題不在於概念本身，而在於如何實作。所以 Rust 並沒有空值，但是它有一個列舉可以表達出這樣的概念，也就是一個值可能是存在或不存在的。此列舉就是 `Option<T>`，它是在標準函式

庫中這樣定義的：

```
enum Option<T> {  
    None,  
    Some(T),  
}
```

`Option<T>` 實在太實用了，所以它早已加進 `prelude` 中，你不需要特地匯入作用域中。它的變體一樣也被加進 `prelude` 中，你可以直接使用 `Some` 和 `None` 而不必加上 `Option::` 的前綴。

`Option<T>` 仍然就只是個列舉，`Some(T)` 與 `None` 仍然是 `Option<T>` 型別的變體。

`<T>` 語法是我們還沒介紹到的 Rust 功能。它是個泛型型別參數，我們會在第十章正式介紹泛型 (generics)。現在你只需要知道 `<T>` 指的是 `Option` 列舉中的 `Some` 變體可以是任意型別。而透過 `Option` 數值來持有數字型別和字串型別的話，它們最終會換掉 `Option<T>` 中的 `T`，成為不同的型別。以下是使用 `Option` 來包含數字與字串型別的範例：

```
let some_number = Some(5);  
let some_char = Some('e');  
  
let absent_number: Option<i32> = None;
```

`some_number` 的型別是 `Option<i32>`，而 `some_char` 的型別是 `Option<char>`，兩者是不同的型別。Rust 可以推導出這些型別，因為我們已經在 `Some` 變體指定數值。至於 `absent_number` 的話，Rust 需要我們寫出完整的 `Option` 型別，因為編譯器無法從 `None` 推導出相對應的 `Some` 變體會持有哪種型別。我們在這裡告訴 Rust 我們 `absent_number` 所指的型別為 `Option<i32>`。

當我們有 `Some` 值時，我們會知道數值是存在的而且就位於 `Some` 內。當我們有 `None` 值時，在某種意義上它代表該值是空的，我們沒有有效的數值。所以為何 `Option<T>` 會比用空值來得好呢？

簡單來說因為 `Option<T>` 與 `T` (`T` 可以是任意型別) 是不同的型別，編譯器不會允許我們像一般有效的值那樣來使用 `Option<T>`。舉例來說，以下範例是無法編譯的，因為這是將 `i8` 與 `Option<i8>` 相加：

```
let x: i8 = 5;  
let y: Option<i8> = Some(5);  
  
let sum = x + y;
```



如果我們執行此程式，我們會得到以下錯誤訊息：

```

$ cargo run
   Compiling enums v0.1.0 (file:///projects/enums)
error[E0277]: cannot add `Option<i8>` to `i8`
  --> src/main.rs:5:17
   |
5  |         let sum = x + y;
   |                     ^ no implementation for `i8 + Option<i8>`
   |
= help: the trait `std::ops::Add<Option<i8>>` is not implemented for `i8`
= help: the following other types implement trait `Add<Rhs>`:
      <&'a f32 as Add<f32>>
      <&'a f64 as Add<f64>>
      <&'a i128 as Add<i128>>
      <&'a i16 as Add<i16>>
      <&'a i32 as Add<i32>>
      <&'a i64 as Add<i64>>
      <&'a i8 as Add<i8>>
      <&'a isize as Add<isize>>
      and 48 others

```

For more information about this error, try `rustc --explain E0277`.
error: could not compile `enums` due to previous error

這樣其實很好！此錯誤訊息事實上指的是 Rust 不知道如何將 `i8` 與 `Option<i8>` 相加，因為它們是不同的型別。當我們在 Rust 中有個型別像是 `i8`，編譯器將會確保我們永遠會擁有有效數值。我們可以很放心地使用該值，而不必檢查是不是空的。我們只有在使用 `Option<i8>`（或者任何其他要使用的型別）時才需要去擔心會不會沒有值。然後編譯器會確保我們在使用該值前，有處理過該有的條件。

換句話說，你必須將 `Option<T>` 轉換為 `T` 你才能對 `T` 做運算。這通常就能幫助我們抓到空值最常見的問題：認為某值不為空，但它其實就是空值。

消除掉非空值是否正確的風險，可以讓你對你寫的程式碼更有信心。要讓一個值變成可能為空的話，你必須顯式建立成對應型別的 `Option<T>`。然後當你要使用該值時，你就得顯式處理數值是否為空的條件。只要一個數值的型別不是 `Option<T>`，你就可以安全地認定該值不為空。這是 Rust 刻意考慮的設計決定，限制無所不在的空值，並增強 Rust 程式碼的安全性。

所以當我們有一個數值型別 `Option<T>`，我們要怎麼從 `Some` 變體取出 `T`，好讓我們可以使用該值呢？`Option<T>` 列舉有大量實用的方法可以在不同的場合下使用。你可以在[它的技術文件](#)查閱。更加熟悉 `Option<T>` 的方法十分益於你接下來的 Rust 旅程。

整體來說，要使用 `Option<T>` 數值的話，你要讓程式碼可以處理每個變體。你會希望有一些程式碼只會在當我們有 `Some(T)` 時執行，然後這些程式碼允許使用內部的 `T`。你會希望有另一部分的程式碼能在只有 `None` 時執行，且這些程式碼不會拿到有效的 `T` 數值。`match` 表達式正是處理此列舉行為的控制流結構：它會針對不同的列舉變體執行不同的程式碼，而且程式碼可以使用配對到的數值資料。

match 控制流建構子

Rust 有個功能非常強大的控制流建構子叫做 `match`，你可以使用一系列模式來配對數值並依據配對到的模式來執行對應的程式。模式（Patterns）可以是字面數值、變數名稱、萬用字元（wildcards）和其他更多元件來組成。[第十八章](#)會涵蓋所有不同類型的模式，以及它們的用途。`match` 強大的地方在於模式表達的清楚程度以及編譯器會確保所有可能的情況都處理了。

你可以想像 `match` 表達式成一個硬幣分類機器：硬幣會滑到不同大小的軌道，然後每個硬幣會滑入第一個符合大小的軌道。同樣地，數值會依序遍歷 `match` 的每個模式，然後進入第一個「配對」到該數值的模式所在的程式碼區塊，並在執行過程中使用。

既然我們都提到硬幣了，就讓我們用它們來作為 `match` 的範例吧！我們可以寫一個接收未知美國硬幣的函式，以類似驗鈔機的方式，決定它是何種硬幣並以美分作為單位回傳其值。如範例 6-3 所示。

```
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter,
}

fn value_in_cents(coin: Coin) -> u8 {
    match coin {
        Coin::Penny => 1,
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter => 25,
    }
}
```

範例 6-3：列舉以及用列舉變體作為模式的 `match` 表達式

讓我們一一介紹 `value_in_cents` 函式中 `match` 的每個部分。首先我們使用 `match` 並加上一個表達式，在此例的話就是指 `coin`。這和 `if` 中條件表達式的用法很像。不過差別在於 `if` 中的條件必須是布林值，而在此它可以回傳任何型別。在此範例中 `coin` 的型別是我們在第一行定義的列舉 `Coin`。

接下來是 `match` 的分支，每個分支有兩個部分：一個模式以及對應的程式碼。這邊第一個分支的模式是 `Coin::Penny` 然後 `=>` 會將模式與要執行的程式碼分開來，而在此例的程式碼就只是個 `1`。每個分支之間由逗號區隔開來。

當 `match` 表達式執行時，他會將計算的數據結果依序與每個分支的模式做比較。如果有模式配對到該值的話，其對應的程式碼就會執行。如果該模式與數值不符的話，就繼續執行下一個分支，就

像硬幣分類機器。

每個分支對應的程式碼都是表達式，然後在配對到的分支中表達式的數值結果就會是整個 `match` 表達式的回傳值。

如果配對分支的程式碼很短的話，我們通常就不會用到大括號，像是範例 6-3 每個分支就只回傳一個數值。如果你想要在配對分支執行多行程式碼的話，你就必須用大括號，然後你可以在括號後選擇性加上逗號。舉例來說，以下程式會在每次配對到 `Coin::Penny` 時印出「幸運幣！」再回傳程式碼區塊最後的數值 `1`：

```
fn value_in_cents(coin: Coin) -> u8 {
    match coin {
        Coin::Penny => {
            println!("幸運幣!");
            1
        }
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter => 25,
    }
}
```

綁定數值的模式

另一項配對分支的實用功能是它們可以綁定配對模式中部分的數值，這讓我們可以取出列舉變體中的數值。

舉例來說，讓我們改變我們其中一個列舉變體成擁有資料。從 1999 年到 2008 年，美國在鑄造 25 美分硬幣時，其中一側會有 50 個州不同的設計。不過其他的硬幣就沒有這樣的設計，只有 25 美分會有特殊值而已。我們可以改變我們的 `enum` 中的 `Quarter` 變體成儲存 `UsState` 數值，如範例 6-4 所示。

```
#[derive(Debug)] // 這讓我們可以顯示每個州
enum UsState {
    Alabama,
    Alaska,
    // --省略--
}

enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}
```


範例 6-4：修改 `Coin` 列舉的 `Quarter` 變體來包含一個 `UsState` 數值

讓我們想像有一個朋友想要收集所有 50 州的 25 美分硬幣。當我們在排序零錢的同時，我們會在拿到 25 美分時喊出該硬幣對應的州，好讓我們的朋友知道，如果他沒有的話就可以納入收藏。

在此程式中的配對表達式中，我們在 `Coin::Quarter` 變體的配對模式中新增了一個變數 `state`。當 `Coin::Quarter` 配對符合時，變數 `state` 會綁定該 25 美分的數值，然後我們就可以在分支程式碼中使用 `state`，如以下所示：

```
fn value_in_cents(coin: Coin) -> u8 {
    match coin {
        Coin::Penny => 1,
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter(state) => {
            println!("此 25 美分所屬的州為 {:?}!", state);
            25
        }
    }
}
```

如果我們呼叫 `value_in_cents(Coin::Quarter(UsState::Alaska))` 的話，`coin` 就會是 `Coin::Quarter(UsState::Alaska)`。當我們比較每個配對分支時，我們會到 `Coin::Quarter(state)` 的分支才配對成功。此時 `state` 綁定的數值就會是 `UsState::Alaska`。我們就可以在 `println!` 表達式中使用該綁定的值，以此取得 `Coin` 列舉中 `Quarter` 變體內的值。

配對 `Option<T>`

在上一個段落，我們想要在使用 `Option<T>` 時取得 `Some` 內部的 `T` 值。如同列舉 `Coin`，我們一樣可以使用 `match` 來處理 `Option<T>`！相對於比較硬幣，我們要比較的是 `Option<T>` 的變體，不過 `match` 表達式運作的方式一模一樣。

假設我們要寫個接受 `Option<i32>` 的函式，而且如果內部有值的話就將其加上 1。如果內部沒有數值的話，該函式就回傳 `None` 且不再嘗試做任何動作。

拜 `match` 所賜，這樣的函式很容易寫出來，長得就像範例 6-5。

```
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        None => None,
        Some(i) => Some(i + 1),
    }
}

let five = Some(5);
let six = plus_one(five);
let none = plus_one(None);
```

範例 6-5：對 `Option<i32>` 使用 `match` 表達式的函式

讓我們來仔細分析 `plus_one` 第一次的執行結果。當我們呼叫 `plus_one(five)` 時，`plus_one` 本體中的變數 `x` 會擁有 `Some(5)`。我們接著就拿去和每個配對分支比較：

```
None => None,
```

`Some(5)` 並不符合 `None` 這樣的模式，所以我們繼續進行下一個分支：

```
Some(i) => Some(i + 1),
```

`Some(5)` 有符合 `Some(i)` 這樣的模式嗎？這是當然的囉！我們有相同的變體。`i` 會綁定 `Some` 中的值，所以 `i` 會取得 `5`。接下來配對分支中的程式碼就會執行，我們將 `1` 加入 `i` 並產生新的 `Some` 其內部的值就會是 `6`。

現在讓我們看看範例 6-5 第二次的 `plus_one` 呼叫，這次的 `x` 是 `None`。我們進入 `match` 然後比較第一個分支：

```
None => None,
```

配對成功！因為沒有任何數值可以相加，程式就停止並在 `=>` 之後馬上回傳 `None`。因為第一個分支就配對成功了，沒有其他的分支需要再做比較。

用 `match` 與列舉組合起來在很多地方都很實用。你將會在許多 Rust 程式碼看到這樣的模式，使用 `match` 配對列舉，綁定內部的資料，然後執行對應的程式碼。一開始使用的確會有點陌生，但當你熟悉以後，你會希望所有語言都能提供這樣的功能。這一直是使用者最愛的功能之一。

配對必須是徹底的

我們還有一個 `match` 的細節要討論：分支的模式必須涵蓋所有可能性。今天要是我們像這樣寫了一個有錯誤的 `plus_one` 函式版本，它會無法編譯：

```
fn plus_one(x: Option<i32>) -> Option<i32> {
    match x {
        Some(i) => Some(i + 1),
    }
}
```



我們沒有處理到 `None` 的情形，所以此程式碼會產生錯誤。幸運的是這是 Rust 能夠抓到的錯誤。如果我們嘗試編譯此程式的話，我們會得到以下錯誤：

```
$ cargo run
   Compiling enums v0.1.0 (file:///projects/enums)
error[E0004]: non-exhaustive patterns: `None` not covered
--> src/main.rs:3:15
   |
3 |         match x {
   |               ^ pattern `None` not covered
   |
note: `Option<i32>` defined here
   = note: the matched value is of type `Option<i32>`
help: ensure that all possible cases are being handled by adding a match arm
with a wildcard pattern or an explicit pattern as shown
   |
4 ~             Some(i) => Some(i + 1),
5 ~             None => todo!(),
   |

For more information about this error, try `rustc --explain E0004`.
error: could not compile `enums` due to previous error
```

Rust 發現我們沒有考慮到所有可能條件，而且還知道我們少了哪些模式！Rust 中的配對必須是徹底 (*exhaustive*) 的：我們必須列舉出所有可能的情形，程式碼才能夠被視為有效。尤其是在 `Option<T>` 的情況下，當 Rust 防止我們忘記處理 `None` 的情形時，它也使我們免於以為擁有一個有效實際上卻是空的值。因此要造成之前提過的十億美元級錯誤在這邊基本上是不可能的。

Catch-all 模式與 `_` 佔位符

使用列舉的話，我們可以針對特定數值作特別的動作，而對其他所有數值採取預設動作。想像一下我們正在做款骰子遊戲，如果你骰出 3 的話，你的角色就動不了，但是可以拿頂酷炫的帽子。如果你骰出 7 你的角色就損失那頂帽子。至於其他的數值，你的角色就按照那個數值在遊戲桌上移動步數。以下是用 `match` 實作出的邏輯，骰子的結果並非隨機數而是寫死的，且所有邏輯對應的函式本體都是空的，因為實際去實作並非本範例的重點：

```

let dice_roll = 9;
match dice_roll {
    3 => add_fancy_hat(),
    7 => remove_fancy_hat(),
    other => move_player(other),
}

fn add_fancy_hat() {}
fn remove_fancy_hat() {}
fn move_player(num_spaces: u8) {}

```

在前兩個分支中的模式分別為數值 `3` 和 `7`。至於最後一個涵蓋其他可能數值的分支，我們用變數 `other` 作為模式。在 `other` 分支執行的程式碼會將該變數傳入函式 `move_player` 中。

此程式碼就算我們沒有列完所有 `u8` 可能的數字也能編譯完成，因為最後的模式會配對所有尚未被列出來的數值。這樣的 catch-all 模式能滿足 `match` 必須要徹底的要求。注意到我們需要將 catch-all 分支放在最後面，因為模式是按照順序配對的。如果我們將 catch-all 放在其他分支前的話，這樣一來其他後面的分支就永遠配對不到了，所以要是我們在 catch-all 之後仍加上分支的話 Rust 會警告我們！

當我們想使用 catch-all 模式但不想使用其數值時，Rust 還有一種模式能讓我們使用：`_` 這是個特殊模式，用來配對任意數值且不綁定該數值。這告訴 Rust 我們不會用到該數值，所以 Rust 不會警告我們沒使用到變數。

讓我們來改變一下遊戲規則：如果你骰到除了 3 與 7 以外的話，你必須要重新擲骰。我們不需要用到 catch-all 的數值，所以我們可以修改我們的程式碼來使用 `_`，而不必繼續用變數 `other`：

```

let dice_roll = 9;
match dice_roll {
    3 => add_fancy_hat(),
    7 => remove_fancy_hat(),
    _ => reroll(),
}

fn add_fancy_hat() {}
fn remove_fancy_hat() {}
fn reroll() {}

```

此範例一樣也滿足徹底的要求，因為我們在最後的分支顯式地忽略其他所有數值，我們沒有遺漏任何值。

我們再改最後一次遊戲規則，改成如果你骰到除了 3 與 7 以外，不會有任何事發生的話，我們可以用單元數值（我們在[元組型別](#)段落提到的空元組）作為 `_` 的程式碼：

```
let dice_roll = 9;
match dice_roll {
    3 => add_fancy_hat(),
    7 => remove_fancy_hat(),
    _ => (),
}

fn add_fancy_hat() {}
fn remove_fancy_hat() {}
```

這裡我們顯式地告訴 Rust 我們不會使用任何其他沒被先前分支配對到的數值，而且我們也不想在此執行任何程式碼。

我們會在[第十八章](#)更進一步探討模式與配對，現在我們要先去看看 `if let` 語法。當 `match` 表達式變得太囉嗦時，這語法就會變得很有用。

透過 `if let` 簡化控制流

`if let` 語法讓你可以用 `if` 與 `let` 的組合來以比較不冗長的方式，來處理只在乎其中一種模式而忽略其餘的數值。現在考慮一支程式如範例 6-6 所示，我們在配對 `config_max` 中 `Option<u8>` 的值，但只想在數值為 `Some` 變體時執行程式。

```
let config_max = Some(3u8);
match config_max {
    Some(max) => println!("最大值被設為 {}", max),
    _ => (),
}
```

範例 6-6： `match` 只在數值為 `Some` 時執行程式

如果數值為 `Some`，我們就在分支中綁定 `max` 變數，印出 `Some` 變體內的數值。我們不想對 `None` 作任何事情。為了滿足 `match` 表達式，我們必須在只處理一種變體的分支後面，再加上 `_ => ()`。這樣就加了不少樣板程式碼。

不過我們可以使用 `if let` 以更精簡的方式寫出來，以下程式碼的行為就與範例 6-6 的 `match` 一樣：

```
let config_max = Some(3u8);
if let Some(max) = config_max {
    println!("最大值被設為 {}", max);
}
```

`if let` 接收一個模式與一個表達式，然後用等號區隔開來。它與 `match` 的運作方式相同，表達式的意義與 `match` 相同，然後前面的模式就是第一個分支。在此例中的模式就是 `Some(max)`，然後 `max` 會綁定 `Some` 內的數值。我們就和 `match` 分支中使用 `max` 一樣，在 `if let` 區塊的本體中使用 `max`。如果數值沒有配對到模式，`if let` 中的程式碼就不會執行。

使用 `if let` 可以少打些字、減少縮排以及不用寫多餘的樣板程式碼。不過你就少了 `match` 強制的徹底窮舉檢查。要何時選擇 `match` 還是 `if let` 得依據你在的場合是要做什麼事情，以及在精簡度與徹底檢查之間做取舍。

換句話說，你可以想像 `if let` 是 `match` 的語法糖 (syntax sugar)，它只會配對一種模式來執行程式碼並忽略其他數值。

我們也可以在 `if let` 之後加上 `else`，`else` 之後的程式碼區塊等同於 `match` 表達式中 `_` 情形的程式碼區塊。這樣一來的 `if let` 和 `else` 組合就等同於 `match` 了。回想一下範例 6-4 的 `Coin` 列舉定義，`Quarter` 變體擁有數值 `UsState`。如果我們希望統計所有不是 25 美分的硬幣的同時，也能繼續回報 25 美分所屬的州的話，我們可以用 `match` 像這樣寫：

```
let mut count = 0;
match coin {
    Coin::Quarter(state) => println!("此 25 美分所屬的州為 {:?}!", state),
    _ => count += 1,
}
```

或是我們也可以用 `if let` 和 `else` 表達式這樣寫：

```
let mut count = 0;
if let Coin::Quarter(state) = coin {
    println!("此 25 美分所屬的州為 {:?}!", state);
} else {
    count += 1;
}
```

如果你的程式碼邏輯遇到使用 `match` 表達會太囉唆的話，記得 `if let` 也在你的 Rust 工具箱中供你使用。

總結

我們現在涵蓋了如何使用列舉來建立一系列列舉數值的自訂型別。我們展示了標準函式庫的 `Option<T>` 型別如何用型別系統來預防錯誤。當列舉數值其內有資料時，你可以依照你想處理的情況數量，使用 `match` 或 `if let` 來取出並使用那些數值。

你的 Rust 程式碼現在能夠使用結構體與列舉來表達你所相關研究領域的概念了。在你的 API 建立自訂型別可以確保型別安全，編譯器會保證你的函式只會取得該函式預期的型別數值。

接下來為了提供組織完善且直觀的 API 供你的使用者使用，並只表達出使用者確切所需要的內容，我們需要瞭解 Rust 的模組。

透過套件、Crate 與模組管理成長中的專案

當你寫的程式規模更大時，組織你的程式碼就很重要。因為用你的腦袋要記住整個程式碼是幾乎不可能的。要是能組織相關功能的程式碼並將它們分成明確功能的話，你就能清楚地找到實作特定功能的程式碼，以及該在哪裏修改該功能的行為。

我們之前寫過的程式都只在一個檔案內的一個模組（module）中。隨著專案成長，我們應該要組織程式碼，拆成數個模組與數個檔案。一個套件（package）可以包含數個執行檔 crate 以及選擇性提供一個函式庫 crate。隨著套件增長，你可以取出不同的部分作為獨立的 crate，成為對外的依賴函式庫。此章節將會介紹這些所有概念。對於非常龐大的專案，需要一系列的關聯套件組合在一起的話，Cargo 有提供工作空間（workspaces），我們會在第十四章的「[Cargo 工作空間](#)」做介紹。

我們還會討論對實作細節進行封裝，讓你的程式碼在頂層更好使用。一旦你實作了某項功能，其他程式就可以用程式碼的公開介面呼叫該程式碼，而不必去知道它實作如何運作。你在寫程式碼時會去定義哪些部分是給其他程式碼公開使用的，以及哪些部分是私底下你可以任意修改的實作細節。這能再減少你的腦袋需要煩惱的細節數量。

還有一個概念需要再提一次，也就是作用域（scope）：程式碼需要能被定義在「作用域內」並要能夠指明此作用域。當讀取寫入或編譯程式碼時，程式設計師與編譯器需要知道特定地點的名稱，才能知道其內的變數、函式、結構體、列舉、常數或其他任何有意義的項目。你可以建立作用域，並改變其在作用域內與作用域外的名稱。你無法在同個作用域內擁有兩個相同名稱的項目。我們可以使用一些工具來解決名稱衝突的問題。

Rust 有一系列的功能能讓你管理你的程式碼組織，包含哪些細節能對外提供、哪些細節是私有的，以及程式中每個作用域的名稱為何。這些功能有時會統一稱作 *模組系統*（*module system*），其中包含：

- **套件（Package）**：讓你建構、測試並分享 crate 的 Cargo 功能
- **Crates**：產生函式庫或執行檔的模組集合
- **模組（Modules）與 use**：讓你控制組織、作用域與路徑的隱私權
- **路徑（Paths）**：對一個項目的命名方式，像是一個結構體、函式或模組

在本章節中，我們會涵蓋所有這些功能，討論它們如何互動，並解釋如何使用它們來管理作用域。在讀完後，你應該就會對模組系統有紮實的認知，並能夠對作用域的使用駕輕就熟！

套件與 Crates

首先我們要介紹的第一個模組系統部分為套件與 crates。

一個 *crate* 是 Rust 編譯器同個時間內視為程式碼的最小單位。就算你執行的是 `rustc` 而非 `cargo`，然後傳入單源源碼檔案（就像我們在第一章的「編寫並執行 Rust 程式」那樣），編譯器會將該檔案視為一個 *crate*。Crate 能包含模組，而模組可以在其他檔案中定義然後同時與 *crate* 一起編譯，我們會在接下來的段落看到。

一個 *crate* 可以有兩種形式：執行檔 *crate* 或函式庫 *crate*。**執行檔 (Binary) crate** 是種你能編譯成執行檔並執行的程式，像是命令列程式或伺服器。這種 *crate* 需要有一個函式 `main` 來定義執行檔執行時該做什麼事。目前我們建立的所有 *crate* 都是執行檔 *crate*。

函式庫 (Library) crate 則不會有 `main` 函式，而且它們也不會編譯成執行檔。這種 *crate* 定義的功能用來分享給多重專案使用。舉例來說，我們在第二章用到的 `rand` *crate* 就提供了產生隨機數值的功能。當大多數的 Rustacean 講到「*crate*」時，他們其實指的是函式庫 *crate*，所以他們講到「*crate*」時相當於就是在講其他程式語言概念中的「函式庫」。

crate 的源頭會是一個原始檔案，讓 Rust 的編譯器可以作為起始點並組織 *crate* 模組的地方（我們會在「[定義模組來控制作用域與隱私權](#)」的段落詳加解釋模組）。

套件 (package) 則是提供一系列功能的一或數個 *crate*。一個套件會包含一個 *Cargo.toml* 檔案來解釋如何建構那些 *crate*。Cargo 本身其實就是個套件，包含了你已經用來建構程式碼的命令列工具。Cargo 套件還包含執行檔 *crate* 需要依賴的函式庫 *crate*。其他專案可以依賴 Cargo 函式庫來使用與 Cargo 命令列工具用到的相同邏輯功能。

一個套件能依照你的喜好擁有數個執行檔 *crate*，但最多只能有一個函式庫 *crate*。而一個套件至少要有一個 *crate*，無論是函式庫或執行檔 *crate*。

讓我們看看當我們建立一個套件時發生了什麼事。首先我們先輸入 `cargo new` 命令：

```
$ cargo new my-project
    Created binary (application) `my-project` package
$ ls my-project
Cargo.toml
src
$ ls my-project/src
main.rs
```

在我們執行 `cargo new` 之後，我們使用 `ls` 來查看 Cargo 建立了什麼。在專案的目錄中會有個 *Cargo.toml* 檔案，這是套件的設定檔。然後還會有個 *src* 目錄底下包含了 *main.rs*。透過你的文字編輯器打開 *Cargo.toml*，你會發現沒有提到 *src/main.rs*。Cargo 遵循的常規是 *src/main.rs* 就是與套件同名的執行檔 *crate* 的 *crate* 源頭。同樣地，Cargo 也會知道如果套件目錄包含 *src/lib.rs* 的

話，則該套件就會包含與套件同名的函式庫 `crate`。Cargo 會將 `crate` 源頭檔案傳遞給 `rustc` 來建構函式庫或執行檔。

我們在此的套件只有包含 `src/main.rs` 代表它只有一個同名的執行檔 `crate` 叫做 `my-project`。如果套件包含 `src/main.rs` 與 `src/lib.rs` 的話，它就有兩個 `crate`：一個執行檔與一個函式庫，兩者都與套件同名。一個套件可以有多個執行檔 `crate`，只要將檔案放在 `src/bin` 目錄底下就好，每個檔案會被視為獨立的執行檔 `crate`。

定義模組來控制作用域與隱私權

在此段落，我們將討論模組以及其他模組系統的部分，像是**路徑**（paths）允許你來命名項目，而 `use` 關鍵字可以將路徑引入作用域，再來 `pub` 關鍵字可以讓指定的項目對外公開。我們還會討論到 `as` 關鍵字、外部套件以及全域（glob）運算子。

首先，讓我們先介紹一些規則好讓你在之後組織程式碼時能更容易理解初步概念。然後我們會再詳細解釋每個規則。

模組懶人包

這裡我們先快速帶過模組、路徑、`use` 關鍵字以及 `pub` 關鍵字在編譯器中是怎麼運作的，以及多數開發者會怎麼組織他們的程式碼。我們會在此章節透過範例逐一介紹，不過這裡能讓你快速理解模組是怎麼運作的。

- **從 `crate` 源頭開始**：在編譯 `crate` 時，編譯器會先尋找 `crate` 源頭檔案（函式庫 `crate` 的話，通常就是 `src/lib.rs`；執行檔 `crate` 的話，通常就是 `src/main.rs`）來編譯程式碼。
- **宣告模組**：在 `crate` 源頭檔案中，你可以宣告新的模組，比如說你宣告了一個「garden」模組 `mod garden`；。編譯器會在以下這幾處尋找模組的程式碼：
 - 同檔案內用 `mod garden` 加上大括號，寫在括號內的程式碼
 - `src/garden.rs` 檔案中
 - `src/garden/mod.rs` 檔案中
- **宣告子模組**：除了 `crate` 源頭以外，其他檔案也可以宣告子模組。舉例來說，你可能會在 `src/garden.rs` 中宣告個 `mod vegetables`；。編譯器會與當前模組同名的目錄底下這幾處尋找子模組的程式碼：
 - 同檔案內，直接用 `mod vegetables` 加上大括號，寫在括號內的程式碼
 - `src/garden/vegetables.rs` 檔案中
 - `src/garden/vegetables/mod.rs` 檔案中
- **模組的路徑**：一旦有個模組成為 `crate` 的一部分，只要隱私權規則允許，你可以在 `crate` 裡任何地方使用該模組的程式碼。舉例來說，「garden」模組底下的「vegetables」模組的 `Asparagus` 型別可以用 `crate::garden::vegetables::Asparagus` 來找到。
- **私有 vs 公開**：模組內的程式碼從上層模組來看預設是私有的。要公開的話，將它宣告為 `pub mod` 而非只是 `mod`。要讓公開模組內的項目也公開的話，在這些項目前面也加上 `pub` 即可。
- **`use` 關鍵字**：在一個作用域內，`use` 關鍵字可以建立項目的捷徑，來縮短冗長的路徑名稱。在任何能使用 `crate::garden::vegetables::Asparagus` 的作用域中，你可以透過 `use crate::garden::vegetables::Asparagus`；來建立捷徑，接著你只需要寫 `Asparagus` 就能在作用域內使用該型別了。

這裡我們建立個執行檔 `crate` 叫做 `backyard` 來展示這些規則。`Crate` 的目錄也叫做 `backyard`，其中包含了這些檔案與目錄：

```
backyard
├── Cargo.lock
├── Cargo.toml
└── src
    ├── garden
    │   └── vegetables.rs
    ├── garden.rs
    └── main.rs
```

此例的 crate 源頭檔案就是 *src/main.rs*，它包含了：

檔案名稱：*src/main.rs*

```
use crate::garden::vegetables::Asparagus;

pub mod garden;

fn main() {
    let plant = Asparagus {};
    println!("I'm growing {:?}!", plant);
}
```

`pub mod garden;` 這行告訴編譯器要包含在 *src/garden.rs* 中的程式碼，也就是：

檔案名稱：*src/garden.rs*

```
pub mod vegetables;
```

這裡的 `pub mod vegetables;` 代表 *src/garden/vegetables.rs* 的程式碼也包含在內。而這段程式碼就是：

```
#[derive(Debug)]
pub struct Asparagus {}
```

現在讓我們詳細介紹這些規則並解釋如何運作的吧！

組織相關程式碼成模組

模組（Modules）能讓我們在 crate 內組織程式碼成數個群組以便使用且增加閱讀性。模組也能控制項目的隱私權，因為模組內的程式碼預設是私有的。私有項目是內部的實作細節，並不打算讓外部能使用。我們能讓模組與其內的項目公開，讓外部程式碼能夠使用並依賴它們。

舉例來說，讓我們建立一個提供餐廳功能的函式庫 crate。我們定義一個函式簽名不過本體會是空的，好讓我們專注在程式組織，而非餐廳程式碼的實作。

在餐飲業中，餐廳有些地方會被稱作前台（front of house）而其他部分則是後台（back of house）。前台是消費者的所在區域，這裡是安排顧客座位、點餐並結帳、吧台調酒的地方。而後台則是主廚與廚師工作的廚房、洗碗工洗碗以及經理管理行政工作的地方。

要讓 crate 架構長這樣的話，我們可以組織函式進入模組中。要建立一個新的函式庫叫做 restaurant 的話，請執行 `cargo new --lib restaurant`。然後將範例 7-1 的程式碼放入 `src/lib.rs` 中，這定義了一些模組與函式簽名。以下是前台的段落：

檔案名稱：`src/lib.rs`

```
mod front_of_house {
    mod hosting {
        fn add_to_waitlist() {}

        fn seat_at_table() {}
    }

    mod serving {
        fn take_order() {}

        fn serve_order() {}

        fn take_payment() {}
    }
}
```

範例 7-1：`front_of_house` 模組包含了其他擁有函式的模組

我們用 `mod` 關鍵字加上模組的名稱（在此例為 `front_of_house`）來定義一個模組，並用大括號涵蓋模組的本體。在模組中，我們可以再包含其他模組，在此例中我們包含了 `hosting` 和 `serving`。模組還能包含其他項目，像是結構體、列舉、常數、特徵、以及像是範例 7-1 的函式。

使用模組的話，我們就能將相關的定義組合起來，並用名稱指出會與它們互相關聯。程式設計師在使用此程式碼時只要觀察依據組合起來的模組名稱就好，不必遍歷所有的定義。這樣就能快速找到他們想使用的定義。要對此程式碼增加新功能的開發者也能知道該將程式碼放在哪裡，以維持程式碼的組織。

稍早我們提到說 `src/main.rs` 和 `src/lib.rs` 屬於 crate 的源頭。之所以這樣命名的原因是因為這兩個文件的內容都會在 crate 源頭模組架構中組成一個模組叫做 `crate`，這樣的結構稱之為**模組樹（module tree）**。

範例 7-2 顯示了範例 7-1 的模組樹架構。

```
crate
├── front_of_house
│   ├── hosting
│   │   ├── add_to_waitlist
│   │   └── seat_at_table
│   └── serving
│       ├── take_order
│       ├── serve_order
│       └── take_payment
```

範例 7-2：範例 7-1 的模組樹

此樹顯示了有些模組是包含在其他模組內的，比方說 `hosting` 就在 `front_of_house` 底下。此樹也顯示了有些模組是其他模組的**同輩 (siblings)**，代表它們是在同模組底下定義的，`hosting` 和 `serving` 都在 `front_of_house` 底下定義。如果模組 A 被包含在模組 B 中，我們會說模組 A 是模組 B 的**下一代 (child)**，而模組 B 是模組 A 的**上一代 (parent)**。注意到整個模組樹的根是一個隱性模組叫做 `crate`。

模組樹可能會讓你想到電腦中檔案系統的目錄樹，這是一個非常恰當的比喻！就像檔案系統中的目錄，你使用模組來組織你的程式碼。而且就像目錄中的檔案，我們需要有方法可以找到我們的模組。

參考模組項目的路徑

要展示 Rust 如何從模組樹中找到一個項目，我們需要使用和查閱檔案系統時一樣的路徑方法。要呼叫函式的話，我們需要知道它的路徑：

路徑可以有兩種形式：

- **絕對路徑** (absolute path) 是從 crate 的源頭起始的完整路徑。如果是外部 crate 的話，絕對路徑起始於該 crate 的名稱；如果是當前 crate 的話，則是 `crate` 作為起頭。
- **相對路徑** (relative path) 則是從本身的模組開始，使用 `self`、`super` 或是當前模組的標識符 (identifiers)。

無論是絕對或相對路徑其後都會接著一或多個標識符，並使用雙冒號 (`::`) 區隔開來。

回頭看看範例 7-1，假設我們想呼叫函式 `add_to_waitlist`。這就像在問函式 `add_to_waitlist` 的路徑在哪？範例 7-3 移除了一些範例 7-1 的模組與函式來精簡程式碼的呈現方式。

我們會展示兩種從 crate 源頭定義的 `eat_at_restaurant` 函式內呼叫 `add_to_waitlist` 的方法。這些路徑是正確的，不過目前還有其他問題會導致此範例無法編譯，我們會在稍後說明。

`eat_at_restaurant` 函式是我們函式庫 crate 公開 API 的一部分，所以我們會加上 `pub` 關鍵字。在「[使用 pub 關鍵字公開路徑](#)」的段落中，我們會提到更多 `pub` 的細節。

檔案名稱：`src/lib.rs`

```
mod front_of_house {
    mod hosting {
        fn add_to_waitlist() {}
    }
}

pub fn eat_at_restaurant() {
    // 絕對路徑
    crate::front_of_house::hosting::add_to_waitlist();

    // 相對路徑
    front_of_house::hosting::add_to_waitlist();
}
```



範例 7-3：使用絕對與相對路徑呼叫 `add_to_waitlist` 函式

我們在 `eat_at_restaurant` 中第一次呼叫 `add_to_waitlist` 函式的方式是用絕對路徑。

`add_to_waitlist` 函式和 `eat_at_restaurant` 都是在同一個 crate 底下，所以我們可以使用 `crate` 關鍵字來作為絕對路徑的開頭。我們接續加上對應的模組直到抵達 `add_to_waitlist`。你可以想像一個有相同架構的檔案系統，然後我們指定

`/front_of_house/hosting/add_to_waitlist` 這樣的路徑來執行 `add_to_waitlist` 程式。使用 `crate` 這樣的名稱作為 `crate` 源頭的開始，就像在你的 shell 使用 `/` 作為檔案系統的根一樣。

而我們第二次在 `eat_at_restaurant` 呼叫 `add_to_waitlist` 的方式是使用相對路徑。路徑的起頭是 `front_of_house`，因為它和 `eat_at_restaurant` 都被定義在模組樹的同一層中。這裡相對應的檔案系統路徑就是 `front_of_house/hosting/add_to_waitlist`。使用一個模組名稱作為開頭通常就是代表相對路徑。

何時該用相對或絕對路徑是你在你的專案中要做的選擇，依照你想將程式碼的定義連帶與使用它們的程式碼一起移動，或是分開移動到不同地方。舉例來說，如果我們同時將 `front_of_house` 模組和 `eat_at_restaurant` 函式移入另一個模組叫做 `customer_experience` 的話，就會需要修改 `add_to_waitlist` 的絕對路徑，但是相對路徑就可以原封不動。而如果我們只單獨將 `eat_at_restaurant` 函式移入一個叫做 `dining` 模組的話，`add_to_waitlist` 的絕對路徑就不用修改，但相對路徑就需要更新。我們通常會傾向於指定絕對路徑，因為分別移動程式碼定義與項目呼叫的位置通常是比較常見的。

讓我們嘗試編譯範例 7-3 並看看為何不能編譯吧！以下範例 7-4 是我們得到的錯誤資訊。

```
$ cargo build
   Compiling restaurant v0.1.0 (file:///projects/restaurant)
error[E0603]: module `hosting` is private
  --> src/lib.rs:9:28
   |
 9 |     crate::front_of_house::hosting::add_to_waitlist();
   |                                ^^^^^^^ private module
   |
note: the module `hosting` is defined here
  --> src/lib.rs:2:5
   |
 2 |     mod hosting {
   |       ^^^^^^^

error[E0603]: module `hosting` is private
  --> src/lib.rs:12:21
   |
12 |     front_of_house::hosting::add_to_waitlist();
   |                   ^^^^^^^ private module
   |
note: the module `hosting` is defined here
  --> src/lib.rs:2:5
   |
 2 |     mod hosting {
   |       ^^^^^^^

For more information about this error, try `rustc --explain E0603`.
error: could not compile `restaurant` due to 2 previous errors
```

範例 7-4：範例 7-3 嘗試編譯程式碼出現的錯誤

錯誤訊息表示 `hosting` 模組是私有的。換句話說，我們指定 `hosting` 模組與 `add_to_waitlist` 函式的路徑是正確的，但是因為它沒有私有部分的存取權，所以 Rust 不讓我們使用。在 Rust 中所有項目（函式、方法、結構體、列舉、模組與常數）的隱私權都是私有的。如果你想要建立私有的函式或結構體，你可以將它們放入模組內。

上層模組的項目無法使用下層模組的私有項目，但下層模組能使用它們上方所有模組的項目。這麼做的原因是因為下層模組用來實現實作細節，而下層模組應該要能夠看到在自己所定義的地方的其他內容。讓我們繼續用餐廳做比喻的話，我們可以想像隱私權規則就像是餐廳的後台辦公室。對餐廳顧客來說裡面發生什麼事情都是未知的，但是辦公室經理可以知道經營餐廳時的所有事物。

Rust 選擇這樣的模組系統，讓內部實作細節預設都是隱藏起來的。這樣一來，你就能知道內部哪些程式碼需要修改，而不會破壞到外部的程式碼。不過 Rust 有提供 `pub` 關鍵字能讓項目公開，讓你可以將下層模組內部的一些程式碼公開給上層模組來使用。

使用 `pub` 關鍵字公開路徑

讓我們再執行一次範例 7-4 的錯誤，它告訴我們 `hosting` 模組是私有的。我們希望上層模組中的 `eat_at_restaurant` 函式可以呼叫下層模組的 `add_to_waitlist` 函式，所以我們將 `hosting` 模組加上 `pub` 關鍵字，如範例 7-5 所示。

檔案名稱：src/lib.rs

```
mod front_of_house {
    pub mod hosting {
        fn add_to_waitlist() {}
    }
}

pub fn eat_at_restaurant() {
    // 絕對路徑
    crate::front_of_house::hosting::add_to_waitlist();

    // 相對路徑
    front_of_house::hosting::add_to_waitlist();
}
```



範例 7-5：宣告 `hosting` 模組為 `pub` 好讓 `eat_at_restaurant` 可以使用

不幸的是範例 7-5 的程式碼仍然回傳了另一個錯誤，如範例 7-6 所示。

```

$ cargo build
   Compiling restaurant v0.1.0 (file:///projects/restaurant)
error[E0603]: function `add_to_waitlist` is private
  --> src/lib.rs:9:37
   |
9  |         crate::front_of_house::hosting::add_to_waitlist();
   |                                         ^^^^^^^^^^^^^^^^^^^^^ private function
   |
note: the function `add_to_waitlist` is defined here
  --> src/lib.rs:3:9
   |
3  |         fn add_to_waitlist() {}
   |         ^^^^^^^^^^^^^^^^^^^^^
   |

error[E0603]: function `add_to_waitlist` is private
  --> src/lib.rs:12:30
   |
12 |         front_of_house::hosting::add_to_waitlist();
   |                                ^^^^^^^^^^^^^^^^^^^^^ private function
   |
note: the function `add_to_waitlist` is defined here
  --> src/lib.rs:3:9
   |
3  |         fn add_to_waitlist() {}
   |         ^^^^^^^^^^^^^^^^^^^^^
   |

For more information about this error, try `rustc --explain E0603`.
error: could not compile `restaurant` due to 2 previous errors

```

範例 7-6：編譯範例 7-5 時產生的錯誤

到底發生了什麼事？在 `mod hosting` 之前加上 `pub` 關鍵字確實公開了模組。有了這項修改後，我們的確可以在取得 `front_of_house` 後，繼續進入 `hosting`。但是 `hosting` 的所有內容仍然是私有的。模組中的 `pub` 關鍵字只會讓該模組公開讓上層模組使用而已，而不是存取它所有的內部程式碼。因為模組相當於一個容器，如果我們只公開模組的話，本身並不能做多少事情。我們需要再進一步選擇公開模組內一些項目才行。

範例 7-6 的錯誤訊息表示 `add_to_waitlist` 函式是私有的。隱私權規則如同模組一樣適用於結構體、列舉、函式與方法。

讓我們在 `add_to_waitlist` 的函式定義加上 `pub` 公開它吧，如範例 7-7 所示。

檔案名稱：src/lib.rs

```

mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}
    }
}

pub fn eat_at_restaurant() {
    // 絕對路徑
    crate::front_of_house::hosting::add_to_waitlist();

    // 相對路徑
    front_of_house::hosting::add_to_waitlist();
}

```

範例 7-7：將 `mod hosting` 和 `fn add_to_waitlist` 都加上 `pub` 關鍵字，讓我們可以從 `eat_at_restaurant` 呼叫函式

現在程式碼就能成功編譯了！要理解為何加上 `pub` 關鍵字讓我們可以在 `add_to_waitlist` 取得這些路徑，同時遵守隱私權規則，讓我們來看看絕對路徑與相對路徑。

在絕對路徑中，我們始於 `crate`，這是 `crate` 模組樹的根。再來 `front_of_house` 模組被定義在 `crate` 源頭中，`front_of_house` 模組不是公開，但因為 `eat_at_restaurant` 函式被定義在與 `front_of_house` 同一層模組中（也就是 `eat_at_restaurant` 與 `front_of_house` 同輩 (siblings)），我們可以從 `eat_at_restaurant` 參考 `front_of_house`。接下來是有 `pub` 標記的 `hosting` 模組，我們可以取得 `hosting` 的上層模組，所以我們可以取得 `hosting`。最後 `add_to_waitlist` 函式也有 `pub` 標記而我們可以取得它的上層模組，所以整個程式呼叫就能執行了！

而在相對路徑中，基本邏輯與絕對路徑一樣，不過第一步有點不同。我們不是從 `crate` 源頭開始，路徑是從 `front_of_house` 開始。`front_of_house` 與 `eat_at_restaurant` 被定義在同一層模組中，所以從 `eat_at_restaurant` 開始定義的相對路徑是有效的。再來因為 `hosting` 與 `add_to_waitlist` 都有 `pub` 標記，其餘的路徑也都是可以進入的，所以此函式呼叫也是有效的！

如果你計畫分享你的函式庫 `crate` 來讓其他專案能使用你的程式碼，你的公開 API 就是你對 `crate` 使用者的合約，這會決定他們能如何使用你的程式碼。這需要考量管理你的公開 API，好讓其他人能輕鬆依賴你的 `crate`。這類的考量不在本書的範疇，如果你對於此議題有興趣的話，請查看 [Rust API Guidelines](#)。

執行檔與函式庫套件的最佳實踐

我們提到套件能同時包含 `src/main.rs` 作為執行檔 `crate` 源頭以及 `src/lib.rs` 作為函式庫 `crate` 源頭，兩者預設都是用套件的名稱。通常來說，一個函式庫與一個執行檔 `crate` 這樣的套件模式，在執行檔中只會留下必要的程式碼，其餘則呼叫函式庫的程式碼。這樣其他專案也能運用到套件提供的多數功能，因為函式庫 `crate` 的程式碼可以分享。

模組要定義在 `src/lib.rs`。然後在執行檔 `crate` 中，任何公開項目都能用套件名稱作為開頭找到。執行檔 `crate` 應視為函式庫 `crate` 的使用者，就像外部 `crate` 那樣使用一樣，只能使用公開 API。這有助於你設計出良好的 API，你不僅是作者，同時還是自己的客戶！

在第十二章中，我們會透過寫個命令列程式來介紹這樣的組織練習，該程式會包含一個執行檔 `crate` 與一個函式庫 `crate`。

使用 `super` 作為相對路徑的開頭

我們可以在路徑開頭使用 `super` 來建構從上層模組出發的相對路徑，而不用從 `crate` 源頭開始。這就像在檔案系統中使用 `..` 作為路徑開頭一樣。使用 `super` 讓我們能參考確定位於上層模組的項目。當模組與上層模組有高度關聯，且上層模組可能以後會被移到模組樹的其他地方時，這能讓組織模組樹更加輕鬆。

請考慮範例 7-8 的程式碼，這模擬了一個主廚修正一個錯誤的訂單，並親自提供給顧客的場景。定義在 `back_of_house` 模組的函式 `fix_incorrect_order` 呼叫了定義在上層模組的函式 `deliver_order`，不過這次是使用 `super` 來指定 `deliver_order` 的路徑：

檔案名稱：`src/lib.rs`

```
fn deliver_order() {}

mod back_of_house {
    fn fix_incorrect_order() {
        cook_order();
        super::deliver_order();
    }

    fn cook_order() {}
}
```

範例 7-8：使用 `super` 作為呼叫函式路徑的開頭

`fix_incorrect_order` 函式在 `back_of_house` 模組中，所以我們可以使用 `super` 前往 `back_of_house` 的上層模組，在此例的話就是源頭 `crate`。然後在此時我們就能找到 `deliver_order`。成功！我們認定 `back_of_house` 模組與 `deliver_order` 函式應該會維持這樣相同的關係，在我們要組織 `crate` 的模組樹時，它們理當一起被移動。因此我們使用 `super` 讓我們在未來程式碼被移動到不同模組時，我們不用更新太多程式路徑。

公開結構體與列舉

我們也可以使用 `pub` 來公開結構體與列舉，但是我們有些額外細節要考慮到。如果我們在結構體定義之前加上 `pub` 的話，我們的確能公開結構體，但是結構體內的欄位仍然會是私有的。我們可

以視情況決定每個欄位要不要公開。在範例 7-9 我們定義了一個公開的結構體

`back_of_house::Breakfast` 並公開欄位 `toast`，不過將欄位 `seasonal_fruit` 維持是私有的。這次範例模擬的情境是，餐廳顧客可以選擇早餐要點什麼類型的麵包，但是由主廚視庫存與當季食材來決定提供何種水果。餐廳提供的水果種類隨季節變化很快，所以顧客無法選擇或預先知道他們會拿到何種水果。

檔案名稱：`src/lib.rs`

```
mod back_of_house {
    pub struct Breakfast {
        pub toast: String,
        seasonal_fruit: String,
    }

    impl Breakfast {
        pub fn summer(toast: &str) -> Breakfast {
            Breakfast {
                toast: String::from(toast),
                seasonal_fruit: String::from("桃子"),
            }
        }
    }
}

pub fn eat_at_restaurant() {
    // 點夏季早餐並選擇黑麥麵包
    let mut meal = back_of_house::Breakfast::summer("黑麥");
    // 我們想改成全麥麵包
    meal.toast = String::from("全麥");
    println!("我想要{}麵包，謝謝", meal.toast);

    // 接下來這行取消註解的話，我們就無法編譯通過
    // 我們無法擅自更改餐點搭配的季節水果
    // meal.seasonal_fruit = String::from("藍莓");
}
```

範例 7-9：一個有些欄位公開而有些是私有欄位的結構體

因為 `back_of_house::Breakfast` 結構體中的 `toast` 欄位是公開的，在 `eat_at_restaurant` 中我們可以加上句點來對 `toast` 欄位進行讀寫。注意我們不能在 `eat_at_restaurant` 使用 `seasonal_fruit` 欄位，因為它是私有的。請嘗試解開修改 `seasonal_fruit` 欄位數值的那行程式註解，看看你會獲得什麼錯誤！

另外因為 `back_of_house::Breakfast` 擁有私有欄位，該結構體必須提供一個公開的關聯函式（associated function）才有辦法產生 `Breakfast` 的實例（我們在此例命名為 `summer`）。如果 `Breakfast` 沒有這樣的函式的話，我們就無法在 `eat_at_restaurant` 建立 `Breakfast` 的實例，因為我們無法在 `eat_at_restaurant` 設置私有欄位 `seasonal_fruit` 的數值。

接下來，如果我們公開列舉的話，那它所有的變體也都會公開。我們只需要在 `enum` 關鍵字之前加上 `pub` 就好，如範例 7-10 所示。

檔案名稱：src/lib.rs

```
mod back_of_house {
    pub enum Appetizer {
        Soup,
        Salad,
    }
}

pub fn eat_at_restaurant() {
    let order1 = back_of_house::Appetizer::Soup;
    let order2 = back_of_house::Appetizer::Salad;
}
```

範例 7-10：公開列舉會讓其所有變體也公開

因為我們公開了 `Appetizer` 列舉，我們可以在 `eat_at_restaurant` 使用 `Soup` 和 `Salad`。

列舉的變體沒有全部都公開的話，通常會讓列舉很不好用。要用 `pub` 標註所有的列舉變體都公開的話又很麻煩。所以公開列舉的話，預設就會公開其變體。相反地，結構體不讓它的欄位全部都公開的話，通常反而比較實用。因此結構體欄位的通用原則是預設為私有，除非有 `pub` 標註。

我們還有一個 `pub` 的使用情境還沒提到，也就是我們模組系統最後一項功能：`use` 關鍵字。我們接下來會先解釋 `use`，再來研究如何組合 `pub` 和 `use`。

透過 `use` 關鍵字引入路徑

要是每次都得寫出呼叫函式的路徑的話是很冗長、重複且不方便的。舉例來說範例 7-7 我們在考慮要使用絕對或相對路徑來呼叫 `add_to_waitlist` 函式時，每次想要呼叫 `add_to_waitlist` 我們都得指明 `front_of_house` 以及 `hosting`。幸運的是，我們有簡化過程的辦法：我們可以用 `use` 關鍵字建立路徑的捷徑，然後在作用域內透過更短的名稱來使用。

在範例 7-11 中，我們引入了 `crate::front_of_house::hosting` 模組進 `eat_at_restaurant` 函式的作用域中，所以我們要呼叫函式 `add_to_waitlist` 的話我們只需要指明 `hosting::add_to_waitlist`。

檔案名稱：src/lib.rs

```
mod front_of_house {  
    pub mod hosting {  
        pub fn add_to_waitlist() {}  
    }  
}  
  
use crate::front_of_house::hosting;  
  
pub fn eat_at_restaurant() {  
    hosting::add_to_waitlist();  
}
```

範例 7-11：使用 `use` 將模組引入

使用 `use` 將路徑引入作用域就像是在檔案系統中產生符號連結一樣（symbolic link）。在 `crate` 源頭加上 `use crate::front_of_house::hosting` 後，`hosting` 在作用域內就是個有效的名稱了。使用 `use` 的路徑也會檢查隱私權，就像其他路徑一樣。

注意到 `use` 只會在它所位在的特定作用域內建立捷徑。範例 7-12 將 `eat_at_restaurant` 移入子模組 `customer`，這樣就會與 `use` 陳述式的作用域不同，所以其函式本體將無法編譯。

檔案名稱：src/lib.rs

```
mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}
    }
}

use crate::front_of_house::hosting;

mod customer {
    pub fn eat_at_restaurant() {
        hosting::add_to_waitlist();
    }
}
```



範例 7-12： `use` 陳述式只適用於所在的作用域

編譯器錯誤顯示了該捷徑無法用在 `customer` 模組內：

```
$ cargo build
   Compiling restaurant v0.1.0 (file:///projects/restaurant)
error[E0433]: failed to resolve: use of undeclared crate or module `hosting`
  --> src/lib.rs:11:9
   |
11 |         hosting::add_to_waitlist();
   |         ^^^^^^^^ use of undeclared crate or module `hosting`

warning: unused import: `crate::front_of_house::hosting`
  --> src/lib.rs:7:5
   |
 7 | use crate::front_of_house::hosting;
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   = note: `#[warn(unused_imports)]` on by default

For more information about this error, try `rustc --explain E0433`.
warning: `restaurant` (lib) generated 1 warning
error: could not compile `restaurant` due to previous error; 1 warning emitted
```

你會發現還有另外一個警告說明 `use` 在它的作用域中並沒有被用到！要解決此問題的話，我們可以將 `use` 也移動到 `customer` 模組內，或是在 `customer` 子模組透過 `super::hosting` 參考上層模組的捷徑。

建立慣用的 `use` 路徑

在範例 7-11 你可能會好奇為何我們指明 `use crate::front_of_house::hosting` 然後在 `eat_at_restaurant` 呼叫，而不是直接用 `use` 指明 `add_to_waitlist` 函式的整個路徑就好。

像範例 7-13 這樣寫。

檔案名稱：src/lib.rs

```
mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}
    }
}

use crate::front_of_house::hosting::add_to_waitlist;

pub fn eat_at_restaurant() {
    add_to_waitlist();
}
```

範例 7-13：使用 `use` 將 `add_to_waitlist` 函式引入作用域，但這較不符合習慣

雖然範例 7-11 與範例 7-13 都能完成相同的任務，但是範例 7-11 的做法比較符合習慣用法。使用 `use` 將函式上層模組引入作用域，讓我們必須在呼叫函式時得指明對應模組。在呼叫函式時指定上層模組能清楚地知道該函式並非本地定義的，同時一樣能簡化路徑。範例 7-13 的程式碼會不清楚 `add_to_waitlist` 是在哪定義的。

另一方面，如果是要使用 `use` 引入結構體、列舉或其他項目的話，直接指明完整路徑反而是符合習慣的方式。範例 7-14 顯示了將標準函式庫的 `HashMap` 引入執行檔 `crate` 作用域的習慣用法。

檔案名稱：src/main.rs

```
use std::collections::HashMap;

fn main() {
    let mut map = HashMap::new();
    map.insert(1, 2);
}
```

範例 7-14：引入 `HashMap` 進作用域的習慣用法

此習慣沒什麼強硬的理由：就只是大家已經習慣這樣的用法來讀寫 Rust 的程式碼。

這樣的習慣有個例外，那就是如果我們將兩個相同名稱的項目使用 `use` 陳述式引入作用域時，因為 Rust 不會允許。範例 7-15 展示了如何引入兩個同名但屬於不同模組的 `Result` 型別進作用域中並使用的方法。

檔案名稱：src/lib.rs

```

use std::fmt;
use std::io;

fn function1() -> fmt::Result {
    // --省略--
}

fn function2() -> io::Result<()> {
    // --省略--
}

```

範例 7-15：要將兩個同名的型別引入相同作用域的話，必須使用它們所屬的模組

如同你所見使用對應的模組可以分辨出是在使用哪個 `Result` 型別。如果我們直接指明 `use std::fmt::Result` 和 `use std::io::Result` 的話，我們會在同一個作用域中擁有兩個 `Result` 型別，這樣一來 Rust 就無法知道我們想用的 `Result` 是哪一個。

使用 `as` 關鍵字提供新名稱

要在相同作用域中使用 `use` 引入兩個同名型別的話，還有另一個辦法。在路徑之後，我們可以用 `as` 指定一個該型別在本地的新名稱，或者說別名（alias）。範例 7-16 展示重寫了範例 7-15，將其中一個 `Result` 型別使用 `as` 重新命名。

檔案名稱：src/lib.rs

```

use std::fmt::Result;
use std::io::Result as IoResult;

fn function1() -> Result {
    // --省略--
}

fn function2() -> IoResult<()> {
    // --省略--
}

```

範例 7-16：使用 `as` 將型別引入作用域的同時重新命名

在第二個 `use` 陳述式，我們選擇了將 `std::io::Result` 型別重新命名為 `IoResult`，這樣就不會和同樣引入作用域內 `std::fmt` 的 `Result` 有所衝突。範例 7-15 與 範例 7-16 都屬於習慣用法，你可以選擇你比較喜歡的方式！

使用 `pub use` 重新匯出名稱

當我們使用 `use` 關鍵字將名稱引入作用域時，該有效名稱在新的作用域中是私有的。要是我們希望呼叫我們這段程式碼時，也可以使用這個名稱的話（就像該名稱是在此作用域內定義的），我們可以組合 `pub` 和 `use`。這樣的技巧稱之為**重新匯出（re-exporting）**，因為我們將項目引入作用域，並同時公開給其他作用域參考。

範例 7-17 將範例 7-11 在源頭模組中原本的 `use` 改成 `pub use`。

檔案名稱：src/lib.rs

```
mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}
    }
}

pub use crate::front_of_house::hosting;

pub fn eat_at_restaurant() {
    hosting::add_to_waitlist();
}
```

範例 7-17：使用 `pub use` 使名稱公開給任何程式的作用域中參考

在此之前，外部程式碼會需要透過

`restaurant::front_of_house::hosting::add_to_waitlist()` 這樣的路徑才能呼叫 `add_to_waitlist`。現在 `pub use` 從源頭模組重新匯出了 `hosting` 模組，外部程式碼現在可以使用 `restaurant::hosting::add_to_waitlist()` 這樣的路徑就好。

當程式碼的內部結構與使用程式的開發者對於該領域所想像的結構不同時，重新匯出會很有用。我們再次用餐廳做比喻的話就像是，經營餐廳的人可能會想像餐廳是由「前台」與「後台」所組成，但光顧的顧客可能不會用這些術語來描繪餐廳的每個部分。使用 `pub use` 的話，我們可以用某種架構寫出程式碼，再以不同的架構對外公開。這樣讓我們的函式庫可以完整的組織起來，且對開發函式庫的開發者與使用函式庫的開發者都提供友善的架構。我們會在第十四章的「[透過 `pub use` 匯出理想的公開 API](#)」段落再看看另一個 `pub use` 的範例並了解它會如何影響 crate 的技術文件。

使用外部套件

在第二章我們寫了一支猜謎遊戲專案時，有用到一個外部套件叫做 `rand` 來取得隨機數字。要在專案內使用 `rand` 的話，我們會在 `Cargo.toml` 加上此行：

檔案名稱：Cargo.toml

```
rand = "0.8.5"
```

在 *Cargo.toml* 新增 `rand` 作為依賴函式庫會告訴 Cargo 要從 crates.io 下載 `rand` 以及其他相關的依賴，讓我們的專案可以使用 `rand`。

接下來要將 `rand` 的定義引入我們套件的作用域的話，我們加上一行 `use` 後面接著 crate 的名稱 `rand` 然後列出我們想要引入作用域的項目。回想一下在第二章「產生隨機數字」的段落，我們將 `Rng` 特徵引入作用域中，並呼叫函式 `rand::thread_rng`：

```
use rand::Rng;

fn main() {
    let secret_number = rand::thread_rng().gen_range(1..=100);
}
```

Rust 社群成員在 crates.io 發佈了不少套件可供使用，要將這些套件引入到你的套件的步驟是一樣的。在你的套件的 *Cargo.toml* 檔案列出它們，然後使用 `use` 將這些 crate 內的項目引入作用域中。

請注意到標準函式庫 `std` 對於我們的套件來說也是一個外部 crate。由於標準函式庫會跟著 Rust 語言發佈，所以我們不需要更改 *Cargo.toml* 來包含 `std`。但是我們仍然需使用 `use` 來將它的項目引入我們套件的作用域中。舉例來說，要使用 `HashMap` 我們可以這樣寫：

```
use std::collections::HashMap;
```

這是個用標準函式庫的 crate 名稱 `std` 起頭的絕對路徑。

使用巢狀路徑來清理大量的 `use` 行數

如果我們要使用在相同 crate 或是相同模組內定義的數個項目，針對每個項目都單獨寫一行的話，會佔據我們檔案內很多空間。舉例來說，範例 2-4 中的猜謎遊戲我們用了這兩個 `use` 陳述式來引入作用域中：

檔案名稱：src/main.rs

```
// --省略--
use std::cmp::Ordering;
use std::io;
// --省略--
```

我們可以改使用巢狀路徑（nested paths）來只用一行就能將數個項目引入作用域中。我們先指明相同路徑的部分，加上雙冒號，然後在大括號內列出各自不同的路徑部分，如範例 7-18 所示。

檔案名稱：src/main.rs

```
// --省略--  
use std::{cmp::Ordering, io};  
// --省略--
```

範例 7-18：使用巢狀路徑引入有部分相同前綴的數個路徑至作用域中

在較大的程式中，使用巢狀路徑將相同 crate 或相同模組中的許多項目引入作用域，可以大量減少 `use` 陳述式的數量！

我們可以在路徑中的任何部分使用巢狀路徑，這在組合兩個享有相同子路徑的 `use` 陳述式時非常有用。舉例來說，範例 7-19 顯示了兩個 `use` 陳述式：一個將 `std::io` 引入作用域，另一個將 `std::io::Write` 引入作用域。

檔案名稱：src/lib.rs

```
use std::io;  
use std::io::Write;
```

範例 7-19：兩個 `use` 陳述式且其中一個是另一個的子路徑

這兩個路徑的相同部分是 `std::io`，這也是整個第一個路徑。要將這兩個路徑合為一個 `use` 陳述式的話，我們可以在巢狀路徑使用 `self`，如範例 7-20 所示。

檔案名稱：src/lib.rs

```
use std::io::{self, Write};
```

範例 7-20：組合範例 7-19 的路徑為一個 `use` 陳述式

此行就會將 `std::io` 和 `std::io::Write` 引入作用域。

全域運算子

如果我們想要將在一個路徑中所定義的**所有**公開項目引入作用域的話，我們可以在指明路徑之後加上全域（glob）運算子 `*`：

```
use std::collections::*;
```

此 `use` 陳述式會將 `std::collections` 定義的所有公開項目都引入作用域中。不過請小心使用全域運算子！它容易讓我們無法分辨作用域內的名稱，以及程式中使用的名稱是從哪定義來的。

全域運算子很常用在 `tests` 模組下，將所有東西引入測試中。我們會在第十一章的「[如何寫測試](#)」段落來討論。全域運算子也常拿來用在 `prelude` 模式中，你可以查閱[標準函式庫的技術文件](#)來瞭解此模式的更多資訊。

將模組拆成不同檔案

本章節目前所有的範例將數個模組定義在同一個檔案中。當模組增長時，你可能會想要將它們的定義拆開到別的檔案中，好讓程式碼容易瀏覽。

舉例來說，讓我們從範例 7-17 餐廳的多重模組開始。我們會將模組拆成數個檔案，而不只是將所有模組都放在 `crate` 源頭檔案。在此例中，源頭檔案為 `src/lib.rs` 不過這步驟在執行檔 `crate` 的 `src/main.rs` 一樣可行。

首先，我們將 `front_of_house` 模組移到獨立的檔案中。刪掉 `front_of_house` 模組大括號內的程式碼，只留下宣告 `mod front_of_house;`，讓 `src/lib.rs` 包含的程式碼如範例 7-21 所示。請注意在我們加上範例 7-22 的 `src/front_of_house.rs` 檔案前這會仍無法編譯。

檔案名稱：`src/lib.rs`

```
mod front_of_house;

pub use crate::front_of_house::hosting;

pub fn eat_at_restaurant() {
    hosting::add_to_waitlist();
}
```



範例 7-21：宣告 `front_of_house` 模組，其本體位於 `src/front_of_house.rs`

接著，將原本大括號內的程式碼寫到新的檔案 `src/front_of_house.rs` 中，如範例 7-22 所示。編譯器知道要查看這個檔案，因為 `crate` 源頭有宣告這個模組的名稱 `front_of_house`。

檔案名稱：`src/front_of_house.rs`

```
pub mod hosting {
    pub fn add_to_waitlist() {}
}
```

範例 7-22：`front_of_house` 模組的定義位於 `src/front_of_house.rs`

你只需要在模組樹中使用 `mod` 宣告一次來讀取檔案就好。一旦編譯器知道該檔案屬於專案的一部分（且知道其位在模組樹中的何處，因為你有宣告 `mod` 陳述式），專案中的其他檔案就能用宣告的路徑讀取檔案的程式碼，如同「[參考模組項目的路徑](#)」段落提到的一樣。換句話說，`mod` 和你在其他程式語言可能會看到的「include」動作並不一樣。

要開始移動 `hosting` 的話，我們先改變 `src/front_of_house.rs`，讓它只包含 `hosting` 模組的宣告：

檔案名稱：`src/front_of_house.rs`

```
pub mod hosting;
```

然後我們建立一個目錄 `src/front_of_house` 以及一個檔案 `src/front_of_house/hosting.rs` 來包含 `hosting` 模組的定義：

檔案名稱：`src/front_of_house/hosting.rs`

```
pub fn add_to_waitlist() {}
```

如果我們將 `hosting.rs` 放在 `src` 目錄下，編譯器會將 `hosting.rs` 的程式碼視為是宣告在 `crate` 源頭底下的 `hosting` 模組。編譯器決定哪些檔案屬於哪些模組的規則讓目錄與檔案架構能更貼近模組樹的架構。

其他種的檔案路徑

目前我們涵蓋了 Rust 編譯器使用的最佳檔案路徑形式，但 Rust 仍然支援舊版的檔案路徑。當 `crate` 源頭宣告了一個模組 `front_of_house` 時，編譯器會在以下幾處尋找模組的程式碼：

- `src/front_of_house.rs`（我們介紹的）
- `src/front_of_house/mod.rs`（舊版風格，仍然支援的路徑形式）

當有個 `front_of_house` 的子模組 `hosting` 宣告時，編譯器會在以下幾處尋找模組的程式碼：

- `src/front_of_house/hosting.rs`（我們介紹的）
- `src/front_of_house/hosting/mod.rs`（舊版風格，仍然支援的路徑形式）

如果你對同個模組同時使用兩種風格的話，你會收到編譯器錯誤。在同個專案對不同模組使用不同風格則是允許的，但這有可能會讓瀏覽專案的人感到困惑。

使用 `mod.rs` 檔案名稱的風格最主要的缺點是你的專案可能最後會有很多檔案都叫做 `mod.rs`，當你在編輯器同時開啟這些檔案時可能會被混淆。

我們將模組的程式碼搬到了不同的檔案，而模組樹仍維持完好如初。就算函式定義被移動不同檔案，`eat_at_restaurant` 內的函式呼叫不用任何修改仍能維持運作。這樣的方式讓你可以隨著模組成長時，移動到新的檔案中。

另外 `src/lib.rs` 內的 `pub use crate::front_of_house::hosting` 陳述式沒有改變，在檔案作為 `crate` 的一部分來編譯時，使用 `use` 的方式也沒有改變。`mod` 關鍵字能宣告模組，然後 Rust 會去同名的檔案尋找該模組的程式碼。

總結

Rust 讓你能夠將套件拆成數個 crate，然後 crate 能再分成數個模組，好讓你可以從一個模組內指定其他模組的項目。而你可以使用絕對或相對路徑來達成。這些路徑可以用 `use` 陳述式來引入作用域，讓你可以在該作用域用更短的路徑來多次呼叫該項目。模組程式碼預設為私有的，但你可以使用 `pub` 關鍵字公開它的定義內容。

在下個章節，我們將探討在標準函式庫中的一些資料結構集合，讓你可以利用它們寫出整潔有組織的程式碼。

常見集合

Rust 的標準函式庫提供一些非常實用的資料結構稱之為**集合 (collections)**。多數其他資料型別只會呈現一個特定數值，但是集合可以包含數個數值。不像內建的陣列與元組型別，這些集合指向的資料位於堆積上，代表資料的數量不必在編譯期就知道，而且可以隨著程式執行增長或縮減。每種集合都有不同的能力以及消耗，依照你的情形選擇適當的集合，是一項你會隨著開發時間漸漸掌握的技能。在本章節我們會介紹三種在 Rust 程式中十分常用的集合：

- **向量 (Vector)** 允許你接二連三地儲存數量不定的數值。
- **字串 (String)** 是字元的集合。我們在之前就提過 `String` 型別，本章會正式深入介紹。
- **雜湊映射 (Hash map)** 允許你將值 (value) 與特定的鍵 (key) 相關聯。這是從一種更通用的資料結構**映射 (map)** 衍生出來的特定實作。

想瞭解更多標準函式庫提供的集合種類的話，歡迎查閱[技術文件](#)。

我們將討論如何建立與更新向量、字串與雜湊映射，以及它們的所長。

透過向量儲存列表

我們第一個要來看的集合是 `Vec<T>` 常稱為**向量** (vector)。向量允許你在一個資料結構儲存不止一個數值，而且該結構的記憶體會接連排列所有數值。它們很適合用來處理你手上的項目列表，像是一個檔案中每行的文字，或是購物車內每項物品。

建立新的向量

要建立一個新的空向量的話，我們呼叫 `Vec::new` 函式，如範例 8-1 所示。

```
let v: Vec<i32> = Vec::new();
```

範例 8-1 建立一個儲存數值型別為 `i32` 的空向量

注意到我們在此加了型別詮釋。因為我們沒有對此向量插入任何數值，Rust 不知道我們想儲存什麼類型的元素。這是一項重點，向量是用泛型 (generics) 實作，我們會在第十章說明如何為你自己的型別使用泛型。現在我們只需要知道標準函式庫提供的 `Vec<T>` 型別可以持有任意型別，然後當特定向量要持有特定型別時，我們可以在尖括號內指定該型別。在範例 8-1，我們告訴 Rust 在 `v` 中的 `Vec<T>` 會持有 `i32` 型別的元素。

不過通常你在建立 `Vec<T>` 時只需要給予初始數值，Rust 就能推導出你想儲存的數值型別，所以你不太常會需要指明型別詮釋。Rust 還提供了 `vec!` 巨集讓我們能方便地建立一個新的向量並取得你提供的數值。在範例 8-2 中，我們建立了一個新的 `Vec<i32>` 並擁有數值 `1`、`2` 和 `3`。整數型別為 `i32` 是因為這是預設整數型別，如同我們在第三章的「[資料型別](#)」段落提到的一樣。

```
let v = vec![1, 2, 3];
```

範例 8-2：建立一個擁有數值的新向量

因為我們給予了初始的 `i32` 數值，Rust 可以推導出 `v` 的型別為 `Vec<i32>`，所以型別詮釋就不是必要的了。接下來，讓我們看看如何修改向量。

更新向量

要在建立向量之後新增元素的話，我們可以使用 `push` 方法，如範例 8-3 所示。

```
let mut v = Vec::new();

v.push(5);
v.push(6);
v.push(7);
v.push(8);
```

範例 8-3：使用 `push` 方法來新增數值到向量

與其他變數一樣，如果我們想要變更其數值的話，我們需要使用 `mut` 關鍵字使它成為可變的，如同第三章提到的一樣。我們插入的數值所屬型別均為 `i32`，然後 Rust 可以從資料推導，所以我們不必指明 `Vec<i32>`。

讀取向量元素

要參考向量儲存的數值有兩種方式。為了更加清楚說明此範例，我們詮釋了函式回傳值的型別。

範例 8-4 顯示了取得向量中數值的方法，可以使用索引語法與 `get` 方法。

```
let v = vec![1, 2, 3, 4, 5];

let third: &i32 = &v[2];
println!("第三個元素是 {third}");

let third: Option<&i32> = v.get(2);
match third {
    Some(third) => println!("第三個元素是 {third}"),
    None => println!("第三個元素並不存在。"),
}
```

範例 8-4：使用索引語法或 `get` 方法來取得向量項目

這邊我們要注意一些地方。我們使用了索引數值 `2` 來獲取第三個元素：向量可以用數字來索引，從零開始計算。使用 `&` 和 `[]` 會給我們索引數值的元素參考，而使用 `get` 方法加上一個索引作為引數，則會給我們 `Option<&T>`，我們可以用 `match` 來配對。

Rust 提供兩種取得元素參考方式，所以當你嘗試使用索引數值取得向量範圍外的元素時，你可以決定程式的行為。讓我們看看一個範例，我們有一個向量擁有五個元素，但我們嘗試用索引 `100` 來取得對應數值，如範例 8-5 所示。

```
let v = vec![1, 2, 3, 4, 5];

let does_not_exist = &v[100];
let does_not_exist = v.get(100);
```



範例 8-5：嘗試對只有五個元素的向量取得索引 100 的值

當我們執行程式時，第一個 `[]` 方法會讓程式恐慌，因為它參考了不存在的元素。此方法適用於當你希望一有無效索引時就讓程式崩潰的狀況。

當你使用 `get` 方法來索取向量不存在的索引時，它會回傳 `None` 而不會恐慌。如果正常情況下偶而會不小心存取超出向量範圍索引的話，你就會想要只用此方法。你的程式碼就會有個邏輯專門處理 `Some(&element)` 或 `None`，如同第六章所述。舉例來說，可能會有由使用者輸入的索引。如果他不小心輸入太大的數字的話，程式可以回傳 `None`，你可以告訴使用者目前向量有多少項目，並讓他們可以再輸入一次。這會比直接讓程式崩潰還來的友善，他們可能只是不小心打錯而已！

當程式有個有效參考時，借用檢查器（borrow checker）會貫徹所有權以及借用規則（如第四章所述）來確保此參考及其他對向量內容的參考都是有效的。回想一下有個規則是我們不能在同個作用域同時擁有可變與不可變參考。這個規則一樣適用於範例 8-6，在此我們有一個向量第一個元素的不可變參考，然後我們嘗試在向量後方新增元素。如果我們嘗試在此動作後繼續使用第一個參考的話，程式會無法執行：

```
let mut v = vec![1, 2, 3, 4, 5];

let first = &v[0];

v.push(6);

println!("第一個元素是：{first}");
```



範例 8-6：在持有一個項目的參考時，還嘗試對向量新增元素

編譯此程式會得到以下錯誤：

```
$ cargo run
   Compiling collections v0.1.0 (file:///projects/collections)
error[E0502]: cannot borrow `v` as mutable because it is also borrowed as
immutable
--> src/main.rs:6:5
   |
4  |     let first = &v[0];
   |                  - immutable borrow occurs here
5  |
6  |     v.push(6);
   |     ^^^^^^^^^ mutable borrow occurs here
7  |
8  |     println!("第一個元素為 {first}");
   |                                     ----- immutable borrow later used here
   |
```

For more information about this error, try ``rustc --explain E0502``.
error: could not compile `collections` due to previous error

範例 8-6 的程式碼看起來好像能執行。為何第一個元素的參考要在意向量的最後端發生了什麼事呢？此錯誤其實跟向量運作的方式有關：由於向量會將元素放在前一位的記憶體位置後方，在向量後方新增元素時，如果當前向量的空間不夠再塞入另一個值的話，可能會需要配置新的記憶體並複製舊的元素到新的空間中。這樣一來，第一個元素的索引可能就會指向已經被釋放的記憶體，借用規則會防止程式遇到這樣的情形。

注意：關於 `Vec<T>` 型別更多的實作細節，歡迎查閱「[The Rustonomicon](#)」。

遍歷向量的元素

想要依序存取向量中每個元素的話，我們可以遍歷所有元素而不必用索引一個一個取得。範例 8-7 闡釋了如何使用 `for` 迴圈來取得一個 `i32` 向量中每個元素的不可變參考並印出他們。

```
let v = vec![100, 32, 57];
for i in &v {
    println!("{i}");
}
```

範例 8-7：使用 `for` 迴圈遍歷向量中每個元素

我們還可以遍歷可變向量中的每個元素取得可變參考來改變每個元素。像是範例 8-8 就使用 `for` 迴圈來為每個元素加上 50。

```
let mut v = vec![100, 32, 57];
for i in &mut v {
    *i += 50;
}
```

範例 8-8：遍歷向量中的元素取得可變參考

要改變可變參考指向的數值，在使用 `+=` 運算子之前，我們需要使用 `*` 解參考運算子來取得 `i` 的數值。我們會在第十五章的「[追蹤指標的數值](#)」段落來講解更多解參考運算子的細節。

當遍歷向量時，無論是不可變或可變地都是安全，因為借用檢查器的規則能確保如此。如果我們嘗試在範例 8-7 或範例 8-8 的 `for` 迴圈本體插入或刪除項目，我們就會獲得和範例 8-6 程式碼類似的編譯錯誤。`for` 迴圈持有的向量參考能避免同時修改整個向量。

使用列舉來儲存多種型別

向量只能儲存同型別的數值，這在某些情況會很不方便，一定會有場合是要儲存不同型別到一個列表中。幸運的是，列舉的變體是定義在相同的列舉型別，所以當我們需要在向量儲存不同型別的元素時，我們可以用列舉來定義！

舉例來說，假設我們想從表格中的一行取得數值，但是有些行內的列會包含整數、浮點數以及一些字串。我們可以定義一個列舉，其變體會持有不同的數值型別，然後所有的列舉變體都會被視為相同型別：就是它們的列舉。接著我們就可以建立一個擁有此列舉型別的向量，最終達成持有不同型別。如範例 8-9 所示。

```
enum SpreadsheetCell {  
    Int(i32),  
    Float(f64),  
    Text(String),  
}  
  
let row = vec![  
    SpreadsheetCell::Int(3),  
    SpreadsheetCell::Text(String::from("藍色")),  
    SpreadsheetCell::Float(10.12),  
];
```

範例 8-9：用 `enum` 定義儲存不同型別的列舉並作為向量的型別

Rust 需要在編譯時期知道向量的型別以及要在堆積上用到多少記憶體才能儲存每個元素。我們必須明確知道哪些型別可以放入向量中。如果 Rust 允許向量一次持有任意型別的話，在對向量中每個元素進行處理時，可能就會有一或多種型別會產生錯誤。使用列舉和 `match` 表達式讓 Rust 可以在編譯期間確保每個可能的情形都已經處理完善了，如同第六章提到的一樣。

如果你無法確切知道執行時程式所處理的所有型別的話，列舉就不管用了。這時使用特徵物件會比較好，我們會在第十七章再來解釋。

現在我們已經講了一些向量常見的用法，有時間的話記得到[向量的 API 技術文件](#)瞭解標準函式庫中 `Vec<T>` 所有實用的方法。舉例來說，除了 `push` 方法以外，還有個 `pop` 方法可以移除並回傳最後一個元素。

釋放向量的同時也會釋放其元素

就像其它 `struct` 一樣，向量會在作用域結束時被釋放，如範例 8-10 所示。

```
{  
    let v = vec![1, 2, 3, 4];  
  
    // 使用 v 做些事情  
} // <- v 在此離開作用域並釋放
```

範例 8-10：顯示向量及其元素在哪裡被釋放

當向量被釋放時，其所有內容也都會被釋放，代表它持有的那些整數都會被清除。這雖然聽起來很直觀，但是當我們開始參考向量中的元素時可能就會變得有點複雜。讓我們看看怎麼處理這種情形吧！

接下來讓我們看看下一個集合型別：`String`！

透過字串儲存 UTF-8 編碼的文字

我們已經在第四章提到字串（String），但現在我們要更加深入探討。Rustaceans 初學者常常會卡在三個環節：Rust 傾向於回報可能的錯誤、字串的資料結構比開發者所熟悉的還要複雜，以及 UTF-8。這些要素讓來自其他程式語言背景的開發者會遇到一些困難。

我們會在集合章節討論字串的原因是，字串本身就是位元組的集合，且位元組作為文字呈現時，它會提供一些實用的方法。在此段落我們將和其他集合型別一樣討論 `String` 的操作，像是建立、更新與讀取。我們還會討論到 `String` 與其他集合不一樣的地方，像是 `String` 的索引就比其他集合還複雜，因為它會依據人們對於 `String` 資料型別的理解而有所不同。

什麼是字串？

首先我們要好好定義字串（String）這個術語。**Rust** 在核心語言中只有一個字串型別，那就是字串切片 `str`，它通常是以借用的形式存在 `&str`。在第四章中我們提到字串切片是一個針對存在某處的 UTF-8 編碼資料的參考。舉例來說，字串字面值（String literals）就儲存在程式的執行檔中，因此就是字串切片。

`String` 型別是 Rust 標準函式庫所提供的型別，並不是核心語言內建的型別，它是可增長的、可變的、可擁有所有權的 UTF-8 編碼字串型別。當 Rustaceans 提及 Rust 中的「字串」時，他們通常指的是 `String` 以及字串切片 `&str` 型別，而不只是其中一種型別。雖然此段落大部分都在討論 `String`，這兩個型別都時常用在 Rust 的標準函式庫中，且 `String` 與字串切片都是 UTF-8 編碼的。

建立新的字串

許多 `Vec<T>` 可使用的方法在 `String` 也都能用，因為 `String` 其實就是一種位元組向量的封裝再加上一些額外的保障、限制與能力。其中一個 `Vec<T>` 與 `String` 都有且用途相同的函式就是 `new`，這用來產生新的實例，如範例 8-11 所示。

```
let mut s = String::new();
```

範例 8-11：建立新的空 `String`

此行會建立新的字串叫做 `s`，我們之後可以再寫入資料。不過通常我們會希望建立字串的時能夠初始化資料。為此我們可以使用 `to_string` 方法，任何有實作 `Display` 特徵的型別都可以使用此方法，就像字串字面值的使用方式一樣。範例 8-12 就展示了兩種例子。


```
let data = "初始內容";

let s = data.to_string();

// 此方法也能直接用於字面值上
let s = "初始內容".to_string();
```

範例 8-12：從字串字面值使用 `to_string` 方法來建立 `String`

此程式碼建立了一個字串內容為 `初始內容`。

我們也可以用函式 `String::from` 從字串字面值建立 `String`。範例 8-13 的程式碼和使用 `to_string` 的範例 8-12 效果一樣。

```
let s = String::from("初始內容");
```

範例 8-13：使用函式 `String::from` 從字串字面值建立 `String`

因為字串用在許多地方，我們可以使用許多不同的通用字串 API 供我們選擇。有些看起來似乎是多餘的，但是它們都有一席之地的！在上面的範例中 `String::from` 和 `to_string` 都在做相同的事，所以你的選擇跟喜好風格與閱讀性比較有關。

另外記得字串是 UTF-8 編碼的，所以我們可以包含任何正確編碼的資料，如範例 8-14 所示。

```
let hello = String::from("السلام عليكم");
let hello = String::from("Dobry den");
let hello = String::from("Hello");
let hello = String::from("ᐃᐣᐣᐣ");
let hello = String::from("नमस्ते");
let hello = String::from("こんにちは");
let hello = String::from("안녕하세요");
let hello = String::from("你好");
let hello = String::from("Olá");
let hello = String::from("Здравствуй");
let hello = String::from("Hola");
```

範例 8-14：用字串儲存各種語言打招呼的文字

以上全是合理的 `String` 數值。

更新字串

就和 `Vec<T>` 一樣，如果你插入更多資料的話，`String` 可以增長大小並變更其內容。除此之外你也可以使用 `+` 運算子或 `format!` 巨集來串接 `String` 數值。

使用 `push_str` 和 `push` 追加字串

我們可以使用 `push_str` 方法來追加一個字串切片使字串增長，如範例 8-15 所示。

```
let mut s = String::from("foo");
s.push_str("bar");
```

範例 8-15：使用 `push_str` 方法向 `String` 追加字串切片

在這兩行之後，`s` 會包含 `foobar`。 `push_str` 方法取得的是字串切片因為我們並不需要取得參數的所有權。舉例來說，在範例 8-16 我們想在 `s2` 追加其內容給 `s1` 之後仍能使用。

```
let mut s1 = String::from("foo");
let s2 = "bar";
s1.push_str(s2);
println!("s2 is {s2}");
```

範例 8-16：在內容追加給 `String` 後繼續使用字串切片

如果 `push_str` 方法會取得 `s2` 的所有權，我們就無法在最後一行印出其數值了。幸好這段程式碼是可以執行的！

而 `push` 方法會取得一個字元作為參數並加到 `String` 上。範例 8-17 顯示了一個使用 `push` 方法將字母 "l" 加到 `String` 的程式碼。

```
let mut s = String::from("lo");
s.push('l');
```

範例 8-17：使用 `push` 將一個字元加到 `String`

結果就是 `s` 會包含 `lol`。

使用 `+` 運算子或 `format!` 巨集串接字串

你通常會想要組合兩個字串在一起，其中一種方式是用 `+` 運算子。如範例 8-18 所示。

```
let s1 = String::from("Hello, ");
let s2 = String::from("world!");
let s3 = s1 + &s2; // 注意到 s1 被移動因此無法再被使用
```

範例 8-18：使用 `+` 運算子組合兩個 `String` 數值成一個新的 `String` 數值

程式碼最後的字串 `s3` 就會獲得 `Hello, world!`。 `s1` 之所以在相加後不再有效，以及 `s2` 是使用參考的原因，都和我們使用 `+` 運算子時呼叫的方法簽名有關。`+` 運算子使用的是 `add` 方法，其簽名會長得像這樣：

```
fn add(self, s: &str) -> String {
```

在標準函式庫中 `add` 是用泛型 (generics) 與關聯型別 (associated types) 定義。我們在此使用實際型別代替的 `add` 簽名。我們會在第十章討論到泛型。此簽名給了一些我們需要瞭解 `+` 運算子的一些線索。

首先 `s2` 有 `&` 代表我們是將第二個字串的參考與第一個字串相加，因為函式 `add` 中的參數 `s` 說明我們只能將 `&str` 與 `String` 相加，我們無法將兩個 `String` 數值相加。但等等 `&s2` 是 `&String` 才對，並非 `add` 第二個參數所指定的 `&str`。為何範例 8-18 可以編譯呢？

我們可以在 `add` 的呼叫中使用 `&s2` 的原因是因為編譯器可以**強制 (coerce)** `&String` 引數轉換成 `&str`。當我們我們呼叫 `add` 方法時，Rust **強制解參考 (deref coercion)** 讓 `&s2` 變成 `&s2[..]`。我們會在第十五章深入探討強制解參考。因為 `add` 不會取得 `s` 參數的所有權，`s2` 在此運算後仍然是個有效的 `String`。

再來，我們可以看到 `add` 的簽名會取得 `self` 的所有權，因為 `self` **沒有** `&`。這代表範例 8-18 的 `s1` 會移動到 `add` 的呼叫內，在之後就不再有效。所以雖然 `let s3 = s1 + &s2;` 看起來像是它拷貝了兩個字串的值並產生了一個新的，但此陳述式實際上是取得 `s1` 的所有權、追加一份 `s2` 的複製內容、然後回傳最終結果的所有權。換句話說，雖然它看起來像是產生了很多拷貝，但實際上並不是。此實作反而比較有效率。

如果我們需要串接數個字串的話，`+` 運算子的行為看起來就顯得有點笨重了：

```
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");

let s = s1 + "-" + &s2 + "-" + &s3;
```

此時 `s` 會是 `tic-tac-toe`。有這麼多的 `+` 和 `"` 字元，我們很難看清楚發生什麼事。如果要完成更複雜的字串組合的話，我們可以改使用 `format!` 巨集：

```
let s1 = String::from("tic");
let s2 = String::from("tac");
let s3 = String::from("toe");

let s = format!("{s1}-{s2}-{s3}");
```

此程式碼一樣能設置 `s` 為 `tic-tac-toe`。`format!` 巨集運作的方式和 `println!` 類似，但不會將輸出結果顯示在螢幕上，它做的是回傳內容的 `String`。使用 `format!` 的程式碼版本看起來比較好讀懂，而且 `format!` 產生的程式碼使用的是參考，所以此呼叫不會取走任何參數的所有權。

索引字串

在其他許多程式語言中，使用索引參考字串來取得獨立字元是有效且常見的操作。然而在 Rust 中如果你嘗試對 `String` 使用索引語法的話，你會得到錯誤。請看看範例 8-19 這段無效的程式碼。

```
let s1 = String::from("hello");
let h = s1[0];
```

範例 8-19：嘗試在字串使用索引語法

此程式會有以下錯誤結果：

```
$ cargo run
   Compiling collections v0.1.0 (file:///projects/collections)
error[E0277]: the type `String` cannot be indexed by `{integer}`
--> src/main.rs:3:13
   |
3  |     let h = s1[0];
   |               ^^^^^ `String` cannot be indexed by `{integer}`
   |
= help: the trait `Index<{integer}>` is not implemented for `String`
= help: the following other types implement trait `Index<Idx>`:
       <String as Index<RangeFrom<usize>>>
       <String as Index<RangeFull>>
       <String as Index<RangeInclusive<usize>>>
       <String as Index<RangeTo<usize>>>
       <String as Index<RangeToInclusive<usize>>>
       <String as Index<std::ops::Range<usize>>>
       <str as Index<I>>

For more information about this error, try `rustc --explain E0277`.
error: could not compile `collections` due to previous error
```

錯誤訊息與提示告訴了我們 Rust 字串並不支援索引。但為何不支援呢？要回答此問題，我們需要先討論 Rust 如何儲存字串進記憶體。

內部呈現

`String` 基本上就是 `Vec<u8>` 的封裝。讓我們看看範例 8-14 中一些正確編碼為 UTF-8 字串的例子，像是這一個：

```
let hello = String::from("Hola");
```

在此例中 `len` 會是 4，也就是向量儲存的字串「Hola」長度為 4 個位元組。每個字母在用 UTF-8 編碼時長度均為 1 個位元組。但接下來這行可能就會讓你感到驚訝了（請注意字串的開頭是西里爾字母 Ze 的大寫，而不是阿拉伯數字 3）。

```
let hello = String::from("Здравствуйते");
```

你可能會以為這字串的長度為 12，事實上 Rust 給的答案卻是 24。這是將「Здравствуйते」用 UTF-8 編碼後的位元組長度，因為該字串的每個 Unicode 純量都佔據兩個位元組。因此字串位元組的索引不會永遠都能對應到有效的 Unicode 純量數值。我們用以下無效的 Rust 程式碼進一步說明：

```
let hello = "Здравствуйते";  
let answer = &hello[0];
```

你已經知道第一個字母 `answer` 不會是 `З`。當經過 UTF-8 編碼時，`З` 的第一個位元組會是 208 然後第二個是 151。所以 `answer` 實際上會拿到 208，但 208 本身又不是個有效字元。回傳 208 可能不會是使用者想要的，他們希望的應該是此字串的第一個字母，但這是 Rust 在位元組索引 0 唯一能回傳的資料。就算字串都只包含拉丁字母，使用者通常也不會希望看到位元組數值作為回傳值。如果 `&"hello"[0]` 是有效程式碼且會回傳位元組數值的話，它會回傳的是 104 並非 `h`。

為了預防回傳意外數值進而導致無法立刻察覺的錯誤，Rust 不會成功編譯這段程式碼，並在開發過程前期就杜絕誤會發生。

位元組、純量數值與形素群集！我的天啊！

UTF-8 還有一個重點是在 Rust 中我們實際上可以有三種觀點來理解字串：位元組、純量數值 (scalar values) 以及形素群集 (grapheme clusters，最接近人們常說的「字母」)。

如果我們觀察用天成體寫的印度語「नमस्ते」，它存在向量中的 `u8` 數值就會長這樣：

```
[224, 164, 168, 224, 164, 174, 224, 164, 184, 224, 165, 141, 224, 164, 164,  
224, 165, 135]
```

這 18 個位元組是電腦最終儲存的資料？如果我們用 Unicode 純量數值觀察的話，也就是 Rust 的 `char` 型別，這些位元組會組成像這樣：

```
['न', 'म', 'स्', 'ते']
```

這邊有六個 `char` 數值，但第四個和第六個卻不是字母，它們是單獨存在不具任何意義的變音符號。最後如果我們以形素群集的角度來看的話，我們就會得到一般人所說的構成此印度語的四個字母：

```
["न", "म", "स्", "ते"]
```

Rust 提供多種不同的方式來解釋電腦中儲存的原始字串資料，讓每個程式無論是何種人類語言的資料，都可以選擇它們需要的呈現方式。

Rust 還有一個不允許索引 `String` 來取得字元的原因是因為，索引運算必須永遠預期是花費常數時間 ($O(1)$)。但在 `String` 上無法提供這樣的效能保證，因為 Rust 會需要從索引的開頭遍歷每個內容才能決定多少有效字元存在。

字串切片

索引字串通常不是個好點子，因為字串索引要回傳的型別是不明確的，是要一個位元組數值、一個字元、一個形素群集還是一個字串切片呢。因此如果你真的想要使用索引建立字串切片的話，Rust 會要你更明確些。要明確指定你的索引與你想要的字串切片。

與其在 `[]` 只使用一個數字來索引，你可以在 `[]` 指定一個範圍來建立包含特定位元組的字串切片：

```
let hello = "Здравствуйте";

let s = &hello[0..4];
```

`s` 在此會是 `&str` 並包含字串前 4 個位元組。稍早我們提過這些字元各佔 2 個位元組，所以這裡的 `s` 就是 `Зд`。

如果我們嘗試只用 `&hello[0..1]` 來取得字元部分的位元組的話，Rust 會和在向量中取得無效索引一樣在執行時恐慌：

```
$ cargo run
  Compiling collections v0.1.0 (file:///projects/collections)
  Finished dev [unoptimized + debuginfo] target(s) in 0.43s
  Running `target/debug/collections`
thread 'main' panicked at 'byte index 1 is not a char boundary; it is inside
'З' (bytes 0..2) of `Здравствуйте`, src/main.rs:4:14
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

你在使用範圍來建立字串切片時要格外小心，因為這樣做有可能會使你的程式崩潰。

遍歷字串的方法

要對字串的部分進行操作最好的方式是明確表達你想要的是字元還是位元組。對獨立的 Unicode 純量型別來說的話，就是使用 `chars` 方法。對「Зд」呼叫 `chars` 會將兩個擁有 `char` 型別的數值拆開並回傳，這樣一來你就可以遍歷每個元素：

```
for c in "Зд".chars() {
    println!("{}", c);
}
```

此程式碼會顯示以下輸出：

```
3
А
```

而 `bytes` 方法會回傳每個原始位元組，可能會在某些場合適合你：

```
for b in "3А".bytes() {
    println!("{}", b);
}
```

此程式碼會印出此字串的四個位元組：

```
208
151
208
180
```

請確定你已經瞭解有效的 Unicode 純量數值可能不止佔 1 個位元組。

而要從天生成組成的字串取得形素群集的話就非常複雜了，所以標準函式庫並未提供這項功能。如果你需要的話，crates.io 上會有提供這項功能的 crate。

字串並不簡單

總結來說，字串是很複雜的。不同的程式語言會選擇不同的決定來呈現給程式設計師。Rust 選擇正確處理 `String` 的方式作為所有 Rust 程式的預設行為，這也代表開發者在處理 UTF-8 資料時需要多加考量。這樣的取舍的確對比其他程式語言來說，增加了不少字串的複雜程度，但是這能讓你在開發週期免於處理非 ASCII 字元相關的錯誤。

好消息是標準函式庫針對 `String` 與 `&str` 型別提供了許多功能，來幫助正確處理這些複雜的情況。別忘了翻翻技術文件來學習這些實用的方法，像是 `contains` 能搜尋字串，而 `replace` 能替換部份字串成另一個字串。

讓我們接下去看一個較簡單地集合吧：雜湊映射（hash maps）！

透過雜湊映射儲存鍵值配對

我們最後一個常見的集合是雜湊映射（**hash map**），`HashMap<K, V>` 型別會儲存一個鍵（key）型別 `K` 對應到一個數值（value）型別 `V`。它透過雜湊函式（hashing function）來決定要將這些鍵與值放在記憶體何處。許多程式語言都有支援這種類型的資料結構，不過通常它們會提供不同的名稱，像是 `hash`、`map`、`object`、`hash table`、`dictionary` 或 `associative array` 等等。

雜湊映射適合用於當你不想像向量那樣用索引搜尋資料，而是透過一個可以為任意型別的鍵來搜尋的情況。舉例來說，在比賽中我們可以使用雜湊映射來儲存每隊的分數，每個鍵代表隊伍名稱，而每個值代表隊伍分數。給予一個隊伍名稱，你就能取得該隊伍分數。

我們會在此段落介紹雜湊映射的基本 API，但還有很多實用的函式定義在標準函式庫的 `HashMap<K, V>` 中，所以別忘了查閱標準函式庫的技術文件來瞭解更多資訊。

建立新的雜湊映射

其中一種建立空的雜湊映射的方式是使用 `new` 並透過 `insert` 加入新元素。在範例 8-20 我們追蹤兩支隊伍的分數，分別為藍隊與黃隊。藍隊初始分數有 10 分，黃隊則有 50 分。

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("藍隊"), 10);
scores.insert(String::from("黃隊"), 50);
```

範例 8-20：建立新的雜湊映射並插入一些鍵值

注意到我們需要先使用 `use` 將標準函式庫的 `HashMap` 集合引入。在我們介紹的三個常見集合中，此集合是最少被用到的，所以它並沒有包含在 `prelude` 內讓我們能自動參考。雜湊映射也沒有像前者那麼多標準函式庫提供的支援，像是內建建構它們的巨集。

和向量一樣，雜湊映射會將它們的資料儲存在堆積上。此 `HashMap` 的鍵是 `String` 型別而值是 `i32` 型別。和向量一樣，雜湊函式宣告後就都得是同類的，所有的鍵都必須是同型別，且所有的值也都必須是同型別。

取得雜湊映射的數值

我們可以透過 `get` 方法並提供鍵來取得其在雜湊映射對應的值，如範例 8-21 所示。


```

use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("藍隊"), 10);
scores.insert(String::from("黃隊"), 50);

let team_name = String::from("藍隊");
let score = scores.get(&team_name).copied().unwrap_or(0);

```

範例 8-21：取得雜湊映射中藍隊的分數

`score` 在此將會是對應藍隊的分數，而且結果會是 `10`。結果是使用 `Some` 的原因是因為 `get` 回傳的是 `Option<&V>`。如果雜湊映射中該鍵沒有對應值的話，`get` 就會回傳 `None`。所以程式會需要透過我們在第六章談到的方式處理 `Option`。

我們也可以使用 `for` 迴圈用類似的方式來遍歷雜湊映射中每個鍵值配對：

```

use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("藍隊"), 10);
scores.insert(String::from("黃隊"), 50);

for (key, value) in &scores {
    println!("{key}: {value}");
}

```

此程式會以任意順序印出每個配對：

```

黃隊： 50
藍隊： 10

```

雜湊映射與所有權

像是 `i32` 這種有實作 `Copy` 特徵的型別其數值可以被拷貝進雜湊映射之中。但對於像是 `String` 這種擁有所有權的數值則會被移動到雜湊映射，並成為該數值新的擁有者，如範例 8-22 所示。

```
use std::collections::HashMap;

let field_name = String::from("Favorite color");
let field_value = String::from("藍隊");

let mut map = HashMap::new();
map.insert(field_name, field_value);
// field_name 和 field_value 在這之後就不能使用了，你可以試著使用它們並看看編譯器回傳
什麼錯誤
```

範例 8-22：展示當鍵值插入雜湊映射後就會擁有它們

我們之後就無法使用變數 `field_name` 和 `field_value`，因為它們的值已經透過呼叫 `insert` 被移入雜湊映射之中。

如果我們插入雜湊映射的數值是參考的話，該值就不會被移動到雜湊映射之中。不過該值的參考就必須一直有效，至少直到該雜湊映射離開作用域為止。我們會在第十章的「[透過生命週期驗證參考](#)」段落討論更多細節。

更新雜湊映射

雖然鍵值配對的數量可以增加，但每個鍵同一時間就只能有一個對應的值而已。（反之並不成立：比如藍隊黃隊可以同時都在 `scores` 雜湊映射內儲存 10 分）

當你想要改變雜湊映射的資料的話，你必須決定如何處理當一個鍵已經有一個值的情況。你可以不管舊的值，直接用新值取代。你也可以保留舊值、忽略新值，只有在該鍵尚未擁有對應數值時才賦值給它。或者你也可以將舊值與新值組合起來。讓我們看看分別怎麼處理吧！

覆蓋數值

如果我們在雜湊映射插入一個鍵值配對，然後又在相同鍵插入不同的數值的話，該鍵相對應的數值就會被取代。如範例 8-23 雖然我們呼叫了兩次 `insert`，但是雜湊映射只會保留一個鍵值配對，因為我們向藍隊的鍵插入了兩次數值。

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("藍隊"), 10);
scores.insert(String::from("藍隊"), 25);

println!("{:?}", scores);
```

範例 8-23：替換某個特定鍵對應的數值

此程式碼會印出 `{"藍隊": 25}`，原本的數值 `10` 會被覆蓋。

只在鍵不存在的情況下插入鍵值

通常檢查雜湊映射有沒有存在某個特定的鍵值是很常見的。我們接下來的動作通常就是檢查如果鍵存在於雜湊映射的話，就不改變其值。但如果鍵不存在的話，就插入數值給它。

雜湊映射提供了一個特別的 API 叫做 `entry` 讓你可以用想要檢查的鍵作為參數。`entry` 方法的回傳值是一個列舉叫做 `Entry`，它代表了一個可能存在或不存在的數值。假設我們想要檢查黃隊的鍵有沒有對應的數值。如果沒有的話，我們想插入 50。而對藍隊也一樣。使用 `entry` API 的話，程式碼會長得像範例 8-24。

```
use std::collections::HashMap;

let mut scores = HashMap::new();

scores.insert(String::from("藍隊"), 10);

scores.entry(String::from("黃隊")).or_insert(50);
scores.entry(String::from("藍隊")).or_insert(50);

println!("{:?}", scores);
```

範例 8-24：使用 `entry` 方法在只有該鍵尚無任何數值時插入數值

`Entry` 中的 `or_insert` 方法定義了如果 `Entry` 的鍵有對應的數值的話，就回傳該值的可變參考；如果沒有的話，那就插入參數作為新數值，並回傳此值的可變參考。這樣的技巧比我們親自寫邏輯還來的清楚，而且更有利於借用檢查器的檢查。

執行範例 8-25 的程式碼會印出 `{"黃隊": 50, "藍隊": 10}`。第一次 `entry` 的呼叫會對黃隊插入數值 50，因為黃隊尚未有任何數值。第二次 `entry` 的呼叫則不會改變雜湊映射，因為藍隊已經有數值 10。

依據舊值更新數值

雜湊映射還有另一種常見的用法是，依照鍵的舊數值來更新它。舉例來說，範例 8-25 展示了一支如何計算一些文字內每個單字各出現多少次的程式碼。我們使用雜湊映射，鍵為單字然後值為我們每次追蹤計算對應單字出現多少次的次數。如果我們是第一次看到該單字的話，我們插入數值 0。

```

use std::collections::HashMap;

let text = "hello world wonderful world";

let mut map = HashMap::new();

for word in text.split_whitespace() {
    let count = map.entry(word).or_insert(0);
    *count += 1;
}

println!("{:?}", map);

```

範例 8-25：使用雜湊映射儲存單字與次數來計算每個字出現的次數

此程式碼會印出 `{"world": 2, "hello": 1, "wonderful": 1}`。你可能會看到鍵值配對的順序不太一樣，回想一下「取得雜湊映射的數值」段落中提過遍歷雜湊映射的順序是任意的。

`split_whitespace` 方法會遍歷 `text` 中被空格分開來的切片。`or_insert` 方法會回傳該鍵對應數值的可變參考（`&mut V`）。在此我們將可變參考儲存在 `count` 變數中，所以要賦值的話，我們必須先使用 `*` 來解參考（dereference）`count`。可變參考會在 `for` 結束時離開作用域，所以所有的改變都是安全的且符合借用規則。

雜湊函式

`HashMap` 預設是使用一種叫做 SipHash 的雜湊函式（hashing function），這可以透過¹ 雜湊表（hash table）抵禦阻斷服務（Denial of Service, DoS）的攻擊。這並不是最快的雜湊演算法，但為了提升安全性而犧牲一點效能是值得的。如果你做評測時覺得預設的雜湊函式太慢無法滿足你的需求的話，你可以指定不同的 *hasher* 來切換成其他雜湊函式。`Hasher` 是一個有實作 `BuildHasher` 特徵的型別。我們會在第十章討論到特徵以及如何實作它們。你不必從頭自己實作一個 hasher，crates.io 上有其他 Rust 使用者分享的函式庫，其中就有不少提供許多常見雜湊演算法的 hasher 實作。

¹ <https://en.wikipedia.org/wiki/SipHash>

總結

當你的程式需要儲存、取得、修改資料時，向量、字串與雜湊映射可以提供大量的功能。以下是一些你應該能夠解決的練習題：

- 給予一個整數列表，請使用向量並回傳中位數（排序列表後正中間的值）以及眾數（出現最多次的值，雜湊映射在此應該會很有用）。

- 將字串轉換成 pig latin。每個單字的第一個字母為子音的話，就將該字母移到單字後方，並加上「ay」，所以「first」會變成「irst-fay」。而單字第一個字母為母音的話，就在單字後方加上「hay」，所以「apple」會變成「apple-hay」。請注意要考慮到 UTF-8 編碼！
- 使用雜湊映射與向量來建立文字介面，讓使用者能新增員工名字到公司內的一個部門。舉來說「將莎莉加入工程部門」或「將阿米爾加入業務部門」。然後讓使用者可以索取一個部門所有的員工列表，或是依據部門用字典順序排序，取得公司內所有的員工。

標準函式庫的 API 技術文件有詳細介紹向量、字串與雜湊映射的所有方法，這對於這些練習題應該會很有幫助！

我們現在已經開始遇到有可能會運作失敗的複雜程式了，所以接下來正是來討論錯誤處理的時候！

錯誤處理

錯誤是軟體開發中不可避免的一環，所以 Rust 有一些特色能夠處理發生錯誤的情形。在許多情況下，Rust 要求你要能知道可能出錯的地方，並在編譯前採取行動。這樣的要求能讓你的程式更穩定，確保你能發現錯誤並在程式碼發佈到生產環境前妥善處理它們！

Rust 將錯誤分成兩大類：**可復原的**（recoverable）和**不可復原的**（unrecoverable）錯誤。像是**找不到檔案**這種可復原的錯誤，我們通常很可能只想回報問題給使用者並重試。而不可復原的錯誤就會是程式錯誤的跡象，像是嘗試取得陣列結尾之後的位置。

許多語言不會區分這兩種錯誤，並以相同的方式處理，使用像是例外（exceptions）這樣統一的機制處理。Rust 沒有例外處理機制，取而代之的是它對可復原的錯誤提供 `Result<T, E>` 型別，對不可復原的錯誤使用 `panic!` 將程式停止執行。本章節會先介紹 `panic!` 再來討論 `Result<T, E>` 數值的回傳。除此之外，我們也將探討何時該從錯誤中復原，何時該選擇停止程式。

對無法復原的錯誤使用 `panic!`

有時候壞事就是會發生在你的程式中，這本來就是你沒辦法全部避免的。在這種情況，Rust 有提供 `panic!` 巨集。在實際情況下我們有兩種方式可以造成恐慌：做出確定會讓程式碼恐慌的動作（像是存取陣列範圍外的元素），或是直接呼叫 `panic!` 巨集。在這兩種狀況下，我們都對程式造成了恐慌。這些恐慌預設會印出程式出錯的訊息，展開並清理堆疊，然後離開程式。再加上環境變數的話，你還可以讓 Rust 顯示恐慌時呼叫的堆疊，讓你能更簡單地追蹤恐慌的源頭。

恐慌時該解開堆疊還是直接終止

當恐慌（panic）發生時，程式預設會開始做解開（unwind）堆疊的動作，這代表 Rust 會回溯整個堆疊，並清理每個它遇到的函式資料。但是這樣回溯並清理的動作很花力氣。另一種方式是直接終止（abort）程式而不清理，程式使用的記憶體會需要由作業系統來清理。

如果你需要你的專案產生的執行檔越小越好，你可以從解開切換成終止，只要在 *Cargo.toml* 檔案中的 `[profile]` 段落加上 `panic = 'abort'` 就好。舉例來說，如果你希望在發佈模式（release mode）恐慌時直接終止，那就加上：

```
[profile.release]
panic = 'abort'
```

讓我們先在小程式內試試呼叫 `panic!`：

檔案名稱：src/main.rs

```
fn main() {
    panic!("🚧 崩潰 (▯▯▯) 潰 (▯▯▯)");
}
```

當你執行程式時，你會看到像這樣的結果：

```
$ cargo run
   Compiling panic v0.1.0 (file:///projects/panic)
   Finished dev [unoptimized + debuginfo] target(s) in 0.25s
   Running `target/debug/panic`
thread 'main' panicked at '🚧 崩潰 (▯▯▯) 潰 (▯▯▯)', src/main.rs:2:5
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

`panic!` 的呼叫導致印出了最後兩行的錯誤訊息。第一行顯示了我們的恐慌訊息以及該恐慌是在原始碼何處發生的：`src/main.rs:2:5` 指的是它發生在我們的 *src/main.rs* 檔案第二行第五個字元。

在此例中，該行指的就是我們寫的程式碼。如果我們查看該行，我們會看到 `panic!` 巨集的呼叫。在其他情形，`panic!` 的呼叫可能會發生在我們呼叫的其他程式碼內，所以錯誤訊息回報的檔案名稱與行數可能就會是其他人呼叫 `panic!` 巨集的程式碼，而不是因為我們的程式碼才導致 `panic!` 的呼叫。我們可以在呼叫 `panic!` 程式碼的地方使用 `backtrace` 來找出出現問題的地方。接下來我們就會深入瞭解 `backtrace`。

使用 `panic!` `Backtrace`

讓我們看看另一個例子，這是函式庫發生錯誤而呼叫 `panic!`，而不是來自於我們在程式碼自己呼叫的巨集。範例 9-1 是個嘗試從向量有效範圍外取得索引的例子。

檔案名稱：src/main.rs

```
fn main() {  
    let v = vec![1, 2, 3];  
  
    v[99];  
}
```



範例 9-1：嘗試取得超出向量長度的元素，進而導致 `panic!` 被呼叫

我們在這邊嘗試取得向量中第 100 個元素（不過因為索引從零開始，所以是索引 99），但是該向量只有 3 個元素。在此情況下，Rust 就會恐慌。使用 `[]` 會回傳元素，但是如果你傳遞了無效的索引，Rust 就回傳不了正確的元素。

在 C 中，嘗試讀取資料結構結束之後的元素屬於未定義行為。你可能會得到該記憶體位置對應其資料結構的元素，即使該記憶體完全不屬於該資料結構。這就稱做緩衝區過讀（buffer overread）而且會導致安全漏洞。攻擊者可能故意操縱該索引來取得在資料結構後面他們原本不應該讀寫的值。

為了保護你的程式免於這樣的漏洞，如果你嘗試用一個不存在的索引讀取元素的話，Rust 會停止執行並拒絕繼續運作下去。讓我們嘗試執行並看看會如何：

```
$ cargo run  
Compiling panic v0.1.0 (file:///projects/panic)  
Finished dev [unoptimized + debuginfo] target(s) in 0.27s  
Running `target/debug/panic`  
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 99', src/main.rs:4:5  
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

此錯誤指向 `main.rs` 的第四行，也就是我們嘗試存取索引 99 的地方。下一行提示告訴我們可以設置 `RUST_BACKTRACE` 環境變數來取得 `backtrace` 以知道錯誤發生時到底發生什麼事。`backtrace` 是一個函式列表，指出得到此錯誤時到底依序呼叫了哪些函式。Rust 的 `backtraces` 運作方式和其他語言一樣：讀取 `backtrace` 關鍵是從最一開始讀取直到你看到你寫的檔案。那就會是問題發生的

源頭。那行以上的行數就是你所呼叫的程式，而以下則是其他呼叫你的程式碼的程式。這些行數可能還會包含 Rust 核心程式碼、標準函式庫程式碼，或是你所使用的 crate。我們設置 `RUST_BACKTRACE` 環境變數的值不為 0，來嘗試取得 backtrace 吧。你應該會看到和範例 9-2 類似的結果。

```
$ RUST_BACKTRACE=1 cargo run
thread 'main' panicked at 'index out of bounds: the len is 3 but the index is 99', src/main.rs:4:5
stack backtrace:
   0: rust_begin_unwind
       at
/rustc/e092d0b6b43f2de967af0887873151bb1c0b18d3/library/std/src/panicking.rs:584:5
   1: core::panicking::panic_fmt
       at
/rustc/e092d0b6b43f2de967af0887873151bb1c0b18d3/library/core/src/panicking.rs:142:14
   2: core::panicking::panic_bounds_check
       at
/rustc/e092d0b6b43f2de967af0887873151bb1c0b18d3/library/core/src/panicking.rs:84:5
   3: <usize as core::slice::index::SliceIndex<T>>::index
       at
/rustc/e092d0b6b43f2de967af0887873151bb1c0b18d3/library/core/src/slice/index.rs:242:10
   4: core::slice::index::<impl core::ops::index::Index<I> for [T]>::index
       at
/rustc/e092d0b6b43f2de967af0887873151bb1c0b18d3/library/core/src/slice/index.rs:18:9
   5: <alloc::vec::Vec<T,A> as core::ops::index::Index<I>>::index
       at
/rustc/e092d0b6b43f2de967af0887873151bb1c0b18d3/library/alloc/src/vec/mod.rs:2591:9
   6: panic::main
       at ./src/main.rs:4:5
   7: core::ops::function::FnOnce::call_once
       at
/rustc/e092d0b6b43f2de967af0887873151bb1c0b18d3/library/core/src/ops/function.rs:248:5
note: Some details are omitted, run with `RUST_BACKTRACE=full` for a verbose backtrace.
```

範例 9-2：當 `RUST_BACKTRACE` 設置時，透過呼叫 `panic!` 產生的 backtrace

輸出結果有點多啊！你看到的實際輸出可能會因你的作業系統與 Rust 版本而有所不同。要取得這些資訊的 backtrace，除錯符號（debug symbols）必須啟用。當我們在使用 `cargo build` 或 `cargo run` 且沒有加上 `--release` 時，除錯符號預設是啟用的。

在範例 9-2 的輸出結果中，第 6 行的 `backtrace` 指向了我們專案中產生問題的地方：`src/main.rs` 中的第四行。如果我們不想讓程式恐慌，我們就要來調查我們所寫的程式中第一個被錯誤訊息指向的位置。在範例 9-1 中，我們故意寫出會恐慌的程式碼。要修正的方法就是不要索取超出向量索引範圍的元素。當在未來你的程式碼恐慌時，你會需要知道是程式碼中的什麼動作造成的、什麼數值導致恐慌以及正確的程式碼該怎麼處理。

我們會在本章節「[要 `panic!` 還是不要 `panic!`](#)」的段落中再回來看 `panic!` 並研究何時該與不該使用 `panic!` 來處理錯誤條件。接下來，我們要看如何使用 `Result` 來處理可回復的錯誤。

Result 與可復原的錯誤

大多數的錯誤沒有嚴重到需要讓整個程式停止執行。有時候當函式失敗時，你是可以輕易理解並作出反應的。舉例來說，如果你嘗試開啟一個檔案，但該動作卻因為沒有該檔案而失敗的話，你可能會想要建立檔案，而不是終止程序。

回憶一下第二章的「使用 `Result` 型別可能的錯誤」提到 `Result` 列舉的定義有兩個變體 `Ok` 和 `Err`，如以下所示：

```
enum Result<T, E> {  
    Ok(T),  
    Err(E),  
}
```

`T` 和 `E` 是泛型型別參數，我們會在第十章深入討論泛型。你現在需要知道的是 `T` 代表我們在成功時會在 `Ok` 變體回傳的型別，而 `E` 則代表失敗時在 `Err` 變體會回傳的錯誤型別。因為 `Result` 有這些泛型型別參數，我們可以將 `Result` 型別和它的函式用在許多不同場合，讓成功與失敗時回傳的型別不相同。

讓我們呼叫一個可能會失敗的函式並回傳 `Result` 型別。在範例 9-3 我們嘗試開啟一個檔案。

檔案名稱：src/main.rs

```
use std::fs::File;  
  
fn main() {  
    let greeting_file_result = File::open("hello.txt");  
}
```

範例 9-3：嘗試開啟一個檔案

`File::open` 的回傳型別為 `Result<T, E>`。泛型參數 `T` 在此已經被 `File::open` 指明成功時會用到的型別 `std::fs::File`，也就是檔案的控制代碼（handle）。用於錯誤時的 `E` 型別則是 `std::io::Error`。這樣的回傳型別代表 `File::open` 的呼叫在成功時會回傳我們可以讀寫的檔案控制代碼，但該函式呼叫也可能失敗。舉例來說，該檔案可能會不存在，或者我們沒有檔案的存取權限。`File::open` 需要有某種方式能告訴我們它的結果是成功或失敗，並回傳檔案控制代碼或是錯誤資訊。這樣的資訊正是 `Result` 列舉想表達的。

如果 `File::open` 成功的話，變數 `greeting_file_result` 的數值就會獲得包含檔案控制代碼的 `Ok` 實例。如果失敗的話，`greeting_file_result` 的值就會是包含為何產生該錯誤的資訊的 `Err` 實例。

我們需要讓範例 9-3 的程式碼依據 `File::open` 回傳不同的結果採取不同的動作。範例 9-4 展示了其中一種處理 `Result` 的方式，我們使用第六章提到的 `match` 表達式。

檔案名稱：src/main.rs

```
use std::fs::File;

fn main() {
    let greeting_file_result = File::open("hello.txt");

    let greeting_file = match greeting_file_result {
        Ok(file) => file,
        Err(error) => panic!("開啟檔案時發生問題：{:?}", error),
    };
}
```

範例 9-4：使用 `match` 表達式來處理回傳的 `Result` 變體

和 `Option` 列舉一樣，`Result` 列舉與其變體都會透過 `prelude` 引入作用域，所以我們不需要指明 `Result::`，可以直接在 `match` 的分支中使用 `Ok` 和 `Err` 變體。

當結果是 `Ok` 時，這裡的程式碼就會回傳 `Ok` 變體中內部的 `file`，然後我們就可以將檔案控制代碼賦值給變數 `greeting_file`。在 `match` 之後，我們就可以使用檔案控制代碼來讀寫。

`match` 的另一個分支則負責處理我們從 `File::open` 中取得的 `Err` 數值。在此範例中，我們選擇呼叫 `panic!` 巨集。如果檔案 `hello.txt` 不存在我們當前的目錄的話，我們就會執行此程式碼，接著就會看到來自 `panic!` 巨集的輸出結果：

```
$ cargo run
Compiling error-handling v0.1.0 (file:///projects/error-handling)
Finished dev [unoptimized + debuginfo] target(s) in 0.73s
Running `target/debug/error-handling`
thread 'main' panicked at '開啟檔案時發生問題：0s { code: 2, kind: NotFound,
message: "No such file or directory" }', src/main.rs:8:23
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

如往常一樣，此輸出告訴我們哪裡出錯了。

配對不同種的錯誤

範例 9-4 的程式碼不管 `File::open` 為何失敗都會呼叫 `panic!`。不過我們想要依據不同的錯誤原因採取不同的動作，如果 `File::open` 是因為檔案不存在的話，我們想要建立檔案並回傳新檔案的控制代碼。如果 `File::open` 是因為其他原因失敗的話，像是我們沒有開啟檔案的權限，我們仍然要像範例 9-4 這樣呼叫 `panic!`。對此我們可以加上 `match` 表達式，如範例 9-5 所示。

檔案名稱：src/main.rs

```

use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file_result = File::open("hello.txt");

    let greeting_file = match greeting_file_result {
        Ok(file) => file,
        Err(error) => match error.kind() {
            ErrorKind::NotFound => match File::create("hello.txt") {
                Ok(fc) => fc,
                Err(e) => panic!("建立檔案時發生問題：{:?}", e),
            },
            other_error => {
                panic!("開啟檔案時發生問題：{:?}", other_error);
            }
        },
    };
}

```

範例 9-5：針對不同種類的錯誤採取不同動作

`File::open` 在 `Err` 變體的回傳型別為 `io::Error`，這是標準函式庫提供的結構體。此結構體有個 `kind` 方法讓我們可以取得 `io::ErrorKind` 數值。標準函式庫提供的列舉 `io::ErrorKind` 有從 `io` 運算可能發生的各種錯誤。我們想處理的變體是 `ErrorKind::NotFound`，這指的是我們嘗試開啟的檔案還不存在。所以我們對 `greeting_file_result` 配對並在用 `error.kind()` 繼續配對下去。

我們從內部配對檢查 `error.kind()` 的回傳值是否是 `ErrorKind` 列舉中的 `NotFound` 變體。如果是的話，我們就嘗試使用 `File::create` 建立檔案。不過 `File::create` 也可能會失敗，所以我們需要第二個內部 `match` 表達式來處理。如果檔案無法建立的話，我們就會印出不同的錯誤訊息。第二個分支的外部 `match` 分支保持不變，如果程式遇到其他錯誤的話就會恐慌。

除了使用 `match` 配對 `Result<T, E>` 以外的方式

我們用的 `match` 的確有點多！`match` 表達式雖然很實用，不過它的行為非常基本。在第十三章你會學到閉包（closure），`Result<T, E>` 型別有很多接收閉包的方法。使用這些方法可以讓你的程式碼更簡潔。

舉例來說，以下是另一種能寫出與範例 9-5 邏輯相同的程式碼，這次則是使用到閉包與 `unwrap_or_else` 方法：

```

use std::fs::File;
use std::io::ErrorKind;

fn main() {
    let greeting_file = File::open("hello.txt").unwrap_or_else(|error| {
        if error.kind() == ErrorKind::NotFound {
            File::create("hello.txt").unwrap_or_else(|error| {
                panic!("建立檔案時發生問題：{:?}", error);
            })
        } else {
            panic!("開啟檔案時發生問題：{:?}", error);
        }
    });
}

```

雖然此程式碼的行為和範例 9-5 一樣，但他沒有包含任何 `match` 表達式而且更易閱讀。當你讀完第十三章後，別忘了回來看看此範例，並查閱標準函式庫中的 `unwrap_or_else` 方法。除此方法以外，還有更多方法可以來解決處理錯誤時龐大的 `match` 表達式。

錯誤發生時產生恐慌的捷徑：`unwrap` 與 `expect`

雖然 `match` 已經足以勝任指派的任務了，但它還是有點冗長，而且可能無法正確傳遞錯誤的嚴重性。`Result<T, E>` 型別有非常多的輔助方法來執行不同的特定任務。`unwrap` 就和我們在範例 9-4 所寫的 `match` 表達式一樣，擁有類似效果的捷徑方法。如果 `Result` 的值是 `Ok` 變體，`unwrap` 會回傳 `Ok` 裡面的值；如果 `Result` 是 `Err` 變體的話，`unwrap` 會呼叫 `panic!` 巨集。以下是使用 `unwrap` 的方式：

檔案名稱：src/main.rs

```

use std::fs::File;

fn main() {
    let greeting_file = File::open("hello.txt").unwrap();
}

```

如果我們沒有 `hello.txt` 這個檔案並執行此程式碼的話，我們會看到從 `unwrap` 方法所呼叫的 `panic!` 回傳訊息：

```

thread 'main' panicked at 'called `Result::unwrap()` on an `Err` value: Os {
code: 2, kind: NotFound, message: "No such file or directory" }',
src/main.rs:4:49

```

還有另一個方法 `expect` 和 `unwrap` 類似，不過能讓我們選擇 `panic!` 回傳的錯誤訊息。使用 `expect` 而非 `unwrap` 並提供完善的錯誤訊息可以表明你的意圖，讓追蹤恐慌的源頭更容易。

`expect` 的語法看起來就像這樣：

檔案名稱：src/main.rs

```
use std::fs::File;

fn main() {
    let greeting_file = File::open("hello.txt")
        .expect("hello.txt 應該要存在此專案中");
}
```

我們使用 `expect` 的方式和 `unwrap` 一樣，不是回傳檔案控制代碼就是呼叫 `panic!` 巨集。使用 `expect` 呼叫 `panic!` 時的錯誤訊息會是我們傳遞給 `expect` 的參數，而不是像 `unwrap` 使用 `panic!` 預設的訊息。訊息看起來就會像這樣：

```
thread 'main' panicked at 'hello.txt 應該要存在此專案中: Os {
code: 2, kind: NotFound, message: "No such file or directory" }',
src/main.rs:5:10
```

在正式環境等級的程式碼，大多數 Rustaceans 會選擇 `expect` 而不是 `unwrap`，這樣能在出錯時提供更多資訊，告訴我們為何預期該動作永遠成功。這樣一來就算你的假設證明錯誤，你都能夠在除錯時有足夠的資訊來理解。

傳播錯誤

當函式實作呼叫的程式碼可能會失敗時，與其直接在該函式本身處理錯誤，你可以回傳錯誤給呼叫此程式的程式碼，由它們決定如何處理。這稱之為**傳播**（propagating）錯誤並讓呼叫者可以有更多的控制權，因為比起你程式碼當下的內容，回傳的錯誤可能提供更多資訊與邏輯以利處理。

舉例來說，範例 9-6 展示了一個從檔案讀取使用者名稱的函式。如果檔案不存在或無法讀取的話，此函式會回傳該錯誤給呼叫該函式的程式碼。

檔案名稱：src/main.rs

```

use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let username_file_result = File::open("hello.txt");

    let mut username_file = match username_file_result {
        Ok(file) => file,
        Err(e) => return Err(e),
    };

    let mut username = String::new();

    match username_file.read_to_string(&mut username) {
        Ok(_) => Ok(username),
        Err(e) => Err(e),
    }
}

```

範例 9-6：使用 `match` 回傳錯誤給呼叫者的函式

此函式還能再更簡化，但我們要先繼續手動處理來進一步探討錯誤處理，最後我們會展示最精簡的方式。讓我們先看看此函式的回傳型別 `Result<String, io::Error>`。這代表此函式回傳的型別為 `Result<T, E>`，其中泛型型別 `T` 已經指明為實際型別 `String`，而泛型型別 `E` 則指明為實際型別 `io::Error`。

如果函式正確無誤的話，程式碼會呼叫此函式並收到擁有 `String` 的 `Ok` 數值。如果程式遇到任何問題的話，呼叫此函式的程式碼就會獲得擁有包含相關問題發生資訊的 `io::Error` 實例的 `Err` 數值。我們選擇 `io::Error` 作為函式的回傳值是因為它正是 `File::open` 函式和 `read_to_string` 方法失敗時的回傳的錯誤型別。

函式本體從呼叫 `File::open` 開始，然後我們使用 `match` 回傳 `Result` 數值，就和範例 9-4 的 `match` 類似。如果 `File::open` 成功的話，變數 `file` 中的檔案控制代碼賦值給可變變數 `username_file` 並讓函式繼續執行下去。但在 `Err` 的情形時，與其呼叫 `panic!`，我們使用 `return` 關鍵字來讓函式提早回傳，並將 `File::open` 的錯誤值，也就是模式中的變數 `e`，作為此函式的錯誤值回傳給呼叫的程式碼。

所以如果我們在 `username_file` 有拿到檔案控制代碼的話，接著函式就會在變數 `username` 建立新的 `String` 並對檔案控制代碼 `username_file` 呼叫 `read_to_string` 方法來讀取檔案內容至 `username`。`read_to_string` 也會回傳 `Result` 因為它也可能失敗，就算 `File::open` 是執行成功的。所以我們需要另一個 `match` 來處理該 `Result`，如果 `read_to_string` 成功的話，我們的函式就是成功的，然後在 `Ok` 回傳 `username` 中該檔案的使用者名稱。如果 `read_to_string` 失敗的話，我們就像處理 `File::open` 的 `match` 一樣回傳錯誤值。不過我們不需要顯式寫出 `return`，因為這是函式中的最後一個表達式。

呼叫此程式碼的程式就會需要處理包含使用者名稱的 `Ok` 數值以及包含 `io::Error` 的 `Err` 數值。這交給呼叫的程式碼來決定如何處理這些數值。舉例來說，如果呼叫此程式碼而獲得錯誤的

話，它可能選擇呼叫 `panic!` 讓程式崩潰，或者使用預設的使用者名稱從檔案以外的地方尋找該使用者。所以我們傳播所有成功或錯誤的資訊給呼叫者，讓它們能妥善處理。

這樣傳播錯誤的模式是非常常見的，所以 Rust 提供了 `?` 來簡化流程。

傳播錯誤的捷徑：`?` 運算子

範例 9-7 是另一個 `read_username_from_file` 的實作，擁有和範例 9-6 一樣的效果，不過這次使用了 `?` 運算子。

檔案名稱：src/main.rs

```
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let mut username_file = File::open("hello.txt");
    let mut username = String::new();
    username_file.read_to_string(&mut username)?;
    Ok(username)
}
```

範例 9-7：使用 `?` 運算子回傳錯誤給呼叫者的函式

定義在 `Result` 數值後的 `?` 運作方式幾乎與範例 9-6 的 `match` 表達式處理 `Result` 的方式一樣。如果 `Result` 的數值是 `Ok` 的話，`Ok` 內的數值就會從此表達式回傳，然後程式就會繼續執行。如果數值是 `Err` 的話，`Err` 就會使用 `return` 關鍵字作為整個函式的回傳值回傳，讓錯誤數值可以傳遞給呼叫者的程式碼。

不過範例 9-6 的 `match` 表達式做的事和 `?` 運算子做的事還是有不同的地方：`?` 運算子呼叫所使用的錯誤數值會傳遞到 `from` 函式中，這是定義在標準函式庫的 `From` 特徵中，用來將數值從一種型別轉換另一種型別。當 `?` 運算子呼叫 `from` 函式時，接收到的錯誤型別會轉換成目前函式回傳值的錯誤型別。這在當函式要回傳一個錯誤型別來代表所有函式可能的失敗是很有用的，即使可能會失敗的原因有很多種。

舉例來說，我們可以將範例 9-7 的函式 `read_username_from_file` 改成回傳一個我們自訂的錯誤型別叫做 `OurError`。如果我們有定義 `impl From<io::Error> for OurError` 能從 `io::Error` 建立一個 `OurError` 實例的話，那麼 `read_username_from_file` 本體中的 `?` 運算就會呼叫 `from` 然後轉換錯誤型別，不必在函式那多加任何程式碼。

在範例 9-7 中，在 `File::open` 的結尾中 `?` 回傳 `Ok` 中的數值給變數 `username_file`。如果有錯誤發生時，`?` 運算子會提早回傳整個函式並將 `Err` 的數值傳給呼叫的程式碼。同理也適用在呼叫 `read_to_string` 結尾的 `?`。

`?` 運算子可以消除大量樣板程式碼並讓函式實作更簡單。我們還可以再進一步將方法直接串接到 `?` 後來簡化程式碼，如範例 9-8 所示。

檔案名稱：src/main.rs

```
use std::fs::File;
use std::io::{self, Read};

fn read_username_from_file() -> Result<String, io::Error> {
    let mut username = String::new();

    File::open("hello.txt")?.read_to_string(&mut username)?;

    Ok(username)
}
```

範例 9-8：在 `?` 運算子後方串接方法呼叫

我們將建立新 `String` 的變數 `username` 移到函式的開頭，這部分沒有任何改變。再來與建立變數 `username_file` 的地方不同的是，我們直接將 `read_to_string` 串接到 `File::open("hello.txt")?` 的結果後方。我們在 `read_to_string` 呼叫的結尾還是有 `?`，然後我們還是在 `File::open` 和 `read_to_string` 成功沒有失敗時，回傳包含 `username` 的 `Ok` 數值。函式達成的效果仍然與範例 9-6 與 9-7 相同。這只是一個比較不同但慣用的寫法。

說到此函式不同的寫法，範例 9-9 展示了使用 `fs::read_to_string` 更短的寫法。

檔案名稱：src/main.rs

```
use std::fs;
use std::io;

fn read_username_from_file() -> Result<String, io::Error> {
    fs::read_to_string("hello.txt")
}
```

範例 9-9：使用 `fs::read_to_string` 而不是開啟檔案後才讀取

讀取檔案至字串中算是個常見動作，所以標準函式庫提供了一個方便的函式

`fs::read_to_string` 來開啟檔案、建立新的 `String`、讀取檔案內容、將內容放入該 `String` 並回傳它。不過使用 `fs::read_to_string` 就沒有機會讓我們來解釋所有的錯誤處理，所以我們一開始才用比較長的寫法。

? 運算子可以用在哪裡？

`?` 運算子只能用在有函式的回傳值相容於 `?` 使用的值才行。這是因為 `?` 運算子會在函式中提早回傳數值，就像我們在範例 9-6 那樣用 `match` 表達式提早回傳一樣。在範例 9-6 中，`match` 使用的是 `Result` 數值，函式的回傳值必須是 `Result` 才能相容於此 `return`。

讓我們看看在範例 9-10 的 `main` 函式中的回傳值要是不相容於我們用在 `?` 的型別，如果我們使用 `?` 運算子會發生什麼事：

```
use std::fs::File;

fn main() {
    let greeting_file = File::open("hello.txt");
}
```



檔案名稱：src/main.rs

範例 9-10：嘗試在回傳 `()` 的 `main` 函式中使用 `?` 會無法編譯

此程式碼會開啟檔案，所以可能會失敗。`?` 運算子會拿到 `File::open` 回傳的 `Result` 數值，但是此 `main` 函式的回傳值為 `()` 而非 `Result`。當我們編譯此程式碼時，我們會得到以下錯誤訊息：

```
$ cargo run
   Compiling error-handling v0.1.0 (file:///projects/error-handling)
error[E0277]: the `?` operator can only be used in a function that returns
`Result` or `Option` (or another type that implements `FromResidual`)
--> src/main.rs:4:48
   |
3 | fn main() {
   | ----- this function should return `Result` or `Option` to accept `?`
4 |     let greeting_file = File::open("hello.txt");
   |                                     ^ cannot use the `?`
operator in a function that returns `()`
   |
   = help: the trait `FromResidual<Result<Infallible, std::io::Error>>` is not
implemented for `()`

For more information about this error, try `rustc --explain E0277`.
error: could not compile `error-handling` due to previous error
```

此錯誤告訴我們只能在回傳型別為 `Result` 或 `Option` 或其他有實作 `FromResidual` 的型別的函式才能使用 `?` 運算子。

要修正此錯誤的話，你有兩種選擇。其中一種是如果你沒有任何限制的話，你可以將函式回傳值變更成與 `?` 運算子相容的型別。另一種則是依照可能的情境使用 `match` 或 `Result<T, E>` 其中一種方法來處理 `Result<T, E>`。

錯誤訊息還提到了 `?` 也能用在 `Option<T>` 的數值。就像 `?` 能用在 `Result` 一樣，你只能在有回傳 `Option` 的函式中，對 `Option` 的值使用 `?`。在 `Option<T>` 呼叫 `?` 的行為與在 `Result<T, E>` 上呼叫類似：如果數值為 `None`，`None` 就會在函式該處被提早回傳；如果數值為 `Some`，`Some` 中的值就會是表達式的結果數值，且程式會繼續執行下去。以下範例 9-11 的函式會尋找給予文字的第一行中最後一個字元：

```
fn last_char_of_first_line(text: &str) -> Option<char> {
    text.lines().next()?.chars().last()
}
```

範例 9-11：Option<T> 的數值上使用在 ? 運算子

此函式會回傳 Option<char>，因為它可能會在此真的找到字元，或者可能根本沒有半個字存在。此程式碼接受引數 text 字串切片，並呼叫它的 lines 方法，這會回傳一個遍歷字串每一行的疊代器。因為此函式想要的是第一行，它對疊代器只呼叫 next 來取得疊代器的第一個數值。如果 text 是空字串的話，這裡 next 的呼叫就會回傳 None。我們這裡就可以使用 ? 來中斷 last_char_of_first_line 並回傳 None。如果 text 不是空字串的話，next 會用 Some 數值來回傳 text 的第一行字串切片。

? 會取出字串切片，然後我們可以對字串切片呼叫 chars 來取得它的疊代器。我們在意的是第一行的最後一個字元，所以我們呼叫 last 來取得疊代器的最後一個值。這也是個 Option 因為第一行可能是個空字串。如果 text 開頭就換行，但在下一行有字元的話，它可能就會是 "\nhi"。不過如果第一行真的有最後一個字元的話，它就會回傳 Some 變體。在這過程中的 ? 運算子讓我們能簡潔地表達此邏輯，並讓我們能只用一行就能實作出來。如果我們對 Option 無法使用 ? 運算子的話，我們使用更多方法呼叫或 match 表達式才能實作此邏輯。

請注意你可以在有回傳 Result 的函式對 Result 的值使用 ? 運算子，你可以在有回傳 Option 的函式對 Option 的值使用 ? 運算子，但你無法混合使用。? 運算子無法自動轉換 Result 與 Option 之間的值。在這種狀況下會需要顯式表達，Result 的話有提供 ok 方法，Option 的話有提供 ok_or 方法。

目前為止，所有我們使用過的 main 函式都是回傳 ()。main 是個特別的函式，因為它是可執行程式的進入點與出口點，而要讓程式可預期執行的話，它的回傳型別就得要有些限制。

幸運的是 main 也可以回傳 Result<(), E>。範例 9-12 取自範例 9-10，不過我們更改 main 的回傳型別為 Result<(), Box<dyn Error>>，並在結尾的回傳數值加上 Ok(())。這樣的程式碼是能編譯的：

```
use std::error::Error;
use std::fs::File;

fn main() -> Result<(), Box<dyn Error>> {
    let greeting_file = File::open("hello.txt");

    Ok(())
}
```

範例 9-12：將 main 改成回傳 Result<(), E> 就能允許在 Result 數值上使用 ? 運算子

Box<dyn Error> 型別使用了特徵物件 (trait object) 我們會在第十七章的「允許不同型別數值的特徵物件」討論到。現在你可以將 Box<dyn Error> 視為它是「任何種類的錯誤」。在有 Box<dyn Error> 錯誤型別的 main 函式中的 Result 使用 ? 是允許的，因為現在 Err 數值可

以被提早回傳。儘管此 `main` 函式本來只會回傳錯誤型別 `std::io::Error`，但有了 `Box<dyn Error>` 的話，此簽名就能允許其他錯誤型別加入 `main` 本體中。

當 `main` 函式回傳 `Result<(), E>` 時，如果 `main` 回傳 `Ok(())` 的話，執行檔就會用 `0` 退出；如果 `main` 回傳 `Err` 數值的話，就會用非零數值退出。用 C 語言寫的執行檔在退出時會回傳整數：程式成功退出的話會回傳整數 `0`，而程式退出錯誤的話則會回傳不是 `0` 的其他整數。而 Rust 執行檔也遵循相容這項規則。

`main` 函式可以回傳任何有實作 `std::process::Termination` 特徵的型別，該特徵包含了一個函式 `report` 來回傳 `ExitCode`。你可以查閱標準函式庫技術文件來了解如何對你的型別實作 `Termination` 特徵。

現在我們已經討論了呼叫 `panic!` 與回傳 `Result` 的細節。現在讓我們回到何時該使用何種辦法的主題上吧。

要 panic! 還是不要 panic!

所以你該如何決定何時要呼叫 `panic!` 還是要回傳 `Result` 呢？當程式碼恐慌時，就沒有任何恢復的方式。你可以在任何錯誤場合呼叫 `panic!`，無論是可能或不可能復原的情況。不過這樣你就等於替呼叫你的程式碼的呼叫者做出決定，讓情況變成無法復原的錯誤了。當你選擇回傳 `Result` 數值，你將決定權交給呼叫者的程式碼。呼叫者可能會選擇符合當下場合的方式嘗試復原錯誤，或者它可以選擇 `Err` 內的數值是不可恢復的，所以它就呼叫 `panic!` 讓你原本可恢復的錯誤轉成不可恢復。因此，當你定義可能失敗的函式時預設回傳 `Result` 是不錯的選擇。

在像是範例、草稿與測試的情況下，程式碼恐慌會比回傳 `Result` 來得恰當。讓我們來探討為何比較好。然後我們再來討論編譯器無法辨別出不可能失敗，但身為人類的你卻可以的情況。本章節會總結一些通用指導原則來決定何時在函式庫程式碼中恐慌。

範例、程式碼原型與測試

當你在寫解釋一些概念的範例時，寫出完善錯誤處理的範例，反而會讓範例變得較不清楚。在範例中，使用像是 `unwrap` 這樣會恐慌的方法可以被視為是一種要求使用者自行決定如何處理錯誤的表現，因為他們可以依據程式碼執行方式來修改此方法。

同樣地 `unwrap` 與 `expect` 方法也很適用在試做原型，你可以在決定準備開始處理錯誤前使用它們。它們會留下清楚的痕跡，當你準備好要讓程式碼更穩固時，你就能回來修改。

如果有方法在測試內失敗時，你會希望整個測試都失敗，就算該方法不是要測試的功能。因為 `panic!` 會將測試標記為失敗，所以在此呼叫 `unwrap` 或 `expect` 是很正確的。

當你知道的比編譯器還多的時候

如果你知道一些編譯器不知道的邏輯的話，直接在 `Result` 呼叫 `unwrap` 或 `expect` 來直接取得 `Ok` 的數值是很有用的。你還是會有個 `Result` 數值需要做處理，你呼叫的程式碼還是有機會失敗的，就算在你的特定場合中邏輯上是不可能的。如果你能保證在親自審閱程式碼後，你絕對不可能會有 `Err` 變體的話，那麼呼叫 `unwrap` 是完全可以接受的。而更好的話，用 `expect` 說明為何你一定不會遇到 `Err` 變體。以下範例就是如此：

```
use std::net::IpAddr;

let home: IpAddr = "127.0.0.1"
    .parse()
    .expect("寫死的 IP 位址應該要有效");
```


我們傳遞寫死的字串來建立 `IpAddr` 的實例。我們可以看出 `127.0.0.1` 是完全合理的 IP 位址，所以這邊我們可以直接 `expect`。不過使用寫死的合理字串並不會改變 `parse` 方法的回傳型別，我們還是會取得 `Result` 數值，編譯器仍然會要我們處理 `Result` 並認為 `Err` 變體是有可能發生的。因為編譯器並沒有聰明到可以看出此字串是個有效的 IP 位址。如果 IP 位址的字串是來自使用者輸入而非我們寫死進程式的話，它的確有可能會失敗，這時我們就得要認真處理 `Result` 了。註明該 IP 位址是寫死的能在未來我們想拿掉 `expect` 並改善錯誤處理時，幫助我們理解需要如何從其他來源處理 IP 位址。

錯誤處理的指導原則

當你的程式碼可能會導致嚴重狀態的話，就建議讓你的程式恐慌。這裡的嚴重狀態是指一些假設、保證、協議或不可變性被打破時的狀態，像是當你的程式碼有無效的數值、互相矛盾的數值或缺少數值。另外還加上以下情形：

- 該嚴重狀態並非預期會發生的，而不是像使用者輸入了錯誤格式這種偶而可能會發生的。
- 你的程式在此時需要避免這種嚴重狀態，而不是在每一步都處理此問題。
- 你所使用的型別沒有適合的方式能夠處理此嚴重狀態。我們會在第十七章的「[定義狀態與行為成型別](#)」段落用範例解釋我們指的是什麼。

如果有人呼叫了你的程式碼卻傳遞了不合理的數值，如果可以的話最好的辦法是回傳個錯誤，這樣函式庫的使用者可以決定在該情況下該如何處理。不過要是繼續執行下去可能會造成危險或不安全的話，最好的辦法是呼叫 `panic!` 並警告使用函式庫的人他們程式碼錯誤發生的位置，好讓他們在開發時就能修正。同樣地，`panic!` 也適合用於如果你呼叫了你無法掌控的外部程式碼，然後它回傳了你無法修正的無效狀態。

不過如果失敗是可預期的，回傳 `Result` 就會比呼叫 `panic!` 來得好。類似的例子有，語法分析器（parser）收到格式錯誤的資訊，或是 HTTP 請求回傳了一個狀態，告訴你已經達到請求上限了。在這樣的案例，回傳 `Result` 代表失敗是預期有時會發生的，而且呼叫者必須決定如何處理。

當你的程式碼可能會因為進行運算時輸入無效數值，而造成使用者安危的話，你的程式需要先驗證該數值，如果數值無效的話就要恐慌。這是基於安全原則，嘗試對無效資料做運算的話可能會導致你的程式碼產生漏洞。這也是標準函式庫在你嘗試取得超出界限的記憶體存取會呼叫 `panic!` 的主要原因。嘗試取得不屬於當前資料結構的記憶體是常見的安全問題。函式通常都會訂下一些合約（**contracts**），它們的行為只有在輸入資料符合特定要求時才帶有保障。當違反合約時恐慌是十分合理的，因為違反合約就代表這是呼叫者的錯誤，這不是你的程式碼該主動處理的錯誤。事實上，呼叫者也沒有任何合理的理由來復原這樣的錯誤。函式的合約應該要寫在函式的技術文件中解釋，尤其是違反時會恐慌的情況。

然而要在你的函式寫一大堆錯誤檢查有時是很冗長且麻煩的。幸運的是你可以利用 Rust 的型別系統（以及編譯器的型別檢查）來幫你完成檢驗。如果你的函式用特定型別作為參數的話，你就可以認定你的程式邏輯是編輯器已經幫你確保你拿到的數值是有效的。舉例來說，如果你有一個型別而非 `Option` 的話，你的程式就會預期取得某個值而不是沒拿到值。你的程式就不必處理 `Some` 和 `None` 這兩個變體情形，它只會有一種情況並絕對會拿到數值。要是有人沒有傳遞任何值給你的函

式會根本無法編譯，所以你的函式就不需要在執行時做檢查。另一個例子是使用非帶號整數像是 `u32` 來確保參數不會是負數。

建立自訂型別來驗證

讓我們來試著使用 Rust 的型別系統來進一步確保我們擁有有效數值，並建立自訂型別來驗證。回想一下第二章的猜謎遊戲，我們的程式碼要使用者從 1 到 100 之間猜一個數字。在開始與祕密數字做比較之前，我們從未驗證使用者輸入的值，我們只驗證了它是否為正的。在這種情況帶來的後果還不算嚴重：我們還是會顯示「太大」或「太小」。但是我們可以改善這段來引導使用者輸入有效數值，並在使用者輸入時猜了超出範圍的數字或字母時呈現不同行為。

我們可以將輸入的猜測分析改成 `i32` 而非 `u32` 來允許負數，並檢查數字是否在範圍內，如以下所示：

```
loop {
    // --省略--

    let guess: i32 = match guess.trim().parse() {
        Ok(num) => num,
        Err(_) => continue,
    };

    if guess < 1 || guess > 100 {
        println!("祕密數字介於 1 到 100 之間。");
        continue;
    }

    match guess.cmp(&secret_number) {
        // --省略--
    }
}
```

`if` 表達式檢查我們的數值是否超出範圍，如果是的話就告訴使用者問題原因，並呼叫 `continue` 來進行下一次的猜測循環，要求再猜一次。在 `if` 表達式之後我們就能用已經知道範圍是在 1 到 100 的 `guess` 與祕密數字做比較。

不過這並非理想解決方案：如果程式必定要求數值一定要是 1 到 100，而且我們有很多函式都有此需求的話，在每個函式都檢查就太囉唆了（而且可能會影響效能）。

對此我們可以建立一個新的型別，並且建立一個驗證產生實例的函式，這樣我們就不必在每個地方都做驗證。這樣一來函式就可以安全地以這個新型別作為簽名，並放心地使用收到的數值。範例 9-13 顯示了定義 `Guess` 型別的例子，它的 `new` 函式只會在接收值為 1 到 100 時才會建立 `Guess` 實例。


```

pub struct Guess {
    value: i32,
}

impl Guess {
    pub fn new(value: i32) -> Guess {
        if value < 1 || value > 100 {
            panic!("猜測數字必須介於 1 到 100 之間，你輸入的是 {}。", value);
        }

        Guess { value }
    }

    pub fn value(&self) -> i32 {
        self.value
    }
}

```

範例 9-13：只會擁有 1 到 100 的 `Guess` 型別

首先我們定義了一個結構體叫做 `Guess`，其欄位叫做 `value` 並會持有 `i32`。這就是數字會被儲存的地方。

接著我們實作一個 `Guess` 的關聯函式叫做 `new` 來建立 `Guess` 的值。`new` 函式定義的參數叫做 `value` 並擁有型別 `i32`，且最後會回傳 `Guess`。函式 `new` 本體中的程式碼會驗證 `value` 確保它位於 1 到 100 之間。如果 `value` 沒有通過驗證，我們呼叫 `panic!` 來警告呼叫此程式碼的開發者，他們可能有需要修正的程式錯誤，因為使用超出範圍的 `value` 來建立 `Guess` 違反了 `Guess::new` 的合約。`Guess::new` 會恐慌的情況需要在公開的 API 技術文件中提及。我們會在第十四章討論如何寫出技術文件並在 API 技術文件中指出可能發生 `panic!` 的情形。如果 `value` 通過驗證的話，我們就建立一個新的 `Guess` 並將參數 `value` 賦值給 `value` 欄位，最後回傳 `Guess`。

接著我們實作了個方法叫做 `value`，它會借用 `self` 且沒有任何參數，並會回傳 `i32`。這種方法有時會被稱為 *getter*，因為它的目的是從它的欄位中取得一些資料並回傳它。此公開方法是必要的，因為 `Guess` 結構體中的 `value` 欄位是私有的。將 `Guess` 結構體的 `value` 欄位設為私有是很重要的，這樣就無法直接設置 `value`，模組外的程式碼必須使用 `Guess::new` 函式來建立 `Guess` 的實例，因而確保 `Guess` 不可能會有沒有經過 `Guess::new` 函式驗證的 `value`。

這樣當函式的參數或回傳值只能是數字 1 到 100 的話，它的簽名就能使用或回傳 `Guess` 而不是 `i32`，因此就不必在它的本體內做任何額外檢查。

總結

Rust 的錯誤檢查功能的設計旨在協助你寫出可靠的程式碼。 `panic!` 巨集告訴你的程式遇到了它無法處理的狀態，並讓你告訴程序停止，而不是繼續嘗試使用無效或不正確的數值。 `Result` 列舉使用 Rust 的型別系統來指出可能會失敗的運算，並讓你的程式碼有辦法恢復。你可以使用 `Result` 來告訴使用你的程式碼的呼叫者，他們需要處理可能成功與失敗的情形。在適當的場合使用 `panic!` 與 `Result` 能讓你的程式碼在不可避免的問題中更加可靠。

現在你已經看過標準函式庫中 `Option` 與 `Result` 使用泛型的優勢了，就讓我們來討論泛型如何運作的，以及你如何在程式碼中使用它們。

泛型型別、特徵與生命週期

每個程式語言都有能夠高效處理概念複製的工具。在 Rust 此工具就是**泛型 (generics)**：實際型別或其他屬性的抽象替代。我們可以表達泛型的行為，或是它們與其他泛型有何關聯，而不必在編譯與執行程式時知道它們實際上是什麼。

函式也可以接受一些泛型型別參數，而不是實際型別像是 `i32` 或 `String`，就像函式有辦法能接收多種未知數值作為參數來執行相同程式碼。事實上我們已經在第六章的 `Option<T>`、第八章的 `Vec<T>` 和 `HashMap<K, V>` 以及第九章的 `Result<T, E>` 使用過泛型了。在本章節，你將會探索如何用泛型定義你自己的型別、函式與方法！

首先我們會先檢視如何提取參數來減少重複的程式碼。接著我們會以相同的技巧使用泛型將兩個只有參數型別不同的函式轉變成泛型函式。我們還會解釋如何在結構體和列舉使用泛型型別。

再來你會學會如何使用**特徵 (traits)** 來定義共同行為。你可以組合特徵與泛型型別來限制泛型型別只適用在有特定行為的型別，而不是任意型別。

最後我們會來介紹**生命週期 (lifetimes)**：一種能讓編譯器知道參考如何互相關聯的泛型。生命週期讓我們能提供給編譯器更多關於借用數值的資訊，好讓它在更多情況下可以確保參考是有效的。

提取函數來減少重複性

泛型讓我們可以用佔位符 (placeholder) 替代特定型別，來表示多重型別並減少程式碼的重複性。在我們深入泛型語法之前，讓我們先來看如何不用泛型型別的情況下，用提取函式的方式減少重複的程式碼。之後我們就會用相同的方式來提取泛型函式！和你透過找出重複的程式碼來提取程式一樣，你也將找出重複的函式來轉成泛型。

我們先從範例 10-1 中一支尋找列表中最大數字的小程式開始。

檔案名稱：src/main.rs

```
fn main() {  
    let number_list = vec![34, 50, 25, 100, 65];  
  
    let mut largest = &number_list[0];  
  
    for number in &number_list {  
        if number > largest {  
            largest = number;  
        }  
    }  
  
    println!("最大數字為 {}", largest);  
}
```

範例 10-1：在數字列表中尋找最大數字的程式碼

我們儲存整數列表到變數 `number_list` 並將列表第一個數字的參考放入變數 `largest`。接著我們遍歷列表中的所有元素，如果目前數字比 `largest` 內儲存的數字還大的話，就會替代成該變數的參考。不過如果目前數值小於或等於最大值的話，變數就不會被改變，程式會接續檢查列表中的下一個數字。在考慮完列表中的所有數字後，`largest` 就應該會指向最大數字，在此例就是 100。

現在我們要從兩個不同的數字列表中找到最大值，我們可以重複範例 10-1 的程式碼，然後在程式中兩個不同的地方使用相同的邏輯，如範例 10-2 所示。

檔案名稱：src/main.rs

```

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let mut largest = &number_list[0];

    for number in &number_list {
        if number > largest {
            largest = number;
        }
    }

    println!("最大數字為 {}", largest);

    let number_list = vec![102, 34, 6000, 89, 54, 2, 43, 8];

    let mut largest = &number_list[0];

    for number in &number_list {
        if number > largest {
            largest = number;
        }
    }

    println!("最大數字為 {}", largest);
}

```

範例 10-2：在兩個數字列表中尋找最大值

雖然這樣的程式碼能執行，寫出重複的程式碼很囉唆而且容易出錯。我們還得記住每次更新時就得一起更新各個地方。

要去除重複的部分，我們可以建立一層抽象，定義一個可以處理任意整數列表作為參數的函式。這樣的解決辦法讓我們的程式更清晰，而且讓我們能抽象表達出從列表中尋找最大值這樣的概念。

在範例 10-3 我們提取了尋找最大值的程式碼成一個函式叫做 `largest`。然後我們呼叫函式來尋找範例 10-2 兩個列表中最大的數字。我們還可以在未來對其他任何 `i32` 的列表使用此函式。

檔案名稱：src/main.rs

```

fn largest(list: &[i32]) -> &i32 {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let result = largest(&number_list);
    println!("最大數字為 {}", result);

    let number_list = vec![102, 34, 6000, 89, 54, 2, 43, 8];

    let result = largest(&number_list);
    println!("最大數字為 {}", result);
}

```

範例 10-3：抽象出尋找最大值的概念並用在兩個不同的列表

`largest` 函式有個參數 `list` 可以代表我們傳遞給函式的 `i32` 型別切片。所以當我們呼叫此函式時，程式可以依據我們傳入的特定數值執行。

總結來說，以下是我們將範例 10-2 的程式碼轉換成範例 10-3 的步驟：

1. 找出重複的程式碼。
2. 將重複的程式碼提取置函式本體內，並指定函式簽名輸入與回傳數值。
3. 更新重複使用程式碼的實例，改呼叫我們定義的函式。

接著我們將以相同的步驟使用泛型來減少重複的程式碼。就像函式本體可以抽象出 `list` 而不用特定數值，泛型允許程式碼執行抽象型別。

舉例來說，假設我們有兩個函式：一個會找出 `i32` 型別切片中的最大值而另一個會找出 `char` 型別切片的最大值。我們要如何刪除重複的部分呢？讓我們拭目以待！

泛型資料型別

我們使用泛型（generics）來建立項目的定義，像是函式簽名或結構體，讓我們之後可以使用在不同的實際資料型別。讓我們先看看如何使用泛型定義函式、列舉與方法。然後我們會再來看泛型對程式碼的效能影響如何。

在函式中定義

當要使用泛型定義函數時，我們通常會將泛型置於函式簽名中指定參數與回傳值資料型別的位置。這樣做能讓我們的程式碼更具彈性並向呼叫者提供更多功能，同時還能防止重複程式碼。

接續我們 `largest` 函式的例子，範例 10-4 展示了兩個都在切片上尋找最大值的函式。我們要用泛型將它們融合成一個函式。

檔案名稱：src/main.rs

```

fn largest_i32(list: &[i32]) -> &i32 {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn largest_char(list: &[char]) -> &char {
    let mut largest = &list[0];

    for item in list {
        if item > largest {
            largest = item;
        }
    }

    largest
}

fn main() {
    let number_list = vec![34, 50, 25, 100, 65];

    let result = largest_i32(&number_list);
    println!("最大數字為 {}", result);

    let char_list = vec!['y', 'm', 'a', 'q'];

    let result = largest_char(&char_list);
    println!("最大字元為 {}", result);
}

```

範例 10-4：兩個名稱與其簽名中的型別都不同的函式

`largest_i32` 函式和我們在範例 10-3 提取的函式一樣都是尋找切片中最大的 `i32`。而 `largest_char` 函式則尋找切片中最大的 `char`。函式本體都擁有相同的程式碼，讓我們來開始用泛型型別參數來消除重複的部分，轉變成只有一個函式吧。

要在新定義的函式中參數化型別的話，我們需要為參數型別命名，就和我們在函式中的參數數值所做的一樣。你可以用任何標識符來命名型別參數名稱。但我們習慣上會用 `T`，因為 Rust 的型別參數名稱都盡量很短，常常只會有一個字母，而且 Rust 對於型別命名的慣用規則是駝峰式大小寫 (CamelCase)。所以 `T` 作為「type」的簡稱是大多數 Rust 程式設計師的選擇。

當我們在函式本體使用參數時，我們必須在簽名中宣告參數名稱，編譯器才能知道該名稱代表什麼。同樣地，當我們要在函式簽名中使用型別參數名稱，我們必須在使用前宣告該型別參數名稱。要定義泛型 `largest` 函式的話，我們在函式名稱與參數列表之間加上尖括號，其內就是型別名稱的宣告，如以下所示：

```
fn largest<T>(list: &[amp;T]) -> &T {
```

我們可以這樣理解定義：函式 `largest` 有泛型型別 `T`，此函式有一個參數叫做 `list`，它的型別為數值 `T` 的切片。`largest` 函式會回傳與型別 `T` 相同型別的參考數值。

範例 10-5 顯示了使用泛型資料型別於函式簽名組合出的 `largest` 函式。此範例還展示了我們如何依序用 `i32` 和 `char` 的切片呼叫函式。注意此程式碼尚未能編譯，不過我們會在本章之後修改它。

檔案名稱：src/main.rs

```
fn largest<T>(list: &[amp;T]) -> &T {  
    let mut largest = &list[0];  
  
    for item in list {  
        if item > largest {  
            largest = item;  
        }  
    }  
  
    largest  
}  
  
fn main() {  
    let number_list = vec![34, 50, 25, 100, 65];  
  
    let result = largest(&number_list);  
    println!("最大數字為 {}", result);  
  
    let char_list = vec!['y', 'm', 'a', 'q'];  
  
    let result = largest(&char_list);  
    println!("最大字元為 {}", result);  
}
```



範例 10-5：使用泛型型別參數的 `largest` 函式，但現在還不能編譯

如果我們現在就編譯程式碼的話，我們會得到此錯誤：

```

$ cargo run
   Compiling chapter10 v0.1.0 (file:///projects/chapter10)
error[E0369]: binary operation `>` cannot be applied to type `&T`
  --> src/main.rs:5:17
   |
5  |         if item > largest {
   |             ---- ^ ----- &T
   |             |
   |             &T
   |
help: consider restricting type parameter `T`
   |
1  | fn largest<T: std::cmp::PartialOrd>(list: &[T]) -> &T {
   |               ++++++

```

For more information about this error, try `rustc --explain E0369`.

error: could not compile `chapter10` due to previous error

提示文字中提到了 `std::cmp::PartialOrd` 這個**特徵 (trait)**。我們會在下一個段落來討論特徵。現在只需要知道 `largest` 本體無法適用於所有可能的 `T` 型別，因為我們想要在本體中比較型別 `T` 的數值，我們只能在能夠排序的型別中做比較。要能夠比較的話，標準函式庫有提供 `std::cmp::PartialOrd` 特徵讓你可以針對你的型別來實作（請查閱附錄 C 來瞭解更多此特徵的細節）。照著提示文字的建議，我們限制 `T` 只對有實作 `PartialOrd` 的型別有效。這樣此範例就能編譯，因為標準函式庫有對 `i32` 與 `char` 實作 `PartialOrd`。

在結構體中定義

我們一樣能以 `<>` 語法來對結構體中一或多個欄位使用泛型型別參數。範例 10-6 展示了定義 `Point<T>` 結構體並讓 `x` 與 `y` 可以是任意型別數值。

檔案名稱：src/main.rs

```

struct Point<T> {
    x: T,
    y: T,
}

fn main() {
    let integer = Point { x: 5, y: 10 };
    let float = Point { x: 1.0, y: 4.0 };
}

```

範例 10-6： `Point<T>` 結構體的 `x` 與 `y` 會有型別 `T` 的數值

在結構體定義使用泛型的語法與函式定義類似。首先，我們在結構體名稱後方加上尖括號，並在其內宣告型別參數名稱。接著我們能在原本指定實際資料型別的地方，使用泛型型別來定義結構體。

注意到我們使用了一個泛型型別來定義 `Point<T>`，此定義代表 `Point<T>` 是某型別 `T` 下之通用的，而且欄位 `x` 與 `y` 擁有相同型別，無論最終是何種型別。如果我們用不同的型別數值來建立 `Point<T>` 實例，我們的程式碼會無法編譯，如範例 10-7 所示。

檔案名稱：src/main.rs

```
struct Point<T> {  
    x: T,  
    y: T,  
}  
  
fn main() {  
    let wont_work = Point { x: 5, y: 4.0 };  
}
```



範例 10-7：欄位 `x` 與 `y` 必須是相同型別，因為它們擁有相同的泛型資料型別 `T`

在此例中，當我們賦值 5 給 `x` 時，我們讓編譯器知道 `Point<T>` 實例中的泛型型別 `T` 會是整數。然後我們將 4.0 賦值給 `y`，這應該要和 `x` 有相同型別，所以我們會獲得以下錯誤：

```
$ cargo run  
    Compiling chapter10 v0.1.0 (file:///projects/chapter10)  
error[E0308]: mismatched types  
--> src/main.rs:7:38  
   |  
7 |     let wont_work = Point { x: 5, y: 4.0 };  
   |                                   ^^^ expected integer, found floating-  
point number  
  
For more information about this error, try `rustc --explain E0308`.  
error: could not compile `chapter10` due to previous error
```

要將結構體 `Point` 的 `x` 與 `y` 定義成擁有不同型別卻仍然是泛型的話，我們可以使用多個泛型型別參數。舉例來說，在範例 10-8 我們改變了 `Point` 的定義為擁有兩個泛型型別 `T` 與 `U`，`x` 擁有型別 `T` 而 `y` 擁有型別 `U`。

檔案名稱：src/main.rs

```

struct Point<T, U> {
    x: T,
    y: U,
}

fn main() {
    let both_integer = Point { x: 5, y: 10 };
    let both_float = Point { x: 1.0, y: 4.0 };
    let integer_and_float = Point { x: 5, y: 4.0 };
}

```

範例 10-8：Point<T, U> 擁有兩個泛型型別，所以 x 和 y 可以有不同的型別數值

現在這些所有的 Point 實例都是允許的了！你要在定義中使用多少泛型型別參數都沒問題，但用太多的話會讓你的程式碼難以閱讀。如果你發現你的程式碼需要使用大量泛型的話，這通常代表你的程式碼需要重新組織成更小的元件。

在列舉中定義

如同結構體一樣，我們可以定義列舉讓它們的變體擁有泛型資料型別。讓我們看看我們在第六章標準函式庫提供的 Option<T> 列舉：

```

enum Option<T> {
    Some(T),
    None,
}

```

此定義現在對你來說應該就說得通了。如同你所看到的 Option<T> 列舉有個泛型型別參數 T 以及兩個變體：Some 擁有型別 T 的數值；而 None 則是不具任何數值的變體。使用 Option<T> 列舉我們可以表達出一個可能擁有的數值這樣的抽象概念。而且因為 Option<T> 是泛型，不管可能的數值型別為何，我們都能使用此抽象。

列舉也能有數個泛型型別。我們在第九章所使用列舉 Result 的定義就是個例子：

```

enum Result<T, E> {
    Ok(T),
    Err(E),
}

```

Result 列舉有兩個泛型型別 T 和 E 且有兩個變體：Ok 擁有型別 T 的數值；而 Err 擁有型別 E 的數值。這樣的定義讓我們很方便能表達 Result 列舉可能擁有一個成功的數值（回傳型別 T 的數值）或失敗的數值（回傳型別為 E 的錯誤值）。事實上這就是我們在範例 9-3 開啟檔案的方式，當我們成功開啟檔案時的 T 就會是型別 std::fs::File，然後當開啟檔案會發生問題時 E 就會是型別 std::io::Error。

當你發現你的程式碼有許多結構體或列舉都只有儲存的值有所不同時，你可以使用泛型型別來避免重複。

在方法中定義

我們可以對結構體或列舉定義方法（如第五章所述）並也可以使用泛型型別來定義。範例 10-9 展示了我們在範例 10-6 定義的結構體 `Point<T>` 並實作了一個叫做 `x` 的方法。

檔案名稱：src/main.rs

```
struct Point<T> {
    x: T,
    y: T,
}

impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

fn main() {
    let p = Point { x: 5, y: 10 };

    println!("p.x = {}", p.x());
}
```

範例 10-9：在 `Point<T>` 結構體實作一個方法叫做 `x`，其會回傳 `x` 欄位中型別為 `T` 的參考

我們在這 `Point<T>` 定義了一個方法叫做 `x` 並回傳欄位 `x` 的資料參考。

注意到我們需要在 `impl` 宣告 `T`，才有 `T` 可以用來標明我們在替型別 `Point<T>` 實作其方法。在 `impl` 之後宣告泛型型別 `T`，Rust 可以識別出 `Point` 尖括號內的型別為泛型型別而非實際型別。我們其實可以選用不同的泛型參數名稱，而不用和結構體定義的泛型參數一樣，不過通常使用相同名稱還是比較常見。無論該泛型型別最終會是何種實際型別，任何方法在有宣告泛型型別的 `impl` 內，都會被定義成適用於各種型別實例。

當我們在定義方法時，我們也可以對泛型型別加上些限制。舉例來說，我們可以只針對 `Point<f32>` 的實例來實作方法，而非適用於任何泛型型別的 `Point<T>` 實例。在範例 10-10 我們使用了實例型別 `f32` 而沒有在 `impl` 宣告任何型別。

檔案名稱：src/main.rs

```
impl Point<f32> {
    fn distance_from_origin(&self) -> f32 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}
```

範例 10-10：一個只適用於擁有泛型 `T` 結構體其中的特定實際型別的 `impl` 區塊

此程式碼代表 `Point<f32>` 會有個方法 `distance_from_origin`，其他 `Point<T>` 只要 `T` 不是型別 `f32` 的實例都不會定義此方法。此方法測量我們的點距離座標 (0.0, 0.0) 有多遠並使用只有浮點數型別能使用的數學運算。

在結構體定義中的泛型型別參數不會總是和結構體方法簽名中的相同。舉例來說，範例 10-11 在 `Point` 結構體中使用泛型型別 `X1` 和 `Y1`，但在 `mixup` 方法中就使用 `X2` `Y2` 以便清楚辨別。該方法用 `self` `Point` 的 `x` 值（型別為 `X1`）與參數傳進來的 `Point` 的 `y` 值（型別為 `Y2`）來建立新的 `Point` 實例。

檔案名稱：src/main.rs

```
struct Point<X1, Y1> {
    x: X1,
    y: Y1,
}

impl<X1, Y1> Point<X1, Y1> {
    fn mixup<X2, Y2>(&self, other: Point<X2, Y2>) -> Point<X1, Y2> {
        Point {
            x: self.x,
            y: other.y,
        }
    }
}

fn main() {
    let p1 = Point { x: 5, y: 10.4 };
    let p2 = Point { x: "Hello", y: 'c' };

    let p3 = p1.mixup(p2);

    println!("p3.x = {}, p3.y = {}", p3.x, p3.y);
}
```

範例 10-11：結構體定義中使用不同的泛型型別的方法

在 `main` 中，我們定義了一個 `Point`，其 `x` 型別為 `i32`（數值為 5），`y` 型別為 `f64`（數值為 10.4）。變數 `p2` 是個 `Point` 結構體，`x` 為字串切片（數值為 "Hello"），`y` 為 `char`（數值為 c）。在 `p1` 呼叫 `mixup` 並加上引數 `p2` 的話會給我們 `p3`，它的 `x` 會有型別 `i32`，因為 `x`

來自 `p1`。而且變數 `p3` 還會有型別為 `char` 的 `y`，因為 `y` 來自 `p2`。 `println!` 巨集的呼叫就會顯示 `p3.x = 5`, `p3.y = c`。

此例是是為了展示一些泛型參數是透過 `impl` 宣告而有些則是透過方法定義來取得。泛型參數 `x1` 和 `y1` 是宣告在 `impl` 之後，因為它們與結構體定義有關聯。泛型參數 `x2` 和 `y2` 則是宣告在 `fn mixup` 之後，因為它們只與方法定義有關聯。

使用泛型的程式碼效能

你可能會好奇當你使用泛型型別參數會不會有執行時的消耗。好消息是使用泛型型別不會比使用實際型別還來的慢。

Rust 在編譯時對使用泛型的程式碼進行單型化 (monomorphization)。**單型化**是個讓泛型程式碼轉換成特定程式碼的過程，在編譯時填入實際的型別。在此過程中，編譯器會做與我們在範例 10-5 建立泛型函式相反的事：編譯器檢查所有泛型程式碼被呼叫的地方，並依據泛型程式碼被呼叫的情況產生實際型別的程式碼。

讓我們看看這在標準函式庫的泛型列舉 `Option<T>` 中是怎麼做到的：

```
let integer = Some(5);
let float = Some(5.0);
```

當 Rust 編譯此程式碼時中，他會進行單型化。在此過程中，會讀取 `Option<T>` 實例中使用的數值並識別出兩種 `Option<T>`：一種是 `i32` 而另一種是 `f64`。接著它就會將 `Option<T>` 的泛型定義展開為兩種定義 `i32` 與 `f64`，以此替換函式定義為特定型別。

單型化的版本看起來會像這樣（編譯器實際使用的名稱會和我們這邊示範的不同）：

檔案名稱：src/main.rs

```
enum Option_i32 {
    Some(i32),
    None,
}

enum Option_f64 {
    Some(f64),
    None,
}

fn main() {
    let integer = Option_i32::Some(5);
    let float = Option_f64::Some(5.0);
}
```

泛型 `Option<T>` 會被替換成編譯器定義的特定定義。因為 Rust 會編譯泛型程式碼成個別實例的特定型別，我們使用泛型就不會造成任何執行時消耗。當程式執行時，它就會和我們親自寫重複定義的版本一樣。單型化的過程讓 Rust 的泛型在執行時十分有效率。

特徵：定義共同行為

特徵 (trait) 會定義特定型別與其他型別共享的功能。我們可以使用特徵定義來抽象出共同行為。我們可以使用**特徵界限** (trait bounds) 來指定泛型型別為擁有特定行為的任意型別。

注意：特徵類似於其他語言常稱作**介面** (interfaces) 的功能，但還是有些差異。

定義特徵

一個型別的行為包含我們對該型別可以呼叫的方法。如果我們可以對不同型別呼叫相同的方法，這些型別就能定義共同行為了。特徵定義是一個將方法簽名統整起來，來達成一些目的而定義一系列行為的方法。

舉例來說，如果我們有數個結構體各自擁有不同種類與不同數量的文字：結構體 `NewsArticle` 儲存特定地點的新聞故事，然後 `Tweet` 則有最多 280 字元的內容，且有個欄位來判斷是全新的推文、轉推或其他推文的回覆。

我們想要建立一個多媒體聚集器函式庫 crate 叫 `aggregator` 來顯示可能存在 `NewsArticle` 或 `Tweet` 實例的資料總結。要達成此目的的話，我們需要每個型別的總結，且我們會呼叫該實例的 `summarize` 方法來索取總結。範例 10-12 顯示了表達此行為的 `Summary` 特徵定義。

檔案名稱：src/lib.rs

```
pub trait Summary {  
    fn summarize(&self) -> String;  
}
```

範例 10-12：`Summary` 特徵包含 `summarize` 方法所定義的行為

我們在此使用 `trait` 關鍵字定義一個特徵，其名為 `Summary`。我們也將特徵宣告成 `pub` 所以其他會依賴此函式庫的 crate 也能用到此特徵，我們之後會再看到其他範例。在大括號中，我們宣告方法簽名來描述有實作此特徵的型別行為，在此例就是 `fn summarize(&self) -> String`。

在方法簽名之後，我們並沒有加上大括號提供實作細節，而是使用分號。每個有實作此特徵的型別必須提供其自訂行為的方法本體。編譯器會強制要求任何有 `Summary` 特徵的型別都要有定義相同簽名的 `summarize` 方法。

特徵本體中可以有多個方法，每行會有一個方法簽名並都以分號做結尾。

為型別實作特徵

現在我們已經用 `Summary` 特徵定義了所需的方法簽名。我們可以在我們多媒體聚集器的型別中實作它。範例 10-13 顯示了 `NewsArticle` 結構體實作 `Summary` 特徵的方式，其使用頭條、作者、位置來建立 `summarize` 的回傳值。至於結構體 `Tweet`，我們使用使用者名稱加上整個推文的文字來定義 `summarize`，因為推文的內容長度已經被限制在 280 個字元以內了。

檔案名稱：src/lib.rs

```
pub struct NewsArticle {
    pub headline: String,
    pub location: String,
    pub author: String,
    pub content: String,
}

impl Summary for NewsArticle {
    fn summarize(&self) -> String {
        format!("{}", {} 著 ({}), self.headline, self.author, self.location)
    }
}

pub struct Tweet {
    pub username: String,
    pub content: String,
    pub reply: bool,
    pub retweet: bool,
}

impl Summary for Tweet {
    fn summarize(&self) -> String {
        format!("{}", {}: {}", self.username, self.content)
    }
}
```

範例 10-13：在型別 `NewsArticle` 與 `Tweet` 實作 `Summary` 特徵

為一個型別實作一個特徵類似於實作一般的方法。不同的地方在於在 `impl` 之後我們加上的是想要實作的特徵，然後再用 `for` 關鍵字加上我們想要實作特徵的型別名稱。在 `impl` 的區塊內我們置入該特徵所定義的方法簽名，我們使用大括號並填入方法本體來為對特定型別實作出特徵方法的指定行為。

現在，我們就能像呼叫正常方法一樣，來呼叫 `NewsArticle` 和 `Tweet` 實例的方法，如以下所示：現在函式庫已經對 `NewsArticle` 和 `Tweet` 實作 `Summary` 特徵了，crate 的使用者能像我們平常呼叫方法那樣，對 `NewsArticle` 和 `Tweet` 的實例呼叫特徵方法。唯一的不同是使用者必須將特徵也加入作用域中。以下的範例展示執行檔 crate 如何使用我們的 `aggregator` 函式庫 crate：

```

use aggregator::{self, Summary, Tweet};

fn main() {
    let tweet = Tweet {
        username: String::from("horse_ebooks"),
        content: String::from(
            "of course, as you probably already know, people",
        ),
        reply: false,
        retweet: false,
    };

    println!("1 則新推文: {}", tweet.summarize());
}

```

此程式碼會印出「1 則新推文：horse_ebooks: of course, as you probably already know, people」。

其他依賴 `aggregator` 函式庫的 crate 也能將 `Summary` 特徵引入作用域並對他們自己的型別實作 `Summary` 特徵。不過實作特徵時有一個限制，那就是我們只能在該特徵或該型別位於我們的 crate 時，才能對型別實作特徵。舉例來說，我們可以對自訂型別像是 `Tweet` 來實作標準函式庫的 `Display` 特徵來為我們 crate `aggregator` 增加更多功能。因為 `Tweet` 位於我們的 `aggregator` crate 裡面。我們也可以在我們的 crate `aggregator` 內對 `Vec<T>` 實作 `Summary`。因為特徵 `Summary` 也位於我們的 `aggregator` crate 裡面。

但是我們無法對外部型別實作外部特徵。舉例來說我們無法在我們的 `aggregator` crate 裡面對 `Vec<T>` 實作 `Display` 特徵。因為 `Display` 與 `Vec<T>` 都定義在標準函式庫中，並沒有在我們 `aggregator` crate 裡面。此限制叫做「連貫性 (coherence)」是程式屬性的一部分。更具體來說我們會稱作「孤兒原則 (orphan rule)」，因為上一代 (parent) 型別不存在。此原則能確保其他人的程式碼不會破壞你的程式碼，反之亦然。沒有此原則的話，兩個 crate 可以都對相同型別實作相同特徵，然後 Rust 就會不知道該用哪個實作。

預設實作

有時候對特徵內的一些或所有方法定義預設行為是很實用的，而不必要求每個型別都實作所有方法。然後當我們對特定型別實作特徵時，我們可以保留或覆蓋每個方法的預設行為。

在範例 10-14 我們在 `Summary` 特徵內指定 `summarize` 方法的預設字串，而不必像範例 10-12 只定義了方法簽名。

檔案名稱：src/lib.rs

```
pub trait Summary {
    fn summarize(&self) -> String {
        String::from("(閱讀更多...)")
    }
}
```

範例 10-14：Summary 特徵定義了 summarize 方法的預設實作

要使用預設實作來總結 NewsArticle 的話，我們可以指定一個空的 impl 區塊，像是 impl Summary for NewsArticle {}。

我們沒有直接對 NewsArticle 定義 summarize 方法，因為我們使用的是預設實作並聲明對 NewsArticle 實作 Summary 特徵。所以最後我們仍然能在 NewsArticle 實例中呼叫 summarize，如以下所示：

```
let article = NewsArticle {
    headline: String::from("Penguins win the Stanley Cup Championship!"),
    location: String::from("Pittsburgh, PA, USA"),
    author: String::from("Iceburgh"),
    content: String::from(
        "The Pittsburgh Penguins once again are the best \
        hockey team in the NHL."),
    },
};

println!("有新文章發佈！{}", article.summarize());
```

此程式碼會印出 有新文章發佈！（閱讀更多...）。

建立預設實作不會影響範例 10-13 中 Tweet 實作的 Summary。因為要取代預設實作的語法，與當沒有預設實作時實作特徵方法的語法是一樣的。

預設實作也能呼叫同特徵中的其他方法，就算那些方法沒有預設實作。這樣一來，特徵就可以提供一堆實用的功能，並要求實作者只需處理一小部分就好。舉例來說，我們可以定義 Summary 特徵，使其擁有一個必須要實作的 summarize_author 方法，以及另一個擁有預設實作會呼叫 summarize_author 的方法：

```
pub trait Summary {
    fn summarize_author(&self) -> String;

    fn summarize(&self) -> String {
        format!("(從 {} 閱讀更多...)", self.summarize_author())
    }
}
```

要使用這個版本的 Summary，我們只需要在對型別實作特徵時定義 summarize_author 就好：

```
impl Summary for Tweet {
    fn summarize_author(&self) -> String {
        format!("@{}", self.username)
    }
}
```

在我們定義 `summarize_author` 之後，我們可以在結構體 `Tweet` 的實例呼叫 `summarize`，然後 `summarize` 的預設實作會呼叫我們提供的 `summarize_author`。因為我們已經定義了 `summarize_author`，且 `Summary` 特徵有提供 `summarize` 方法的預設實作，所以我們不必再寫任何程式碼。

```
let tweet = Tweet {
    username: String::from("horse_ebooks"),
    content: String::from(
        "of course, as you probably already know, people",
    ),
    reply: false,
    retweet: false,
};

println!("1 則新推文: {}", tweet.summarize());
```

此程式碼會印出 `1 則新推文: (從 @horse_ebooks 閱讀更多...)`。

注意要是對相同方法覆寫實作的話，就無法呼叫預設實作。

特徵作為參數

現在你知道如何定義與實作特徵，我們可以來探討如何使用特徵來定義函式來接受多種不同的型別。我們會使用範例 10-13 中 `NewsArticle` 與 `Tweet` 實作的 `Summary` 特徵，來定義一個函式 `notify` 使用它自己的參數 `item` 來呼叫 `summarize` 方法，所以此參數的型別預期有實作 `Summary` 特徵。為此我們可以使用 `impl Trait` 語法，如下所示：

```
pub fn notify(item: &impl Summary) {
    println!("頭條新聞! {}", item.summarize());
}
```

與其在 `item` 參數指定實際型別，我們用的是 `impl` 關鍵字並加上特徵名稱。這樣此參數就會接受任何有實作指定特徵的型別。在 `notify` 本體中我們就可以用 `item` 呼叫 `Summary` 特徵的任何方法，像是 `summarize`。我們可以呼叫 `notify` 並傳遞任何 `NewsArticle` 或 `Tweet` 的實例。但如果用其他型別像是 `String` 或 `i32` 來呼叫此程式碼的話會無法編譯，因為那些型別沒有實作 `Summary`。

特徵界限語法

`impl Trait` 語法看起來很直觀，不過它其實是一個更長格式的語法糖，這個格式稱之為「特徵界限 (trait bound)」，它長得會像這樣：

```
pub fn notify<T: Summary>(item: &T) {  
    println!("頭條新聞! {}", item.summarize());  
}
```

此格式等同於之前段落的範例，只是比較長一點。我們將特徵界限置於泛型型別參數的宣告中，在尖括號內接在冒號之後。

`impl Trait` 語法比較方便，且在簡單的案例中可以讓程式碼比較簡潔；而特徵界限語法則適合用於其他比較複雜的案例。舉例來說我們可以有兩個有實作 `Summary` 的參數，使用 `impl Trait` 語法看起來會像這樣：

```
pub fn notify(item1: &impl Summary, item2: &impl Summary) {
```

如果我們想要此函式允許 `item1` 和 `item2` 是不同型別的話，使用 `impl Trait` 的確是正確的（只要它們都有實作 `Summary`）。不過如果我們希望兩個參數都是同一型別的話，我們就得使用特徵界限來表達，如以下所示：

```
pub fn notify<T: Summary>(item1: &T, item2: &T) {
```

泛型型別 `T` 作為 `item1` 和 `item2` 的參數會限制函式，讓傳遞給 `item1` 和 `item2` 參數的數值型別必須相同。

透過 `+` 來指定多個特徵界限

我們也可以指定不只一個特徵界限。假設我們還想要 `notify` 中的 `item` 不只能夠呼叫 `summarize` 方法，還能顯示格式化訊息的話，我們可以在 `notify` 定義中指定 `item` 必須同時要有 `Display` 和 `Summary`。這可以使用 `+` 語法來達成：

```
pub fn notify(item: &(impl Summary + Display)) {
```

`+` 也能用在泛型型別的特徵界限中：

```
pub fn notify<T: Summary + Display>(item: &T) {
```

有了這兩個特徵界限，`notify` 本體就能呼叫 `summarize` 以及使用 `{}` 來格式化 `item`。

透過 `where` 來使特徵界限更清楚

使用太多特徵界限也會帶來壞處。每個泛型都有自己的特徵界限，所以有數個泛型型別的函式可以在函式名稱與參數列表之間包含大量的特徵界限資訊，讓函式簽名難以閱讀。因此 Rust 有提供另一個在函式簽名之後指定特徵界限的語法 `where`。所以與其這樣寫：

```
fn some_function<T: Display + Clone, U: Clone + Debug>(t: &T, u: &U) -> i32 {
```

我們可以這樣寫 `where` 的語法，如以下所示：

```
fn some_function<T, U>(t: &T, u: &U) -> i32
where
    T: Display + Clone,
    U: Clone + Debug,
{
```

此函式簽名就沒有這麼複雜了，函式名稱、參數列表與回傳型別能靠得比較近，就像沒有一堆特徵界限的函式一樣。

回傳有實作特徵的型別

我們也能在回傳的位置使用 `impl Trait` 語法來回傳某個有實作特徵的型別數值，如以下所示：

```
fn returns_summarizable() -> impl Summary {
    Tweet {
        username: String::from("horse_ebooks"),
        content: String::from(
            "of course, as you probably already know, people",
        ),
        reply: false,
        retweet: false,
    }
}
```

將 `impl Summary` 作為回傳型別的同時，我們在函式 `returns_summarizable` 指定回傳有實作 `Summary` 特徵的型別而不必指出實際型別。在此例中，`returns_summarizable` 回傳 `Tweet`，但呼叫此函式的程式碼不需要知道。

回傳一個只有指定所需實作特徵的型別在閉包（closures）與疊代器（iterators）中非常有用，我們會在第十三章介紹它們。閉包與疊代器能建立只有編譯器知道的型別，或是太長而難以指定的型別。`impl Trait` 語法允許你不用寫出很長的型別，而是只要指定函數會回傳有實作 `Iterator` 特徵的型別就好。

然而如果你使用 `impl Trait` 的話，你就只能回傳單一型別。舉例來說此程式碼指定回傳型別為 `impl Summary`，但是寫說可能會回傳 `NewsArticle` 或 `Tweet` 的話就會無法執行：

```
fn returns_summarizable(switch: bool) -> impl Summary {
    if switch {
        NewsArticle {
            headline: String::from(
                "Penguins win the Stanley Cup Championship!",
            ),
            location: String::from("Pittsburgh, PA, USA"),
            author: String::from("Iceburgh"),
            content: String::from(
                "The Pittsburgh Penguins once again are the best \
                hockey team in the NHL.",
            ),
        }
    } else {
        Tweet {
            username: String::from("horse_ebooks"),
            content: String::from(
                "of course, as you probably already know, people",
            ),
            reply: false,
            retweet: false,
        }
    }
}
```



寫說可能回傳 `NewsArticle` 或 `Tweet` 的話是不被允許的，因為 `impl Trait` 語法會限制在編譯器中最終決定的型別。我們會在第十七章的「[允許不同型別數值的特徵物件](#)」來討論如何寫出這種行為的函式。

透過特徵界限來選擇性實作方法

在有使用泛型型別參數 `impl` 區塊中使用特徵界限，我們可以選擇性地對有實作特定特徵的型別來實作方法。舉例來說，範例 10-15 的 `Pair<T>` 對所有 `T` 實作了 `new` 函式來回傳新的 `Pair<T>` 實例（回想一下第五章的「[定義方法](#)」段落，`Self` 是 `impl` 區塊內的型別別名，在此例就是 `Pair<T>`）。但在下一個 `impl` 區塊中，只有在其內部型別 `T` 有實作能夠做比較的 `PartialOrd` 特徵以及能夠顯示在螢幕的 `Display` 特徵的話，才會實作 `cmp_display` 方法。

檔案名稱：src/lib.rs


```

use std::fmt::Display;

struct Pair<T> {
    x: T,
    y: T,
}

impl<T> Pair<T> {
    fn new(x: T, y: T) -> Self {
        Self { x, y }
    }
}

impl<T: Display + PartialOrd> Pair<T> {
    fn cmp_display(&self) {
        if self.x >= self.y {
            println!("最大的是 x = {}", self.x);
        } else {
            println!("最大的是 y = {}", self.y);
        }
    }
}

```

範例 10-15：依據特徵界限來選擇性地在泛型型別實作方法

我們還可以對有實作其他特徵的型別選擇性地來實作特徵。對滿足特徵界限的型別實作特徵會稱之為**全面實作 (blanket implementations)**，這被廣泛地用在 Rust 標準函式庫中。舉例來說，標準函式庫會對任何有實作 `Display` 特徵的型別實作 `ToString`。標準函式庫中的 `impl` 區塊會有類似這樣的程式碼：

```

impl<T: Display> ToString for T {
    // --省略--
}

```

因為標準函式庫有此全面實作，我們可以在任何有實作 `Display` 特徵的型別呼叫 `ToString` 特徵的 `to_string` 方法。舉例來說，我們可以像這樣將整數轉變成對應的 `String` 數值，因為整數有實作 `Display`：

```

let s = 3.to_string();

```

全面實作在特徵技術文件的「Implementors」段落有做說明。

特徵與特徵界限讓我們能使用泛型型別參數來減少重複的程式碼的同時，告訴編譯器該泛型型別該擁有何種行為。編譯器可以利用特徵界限資訊來檢查程式碼提供的實際型別有沒有符合特定行為。在動態語言中，我們要是呼叫一個該型別沒有的方法的話，我們會在執行時才發生錯誤。但是 Rust 將此錯誤移到編譯期間，讓我們必須在程式能夠執行之前確保有修正此問題。除此之外，我

們還不用寫在執行時檢查此行為的程式碼，因為我們已經在編譯時就檢查了。這麼做我們可以在不失去泛型彈性的情況下，提升效能。

透過生命週期驗證參考

生命週期 (lifetime) 是另一種我們已經使用過的泛型。不同於確保一個型別有沒有我們要的行為，生命週期確保我們在需要參考的時候，它們都是有效的。

我們在第四章的「[參考與借用](#)」段落沒談到的是，Rust 中的每個參考都有個**生命週期**，這是決定該參考是否有效的作用域。大多情況下生命週期是隱式且可推導出來的，就像大多情況下型別是可推導出來的。當多種型別都有可能時，我們就得詮釋型別。同樣地，當生命週期的參考能以不同方式關聯的話，我們就得詮釋生命週期。Rust 要求我們用泛型生命週期參數來詮釋參考之間的關係，以確保實際在執行時的參考絕對是有效的。

詮釋生命週期在大多數的程式語言中都沒有這個概念，所以這段可能會有點讓你覺得陌生。雖然我們不會在此章涵蓋所有生命週期的內容，但是我們會講些你可能遇到生命週期的常見場景，好讓你更加熟悉這個概念。

透過生命週期預防迷途參考

生命週期最主要的目的就是要預防**迷途參考** (dangling references)，其會導致程式參考到其他資料，而非它原本想要的參考。請看一下範例 10-16 的程式，它有一個外部作用域與內部作用域。

```
fn main() {  
    let r;  
  
    {  
        let x = 5;  
        r = &x;  
    }  
  
    println!("r: {}", r);  
}
```



範例 10-16：嘗試使用其值已經離開作用域的參考

注意：範例 10-16、10-17 與 10-23 宣告變數時都沒有給予初始數值，所以變數名稱可以存在於外部作用域。乍看之下這似乎違反 Rust 不存在空值的原則。但是如果我們嘗試在賦值前使用變數的話，我們就會獲得編譯期錯誤，這證明 Rust 的確不允許空值。

外部作用域宣告了一個沒有初始值的變數 `r`，然後內部作用域宣告了一個初始值為 5 的變數 `x`。在內部作用域中，我們嘗試將 `x` 的參考賦值給 `r`。然後內部作用域結束後，我們嘗試印出 `r`。此程式碼不會編譯成功，因為數值 `r` 指向的數值在我們嘗試使用它時已經離開作用域。以下是錯誤訊息。

```
$ cargo run
   Compiling chapter10 v0.1.0 (file:///projects/chapter10)
error[E0597]: `x` does not live long enough
  --> src/main.rs:6:13
   |
6  |         r = &x;
   |         ^^ borrowed value does not live long enough
7  |     }
   |     - `x` dropped here while still borrowed
8  |
9  |     println!("r: {}", r);
   |                     - borrow later used here

For more information about this error, try `rustc --explain E0597`.
error: could not compile `chapter10` due to previous error
```

變數 `x` 「存在的不夠久」。原因是因為當內部作用域在第 7 行結束時，`x` 會離開作用域。但是 `r` 卻還在外部作用域中有效，我們會說的「活得比較久」。如果 Rust 允許此程式碼可以執行的話，`r` 就會參考到 `x` 離開作用域後被釋放的記憶體位置，然後我們嘗試對 `r` 做的事情都不會是正確的了。所以 Rust 如何決定此程式碼無效呢？它使用了借用檢查器。

借用檢查器

Rust 編譯器有個**借用檢查器** (borrow checker) 會比較作用域來檢測所有的借用是否有效。範例 10-17 顯示了範例 10-16 的程式碼，但加上了變數生命週期的註釋。

```
fn main() {
    let r;           // -----+--- 'a
                    //          |
    {
        let x = 5;   // -+--- 'b |
        r = &x;       //  |      |
    }                // -+      |
                    //          |
    println!("r: {}", r); //      |
}                    // -----+---
```



範例 10-17：變數 `r` 與 `x` 的生命週期註釋，分別以 `'a` 和 `'b` 作為表示

我們在此定義 `r` 的生命週期註釋為 `'a` 而 `x` 的生命週期為 `'b`。如同你所見，內部的 `'b` 區塊比外部的 `'a` 生命週期區塊還小。在編譯期間，Rust 會比較兩個生命週期的大小，並看出 `r` 有生命週期 `'a` 但它參考的記憶體有生命週期 `'b`。程式被回絕的原因是因為 `'b` 比 `'a` 還短：被參考的對象比參考者存在的時間還短。

範例 10-18 修正了此程式碼讓它不會存在迷途參考，並能夠正確編譯。

```
fn main() {
    let x = 5;           // -----+-- 'b
                        //          |
    let r = &x;          // --+-- 'a  |
                        //      |    |
    println!("r: {}", r); //      |    |
                        // --+      |
}                       // -----+
```

範例 10-18：一個有效參考，因為資料比參考的生命週期還長

`x` 在此有生命週期 `'b`，此時它比 `'a` 還長。這代表 `r` 可以參考 `x`，因為 Rust 知道 `r` 的參考在 `x` 是有效的時候永遠是有效的。

現在你知道參考的生命週期，以及 Rust 如何分析生命週期以確保參考永遠有效了。讓我們來探索函式中參數與回傳值的泛型生命週期。

函式中的泛型生命週期

讓我們寫個回傳兩個字串切片中較長者的函式。此函式會接收兩個字串切片並回傳一個字串切片。在我們實作 `longest` 函式後，範例 10-19 的程式碼應該要印出 最長的字串為 `abcd`。

檔案名稱：src/main.rs

```
fn main() {
    let string1 = String::from("abcd");
    let string2 = "xyz";

    let result = longest(string1.as_str(), string2);
    println!("最長的字串為 {}", result);
}
```

範例 10-19：`main` 函式呼叫 `longest` 函式來找出兩個字串切片中較長的

注意我們想要函式接收的是字串切片的參考，而不是字串，因為我們不希望 `longest` 函式會取得它參數的所有權。第四章的「字串切片作為參數」段落有提到為何範例 10-19 的參數正是我們所想要使用的參數。

如果我們嘗試實作 `longest` 函式時，如範例 10-20 所示，它不會編譯過。

檔案名稱：src/main.rs

```
fn longest(x: &str, y: &str) -> &str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```



範例 10-20：回傳兩個字串中較長者的 `longest` 函式實作，不過無法編譯成功

我們會看到以下關於生命週期的錯誤：

```
$ cargo run
   Compiling chapter10 v0.1.0 (file:///projects/chapter10)
error[E0106]: missing lifetime specifier
  --> src/main.rs:9:33
   |
 9 | fn longest(x: &str, y: &str) -> &str {
   |           ----      ----      ^ expected named lifetime parameter
   |
   = help: this function's return type contains a borrowed value, but the
signature does not say whether it is borrowed from `x` or `y`
help: consider introducing a named lifetime parameter
   |
 9 | fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
   |           ++++      ++          ++          ++

For more information about this error, try `rustc --explain E0106`.
error: could not compile `chapter10` due to previous error
```

提示文字表示回傳型別需要有一個泛型生命週期參數，因為 Rust 無法辨別出回傳的參考指的是 `x` 還是 `y`。事實上，我們也不知道，因為函式本體中的 `if` 區塊會回傳 `x` 的參考而 `else` 區塊會回傳 `y` 的參考！

當我們定義函式時，我們不知道傳遞進此函式的實際數值會是什麼，所以我們不知道到底是 `if` 或 `else` 的區塊會被執行。我們也不知道傳遞進來的參考實際的生命週期為何，所以我們無法像範例 10-17 和 10-18 那樣觀察作用域，來判定我們回傳的參考會永遠有效。要修正此錯誤，我們加上泛型生命週期參數來定義參考之間的關係，讓借用檢查器能夠進行分析。

生命週期詮釋語法

生命週期詮釋（Lifetime Annotation）不會改變參考能存活多久，它們僅描述了數個參考的生命週期之間互相的關係，而不會影響其生命週期。就像當函式簽名指定了一個泛型型別參數時，函式便能夠接受任意型別一樣。函式可以指定一個泛型生命週期參數，這樣函式就能接受任何生命週期。

生命週期詮釋的語法有一點不一樣：生命週期參數的名稱必須以撇號（'）作為開頭，通常全是小寫且很短，就像泛型型別一樣。大多數的人會使用名稱 `'a` 作為第一個生命週期詮釋。我們將生命週期參數置於參考的 `&` 之後，並使用空格區隔詮釋與參考的型別。

以下是一些例子：沒有生命週期參數的 `i32` 參考、有生命週期 `'a` 的 `i32` 參考以及有生命週期 `'a` 的 `i32` 可變參考。

```
&i32           // 一個參考
&'a i32        // 一個有顯式生命週期的參考
&'a mut i32    // 一個有顯式生命週期的可變參考
```

只有自己一個生命週期本身沒有多少意義，因為該詮釋是為了告訴 Rust 數個參考的泛型生命週期參數之間互相的關係。讓我們來研究生命週期詮釋如何在 `longest` 函式中相互關聯吧。

函式簽名中的生命週期詮釋

要在函式簽名使用生命週期詮釋的話，我們需要在函式名稱與參數列表之間的尖括號內宣告泛型生命週期參數，就像泛型型別參數那樣。

我們想在此簽名表達這樣的限制：只要所有參數都要是有效的，那麼回傳的參考才也會是有效的。也就是參數的生命週期與回傳參考的生命週期是相關的。我們會將生命週期命名為 `'a` 然後將它加到每個參考，如範例 10-21 所示。

檔案名稱：src/main.rs

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

範例 10-21： `longest` 函式定義指定所有簽名中的參考必須有相同的生命週期 `'a`

此程式碼能夠編譯成功並產生我們希望在範例 10-19 的 `main` 函式中得到的結果。

此函式簽名告訴 Rust 它有個生命週期 `'a`，函式的兩個參數都是字串切片，並且會有生命週期 `'a`。此函式簽名還告訴了 Rust 從函式回傳的字串切片也會和生命週期 `'a` 存活的一樣久。實際上它代表 `longest` 函式回傳參考的生命週期與函式引數傳入時生命週期較短的參考的生命週期一樣。這樣的關係正是我們想讓 Rust 知道以便分析這段程式碼。

注意當我們在此函式簽名指定生命週期參數時，我們不會變更任何傳入或傳出數值的生命週期。我們只是告訴借用檢查器應該要拒絕任何沒有服從這些約束的數值。注意到 `longest` 函式不需要知道 `x` 和 `y` 實際上會活多久，只需要知道有某個作用域會用 `'a` 取代來滿足此簽名。

當要在函式詮釋生命週期時，詮釋會位於函式簽名中，而不是函式本體。就像型別會寫在簽名中一樣，生命週期詮釋會成為函式的一部份。在函式簽名加上生命週期能讓 Rust 編譯器的分析工作變得更輕鬆。如果當函式的詮釋或呼叫的方式出問題時，編譯器錯誤就能限縮到我們的程式碼中指出來。如果都改讓 Rust 編譯器去推導可能的生命週期關係的話，編譯器可能會指到程式碼真正出錯之後的好幾步之後。

當我們向 `longest` 傳入實際參考時，`'a` 實際替代的生命週期為 `x` 作用域與 `y` 作用域重疊的部分。換句話說，泛型生命週期 `'a` 取得的生命週期會等於 `x` 與 `y` 的生命週期中較短的。因為我們將回傳的參考詮釋了相同的生命週期參數 `'a`，回傳參考的生命週期也會保證在 `x` 和 `y` 的生命週期較短的結束前有效。

讓我們來看看如何透過傳入不同實際生命週期的參考來使生命週期詮釋能約束 `longest` 函式，如範例 10-22 所示。

檔案名稱：src/main.rs

```
fn main() {
    let string1 = String::from("很長的長字串");

    {
        let string2 = String::from("xyz");
        let result = longest(string1.as_str(), string2.as_str());
        println!("最長的字串為 {}", result);
    }
}
```

範例 10-22 使用 `longest` 函式並傳入 `String` 數值的參考，但兩個參數的實際生命週期均不相同

在此例中 `string1` 在外部作用域結束前都有效，而 `string2` 在內部作用域結束前都有效，然後 `result` 會取得某個有效參考直到內部作用域結束為止。執行此程式的話，你會看到借用檢查器認可此程式碼，它會編譯成功然後印出 最長的字串為 很長的長字串 。

接下來，讓我們寫一個範例能要求 `result` 生命週期的參考必須是兩個引數中較短的才行。我們會移動變數 `result` 的宣告到外部作用域，但保留變數 `result` 的賦值與 `string2` 一樣在內部作用域。然後我們也將使用到 `result` 的 `println!` 移到外部作用域，緊接在內部作用域結束之後。如範例 10-23 所示，此程式碼會編譯不過。

檔案名稱：src/main.rs


```
fn main() {
    let string1 = String::from("很長的長字串");
    let result;
    {
        let string2 = String::from("xyz");
        result = longest(string1.as_str(), string2.as_str());
    }
    println!("最長的字串為 {}", result);
}
```



範例 10-23：嘗試在 `string2` 離開作用域後使用 `result`

當我們嘗試編譯此程式碼，我們會看到以下錯誤：

```
$ cargo run
   Compiling chapter10 v0.1.0 (file:///projects/chapter10)
error[E0597]: `string2` does not live long enough
  --> src/main.rs:6:44
   |
6  |         result = longest(string1.as_str(), string2.as_str());
   |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^ borrowed value
does not live long enough
7  |     }
   |     - `string2` dropped here while still borrowed
8  |     println!("最長的字串為 {}", result);
   |                                     ----- borrow later used here

For more information about this error, try `rustc --explain E0597`.
error: could not compile `chapter10` due to previous error
```

錯誤訊息表示要讓 `result` 在 `println!` 陳述式有效的話，`string2` 必須在外部作用域結束前都是有效的。Rust 會知道是因為我們在函式的參數與回傳值使用相同的生命週期 'a 來詮釋。

身為人類我們能看出此程式碼的 `string1` 字串長度的確比 `string2` 長，因此 `result` 會包含 `string1` 的參考。因為 `string1` 尚未離開作用域，所以 `string1` 的參考在 `println!` 陳述式中仍然是有效的才對。然而編譯器在此情形會無法看出參考是有效的。所以我們才告訴 Rust `longest` 函式回傳參考的生命週期等同於傳入參考中較短的生命週期。這樣一來借用檢查器就會否決範例 10-23 的程式碼，因為它可能會有無效的參考。

歡迎嘗試設計更多採用不同數值與不同生命週期的參考作為 `longest` 函式參數與回傳值的實驗，並在編譯前假設你的實驗會不會通過借用檢查器，然後看看你的理解是不是正確的！

深入理解生命週期

你要指定生命週期參數的方式取決於函式的行為。舉例來說如果我們改變函式 `longest` 的實作為永遠只回傳第一個參數而不是最長的字串切片，我們就不需要在參數 `y` 指定生命週期。以下的程

式碼就能編譯：

檔案名稱：src/main.rs

```
fn longest<'a>(x: &'a str, y: &str) -> &'a str {  
    x  
}
```

我們指定生命週期參數 `'a` 給參數 `x` 與回傳型別，但參數 `y` 則沒有，因為 `y` 的生命週期與 `x` 和回傳型別的生命週期之間沒有任何關係。

當函式回傳參考時，回傳型別的生命週期參數必須符合其中一個參數的生命週期參數。如果回傳參考沒有和任何參數有關聯的話，代表它參考的是函式本體中的數值。但這會是迷途參考，因為該數值會在函式結尾離開作用域。請看看以下嘗試在函式 `longest` 的實作做法，它並不會編譯成功：

檔案名稱：src/main.rs

```
fn longest<'a>(x: &str, y: &str) -> &'a str {  
    let result = String::from("超長的字串");  
    result.as_str()  
}
```



我們在這邊雖然有對回傳型別指定生命週期參數 `'a`，但此實作還是會失敗，因為回傳值的生命週期與參數的生命週期完全無關。以下是我們獲得的錯誤訊息：

```
$ cargo run  
Compiling chapter10 v0.1.0 (file:///projects/chapter10)  
error[E0515]: cannot return reference to local variable `result`  
--> src/main.rs:11:5  
   |  
11 |     result.as_str()  
   |     ^^^^^^^^^^^^^^^ returns a reference to data owned by the current  
function  
  
For more information about this error, try `rustc --explain E0515`.  
error: could not compile `chapter10` due to previous error
```

問題在於 `result` 會離開作用域並在 `longest` 函式結尾被清除。我們卻嘗試從函式中回傳 `result` 的參考。我們無法指定生命週期參數來改變迷途參考，而且 Rust 不會允許我們將建立迷途參考。在此例中，最好的解決辦法是回傳有所有權的資料型別而非參考，並讓呼叫的函式自行決定如何清理數值。

總結來說，生命週期語法是用來連接函式中不同參數與回傳值的生命週期。一旦連結起來，Rust 就可以獲得足夠的資訊來確保記憶體安全的運算並防止會產生迷途指標或違反記憶體安全的操作。

結構體定義中的生命週期詮釋

目前為止，我們定義過的結構體都持有型別的所有權。結構體其實也能持有參考，不過我們會需要在結構體定義中每個參考加上生命週期詮釋。範例 10-24 有個持有字串切片的結構體

`ImportantExcerpt`。

檔案名稱：src/main.rs

```
struct ImportantExcerpt<'a> {
    part: &'a str,
}

fn main() {
    let novel = String::from("Call me Ishmael. Some years ago...");
    let first_sentence = novel.split('.').next().expect("無法找到 '. '");
    let i = ImportantExcerpt {
        part: first_sentence,
    };
}
```

範例 10-24：擁有參考的結構體需要加上生命週期詮釋

此結構體有個欄位 `part` 並擁有字串切片參考。如同泛型資料型別，我們在結構體名稱之後的尖括號內宣告泛型生命週期參數，所以我們就可以在結構體定義的本體中使用生命週期參數。此詮釋代表 `ImportantExcerpt` 的實例不能比它持有的欄位 `part` 活得還久。

`main` 函式在此產生一個結構體 `ImportantExcerpt` 的實例並持有一個參考，其為變數 `novel` 所擁有的 `String` 中的第一個句子的參考。`novel` 的資料是在 `ImportantExcerpt` 實例之前建立的。除此之外，`novel` 在 `ImportantExcerpt` 離開作用域之前不會離開作用域，所以 `ImportantExcerpt` 實例中的參考是有效的。

生命週期省略

你已經學到了每個參考都有個生命週期，而且你需要在有使用參考的函式與結構體中指定生命週期參數。然而在第四章的範例 4-9 我們有函式可以不詮釋生命週期並照樣編譯成功，我們在範例 10-25 再展示一次。

檔案名稱：src/lib.rs

```
fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();

    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }

    &s[..]
}
```

範例 10-25：在範例 4-9 定義過的函式，雖然其參數與回傳值均為參考，卻仍可編譯成功

此函式可以不用生命週期註釋仍照樣編譯過是有歷史因素的：在早期版本的 Rust（1.0 之前），此程式碼是無法編譯的，因為每個參考都得有顯式生命週期。在當時的情況下，此函式簽名會長得像這樣：

```
fn first_word<'a>(s: &'a str) -> &'a str {
```

在寫了大量的 Rust 程式碼後，Rust 團隊發現 Rust 開發者會在特定情況反覆輸入同樣的生命週期註釋。這些情形都是可預期的，而且可以遵循一些明確的模式。開發者將這些模式加入編譯器的程式碼中，所以借用檢查器可以依據這些情況自行推導生命週期，而讓我們不必顯式註釋。

這樣的歷史值得提起的原因是因為很可能會有更多明確的模式被找出來並加到編譯器中，意味著未來對於生命週期註釋的要求會更少。

被寫進 Rust 參考分析的模式被稱作**生命週期省略規則（lifetime elision rules）**。這些不是程式設計師要遵守的規則，而是一系列編譯器能去考慮的情形。而如果你的程式碼符合這些情形時，你就不必顯式寫出生命週期。

省略規則無法提供完整的推導。如果 Rust 能明確套用規則，但在這之後還是有參考存在模稜兩可的生命週期，編譯器就無法猜出剩餘參考的生命週期。編譯器不會亂猜，它會回傳錯誤給你，說明你需要指定生命週期註釋。

在函式或方法參數上的生命週期稱為**輸入生命週期（input lifetimes）**，而在回傳值的生命週期則稱為**輸出生命週期（output lifetimes）**。

當參考沒有顯式註釋生命週期時，編譯器會用三項規則來推導它們。第一個規則適用於輸入生命週期，而第二與第三個規則適用於輸出生命週期。如果編譯器處理完這三個規則，卻仍有參考無法推斷出生命週期時，編譯器就會停止並回傳錯誤。這些適用於 `fn` 定義的規則一樣適用於 `impl` 區塊。

第一個規則是編譯器會給予每個參考參數一個生命週期參數。換句話說，一個函式只有一個參數的話，就只會有一個生命週期：`fn foo<'a>(x: &'a i32)`；一個函式有兩個參數的話，就會有分別兩個生命週期參數：`fn foo<'a, 'b>(x: &'a i32, y: &'b i32)`，以此類推。

第二個規則是如果剛好只有一個輸入生命週期參數，該參數就會賦值給所有輸出生命週期參數：

```
fn foo<'a>(x: &'a i32) -> &'a i32。
```

第三個規則是如果有多個輸入生命週期參數，但其中一個是 `&self` 或 `&mut self`，由於這是方法，`self` 的生命週期會賦值給所有輸出生命週期參數。此規則讓方法更容易讀寫，因為不用寫更多符號出來。

讓我們假裝我們是編譯器。我們會檢查這些規則並找出範例 10-25 中函式 `first_word` 簽名中參考的生命週期。簽名的參考一開始沒有任何生命週期：

```
fn first_word(s: &str) -> &str {
```

接著編譯器檢查第一個規則，指明每個參數都有自己的生命週期。我們如往常一樣指定 `'a`，所以簽名就會變成：

```
fn first_word<'a>(s: &'a str) -> &str {
```

然後第二個規則也適用因為這裡剛好就一個輸入生命週期而已。第二個規則指明只有一個輸入生命週期的話，就會賦值給所有其他輸出生命週期。所以簽名現在變成這樣：

```
fn first_word<'a>(s: &'a str) -> &'a str {
```

現在此函式所有的參考都有生命週期了，而且編譯器可以繼續分析，不必要求程式設計師在此詮釋函式簽名的生命週期。

讓我們再看看一個例子，這次是範例 10-20 一開始沒有任何生命週期參數的 `longest` 函式：

```
fn longest(x: &str, y: &str) -> &str {
```

讓我們先檢查第一項規則：每個參數都有自己的生命週期。這次我們有兩個參數，所以我們有兩個生命週期：

```
fn longest<'a, 'b>(x: &'a str, y: &'b str) -> &str {
```

你可以看出來第二個規則並不適用，因為我們有不只一個輸入生命週期。而第三個也不適用，因為 `longest` 是函式而非方法，其參數不會有 `self`。遍歷這三個規則下來，我們仍然無法推斷出回傳型別的生命週期。這就是為何我們嘗試編譯範例 10-20 的程式碼會出錯的原因：編譯器遍歷生命週期省略規則，但仍然無法推導出簽名中所有參考的生命週期。

因為第三個規則僅適用於方法簽名，我們接下來就會看看這種情況時的生命週期，看看為何第三個規則讓我們不必常常在方法簽名詮釋生命週期。

在方法定義中的生命週期詮釋

當我們在有生命週期的結構體上實作方法時，其語法類似於我們在範例 10-11 中泛型型別參數的語法。宣告並使用生命週期參數的地方會依據它們是否與結構體欄位或方法參數與回傳值相關。

結構體欄位的生命週期永遠需要宣告在 `impl` 關鍵字後方以及結構體名稱後方，因為這些生命週期是結構體型別的一部分。

在 `impl` 區塊中方法簽名的參考可能會與結構體欄位的參考生命週期綁定，或者它們可能是互相獨立的。除此之外，生命週期省略規則常常可以省略方法簽名中的生命週期詮釋。讓我們看看範例 10-24 定義過的 `ImportantExcerpt` 來作為範例。

首先我們使用一個方法叫做 `level` 其參數只有 `self` 的參考而回傳值是 `i32`，這不是任何參考：

```
impl<'a> ImportantExcerpt<'a> {  
    fn level(&self) -> i32 {  
        3  
    }  
}
```

我們必須在 `impl` 之後宣告生命週期參數，並在型別名稱後使用該生命週期。但是我們不必在 `self` 的參考加上生命週期詮釋，因為其適用於第一個省略規則。

以下是第三個生命週期省略規則適用的地方：

```
impl<'a> ImportantExcerpt<'a> {  
    fn announce_and_return_part(&self, announcement: &str) -> &str {  
        println!("請注意：{}", announcement);  
        self.part  
    }  
}
```

這裡有兩個輸入生命週期，所以 Rust 用第一個生命週期省略規則給予 `&self` 和 `announcement` 它們自己的生命週期。然後因為其中一個參數是 `&self`，回傳型別會取得 `&self` 的生命週期，如此一來所有的生命週期都推導出來了。

靜態生命週期

其中有個特殊的生命週期 `'static` 我們需要進一步討論，這是指該參考可以存活在整個程式期間。所有的字串字面值都有 `'static` 生命週期，我們可以這樣詮釋：

```
let s: &'static str = "我有靜態生命週期。";
```

此字串的文字會直接儲存在程式的執行檔中，所以永遠有效。因此所有的字串字面值的生命週期都是 `'static`。

你有時可能會看到錯誤訊息建議使用 `'static` 生命週期。但在你對參考指明 `'static` 生命週期前，最好想一下該參考的生命週期是否真的會存在於整個程式期間，以及是否真的該活得這麼久。大多數錯誤訊息會建議 `'static` 生命週期的情況都來自於嘗試建立迷途參考或可用的生命週期不符。這樣的情況下，應該是要實際嘗試解決問題，而不是指明 `'static` 生命週期。

組合泛型型別參數、特徵界限與生命週期

讓我們用一個函式來總結泛型型別參數、特徵界限與生命週期的語法！

```
use std::fmt::Display;

fn longest_with_an_announcement<'a, T>(
    x: &'a str,
    y: &'a str,
    ann: T,
) -> &'a str
where
    T: Display,
{
    println!("公告! {}", ann);
    if x.len() > y.len() {
        x
    } else {
        y
    }
}
```

這是範例 10-21 會回傳兩個字串切片較長者的 `longest` 函式。不過現在它有個額外的參數 `ann`，使用的是泛型型別 `T`，它可以是任何在 `where` 中所指定有實作 `Display` 特徵的型別。此額外參數會在 `{}` 的地方印出來，這正是為何 `Display` 的特徵界限是必須的。因為生命週期也是一種泛型，生命週期參數 `'a` 與泛型型別參數 `T` 都宣告在函式名稱後的尖括號內。

總結

我們在此章節涵蓋了許多內容！現在你已經知道泛型型別參數、特徵與特徵界限以及泛型生命週期參數，你已經準備好能寫出適用於許多不同情況且不重複的程式碼了。泛型型別參數讓你可以讓程式碼適用於不同型別；特徵與特徵界限確保就算型別為泛型，它們都會有相同的行為。你還學到了使用生命週期詮釋確保此如此彈性的程式碼不會造成迷途參考。而且這些分析都發生在編譯期間，完全不影響執行的效能！

不管你信不信，本章節還有很多延伸主題可以導論，像是第十七章就會討論特徵物件 (trait objects)，這是另一個使用特徵的方法。另外還有一些更複雜的場合會涉及到更進階的生命週期詮釋。對此你可能就會想閱讀 [Rust Reference](#)。接下來，你將學習如何在 Rust 寫測試，讓你可以確保程式碼能如期執行。

編寫自動化測試

Edsger W. Dijkstra 曾在 1972 年的演講「謙遜的程式設計師」中提到：「程式測試是個證明程式錯誤存在非常有效的方法，但要證明它不存在卻反而顯得十分無力。」這不代表我們不應該盡可能地做測試！

程式碼的正確性意謂著我們的程式碼可以如我們的預期執行。Rust 就被設計為特別注重程式的正確性，但正確性是很複雜且難以證明的。Rust 的型別系統就承擔了很大一部分的負擔，但是型別系統還是沒辦法抓到全部。所以 Rust 在語言內提供了編寫自動化程式測試的支援。

假設我們要寫個程式叫做 `add_two`，其會將傳入任意數字加上 2。此函式簽名接受整數作為參數並回傳一個整數作為結果。當我們實作並編譯函式時，Rust 會做所有你已經學過的型別檢查與借用檢查，來確保像是我們不會中傳入 `String` 數值或任意無效參考至此函式。但 Rust **無法**檢查其是否能執行我們預期此函式會完成的任務，也就是回傳加上 2 的參數。說不定它會將參數加上 10 或減 50！這就是我們要做測試的地方。

舉例來說，我們可以寫測試來判定當我們傳入 3 給函式 `add_two` 時，回傳值是不是 5。我們可以再變更我們的程式碼時來執行這些測試，以確保原本就正確的行為不會被改變。

測試是個複雜的技能，雖然我們無法在一個章節就涵蓋如何寫出好測試的細節，但我們還是會討論 Rust 測試功能機制。我們會介紹當你寫測試時可以用的註釋與巨集、執行測試時的預設行為與選項以及如何組織測試成單元測試與整合測試。

如何寫測試

測試是一種 Rust 函式來驗證非測試程式碼是否以預期的方式執行。測試函式的本體通常會做三件動作：

1. 設置任何所需要的資料或狀態。
2. 執行你希望測試的程式碼
3. 判定結果是否與你預期的相符。

讓我們看看 Rust 特地提供給測試的功能：包含 `test` 屬性 (attribute)、一些巨集以及 `should_panic` 屬性。

測試函式剖析

最簡單的形式來看，測試在 Rust 中就是附有 `test` 屬性的函式。屬性是一種關於某段 Rust 程式碼的詮釋資料 (metadata)，其中一個例子是我們在第五章使用的 `derive` 屬性。要將一個函式轉換成測試函式，在 `fn` 前一行加上 `#[test]` 即可。當你用 `cargo test` 命令來執行你的測試時，Rust 會建構一個測試執行檔並執行被標注的函式，並回報每個測試函式是否通過或失敗。

當我們用 Cargo 建立新的函式庫專案時，同時會自動建立一個擁有測試函式的測試模組。此模組能協助我們開始寫測試，讓你不必在每次建立新專案時，尋找特定結構體與測試函式的語法。你可以新增多少測試函式與多少測試模組都沒問題！

在實際測試任何程式碼之前，我們將會透過實驗測試產生的樣板，來探索測試如何運作的每個環節。然後我們會寫些現實世界會寫的測試，呼叫我們寫的程式碼並判定其行為是否正確。

讓我們建立個會相加兩個數字的函式庫專案 `adder`：

```
$ cargo new adder --lib
    Created library `adder` project
$ cd adder
```

函式庫專案 `adder` 中的 `src/lib.rs` 檔案內容會長得像範例 11-1 所示。

檔案名稱：`src/lib.rs`

```
#[cfg(test)]
mod tests {
    #[test]
    fn it_works() {
        let result = 2 + 2;
        assert_eq!(result, 4);
    }
}
```

範例 11-1：透過 `cargo new` 自動產生的測試模組與函式

現在我們先忽略開頭前兩行並專注在函式。先注意到 `#[test]` 註釋：此屬性指出這是測試函式，所以測試者會知道此函式是用來測試的。我們也可以在 `tests` 模組中加入非測試函式來協助設置常見場景或是執行常見運算，所以我們需要標注哪些是想要測試的函式。

範例函式本體使用 `assert_eq!` 巨集來判定該 `result`，也就是 $2 + 2$ 的結果是否等於 4。此判定是作為典型測試的範例格式。讓我們執行它來看看此測試是否會通過。

`cargo test` 命令會執行專案中的所有測試，如範例 11-2 所示。

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.57s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 1 test
test tests::it_works ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

範例 11-2：執行自動產生的測試的輸出結果

Cargo 會編譯並執行測試。在 `running 1 test` 這行之後會顯示自動產生的測試函式 `it_works` 以及測試執行的結果 `ok`。再來可以看到整體總結，`test result: ok.` 代表所有測試都有通過，然後 `1 passed; 0 failed` 指出所有測試成功或失敗的數量。

我們可以選擇忽略測試，讓它在特定情形不會執行，我們會在本章的「[忽略某些測試除非特別指定](#)」段落再做說明。因為我們尚未有任何會忽略的程式碼，所以總結會顯示 `0 ignored`。我們也

可以在 `cargo test` 傳入引數，只執行名稱符合字串的測試。這叫做過濾 (filtering)，我們會在「透過名稱來執行部分測試」段落做說明。我們也沒有過濾會執行的測試，所以總結最後顯示 `0 filtered out`。

`0 measured` 的統計數值是指評測效能的效能測試。效能測試 (Benchmark tests) 在本書撰寫時，仍然僅在 nightly Rust 可用。請查閱[效能測試的技術文件](#)來瞭解詳情。

測試輸出結果的下一部分，也就是 `Doc-tests adder`，是指任何技術文件測試的結果。我們還沒有任何技術文件測試，但是 Rust 可以編譯在 API 技術文件中的任何程式碼範例。此功能能幫助我們將技術文件與程式碼保持同步！我們會在第十四章的「將技術文件註解作為測試」段落討論如何寫技術文件測試。現在我們會先忽略 `Doc-tests` 的輸出結果。

讓我們變更程式碼的名稱來看看測試輸出會變成什麼。將 `it_works` 函式變更名稱，像是以下改成 `exploration` 這樣：

檔案名稱：src/lib.rs

```
#[cfg(test)]
mod tests {
    #[test]
    fn exploration() {
        assert_eq!(2 + 2, 4);
    }
}
```

然後再執行一次 `cargo test`，輸出會顯示 `exploration` 而非 `it_works`：

```
$ cargo test
   Compiling adder v0.1.0 (file:///projects/adder)
   Finished test [unoptimized + debuginfo] target(s) in 0.59s
   Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 1 test
test tests::exploration ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s


Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

讓我們再加上另一個測試，不過這次我們要讓測試失敗！測試會在測試函式恐慌時失敗，每個測試會跑在新的執行緒（thread）上，然後當主執行緒看到測試執行緒死亡時，就會將該測試標記為失敗的。我們有在第九章提及引發恐慌最簡單的辦法，那就是呼叫 `panic!` 巨集。將它寫入新的測試 `another` 中，所以你在 `src/lib.rs` 的檔案中會看到向範例 11-3 這樣。

檔案名稱：src/lib.rs

```
#[cfg(test)]
mod tests {
    #[test]
    fn exploration() {
        assert_eq!(2 + 2, 4);
    }

    #[test]
    fn another() {
        panic!("此測試會失敗");
    }
}
```



範例 11-3：新增第二個會失敗的測試，因為我們會呼叫 `panic!` 巨集

使用 `cargo test` 再執行一次測試，輸出結果應該會像範例 11-4 這樣，顯示出我們的 `exploration` 測試通過但 `another` 失敗。

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.72s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 2 tests
test tests::another ... FAILED
test tests::exploration ... ok

failures:

---- tests::another stdout ----
thread 'main' panicked at '此測試會失敗', src/lib.rs:10:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::another

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

範例 11-4：其中一個測試通過，而另一個失敗的輸出結果

`test tests::another` 這行會顯示 `FAILED` 而非 `ok`。在獨立結果與總結之間出現了兩個新的段落，第一個段落會顯示每個測試失敗的原因細節。在此例中，我們會收到 `another` 失敗的緣由，因為 `src/lib.rs` 檔案中第十行的恐慌 `panicked at '此測試會失敗'`。下一個段落則是會列出所有失敗的測試，要是測試很多且失敗測試輸出結果很長的話，此資訊就很實用。我們可以使用失敗測試的名稱來只執行這個測試以便除錯。我們會在「[控制程式如何執行](#)」段落討論更多執行測試的方法。

總結會顯示在最後一行，在此例中它表示我們有一個測試結果是 `FAILED`。也就是我們有一個測試通過，一個測試失敗。

現在你知道測試結果在不同場合看起來的樣子，讓我們來看看除了 `panic!` 以外對測試也很有幫助的巨集吧。

透過 `assert!` 巨集檢查結果

標準函式庫提供的 `assert!` 巨集可以在你要確保測試中的一些條件評估為 `true` 時使用。我們給予 `assert!` 巨集一個引數來計算出布林值。如果數值為 `true`，`assert!` 不會做任何動作然後測試就會通過。如果數值為 `false`，`assert!` 巨集會呼叫 `panic!` 巨集導致測試失敗。使用 `assert!` 巨集能幫助我們檢查我們的程式碼是否以我們預期的方式運作。

在第五章的範例 5-15，我們有結構體 `Rectangle` 與方法 `can_hold`，我們在範例 11-5 再看一次。讓我們將此程式碼寫入 `src/lib.rs` 檔案中，並寫些對它使用 `assert!` 巨集的測試。

檔案名稱：`src/lib.rs`

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

impl Rectangle {
    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width > other.width && self.height > other.height
    }
}
```

範例 11-5：第五章中的結構體 `Rectangle` 與其方法 `can_hold`

`can_hold` 方法會回傳布林值，這代表它是 `assert!` 巨集的絕佳展示機會。在範例 11-6 中，我們寫了個測試來練習 `can_hold` 方法，我們建立了一個寬度為 8 長度為 7 的 `Rectangle` 實例，並判定它可以包含另一個寬度為 5 長度為 1 的 `Rectangle` 實例。

檔案名稱：`src/lib.rs`

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn larger_can_hold_smaller() {
        let larger = Rectangle {
            width: 8,
            height: 7,
        };
        let smaller = Rectangle {
            width: 5,
            height: 1,
        };

        assert!(larger.can_hold(&smaller));
    }
}
```

範例 11-6：一支檢查一個大長方形是否能包含一個小長方形的 `can_hold` 測試

注意到我們已經在 `tests` 模組中加了一行 `use super::*;`。 `tests` 和一般的模組一樣都遵循我們在第七章「[參考模組項目的路徑](#)」提及的常見能見度規則。因為 `tests` 模組是內部模組，我們

需要將外部模組的程式碼引入內部模組的作用域中。我們使用全域運算子（glob）讓外部模組定義的所有程式碼在此 `tests` 模組都可以使用。

我們將我們的測試命名為 `larger_can_hold_smaller`，然後我們建立兩個我們需要用到的 `Rectangle` 實例。然後我們呼叫 `assert!` 巨集並將 `larger.can_hold(&smaller)` 的結果傳給它。此表達式應該要回傳 `true`，所以我們的程式應該會通過。讓我們看看結果吧！

```
$ cargo test
  Compiling rectangle v0.1.0 (file:///projects/rectangle)
  Finished test [unoptimized + debuginfo] target(s) in 0.66s
  Running unittests src/lib.rs (target/debug/deps/rectangle-6584c4561e48942e)

running 1 test
test tests::larger_can_hold_smaller ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s


Doc-tests rectangle

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

它通過了！讓我們再加另一個測試，這是判定小長方形無法包含大長方形：

檔案名稱：`src/lib.rs`


```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn larger_can_hold_smaller() {
        // --省略--
    }

    #[test]
    fn smaller_cannot_hold_larger() {
        let larger = Rectangle {
            width: 8,
            height: 7,
        };
        let smaller = Rectangle {
            width: 5,
            height: 1,
        };

        assert!(!smaller.can_hold(&larger));
    }
}

```

因為函式 `can_hold` 的正確結果在此例為 `false`，我們需要將該結果反轉後才能傳給 `assert!` 巨集。因此我們的測試在 `can_hold` 回傳 `false` 時才會通過：

```
$ cargo test
Compiling rectangle v0.1.0 (file:///projects/rectangle)
Finished test [unoptimized + debuginfo] target(s) in 0.66s
Running unittests src/lib.rs (target/debug/deps/rectangle-6584c4561e48942e)

running 2 tests
test tests::larger_can_hold_smaller ... ok
test tests::smaller_cannot_hold_larger ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests rectangle

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

兩個測試都過了！現在讓我們看看當我們在程式碼中引入程式錯誤的話，測試結果會為何。讓我們來改變 `can_hold` 方法的實作將比較時的大於符號改成小於符號：

```
// --省略--
impl Rectangle {
    fn can_hold(&self, other: &Rectangle) -> bool {
        self.width < other.width && self.height > other.height
    }
}
```



執行測試的話現在就會顯示以下結果：

```
$ cargo test
Compiling rectangle v0.1.0 (file:///projects/rectangle)
Finished test [unoptimized + debuginfo] target(s) in 0.66s
Running unittests src/lib.rs (target/debug/deps/rectangle-6584c4561e48942e)

running 2 tests
test tests::larger_can_hold_smaller ... FAILED
test tests::smaller_cannot_hold_larger ... ok

failures:

---- tests::larger_can_hold_smaller stdout ----
thread 'main' panicked at 'assertion failed: larger.can_hold(&smaller)',
src/lib.rs:28:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::larger_can_hold_smaller

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

我們的測試抓到了錯誤！因為 `larger.width` 是 8 而 `smaller.width` 是 5，`can_hold` 比較寬度時現在會回傳 `false`，因為 8 沒有比 5 小。

透過 `assert_eq!` 與 `assert_ne!` Macros 測試相等

有一種常見驗證程式的方式是將程式碼的結果與你預期程式碼會回傳的數值做測試，檢查它們是否相等。你可以使用 `assert!` 巨集並傳入使用 `==` 運算子的表達式來辦到。不過這種測試方法是很常見的，所以標準函式庫提供了一對巨集 `assert_eq!` 與 `assert_ne!` 來讓你能更方便地測試。這兩個巨集分別比較兩個引數是否相等或不相等。如果判定失敗的話，它們還會印出兩個數值，讓我們能清楚看到為何測試失敗。相對地，`assert!` 巨集只會說明它在 `==` 表達式中取得 `false` 值，而不會告訴你導致 `false` 的那兩個值。

在範例 11-7 中，我們寫了個函式叫做 `add_two` 並對參數加上 2，然後我們使用 `assert_eq!` 巨集來測試此函式。

檔案名稱：src/lib.rs

```
pub fn add_two(a: i32) -> i32 {
    a + 2
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_adds_two() {
        assert_eq!(4, add_two(2));
    }
}
```

範例 11-7：使用 `assert_eq!` 巨集測試函式 `add_two`

讓我們檢查後它的確通過了！

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.58s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 1 test
test tests::it_adds_two ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

我們傳入 `assert_eq!` 巨集的引數 `4` 與呼叫 `add_two(2)` 的結果相等。測試的結果為 `test tests::it_adds_two ... ok` 而 `ok` 就代表我們的測試通過了！

讓我們在我們的程式碼引入個錯誤，看看使使用 `assert_eq!` 的測試失敗時看起來為何。變更函式 `add_two` 的實作改成加 `3`：

```
pub fn add_two(a: i32) -> i32 {
    a + 3
}
```

再執行一次測試：

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.61s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 1 test
test tests::it_adds_two ... FAILED

failures:

---- tests::it_adds_two stdout ----
thread 'main' panicked at 'assertion failed: `(left == right)`
  left: `4`,
 right: `5`', src/lib.rs:11:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::it_adds_two

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

我們的測試抓到了錯誤！`it_adds_two` 測試失敗了，然後訊息會告訴我們失敗的判斷來自於 `assertion failed: `(left == right)``，以及 `left` 與 `right` 的數值為何。此訊息能協助我們開始除錯：`left` 的引數是 4 但是擁有 `add_two(2)` 的引數 `right` 卻是 5。你應該能想像這會在有一大堆測試時是非常有幫助的。

注意到在有些語言或測試框架中，判定相等的函式的參數會稱作 `expected` 和 `actual`，然後它們會因為指定的引數順序而有差。但在 Rust 中它們被稱為 `left` 和 `right`，且我們預期的值與測試中程式碼產生的值之間的順序沒有任何影響。我們可以在此測試這樣寫判定 `assert_eq!` (`add_two(2)`, 4)，而錯誤訊息一樣會顯示 `assertion failed: `(left == right)``。

`assert_ne!` 巨集會在我們給予的兩個值不相等時通過，相等時失敗。此巨集適用於當我們不確定一個數值會是什麼樣子，但是我們確定該數值不該是某種樣子。舉例來說，如果我們要測試一個保證會以某種形式更改其輸入的函式，但輸入變更的方式是依照我們執行程式時的當天是星期幾來決定，此時最好的判定方式就是檢查函式的輸出不等於輸入。

`assert_eq!` 和 `assert_ne!` 巨集底下分別使用了 `==` 和 `!=` 運算子。當判定失敗時，巨集會透過除錯格式化資訊來顯示它們的引數，代表要比較的數值必須要實作 `PartialEq` 和 `Debug` 特徵。所有的基本型別與大多數標準函式庫中提供的型別都有實作這些特徵。對於你自己定義的結構體與列舉，你需要實作 `PartialEq`，這樣該型別的數值才能判定相等或不相等。你需要實作 `Debug` 來顯示判定失敗時的數值。因為這兩個特徵都是可推導的特徵，就像第五章的範例 5-12 所

寫的那樣，我們通常只要在你定義的結構體或列舉前加上 `#[derive(PartialEq, Debug)]` 的註釋就好。你可以查閱附錄 C 「可推導的特徵」來發現更多可推導的特徵。

加入自訂失敗訊息

你可以寫一個與失敗訊息一同顯示的自訂訊息，作為 `assert!`、`assert_eq!` 與 `assert_ne!` 巨集的選擇性引數。任何指定在必要引數後方的任何引數都會傳給 `format!` 巨集（我們在第八章「使用 + 運算子或 `format!` 巨集串接字串」的段落討論過），所以你可以傳入一個包含 `{}` 佔位符（placeholder）的格式化字串以及其對應的數值。自訂訊息可以用來紀錄判定的意義，當測試失敗時，你可以更清楚知道程式碼的問題。

舉例來說，假設我們有個函式會以收到的名字向人們打招呼，而且我們希望測試我們傳入的名字有出現在輸出：

檔案名稱：src/lib.rs

```
pub fn greeting(name: &str) -> String {
    format!("哈囉{}", name)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn greeting_contains_name() {
        let result = greeting("卡爾");
        assert!(result.contains("卡爾"));
    }
}
```

此函式的要求還沒完全確定，而我們招呼開頭的文字 哈囉 很可能會在之後改變。我們決定當需求改變時，我們不想要得同時更新測試。所以我們不打算檢查 `greeting` 函式回傳的整個數值，我們只需要判定輸出有沒有包含輸入參數。

現在讓我們將錯誤引進程式中吧，將 `greeting` 改成不包含 `name` 然後看看預設的測試失敗會如何呈現：

```
pub fn greeting(name: &str) -> String {
    String::from("哈囉!")
}
```

執行此程式會產生以下錯誤：

```

$ cargo test
Compiling greeter v0.1.0 (file:///projects/greeter)
Finished test [unoptimized + debuginfo] target(s) in 0.91s
Running unittests src/lib.rs (target/debug/deps/greeter-170b942eb5bf5e3a)

running 1 test
test tests::greeting_contains_name ... FAILED

failures:

---- tests::greeting_contains_name stdout ----
thread 'main' panicked at 'assertion failed: result.contains(\"卡爾\")',
src/lib.rs:12:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::greeting_contains_name

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`

```

此結果指出判定失敗以及發生的位置。要是錯誤訊息可以提供我們從 `greeting` 函式取得的數值會更好。讓我們來在測試函式中加入自訂訊息，該訊息會是個格式化字串，並有個佔位符（placeholder）來填入我們從 `greeting` 函式取得的確切數值：

```

#[test]
fn greeting_contains_name() {
    let result = greeting("卡爾");
    assert!(
        result.contains("卡爾"),
        "打招呼時並沒有喊出名稱，其數值為 `{}`",
        result
    );
}

```

現在當我們執行測試，我們能從錯誤訊息得到更多資訊：

```
$ cargo test
Compiling greeter v0.1.0 (file:///projects/greeter)
Finished test [unoptimized + debuginfo] target(s) in 0.93s
Running unittests src/lib.rs (target/debug/deps/greeter-170b942eb5bf5e3a)

running 1 test
test tests::greeting_contains_name ... FAILED

failures:

---- tests::greeting_contains_name stdout ----
thread 'main' panicked at '打招呼時並沒有喊出名稱，其數值為 `哈囉!`',
src/lib.rs:12:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::greeting_contains_name

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

我們可以看到我們實際從測試輸出拿到的數值，這能幫助我們除錯找到實際發生什麼，而不只是預期會是什麼。

透過 `should_panic` 檢查恐慌

除了檢查我們的程式碼有沒有回傳我們預期的正確數值，檢查我們的程式碼有沒有如我們預期處理錯誤條件也是很重要的。舉例來說，考慮我們在第九章範例 9-13 建立的 `Guess` 型別。其他使用 `Guess` 的程式碼保證會拿到數值為 1 到 100 的 `Guess` 實例。我們可以寫個會恐慌的程式，嘗試用範圍之外的數字建立 `Guess` 實例。

為此我們可以加上屬性 `should_panic` 到我們的測試函式。此屬性讓函式的程式碼恐慌時才會通過測試，反之如果函式的程式碼沒有恐慌的話測試就會失敗。

範例 11-8 展示一支檢查 `Guess::new` 是否以我們預期的錯誤條件出錯的測試。

檔案名稱：src/lib.rs


```

pub struct Guess {
    value: i32,
}

impl Guess {
    pub fn new(value: i32) -> Guess {
        if value < 1 || value > 100 {
            panic!("猜測數字必須介於 1 到 100 之間，你輸入的是 {}。", value);
        }

        Guess { value }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    #[should_panic]
    fn greater_than_100() {
        Guess::new(200);
    }
}

```

範例 11-8：測試造成 `panic!` 的條件

我們將 `#[should_panic]` 屬性置於 `#[test]` 屬性之後與測試函式之前。讓我們看看測試通過的結果：

```
$ cargo test
Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
Finished test [unoptimized + debuginfo] target(s) in 0.58s
Running unittests src/lib.rs (target/debug/deps/guessing_game-57d70c3acb738f4d)

running 1 test
test tests::greater_than_100 - should panic ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s


Doc-tests guessing_game

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

看起來不錯！現在我們將錯誤引入程式碼中，移除會讓 `new` 函式在數值大於 100 會恐慌的程式碼：

```
// --省略--
impl Guess {
    pub fn new(value: i32) -> Guess {
        if value < 1 {
            panic!("猜測數字必須介於 1 到 100 之間，你輸入的是 {}。", value);
        }

        Guess { value }
    }
}
```



當我們執行範例 11-8 的測試，它就會失敗：

```
$ cargo test
  Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
  Finished test [unoptimized + debuginfo] target(s) in 0.62s
  Running unittests src/lib.rs (target/debug/deps/guessing_game-57d70c3acb738f4d)

running 1 test
test tests::greater_than_100 - should panic ... FAILED

failures:

---- tests::greater_than_100 stdout ----
note: test did not panic as expected

failures:
    tests::greater_than_100

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

我們在此情況得到的訊息並不是很有用，但是當我們查看測試函式，我們會看到它詮釋了 `# [should_panic]`。這個測試失敗代表測試函式內的程式碼沒有造成恐慌。

使用 `should_panic` 的測試可能會有點模稜兩可。`should_panic` 測試只要有恐慌都會通過，就算是不同於我們預期發生的恐慌而造成的也一樣。要讓測試 `should_panic` 更精準的話，我們可以加上選擇性的 `expected` 參數到 `should_panic` 中。這樣測試就會確保錯誤訊息會包含我們所寫的文字。舉例來說，範例 11-9 更改了 `Guess` 讓 `new` 函式會依據數值太大或大小而有不同的錯誤訊息。

檔案名稱：src/lib.rs

```
// --省略--

impl Guess {
    pub fn new(value: i32) -> Guess {
        if value < 1 {
            panic!(
                "猜測數字必須大於等於 1，取得的數值是 {}。",
                value
            );
        } else if value > 100 {
            panic!(
                "猜測數字必須小於等於 100，取得的數值是 {}。",
                value
            );
        }

        Guess { value }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    #[should_panic(expected = "小於等於 100")]
    fn greater_than_100() {
        Guess::new(200);
    }
}
```

範例 11-9：panic! 的錯誤訊息包含特定子字串才會通過的測試

此測試會通過是因為我們在 `should_panic` 屬性加上的 `expected` 就是 `Guess::new` 函式恐慌時的子字串。我們也可以指定整個恐慌訊息，在此例的話就是 猜測數字必須小於等於 100，取得的數值是 200。你所指定的預期參數取決於該恐慌訊息是獨特或動態的，以及你希望你的測試要多精準。在此例中，恐慌訊息的子訊息就足以確認測試函式中的程式碼會執行 `else if value > 100` 的分支。

為了觀察擁有 `expected` 訊息的 `should_panic` 失敗時會發生什麼事。讓我同樣再次將錯誤引入程式中，將 `if value < 1` 與 `else if value > 100` 的區塊本體對調：

```
if value < 1 {
    panic!(
        "猜測數字必須小於等於 100，取得的數值是 {}。",
        value
    );
} else if value > 100 {
    panic!(
        "猜測數字必須大於等於 1，取得的數值是 {}。",
        value
    );
}
```



這次當我們執行 `should_panic` 測試，它就會失敗：

```
$ cargo test
Compiling guessing_game v0.1.0 (file:///projects/guessing_game)
Finished test [unoptimized + debuginfo] target(s) in 0.66s
Running unittests src/lib.rs (target/debug/deps/guessing_game-57d70c3acb738f4d)

running 1 test
test tests::greater_than_100 - should panic ... FAILED

failures:

---- tests::greater_than_100 stdout ----
thread 'main' panicked at '猜測數字必須大於等於 1，取得的數值是 200。',
src/lib.rs:13:13
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
note: panic did not contain expected string
      panic message: `"猜測數字必須大於等於 1，取得的數值是 200。"`，
      expected substring: `"小於等於 100"`

failures:
    tests::greater_than_100

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

錯誤訊息表示此程式碼的確有如我們預期地恐慌，但是恐慌訊息並沒有包含預期的字串「猜測數字必須小於等於 100」。在此例我們的會得到的恐慌訊息為「猜測數字必須大於等於 1，取得的數值是 200」。這樣我們就能尋找錯誤在哪了！

在測試中使用 `Result<T, E>`

我們目前為止的測試在失敗時都會恐慌。我們也可以寫出使用 `Result<T, E>` 的測試！以下是範例 11-1 的測試，不過重寫成 `Result<T, E>` 的版本並回傳 `Err` 而非恐慌：

```
#[cfg(test)]
mod tests {
    #[test]
    fn it_works() -> Result<(), String> {
        if 2 + 2 == 4 {
            Ok(())
        } else {
            Err(String::from("二加二不等於四"))
        }
    }
}
```

`it_works` 函式現在有個回傳型別 `Result<(), String>`。在函式本體中，我們不再呼叫 `assert_eq!` 巨集，而是當測試成功時回傳 `Ok(())`，當程式失敗時回傳存有 `String` 的 `Err`。

測試中回傳 `Result<T, E>` 讓你可以在測試本體中使用問號運算子，這樣能方便地寫出任何運算回傳 `Err` 時該失敗的測試。

不過你就不能將 `#[should_panic]` 詮釋用在使用 `Result<T, E>` 的測試。要判斷一個操作是否回傳 `Err` 的話，不要在 `Result<T, E>` 數值後加上 `?`，而是改用 `assert!(value.is_err())`。

現在你知道了各種寫測試的方法，讓我們看看執行程式時發生了什麼事，並探索我們可以對 `cargo test` 使用的選項。

控制程式如何執行

就像 `cargo run` 會編譯你的程式碼並執行產生的執行檔，`cargo test` 會在測試模式編譯你的程式碼並執行產生的測試執行檔。`cargo test` 產生的執行檔預設行為會平行執行所有測試，並獲取測試執行時的輸出，讓測試各自的輸出結果不會顯示出來，以更容易讀取相關測試的結果。然而你可以指定命令列選項來改變預設行為。

有些命令列選項用於 `cargo test` 而有些則用於產生的測試執行檔。要分開這兩種引數，你可以先列出要用於 `cargo test` 的引數然後加上 `--` 分隔線來區隔要用於測試執行檔的引數。執行 `cargo test --help` 可以顯示你能用在 `cargo test` 的選項，而執行 `cargo test -- --help` 則會顯示你在 `--` 之後能用的選項。

平行或接續執行測試

當你執行數個測試時，它們預設會使用執行緒（thread）來平行執行。這樣測試可以更快完成，讓你可以從你或其他人的程式碼更快獲得回饋。因為測試是同時一起執行的，請確保你的測試並不依賴其他測試或是共享的狀態。這包含共享環境，像是目前的工作目錄或是環境變數。

舉例來說，假設你的每個測試都會執行一些程式碼，用以在硬碟上產生一個檔案叫做 `test-output.txt`，並將一些資料寫入檔案中。然後每個測試讀取檔案中的資料，並判定該檔案有沒有包含特定的值，而這個值在每個測試都不相同。因為測試同時執行，其中的測試可能覆蓋其他測試寫入與讀取的內容。這樣其他測試就會失敗，並不是因為程式碼不正確，而是因為平行執行時該測試會被其他測試所影響。其中一個解決辦法是確保每個測試都寫入不同的檔案，或者也可以選擇一次只執行一個測試。

如果你不想平行執行測試，或者你想要能更加掌控使用的執行緒數量，你可以傳遞 `--test-threads` 的選項以及你希望在測試執行檔使用的執行緒數量。請看一下以下範例：

```
$ cargo test -- --test-threads=1
```

我們將測試執行緒設為 `1`，告訴程式不要做任何平行化。使用一條執行緒執行測試會比平行執行它們還來的久，但是如果測試有共享狀態的話，它們就會不互相影響到對方了。

顯示函式輸出結果

如果測試通過的話，Rust 的測試函式庫預設會獲取所有印出的標準輸出。舉例來說，如果我們在測試中呼叫 `println!` 然後測試通過的話，我們不會在終端機看到 `println!` 的輸出，我們只會看到一行表達測試通過的訊息。如果測試失敗，我們才會看到所有印出的標準輸出與失敗訊息。

舉例來說，範例 11-10 有個蠢蠢的函式只會印出它的參數並回傳 10，以及一個會通過的測試與一個會失敗的測試。

檔案名稱：src/lib.rs

```
fn prints_and_returns_10(a: i32) -> i32 {  
    println!("我得到的數值為 {}", a);  
    10  
}  
  
#[cfg(test)]  
mod tests {  
    use super::*;  
  
    #[test]  
    fn this_test_will_pass() {  
        let value = prints_and_returns_10(4);  
        assert_eq!(10, value);  
    }  
  
    #[test]  
    fn this_test_will_fail() {  
        let value = prints_and_returns_10(8);  
        assert_eq!(5, value);  
    }  
}
```



範例 11-10：測試會呼叫 `println!` 的函式

當我們使用 `cargo test` 執行這些程式時，我們會看到以下輸出結果：


```
$ cargo test
   Compiling silly-function v0.1.0 (file:///projects/silly-function)
   Finished test [unoptimized + debuginfo] target(s) in 0.58s
   Running unittests src/lib.rs (target/debug/deps/silly_function-160869f38cff9166)

running 2 tests
test tests::this_test_will_fail ... FAILED
test tests::this_test_will_pass ... ok

failures:

---- tests::this_test_will_fail stdout ----
我得到的數值為 8
thread 'main' panicked at 'assertion failed: `(left == right)`
  left: `5`,
 right: `10`', src/lib.rs:19:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::this_test_will_fail

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

注意到此輸出結果我們看不到 我得到的數值為 4，這是當測試通過時印出的訊息。這個輸出被獲取走了。而測試會失敗的標準輸出 我得到的數值為 8 則會出現在測試總結輸出的段落上，並同時顯示錯誤發生的原因。

如果我們希望在測試通過時也能看到印出的數值，我們可以用 `--show-output` 告訴 Rust 也在成功的測試顯示輸出結果。

```
$ cargo test -- --show-output
```

當我們使用 `--show-output` 再次執行範例 11-10 的話，我們就能看到以下輸出：

```
$ cargo test -- --show-output
   Compiling silly-function v0.1.0 (file:///projects/silly-function)
   Finished test [unoptimized + debuginfo] target(s) in 0.60s
   Running unittests src/lib.rs (target/debug/deps/silly_function-160869f38cff9166)

running 2 tests
test tests::this_test_will_fail ... FAILED
test tests::this_test_will_pass ... ok

successes:

---- tests::this_test_will_pass stdout ----
我得到的數值為 4

successes:
    tests::this_test_will_pass

failures:

---- tests::this_test_will_fail stdout ----
我得到的數值為 8
thread 'main' panicked at 'assertion failed: `(left == right)`
  left: `5`,
 right: `10`', src/lib.rs:19:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::this_test_will_fail

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

透過名稱來執行部分測試

有時執行完整所有的測試會很花時間。如果你正專注於程式碼的特定部分，你可能會想要只執行與該程式碼有關的測試。你可以向 `cargo test` 傳遞你想要執行的測試名稱作為引數。

為了解釋如何執行部分測試，我們將為 `add_two` 函式建立三個測試，如範例 11-11 所示，然後選擇其中一個執行。

檔案名稱：src/lib.rs

```

pub fn add_two(a: i32) -> i32 {
    a + 2
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn add_two_and_two() {
        assert_eq!(4, add_two(2));
    }

    #[test]
    fn add_three_and_two() {
        assert_eq!(5, add_two(3));
    }

    #[test]
    fn one_hundred() {
        assert_eq!(102, add_two(100));
    }
}

```

範例 11-11：三個名稱不同的測試

如果我們沒有傳遞任何引數來執行測試的話，如我們前面看過的一樣，所有測試會平行執行：

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.62s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 3 tests
test tests::add_three_and_two ... ok
test tests::add_two_and_two ... ok
test tests::one_hundred ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

執行單獨一個測試

我們可以傳遞任何測試函式的名稱給 `cargo test` 來只執行該測試：

```
$ cargo test one_hundred
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.69s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 1 test
test tests::one_hundred ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 2 filtered out;
finished in 0.00s
```

只有名為 `one_hundred` 的測試會執行，其他兩個的名稱並不符合。測試輸出會在總結的最後顯示 `2 filtered out` 告訴我們除了命令列執行的測試以外，還有更多其他測試。

我們無法用此方式指定多個測試名稱，只有第一個傳給 `cargo test` 有用。但我們有其他方式能執行數個測試。

過濾執行數個測試

我們可以指定部分測試名稱，然後任何測試名稱中有相符的就會被執行。舉例來說，因為我們有兩個測試的名稱都包含 `add`，我們可以透過執行 `cargo test add` 來執行這兩個測試：

```
$ cargo test add
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.61s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 2 tests
test tests::add_three_and_two ... ok
test tests::add_two_and_two ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out;
finished in 0.00s
```

此命令會執行所有名稱中包含 `add` 的測試，並過濾掉 `one_hundred` 的測試名稱。另外測試所在的模組也屬於測試名稱中，所以我們可以透過過濾模組名稱來執行該模組的所有測試。

忽略某些測試除非特別指定

有時候有些特定的測試執行會花非常多時間，所以你可能希望在執行 `cargo test` 時能排除它們。與其列出所有你想要的測試作為引數，你可以在花時間的測試前加上 `ignore` 屬性註釋來排除它們，如以下所示：

檔案名稱：src/lib.rs

```
#[test]
fn it_works() {
    assert_eq!(2 + 2, 4);
}

#[test]
#[ignore]
fn expensive_test() {
    // 會執行一小時的程式碼
}
```

對於想排除的測試，我們在 `#[test]` 之後我們加上 `#[ignore]`。現在當我們執行我們的測試時，`it_works` 會執行但 `expensive_test` 就不會：

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.60s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 2 tests
test expensive_test ... ignored
test it_works ... ok

test result: ok. 1 passed; 0 failed; 1 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.02s
```

`expensive_test` 函式會列在 `ignored`，如果我們希望只執行被忽略的測試，我們可以使用 `cargo test -- --ignored`：

```
$ cargo test -- --ignored
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.61s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 1 test
test expensive_test ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 1 filtered out;
finished in 0.00s

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

透過控制哪些測試能執行，你能夠確保快速執行 `cargo test`。當你有時間能夠執行 `ignored` 的測試時，你可以執行 `cargo test -- --ignored` 來等待結果。如果你想執行所有程式，無論他們是不是被忽略的話，你可以執行 `cargo test -- --include-ignored`。

測試組織架構

如同本章開頭提到的，測試是個複雜的領域，不同的人可能使用不同的術語與組織架構。Rust 社群將測試分為兩大分類術語：單元測試和整合測試。**單元測試**（unit tests）比較小且較專注，傾向在隔離環境中一次只測試一個模組，且能夠測試私有介面。**整合測試**（integration tests）對於你的函式庫來說是個完全外部的程式碼，所以會如其他外部程式碼一樣使用你的程式碼，只能使用公開介面且每個測試可能會有數個模組。

這兩種測試都很重要，且能確保函式庫每個部分能在分別或一起執行的情況下，如你預期的方式運作。

單元測試

單元測試的目的是要在隔離其他程式碼的狀況下測試每個程式碼單元，迅速查明程式碼有沒有如預期或非預期的方式運作。你會將單元測試放在 `src` 目錄中每個你要測試的程式同個檔案下。我們常見的做法是在每個檔案建立一個模組 `tests` 來包含測試函式，並用 `cfg(test)` 來詮釋模組。

測試模組與 `#[cfg(test)]`

測試模組上的 `#[cfg(test)]` 詮釋會告訴 Rust 當你執行 `cargo test` 才會編譯並執行測試程式碼。而不是當你執行 `cargo build`。當你想要建構函式庫時，這能節省編譯時間並降低編譯出的檔案所佔的空間，因為這些測試沒有被包含到。整合測試位於不同目錄，所以它們不需要 `#[cfg(test)]`。但是因為單元測試與程式碼位於相同的檔案下，你需要使用 `#[cfg(test)]` 來指明它們不應該被包含在編譯結果。

回想一下本章節第一個段落中我們建立了一個新專案 `adder`，並用 Cargo 為我們產生以下程式碼：

檔案名稱：`src/lib.rs`

```
#[cfg(test)]
mod tests {
    #[test]
    fn it_works() {
        let result = 2 + 2;
        assert_eq!(result, 4);
    }
}
```

此程式碼是自動產生的測試模組。`cfg` 屬性代表的是 *configuration* 並告訴 Rust 以下項目只有在給予特定配置選項時才會被考慮。在此例中配置選項是 `test`，這是 Rust 提供用來編譯與執行測

試的選項。使用 `cfg` 屬性的話，Cargo 只有在我們透過 `cargo test` 執行測試時才會編譯我們的測試程式碼。這包含此模組能可能需要的輔助函式，以及用 `#[test]` 詮釋的測試函式。

測試私有函式

在測試領域的社群中對於是否應該直接測試私有函式一直存在著爭議，而且有些其他語言會讓測試私有函式變得很困難，甚至不可能。不管你認為哪個論點比較理想，Rust 的隱私權規則還是能讓你測試私有函式。考慮以下範例 11-12 擁有私有函式 `internal_adder` 的程式碼。

檔案名稱：src/lib.rs

```
pub fn add_two(a: i32) -> i32 {
    internal_adder(a, 2)
}

fn internal_adder(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn internal() {
        assert_eq!(4, internal_adder(2, 2));
    }
}
```

範例 11-12：測試私有函式

注意到函式 `internal_adder` 沒有標記為 `pub`。測試也只是 Rust 的程式碼，且 `tests` 也只是另一個模組。如同我們在[參考模組項目的路徑](#)段落討論到的，下層模組的項目可以使用該項目以上的模組。在此測試中，我們透過 `use super::*` 引入 `test` 模組上層的所有項目，所以測試能呼叫 `internal_adder`。如果你不認為私有函式應該測試，Rust 也沒有什麼好阻止你的地方。

整合測試

在 Rust 中，整合測試對你的函式庫來說是完全外部的程式。它們使用你的函式庫的方式與其他程式碼一樣，所以它們只能呼叫屬於函式庫中公開 API 的函式。它們的目的是要測試你的函式庫數個部分一起運作時有沒有正確無誤。單獨運作無誤的程式碼單元可能會在整合時出現問題，所以整合測試的程式碼的涵蓋率也很重要。要建立整合測試，你需要先有個 `tests` 目錄。

tests 目錄

我們在專案目錄最上層在 `src` 旁建立一個 `tests` 目錄。Cargo 知道要從此目錄來尋找整合測試。我們接著就可以建立多少個測試都沒問題，Cargo 會編譯每個檔案成獨立的 crate。

讓我們來建立一個整合測試，將範例 11-12 的程式碼保留在 `src/lib.rs` 檔案中，然後建立一個 `tests` 目錄、一個叫做 `tests/integration_test.rs` 的檔案。你的目錄架構應該要長的像這樣：

```
addler
├── Cargo.lock
├── Cargo.toml
├── src
│   └── lib.rs
└── tests
    └── integration_test.rs
```

請在 `tests/integration_test.rs` 輸入範例 11-13 的程式碼：

檔案名稱：`tests/integration_test.rs`

```
use addler;

#[test]
fn it_adds_two() {
    assert_eq!(4, addler::add_two(2));
}
```

範例 11-13：`addler` crate 中函式的整合測試

`tests` 目錄的每個檔案都是獨立的 crate，所以我們需要將函式庫引入每個測試 crate 的作用域中。因此我們在程式最上方加了 `use addler`，這在單元測試是不需要的。

我們不用對 `tests/integration_test.rs` 的任何程式碼詮釋 `#[cfg(test)]`。Cargo 會特別對待 `tests` 目錄並只在我們執行 `cargo test` 時，編譯此目錄的檔案。現在請執行 `cargo test`：

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 1.31s
Running unittests src/lib.rs (target/debug/deps/adder-1082c4b063a8fbe6)

running 1 test
test tests::internal ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Running tests/integration_test.rs (target/debug/deps/integration_test-
1082c4b063a8fbe6)

running 1 test
test it_adds_two ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

輸出結果中有三個段落，包含單元測試、整合測試與技術文件測試。要是有個段落的任何一個測試失敗的話，接下來的段落就不會執行。舉例來說，如果單元測試失敗了，我們就不會看到整合測試與技術文件測試的輸出，因為它們只會在所有單元測試都通過之後才會執行。

第一個段落的單元測試與我們看過的相同：每行會是每個單元測試（在此例是我們在範例 11-12 寫的 `internal`）最後附上單元測試的總結。

整合測試段落從 `Running tests/integration_test.rs` 開始，接著每行會是每個整合測試的測試函式，最後在 `Doc-tests adder` 段落開始前的那一行則是整合測試的總結結果。

每個整合測試檔案會有自己的段落，如果我們在 `tests` 目錄加入更多檔案的話，就會出現更多整合測試段落。

我們一樣能用測試函式的名稱來作為 `cargo test` 的引數，來執行特定整合測試。要執行特定整合測試檔案內的所有測試，可以用 `--test` 作為 `cargo test` 的引數並加上檔案名稱：

```
$ cargo test --test integration_test
   Compiling adder v0.1.0 (file:///projects/adder)
   Finished test [unoptimized + debuginfo] target(s) in 0.64s
   Running tests/integration_test.rs (target/debug/deps/integration_test-82e7799c1bc62298)

running 1 test
test it_adds_two ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

此命令會只執行 *tests/integration_test.rs* 檔案內的測試。

整合測試的子模組

隨著你加入的整合測試越多，你可能會想要在 *tests* 目錄下產生更多檔案來協助組織它們。舉例來說，你可以用測試函式測試的功能來組織它們。如同稍早提到的，*tests* 目錄下的每個檔案都會編譯成自己獨立的 crate，這有助於建立不同的作用域，這就像是使用者使用你的 crate 的可能環境。然而這也代表 *tests* 目錄的檔案不會和 *src* 的檔案行為一樣，也就是你在第七章學到如何拆開程式碼成模組與檔案的部分。

當你希望擁有一些能協助數個整合測試檔案的輔助函式，並遵循第七章的「[將模組拆成不同檔案](#)」段落來提取它們到一個通用模組時，你就會發現 *tests* 目錄下的檔案行為是不同的。舉例來說，我們建立了 *tests/common.rs* 並寫了一個函式 `setup`，然後我們希望 `setup` 能被不同測試檔案的數個測試函式呼叫：

檔案名稱：*tests/common.rs*

```
pub fn setup() {
    // 在此設置測試函式庫會用到的程式碼
}
```

當我們再次執行程式時，我們會看到測試輸出多了一個 *common.rs* 檔案的段落，就算該檔案沒有包含任何測試函式，而且我們也還沒有在任何地方呼叫 `setup` 函式：

```
$ cargo test
Compiling adder v0.1.0 (file:///projects/adder)
Finished test [unoptimized + debuginfo] target(s) in 0.89s
Running unittests src/lib.rs (target/debug/deps/adder-92948b65e88960b4)

running 1 test
test tests::internal ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Running tests/common.rs (target/debug/deps/common-92948b65e88960b4)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Running tests/integration_test.rs (target/debug/deps/integration_test-
92948b65e88960b4)

running 1 test
test it_adds_two ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests adder

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

讓 `common` 出現在測試結果並顯示 `running 0 tests` 並不是我們想做的事。我們只是想要分享一些程式碼給其他整合測試檔案而已。

要防止 `common` 出現在測試輸出，我們不該建立 `tests/common.rs`，而是要建立 `tests/common/mod.rs`。專案目錄現在應該要長的像這樣：

```
├── Cargo.lock
├── Cargo.toml
├── src
│   └── lib.rs
└── tests
    ├── common
    │   └── mod.rs
    └── integration_test.rs
```

這是另一個 Rust 知道的舊版命名形式，我們在第七章的「[其他種的檔案路徑](#)」段落有提過。這樣命名檔案的話會告訴 Rust 不要將 `common` 模組視為整合測試檔案。當我們將 `setup` 函式程式碼移到 `tests/common/mod.rs` 並刪除 `tests/common.rs` 檔案時，原本的段落就不會再出現在測試輸出。`tests` 目錄下子目錄的檔案不會被編譯成獨立 crate 或在測試輸出顯示段落。

在我們建立 `tests/common/mod.rs` 之後，我們可以將它以模組的形式用在任何整合測試檔案中。以下是在 `tests/integration_test.rs` 的 `it_adds_two` 測試中呼叫函式 `setup` 的範例：

檔案名稱：`tests/integration_test.rs`

```
use adder;

mod common;

#[test]
fn it_adds_two() {
    common::setup();
    assert_eq!(4, adder::add_two(2));
}
```

注意到 `mod common;` 的宣告與我們在範例 7-21 說明的模組宣告方式一樣。之後在測試函式中，我們就可以呼叫函式 `common::setup()`。

執行檔 Crate 的整合測試

如果我們的專案只包含 `src/main.rs` 檔案的執行檔 crate 而沒有 `src/lib.rs` 檔案的話，我們無法在 `tests` 目錄下建立整合測試，也無法將 `src/main.rs` 檔案中定義的函式透過 `use` 陳述式引入作用域。只有函式庫 crate 能公開函式給其他 crate 使用，執行檔 crate 只用於獨自執行。

這也是為何 Rust 專案為執行檔提供直白的 `src/main.rs` 檔案並允許呼叫 `src/lib.rs` 檔案中的邏輯程式碼。使用這樣子的架構的話，整合測試可以透過 `use` 來測試函式庫 crate，並讓重點功能可以公開使用。如果重點功能可以運作的話，那 `src/main.rs` 檔案中剩下的程式碼部分也能夠如期執行，而這一小部分就不必特地做測試。

總結

Rust 的測試功能提供了判定程式碼怎樣才算正常運作的方法，以確保它能以你預期的方式運作，就算當你做了改變時也是如此。單元測試分別測試函式庫中每個不同的部分，且能測試私有實作細節。整合測試檢查函式庫數個部分一起執行時是否正確無誤，且它們使用函式庫公開 API 來測試程式碼的行為與外部程式碼使用的方式一樣。雖然 Rust 型別系統與所有權規則能避免某些種類的程式錯誤，測試還是減少邏輯程式錯誤的重要辦法，讓你的程式碼能如預期行為運作。

讓我們統整此章節以及之前的章節所學到的知識來寫一支專案吧！

I/O 專案：建立一個命令列程式

本章節用來回顧你目前學過的許多技能，並探索些標準函式庫中的更多功能。我們會來建立個命令列工作來處理檔案與命令列輸入 / 輸出，以此練習些你已經掌握的 Rust 概念。

Rust 的速度、安全、單一執行檔輸出與跨平台支援使其成為建立命令列工具的絕佳語言。所以在我們的專案中，我們要寫出我們自己的經典命令列工具 `grep` (**g**lobally search a **r**egular **e**xpression and **p**rint)。在最簡單的使用場合中，`grep` 會搜尋指定檔案中的指定字串。為此 `grep` 會接收一個檔案名稱與一個字串作為其引數。然後它會讀取檔案、在該檔案中找到包含字串引數的行數，並印出這些行數。

在過程中，我們會展示如何讓我們的命令列工具和其他許多命令列工具一樣使用終端機的功能。我們會讀取一個環境變數的數值來讓使用者可以配置此工具的行為。我們還會將錯誤訊息在控制台中的標準錯誤（`stderr`）顯示而非標準輸出（`stdout`）。所以舉例來說，使用者可以將成功的標準輸出重新導向至一個檔案，並仍能在螢幕上看到錯誤訊息。

其中一位 Rust 社群成員 Andrew Gallant 已經有建立個功能完善且十分迅速的 `grep` 版本，叫做 `ripgrep`。相比之下，我們的版本會相對簡單許多，但此章節能給你些背景知識，來幫你理解像是 `ripgrep` 等真實專案。

我們的 `grep` 專案會組合你所學過的各種概念：

- 組織程式碼（使用你在[第七章](#)所學的模組）
- 使用向量與字串（[第八章](#)的集合）
- 錯誤處理（[第九章](#)）
- 合理的使用特徵與生命週期（[第十章](#)）
- 測試（[第十一章](#)）

我們還會簡單介紹閉包、疊代器與特徵物件，這些在[第十三章](#)與[第十七章](#)會做詳細介紹。

接受命令列引數

一如往常我們用 `cargo new` 建立新的專案，我們將我們的專案命名為 `minigrep` 來與很可能已經在你系統中的 `grep` 工具做區別。

```
$ cargo new minigrep
    Created binary (application) `minigrep` project
$ cd minigrep
```

第一項任務是要讓 `minigrep` 能接收兩個命令列引數：檔案路徑與欲搜尋的字串。也就是說，我們想要能夠使用 `cargo run` 加上兩條連字號來指示接下來的引號用於我們的程式而不是 `cargo`，然後輸入欲搜尋的字串與要被搜尋的檔案路徑來執行程式，如以下所示：

```
$ cargo run -- searchstring example-filename.txt
```

但現在由 `cargo new` 產生的程式還無法處理我們給予的引數。crates.io 有些函式庫可以幫助程式接收命令列中的引數，但有鑑於你要學習此概念，讓我們親自來實作一個。

讀取引數數值

要讓 `minigrep` 能夠讀取我們傳入的命令列引數數值，我們需要使用 Rust 標準函式庫中提供的 `std::env::args` 函式。此函式會回傳一個包含我們傳給 `minigrep` 的命令列引數的疊代器 (iterator)。我們會在第十三章詳細解釋疊代器。現在你只需要知道疊代器的兩項重點：疊代器會產生一系列的數值，然後我們可以對疊代器呼叫 `collect` 方法來將其轉換成像是向量的集合，來包含疊代器產生的所有元素。

範例 21-1 的程式碼能讓你的 `minigrep` 程式能夠讀取任何傳入的命令列引數，然後收集數值成一個向量。

檔案名稱：src/main.rs

```
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();
    dbg!(args);
}
```

範例 12-1：收集命令列引數至向量中並顯示它們

首先我們透過 `use` 陳述式將 `std::env` 模組引入作用域，讓我們可以使用它的 `args` 函式。注意到 `std::env::args` 函式位於兩層模組下。如同我們在第七章談過的，如果我們要用的函式模組路徑超過一層以上的話，我們選擇將上層模組引入作用域中，而不是函式本身。這樣的話，我們可以輕鬆使用 `std::env` 中的其他函式。而且這也比直接加上 `use std::env::args` 然後只使用 `args` 來呼叫函式還要明確些，因為 `args` 容易被誤認成是由目前模組定義的函式。

args 函式與無效的 Unicode

值得注意的是如果任何引數包含無效 Unicode 的話，`std::env::args` 就會恐慌。如果你的程式想要接受包含無效 Unicode 引數的話，請改使用 `std::env::args_os`。該函式回傳會產生 `OsString` 數值的疊代器，而非 `String` 數值。我們出於簡單方便所以在此使用 `std::env::args`，因為 `OsString` 在不同平台中數值會有所差異，且會比 `String` 數值還要難處理。

我們在 `main` 中的第一行呼叫 `env::args`，然後馬上使用 `collect` 來將疊代器轉換成向量，這會包含疊代器產生的所有數值。我們可以使用 `collect` 函式來建立許多種集合，所以我們顯式詮釋 `args` 的型別來指定我們想要字串向量。雖然我們很少需要在 Rust 中詮釋型別，`collect` 是其中一個你常常需要詮釋的函式，因為 Rust 無法推斷出你想要何種集合。

最後，我們使用除錯巨集來顯示向量。讓我們先嘗試不用引數來執行程式碼，再用兩個引數來執行：

```
$ cargo run
  Compiling minigrep v0.1.0 (file:///projects/minigrep)
  Finished dev [unoptimized + debuginfo] target(s) in 0.61s
  Running `target/debug/minigrep`
[src/main.rs:5] args = [
  "target/debug/minigrep",
]
```

```
$ cargo run -- needle haystack
  Compiling minigrep v0.1.0 (file:///projects/minigrep)
  Finished dev [unoptimized + debuginfo] target(s) in 1.57s
  Running `target/debug/minigrep needle haystack`
[src/main.rs:5] args = [
  "target/debug/minigrep",
  "needle",
  "haystack",
]
```

值得注意的是向量中第一個數值為 `"target/debug/minigrep"`，這是我們的執行檔名稱。這與 C 的引數列表行為相符，讓程式在執行時能使用它們被呼叫的名稱路徑。存取程式名稱通常是很實用的。

的，像是你能將它顯示在訊息中，或是依據程式被呼叫的命令列別名來改變程式的行為。但考慮本章節的目的，我們會忽略它並只儲存我們想要的兩個引數。

將引數數值儲存至變數

目前程式能夠取得命令列引數指定的數值。現在我們想要將這兩個引數存入變數中，讓我們可以在接下來的程式中使用數值，如範例 12-2 所示。

檔案名稱：src/main.rs

```
use std::env;

fn main() {
    let args: Vec<String> = env::args().collect();

    let query = &args[1];
    let file_path = &args[2];

    println!("搜尋 {}", query);
    println!("目標檔案為 {}", file_path);
}
```

範例 12-2：建立變數來儲存搜尋引數與檔案路徑引數

如我們印出向量時所看到的，向量的第一個數值 `args[0]` 會是程式名稱，所以我們從引數 1 開始。`minigrep` 接收的第一個引數會是我們要搜尋的字串，所以我們將第一個引數的參考賦值給變數 `query`。第二個引數會是檔案路徑，所以我們將第二個引數的參考賦值給 `file_path`。

我們暫時印出這些變數的數值來證明程式碼運作無誤。讓我們用引數 `test` 與 `sample.txt` 來再次執行程式：

```
$ cargo run -- test sample.txt
   Compiling minigrep v0.1.0 (file:///projects/minigrep)
   Finished dev [unoptimized + debuginfo] target(s) in 0.0s
   Running `target/debug/minigrep test sample.txt`
搜尋 test
目標檔案為 sample.txt
```

很好，程式能執行！我們想要的引數數值都有儲存至正確的變數中。之後我們會對特定潛在錯誤的情形來加上一些錯誤處理，像是當使用者沒有提供引數的情況。現在我們先忽略這樣的情況，並開始加上讀取檔案的功能。

讀取檔案

現在我們要加上能夠讀取 `file_path` 中命令列引數指定的檔案功能。首先我們需要有個檔案範本能讓我們測試，我們可以建立一個文字檔，其中由數行重複的單字組成少量文字。範例 12-3 Emily Dickinson 的詩就是不錯的選擇！在專案根目錄建立一個檔案叫做 *poem.txt*，然後輸入此詩「I'm Nobody! Who are you?」

檔案名稱：poem.txt

```
I'm nobody! Who are you?  
Are you nobody, too?  
Then there's a pair of us - don't tell!  
They'd banish us, you know.  
  
How dreary to be somebody!  
How public, like a frog  
To tell your name the livelong day  
To an admiring bog!
```

範例 12-3：以 Emily Dickinson 的詩作為絕佳測試範本

有了這些文字，接著修改 *src/main.rs* 來加上讀取檔案的程式碼，如範例 12-4 所示。

檔案名稱：src/main.rs

```
use std::env;  
use std::fs;  
  
fn main() {  
    // --省略--  
    println!("目標檔案為 {}", file_path);  
  
    let contents = fs::read_to_string(file_path)  
        .expect("應該要能夠讀取檔案");  
  
    println!("文字內容：\n{contents}");  
}
```

範例 12-4：讀取第二個引數指定的檔案內容

首先，我們加上另一個 `use` 陳述式來將標準函式庫中的另一個相關部分引入：我們需要 `std::fs` 來處理檔案。

在 `main` 中，我們加上新的陳述式：`fs::read_to_string` 會接收 `file_path`、開啟該檔案並回傳檔案內容的 `Result<String>`。

在陳述式之後，我們再次加上暫時的 `println!` 陳述式來在讀取檔案之後，顯示 `contents` 的數值，讓我們能檢查程式目前運作無誤。

讓我們用任何字串作為第一個命令列引數（因為我們還沒實作搜尋的部分）並與 `poem.txt` 檔案作為第二個引數來執行此程式碼：

```
$ cargo run -- the poem.txt
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.0s
Running `target/debug/minigrep the poem.txt`
搜尋 the
目標檔案為 poem.txt
文字內容：
I'm nobody! Who are you?
Are you nobody, too?
Then there's a pair of us - don't tell!
They'd banish us, you know.

How dreary to be somebody!
How public, like a frog
To tell your name the livelong day
To an admiring bog!
```

很好！程式碼有讀取並印出檔案內容。但此程式碼有些缺陷。`main` 函式負責太多事情了，通常如果每個函式都只負責一件事的話，函式才能清楚直白且易於維護。另一個問題是我們盡可能地處理錯誤。由於程式還很小，此缺陷不算什麼大問題，但隨著程式增長時，這會越來越難清楚地修正。在開發程式時盡早重構是很好的做法，因為重構少量的程式碼會比較簡單。接下來就讓我們開始吧。

透過重構來改善模組性與錯誤處理

為了改善我們的程式，我們需要修正四個問題，這與程式架構與如何處理潛在錯誤有關。首先，我們的 `main` 函式會處理兩件任務：它得解析引數並讀取檔案。隨著我們的程式增長，`main` 函式中要處理的任務就會增加。要是一個函式有這麼多責任，它就會越來越難理解、越難測試並且難在不破壞其他部分的情況下做改變。我們最好能將不同功能拆開，讓每個函式只負責一項任務。

而這也和第二個問題有關：雖然 `query` 與 `file_path` 是我們程式的設置變數，而變數 `contents` 則用於程式邏輯。隨著 `main` 增長，我們會需要引入越多變數至作用域中。而作用域中有越多變數，我們就越難追蹤每個變數的用途。我們最好是將設置變數集結成一個結構體，讓它們的用途清楚明白。

第三個問題是當讀取檔案失敗時，我們使用 `expect` 來印出錯誤訊息，但是錯誤訊息只印出 `應該要能夠讀取檔案`。讀取檔案可以有好幾種失敗的方式：舉例來說，檔案可能不存在，或是我們可能沒有權限能開啟它。目前不管原因為何，我們都只印出相同的錯誤訊息，這並沒有給使用者足夠的資訊！

第四，我們重複使用 `expect` 來處理不同錯誤，而如果有使用者沒有指定足夠的引數來執行程式的話，他們會從 Rust 獲得 `index out of bounds` 的錯誤，這並沒有清楚解釋問題。最好是所有的錯誤處理程式碼都可以位於同個地方，讓未來的維護者只需要在此處來修改錯誤處理的程式碼。將所有錯誤處理的程式碼置於同處也能確保我們能提供對終端使用者有意義的訊息。

讓我們來重構專案以解決這四個問題吧。

分開執行檔專案的任務

`main` 函式負責多數任務的組織分配問題在許多執行檔專案中都很常見。所以 Rust 社群開發出了一種流程，這在當 `main` 開始變大時，能作為分開執行檔程式中任務的指導原則。此流程有以下步驟：

- 將你的程式分成 `main.rs` 與 `lib.rs` 並將程式邏輯放到 `lib.rs`。
- 只要你的命令列解析邏輯很小，它可以留在 `main.rs`。
- 當命令行解析邏輯變得複雜時，就將其從 `main.rs` 移至 `lib.rs`。

在此流程之後的 `main` 函式應該要只負責以下任務：

- 透過引數數值呼叫命令列解析邏輯
- 設置任何其他配置
- 呼叫 `lib.rs` 中的 `run` 函式
- 如果 `run` 回傳錯誤的話，處理該錯誤

此模式用於分開不同任務：`main.rs` 處理程式的執行，然後 `lib.rs` 處理眼前的所有任務邏輯。因為你無法直接測試 `main`，此架構讓你能測試所有移至 `lib.rs` 的程式函式邏輯。留在 `main.rs` 的程式

碼會非常小，所以容易直接用閱讀來驗證。讓我們用此流程來重構程式吧。

提取引數解析器

我們會提取解析引數的功能到一個 `main` 會呼叫的函式中，以將命令列解析邏輯妥善地移至 `src/lib.rs`。範例 12-5 展示新的 `main` 會呼叫新的函式 `parse_config`，而此函式我們先暫時留在 `src/main.rs`。

檔案名稱：`src/main.rs`

```
fn main() {
    let args: Vec<String> = env::args().collect();

    let (query, file_path) = parse_config(&args);

    // --省略--
}

fn parse_config(args: &[String]) -> (&str, &str) {
    let query = &args[1];
    let file_path = &args[2];

    (query, file_path)
}
```

範例 12-5：從 `main` 提取 `parse_config` 函式

我們仍然收集命令列引數至向量中，但不同於在 `main` 函式中將索引 1 的引數數值賦值給變數 `query` 且將索引 2 的引數數值賦值給變數 `file_path`，我們將整個向量傳至 `parse_config` 函式。`parse_config` 函式會擁有決定哪些引數要賦值給哪些變數的邏輯，並將數值回傳給 `main`。我們仍然在 `main` 中建立變數 `query` and `file_path`，但 `main` 不再負責決定命令列引數與變數之間的關係。

此重構可能對我們的小程式來說有點像是殺雞焉用牛刀，但是我們正一小步一小步地累積重構。做了這項改變後，請再次執行程式來驗證引數解析有沒有正常運作。經常檢查你的進展是很好的，這能幫助你找出問題發生的原因。

集結配置數值

我們可以再進一步改善 `parse_config` 函式。目前我們回傳的是元組，但是我們馬上又將元組拆成獨立部分。這是個我們還沒有建立正確抽象的信號。

另外一個告訴我們還有改善空間的地方是 `parse_config` 名稱中的 `config`，這指示我們回傳的兩個數值是相關的，且都是配置數值的一部分。我們現在沒有確實表達出這樣的資料結構，而只有將兩個數值組合成一個元組而已，我們可以將這兩個數值存入一個結構體，並對每個結構體欄位給

予有意義的名稱。這樣做能讓未來的維護者可以清楚知道這些數值的不同與關聯，以及它們的用途。

範例 12-6 改善了 `parse_config` 函式。

檔案名稱：src/main.rs

```
fn main() {
    let args: Vec<String> = env::args().collect();

    let config = parse_config(&args);

    println!("搜尋 {}", config.query);
    println!("目標檔案為 {}", config.file_path);

    let contents = fs::read_to_string(config.file_path)
        .expect("應該要能夠讀取檔案");

    // --省略--
}

struct Config {
    query: String,
    file_path: String,
}

fn parse_config(args: &[String]) -> Config {
    let query = args[1].clone();
    let file_path = args[2].clone();

    Config { query, file_path }
}
```

範例 12-6：重構 `parse_config` 來回傳 `Config` 結構體實例

我們定義了一個結構體 `Config` 其欄位有 `query` 與 `file_path`。`parse_config` 的簽名現在指明它會回傳一個 `Config` 數值。在 `parse_config` 的本體中，我們原先回傳 `args` 中 `String` 數值參考的字串切片，現在我們定義 `Config` 來包含具所有權的 `String` 數值。`main` 中的 `args` 變數是引數數值的擁有者，而且只是借用它們給 `parse_config` 函式，這意味著如果 `Config` 嘗試取得 `args` 中數值的所有權的話，我們會違反 Rust 的借用規則。

我們可以用許多不同的方式來管理 `String` 的資料，最簡單（卻較無效率）的方式是對數值呼叫 `clone` 方法。這會複製整個資料讓 `Config` 能夠擁有，這會比參考字串資料還要花時間與記憶體。然而克隆資料讓我們的程式碼比較直白，因為在此情況下我們就不需要管理參考的生命週期，犧牲一點效能以換取簡潔性是值得的。

使用 `clone` 的權衡取舍

由於 `clone` 會有運行時消耗，所以許多 Rustaceans 傾向於避免使用它來修正所有權問題。在第十三章中，你會學到如何更有效率的處理這種情況。但現在我們可以先複製字串來繼續進行下去，因為你只複製了一次，而且檔案路徑與搜尋字串都算很小。先寫出較沒有效率但可執行的程式會比第一次就要過分優化還來的好。隨著你對 Rust 越熟練，你的確就可以從有效率的解決方案開始，但現在呼叫 `clone` 是完全可以接受的。

我們更新 `main` 來將 `parse_config` 回傳的 `Config` 實例儲存至 `config` 變數中，並更新之前分別使用變數 `query` 與 `file_path` 的程式碼段落來改使用 `Config` 結構體中的欄位。

現在我們的程式碼更能表達出 `query` 與 `file_path` 是相關的，而且它們的目的是配置程式的行為。任何使用這些數值的程式碼都會從 `config` 實例中的欄位名稱知道它們的用途。

建立 `Config` 的建構子

目前我們將負責解析命令列引數的邏輯從 `main` 移至 `parse_config` 函式。這樣做能幫助我們理解 `query` 與 `file_path` 數值是相關的，且此關係應該要能在我們的程式碼中表達出來。然後我們增加了結構體 `Config` 來描述 `query` 與 `file_path` 的相關性，並在 `parse_config` 函式中將數值名稱作為結構體欄位名稱來回傳。

所以現在 `parse_config` 函式的目的是要建立 `Config` 實例，我們可以將 `parse_config` 從普通的函式變成與 `Config` 結構體相關連的 `new` 函式。這樣做能讓程式碼更符合慣例。我們可以對像是 `String` 等標準函式庫中的型別呼叫 `String::new` 來建立實例。同樣地，透過將 `parse_config` 改為 `Config` 的關聯函式 `new`，我們可以透過呼叫 `Config::new` 來建立 `Config` 的實例。範例 12-7 正是我們要作出的改變。

檔案名稱：src/main.rs


```

fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::new(&args);

    // --省略--
}

// --省略--

impl Config {
    fn new(args: &[String]) -> Config {
        let query = args[1].clone();
        let file_path = args[2].clone();

        Config { query, file_path }
    }
}

```

範例 12-7：變更 `parse_config` 成 `Config::new`

我們更新了 `main` 原先呼叫 `parse_config` 的地方來改呼叫 `Config::new`。我們變更了 `parse_config` 的名稱成 `new` 並移入 `impl` 區塊中，讓 `new` 成為 `Config` 的關聯函式。請嘗試再次編譯此程式碼來確保它能執行。

修正錯誤處理

現在我們要來修正錯誤處理。回想一下要是 `args` 向量中的項目太少的話，嘗試取得向量中索引 1 或索引 2 的數值的話可能就會導致程式恐慌。試著不用任何引數執行程式的話，它會產生以下結果：

```

$ cargo run
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.0s
Running `target/debug/minigrep`
thread 'main' panicked at 'index out of bounds: the len is 1 but the index is 1', src/main.rs:27:21
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

```

`index out of bounds: the len is 1 but the index is 1` 這行是給程式設計師看的錯誤訊息。這無法協助我們的終端使用者理解他們該怎麼處理。讓我們來修正吧。

改善錯誤訊息

在範例 12-8 中，我們在 `new` 函式加上了一項檢查來驗證 `slice` 是否夠長，接著才會取得索引 1 和 2。如果 `slice` 不夠長的話，程式就會恐慌並顯示更清楚的錯誤訊息。

檔案名稱：src/main.rs

```
// --省略--
fn new(args: &[String]) -> Config {
    if args.len() < 3 {
        panic!("引數不足");
    }
    // --省略--
}
```

範例 12-8：新增對引數數量的檢查

此程式碼類似於我們在範例 9-13 寫的 `Guess::new` 函式，在那裡當 `value` 超出有效數值的範圍時，我們就呼叫 `panic!`。然而在此我們不是檢查數值的範圍，而是檢查 `args` 的長度是否至少為 3，然後函式剩餘的段落都能在假設此條件成立情況下正常執行。如果 `args` 的項目數量少於三的話，此條件會為真，然後我們就會立即呼叫 `panic!` 巨集來結束程式。

在 `new` 多了這些額外的程式碼之後，讓我們不用任何引數再次執行程式，來看看錯誤訊息為何：

```
$ cargo run
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.0s
Running `target/debug/minigrep`
thread 'main' panicked at '引數不足', src/main.rs:26:13
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

這樣的輸出就好多了，我們現在有個合理的錯誤訊息。然而我們還是顯示了一些額外資訊給使用者。也許在此使用範例 9-13 的技巧並不是最好的選擇，如同第九章所提及的，`panic!` 的呼叫比較屬於程式設計問題，而不是使用問題。我們可以改使用第九章的其他技巧，像是回傳 `Result` 來表達是成功還是失敗。

回傳 `Result` 而非呼叫 `panic!`

我們可以回傳 `Result` 數值，在成功時包含 `Config` 的實例並在錯誤時描述問題原因。我們也將函式名稱從 `new` 改為 `build`，因為許多開發者通常預期 `new` 函式不會失敗。當 `Config::build` 與 `main` 溝通時，我們可以使用 `Result` 型別來表達這裡有問題發生。然後我們改變 `main` 來將 `Err` 變體轉換成適當的錯誤訊息給使用者，而不是像呼叫 `panic!` 時出現圍繞著 `thread 'main'` 與 `RUST_BACKTRACE` 的文字。

範例 12-9 顯示我們得改變 `Config::build` 的回傳值並讓函式本體回傳 `Result`。注意到這還不能編譯，直到我們也更新 `main` 為止，這會在下個範例解釋。

檔案名稱：src/main.rs

```
impl Config {
    fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("引數不足");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        Ok(Config { query, file_path })
    }
}
```



範例 12-9：從 `Config::build` 回傳 `Result`

我們的 `build` 函式現在會回傳 `Result`，在成功時會有 `Config` 實例，而在錯誤時會有個 `&'static str`。我們的錯誤值永遠會是有 `'static` 生命週期的字串字面值。

我們在 `build` 函式本體作出了兩項改變：不同於呼叫 `panic!`，當使用者沒有傳遞足夠引數時，我們現在會回傳 `Err` 數值。此外我們也將 `Config` 封裝進 `Ok` 作為回傳值。這些改變讓函式能符合其新的型別簽名。

從 `Config::build` 回傳 `Err` 數值讓 `main` 函式能處理 `build` 函式回傳的 `Result` 數值，並明確地在錯誤情況下離開程序。

呼叫 `Config::build` 並處理錯誤

為了能處理錯誤情形並印出對使用者友善的訊息，我們需要更新 `main` 來處理 `Config::build` 回傳的 `Result`，如範例 12-10 所示。我們還要負責用一個非零的錯誤碼來離開命令列工具，這原先是 `panic!` 會處理的，現在我們得自己實作。非零退出狀態是個常見信號，用來告訴呼叫程式的程序，該程式離開時有個錯誤狀態。

檔案名稱：src/main.rs

```
use std::process;

fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::build(&args).unwrap_or_else(|err| {
        println!("解析引數時出現問題：{err}");
        process::exit(1);
    });

    // --省略--
```

範例 12-10：如果建立新的 `Config` 失敗時會用錯誤碼離開

在此範例中，我們使用一個還沒詳細介紹的方法 `unwrap_or_else`，這定義在標準函式庫的 `Result<T, E>` 中。使用 `unwrap_or_else` 讓我們能定義一些自訂的非 `panic!` 錯誤處理。如果 `Result` 數值為 `Ok`，此方法行為就類似於 `unwrap`，它會回傳 `Ok` 所封裝的內部數值。然而，如果數值為 `Err` 的話，此方法會呼叫閉包（closure）內的程式碼，這會是由我們所定義的匿名函式並作為引數傳給 `unwrap_or_else`。我們會在第十三章詳細介紹閉包。現在你只需要知道 `unwrap_or_else` 會傳遞 `Err` 的內部數值，在此例中就是我們在範例 12-9 新增的靜態字串「引數不足」，將此數值傳遞給閉包中兩條直線之間的 `err` 引數。閉包內的程式碼就可以在執行時使用 `err` 數值。

我們新增了一行 `use` 來將標準函式庫中的 `process` 引入作用域。在錯誤情形下要執行的閉包程式碼只有兩行：我們印出 `err` 數值並呼叫 `process::exit`。`process::exit` 函式會立即停止程式並回傳給予的數字來作為退出狀態碼。這與範例 12-8 我們使用 `panic!` 來處理的方式類似，但我們不再顯示多餘的輸出結果。讓我們試試看：

```
$ cargo run
  Compiling minigrep v0.1.0 (file:///projects/minigrep)
  Finished dev [unoptimized + debuginfo] target(s) in 0.48s
  Running `target/debug/minigrep`
解析引數時出現問題：引數不足
```

很好！這樣的輸出結果對使用者友善多了。

從 `main` 提取邏輯

現在我們完成配置解析的重構了，接下來輪到程式邏輯了。如同我們在「[分開執行檔專案的任務](#)」中所提及的，我們會提取一個函式叫做 `run`，這會存有目前 `main` 函式中除了設置配置或處理錯誤以外的所有邏輯。當我們完成後，`main` 會變得非常簡潔，且能輕鬆用肉眼來驗證，然後我就能對所有其他邏輯進行測試了。

範例 12-11 提取了 `run` 函式。目前我們在提取函式時，會逐步作出小小的改善。我們仍然在 `src/main.rs` 底下定義函式。

檔案名稱：`src/main.rs`

```

fn main() {
    // --省略--

    println!("搜尋 {}", config.query);
    println!("目標檔案為 {}", config.file_path);

    run(config);
}

fn run(config: Config) {
    let contents = fs::read_to_string(config.file_path)
        .expect("應該要能夠讀取檔案");

    println!("文字內容：\n{contents}");
}

// --省略--

```

範例 12-11：提取 `run` 函式來包含剩餘的程式邏輯

`run` 現在會包含 `main` 中從讀取文件開始的所有剩餘邏輯。`run` 函式會接收 `Config` 實例來作為引數。

從 `run` 函式回傳錯誤

隨著剩餘程式邏輯都移至 `run` 函式，我們可以像範例 12-9 的 `Config::build` 一樣來改善錯誤處理。不同於讓程式呼叫 `expect` 來恐慌，當有問題發生時，`run` 函式會回傳 `Result<T, E>`。這能讓我們進一步穩固 `main` 中對使用者友善的處理錯誤邏輯。範例 12-12 展示我們對 `run` 的簽名與本體所需要做的改變。

檔案名稱：src/main.rs

```

use std::error::Error;

// --省略--

fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let contents = fs::read_to_string(config.file_path)?;

    println!("文字內容：\n{contents}");

    Ok(())
}

```

範例 12-12：變更 `run` 函式來回傳 `Result`

我們在此做了三項明顯的修改。首先，我們改變了 `run` 函式的回傳型別為 `Result<(), Box<dyn Error>>`，此函式之前回傳的是單元型別 `()`，而它現在仍作為 `Ok` 條件內的數值。

對於錯誤型別，我們使用特徵物件 (**trait object**) `Box<dyn Error>` (然後我們在最上方透過 `use` 陳述式來將 `std::error::Error` 引入作用域)。我們會在[第十七章](#)討論特徵物件。現在你只需要知道 `Box<dyn Error>` 代表函式會回傳有實作 `Error` 特徵的型別，但我們不必指定回傳值的明確型別。這增加了回傳錯誤數值的彈性，其在不同錯誤情形中可能有不同的型別。`dyn` 關鍵字是「動態 (dynamic)」的縮寫。

再來，我們移除了 `expect` 的呼叫並改為[第九章](#)所介紹的 `?` 運算子。所以與其對錯誤 `panic!`，`?` 會回傳當前函式的錯誤數值，並交由呼叫者處理。

第三，`run` 函式現在成功時會回傳 `Ok` 數值。我們在 `run` 函式簽名中的成功型別為 `()`，這意味著我們需要將單元型別封裝進 `Ok` 數值。`Ok(())` 這樣的語法一開始看可能會覺得有點奇怪，但這樣子使用 `()` 的確符合慣例，說明我們呼叫 `run` 只是為了它的副作用，它不會回傳我們需要的數值。

當你執行此程式時，它雖然能編譯但會顯示一個警告：

```
$ cargo run the poem.txt
   Compiling minigrep v0.1.0 (file:///projects/minigrep)
warning: unused `Result` that must be used
--> src/main.rs:19:5
   |
19 |     run(config);
   |     ^^^^^^^^^^^
   |
   = note: `#[warn(unused_must_use)]` on by default
   = note: this `Result` may be an `Err` variant, which should be handled

warning: `minigrep` (bin "minigrep") generated 1 warning
   Finished dev [unoptimized + debuginfo] target(s) in 0.71s
   Running `target/debug/minigrep the poem.txt`
搜尋 the
目標檔案為 poem.txt
文字內容：
I'm nobody! Who are you?
Are you nobody, too?
Then there's a pair of us - don't tell!
They'd banish us, you know.

How dreary to be somebody!
How public, like a frog
To tell your name the livelong day
To an admiring bog!
```

Rust 告訴我們程式碼忽略了 `Result` 數值且 `Result` 數值可能代表會有錯誤發生。但我們沒有檢查是不是會發生錯誤，所以編譯器提醒我們可能要在這寫些錯誤處理的程式碼！我們現在就來修正此問題。

在 `main` 中處理 `run` 回傳的錯誤

我們會用類似範例 12-10 中處理 `Config::build` 的技巧來處理錯誤，不過會有一些差別：

檔案名稱：`src/main.rs`

```
fn main() {  
    // --省略--  
  
    println!("搜尋 {}", config.query);  
    println!("目標檔案為 {}", config.file_path);  
  
    if let Err(e) = run(config) {  
        println!("應用程式錯誤：{e}");  
        process::exit(1);  
    }  
}
```

我們使用 `if let` 而非 `unwrap_or_else` 來檢查 `run` 是否有回傳 `Err` 數值，並以此呼叫 `process::exit(1)`。 `run` 函式沒有回傳數值，所以我們不必像處理 `Config::build` 得用 `unwrap` 取得 `Config` 實例。因為 `run` 在成功時會回傳 `()`，而我們只在乎偵測錯誤，所以我們不需要 `unwrap_or_else` 來回傳解封裝後的數值，因為它只會是 `()`。

`if let` 的本體與 `unwrap_or_else` 函式則都做一樣的事情：印出錯誤並離開。

將程式碼拆到函式庫 `Crate`

我們的 `minigrep` 專案目前看起來不錯！接下來我們要將 `src/main.rs` 檔案分開來，將一些程式碼放入 `src/lib.rs` 檔案中。這樣讓我們可以測試程式碼，並讓 `src/main.rs` 檔案的負擔變得少一點。

讓我們將 `main` 以外的所有程式碼從 `src/main.rs` 移到 `src/lib.rs`：

- `run` 函式定義
- 相關的 `use` 陳述式
- `Config` 的定義
- `Config::build` 的函式定義

`src/lib.rs` 的內容應該要如範例 12-13 所示（為了簡潔，我們省略了函式本體）。注意到這還無法編譯，直到我們也修改 `src/main.rs` 成範例 12-14 為止。

檔案名稱：`src/lib.rs`

```

use std::error::Error;
use std::fs;

pub struct Config {
    pub query: String,
    pub file_path: String,
}

impl Config {
    pub fn build(args: &[String]) -> Result<Config, &'static str> {
        // --省略--
    }
}

pub fn run(config: Config) -> Result<(), Box<dyn Error>> {
    // --省略--
}

```



範例 12-13：將 `Config` 與 `run` 移至 `src/lib.rs`

我們對許多項目都使用了 `pub` 關鍵字，這包含 `Config` 與其欄位，以及其 `build` 方法，還有 `run` 函式。我們現在有個函式庫會提供公開 API 能讓我們來測試！

現在我們需要將移至 `src/lib.rs` 的程式碼引入執行檔 `crate` 的 `src/main.rs` 作用域中，如範例 12-14 所示。

檔案名稱：`src/main.rs`

```

use std::env;
use std::process;

use minigrep::Config;

fn main() {
    // --省略--
    if let Err(e) = minigrep::run(config) {
        // --省略--
    }
}

```

範例 12-14：在 `src/main.rs` 使用 `minigrep` 函式庫 `crate`

我們加上 `use minigrep::Config` 這行來將 `Config` 型別從函式庫 `crate` 引入執行檔 `crate` 的作用域中，然後我們使用 `run` 函式的方式是在其前面再加上 `crate` 的名稱。現在所有的功能都應該正常並能執行了。透過 `cargo run` 來執行程式並確保一切正常。

哇！辛苦了，不過我們為未來的成功打下了基礎。現在處理錯誤就輕鬆多了，而且我們讓程式更模組化。現在幾乎所有的工作都會在 `src/lib.rs` 中進行。

讓我們利用這個新的模組化優勢來進行些原本在舊程式碼會很難處理的工作，但在新的程式碼會變得非常容易，那就是寫些測試！

透過測試驅動開發完善函式庫功能

現在我們提取邏輯到 `src/lib.rs` 並在 `src/main.rs` 留下引數收集與錯誤處理的任務，現在對程式碼中的核心功能進行測試會簡單許多。我們可以使用各種引數直接呼叫函式來檢查回傳值，而不用從命令列呼叫我們的執行檔。

在此段落中，我們會在 `minigrep` 程式中利用測試驅動開發（Test-driven development，TDD）來新增搜尋邏輯。此程式開發技巧遵循以下步驟：

1. 寫出一個會失敗的測試並執行它來確保它失敗的原因如你所預期。
2. 寫出或修改足夠的程式碼來讓新測試可以通過。
3. 重構你新增或變更的程式碼並確保測試仍能持續通過。
4. 重複第一步！

雖然這只是編寫軟體的許多方式之一，但 TDD 也有助於程式碼的設計。在寫出能通過測試的程式碼之前先寫好測試能夠協助在開發過程中維持高測試覆蓋率。

我們將用測試驅動功能的實作，而要實作的功能就是在檔案內容中找到欲搜尋的字串，並產生符合查詢字串的行數列表。我們會在一個叫做 `search` 的函式新增此功能。

編寫失敗的測試

讓我們移除 `src/lib.rs` 與 `src/main.rs` 中用來檢查程式行為的 `println!` 陳述式，因為我們不再需要它們了。然後在 `src/lib.rs` 中，我們加上 `tests` 模組與一個測試函式，如我們在[第十一章](#)所做的一樣。測試函式會指定我們希望 `search` 函式所能擁有的行為，它會接收搜尋字串與一段要被搜尋的文字，然後它只回傳文字中包含該搜尋字串的行數。範例 12-15 展示了此測試，但還不能編譯。

檔案名稱：`src/lib.rs`

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn one_result() {
        let query = "duct";
        let contents = "\
Rust:
safe, fast, productive.
Pick three.";

        assert_eq!(vec!["safe, fast, productive."], search(query, contents));
    }
}
```



範例 12-15：建立一個我們預期 `search` 函式該有的行為的失敗測試

此測試搜尋字串 `"duct"`。而要被搜尋的文字有三行，只有一行包含 `"duct"`（在雙引號開頭後方的斜線會告訴 Rust 別在此字串內容開始處換行）。我們判定 `search` 函式回傳的數值只會包含我們預期的那一行。

我們還無法執行此程式並觀察其失敗，因為測試還無法編譯，`search` 函式根本還不存在！按照 TDD 的準則，我們只要加上足夠的程式碼讓測試可以編譯並執行，而我們要加上的是 `search` 函式的定義並永遠回傳一個空的向量，如範例 12-16 所示。然後測試應該就能編譯並失敗，因為空向量並不符合包含 `"safe, fast, productive."` 此行的向量。

檔案名稱：`src/lib.rs`

```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    vec![]
}
```

範例 12-16：定義足夠的 `search` 函式讓我們的測試能夠編譯

值得注意的是在 `search` 的簽名中需要定義一個顯式的生命週期 `'a`，並用於 `contents` 引數與回傳值。回想一下在第十章中生命週期參數會連結引數生命週期與回傳值生命週期。在此例中，我們指明回傳值應包含字串切片且其會參考 `contents` 引數的切片（而非引數 `query`）。

換句話說，我們告訴 Rust `search` 函式回傳的資料會跟傳遞給 `search` 函式的引數 `contents` 資料存活的一樣久。這點很重要！被切片參考的資料必須有效，這樣其參考才會有效。如果編譯器假設是在建立 `query` 而非 `contents` 的字串切片，它的安全檢查就會不正確。

如果我們忘記詮釋生命週期並嘗試編譯此函式，我們會得到以下錯誤：

```

$ cargo build
   Compiling minigrep v0.1.0 (file:///projects/minigrep)
error[E0106]: missing lifetime specifier
  --> src/lib.rs:28:51
   |
28 | pub fn search(query: &str, contents: &str) -> Vec<&str> {
   |               -----             -----             ^ expected named
lifetime parameter
   |
   = help: this function's return type contains a borrowed value, but the
signature does not say whether it is borrowed from `query` or `contents`
help: consider introducing a named lifetime parameter
   |
28 | pub fn search<'a>(query: &'a str, contents: &'a str) -> Vec<&'a str> {
   |               +++++      ++                ++                ++

For more information about this error, try `rustc --explain E0106`.
error: could not compile `minigrep` due to previous error

```

Rust 無法知道這兩個引數哪個才是我們需要的，所以我們得告訴它。由於引數 `contents` 包含所有文字且我們想要回傳符合條件的部分文字，所以我們知道 `contents` 引數要用生命週期語法與回傳值做連結。

其他程式設計語言不會要求你要在簽名中連結引數與回傳值，但這寫久就會習慣了。你可能會想要將此例與第十章的「[透過生命週期驗證參考](#)」段落做比較。

現在讓我們執行測試：

```
$ cargo test
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished test [unoptimized + debuginfo] target(s) in 0.97s
Running unittests src/lib.rs (target/debug/deps/minigrep-9cd200e5fac0fc94)

running 1 test
test tests::one_result ... FAILED

failures:

---- tests::one_result stdout ----
thread 'main' panicked at 'assertion failed: `(left == right)`
  left: `["safe, fast, productive."]',
 right: `[]`, src/lib.rs:44:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::one_result

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`
```

很好！測試如我們所預期地失敗。接下來我們要讓測試通過！

寫出讓測試成功的程式碼

目前我們的測試會失敗，因為我們永遠只回傳一個空向量。要修正並實作 `search`，我們的程式需要完成以下步驟：

- 遍歷內容的每一行。
- 檢查該行是否包含我們要搜尋的字串。
- 如果有的話，將它加入我們要回傳的數值列表。
- 如果沒有的話，不做任何事。
- 回傳符合的結果列表。

讓我們來完成每個步驟，先從遍歷每一行開始。

透過 `lines` 方法來遍歷每一行

Rust 有個實用的方法能逐步處理字串的每一行，這方法就叫 `lines`，而使用方式就如範例 12-17 所示。注意此例還無法編譯。

檔案名稱：src/lib.rs

```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    for line in contents.lines() {
        // 對每行做些事情
    }
}
```



範例 12-17：在 `contents` 中遍歷每一行

`lines` 方法會回傳疊代器 (iterator)。我們會在第十三章詳細解釋疊代器，不過回想一下你在範例 3-5 就看過疊代器的用法了，我們對疊代器使用 `for` 迴圈來對集合中的每個項目執行一些程式碼。

檢查每行是否有要搜尋的字串

接著，我們要檢查目前的行數是否有包含我們要搜尋的字串。幸運的是，字串有個好用的方法叫做 `contains` 能幫我處理這件事！在 `search` 函式中加上方法 `contains` 的呼叫，如範例 12-18 所示。注意這仍然無法編譯。

檔案名稱：src/lib.rs

```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    for line in contents.lines() {
        if line.contains(query) {
            // 對每行做些事情
        }
    }
}
```



範例 12-18：增加檢查行數是否包含 `query` 字串的功能

目前我們正在將功能實作出來。但要能夠編譯的話，我們需要從本體回傳函式簽名中指定的數值。

儲存符合條件的行數

要完成此函式的話，我們需要有個方式能儲存包含搜尋字串的行數。為此我們可以在 `for` 迴圈前建立一個可變向量然後對向量呼叫 `push` 方法來儲存 `line`。在 `for` 迴圈之後，我們回傳向量，如範例 12-19 所示。

檔案名稱：src/lib.rs

```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    let mut results = Vec::new();

    for line in contents.lines() {
        if line.contains(query) {
            results.push(line);
        }
    }

    results
}
```

範例 12-19：儲存符合的行數讓我們可以回傳它們

現在 `search` 函式應該只會回傳包含 `query` 的行數，而我們的測試也該通過。讓我們執行測試：

```
$ cargo test
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished test [unoptimized + debuginfo] target(s) in 1.22s
Running unittests src/lib.rs (target/debug/deps/minigrep-9cd200e5fac0fc94)

running 1 test
test tests::one_result ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Running unittests src/main.rs (target/debug/deps/minigrep-9cd200e5fac0fc94)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests minigrep

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

我們的測試通過了，所以我們確定它運作無誤！

在此刻之後，我們可以考慮重構搜尋函式的實作，並確保測試能通過以維持功能不變。搜尋函式的程式碼並沒有很糟，但它沒有用到疊代器中的一些實用功能優勢。我們會在[第十三章](#)詳細探討疊代器之後，再回過頭來看這個例子，來看看如何改善。

在 `run` 函式中使用 `search` 函式

現在 `search` 函式能夠執行且也有測試過了，我們需要從 `run` 函式呼叫 `search`。我們需要將 `config.query` 數值與 `run` 從檔案讀取到的 `contents` 傳給 `search` 函式。然後 `run` 會印出 `search` 回傳的每一行：

檔案名稱：src/lib.rs

```
pub fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let contents = fs::read_to_string(config.file_path)?;

    for line in search(&config.query, &contents) {
        println!("{}", line);
    }

    Ok(())
}
```

我們仍會使用 `for` 迴圈來取得 `search` 回傳的每一行並顯示出來。

現在整支程式應該都能執行了！讓我們來試試看。首先用一個只會在 Emily Dickinson 的詩中回傳剛好一行的單字「frog」：

```
$ cargo run -- frog poem.txt
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.38s
Running `target/debug/minigrep frog poem.txt`
How public, like a frog
```

酷喔！現在讓我們試試看能符合多行的單字，像是「body」：

```
$ cargo run -- body poem.txt
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.0s
Running `target/debug/minigrep body poem.txt`
I'm nobody! Who are you?
Are you nobody, too?
How dreary to be somebody!
```

最後，讓我們確保使用詩中沒出現的單字來搜尋時，我們不會得到任何一行，像是「monomorphization」：

```
$ cargo run -- monomorphization poem.txt
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.0s
Running `target/debug/minigrep monomorphization poem.txt`
```


漂亮！我們建立了一個屬於自己的迷你經典工具，並學到了很多如何架構應用程式的知識。我們也學了一些檔案輸入與輸出、生命週期、測試與命令列解析。

為了讓此專案更完整，我們會簡單介紹如何使用環境變數，以及如何印出到標準錯誤（standard error），這兩項在寫命令列程式時都很實用。

處理環境變數

我們會新增一個額外的功能來改善 `minigrep`：使用者可以透過環境變數來啟用不區分大小寫的搜尋功能。我們可以將此功能設為命令列選項並要求使用者每次需要時就要加上它，但是這次我們選擇使用環境變數。這樣一來能讓使用者設置環境變數一次就好，然後在該終端機 session 中所有的搜尋都會是不區分大小寫的。

寫個不區分大小寫的 `search` 函式的失敗測試

我們首先新增個 `search_case_insensitive` 函式在環境變數有數值時呼叫它。我們將繼續遵守 TDD 流程，所以第一步一樣是先寫個會失敗的測試。我們會為新函式

`search_case_insensitive` 新增一個測試，並將舊測試從 `one_result` 改名為 `case_sensitive` 以便清楚兩個測試的差別，如範例 12-20 所示。

檔案名稱：src/lib.rs

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn case_sensitive() {
        let query = "duct";
        let contents = "\
Rust:
safe, fast, productive.
Pick three.
Duct tape.";

        assert_eq!(vec!["safe, fast, productive."], search(query, contents));
    }

    #[test]
    fn case_insensitive() {
        let query = "rUsT";
        let contents = "\
Rust:
safe, fast, productive.
Pick three.
Trust me.";

        assert_eq!(
            vec!["Rust:", "Trust me."],
            search_case_insensitive(query, contents)
        );
    }
}
```



範例 12-20：為準備加入的不區分大小寫函式新增個失敗測試

注意到我們也修改了舊測試 `contents`。我們新增了一行使用大寫 D 的 `"Duct tape."`，當我們以區分大小寫來搜尋時，就不會符合要搜尋的 `"duct"`。這樣變更舊測試能確保我們沒有意外破壞我們已經實作好的區分大小寫的功能。此測試應該要能通過，並在我們實作不區分大小寫的搜尋時仍能繼續通過。

新的不區分大小寫的搜尋測試使用 `"rUsT"` 來搜尋。在我們準備要加入的 `search_case_insensitive` 函式中，要搜尋的 `"rUsT"` 應該要能符合有大寫 R 的 `"Rust:"` 以及 `"Trust me."` 這幾行，就算兩者都與搜尋字串有不同的大小寫。這是我們的失敗測試而且它還無法編譯，因為我們還沒有定義 `search_case_insensitive` 函式。歡迎自行加上一個永遠回傳空向量的骨架實作，就像我們在範例 12-16 所做的一樣，然後看看測試能不能編譯過並失敗。

實作 `search_case_insensitive` 函式

範例 12-21 顯示的 `search_case_insensitive` 函式會與 `search` 函式幾乎一樣。唯一的不同在於我們將 `query` 與每個 `line` 都變成小寫，所以無論輸入引數是大寫還是小寫，當我們在檢查行數是否包含搜尋的字串時，它們都會是小寫。

檔案名稱：src/lib.rs

```
pub fn search_case_insensitive<'a>(
    query: &str,
    contents: &'a str,
) -> Vec<&'a str> {
    let query = query.to_lowercase();
    let mut results = Vec::new();

    for line in contents.lines() {
        if line.to_lowercase().contains(&query) {
            results.push(line);
        }
    }

    results
}
```

範例 12-21：定義 `search_case_insensitive` 並在比較前將搜尋字串與行數均改為小寫

首先我們將 `query` 字串變成小寫並儲存到同名的遮蔽變數中。我們必須呼叫對要搜尋的字串 `to_lowercase`，這樣無論使用者輸入的是 `"rust"`、`"RUST"`、`"Rust"` 或 `"rUsT"`，我們都會將字串視為 `"rust"` 並以此來不區分大小寫。雖然 `to_lowercase` 能處理基本的 Unicode，但它不會是 100% 準確的。如果我們是在寫真正的應用程式的話，我們需要處理更多條件，但在此段落是為了理解環境變數而非 Unicode，所以我們維持這樣寫就好。

注意到 `query` 現在是個 `String` 而非字串切片，因為呼叫 `to_lowercase` 會建立新的資料而非參考現存的資料。假設要搜尋的字串是 `"rUsT"` 的話，該字串切片並沒有包含小寫的 `u` 或 `t` 能讓我們來使用，所以我們必須配置一個包含 `"rust"` 的新 `String`。現在當我們將 `query` 作為引數傳給 `contains` 方法時，我們需要加上「&」，因為 `contains` 所定義的簽名接收的是一個字串切片。

接著，我們對每個 `line` 加上 `to_lowercase` 的呼叫。現在我們將 `line` 和 `query` 都轉換成小寫了。我們可以不區分大小寫來找到符合的行數。

讓我們來看看實作是否能通過測試：

```
$ cargo test
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished test [unoptimized + debuginfo] target(s) in 1.33s
Running unittests src/lib.rs (target/debug/deps/minigrep-9cd200e5fac0fc94)

running 2 tests
test tests::case_insensitive ... ok
test tests::case_sensitive ... ok

test result: ok. 2 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Running unittests src/main.rs (target/debug/deps/minigrep-9cd200e5fac0fc94)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests minigrep

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

很好！測試通過。現在讓我們從 `run` 函式呼叫新的 `search_case_insensitive` 函式。首先，我們要在 `Config` 中新增一個配置選項來切換區分大小寫與不區分大小寫之間的搜尋。新增此欄位會造成編譯錯誤，因為我們還沒有初始化該欄位：

檔案名稱：src/lib.rs

```
pub struct Config {
    pub query: String,
    pub file_path: String,
    pub ignore_case: bool,
}
```



注意到我們新增了 `ignore_case` 欄位並存有布林值。接著，我們需要 `run` 函式檢查 `ignore_case` 欄位的數值並以此決定要呼叫 `search` 函式或是 `search_case_insensitive` 函式，如範例 12-22 所示。不過目前還無法編譯。

檔案名稱：src/lib.rs

```
pub fn run(config: Config) -> Result<(), Box<dyn Error>> {
    let contents = fs::read_to_string(config.file_path)?;

    let results = if config.ignore_case {
        search_case_insensitive(&config.query, &contents)
    } else {
        search(&config.query, &contents)
    };

    for line in results {
        println!("{line}");
    }

    Ok(())
}
```



範例 12-22：依據 `config.ignore_case` 的數值來呼叫 `search` 或 `search_case_insensitive`

最後，我們需要檢查環境變數。處理環境變數的函式位於標準函式庫中的 `env` 模組中，所以我們在 `src/lib.rs` 最上方加上 `use std::env;` 來將該模組引入作用域。然後我們使用 `env` 模組中的 `var` 函式來檢查一個叫做 `IGNORE_CASE` 的環境變數有沒有設任何數值，如範例 12-23 所示。

檔案名稱：`src/lib.rs`

```
use std::env;
// --省略--

impl Config {
    pub fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("引數不足");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        let ignore_case = env::var("IGNORE_CASE").is_ok();

        Ok(Config {
            query,
            file_path,
            ignore_case,
        })
    }
}
```

範例 12-23：檢查環境變數 `IGNORE_CASE`

我們在此建立了一個新的變數 `ignore_case`。要設置其數值，我們可以呼叫 `env::var` 函式並傳入環境變數 `IGNORE_CASE` 的名稱。`env::var` 函式會回傳 `Result`，如果有設置環境變數的話，這就會是包含環境變數數值的成功 `Ok` 變體；如果環境變數沒有設置的話，這就會回傳 `Err` 變體。

我們在 `Result` 使用 `is_ok` 方法來檢查環境變數是否有設置，也就是程式是否該使用區分大小寫的搜尋。如果 `IGNORE_CASE` 環境變數沒有設置任何數值的話，`is_ok` 會回傳否，所以程式就會進行區分大小寫的搜尋。我們不在乎環境變數的數值，只在意它有沒有被設置而已，所以我們使用 `is_ok` 來檢查而非使用 `unwrap`、`expect` 或其他任何我們看過的 `Result` 方法。

我們將變數 `ignore_case` 的數值傳給 `Config` 實例，讓 `run` 函式可以讀取該數值並決定該呼叫 `search_case_insensitive` 還是 `search`，如範例 12-22 所實作的一樣。

讓我們試看看吧！首先，我們先不設置環境變數並執行程式來搜尋 `to`，任何包含小寫單字「to」的行數都應要符合：

```
$ cargo run -- to poem.txt
Compiling minigrep v0.1.0 (file:///projects/minigrep)
Finished dev [unoptimized + debuginfo] target(s) in 0.0s
Running `target/debug/minigrep to poem.txt`
Are you nobody, too?
How dreary to be somebody!
```

看起來運作仍十分正常！現在，讓我們設置 `IGNORE_CASE` 為 `1` 並執行程式來搜尋相同的字串 `to`。

```
$ IGNORE_CASE=1 cargo run -- to poem.txt
```

如果你使用的是 PowerShell，你需要將設置變數與執行程式分為不同的命令：

```
PS> $Env:IGNORE_CASE=1; cargo run -- to poem.txt
```

這會在你的 shell session 中設置 `IGNORE_CASE`。它可以透過 `Remove-Item cmdlet` 來取消設置：

```
PS> Remove-Item Env:IGNORE_CASE
```

我們應該會得到包含可能有大寫的「to」的行數：

```
Are you nobody, too?
How dreary to be somebody!
To tell your name the livelong day
To an admiring bog!
```

太好了，我們也取得了包含「To」的行數！我們的 `minigrep` 程式現在可以進行不區分大小寫的搜尋並以環境變數配置。現在你知道如何使用命令列引數或環境變數來管理設置選項了。

有些程式允許同時使用引數與環境變數來配置。在這種情況下，程式會決定各種選項的優先層級。你想要練習的話，嘗試使用命令列引數與環境變數來控制區分大小寫的選項。並在程式執行時，其中一個設置為區分大小寫，而另一個為不區分大小寫時，自行決定該優先使用命令列引數還是環境變數。

`std::env` 模組還包含很多處理環境變數的實用功能，歡迎查閱其官方文件來瞭解有哪些可用。

將錯誤訊息寫入標準錯誤而非標準輸出

目前我們使用 `println!` 巨集來將所有的輸出顯示到終端機。大多數的終端機都提供兩種輸出方式：用於通用資訊的標準輸出（**standard output, `stdout`**）以及用於錯誤訊息的標準錯誤（**standard error, `stderr`**）。這樣的區別讓使用者可以選擇將程式的成功輸出導向到一個檔案中，並仍能在螢幕上顯示錯誤訊息。

`println!` 巨集只能夠印出標準輸出，所以我們得用其他方式來印出標準錯誤。

檢查該在哪裡寫錯誤

首先，我們先來觀察 `minigrep` 目前寫到標準輸出的顯示內容，其中包含任何我們想改寫成標準錯誤的錯誤訊息。我們會將標準輸出重新導向至一個檔案並故意產生一個錯誤。因為我們不會重新導向標準錯誤，所以任何傳送至標準錯誤的內容會繼續顯示在螢幕上。

命令列程式應該要傳送錯誤訊息至標準錯誤，讓我們可以在重新導向標準輸出至檔案時，仍能在螢幕上看到錯誤訊息。所以我們的程式目前並不符合預期：我們會看到它儲存錯誤訊息輸出到檔案中！

要觀察此行為的方式的話，我們要透過 `>` 來執行程式並加上檔案名稱 `output.txt`，這是我們要重新導向標準輸出到的地方。我們不會傳遞任何引數，這樣就應該會造成錯誤：

```
$ cargo run > output.txt
```

`>` 語法告訴 shell 要將標準輸出的內容寫入 `output.txt` 而不是顯示在螢幕上。我們沒有看到應顯示在螢幕上的錯誤訊息，這代表它一定跑到檔案中了。以下是 `output.txt` 包含的內容：

```
解析引數時出現問題：引數不足
```

是的，我們的錯誤訊息印到了標準輸出。像這樣的錯誤訊息印到標準錯誤會比較好，這樣才能只讓成功執行的資料存至檔案中。讓我們來修正吧。

將錯誤印出至標準錯誤

我們會使用範例 12-24 的程式碼來改變錯誤訊息印出的方式。由於我們在本章前幾篇的重構，所有印出錯誤訊息的程式碼都位於 `main` 函式中。標準函式庫有提供 `eprintln!` 巨集來印到標準錯誤，所以讓我們變更兩個原本呼叫 `println!` 來印出錯誤的段落來改使用 `eprintln!`。

檔案名稱：`src/main.rs`

```
fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::build(&args).unwrap_or_else(|err| {
        eprintln!("解析引數時出現問題：{err}");
        process::exit(1);
    });

    if let Err(e) = minigrep::run(config) {
        eprintln!("應用程式錯誤：{e}");
        process::exit(1);
    }
}
```

範例 12-24：使用 `eprintln!` 來將錯誤訊息印至標準錯誤而非標準輸出

讓我們以相同方式再執行程式一次，沒有任何引數並用 `>` 重新導向標準輸出：

```
$ cargo run > output.txt
解析引數時出現問題：引數不足
```

現在我們看到錯誤顯示在螢幕上而且 `output.txt` 裡什麼也沒有，這正是命令列程式所預期的行為。

讓我們加上不會產生錯誤的引數來執行程式，並仍重新導向標準輸出至檔案中，如以下所示：

```
$ cargo run -- to poem.txt > output.txt
```

我們在終端機不會看到任何輸出，而 `output.txt` 會包含我們的結果：

檔案名稱：output.txt

```
Are you nobody, too?
How dreary to be somebody!
```

這說明我們現在有對成功的輸出使用標準輸出，而且有妥善地將錯誤輸出傳至標準錯誤。

總結

本章節回顧了你目前所學的一些重要概念，並介紹了如何在 Rust 中進行常見的 I/O 操作。透過使用命令列引數、檔案、環境變數與用來印出錯誤的 `eprintln!` 巨集，你現在已經準備好能寫出命令列應用程式了。結合前幾章的概念，你的程式的組織架構會非常穩固、資料都能有效率地儲存至適當的資料結構、完善地處理錯誤，並通過測試檢驗。

接下來，我們要探討些 Rust 受到函式語言啟發的功能：閉包與疊代器。

函式語言功能：疊代器與閉包

Rust 的設計靈感啟發自許多現有的語言與技術，其中一個影響十分顯著的就是函式程式設計 (*functional programming*)。以函式風格的程式設計通常包含將函式視為數值並作為引數傳遞、將它們從其他函式回傳、將它們賦值給變數以便之後使用，以及更多。

在本章節中，我們不會討論哪些才是屬於函式程式設計或哪些不是，而是介紹一些 Rust 中類似於許多語言常視為函式語言特色的功能。

更明確來說，我們會涵蓋：

- 閉包 (*Closures*)：類似函式的結構並可以賦值給變數
- 疊代器 (*Iterators*)：遍歷一系列元素的方法
- 如何用閉包與疊代器來改善第十二章的 I/O 專案
- 閉包與疊代器的效能（先偷偷跟你說：它們比你想像的還要快！）

我們已經在其他章節提到的功能像是模式配對與列舉也都有被函式風格所影響。因為掌握閉包與疊代器是寫出符合語言風格與高效 Rust 程式碼中重要的一環，所以我們將用一章來介紹它們。

閉包：獲取其環境的匿名函式

Rust 的閉包（closures）是個你能賦值給變數或作為其他函式引數的匿名函式。你可以在某處建立閉包，然後在不同的地方呼叫閉包並執行它。而且不像函式，閉包可以從它們所定義的作用域中獲取數值。我們將會解釋這些閉包功能如何允許程式碼重用以及自訂行為。

透過閉包獲取環境

我們首先會來研究我們如何用閉包來獲取定義在環境的數值並在之後使用。讓我們考慮以下假設情境：每隔一段時間，我們的襯衫公司會送出獨家限量版襯衫給郵寄清單的某位顧客來作為宣傳手段。郵寄清單的顧客可以在他們的設定中加入他們最愛的顏色。如果被選中的人有設定最愛顏色的話，他們就會獲得該顏色的襯衫。如果他們沒有指定任何最愛顏色的話，公司就會選擇目前顏色最多的選項。

要實作的方式有很多種。舉例來說，我們可以使用一個列舉叫做 `ShirtColor` 然後其變體有 `Red` 和 `Blue`（為了簡潔我們限制顏色的種類）。我們用 `Inventory` 來代表公司的庫存，然後用 `shirts` 欄位來包含 `Vec<ShirtColor>` 來代表目前庫存有的襯衫顏色。定義在 `Inventory` 的 `giveaway` 方法會取得免費襯衫得主的選擇性襯衫顏色偏好，然後回傳他們會拿到的襯衫顏色。如範例 13-1 所示：

檔案名稱：`src/main.rs`

```

#[derive(Debug, PartialEq, Copy, Clone)]
enum ShirtColor {
    Red,
    Blue,
}

struct Inventory {
    shirts: Vec<ShirtColor>,
}

impl Inventory {
    fn giveaway(&self, user_preference: Option<ShirtColor>) -> ShirtColor {
        user_preference.unwrap_or_else(|| self.most_stocked())
    }

    fn most_stocked(&self) -> ShirtColor {
        let mut num_red = 0;
        let mut num_blue = 0;

        for color in &self.shirts {
            match color {
                ShirtColor::Red => num_red += 1,
                ShirtColor::Blue => num_blue += 1,
            }
        }
        if num_red > num_blue {
            ShirtColor::Red
        } else {
            ShirtColor::Blue
        }
    }
}

fn main() {
    let store = Inventory {
        shirts: vec![ShirtColor::Blue, ShirtColor::Red, ShirtColor::Blue],
    };

    let user_pref1 = Some(ShirtColor::Red);
    let giveaway1 = store.giveaway(user_pref1);
    println!(
        "偏好 {:?} 的使用者獲得 {:?}",
        user_pref1, giveaway1
    );

    let user_pref2 = None;
    let giveaway2 = store.giveaway(user_pref2);
    println!(
        "偏好 {:?} 的使用者獲得 {:?}",

```

```
        user_pref2, giveaway2
    );
}
```

範例 13-1：襯衫公司送禮的情境

定義在 `main` 中的 `store` 在這次的限量版宣傳中的庫存有兩件藍色襯衫與一件紅色襯衫。我們呼叫了 `giveaway` 方法兩次，一次是給偏好紅色襯衫的使用者，另一次則是給無任何偏好的使用者。

再次強調這可以用各種方式實作，只是在此我們想專注在閉包，所以除了用到我們已經學過的概念以外，`giveaway` 方法中還使用了閉包。在 `giveaway` 方法中，我們從參數型別

`Option<ShirtColor>` 取得使用者偏好，然後對 `user_preference` 呼叫 `unwrap_or_else` 方法。`Option<T>` 的 `unwrap_or_else` 方法定義在標準函式庫中。它接收一個引數：一個沒有任何引數的閉包然後會回傳數值 `T`（該型別為 `Option<T>` 的 `Some` 儲存的型別，在此例中就是 `ShirtColor`）。如果 `Option<T>` 是 `Some` 變體，`unwrap_or_else` 就會回傳 `Some` 裡的數值。如果 `Option<T>` 是 `None` 變體，`unwrap_or_else` 會呼叫閉包並回傳閉包回傳的數值。

我們寫上閉包表達式 `|| self.most_stocked()` 作為 `unwrap_or_else` 的引數。這是個沒有任何參數的閉包（如果閉包有參數的話，它們會出現在兩條直線中間）。閉包本體會呼叫 `self.most_stocked()`。我們直接在此定義閉包，然後 `unwrap_or_else` 的實作就會在需要結果時執行閉包。

執行此程式的話就會印出：

```
$ cargo r
  Compiling shirt-company v0.1.0 (file:///projects/shirt-company)
    Finished dev [unoptimized + debuginfo] target(s) in 0.00s
     Running `target/debug/shirt-company`
偏好 Some(Red) 的使用者獲得 Red
偏好 None 的使用者獲得 Blue
```

這裡值得注意的是我們對當前 `Inventory` 實例傳入的是一個呼叫 `self.most_stocked()` 的閉包。標準函式庫不需要知道我們定義的任何型別像 `Inventory` 與 `ShirtColor`，或是在此情境中我們需要使用的任何邏輯，閉包就會獲取 `Inventory` 實例的不可變參考 `self`，然後傳給我們在 `unwrap_or_else` 方法中指定的程式碼。反之，函式就無法像這樣獲取它們周圍的環境。

閉包型別推導與詮釋

函式與閉包還有更多不同的地方。閉包通常不必像 `fn` 函式那樣要求你要詮釋參數或回傳值的型別。函式需要型別詮釋是因為它們是顯式公開給使用者的介面。嚴格定義此介面是很重要的，這能確保每個人同意函式使用或回傳的數值型別為何。但是閉包並不是為了對外公開使用，它們儲存在變數且沒有名稱能公開給我們函式庫的使用者。

閉包通常很短，而且只與小範圍內的程式碼有關，而非適用於任何場合。有了這樣限制的環境，編譯器能可靠地推導出參數與回傳值的型別，如同其如何推導出大部分的變數型別一樣。（但在有些例外情形下編譯器還是需要閉包的型別詮釋）

至於變數的話，雖然不是必要的，但如果我們希望能夠增加閱讀性與清楚程度，我們還是可以加上型別詮釋。要在閉包詮釋型別的話，就會如範例 13-2 的定義所示。在此範例中，我們定義一個閉包並儲存至一個變數中，而非像範例 13-1 我們將閉包作為引數傳入。

檔案名稱：src/main.rs

```
let expensive_closure = |num: u32| -> u32 {
    println!("緩慢計算中...");
    thread::sleep(Duration::from_secs(2));
    num
};
```

範例 13-2：對閉包加上選擇性的參數與回傳值型別詮釋

加上型別詮釋後，閉包的語法看起來就更像函式的語法了。我們在此定義了一個對參數加 1 的函式，以及一個有相同行為的閉包做為比較。我們加了一些空格來對齊相對應的部分。這顯示了閉包語法和函式語法有多類似，只是改用直線以及有些語法是選擇性的。

```
fn add_one_v1 (x: u32) -> u32 { x + 1 }
let add_one_v2 = |x: u32| -> u32 { x + 1 };
let add_one_v3 = |x|           { x + 1 };
let add_one_v4 = |x|           x + 1 ;
```

第一行顯示的是函式定義，而第二行則顯示有完成型別詮釋的閉包定義。在第三行我們移除了閉包定義的型別詮釋，然後在第四行我們移除了大括號，因為閉包本體只有一個表達式，所以這是選擇性的。這些都是有效的定義，並會在被呼叫時產生相同行為。而 `add_one_v3` 和 `add_one_v4` 一定要被呼叫，這樣編譯器才能從它們的使用方式中推導出型別。這就像 `let v = Vec::new();` 需要型別詮釋，或是有某種型別的數值插入 `Vec` 中，Rust 才能推導出型別。

對於閉包定義，編譯器會對每個參數與它們的回傳值推導出一個實際型別。舉例來說，範例 13-3 展示一支只會將收到的參數作為回傳值的閉包定義。此閉包並沒有什麼意義，純粹作為範例解釋。注意到我們沒有在定義中加上任何型別詮釋。由於沒有型別詮釋，我們可以用任何型別來呼叫閉包，像我們第一次呼叫就用 `String`。如果我們接著嘗試用整數呼叫 `example_closure`，我們就會得到錯誤。

檔案名稱：src/main.rs

```
let example_closure = |x| x;

let s = example_closure(String::from("哈囉"));
let n = example_closure(5);
```



範例 13-3：嘗試呼叫被推導出兩個不同型別的閉包

編譯器會給我們以下錯誤：

```
$ cargo run
  Compiling closure-example v0.1.0 (file:///projects/closure-example)
error[E0308]: mismatched types
  --> src/main.rs:5:29
   |
5 |         let n = example_closure(5);
   |                        ^- help: try using a conversion method:
   |                        `.to_string()`
   |                        |
   |                        expected struct `String`, found integer
   |                        arguments to this function are incorrect
   |
note: closure parameter defined here
  --> src/main.rs:3:28
   |
2 |         let example_closure = |x| x;
   |                                ^

For more information about this error, try `rustc --explain E0308`.
error: could not compile `closure-example` due to previous error
```

當我們第一次使用 `String` 數值呼叫 `example_closure` 時，編譯器會推導 `x` 與閉包回傳值的型別為 `String`。這樣 `example_closure` 閉包內的型別就會鎖定，然後我們如果對同樣的閉包嘗試使用不同的型別的話，我們就會得到型別錯誤。

獲取參考或移動所有權

閉包要從它們周圍環境取得數值有三種方式，這能直接對應於函式取得參數的三種方式：不可變借用、可變借用，與取得所有權。閉包會依照函式本體如何使用獲取的數值，來決定要用哪種方式。

在範例 13-4 中，我們定義一個閉包來獲取 `list` 向量的不可變參考，因為它只需要不可變參考就能印出數值：

檔案名稱：src/main.rs


```
fn main() {
    let list = vec![1, 2, 3];
    println!("定義閉包前：{:?}", list);

    let only_borrows = || println!("來自閉包：{:?}", list);

    println!("呼叫閉包前：{:?}", list);
    only_borrows();
    println!("呼叫閉包後：{:?}", list);
}
```

範例 13-4：定義並呼叫會獲取不可變參考的閉包

此範例還示範了變數能綁定閉包的定義，然後我們之後就可以用變數名稱加上括號來呼叫閉包，這樣變數名稱就像函式名稱一樣。

由於我們可以同時擁有 `list` 的多重不可變參考，`list` 在閉包定義前、在閉包定義後閉包呼叫前以及閉包呼叫時的程式碼中都是能使用的。此程式碼就會編譯、執行並印出：

```
$ cargo run
   Compiling closure-example v0.1.0 (file:///projects/closure-example)
   Finished dev [unoptimized + debuginfo] target(s) in 0.43s
   Running `target/debug/closure-example`
定義閉包前：[1, 2, 3]
呼叫閉包前：[1, 2, 3]
來自閉包：[1, 2, 3]
呼叫閉包後：[1, 2, 3]
```

接著在範例 13-5 中我們改變閉包本體，對 `list` 向量加上一個元素。這樣閉包現在就會獲取可變參考：

檔案名稱：src/main.rs

```
fn main() {
    let mut list = vec![1, 2, 3];
    println!("呼叫閉包前{:?}", list);

    let mut borrows_mutably = || list.push(7);

    borrows_mutably();
    println!("呼叫閉包後：{:?}", list);
}
```

範例 13-11：定義並呼叫會獲取可變參考的閉包

此程式碼會編譯、執行並印出：

```
$ cargo run
Compiling closure-example v0.1.0 (file:///projects/closure-example)
Finished dev [unoptimized + debuginfo] target(s) in 0.43s
Running `target/debug/closure-example`
呼叫閉包前[1, 2, 3]
呼叫閉包後：[1, 2, 3, 7]
```

注意到在 `borrowably` 閉包的定義與呼叫之間的 `println!` 不見了：當 `borrowably` 定義時，它會獲取 `list` 的可變參考。我們在閉包呼叫之後沒有再使用閉包，所以可變參考就結束。在閉包定義與呼叫之間，利用不可變參考印出輸出是不允許的，因為在可變參考期間不能再有其他參考。你可以試試看在那加上 `println!` 然後看看會收到什麼錯誤訊息！

如果你想要強迫閉包取得周圍環境數值的所有權的話，你可以在參數列表前使用 `move` 關鍵字。

此技巧適用於將閉包傳給新執行緒來移動資料，讓新的執行緒能擁有該資料。我們會在第十六章討論並行時，介紹為何你會想使用它們。但現在讓我們簡單探索怎麼在閉包使用 `move` 關鍵字開個新的執行緒就好。範例 13-6 更改了範例 13-4 讓向量在新的執行緒印出而非原本的主執行緒：

檔案名稱：src/main.rs

```
use std::thread;

fn main() {
    let list = vec![1, 2, 3];
    println!("呼叫閉包前：{:?}", list);

    thread::spawn(move || println!("來自執行緒：{:?}", list))
        .join()
        .unwrap();
}
```

範例 13-6：使用 `move` 來迫使執行緒的閉包取得 `list` 的所有權

我們開了一個新的執行緒，將閉包作為引數傳入，閉包本體會印出 `list`。在範例 13-4 中，閉包只用不可變參考獲取 `list`，因為要印出 `list` 的需求只要這樣就好。而在此例中，儘管閉包本體仍然只需要不可變參考就好，我們在閉包定義時想要指定 `list` 應該要透過 `move` 關鍵字移入閉包。新的執行緒可能會在主執行緒之前結束，或者主執行緒也有可能先結束。如果主執行緒持有 `list` 的所有權卻在新執行緒之前結束並釋放 `list` 的話，執行緒拿到的不可變參考就會無效了。因此編譯器會要求 `list` 移入新執行緒的閉包中，這樣參考才會有效。嘗試看看將 `move` 關鍵字刪掉，或是在主執行緒的閉包定義之後使用 `list`，看看你會收到什麼編譯器錯誤訊息！

Fn 特徵以及將獲取的數值移出閉包

一旦閉包從其定義的周圍環境獲取了數值的參考或所有權（也就是說被**移入**閉包中），閉包本體的程式碼會定義閉包在執行結束後要對參考或數值做什麼事情（也就是說被**移出**閉包）。閉包本體可

以做以下的事情：將獲取的數值移出閉包、改變獲取的數值、不改變且不移動數值，或是一開始就不從環境獲取任何值。

閉包從周圍環境獲取並處理數值的方式會影響閉包會實作哪種特徵，而這些特徵能讓函式與結構體決定它們想使用哪種閉包。閉包會依照閉包本體處理數值的方式，自動實作一種或多種 `Fn` 特徵：

1. `FnOnce` 適用於可以呼叫一次的閉包。所有閉包至少都會有此特徵，因為所有閉包都能被呼叫。會將獲取的數值移出本體的閉包只會實作 `FnOnce` 而不會再實作其他 `Fn` 特徵，因為這樣它只能被呼叫一次。
2. `FnMut` 適用於不會將獲取數值移出本體，而且可能會改變獲取數值的閉包。這種閉包可以被呼叫多次。
3. `Fn` 適用於不會將獲取數值移出本體，而且不會改變獲取數值或是甚至不從環境獲取數值的閉包。這種閉包可以被呼叫多次，而且不會改變周圍環境，這對於並行呼叫閉包多次來說非常重要。

讓我們來觀察範例 13-1 中 `Option<T>` 用到的 `unwrap_or_else` 方法定義：

```
impl<T> Option<T> {
    pub fn unwrap_or_else<F>(self, f: F) -> T
    where
        F: FnOnce() -> T
    {
        match self {
            Some(x) => x,
            None => f(),
        }
    }
}
```

回想一下 `T` 是一個泛型型別，代表著 `Option` 的 `Some` 變體內的數值型別。型別 `T` 同時也是函式 `unwrap_or_else` 的回傳型別：比如說對 `Option<String>` 呼叫 `unwrap_or_else` 的話就會取得 `String`。

接著注意到函式 `unwrap_or_else` 有個額外的泛型型別參數 `F`。型別 `F` 是參數 `f` 的型別，也正是當我們呼叫 `unwrap_or_else` 時的閉包。

泛型型別 `F` 指定的特徵界限是 `FnOnce() -> T`，也就是說 `F` 必須要能夠呼叫一次、不帶任何引數然後回傳 `T`。在特徵界限中使用 `FnOnce` 限制了 `unwrap_or_else` 只能呼叫 `f` 最多一次。在 `unwrap_or_else` 本體中，如果 `Option` 是 `Some` 的話，`f` 就不會被呼叫。如果 `Option` 是 `None` 的話，`f` 就會被呼叫一次。由於所有閉包都有實作 `FnOnce`，`unwrap_or_else` 能接受大多數各種不同的閉包，讓它的用途非常彈性。

注意：函式也可以實作這三種 `Fn` 特徵。如果我們不必獲取環境數值，在我們需要有實作其中一種 `Fn` 特徵的項目時，我們可以使用函式名稱而不必用到閉包。舉例來說，對於

`Option<Vec<T>>` 的數值，我們可以呼叫 `unwrap_or_else(Vec::new)` 在數值為 `None` 時取得新的空向量。

現在讓我們來看看標準函式庫中切片定義的 `sort_by_key` 方法，來觀察它和 `unwrap_or_else` 有什麼不同，以及為何 `sort_by_key` 的特徵界限使用的是 `FnMut` 而不是 `FnOnce`。閉包會取得一個引數，這會是該切片當下項目的參考，然後回傳型別 `K` 的數值以供排序。當你想透過切片項目的特定屬性做排序時，此函式會很實用。在範例 13-7 中，我們有個 `Rectangle` 實例的列表，然後我們使用 `sort_by_key` 透過 `width` 屬性由低至高排序它們：

檔案名稱：src/main.rs

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    list.sort_by_key(|r| r.width);
    println!("{:?}", list);
}
```

範例 13-7：使用 `sort_by_key` 依據寬度來排序長方形

此程式碼會印出：

```
$ cargo run
Compiling rectangles v0.1.0 (file:///projects/rectangles)
Finished dev [unoptimized + debuginfo] target(s) in 0.41s
Running `target/debug/rectangles`
[
  Rectangle {
    width: 3,
    height: 5,
  },
  Rectangle {
    width: 7,
    height: 12,
  },
  Rectangle {
    width: 10,
    height: 1,
  },
]
```

`sort_by_key` 的定義會需要 `FnMut` 閉包的原因是因為它得呼叫閉包好幾次，對切片的每個項目都要呼叫一次。閉包 `|r| r.width` 沒有獲取、改變或移動周圍環境的任何值，所以它符合特徵界限的要求。

反之，範例 13-8 示範了一個只實作 `FnOnce` 特徵的閉包，因為它有將數值移出環境。編譯器不會允許我們將此閉包用在 `sort_by_key`：

檔案名稱：src/main.rs

```

#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    let mut sort_operations = vec![];
    let value = String::from("by key called");

    list.sort_by_key(|r| {
        sort_operations.push(value);
        r.width
    });
    println!("{:#?}", list);
}

```



範例 13-8：嘗試在 `sort_by_key` 使用 `FnOnce` 閉包

這裡嘗試用很糟糕且令人費解的方式計算 `list` 在排序時 `sort_by_key` 被呼叫了幾次。此程式碼嘗試計數的方式是把閉包周圍環境中型別為 `String` 的 `value` 變數放入 `sort_operations` 向量中。閉包會獲取 `value`，然後將 `value` 移出閉包，也就是將 `value` 的所有權轉移到 `sort_operations` 向量裡。此向量只能呼叫一次，嘗試呼叫第二次是無法成功的，因為 `value` 已經不存在於環境中了，無法再次放入 `sort_operations`！因此，此閉包僅實作了 `FnOnce`。當我們嘗試編譯此程式碼時，我們會收到錯誤訊息說明 `value` 無法移出閉包，因為閉包必須實作 `FnMut`：

```
$ cargo run
Compiling rectangles v0.1.0 (/Users/wuwayne/Desktop/book-tw/listings/ch13-
functional-features/listing-13-08)
error[E0507]: cannot move out of `value`, a captured variable in an `FnMut`
closure
  --> src/main.rs:18:30
   |
15 |     let value = String::from("by key called");
   |         ----- captured outer variable
16 |
17 |     list.sort_by_key(|r| {
   |                     --- captured by this `FnMut` closure
18 |         sort_operations.push(value);
   |                               ^^^^^^ move occurs because `value` has type
`String`, which does not implement the `Copy` trait

For more information about this error, try `rustc --explain E0507`.
error: could not compile `rectangles` due to previous error
```

錯誤訊息指出閉包本體將 `value` 移出環境的地方。要修正此問題的話，我們需要改變閉包本體，讓它不再將數值移出環境。要計算 `sort_by_key` 呼叫次數的話，在環境中放置一個計數器，然後在閉包本體增加其值是更直觀的計算方法。範例 13-9 的閉包就能用在 `sort_by_key`，因為它只獲取了 `num_sort_operations` 計數器的可變參考，因此可以被呼叫不只一次：

檔案名稱：src/main.rs

```
#[derive(Debug)]
struct Rectangle {
    width: u32,
    height: u32,
}

fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    let mut num_sort_operations = 0;
    list.sort_by_key(|r| {
        num_sort_operations += 1;
        r.width
    });
    println!("{:#?} 的排序經過 {num_sort_operations} 次運算", list);
}
```

範例 13-9：在 `sort_by_key` 使用 `FnMut` 閉包是允許的

當我們要在函式或型別中定義與使用閉包時，`Fn` 特徵是很重要的。在下個段落中，我們將討論疊代器。疊代器有許多方法都需要閉包引數，所以隨著我們繼續下去別忘了複習閉包的用法！

使用疊代器來處理一系列的項目

疊代器（Iterator）模式讓你可以對一個項目序列依序進行某些任務。疊代器的功用是遍歷序列中每個項目，並決定該序列何時結束。當你使用疊代器，你不需要自己實作這些邏輯。

在 Rust 中疊代器是惰性（lazy）的，代表除非你呼叫方法來使用疊代器，不然它們不會有任何效果。舉例來說，範例 13-10 的程式碼會透過 `Vec<T>` 定義的方法 `iter` 從向量 `v1` 建立一個疊代器來遍歷它的項目。此程式碼本身沒有啥實用之處。

```
let v1 = vec![1, 2, 3];

let v1_iter = v1.iter();
```

範例 13-10：建立一個疊代器

疊代器儲存在變數 `v1_iter` 中。一旦我們建立了疊代器，我們可以有很多使用它的方式。在第三章的範例 3-5 中，我們在 `for` 迴圈中使用疊代器來對每個項目執行一些程式碼。在過程中這就隱性建立並使用了一個疊代器，雖然我們當時沒有詳細解釋細節。

在範例 13-11 中，我們區隔了疊代器的建立與使用疊代器 `for` 迴圈。當使用 `v1_iter` 疊代器的 `for` 迴圈被呼叫時，疊代器中的每個元素才會在迴圈中每次疊代中使用，以此印出每個數值。

```
let v1 = vec![1, 2, 3];

let v1_iter = v1.iter();

for val in v1_iter {
    println!("取得：{}", val);
}
```

範例 13-11：在 `for` 迴圈使用疊代器

在標準函式庫沒有提供疊代器的語言中，你可能會用別種方式寫這個相同的函式，像是先從一個變數 0 作為索引開始、使用該變數索引向量來獲取數值，然後在迴圈中增加變數的值直到它抵達向量的總長。

疊代器會為你處理這些所有邏輯，減少重複且你可能會搞砸的程式碼。疊代器還能讓你靈活地將相同的邏輯用於不同的序列，而不只是像向量這種你能進行索引的資料結構。讓我們研究看看疊代器怎麼辦到的。

Iterator 特徵與 next 方法

所有的疊代器都會實作定義在標準函式庫的 `Iterator` 特徵。特徵的定義如以下所示：

```
pub trait Iterator {
    type Item;

    fn next(&mut self) -> Option<Self::Item>;

    // 以下省略預設實作
}
```

注意到此定義使用了一些新的語法：`type Item` 與 `Self::Item`，這是此特徵定義的**關聯型別 (associated type)**。我們會在第十九章進一步探討關聯型別。現在你只需要知道此程式碼表示要實作 `Iterator` 特徵的話，你還需要定義 `Item` 型別，而此 `Item` 型別會用在方法 `next` 的回傳型別中。換句話說，`Item` 型別會是從疊代器回傳的型別。

`Iterator` 型別只要求實作者定義一個方法：`next` 方法會用 `Some` 依序回傳疊代器中的每個項目，並在疊代器結束時回傳 `None`。

我們可以直接在疊代器呼叫 `next` 方法。範例 13-12 展示從向量建立的疊代器重複呼叫 `next` 每次會得到什麼數值。

檔案名稱：src/lib.rs

```
#[test]
fn iterator_demonstration() {
    let v1 = vec![1, 2, 3];

    let mut v1_iter = v1.iter();

    assert_eq!(v1_iter.next(), Some(&1));
    assert_eq!(v1_iter.next(), Some(&2));
    assert_eq!(v1_iter.next(), Some(&3));
    assert_eq!(v1_iter.next(), None);
}
```

範例 13-12：對疊代器呼叫 `next` 方法

注意到 `v1_iter` 需要是可變的：在疊代器上呼叫 `next` 方法會改變疊代器內部用來紀錄序列位置的狀態。換句話說，此程式碼消耗或者說使用了疊代器。每次 `next` 的呼叫會從疊代器消耗一個項目。而我們不必在 `for` 迴圈指定 `v1_iter` 為可變是因為迴圈會取得 `v1_iter` 的所有權並在內部將其改為可變。

另外還要注意的是我們從 `next` 呼叫取得的是向量中數值的不可變參考。`iter` 方法會從疊代器中產生不可變參考。如果我們想要一個取得 `v1` 所有權的疊代器，我們可以呼叫 `into_iter` 而非 `iter`。同樣地，如果我們想要遍歷可變參考，我們可以呼叫 `iter_mut` 而非 `iter`。

消耗疊代器的方法

標準函式庫提供的 `Iterator` 特徵有一些不同的預設實作方法，你可以查閱標準函式庫的 `Iterator` 特徵 API 技術文件來找到這些方法。其中有些方法就是在它們的定義呼叫 `next` 方法，這就是為何當你實作 `Iterator` 特徵時需要提供 `next` 方法的實作。

會呼叫 `next` 的方法被稱之為**消耗配接器 (consuming adaptors)**，因為呼叫它們會使用掉疊代器。其中一個例子就是方法 `sum`，這會取得疊代器的所有權並重複呼叫 `next` 來遍歷所有項目，因而消耗掉疊代器。隨著遍歷的過程中，他會將每個項目加到總計中，並在疊代完成時回傳總計數值。範例 13-13 展示了一個使用 `sum` 方法的測試：

檔案名稱：src/lib.rs

```
#[test]
fn iterator_sum() {
    let v1 = vec![1, 2, 3];

    let v1_iter = v1.iter();

    let total: i32 = v1_iter.sum();

    assert_eq!(total, 6);
}
```

範例 13-13：呼叫 `sum` 方法來取得疊代器中所有項目的總計數值

我們呼叫 `sum` 之後就不再被允許使用 `v1_iter` 了，因為 `sum` 取得了疊代器的所有權。

產生其他疊代器的方法

疊代配接器 (iterator adaptors) 是定義在 `Iterator` 特徵的方法，它們不會消耗掉疊代器。它們會改變原本疊代器的一些屬性來產生不同的疊代器。

範例 13-14 呼叫了疊代器的疊代配接器方法 `map`，它會取得一個閉包在進行疊代時對每個項目進行呼叫。`map` 方法會回傳個項目被改變過的新疊代器。這裡的閉包會將向量中的每個項目加 1 來產生新的疊代器：

檔案名稱：src/main.rs

```
let v1: Vec<i32> = vec![1, 2, 3];

v1.iter().map(|x| x + 1);
```

範例 13-14：呼叫疊代配接器 `map` 來建立新的疊代器

不過此程式碼會產生個警告：

```

$ cargo run
   Compiling iterators v0.1.0 (file:///projects/iterators)
warning: unused `Map` that must be used
--> src/main.rs:4:5
  |
4 |     v1.iter().map(|x| x + 1);
  |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |
= note: `#[warn(unused_must_use)]` on by default
= note: iterators are lazy and do nothing unless consumed

warning: `iterators` (bin "iterators") generated 1 warning
   Finished dev [unoptimized + debuginfo] target(s) in 0.47s
   Running `target/debug/iterators`

```

範例 13-14 的程式碼不會做任何事情，我們指定的閉包沒有被呼叫到半次。警告提醒了我們原因：疊代配接器是惰性的，我們必須在此消耗疊代器才行。

要修正並消耗此疊代器，我們將使用 `collect` 方法，這是我們在範例 12-1 搭配 `env::args` 使用的方法。此方法會消耗疊代器並收集結果數值至一個資料型別集合。

在範例 13-15 中，我們將遍歷 `map` 呼叫所產生的疊代器結果數值收集到一個向量中。此向量最後會包含原本向量每個項目都加 1 的數值。

檔案名稱：src/main.rs

```

let v1: Vec<i32> = vec![1, 2, 3];

let v2: Vec<_> = v1.iter().map(|x| x + 1).collect();

assert_eq!(v2, vec![2, 3, 4]);

```

範例 13-15：呼叫方法 `map` 來建立新的疊代器並呼叫 `collect` 方法來消耗新的疊代器來產生向量

因為 `map` 接受一個閉包，我們可以對每個項目指定任何我們想做的動作。這是一個展示如何使用閉包來自訂行為，同時又能重複使用 `Iterator` 特徵提供的遍歷行為的絕佳例子。

你可以透過疊代配接器串連多重呼叫，在進行一連串複雜運算的同時，仍保持良好的閱讀性。但因為所有的疊代器都是惰性的，你必須呼叫能消耗配接器的方法來取得疊代配接器的結果。

使用閉包獲取它們的環境

許多疊代配接器都會拿閉包作為引數，而通常我們向疊代配接器指定的閉包引數都能獲取它們周圍的環境。

在以下例子中，我們使用 `filter` 方法來取得閉包。閉包會取得疊代器的每個項目並回傳布林值。如果閉包回傳 `true`，該數值就會被包含在 `filter` 產生的疊代器中；如果閉包回傳 `false`，該數值就不會被包含在結果疊代器中。

在範例 13-16 中我們使用 `filter` 與一個從它的環境獲取變數 `shoe_size` 的閉包來遍歷一個有 `Shoe` 結構體實例的集合。它會回傳只有符合指定大小的鞋子：

檔案名稱：src/lib.rs

```

#[derive(PartialEq, Debug)]
struct Shoe {
    size: u32,
    style: String,
}

fn shoes_in_size(shoes: Vec<Shoe>, shoe_size: u32) -> Vec<Shoe> {
    shoes.into_iter().filter(|s| s.size == shoe_size).collect()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn filters_by_size() {
        let shoes = vec![
            Shoe {
                size: 10,
                style: String::from("運動鞋"),
            },
            Shoe {
                size: 13,
                style: String::from("涼鞋"),
            },
            Shoe {
                size: 10,
                style: String::from("靴子"),
            },
        ];

        let in_my_size = shoes_in_size(shoes, 10);

        assert_eq!(
            in_my_size,
            vec![
                Shoe {
                    size: 10,
                    style: String::from("運動鞋")
                },
                Shoe {
                    size: 10,
                    style: String::from("靴子")
                },
            ]
        );
    }
}

```

範例 13-16：使用 `filter` 方法與一個獲取 `shoe_size` 的閉包

函式 `shoes_in_size` 會取得鞋子向量的所有權以及一個鞋子大小作為參數。它會回傳只有符合指定大小的鞋子向量。

在 `shoes_in_size` 的本體中，我們呼叫 `into_iter` 來建立一個會取得向量所有權的疊代器。然後我們呼叫 `filter` 來將該疊代器轉換成只包含閉包回傳為 `true` 的元素的新疊代器。

閉包會從環境獲取 `shoe_size` 參數並比較每個鞋子數值的大小，讓只有符合大小的鞋子保留下來。最後呼叫 `collect` 來收集疊代器回傳的數值進一個函式會回傳的向量。

此測試顯示了當我們呼叫 `shoes_in_size` 時，我們會得到我們指定相同大小的鞋子。

改善我們的 I/O 專案

有了疊代器這樣的新知識，我們可以使用疊代器來改善第十二章的 I/O 專案，讓程式碼更清楚與簡潔。我們來看看疊代器如何改善 `Config::build` 函式與 `search` 函式的實作。

使用疊代器移除 `clone`

在範例 12-6 中，我們加了些程式碼來取得 `String` 數值的切片並透過索引切片與克隆數值來產生 `Config` 實例，讓 `Config` 結構體能擁有其數值。在範例 13-17 中，我們重現了範例 12-23 的 `Config::build` 函式實作：

檔案名稱：src/lib.rs

```
impl Config {
    pub fn build(args: &[String]) -> Result<Config, &'static str> {
        if args.len() < 3 {
            return Err("引數不足");
        }

        let query = args[1].clone();
        let file_path = args[2].clone();

        let ignore_case = env::var("IGNORE_CASE").is_ok();

        Ok(Config {
            query,
            file_path,
            ignore_case,
        })
    }
}
```

範例 13-17：重現範例 12-23 的 `Config::build` 函式

當時我們說先不用擔心 `clone` 呼叫帶來的效率問題，因為我們會在之後移除它們。現在正是絕佳時機！

我們在此需要 `clone` 的原因為我們的參數 `args` 是擁有 `String` 元素的切片，但是 `build` 函式並不擁有 `args`。要回傳 `Config` 實例的所有權，我們必須克隆數值給 `Config` 的 `query` 與 `file_path` 欄位，`Config` 實例才能擁有其值。

有了我們新學到的疊代器，我們可以改變 `build` 函式來取得疊代器的所有權來作為引數，而非借用切片。我們會來使用疊代器的功能，而不是檢查切片長度並索引特定位置。這能讓

`Config::build` 函式的意圖更清楚，因為疊代器會存取數值。

一旦 `Config::build` 取得疊代器的所有權並不再使用借用的索引動作，我們就可以從疊代器中移動 `String` 的數值至 `Config` 而非呼叫 `clone` 來產生新的配置。

直接使用回傳的疊代器

請開啟你的 I/O 專案下的 `src/main.rs` 檔案，這應該會看起來像這樣：

檔案名稱：`src/main.rs`

```
fn main() {
    let args: Vec<String> = env::args().collect();

    let config = Config::build(&args).unwrap_or_else(|err| {
        eprintln!("解析引數時出現問題：{err}");
        process::exit(1);
    });

    // --省略--
}
```

我們首先會改變範例 12-24 的 `main` 函式開頭段落成範例 13-18 這段使用疊代器的程式碼。這在我們更新 `Config::build` 之前都還無法編譯。

檔案名稱：`src/main.rs`

```
fn main() {
    let config = Config::build(env::args()).unwrap_or_else(|err| {
        eprintln!("解析引數時出現問題：{err}");
        process::exit(1);
    });

    // --snip--
}
```



範例 13-18：傳遞 `env::args` 的回傳值給 `Config::build`

`env::args` 函式回傳的是疊代器！與其收集疊代器的數值成一個向量再傳遞切片給 `Config::build`，現在我們可以直接傳遞 `env::args` 回傳的疊代器所有權給 `Config::build`。

接下來，我們需要更新 `Config::build` 的定義。在 I/O 專案的 `src/lib.rs` 檔案中，讓我們變更 `Config::build` 的簽名成範例 13-19 的樣子。這還無法編譯，因為我們需要更新函式本體。

檔案名稱：`src/lib.rs`

```
impl Config {
    pub fn build(
        mut args: impl Iterator<Item = String>,
    ) -> Result<Config, &'static str> {
        // --snip--
    }
}
```



範例 13-19：更新 `Config::build` 的簽名來接收疊代器

標準函式庫技術文件顯示 `env::args` 函式回傳的疊代器型別為 `std::env::Args`，而該疊代器有實作 `Iterator` 特徵並回傳 `String` 數值。

我們更新了 `Config::build` 函式的簽名，讓參數 `args` 擁有個特徵界限為 `impl Iterator<Item = String>` 的泛型型別而非 `&[String]`。我們在第十章的「特徵作為參數」段落討論過 `impl Trait` 的語法，這樣的用法讓 `args` 可以接收任何有實作 `Iterator` 並回傳 `String` 數值的型別。

因為我們取得了 `args` 的所有權，而且我們需要將 `args` 成為可變的讓我們可以疊代它，所以我們將關鍵字 `mut` 加到 `args` 的參數指定使其成為可變的。

使用 `Iterator` 特徵方法而非索引

接下來，我們要修正 `Config::build` 的本體。因為 `args` 有實作 `Iterator` 特徵，所以我們知道我們可以對它呼叫 `next` 方法！範例 13-20 更新了範例 12-23 的程式碼來使用 `next` 方法：

檔案名稱：`src/lib.rs`

```
impl Config {
    pub fn build(
        mut args: impl Iterator<Item = String>,
    ) -> Result<Config, &'static str> {
        args.next();

        let query = match args.next() {
            Some(arg) => arg,
            None => return Err("無法取得搜尋字串"),
        };

        let file_path = match args.next() {
            Some(arg) => arg,
            None => return Err("無法取得檔案路徑"),
        };

        let ignore_case = env::var("IGNORE_CASE").is_ok();

        Ok(Config {
            query,
            file_path,
            ignore_case,
        })
    }
}
```

範例 13-20：變更 `Config::build` 的本體來使用疊代器方法

我們還記得 `env::args` 回傳的第一個數值會是程式名稱。我們想要忽略該值並取得下個數值，所以我們第一次呼叫 `next` 時不會對回傳值做任何事。再來我們才會呼叫 `next` 來取得我們想要的數值置入 `Config` 中的 `query` 欄位。如果 `next` 回傳 `Some` 的話，我們使用 `match` 來提取數值。如果它回傳 `None` 的話，這代表引數不足，所以我們提早用 `Err` 數值回傳。我們對 `file_path` 數值也做一樣的事。

透過疊代配接器讓程式碼更清楚

我們也可以對 I/O 專案中的 `search` 函式利用疊代器的優勢，範例 13-21 重現了範例 12-19 的程式碼：

檔案名稱：src/lib.rs

```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    let mut results = Vec::new();

    for line in contents.lines() {
        if line.contains(query) {
            results.push(line);
        }
    }

    results
}
```

範例 13-21：範例 12-19 的 `search` 函式實作

我們可以使用疊代配接器（iterator adaptor）方法讓此程式碼更精簡。這樣做也能避免我們產生過程中的 `results` 可變向量。函式程式設計風格傾向於最小化可變狀態的數量使程式碼更加簡潔。移除可變狀態還在未來有機會讓搜尋可以平行化，因為我們不需要去管理 `results` 向量的並行存取。範例 13-22 展示了此改變：

檔案名稱：src/lib.rs

```
pub fn search<'a>(query: &str, contents: &'a str) -> Vec<&'a str> {
    contents
        .lines()
        .filter(|line| line.contains(query))
        .collect()
}
```

範例 13-22：對 `search` 函式實作使用疊代配接器方法

回想一下 `search` 函式的目的是要回傳 `contents` 中所有包含 `query` 的行數。類似於範例 13-16 的 `filter` 範例，此程式碼使用 `filter` 配接器來只保留 `line.contains(query)` 回傳為 `true` 的行數。我們接著就可以用 `collect` 收集符合的行數成另一個向量。這樣簡單多了！你也可以對 `search_case_insensitive` 函式使用疊代器方法做出相同的改變。

迴圈與疊代器之間的選擇

接下來的邏輯問題是在你自己的程式碼中你應該與為何要使用哪種風格呢：是要原本範例 13-21 的程式碼，還是範例 13-22 使用疊代器的版本呢？大多數的 Rust 程式設計師傾向於使用疊代器。一開始的確會有點難上手，不過一旦你熟悉各種疊代配接器與它們的用途後，疊代器就會很好理解了。不同於用迴圈迂迴處理每一步並建構新的向量，疊代器能更專注在迴圈的高階抽象上。這能抽象出常見的程式碼，並能更容易看出程式碼中的重點部位，比如疊代器中每個元素要過濾的條件。

但是這兩種實作真的完全相等嗎？你的直覺可能會假設低階的迴圈可能更快些。讓我們來討論效能吧。

比較效能：迴圈 vs. 疊代器

為了決定該使用迴圈還是疊代器，你需要知道哪個實作比較快：是顯式 `for` 迴圈的版本，還是疊代器的版本。

我們可以透過讀取整本 Sir Arthur Conan Doyle 寫的 *The Adventures of Sherlock Holmes* 到 `String` 中並搜尋內容中的 *the* 來進行評測。以下為針對 `search` 函式使用 `for` 迴圈與使用疊代器的版本評測 (benchmark)：

```
test bench_search_for ... bench: 19,620,300 ns/iter (+/- 915,700)
test bench_search_iter ... bench: 19,234,900 ns/iter (+/- 657,200)
```

疊代器版本竟然比較快一些！我們在此不會解釋評測的程式碼，因為這裡的重點不在於證明這兩種版本是一樣快的，而是要理解這兩種實作對效能的影響。

要做更全面的評測的話，你應該要檢查使用不同大小的不同文字來作為 `contents`、不同單字與不同長度來作為 `query`，以及所有各式各樣的可能性。這邊的重點在於：疊代器雖然是高階抽象，但其編譯出來的程式碼與你親自寫出低階的程式碼幾乎相同。疊代器是 Rust 其中一種**零成本抽象** (zero-cost abstractions)，這指的是使用的抽象不會在執行時有額外的開銷。這是 C++ 的初始設計暨實作者 Bjarne Stroustrup 在《Foundations of C++》(2012) 書中所定義的**零開銷** (zero-overhead) 的概念：

大致上來說，C++ 的實作遵守著零開銷的原則：你沒有使用到的話，你就不必買單。而且你有使用到的話，你不可能再寫出更好的程式碼。

作為另一個例子，以下程式碼是音訊解碼器的其中一個段落。解碼演算碼使用線性預測數學運算，並依據之前樣本的線性函式來預估未來的數值。此程式碼使用一連串的疊代器來對作用域中的三個變數進行數學運算：資料切片 `buffer`、長度為 12 的 `coefficients` 陣列以及要偏移的數量 `qlp_shift`。我們在範例中宣告變數但沒有給予任何數值，雖然只看此程式碼的確沒有什麼意義，但是這仍然是個現實世界中的其中一個簡例，可以來看出 Rust 如何將高階的想法轉換成低階的程式碼。

```

let buffer: &mut [i32];
let coefficients: [i64; 12];
let qlp_shift: i16;

for i in 12..buffer.len() {
    let prediction = coefficients.iter()
        .zip(&buffer[i - 12..i])
        .map(|(&c, &s)| c * s as i64)
        .sum::<i64>() >> qlp_shift;

    let delta = buffer[i];
    buffer[i] = prediction as i32 + delta;
}

```

要計算 `prediction` 的數值的話，此程式碼會遍歷 `coefficients` 的 12 個數值並使用 `zip` 方法將係數數值與 `buffer` 中之前 12 個數值做配對。然後在每個配對中，我們將數值相乘，然後相加所有結果，最後對總和往右偏移 `qlp_shift` 位。

像音訊解碼器這種的應用程式運算通常最注重效率。我們在此建立了一個疊代器，使用兩個配接器，然後消化數值。這段 Rust 程式碼會產生什麼樣的組合語言程式碼呢？在本書撰寫時，它會編譯出與你自己手寫一樣的組合語言。遍歷 `coefficients` 的數值完全不需用到迴圈：Rust 知道一共有 12 次疊代，所以它會「展開 (unroll)」迴圈。**展開**是一種優化方式，這會移除迴圈控制程式碼並改產生針對迴圈中每次疊代的重複程式碼。

所有的係數都會存在暫存器中，這意味著存取數值會非常地快。在執行時不會有對陣列的界限檢查。這些所有 Rust 能做的優化讓產生的程式碼可以十分迅速。現在既然你已經知道了，你就可以自在地使用疊代器與閉包！它們可以寫出高階的程式碼，但不會犧牲執行時的效能。

總結

閉包與疊代器是 Rust 啟發自函式程式語言的概念。它們讓 Rust 在表達高階概念的同時，仍能擁有低階程式碼的效能。閉包與疊代器的實作對執行時效能不會有影響。這是 Rust 竭力提供零成本抽象的目標之一。

現在我們改善了 I/O 專案的可讀性，讓我們看看 `cargo` 一些更多的功能，來幫助我們分享專案給全世界。

更多關於 Cargo 與 Crates.io 的內容

目前我們只使用了 Cargo 最基本的功能來建構、執行與測試我們的程式碼，但它還能做更多事。在本章節中我們將討論這些其他的進階功能，你將瞭解如何做到以下動作：

- 透過發佈設定檔來自訂你的建構
- 發佈函式庫到 crates.io
- 透過工作空間組織大型專案
- 從 crates.io 安裝執行檔
- 使用自訂命令擴展 Cargo 的功能

Cargo 能做的事比本章會介紹到的功能還多，所以想要知道它所有功能的話，歡迎查閱[它的技術文件](#)。

透過發佈設定檔自訂建構

在 Rust 中發佈設定檔（release profiles）是個預先定義好並可用不同配置選項來自訂的設定檔，能讓程式設計師掌控更多選項來編譯程式碼。每個設定檔的配置彼此互相獨立。

Cargo 有兩個主要的設定檔：`dev` 設定檔會在當你對 Cargo 執行 `cargo build` 時所使用；`release` 設定檔會在當你對 Cargo 執行 `cargo build --release` 時所使用。`dev` 設定檔預設定義為適用於開發時使用，而 `release` 設定檔預設定義為適用於發佈時使用。

你可能會覺得這些設定檔名稱很眼熟，因為它們就已經顯示在輸出結果過：

```
$ cargo build
  Finished dev [unoptimized + debuginfo] target(s) in 0.0s
$ cargo build --release
  Finished release [optimized] target(s) in 0.0s
```

`dev` 與 `release` 是編譯器會使用到的不同設定檔。

當專案的 *Cargo.toml* 中你沒有顯式加上任何 `[profile.*]` 段落的話，Cargo 就會使用每個設定檔的預設設置。透過對你想要自訂的任何設定檔加上 `[profile.*]` 段落，你可以覆寫任何預設設定的子集。舉例來說，以下是 `dev` 與 `release` 設定檔中 `opt-level` 設定的預設數值：

檔案名稱：*Cargo.toml*

```
[profile.dev]
opt-level = 0

[profile.release]
opt-level = 3
```

`opt-level` 設定控制了 Rust 對程式碼進行優化的程度，範圍從 0 到 3。提高優化程度會增加編譯時間，所以如果你在開發過程中得時常編譯程式碼的話，你傾向於編譯快一點而不管優化的多寡，就算結果程式碼會執行的比較慢。這就是 `dev` 的 `opt-level` 預設為 0 的原因。當你準備好要發佈你的程式碼時，則最好花多點時間來編譯。你只需要在發佈模式編譯一次，但你的編譯程式則會被執行很多次，所以發佈模式選擇花費多點編譯時間來讓程式跑得比較快。這就是 `release` 的 `opt-level` 預設為 3 的原因。

你可以在 *Cargo.toml* 加上不同的數值來覆蓋預設設定。舉例來說，如果我們希望在開發設定檔使用優化等級 1 的話，我們可以在專案的 *Cargo.toml* 檔案中加上這兩行：

檔案名稱：*Cargo.toml*


```
[profile.dev]
opt-level = 1
```

這樣就會覆蓋預設設定 `0`。現在當我們執行 `cargo build`，Cargo 就會使用 `dev` 設定檔的預設值以及我們自訂的 `opt-level`。因為我們將 `opt-level` 設為 `1`，Cargo 會比原本的預設進行更多優化，但沒有發佈建構那麼多。

對於完整的設置選項與每個設定檔的預設列表，請查閱 [Cargo 的技術文件](#)。

發佈 Crate 到 Crates.io

我們已經使用過 crates.io 的套件來作為我們專案的依賴函式庫，但是你也可以發佈你自己的套件來將你的程式碼提供給其他人使用。crates.io 會發行套件的原始碼，所以它主要用來託管開源程式碼。

Rust 與 Cargo 有許多功能可以幫助其他人更容易找到並使用你發佈的套件。我們會介紹其中一些功能並解釋如何發佈套件。

寫上有幫助的技術文件註解

準確地加上套件的技術文件有助於其他使用者知道如何及何時使用它們，所以投資時間在寫技術文件上是值得的。在第三章我們提過如何使用兩條斜線 `//` 來加上 Rust 程式碼註解。Rust 還有個特別的註解用來作為技術文件，俗稱為**技術文件註解 (documentation comment)**，這能用來產生 HTML 技術文件。這些 HTML 顯示公開 API 項目中技術文件註解的內容，讓對此函式庫有興趣的開發者知道如何使用你的 crate，而不需知道 crate 是如何實作的。

技術文件註解使用三條斜線 `///` 而不是兩條，並支援 Markdown 符號來格式化文字。技術文件註解位於它們對應項目的上方。範例 14-1 顯示了 `my_crate` crate 中 `add_one` 的技術文件註解。

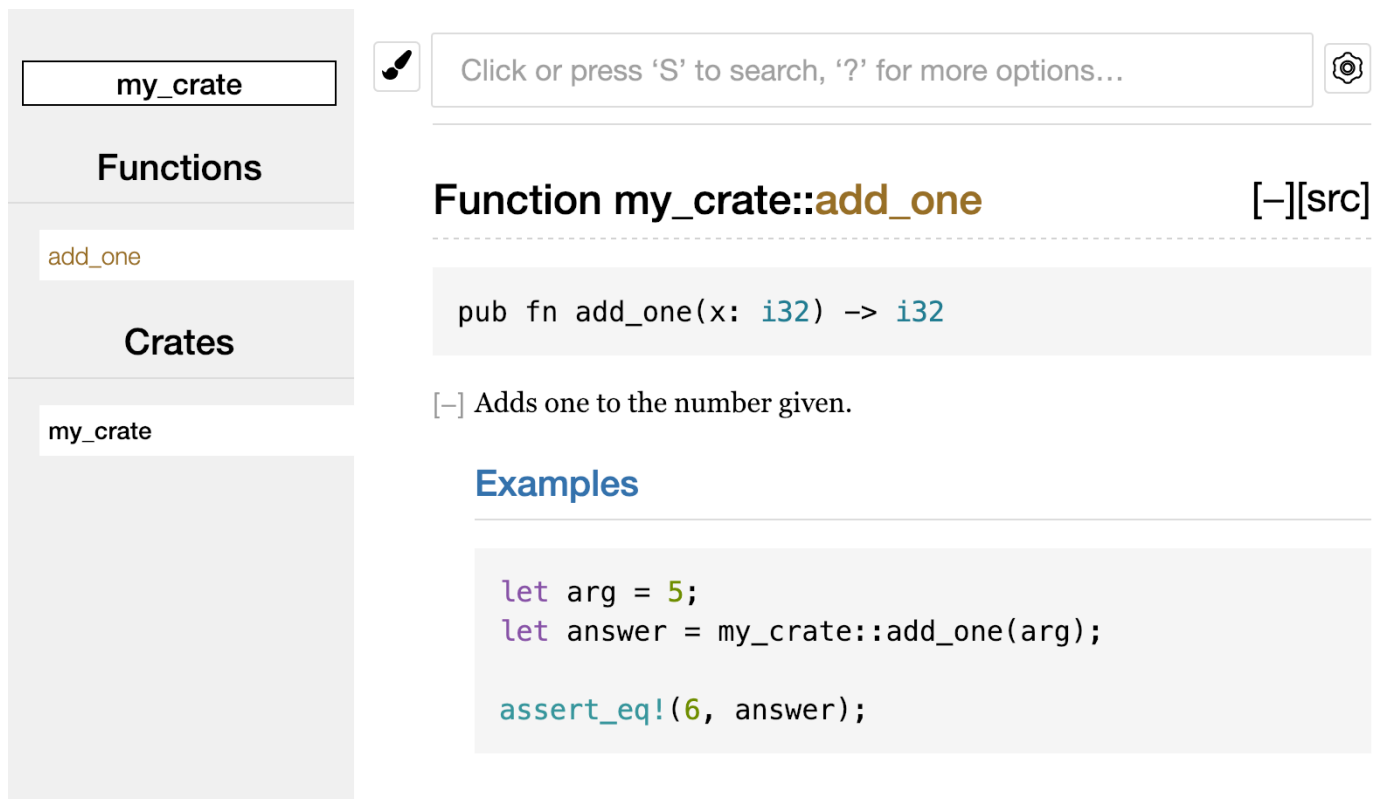
檔案名稱：`src/lib.rs`

```
/// Adds one to the number given.
///
/// # Examples
///
/// ```
/// let arg = 5;
/// let answer = my_crate::add_one(arg);
///
/// assert_eq!(6, answer);
/// ```
pub fn add_one(x: i32) -> i32 {
    x + 1
}
```

範例 14-1：函式的技術文件註解

我們在這裡加上了解釋函式 `add_one` 行為的描述、加上一個標題為 `Examples` 的段落並附上展示如何使用 `add_one` 函式的程式碼。我們可以透過執行 `cargo doc` 來從技術文件註解產生 HTML 技術文件。此命令會執行隨著 Rust 一起發佈的工具 `rustdoc`，並在 `target/doc` 目錄下產生 HTML 技術文件。

為了方便起見，你可以執行 `cargo doc --open` 來建構當前 crate 的 HTML 技術文件（以及 crate 所有依賴的技術文件）並在網頁瀏覽器中開啟結果。導向到函式 `add_one` 而你就能看到技術文件註解是如何呈現的，如圖示 14-1 所示：



圖示 14-1：函式 `add_one` 的 HTML 技術文件

常見技術文件段落

我們在範例 14-1 使用 `# Examples` Markdown 標題來在 HTML 中建立一個標題為「Examples」的段落。以下是 crate 技術文件中常見的段落標題：

- **Panics**：該函式可能會導致恐慌的可能場合。函式的呼叫者不希望他們的程式恐慌的話，就要確保他們沒有在這些情況下呼叫該函式。
- **Errors**：如果函式回傳 `Result`，解釋發生錯誤的可能種類以及在何種條件下可能會回傳這些錯誤有助於呼叫者，讓他們可以用不同方式來寫出處理不同種錯誤的程式碼。
- **Safety**：如果呼叫的函式是 `unsafe` 的話（我們會在第十九章討論不安全的議題），就必須要有個段落解釋為何該函式是不安全的，並提及函式預期呼叫者要確保哪些不變條件（invariants）。

大多數的技術文件註解不全都需要這些段落，但這些可能是使用者有興趣瞭解的內容，你可以作為提醒你的檢查列表。

將技術文件註解作為測試

在技術文件註解加上範例程式碼區塊有助於解釋如何使用你的函式庫，而且這麼做還有個額外好處：執行 `cargo test` 也會將你的技術文件視為測試來執行！在技術文件加上範例的確是最佳示範，但是如果程式碼在技術文件寫完之後變更的話，該範例可能就會無法執行了。如果我們對範例

14-1 中有附上技術文件的函式 `add_one` 執行 `cargo test` 的話，我們會看見測試結果有以下這樣的段落：

```
Doc-tests my_crate

running 1 test
test src/lib.rs - add_one (line 5) ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.27s
```

現在如果我們變更函式或範例使其內的 `assert_eq!` 會恐慌並再次執行 `cargo test` 的話，我們會看到技術文件測試能互相獲取錯誤，告訴我們範例與程式碼已經不同步了！

包含項目結構的註解

風格為 `///!` 技術文件註解會對其包含該註解的項目加上技術文件，而不是對註解後的項目加上技術文件。我們通常將此技術文件註解用於 crate 源頭檔（通常為 `src/lib.rs`）或模組來對整個 crate 或模組加上技術文件。

舉例來說，如果我們希望能加上技術文件來描述包含 `add_one` 函式的 `my_crate` 目的，我們可以用 `///!` 在 `src/lib.rs` 檔案開頭加上技術文件註解，如範例 14-2 所示：

檔案名稱：`src/lib.rs`

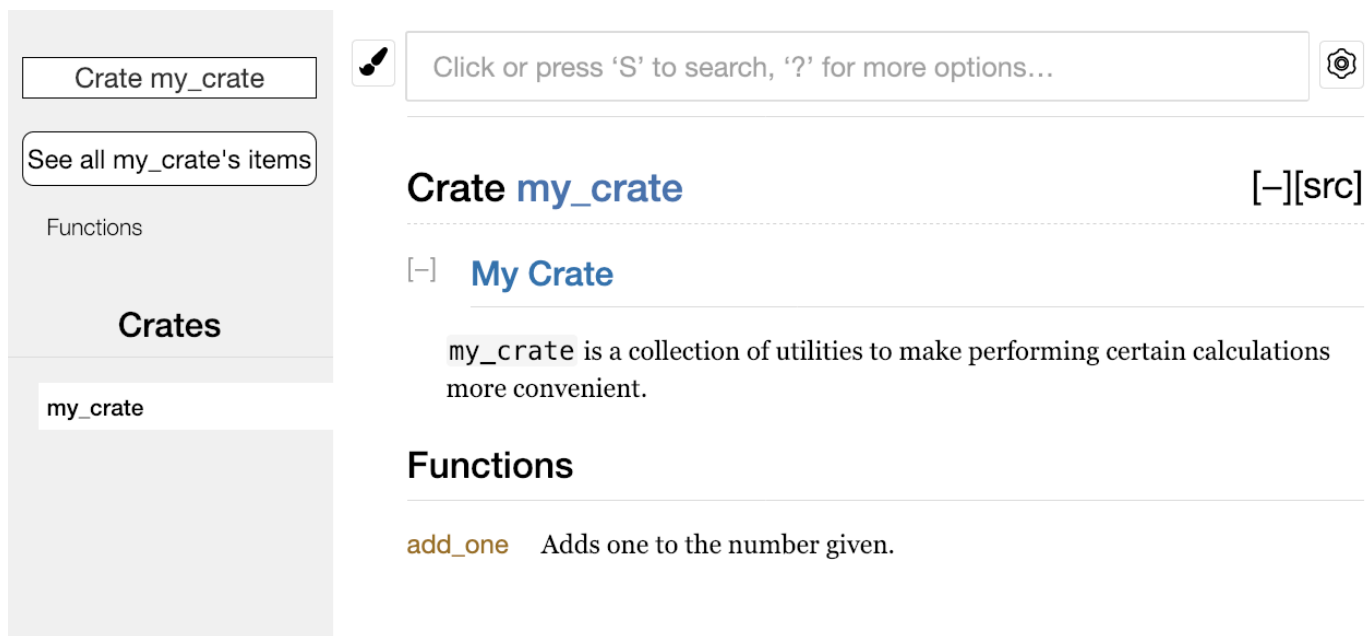
```
///! # My Crate
///!
///! `my_crate` is a collection of utilities to make performing certain
///! calculations more convenient.

/// Adds one to the number given.
// --省略--
```

範例 14-2：描述整個 `my_crate` crate 的技術文件

注意到 `///!` 最後一行之後並沒有緊貼任何程式碼，因為我們是用 `///!` 而非 `///` 來下註解，我們是對包含此註解的整個項目加上技術文件，而不是此註解之後的項目。在此例中，該項目就是 `src/lib.rs` 檔案，也就是 crate 的源頭。這些註解會描述整個 crate。

當我們執行 `cargo doc --open`，這些註解會顯示在 `my_crate` 技術文件的首頁，位於 crate 公開項目列表的上方，如圖示 14-2 所示：



圖示 14-2： `my_crate` 的技術文件，包含描述整個 crate 的註解

項目中的技術文件註解可以用來分別描述 crate 和模組。用它們來將解釋容器整體的目的有助於你的使用者瞭解該 crate 的程式碼組織架構。

透過 `pub use` 匯出理想的公開 API

公開 API 的架構是發佈 crate 時要考量到的一大重點。使用 crate 的人可能並沒有你那麼熟悉其中的架構，而且如果你的 crate 模組分層越深的話，他們可能就難以找到他們想使用的部分。

在第七章中，我們介紹了如何使用 `mod` 關鍵字來組織我們的程式碼成模組、如何使用 `pub` 關鍵字來公開項目，以及如何使用 `use` 關鍵字來將項目引入作用域。然而在開發 crate 時的架構雖然對你來說是合理的，但對你的使用者來說可能就不是那麼合適了。你可能會希望用有數個層級的分層架構來組織你的程式碼，但是要是有人想使用你定義在分層架構裡的型別時，它們可能就很難發現這些型別的存在。而且輸入 `use my_crate::some_module::another_module::UsefulType;` 是非常惱人的，我們會希望輸入 `use my_crate::UsefulType;` 就好。

好消息是如果你的架構不便於其他函式庫所使用的話，你不必重新組織你的內部架構：你可以透過使用 `pub use` 選擇重新匯出（re-export）項目來建立一個不同於內部私有架構的公開架構。重新匯出會先取得某處的公開項目，再從其他地方使其公開，讓它像是被定義在其他地方一樣。

舉例來說，我們建立了一個函式庫叫做 `art` 來模擬藝術概念。在函式庫中有兩個模組：`kinds` 模組包含兩個列舉 `PrimaryColor` 和 `SecondaryColor`；而 `utils` 模組包含一個函式 `mix`，如範例 14-3 所示：

檔案名稱：`src/lib.rs`

```

//! # Art
//!
//! A library for modeling artistic concepts.

pub mod kinds {
    /// The primary colors according to the RYB color model.
    pub enum PrimaryColor {
        Red,
        Yellow,
        Blue,
    }

    /// The secondary colors according to the RYB color model.
    pub enum SecondaryColor {
        Orange,
        Green,
        Purple,
    }
}

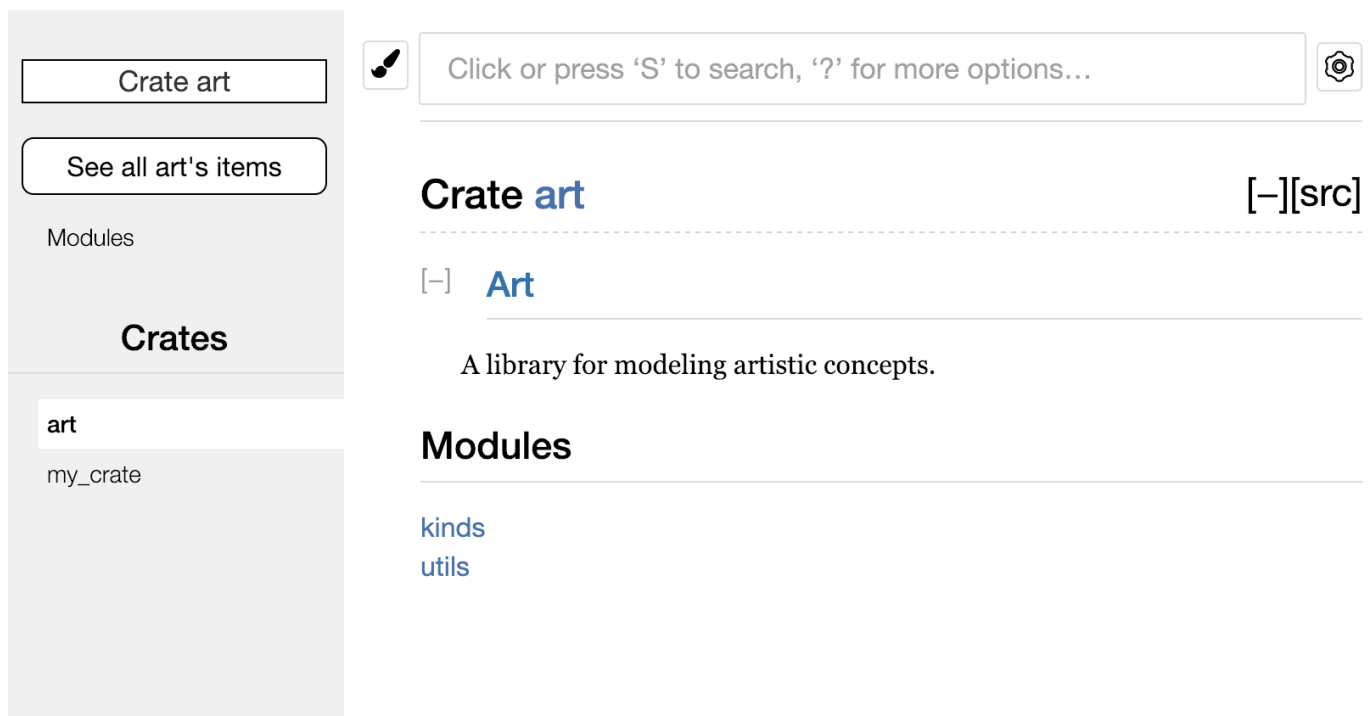
pub mod utils {
    use crate::kinds::*;

    /// Combines two primary colors in equal amounts to create
    /// a secondary color.
    pub fn mix(c1: PrimaryColor, c2: PrimaryColor) -> SecondaryColor {
        // --省略--
    }
}

```

範例 14-3：函式庫 `art` 有兩個模組項目 `kinds` 和 `utils`

圖示 14-3 顯示了此 crate 透過 `cargo doc` 產生的技術文件首頁：



圖示 14-3：art 的技術文件首頁陳列了 kinds 和 utils 模組

注意到 `PrimaryColor` 與 `SecondaryColor` 型別沒有列在首頁，而函式 `mix` 也沒有。我們必須點擊 `kinds` 與 `utils` 才能看到它們。

其他依賴此函式庫的 crate 需要使用 `use` 陳述式來將 `art` 的項目引入作用域中，並指定當前模組定義的架構。範例 14-4 顯示了從 `art` crate 使用 `PrimaryColor` 和 `mix` 項目的 crate 範例：

檔案名稱：src/main.rs

```
use art::kinds::PrimaryColor;
use art::utils::mix;

fn main() {
    let red = PrimaryColor::Red;
    let yellow = PrimaryColor::Yellow;
    mix(red, yellow);
}
```

範例 14-4：一個使用 art 並匯出內部架構項目的 crate

範例 14-4 中使用 `art` crate 的程式碼作者必須搞清楚 `PrimaryColor` 位於 `kinds` 模組中而 `mix` 位於 `utils` 模組中。`art` crate 的模組架構對開發 `art` crate 的開發者才比較有意義，對使用者來說就沒那麼重要。內部架構沒有提供什麼有用的資訊給想要知道如何使用 `art` crate 的人，還容易造成混淆，因為開發者得自己搞清楚要從何處找起，而且必須在 `use` 陳述式中指定每個模組名稱。

要從公開 API 移除內部架構，我們可以修改範例 14-3 中 `art` crate 的程式碼，並加上 `pub use` 陳述式來在頂層重新匯出項目，如範例 14-5 所示：

檔案名稱：src/lib.rs

```
//! # Art
//!
//! A library for modeling artistic concepts.

pub use self::kinds::PrimaryColor;
pub use self::kinds::SecondaryColor;
pub use self::utils::mix;

pub mod kinds {
    // --省略--
}

pub mod utils {
    // --省略--
}
```

範例 14-5：加上 `pub use` 陳述式來重新匯出項目

`cargo doc` 對此 crate 產生的 API 技術文件現在就會顯示與連結重新匯出的項目到首頁中，如圖示 14-4 所示。讓 `PrimaryColor` 與 `SecondaryColor` 型別以及函式 `mix` 更容易被找到。

The screenshot shows the Rust documentation page for the 'art' crate. The page layout includes a sidebar on the left with a search bar and navigation links. The main content area displays the crate name 'Crate art' with a '[src]' link, followed by 'Art' with a '[-]' link. Below this is the description 'A library for modeling artistic concepts.' and a 'Re-exports' section listing 'PrimaryColor', 'SecondaryColor', and 'mix'. The 'Modules' section lists 'kinds' and 'utils'.

圖示 14-4：art 的技術文件首頁會連結重新匯出的結果

`art` crate 使用者仍可以看到並使用範例 14-3 的內部架構，如範例 14-4 所展示的方式，或者它們可以使用像範例 14-5 這樣更方便的架構，如範例 14-6 所示：

檔案名稱：src/main.rs

```
use art::mix;
use art::PrimaryColor;

fn main() {
    // --省略--
}
```

範例 14-6：使用從 `art` crate 重新匯出項目的程式

如果你有許多巢狀模組（nested modules）的話，在頂層透過 `pub use` 重新匯出型別可以大大提升使用 crate 的體驗。另一項 `pub use` 的常見用途是重新匯出目前 crate 依賴的定義，讓那些 crate 定義成會你的 crate 公開 API 的一部分。

提供實用的公開 API 架構更像是一門藝術而不只是科學，而你可以一步步來尋找最適合使用者的 API 架構。使用 `pub use` 可以給你更多組織 crate 內部架構的彈性，並將內部架構與你要呈現給使用者的介面互相解偶（decouple）。你可以觀察一些你安裝過的程式碼，看看它們的內部架構是不是不同於它們的公開 API。

設定 Crates.io 帳號

在你可以發佈任何 crate 之前，你需要建立一個 crates.io 的帳號並取得一個 API token。請前往 crates.io 的首頁並透過 GitHub 帳號來登入（GitHub 目前是必要的，但未來可能會支援其他建立帳號的方式）一旦你登入好了之後，到你的帳號設定 <https://crates.io/me/> 並索取你的 API key，然後用這個 API key 來執行 `cargo login` 命令，如以下所示：

```
$ cargo login abcdefghijklmnopqrstuvwxyz012345
```

此命令會傳遞你的 API token 給 Cargo 並儲存在本地的 `~/.cargo/credentials`。注意此 token 是個秘密（**secret**），千萬不要分享給其他人。如果你因為任何原因分享給任何人的話，你最好撤銷掉並回到 crates.io 產生新的 token。

新增詮釋資料到新的 Crate

讓我們假設你有個 crate 想要發佈。在發佈之前，你需要加上一些詮釋資料（metadata），也就是在 crate 的 `Cargo.toml` 檔案中 `[package]` 的段落內加上更多資料。

你的 crate 必須要有個獨特的名稱。雖然你在本地端開發 crate 時，你的 crate 可以是任何你想要的名稱。但是 crates.io 上的 crate 名稱採先搶先贏制。一旦有 crate 名稱被取走了，其他人就不能再使用該名稱來發佈 crate。在嘗試發佈 crate 前，最好先搜尋你想使用的名稱。如果該名稱已

被使用了，你就需要想另一個名稱，並在 *Cargo.toml* 檔案中 `[package]` 段落的 `name` 欄位使用新的名稱來發佈，如以下所示：

檔案名稱：Cargo.toml

```
[package]
name = "guessing_game"
```

當你選好獨特名稱後，此時執行 `cargo publish` 來發佈 crate 的話，你會得到以下警告與錯誤：

```
$ cargo publish
    Updating crates.io index
warning: manifest has no description, license, license-file, documentation,
homepage or repository.
See https://doc.rust-lang.org/cargo/reference/manifest.html#package-metadata
for more info.
--省略--
error: failed to publish to registry at https://crates.io

Caused by:
  the remote server responded with an error: missing or empty metadata fields:
description, license. Please see https://doc.rust-
lang.org/cargo/reference/manifest.html for how to upload metadata
```

這是因為你還缺少一些關鍵資訊：描述與授權條款是必須的，所以人們才能知道你的 crate 在做什麼以及在何種情況下允許使用。在 *Cargo.toml* 檔案中加上一兩句描述，它就會顯示在你的 crate 的搜尋結果中。至於 `license` 欄位，你需要給予 *license identifier value*。Linux Foundation's Software Package Data Exchange (SPDX) 有列出你可以使用的標識符數值。舉例來說，要指定你的 crate 使用 MIT 授權條款的話，就加上 `MIT` 標識符：

檔案名稱：Cargo.toml

```
[package]
name = "guessing_game"
license = "MIT"
```

如果你想使用沒有出現在 SPDX 的授權條款，你需要將該授權條款的文字儲存在一個檔案中、將該檔案加入你的專案中並使用 `license-file` 來指定該檔案名稱，而不使用 `license`。

你的專案適合使用什麼樣的授權條款超出了本書的範疇。不過 Rust 社群中許多人都會用 `MIT OR Apache-2.0` 雙授權條款作為它們專案的授權方式，這和 Rust 的授權條款一樣。這也剛好展示你也可以用 `OR` 指定數個授權條款，讓你的專案擁有數個不同的授權方式。

有了獨特名稱、版本、描述與授權條款，已經準備好發佈的 *Cargo.toml* 檔案會如以下所示：

檔案名稱：Cargo.toml

```
[package]
name = "guessing_game"
version = "0.1.0"
edition = "2021"
description = "A fun game where you guess what number the computer has chosen."
license = "MIT OR Apache-2.0"

[dependencies]
```

`Cargo` 技術文件還介紹了其他你可以指定的詮釋資料，讓你的 crate 更容易被其他人發掘並使用。

發佈至 Crates.io

現在你已經建立了帳號、儲存了 API token、選擇了 crate 的獨特名稱並指定了所需的詮釋資料，你現在已經準備好發佈了！發佈 crate 會上傳一個指定版本到 crates.io 供其他人使用。

發佈 crate 時請格外小心，因為發佈是會永遠存在的。該版本無法被覆寫，而且程式碼無法被刪除。crates.io 其中一個主要目標就是要作為儲存程式碼的永久伺服器，讓所有依賴 crates.io 的 crate 的專案可以持續正常運作。允許刪除版本會讓此目標幾乎無法達成。不過你能發佈的 crate 版本不會有數量限制。

再次執行 `cargo publish` 命令，這次就應該會成功了：

```
$ cargo publish
  Updating crates.io index
  Packaging guessing_game v0.1.0 (file:///projects/guessing_game)
  Verifying guessing_game v0.1.0 (file:///projects/guessing_game)
  Compiling guessing_game v0.1.0
(file:///projects/guessing_game/target/package/guessing_game-0.1.0)
  Finished dev [unoptimized + debuginfo] target(s) in 0.19s
  Uploading guessing_game v0.1.0 (file:///projects/guessing_game)
```

恭喜！你現在將你的程式碼分享給 Rust 社群了，任何人現在都可以輕鬆將你的 crate 加到他們的專案中作為依賴了。

對現有 Crate 發佈新版本

當你對你的 crate 做了一些改變並準備好發佈新版本時，你可以變更 `Cargo.toml` 中的 `version` 數值，並再發佈一次。請使用語意化版本規則依據你作出的改變來決定下一個妥當的版本數字。接著執行 `cargo publish` 來上傳新版本。

透過 `cargo yank` 棄用 Crates.io 的版本

雖然你無法刪除 crate 之前的版本，你還是可以防止任何未來的專案加入它們作為依賴。這在 crate 版本因某些原因而被破壞時會很有用。在這樣的情況下，Cargo 支援撤回 (yanking) crate 版本。

撤回一個版本能防止新專案用該版本作為依賴，同時允許現存依賴它的專案能夠繼續依賴該版本。實際上，撤回代表所有專案的 *Cargo.lock* 都不會被破壞，且任何未來產生的 *Cargo.lock* 檔案不會使用被撤回的版本。

要撤回一個 crate 的版本，在你先前發布的 crate 目錄底下執行 `cargo yank` 並指定你想撤回的版本。舉例來說，如果我們發布了一個 `guessing_game` crate 的版本 1.0.1，然讓我們想撤回的話，我們可以在 `guessing_game` 專案目錄底下執行：

```
$ cargo yank --vers 1.0.1
    Updating crates.io index
    Yank guessing_game@1.0.1
```

而對命令加上 `--undo` 的話，你還可以在復原撤回的動作，允許其他專案可以再次依賴該版本：

```
$ cargo yank --vers 1.0.1 --undo
    Updating crates.io index
    Yank guessing_game@1.0.1
```

撤回**並不會**刪除任何程式碼。舉例來說，它並不會刪除任何不小心上傳的祕密訊息。如果真的出現這種情形，你必須立即重設那些資訊。

Cargo 工作空間

在第十二章中，我們建立的套件包含一個執行檔 crate 與一個函式庫 crate。隨著專案開發，你可能會發現函式庫 crate 變得越來越大，而你可能會想要將套件拆成數個函式庫 crate。Cargo 提供了一個功能叫做**工作空間**（workspaces）能來幫助管理並開發數個相關的套件。

建立工作空間

工作空間是一系列的共享相同 *Cargo.lock* 與輸出目錄的套件。讓我們建立一個使用工作空間的專案，我們會使用簡單的程式碼，好讓我們能專注在工作空間的架構上。組織工作空間的架構有很多種方式，我們會介紹其中一種常見的方式。我們的工作空間將會包含一個執行檔與兩個函式庫。執行檔會提供主要功能，並依賴其他兩個函式庫。其中一個函式庫會提供函式 `add_one`，而另一個函式庫會提供函式 `add_two`。這三個 crate 會包含在相同的工作空間中，我們先從建立工作空間的目錄開始：

```
$ mkdir add
$ cd add
```

接著在 *add* 目錄中，我們建立會設置整個工作空間的 *Cargo.toml* 檔案。此檔案不會有 `[package]` 段落。反之，他會使用一個 `[workspace]` 段落作為起始，讓我們可以透過指定執行檔 crate 的套件路徑來將它加到工作空間的成員中。在此例中，我們的路徑是 *adder*：

檔案名稱：Cargo.toml

```
[workspace]

members = [
    "adder",
]
```

接下來我們會在 *add* 目錄下執行 `cargo new` 來建立 *adder* 執行檔 crate：

```
$ cargo new adder
Created binary (application) `adder` package
```

在這個階段，我們已經可以執行 `cargo build` 來建構工作空間。目錄 *add* 底下的檔案應該會看起來像這樣：

```
├── Cargo.lock
├── Cargo.toml
├── adder
│   ├── Cargo.toml
│   └── src
│       └── main.rs
└── target
```

工作空間在頂層有一個 *target* 目錄用來儲存編譯結果。 *adder* 套件不會有自己的 *target* 目錄。就算我們在 *adder* 目錄底下執行 `cargo build`，編譯結果仍然會在 *add/target* 底下而非 *add/adder/target*。Cargo 之所以這樣組織工作空間的 *target* 目錄是因為工作空間的 crate 是會彼此互相依賴的。如果每個 crate 都有自己的 *target* 目錄，每個 crate 就得重新編譯工作空間中的其他每個 crate 才能將編譯結果放入它們自己的 *target* 目錄。共享 *target* 目錄的話，crate 可以避免不必要的重新建構。

在工作空間中建立第二個套件

接下來讓我們在工作空間中建立另一個套件成員 *add_one*。請修改頂層 *Cargo.toml* 來指定 *add_one* 的路徑到 *members* 列表中：

檔案名稱：Cargo.toml

```
[workspace]

members = [
    "adder",
    "add_one",
]
```

然後產生新的函式庫 crate *add_one*：

```
$ cargo new add_one --lib
   Created library `add_one` package
```

add 目錄現在應該要擁有這些目錄與檔案：

```

├── Cargo.lock
├── Cargo.toml
├── add_one
│   ├── Cargo.toml
│   └── src
│       └── lib.rs
├── adder
│   ├── Cargo.toml
│   └── src
│       └── main.rs
└── target

```

在 `add_one/src/lib.rs` 檔案中，讓我們加上一個函式 `add_one`：

檔案名稱：`add_one/src/lib.rs`

```

pub fn add_one(x: i32) -> i32 {
    x + 1
}

```

現在我們可以讓我們 `adder` 套件的執行檔依賴擁有函式庫的 `add_one` 套件。首先，我們需要將 `add_one` 的路徑依賴加到 `adder/Cargo.toml`。

檔案名稱：`adder/Cargo.toml`

```

[dependencies]
add_one = { path = "../add_one" }

```

Cargo 不會假設工作空間下的 crate 會彼此依賴，我們要指定彼此之間依賴的關係。

接著讓我們在 `adder` 內使用 `add_one` crate 的 `add_one` 函式。開啟 `adder/src/main.rs` 檔案並在最上方加上 `use` 來將 `add_one` 函式庫引入作用域。然後變更 `main` 函式來呼叫 `add_one` 函式，如範例 14-7 所示。

檔案名稱：`adder/src/main.rs`

```

use add_one;

fn main() {
    let num = 10;
    println!(
        "你好，世界！{num} 加一會是 {}！", add_one::add_one(num);
    );
}

```

範例 14-7：在 `adder` crate 中使 `add_one` 函式庫 crate

讓我們在頂層的 `add` 目錄執行 `cargo build` 來建構工作空間吧！

```
$ cargo build
Compiling add_one v0.1.0 (file:///projects/add/add_one)
Compiling adder v0.1.0 (file:///projects/add/adder)
Finished dev [unoptimized + debuginfo] target(s) in 0.68s
```

要執行 `add` 目錄的執行檔 `crate`，我們可以透過 `-p` 加上套件名稱使用 `cargo run` 來執行我們想要在工作空間中指定的套件：

```
$ cargo run -p adder
Finished dev [unoptimized + debuginfo] target(s) in 0.0s
Running `target/debug/adder`
你好，世界！10 加一會是 11！
```

這就會執行 `adder/src/main.rs` 的程式碼，其依賴於 `add_one` `crate`。

在工作空間中依賴外部套件

注意到工作空間只有在頂層有一個 `Cargo.lock` 檔案，而不是在每個 `crate` 目錄都有一個 `Cargo.lock`。這確保所有的 `crate` 都對所有的依賴使用相同的版本。如果我們加了 `rand` 套件到 `adder/Cargo.toml` 與 `add_one/Cargo.toml` 檔案中，Cargo 會將兩者的版本解析為同一個 `rand` 版本並記錄到同個 `Cargo.lock` 中。確保工作空間所有 `crate` 都會使用相同依賴代表工作空間中的 `crate` 永遠都彼此相容。讓我們將 `rand` `crate` 加到 `add_one/Cargo.toml` 檔案的 `[dependencies]` 段落中，使 `add_one` `crate` 可以使用 `rand` `crate`：

檔案名稱：`add_one/Cargo.toml`

```
rand = "0.8.5"
```

我們現在就可以將 `use rand;` 加到 `add_one/src/lib.rs` 檔案中，接著在 `add` 目錄下執行 `cargo build` 來建構整個工作空間就會引入並編譯 `rand` `crate`。我們會得到一個警告，因為我們還沒有開始使用引入作用域的 `rand`：


```
$ cargo build
  Updating crates.io index
  Downloaded rand v0.8.5
  --省略--
  Compiling rand v0.8.5
  Compiling add_one v0.1.0 (file:///projects/add/add_one)
warning: unused import: `rand`
--> add_one/src/lib.rs:1:5
|
1 | use rand;
|     ^^^^
|
= note: `[warn(unused_imports)]` on by default
warning: `add_one` (lib) generated 1 warning
  Compiling adder v0.1.0 (file:///projects/add/adder)
  Finished dev [unoptimized + debuginfo] target(s) in 10.18s
```

頂層的 *Cargo.lock* 現在就包含 `add_one` 有 `rand` 作為依賴的資訊。不過就算我們能在工作空間的某處使用 `rand`，並不代表我們可以在工作空間的其他 crate 中使用它，除非它們的 *Cargo.toml* 也加上了 `rand`。舉例來說，如果我們將 `use rand;` 加到 *adder/src/main.rs* 檔案中想讓 `adder` 套件也使用的話，我們就會得到錯誤：

```
$ cargo build
--省略--
  Compiling adder v0.1.0 (file:///projects/add/adder)
error[E0432]: unresolved import `rand`
--> adder/src/main.rs:2:5
|
2 | use rand;
|     ^^^^ no external crate `rand`
```

要修正此問題，只要修改 `adder` 套件的 *Cargo.toml* 檔案，指示它也加入 `rand` 作為依賴就好了。這樣建構 `adder` 套件就會將在 *Cargo.lock* 中將 `rand` 加入 `adder` 的依賴，但是沒有額外的 `rand` 會被下載。Cargo 會確保工作空間中每個套件的每個 crate 都會使用相同的 `rand` 套件版本。這可以節省空間，並能確保工作空間中的 crate 彼此相容。

在工作空間中新增測試

讓我們再進一步加入一個測試函式 `add_one::add_one` 到 `add_one` crate 之中：

檔案名稱：`add_one/src/lib.rs`

```
pub fn add_one(x: i32) -> i32 {
    x + 1
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        assert_eq!(3, add_one(2));
    }
}
```

現在在頂層的 `add` 目錄執行 `cargo test`。像這樣在工作空間的架構下執行 `cargo test` 會執行工作空間下所有 crate 的測試：

```
$ cargo test
   Compiling add_one v0.1.0 (file:///projects/add/add_one)
   Compiling adder v0.1.0 (file:///projects/add/adder)
   Finished test [unoptimized + debuginfo] target(s) in 0.27s
   Running unittests src/lib.rs (target/debug/deps/add_one-f0253159197f7841)

running 1 test
test tests::it_works ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

   Running unittests src/main.rs (target/debug/deps/adder-49979ff40686fa8e)

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

   Doc-tests add_one

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

輸出的第一個段落顯示了 `add_one` crate 中的 `it_works` 測試通過。下一個段落顯示 `adder` crate 沒有任何測試，然後最後一個段落顯示 `add_one` 中沒有任何技術文件測試。

我們也可以在頂層目錄使用 `-p` 並指定我們想測試的 crate 名稱來測試工作空間中特定的 crate：

```
$ cargo test -p add_one
    Finished test [unoptimized + debuginfo] target(s) in 0.00s
    Running unittests src/lib.rs (target/debug/deps/add_one-b3235fea9a156f74)

running 1 test
test tests::it_works ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

Doc-tests add_one

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s
```

此輸出顯示 `cargo test` 只執行了 `add_one` crate 的測試並沒有執行 `adder` crate 的測試。

如果你想要發佈工作空間的 crate 到 crates.io，工作空間中的每個 crate 必須分別獨自發佈。和 `cargo test` 一樣，我們可以用 `-p` 的選項來指定想要的 crate 名稱，來發布工作空間內的特定 crate。

之後想嘗試練習的話，你可以在工作空間中在加上 `add_two` crate，方式和 `add_one` crate 類似！

隨著你的專案成長，你可以考慮使用工作空間：拆成各個小部分比一整塊大程式還更容易閱讀。再者，如果需要經常同時修改的話，將 crate 放在同個工作空間中更易於彼此的協作。

透過 `cargo install` 安裝執行檔

`cargo install` 命令讓你能本地安裝並使用執行檔 crates。這並不是打算要取代系統套件，這是為了方便讓 Rust 開發者可以安裝 crates.io 上分享的工具。注意你只能安裝有執行檔目標的套件。**執行檔目標** (binary target) 是在 crate 有 `src/main.rs` 檔案或其他指定的執行檔時，所建立的可執行程式。而相反地，函式庫目標就無法單獨執行，因為它提供給其他程式使用的函式庫。通常 crate 都會提供 `README` 檔案說明此 crate 是函式庫還是執行檔目標，或者兩者都是。

所有透過 `cargo install` 安裝的執行檔都儲存在安裝根目錄的 `bin` 資料夾中。如果你是用 `rustup.rs` 安裝 Rust 且沒有任何自訂設置的話，此目錄會是 `$HOME/.cargo/bin`。請確定該目錄在你的 `$PATH` 中，這樣才能夠執行 `cargo install` 安裝的程式。

舉例來說，第十二章我們提到有個 Rust 版本的 `grep` 工具叫做 `ripgrep` 能用來搜尋檔案。要安裝 `ripgrep` 的話，我們可以執行以下命令：

```
$ cargo install ripgrep
  Updating crates.io index
  Downloaded ripgrep v13.0.0
  Downloaded 1 crate (243.3 KB) in 0.88s
  Installing ripgrep v13.0.0
--省略--
  Compiling ripgrep v13.0.0
    Finished release [optimized + debuginfo] target(s) in 3m 10s
  Installing ~/.cargo/bin/rg
  Installed package `ripgrep v13.0.0` (executable `rg`)
```

輸出的最後兩行顯示了執行檔的安裝位置與名稱，在 `ripgrep` 此例中就是 `rg`。如稍早提到的，只要你的 `$PATH` 有包含安裝目錄，你就可以執行 `rg --help` 並開始使用更快更鏽的搜尋檔案工具！

透過自訂命令來擴展 Cargo 的功能

Cargo 的設計能讓你在不用修改 Cargo 的情況下擴展新的子命令。如果你 `$PATH` 中有任何叫做 `cargo-something` 的執行檔，你就可以用像是執行 Cargo 子命令的方式 `cargo something` 來執行它。像這樣的自訂命令在你執行 `cargo --list` 時也會顯示出來。能夠透過 `cargo install` 來安裝擴展外掛並有如內建 Cargo 工具般來執行使用是 Cargo 設計上的一大方便優勢！

總結

透過 Cargo 與 crates.io 分享程式碼是讓 Rust 生態系統能適用於許多不同任務的重要部分之一。Rust 的標準函式庫既小又穩定，但是 crate 可以很容易地分享、使用，並在語言本身不同的時間線來進行改善。千萬別吝嗇於分享你認為實用的程式碼到 crates.io，其他人可能也會覺得它很有幫助！

智慧指標

指標 (pointer) 是一個將變數儲存記憶體位址的通用概念。此位址參考或者說是「指向」一些其他資料。Rust 中最常見的指標種類就是第四章介紹的參考 (reference)。參考以 `&` 符號作為指示並借用它們指向的數值。它們除了參考資料以外，沒有其他的特殊能力，也沒有任何額外開銷。

另一方面，**智慧指標** (Smart pointers) 是個不只會有像是指標的行為，還會包含擁有的詮釋資料與能力。智慧指標的概念並不是 Rust 獨有的，智慧指標起源於 C++ 且也都存在於其他語言。Rust 在標準函式庫中有提供許多不同的智慧指標，不只能參考還具備更多的功能。為了探索各個概念，我們會來研究一些各種不同的智慧指標範例，包含**參考計數** (reference counting) 智慧指標型別。此指標允許一個資料可以有多個擁有者，並追蹤擁有者的數量，當沒有任何擁有者時，就清除資料。

在 Rust 中，我們有所有權與借用的概念，所以參考與智慧指標之間還有一項差別：參考是只有借用資料的指標，但智慧指標在很多時候都**擁有**它們指向的資料。

雖然在前面的章節我們沒有這樣稱呼，但我們已經在本書中遇過一些智慧指標了，像是第八章的 `String` 和 `Vec<T>`，雖然當時我們沒有稱呼它們為智慧指標。這些型別都算是智慧指標，因為它們都擁有一些記憶體並允許你操控它們。它們也有詮釋資料以及額外的能力或保障。像是 `String` 就會將容量儲存在詮釋資料中，並確保其資料永遠是有效的 UTF-8。

智慧指標通常都使用結構體實作。和一般結構體不同，智慧指標會實作 `Deref` 與 `Drop` 特徵。`Deref` 特徵允許智慧指標結構體的實例表現的像是參考一樣，讓你可以寫出能用在參考與智慧指標的程式碼。`Drop` 特徵允許你自訂當智慧指標實例離開作用域時要執行的程式碼。在本章節我們會討論這兩個特徵並解釋為何它們對智慧指標很重要。

有鑑於智慧指標在 Rust 是個常用的通用設計模式，本章不會涵蓋每一個現有的智慧指標。許多函式庫也都會提供它們自己的智慧指標，你甚至能寫個你自己的。我們會提及標準函式庫中最常用到的智慧指標：

- `Box<T>` 將數值配置到堆積上
- `Rc<T>`，參考計數型別來允許資料能有數個擁有者
- 透過 `RefCell<T>` 來存取 `Ref<T>` 與 `RefMut<T>`，這是在執行時而非編譯時強制執行借用規則的型別

除此之外，我們還會涵蓋到**內部可變性** (interior mutability) 模式，這讓不可變參考的型別能提供改變內部數值的 API。我們還會討論**參考循環** (reference cycles) 為何會導致記憶體洩漏以及如何預防它們。

讓我們開始吧！

使用 `Box<T>` 指向堆積上的資料

最直白的智慧指標是 `box` 其型別為 `Box<T>`。Box 允許你儲存資料到堆積上，而不是堆疊。留在堆疊上的會是指向堆積資料的指標。你可以回顧第四章瞭解堆疊與堆積的差別。

Box 沒有額外的效能開銷，就只是將它們的資料儲存在堆積上而非堆疊而已。不過相對地它們也沒有多少額外功能。你大概會在這些場合用到它們：

- 當你有個型別無法在編譯時期確定大小，而你又想在需要知道確切大小的情況下使用該型別的數值。
- 當你有個龐大的資料，而你想要轉移所有權並確保資料不會被拷貝。
- 當你想要擁有某個值，但你只在意該型別有實作特定的特徵，而不是何種特定型別。

我們會在「[透過 Box 建立遞迴型別](#)」段落解說第一種情形。而在第二種情形，轉移龐大的資料的所有權可能會很花費時間，因為在堆疊上的話會拷貝所有資料。要改善此情形，我們可以用 `box` 將龐大的資料儲存在堆積上。這樣就只有少量的指標資料在堆疊上被拷貝，而其參考的資料仍然保留在堆積上的同個位置。第三種情況被稱之為**特徵物件 (trait object)**，第十七章會花整個「[允許不同型別數值的特徵物件](#)」段落來討論此議題。所以你在此學到的到第十七章會再次用上！

使用 `Box<T>` 儲存資料到堆積上

在我們討論 `Box<T>` 在堆積儲存空間上的使用場合前，我們會先介紹語法以及如何對 `Box<T>` 內儲存的數值進行互動。

範例 15-1 顯示如何使用 `box` 在堆積上儲存一個 `i32` 數值：

檔案名稱：src/main.rs

```
fn main() {  
    let b = Box::new(5);  
    println!("b = {}", b);  
}
```

範例 15-1：使用 `box` 在堆積上儲存一個 `i32` 數值

我們定義了變數 `b` 其數值為 `Box` 配置在堆積上指向的數值 `5`。程式在此例會印出 `b = 5`，在此例中我們可以用在堆疊上相同的方式取得 `box` 的資料。就像任何有所有權的數值一樣，當 `box` 離開作用域時會釋放記憶體，在此例就是當 `b` 抵達 `main` 結尾的時候。釋放記憶體作用於 `box`（儲存在堆疊上）以及其所指向的資料（儲存在堆積上）。

將單一數值放在堆積上的確沒什麼用處，所以你不會對這種類型經常使用 `box`。在大多數情況下將像 `i32` 這種單一數值預設儲存在堆疊的確比較適合。

透過 Box 建立遞迴型別

遞迴型別 (recursive type) 的數值可以用相同型別的其他數值作為自己的一部分。遞迴型別對 Rust 來說會造成問題，因為 Rust 得在編譯期間時知道型別佔用的空間。由於這種巢狀數值理論上可以無限循環下去，Rust 無法知道一個遞迴型別的數值需要多大的空間。然而 box 則有已知大小，所以將 box 填入遞迴型別定義中，你就可以有遞迴型別了。

讓我們來探索 *cons list* 來作為遞迴型別的範例。這是個在函式程式語言中常見的資料型別，很適合作為遞迴型別的範例。我們要定義的 *cons list* 型別除了遞迴的部分以外都很直白，因此這個例子的概念在往後你遇到更複雜的遞迴型別時會很實用。

更多關於 Cons List 的資訊

cons list 是個起源於 Lisp 程式設計語言與其方言的資料結構，用巢狀配對組成，相當於 Lisp 版的鏈結串列 (linked list)。這名字來自於 Lisp 中的 `cons` 函式 (「construct function」的縮寫)，它會從兩個引數建構一個新的配對，而這通常包含一個數值與另一個配對。對其呼叫 `cons` 就我們能建構出擁有遞迴配對的 *cons list*。

舉例來說，以下是個 *cons list* 的範例，包含了用括號包起來的 1、2、3 列表配對：

```
(1, (2, (3, Nil)))
```

每個 *cons list* 的項目都包含兩個元素：目前項目的數值與下一個項目。列表中的最後一個項目只會包含一個數值叫做 `Nil`，並不會再連接下一個項目。*cons list* 透過遞迴呼叫 `cons` 函式來產生。表示遞迴終止條件的名稱為 `Nil`。注意這和第六章提到的「null」或「nil」的概念不全然相同，這些代表的是無效或空缺的數值。

在 Rust 中 *cons lists* 不是常見的資料結構。大多數當你在 Rust 需要項目列表時，`Vec<T>` 會比較好的選擇。而其他時候夠複雜的遞迴資料型別**確實**在各種特殊情形會很實用，不過先從 *cons list* 開始的話，我們可以專注探討 box 如何讓我們定義遞迴資料型別。

範例 15-2 包含了 *cons list* 的列舉定義。注意到此程式碼還不能編譯過，因為 `List` 型別並沒有已知的大小，我們接下來會繼續說明。

檔案名稱：src/main.rs

```
enum List {  
    Cons(i32, List),  
    Nil,  
}
```



範例 15-2：第一次嘗試定義一個列舉來代表有 `i32` 數值的 *cons list* 資料結構

注意：我們定義的 *cons list* 只有 `i32` 數值是為了範例考量。我們當然可以使用第十章討論

過的泛型來定義它，讓 `cons list` 定義的型別可以儲存任何型別數值。

使用 `List` 型別來儲存 `1, 2, 3` 列表的話會如範例 15-3 的程式碼所示：

檔案名稱：`src/main.rs`

```
use crate::List::{Cons, Nil};

fn main() {
    let list = Cons(1, Cons(2, Cons(3, Nil)));
}
```



範例 15-3：使用 `List` 列舉儲存列表 `1, 2, 3`

第一個 `Cons` 值會得到 `1` 與另一個 `List` 數值。此 `List` 數值是另一個 `Cons` 數值且持有 `2` 與另一個 `List` 數值。此 `List` 數值是另一個 `Cons` 數值且擁有 `3` 與一個 `List` 數值，其就是最後的 `Nil`，這是傳遞列表結尾訊號的非遞迴變體。

如果我們嘗試編譯範例 15-3 的程式碼，我們會得到範例 15-4 的錯誤：

```
$ cargo run
   Compiling cons-list v0.1.0 (file:///projects/cons-list)
error[E0072]: recursive type `List` has infinite size
--> src/main.rs:1:1
 |
1 | enum List {
 | ^^^^^^^^^ recursive type has infinite size
2 |     Cons(i32, List),
 |               ---- recursive without indirection
 |
help: insert some indirection (e.g., a `Box`, `Rc`, or `&`) to make `List`
representable
2 |     Cons(i32, Box<List>),
 |               ++++++
 |
For more information about this error, try `rustc --explain E0072`.
error: could not compile `cons-list` due to previous error
```

範例 15-4：嘗試定義遞迴列舉所得到的錯誤

錯誤顯示此型別的「大小為無限」，原因是因為我們定義的 `List` 有個變體是遞迴：它直接存有另一個相同類型的數值。所以 Rust 無法判別出它需要多少空間才能儲存一個 `List` 的數值。讓我們進一步研究為何會得到這樣的錯誤，首先來看 Rust 如何決定要配置多少空間來儲存非遞迴型別。

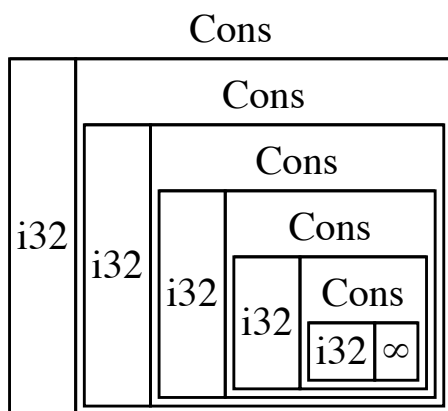
計算非遞迴型別的大小

回想一下第六章中，當我們在討論列舉定義時，我們在範例 6-2 定義的 `Message` 列舉：

```
enum Message {  
    Quit,  
    Move { x: i32, y: i32 },  
    Write(String),  
    ChangeColor(i32, i32, i32),  
}
```

要決定一個 `Message` 數值需要配置多少空間，Rust 會遍歷每個變體來看哪個變體需要最大的空間。Rust 會看到 `Message::Quit` 不佔任何空間、`Message::Move` 需要能夠儲存兩個 `i32` 的空間，以此類推。因為只有一個變體會被使用，一個 `Message` 數值所需的最大空間就是其最大變體的大小。

將此對應到當 Rust 嘗試檢查像是範例 15-2 的 `List` 列舉來決定遞迴型別需要多少空間時，究竟會發生什麼事。編譯器先從查看 `Cons` 的變體開始，其存有一個 `i32` 型別與一個 `List` 型別。因此 `Cons` 需要的空間大小為 `i32` 的大小加上 `List` 的大小。為了要瞭解 `List` 型別需要的多少記憶體，編譯器再進一步看它的變體，也是從 `Cons` 變體開始。`Cons` 變體存有一個型別 `i32` 與一個型別 `List`，而這樣的過程就無限處理下去，如圖示 15-1 所示。



圖示 15-1：無限個 `List` 包含著無限個 `Cons` 變體

使用 `Box<T>` 取得已知大小的遞迴型別

由於 Rust 無法判別出遞迴定義型別要配置多少空間，所以編譯器會針對此錯誤提供些實用的建議：

```
help: insert some indirection (e.g., a `Box`, `Rc`, or `&`) to make `List`
representable
```

```
|
2 |     Cons(i32, Box<List>),
|           ++++++  +
```

在此建議中，「indirection」代表與其直接儲存數值，我們可以變更資料結構，間接儲存指向數值的指標。

因為 `Box<T>` 是個指標，Rust 永遠知道 `Box<T>` 需要多少空間：指標的大小不會隨著指向的資料數量而改變。這代表我們可以將 `Box<T>` 存入 `Cons` 變體而非直接儲存另一個 `List` 數值。

`Box<T>` 會指向另一個存在於堆積上的 `List` 數值而不是存在 `Cons` 變體中。概念上我們仍然有建立一個持有其他列表的列表，但此實作更像是將項目接著另一個項目排列，而非包含另一個在內。

我們可以改變範例 15-2 的 `List` 列舉定義以及範例 15-3 `List` 的使用方式，將其寫入範例 15-5，這次就能夠編譯過了：

檔案名稱：src/main.rs

```
enum List {
    Cons(i32, Box<List>),
    Nil,
}

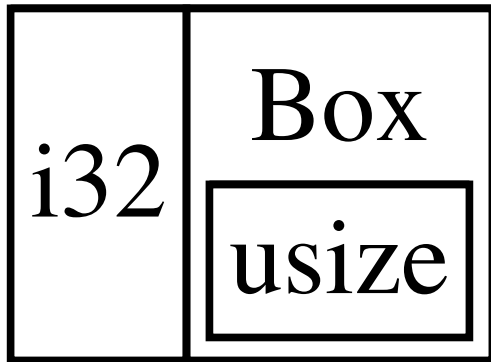
use crate::List::{Cons, Nil};

fn main() {
    let list = Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))));
}
```

範例 15-5：使用 `Box<T>` 定義的 `List` 就有已知大小

`Cons` 變體需要的大小為 `i32` 加上儲存 `box` 指標的空間。`Nil` 變體沒有儲存任何數值，所以它需要的空間比 `Cons` 變體少。現在我們知道任何 `List` 數值會佔的空間都是一個 `i32` 加上 `box` 指標的大小。透過使用 `box`，我們打破了無限遞迴，所以編譯器可以知道儲存一個 `List` 數值所需要的大小。圖示 15-2 顯示了 `Cons` 變體看起來的樣子。

Cons



圖示 15-2：不再是無限大小的 `List`，因為其 `Cons` 存的是 `Box`

`Boxes` 只提供了間接儲存與堆積配置，它們沒有其他任何特殊功能，比如我們等下就會看到的其他智慧指標型別。它們也沒有任何因這些特殊功能產生的額外效能開銷，所以它們很適合用於像是 `cons list` 這種我們只需要間接儲存的場合。我們在第十七章還會再介紹到更多 `box` 的使用情境。

`Box<T>` 型別是智慧指標是因為它有實作 `Deref` 特徵，讓 `Box<T>` 的數值可以被視為參考所使用。當 `Box<T>` 數值離開作用域時，該 `box` 指向的堆積資料也會被清除，因為其有 `Drop` 特徵實作。這兩種特徵對於本章將會討論的其他智慧指標型別所提供的功能，將會更加重要。讓我們來探討這兩種特徵的細節吧。

透過 `Deref` 特徵將智慧指標視為一般參考

實作 `Deref` 特徵讓你可以自訂解參考運算子 (**dereference operator**) `*` 的行為 (這不是相乘或全域運算子)。透過這種方式實作 `Deref` 的智慧指標可以被視為正常參考來對待，這樣操作參考的程式碼也能用在智慧指標中。

讓我們先看解參考運算子如何在正常參考中使用。然後我們會嘗試定義一個行為類似 `Box<T>` 的自定型別，並看看為何解參考運算子無法像參考那樣用在我們新定義的型別。我們將會探討如何實作 `Deref` 特徵使智慧指標能像類似參考的方式運作。接著我們會看看 Rust 的**強制解參考** (**deref coercion**) 功能並瞭解它如何處理參考與智慧指標。

注意：我們即將定義的 `MyBox<T>` 型別與真正的 `Box<T>` 有一項很大的差別，就是我們的版本不會將其資料儲存在堆積上。我們在此例會專注在 `Deref` 上，所以資料實際上儲存在何處，並沒有比指標相關行為來得重要。

追蹤指標的數值

一般的參考是一種指標，其中一種理解指標的方式是看成一個會指向存於某處數值的箭頭。在範例 15-6 中我們建立了數值 `i32` 的參考，接著使用解參考運算子來追蹤參考的數值：

檔案名稱：src/main.rs

```
fn main() {
    let x = 5;
    let y = &x;

    assert_eq!(5, x);
    assert_eq!(5, *y);
}
```

範例 15-6：使用解參考運算子來追蹤數值 `i32` 的參考

變數 `x` 存有 `i32` 數值 `5`。我們將 `y` 設置為 `x` 的參考。我們可以判定 `x` 等於 `5`。不過要是我們想要判定 `y` 數值的話，我們需要使用 `*y` 來追蹤參考指向的數值 (也就是**解參考**)，這樣編譯器才能比較實際數值。一旦我們解參考 `y`，我們就能取得 `y` 指向的整數數值並拿來與 `5` 做比較。

如果我們嘗試寫說 `assert_eq!(5, y);` 的話，我們會得到此編譯錯誤：

```

$ cargo run
   Compiling deref-example v0.1.0 (file:///projects/deref-example)
error[E0277]: can't compare `{integer}` with `{integer}`
  --> src/main.rs:6:5
   |
6 |     assert_eq!(5, y);
   |     ^^^^^^^^^^^^^^^^^ no implementation for `{integer} == {integer}`
   |
   = help: the trait `PartialEq<{integer}>` is not implemented for `{integer}`
   = note: this error originates in the macro `assert_eq` (in Nightly builds,
run with -Z macro-backtrace for more info)
   = help: the following other types implement trait `PartialEq<Rhs>`:
           f32
           f64
           i128
           i16
           i32
           i64
           i8
           isize
           and 6 others

For more information about this error, try `rustc --explain E0277`.
error: could not compile `deref-example` due to previous error

```

比較一個數字與一個數字的參考是不允許的，因為它們是不同的型別。我們必須使用解參考運算子來追蹤其指向的數值。

像參考般使用 `Box<T>`

我們將範例 15-6 的參考改用 `Box<T>` 重寫。範例 15-7 對 `Box<T>` 使用解參考運算子的方式如就和範例 15-6 對參考使用解參考運算子的方式一樣：

檔案名稱：src/main.rs

```

fn main() {
    let x = 5;
    let y = Box::new(x);

    assert_eq!(5, x);
    assert_eq!(5, *y);
}

```

範例 15-7：對 `Box<i32>` 使用解參考運算子

範例 15-7 與範例 15-6 主要的差別在於這裡我們設置 `y` 為一個指向 `x` 的拷貝數值的 `Box<T>` 實例，而不是指向 `x` 數值的參考。在最後的判定中，我們可以對 `Box<T>` 的指標使用解參考運算子，跟我們對當 `y` 還是參考時所做的動作一樣。接下來，我們要來探討 `Box<T>` 有何特別之處，讓我們可以對自己定義的型別也可以使用解參考運算子。

定義我們自己的智慧指標

讓我們定義一個與標準函式庫所提供的 `Box<T>` 型別類似的智慧指標，並看看智慧指標預設行為與參考有何不同。然後我們就會來看能夠使用解參考運算子的方式。

`Box<T>` 本質上就是定義成只有一個元素的元組結構體，所以範例 15-8 用相同的方式來定義 `MyBox<T>`。我們也定義了 `new` 函式來對應於 `Box<T>` 的 `new` 函式。

檔案名稱：src/main.rs

```
struct MyBox<T>(T);

impl<T> MyBox<T> {
    fn new(x: T) -> MyBox<T> {
        MyBox(x)
    }
}
```

範例 15-8：定義 `MyBox<T>` 型別

我們定義了一個結構體叫做 `MyBox` 並宣告一個泛型參數 `T`，因為我們希望我們的型別能存有任何型別的數值。`MyBox` 是個只有一個元素型別為 `T` 的元組結構體。`MyBox::new` 函式接受一個參數型別為 `T` 並回傳存有該數值的 `MyBox` 實例。

讓我們將範例 15-7 的 `main` 函式加到範例 15-8 並改成使用我們定義的 `MyBox<T>` 型別而不是原本的 `Box<T>`。範例 15-9 的程式碼無法編譯，因為 Rust 不知道如何解參考 `MyBox`。

檔案名稱：src/main.rs

```
fn main() {
    let x = 5;
    let y = MyBox::new(x);

    assert_eq!(5, x);
    assert_eq!(5, *y);
}
```



範例 15-9：嘗試像使用 `Box<T>` 和參考一樣的方式來使用 `MyBox<T>`

以下是編譯結果出現的錯誤：

```
$ cargo run
Compiling deref-example v0.1.0 (file:///projects/deref-example)
error[E0614]: type `MyBox<{integer}>` cannot be dereferenced
--> src/main.rs:14:19
   |
14 |         assert_eq!(5, *y);
   |                        ^^

For more information about this error, try `rustc --explain E0614`.
error: could not compile `deref-example` due to previous error
```

我們的 `MyBox<T>` 型別無法解參考因為我們還沒有對我們的型別實作該能力。要透過 `*` 運算子來解參考的話，我們要實作 `Deref` 特徵。

透過實作 `Deref` 特徵來將一個型別能像參考般對待

如同第十章的「為型別實作特徵」段落所講過的，要實作一個特徵的話，我們需要提供該特徵要求的方法實作。標準函式庫所提供的 `Deref` 特徵要求我們實作一個方法叫做 `deref`，這會借用 `self` 並回傳內部資料的參考。範例 15-10 包含了對 `MyBox` 定義加上的 `Deref` 實作：

檔案名稱：src/main.rs

```
use std::ops::Deref;

impl<T> Deref for MyBox<T> {
    type Target = T;

    fn deref(&self) -> &Self::Target {
        &self.0
    }
}
```

範例 15-10：對 `MyBox<T>` 實作 `Deref`

`type Target = T;` 語法定義了一個供 `Deref` 特徵使用的關聯型別。關聯型別與宣告泛型參數會有一點差別，但是你現在先不用擔心它們，我們會在第十九章深入探討。

我們對 `deref` 的方法本體加上 `&self.0`，`deref` 就可以回傳一個參考讓我們可以使用 `*` 運算子取得數值。回想一下第五章的「使用無名稱欄位的元組結構體來建立不同型別」段落，`.0` 可以取的元組結構體的第一個數值。範例 15-9 的 `main` 函式現在對 `MyBox<T>` 數值的 `*` 呼叫就可以編譯了，而且判定也會通過！

沒有 `Deref` 特徵的話，編譯器只能解參考 `&` 的參考。`deref` 方法讓編譯器能夠從任何有實作 `Deref` 的型別呼叫 `deref` 方法取得 `&` 參考，而它就可以進一步解參考獲取數值。

當我們在範例 15-9 中輸入 `*y` 時，Rust 背後實際上是執行此程式碼：


```
*(y.deref())
```

Rust 將 `*` 運算子替換為方法 `deref` 的呼叫再進行普通的解參考，所以我們不必煩惱何時該或不該呼叫 `deref` 方法。此 Rust 特性讓我們可以對無論是參考或是有實作 `Deref` 的型別都能寫出一致的程式碼。

`deref` 方法會回傳一個數值參考，以及括號外要再加上普通解參考的原因，都是因為所有權系統。如果 `deref` 方法直接回傳數值而非參考數值的話，該數值就會移出 `self`。我們不希望在此例或是大多數使用解參考運算子的場合下，取走 `MyBox<T>` 內部數值的所有權。

注意到每次我們在程式碼中使用 `*` 時，`*` 運算子被替換成 `deref` 方法呼叫，然後再呼叫 `*` 剛好一次。因為 `*` 運算子不會被無限遞迴替換，我們能剛好取得型別 `i32` 並符合範例 15-9 `assert_eq!` 中與 `5` 的判定。

函式與方法的隱式強制解參考

強制解參考 (Deref coercion) 會將有實作 `Deref` 特徵的型別參考轉換成其他型別的參考。舉例來說，強制解參考可以轉換 `&String` 成 `&str`，因為 `String` 有實作 `Deref` 特徵並能用它來回傳 `&str`。強制解參考是一個 Rust 針對函式或方法的引數的便利設計，且只會用在有實作 `Deref` 特徵的型別。當我們將某個特定型別數值的參考作為引數傳入一個函式或方法，但該函式或方法所定義的參數卻不相符時，強制解參考就會自動發生，並進行一系列的 `deref` 方法呼叫，將我們提供的型別轉換成參數所需的型別。

Rust 會加入強制解參考的原因是因為程式設計師在寫函式與方法呼叫時，就不必加上許多顯式參考 `&` 與解參考 `*`。強制解參考還讓我們可以寫出能同時用於參考或智慧指標的程式碼。

為了展示強制解參考，讓我們使用範例 15-8 定義的 `MyBox<T>` 型別以及範例 15-10 所加上的 `Deref` 實作。範例 15-11 中定義的函式使用字串切片作為參數：

檔案名稱：src/main.rs

```
fn hello(name: &str) {  
    println!("Hello, {name}!");  
}
```

範例 15-11：hello 函式且有參數 `name` 其型別為 `&str`

我們可以使用字串切片作為引數來呼叫函式 `hello`，比方說 `hello("Rust");`。強制解參考讓我們可以透過 `MyBox<String>` 型別數值的參考來呼叫 `hello`，如範例 15-12 所示：

檔案名稱：src/main.rs

```
fn main() {
    let m = MyBox::new(String::from("Rust"));
    hello(&m);
}
```

範例 15-12：利用強制解參考透過 `MyBox<String>` 數值的參考來呼叫 `hello`

我們在此使用 `&m` 作為引數來呼叫函式 `hello`，這是 `MyBox<String>` 數值的參考。因為我們在範例 15-10 有對 `MyBox<T>` 實作 `Deref` 特徵，Rust 可以呼叫 `deref` 將 `&MyBox<String>` 變成 `&String`。標準函式庫對 `String` 也有實作 `Deref` 並會回傳字串切片，這可以在 `Deref` 的 API 技術文件中看到。所以 Rust 會再呼叫 `deref` 一次來將 `&String` 變成 `&str`，這樣就符合函式 `hello` 的定義了。

如果 Rust 沒有實作強制解參考的話，我們就得用範例 15-13 的方式才能辦到範例 15-12 使用型別 `&MyBox<String>` 的數值來呼叫 `hello` 的動作。

檔案名稱：src/main.rs

```
fn main() {
    let m = MyBox::new(String::from("Rust"));
    hello(&(*m)[..]);
}
```

範例 15-13：如果 Rust 沒有強制解參考，我們就得這樣寫程式碼

`(*m)` 會將 `MyBox<String>` 解參考成 `String`，然後 `&` 和 `[..]` 會從 `String` 中取得等於整個字串的字串切片，這就符合 `hello` 的簽名。沒有強制解參考的程式碼就難以閱讀、寫入或是理解，因為有太多的符號參雜其中。強制解參考能讓 Rust 自動幫我們做這些轉換。

當某型別有定義 `Deref` 特徵時，Rust 會分析該型別並重複使用 `Deref::deref` 直到能取得與參數型別相符的參考。`Deref::deref` 需要呼叫的次數會在編譯時期插入，所以使用強制解參考沒有任何的執行時開銷！

強制解參考如何處理可變性

類似於你使用 `Deref` 特徵來覆蓋不可變參考的 `*` 運算子的方式，你也可以使用 `DerefMut` 特徵來覆蓋可變參考的 `*` 運算子。

當 Rust 發現型別與特徵實作符合以下三種情況時，它就會進行強制解參考：

- 從 `&T` 到 `&U` 且 `T: Deref<Target=U>`
- 從 `&mut T` 到 `&mut U` 且 `T: DerefMut<Target=U>`
- 從 `&mut T` 到 `&U` 且 `T: Deref<Target=U>`

前兩個除了第二個有實作可變性之外是相同的。第一個情況表示如果你有個 `&T` 且 `T` 有實作 `Deref` 到某個型別 `U`，你就可以直接得到 `&U`。第二種情況指的則是對可變參考的強制解參考。

第三種情況比較棘手：Rust 也能強制將可變參考轉為一個不可變參考。但反過來是**不可行**的：不可變參考永遠不可能強制解參考成可變參考。由於借用規則，如果你有個可變參考，該可變參考必須是該資料的唯一參考（不然程式無法編譯）。轉換可變參考成不可變參考不會破壞借用規則。轉換不可變參考成可變參考的話，就需要此不可變參考是該資料的唯一參考，但借用規則無法做擔保。因此 Rust 無法將不可變參考轉換成可變參考。

透過 `Drop` 特徵執行清除程式碼

第二個對智慧指標模式很重要的特徵是 `Drop`，這讓你能自訂數值離開作用域時的行為。你可以對任何型別實作 `Drop` 特徵，然後你指定的程式碼就能用來釋放像是檔案或網路連線等資源。我們在智慧指標的章節介紹 `Drop` 的原因是因為 `Drop` 特徵的功能幾乎永遠會在實作智慧指標時用到。舉例來說，當 `Box<T>` 離開作用域時，它會釋放該 `box` 在堆積上指向的記憶體空間。

在某些語言中，當程式設計師使用完某些型別的實例後，每次都得呼叫釋放記憶體與資源的程式碼。例子包括檔案控制代碼（file handle）、插座（socket）或鎖。如果他們忘記的話，系統可能就會過載並崩潰。在 Rust 中你可以對數值離開作用域時指定一些程式碼，然後編譯器就會自動插入此程式碼。所以你就不用每次在特定型別實例使用完時，在程式的每個地方都寫上清理程式碼。而且你還不會泄漏資源！

透過實作 `Drop` 特徵我們可以指定當數值離開作用域時要執行的程式碼。`Drop` 特徵會要求我們實作一個方法叫做 `drop`，這會取得 `self` 的可變參考。為了觀察 Rust 何時會呼叫 `drop`，讓我們先用 `println!` 陳述式實作 `drop`。

範例 15-14 的結構體 `CustomSmartPointer` 只有一個功能，那就是在實例離開作用域時印出 `Dropping CustomSmartPointer!`。此範例能夠展示 Rust 何時會執行 `drop` 函式。

檔案名稱：src/main.rs

```
struct CustomSmartPointer {
    data: String,
}

impl Drop for CustomSmartPointer {
    fn drop(&mut self) {
        println!("釋放 CustomSmartPointer 的資料 `{}`!", self.data);
    }
}

fn main() {
    let c = CustomSmartPointer {
        data: String::from("我的東東"),
    };
    let d = CustomSmartPointer {
        data: String::from("其他東東"),
    };
    println!("CustomSmartPointers 建立完畢。");
}
```

範例 15-14： `CustomSmartPointer` 結構體實作了會放置清理程式碼的 `Drop` 特徵

`Drop` 特徵包含在 `prelude` 中，所以我們不需要特地引入作用域。我們對 `CustomSmartPointer` 實作 `Drop` 特徵並提供會呼叫 `println!` 的 `drop` 方法實作。`drop` 的函式本體用來放置你想要在型別實例離開作用域時執行的邏輯。我們在此印出一些文字來展示 Rust 如何呼叫 `drop`。

在 `main` 中，我們建立了兩個 `CustomSmartPointer` 實例並印出 `CustomSmartPointers` 建立完畢。在 `main` 結尾，我們的 `CustomSmartPointer` 實例會離開作用域，然後 Rust 就會呼叫我們放在 `drop` 方法的程式碼，也就是印出我們的最終訊息。注意到我們不需要顯式呼叫 `drop` 方法。

當我們執行此程式時，我們會看到以下輸出：

```
$ cargo run
  Compiling drop-example v0.1.0 (file:///projects/drop-example)
  Finished dev [unoptimized + debuginfo] target(s) in 0.60s
  Running `target/debug/drop-example`
CustomSmartPointers 建立完畢。
釋放 CustomSmartPointer 的資料 `其他東東`!
釋放 CustomSmartPointer 的資料 `我的東東`!
```

當我們的實例離開作用域時，Rust 會自動呼叫 `drop`，呼叫我們指定的程式碼。變數會以與建立時相反的順序被釋放，所以 `d` 會在 `c` 之前被釋放。此範例給了我們一個觀察 `drop` 如何執行的視覺化指引，通常你會指定該型別所需的清除程式碼，而不是印出訊息。

透過 `std::mem::drop` 提早釋放數值

不幸的是，我們無法直接了當地取消自動 `drop` 的功能。停用 `drop` 通常是不必要的，整個 `Drop` 的目的本來就是要能自動處理。不過有些時候你可能會想要提早清除數值。其中一個例子是使用智慧指標來管理鎖：你可能會想要強制呼叫 `drop` 方法來釋放鎖，好讓作用域中的其他程式碼可以取得該鎖。Rust 不會讓你手動呼叫 `Drop` 特徵的 `drop` 方法。不過如果你想要一個數值在離開作用域前就被釋放的話，你可以使用標準函式庫提供的 `std::mem::drop` 函式來呼叫。

如果我們嘗試修改範例 15-14 的 `main` 函式來手動呼叫 `Drop` 特徵的 `drop` 方法，如範例 15-15 所示，我們會得到編譯錯誤：

檔案名稱：src/main.rs

```
fn main() {
    let c = CustomSmartPointer {
        data: String::from("某些資料"),
    };
    println!("CustomSmartPointer 建立完畢。");
    c.drop();
    println!("CustomSmartPointer 在 main 結束前就被釋放了。");
}
```



範例 15-15：嘗試呼叫 `Drop` 特徵的 `drop` 方法來手動提早清除

當我們嘗試編譯此程式碼，我們會獲得以下錯誤：

```
$ cargo run
   Compiling drop-example v0.1.0 (file:///projects/drop-example)
error[E0040]: explicit use of destructor method
  --> src/main.rs:16:7
   |
16 |     c.drop();
   |     ^^^^^^^
   |
   | explicit destructor calls not allowed
   | help: consider using `drop` function: `drop(c)`

For more information about this error, try `rustc --explain E0040`.
error: could not compile `drop-example` due to previous error
```

此錯誤訊息表示我們不允許顯式呼叫 `drop`。錯誤訊息使用了一個術語**解構子 (destructor)**，這是通用程式設計術語中表達會清除實例的函式。**解構子**對應的術語就是**建構子 (constructor)**，這會建立實例。Rust 中的 `drop` 函式就是一種特定的解構子。

Rust 不讓我們顯式呼叫 `drop`，因為 Rust 還是會在 `main` 結束時自動呼叫 `drop`。這樣可能會導致**重複釋放 (double free)** 的錯誤，因為 Rust 可能會嘗試清除相同的數值兩次。

當數值離開作用域時我們無法停用自動插入的 `drop`，而且我們無法顯式呼叫 `drop` 方法，所以如果我必須強制讓一個數值提早清除的話，我們可以用 `std::mem::drop` 函式。

`std::mem::drop` 函式不同於 `Drop` 中的 `drop` 方法，我們傳入想要強制提早釋放的數值作為引數。此函式也包含在 `prelude`，所以我們可以修改範例 15-15 的 `main` 來呼叫 `drop` 函式，如範例 15-16 所示：

檔案名稱：src/main.rs

```
fn main() {
    let c = CustomSmartPointer {
        data: String::from("某些資料"),
    };
    println!("CustomSmartPointer 建立完畢。");
    drop(c);
    println!("CustomSmartPointer 在 main 結束前就被釋放了。");
}
```

範例 15-16：在數值離開作用域前呼叫 `std::mem::drop` 來顯示釋放數值

執行此程式會印出以下結果：

```
$ cargo run
Compiling drop-example v0.1.0 (file:///projects/drop-example)
Finished dev [unoptimized + debuginfo] target(s) in 0.73s
Running `target/debug/drop-example`
CustomSmartPointer 建立完畢。
釋放 CustomSmartPointer 的資料 `某些資料`！
CustomSmartPointer 在 main 結束前就被釋放了。
```

釋放 CustomSmartPointer 的資料 `某些資料`！這段文字會在 CustomSmartPointer 建立完畢。與 CustomSmartPointer 在 main 結束前就被釋放了。文字之間印出，顯示 drop 方法會在那時釋放 c。

你可以在許多地方使用 Drop 特徵實作所指定的程式碼，讓清除實例變得方便又安全。舉例來說，你可以用它來建立你自己的記憶體配置器！透過 Drop 特徵與 Rust 的所有權系統，你不必去擔心要記得清理，因為 Rust 會自動處理。

你也不必擔心會意外清理仍在使用的數值：所有權系統會確保所有參考永遠有效，並確保當數值不再需要時只會呼叫 drop 一次。

現在你看過 Box<T> 以及一些智慧指標的特性了，讓我們來看看一些其他定義在標準函式庫的智慧指標吧。

Rc<T> 參考計數智慧指標

在大多數的場合，所有權是很明確的：你能確切知道哪些變數擁有哪些數值。然而還是有些情況會需要讓一個數值能有數個擁有者。舉例來說，在圖資料結構中數個邊可能就會指向同個節點，而該節點概念上就被所有指向它的邊所擁有。節點直到沒有任何邊指向它，也就是沒有任何擁有者時才會被清除。

你必須使用 Rust 的型別 `Rc<T>` 才能擁有多重所有權，這是**參考計數**（reference counting）的簡寫。`Rc<T>` 型別會追蹤參考其數值的數量來決定該數值是否還在使用中。如果數值沒有任何參考的話，該數值就可以被清除，因為不會產生任何無效參考。

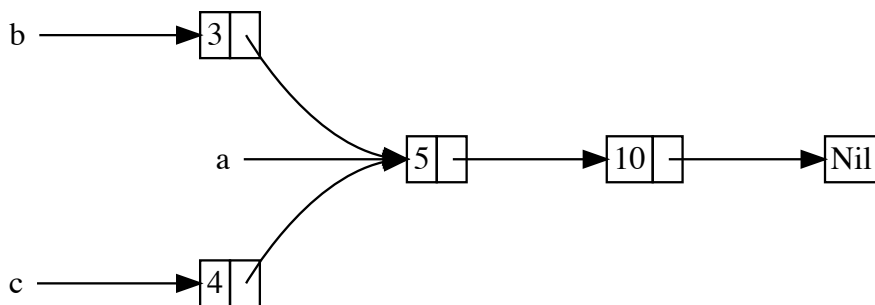
想像 `Rc<T>` 是個在客廳裡的電視，當有人進入客廳要看電視時，它們就會打開它。其他人也能進來觀看電視。當最後一個人離開客廳時，它們會關掉電視，因為沒有任何人會再看了。如果當其他人還在看電視時，有人關掉了它，其他在看電視的人肯定會生氣。

`Rc<T>` 型別的使用時機在於當我們想要在堆積上配置一些資料給程式中數個部分讀取，但是我們無法在編譯時期決定哪個部分會最後一個結束使用數值的部分。如果我們知道哪個部分會最後結束的話，我們可以將那個部分作為資料的擁有者就好，然後正常的所有權規則就會在編譯時生效。

注意到 `Rc<T>` 只適用於單一執行緒（single-threaded）的場合。當我們在第十六章討論並行（concurrency）時，我們會介紹如何在多執行緒程式達成參考計數。

使用 `Rc<T>` 來分享資料

讓我們回顧範例 15-5 的 cons list 範例。回想一下我們當時是用 `Box<T>` 定義。這次我們會建立兩個列表，它們會同時共享第三個列表的所有權。概念上會如圖示 15-3 所示：



圖示 15-3：兩個列表 `b` 和 `c` 共享第三個列表 `a` 的所有權

我們會建立列表 `a` 來包含 5 然後是 10。然後我們會在建立兩個列表：`b` 以 3 為開頭而 `c` 以 4 為開頭。`b` 與 `c` 列表會同時連接包含 5 與 10 的第一個列表 `a`。換句話說，兩個列表會同時共享包含 5 與 10 的第一個列表。

嘗試使用 `Box<T>` 來定義這種情境的 `List` 的話會無法成功，如範例 15-17 所示：

檔案名稱：src/main.rs

```
enum List {
    Cons(i32, Box<List>),
    Nil,
}

use crate::List::{Cons, Nil};

fn main() {
    let a = Cons(5, Box::new(Cons(10, Box::new(Nil))));
    let b = Cons(3, Box::new(a));
    let c = Cons(4, Box::new(a));
}
```



範例 15-17：展示我們無法用 `Box<T>` 讓兩個列表嘗試共享第三個列表的所有權

當我們編譯此程式碼，我們會得到以下錯誤：

```
$ cargo run
Compiling cons-list v0.1.0 (file:///projects/cons-list)
error[E0382]: use of moved value: `a`
--> src/main.rs:11:30
   |
9  |     let a = Cons(5, Box::new(Cons(10, Box::new(Nil))));
   |         - move occurs because `a` has type `List`, which does not
   |         implement the `Copy` trait
10 |     let b = Cons(3, Box::new(a));
   |                           - value moved here
11 |     let c = Cons(4, Box::new(a));
   |                           ^ value used here after move

For more information about this error, try `rustc --explain E0382`.
error: could not compile `cons-list` due to previous error
```

`Cons` 變體擁有它們存有的資料，所以當我們建立列表 `b` 時，`a` 會移動到 `b`，所以 `b` 就擁有 `a`。然後當我們嘗試再次使用 `a` 來建立 `c` 時，這就不會被允許，因為 `a` 已經被移走了。

我們可以嘗試改用參考來變更 `Cons` 的定義，但是這樣我們就必須指定生命週期參數。透過指定生命週期參數，我們會指定列表中的每個元素會至少活得跟整個列表一樣久。範例 15-17 的元素和列表雖然可以這樣，但不是所有的場合都是如此。

我們最後可以改用 `Rc<T>` 來變更 `List` 的定義，如範例 15-18 所示。每個 `Cons` 變體都會存有一個數值以及一個由 `Rc<T>` 指向的 `List`。當我們建立 `b` 時，不會取走 `a` 的所有權，我們會克隆 (clone) `a` 存有的 `Rc<List>`，因而增加參考的數量從一增加到二，並讓 `a` 與 `b` 共享 `Rc<List>` 資料的所有權。我們也在建立 `c` 時克隆 `a`，增加參考的數量從二增加到三。每次我們

呼叫 `Rc::clone` 時，對 `Rc<List>` 資料的參考計數就會成增加，然後資料不會被清除直到沒有任何參考為止。

檔案名稱：src/main.rs

```
enum List {
    Cons(i32, Rc<List>),
    Nil,
}

use crate::List::{Cons, Nil};
use std::rc::Rc;

fn main() {
    let a = Rc::new(Cons(5, Rc::new(Cons(10, Rc::new(Nil)))))
    let b = Cons(3, Rc::clone(&a));
    let c = Cons(4, Rc::clone(&a));
}
```

範例 15-18：使用 `Rc<T>` 來定義 `List`

我們需要使用 `use` 陳述式來將 `Rc<T>` 引入作用域，因為它沒有被包含在 `prelude` 中。在 `main` 中，我們建立了一個包含 5 與 10 的列表並存入 `a` 的 `Rc<List>`。然後當我們建立 `b` 與 `c` 時，我們會呼叫函式 `Rc::clone` 來將 `a` 的 `Rc<List>` 參考作為引數傳入。

當然我們可以呼叫 `a.clone()` 而非 `Rc::clone(&a)`，但是在此情形中 Rust 的慣例是使用 `Rc::clone`。 `Rc::clone` 的實作不會像大多數型別的 `clone` 實作會深拷貝（deep copy）所有的資料。 `Rc::clone` 的呼叫只會增加參考計數，這花費的時間就相對很少。深拷貝通常會花費比較多的時間。透過使用 `Rc::clone` 來參考計數，我們可以以視覺辨別出這是深拷貝的克隆還是增加參考計數的克隆。當我們需要調查程式碼的效能問題時，我們就只需要考慮深拷貝的克隆，而不在意 `Rc::clone`。

克隆 `Rc<T>` 實例會增加其參考計數

讓我們改變範例 15-18 的範例，好讓我們能觀察參考計數在我們建立與釋放 `a` 的 `Rc<List>` 參考時產生的變化。

範例 15-19，我們改變了 `main` 讓列表 `c` 寫在一個內部作用域中，然後我們就能觀察到當 `c` 離開作用域時參考計數產生的改變。

檔案名稱：src/main.rs

範例 15-19：印出參考計數

此程式碼印出以下結果：

我們可以看到 `a` 的 `Rc<List>` 會有個初始參考計數 1，然後我們每次呼叫 `clone` 時，計數會加 1。當 `c` 離開作用域時，計數會減 1。我們不必呼叫任何函式來減少參考計數，像呼叫 `Rc::clone` 時才會增加參考計數那樣。當 `Rc<T>` 數值離開作用域時，`Drop` 特徵的實作就會自動減少參考計數。

我們無法從此例觀察到的是當 `b` 然後是 `a` 從 `main` 的結尾離開作用域時，計數會是 0，然後 `Rc<List>` 在此時就會完全被清除。使用 `Rc<T>` 能允許單一數值能有數個擁有者，然後計數會確保只要有任何擁有者還存在的狀況下，數值會保持是有效的。

透過不可變參考，`Rc<T>` 能讓你分享資料給程式中數個部分來只做讀取的動作。如果 `Rc<T>` 允許你也擁有數個可變參考的話，你可能就違反了第四章提及的借用規則：數個對相同位置的可變借用會導致資料競爭（data races）與不一致。但可變資料還是非常實用的！在下個段落，我們會討論內部可變性模式與 `RefCell<T>` 型別，此型別能讓你搭配 `Rc<T>` 使用來處理不可變的限制。

RefCell<T> 與內部可變性模式

內部可變性 (Interior mutability) 是 Rust 中的一種設計模式，能讓你對即使是不可變參考的資料也能改變。正常狀況下，借用規則是不允許這種動作的。為了改變資料，這樣的模式會在資料結構內使用 `unsafe` 程式碼來繞過 Rust 的常見可變性與借用規則。不安全 (`unsafe`) 的程式碼等於告訴編譯器我們會自己手動檢查，編譯器不會檢查全部的規則，我們會在第十九章討論更多關於不安全的程式碼。

當編譯器無法保障，但我們可以確保借用規則在執行時能夠遵循的話，我們就可以使用擁有內部可變性模式的型別。其內的 `unsafe` 程式碼會透過安全的 API 封裝起來，讓外部型別仍然是不可變的。

讓我們觀察擁有內部可變性模式的 `RefCell<T>` 型別來探討此概念。

透過 `RefCell<T>` 在執行時強制檢測借用規則

不像 `Rc<T>`，`RefCell<T>` 型別的資料只會有一個所有權。所以 `RefCell<T>` 與 `Box<T>` 這種型別有何差別呢？回憶一下你在第四章學到的借用規則：

- 在任何時候，我們要麼**只能**有一個可變參考，要麼可以有**任意數量**的不可變參考。
- 參考必須永遠有效。

對於參考與 `Box<T>`，借用規則會在編譯期強制檢測。對於 `RefCell<T>`，這些規則會在**執行時**才強制執行。對於參考來說，如果你打破這些規則，你會得到編譯錯誤。而對 `RefCell<T>` 來說，如果你打破這些規則，你的程式會恐慌並離開。

在編譯時期檢查借用規則的優勢在於錯誤能在開發過程及早獲取，而且這對執行時的效能沒有任何影響，因為所有的分析都預先完成了。基於這些原因，在編譯時檢查借用規則在大多數情形都是最佳選擇，這也是為何這是 Rust 預設設置的原因。

在執行時檢查借用規則的優勢則在於能允許一些特定記憶體安全的場合，而這些原本是不被編譯時檢查所允許的。像 Rust 編譯器這種靜態分析本質上是保守的。有些程式碼特性是無法透過分析程式碼檢測出的，最著名的範例就是停機問題 (Halting Problem)，這超出本書的範疇，但是是個有趣的研究議題。

因為有些分析是不可能的，如果 Rust 編譯器無法確定程式碼是否符合所有權規則，它可能會拒絕一支正確的程式，所以由此觀點來看能知道 Rust 編譯器是保守的。如果 Rust 接受不正確的程式，使用者就無法信任 Rust 帶來的保障。然而如果 Rust 拒絕正確的程式，對程式設計師就會很不方便，但沒有任何嚴重的災難會發生。`RefCell<T>` 型別就適用於當你確定你的程式碼有遵循借用規則，但是編譯器無法理解並保證的時候。

類似於 `Rc<T>`，`RefCell<T>` 也只能用於單一執行緒 (single-threaded) 的場合，所以如果你嘗試用在多執行緒上的話就會出現編譯時錯誤。我們會在第十六章討論如何在多執行緒程式擁有

`RefCell<T>` 的功能。

以下是何時選擇 `Box<T>`、`Rc<T>` 或 `RefCell<T>` 的理由：

- `Rc<T>` 讓數個擁有者能共享相同資料；`Box<T>` 與 `RefCell<T>` 只能有一個擁有者。
- `Box<T>` 能有不可變或可變的借用並在編譯時檢查；`Rc<T>` 則只能有不可變借用並在編譯時檢查；`RefCell<T>` 能有不可變或可變借用但是在執行時檢查。
- 由於 `RefCell<T>` 允許在執行時檢查可變參考，你可以改變 `RefCell<T>` 內部的數值，就算 `RefCell<T>` 是不可變的。

改變不可變數值內部的值稱為**內部可變性**模式。讓我們看看內部可變性何時會有用，且觀察為何是可行的。

內部可變性：不可變數值的可變借用

借用規則的影響是當你有個不可變數值，你就無法取得可變參考。舉例來說，以下程式碼會無法編譯：

```
fn main() {  
    let x = 5;  
    let y = &mut x;  
}
```



如果你嘗試編譯此程式碼，你會獲得以下錯誤：

```
$ cargo run  
    Compiling borrowing v0.1.0 (file:///projects/borrowing)  
error[E0596]: cannot borrow `x` as mutable, as it is not declared as mutable  
--> src/main.rs:3:13  
  |  
2 |     let x = 5;  
  |         - help: consider changing this to be mutable: `mut x`  
3 |     let y = &mut x;  
  |             ^^^^^^ cannot borrow as mutable  
  
For more information about this error, try `rustc --explain E0596`.  
error: could not compile `borrowing` due to previous error
```

然而在某些特定情況，我們會想要能夠有個方法可以改變一個數值，但該數值對其他程式碼而言仍然是不可變的。數值提供的方法以外的程式碼都無法改變其值。使用 `RefCell<T>` 是取得內部可變性的方式之一。但 `RefCell<T>` 仍然要完全遵守借用規則：編譯器的借用檢查器會允許這些內部可變性，然後在執行時才檢查借用規則。如果你違反規則，你就會得到 `panic!` 而非編譯錯誤。

讓我們用一個實際例子來探討如何使用 `RefCell<T>` 來改變不可變數值，並瞭解為何這是很實用的。

內部可變性的使用案例：模擬物件

程式設計師有時在進行測試時會將一個型別替換成其他型別，用以觀察特定行為並判定是否有正確實作。這種型別就稱為**測試替身**（test double）。你可以想成這和影視產業中的「特技替身演員」類似，有個人會代替原本的演員來拍攝一些特定的場景。測試替身會在執行測試時代替其他型別。**模擬物件**（Mock objects）是測試替身其中一種特定型別，這能紀錄測試過程中發生什麼事並讓你判斷動作是否正確。

Rust 的物件與其他語言中的物件概念並不全然相同，而且 Rust 的標準函式庫內也沒有如其他語言會內建的模擬物件功能。不過你還是可以有方法來建立結構體來作為模擬物件。

以下是我們要測試的情境：我們建立一個函式庫來追蹤一個數值與最大值的差距，並依據該差距傳送訊息。舉例來說，此函式庫就能用來追蹤使用者允許呼叫 API 次數的上限。

我們的函式庫提供的功能只有追蹤與最大值的距離以及何時該傳送什麼訊息。使用函式庫的應用程式要提供傳送訊息的機制，應用程式可以將訊息存在應用程式內、傳送電子郵件、傳送文字訊息或其他等等。函式庫不需要知道細節，它只需要在意會有項目實作我們提供的 `Messenger` 特徵。範例 15-20 顯示了函式庫的程式碼：

檔案名稱：`src/lib.rs`

```

pub trait Messenger {
    fn send(&self, msg: &str);
}

pub struct LimitTracker<'a, T: Messenger> {
    messenger: &'a T,
    value: usize,
    max: usize,
}

impl<'a, T> LimitTracker<'a, T>
where
    T: Messenger,
{
    pub fn new(messenger: &'a T, max: usize) -> LimitTracker<'a, T> {
        LimitTracker {
            messenger,
            value: 0,
            max,
        }
    }

    pub fn set_value(&mut self, value: usize) {
        self.value = value;

        let percentage_of_max = self.value as f64 / self.max as f64;

        if percentage_of_max >= 1.0 {
            self.messenger.send("錯誤：你超過使用上限了！");
        } else if percentage_of_max >= 0.9 {
            self.messenger
                .send("緊急警告：你已經使用 90% 的配額了！");
        } else if percentage_of_max >= 0.75 {
            self.messenger
                .send("警告：你已經使用 75% 的配額了！");
        }
    }
}

```

範例 15-20：追蹤某個值與最大值差距的函式庫並以此值的特定層級傳送警告

此程式碼其中一個重點是 `Messenger` 特徵有個方法叫做 `send`，這會接收一個 `self` 的不可變參考與一串訊息文字。此特徵就是我們的模擬物件所需實作的介面，讓我們能模擬和實際物件一樣的行爲。另一個重點是我們想要測試 `LimitTracker` 中 `set_value` 方法的行爲。我們可以改變傳給參數 `value` 的值，但是 `set_value` 沒有回傳任何東西好讓我們做判斷。我們希望如果我們透過某個實作 `Messenger` 的型別與特定數值 `max` 來建立 `LimitTracker` 時，傳送訊息者能被通知要傳遞合適的訊息。

我們需要有個模擬物件，而不是在呼叫 `send` 時真的傳送電子郵件或文字訊息，我們只想紀錄訊息被通知要傳送了。我們可以建立模擬物件的實例，以此建立 `LimitTracker`、呼叫 `LimitTracker` 的 `set_value`，並檢查模擬物件有我們預期的訊息。範例 15-21 展示一個嘗試實作此事的模擬物件，但借用檢查器卻不允許：

檔案名稱：src/lib.rs

```
#[cfg(test)]
mod tests {
    use super::*;

    struct MockMessenger {
        sent_messages: Vec<String>,
    }

    impl MockMessenger {
        fn new() -> MockMessenger {
            MockMessenger {
                sent_messages: vec![],
            }
        }
    }

    impl Messenger for MockMessenger {
        fn send(&self, message: &str) {
            self.sent_messages.push(String::from(message));
        }
    }

    #[test]
    fn it_sends_an_over_75_percent_warning_message() {
        let mock_messenger = MockMessenger::new();
        let mut limit_tracker = LimitTracker::new(&mock_messenger, 100);

        limit_tracker.set_value(80);

        assert_eq!(mock_messenger.sent_messages.len(), 1);
    }
}
```



範例 15-21：嘗試實作 `MockMessenger` 但借用檢查器不允許

此測試程式碼定義了一個結構體 `MockMessenger` 其有個 `sent_messages` 欄位並存有 `String` 數值的 `Vec` 來追蹤被通知要傳送的訊息。我們也定義了一個關聯函式 `new` 讓我們可以方便建立起始訊息列表為空的 `MockMessenger`。我們對 `MockMessenger` 實作 `Messenger` 特徵，這樣我們才能將 `MockMessenger` 交給 `LimitTracker`。在 `send` 方法的定義中，我們取得由參數傳遞的訊息，並存入 `MockMessenger` 的 `sent_messages` 列表中。

但是此測試有個問題，如以下所示：

我們無法修改 `MockMessenger` 來追蹤訊息，因為 `send` 方法取得的是 `self` 的不可變參考。而我們也無法使用錯誤訊息中推薦使用的 `&mut self`，因為 `send` 的簽名就會與 `Messenger` 特徵所定義的不相符（你可以試看看並觀察錯誤訊息）。

檔案名稱：src/lib.rs

```

#[cfg(test)]
mod tests {
    use super::*;
    use std::cell::RefCell;

    struct MockMessenger {
        sent_messages: RefCell<Vec<String>>,
    }

    impl MockMessenger {
        fn new() -> MockMessenger {
            MockMessenger {
                sent_messages: RefCell::new(vec![]),
            }
        }
    }

    impl Messenger for MockMessenger {
        fn send(&self, message: &str) {
            self.sent_messages.borrow_mut().push(String::from(message));
        }
    }

    #[test]
    fn it_sends_an_over_75_percent_warning_message() {
        // --省略--

        assert_eq!(mock_messenger.sent_messages.borrow().len(), 1);
    }
}

```

範例 15-22：在外部數值為不可變時，使用 `RefCell<T>` 來改變內部數值

`sent_messages` 欄位現在是型別 `RefCell<Vec<String>>` 而非 `Vec<String>`。在 `new` 函式中，我們用空的向量來建立新的 `RefCell<Vec<String>>`。

至於 `send` 方法的實作，第一個參數仍然是 `self` 的不可變借用，這就符合特徵所定義的。我們在 `self.sent_messages` 對 `RefCell<Vec<String>>` 呼叫 `borrow_mut` 來取得 `RefCell<Vec<String>>` 內的可變參考數值，也就是向量。然後我們對向量的可變參考呼叫 `push` 來追蹤測試中的訊息。

最後一項改變是判定：要看到內部向量有多少項目的話，我們對 `RefCell<Vec<String>>` 呼叫 `borrow` 來取得向量的不可變參考。

現在你已經知道如何使用 `RefCell<T>`，讓我們進一步探討它如何運作的吧！

透過 `RefCell<T>` 在執行時追蹤借用

當建立不可變與可變參考時，我們分別使用 `&` 和 `&mut` 語法。而對於 `RefCell<T>` 的話，我們使用 `borrow` 和 `borrow_mut` 方法，這是 `RefCell<T>` 所提供的安全 API 之一。`borrow` 方法回傳一個智慧指標型別 `Ref<T>`，而 `borrow_mut` 回傳智慧指標型別 `RefMut<T>`。這兩個型別都有實作 `Deref`，所以我們可以像一般參考來對待它們。

`RefCell<T>` 會追蹤當前有多少 `Ref<T>` 和 `RefMut<T>` 智慧指標存在。每次我們呼叫 `borrow` 時，`RefCell<T>` 會增加不可變借用計數。當 `Ref<T>` 離開作用域時，不可變借用計數就會減一。就和編譯時借用規則一樣，`RefCell<T>` 讓我們同一時間要麼只能有一個可變參考，要麼可以有數個不可變參考。

如果我們嘗試違反這些規則，我們不會像參考那樣得到編譯器錯誤，`RefCell<T>` 的實作會在執行時恐慌。範例 15-23 修改了範例 15-22 的 `send` 實作。我們故意嘗試在同個作用域下建立兩個可變參考，來說明 `RefCell<T>` 會不允許我們在執行時這樣做。

檔案名稱：src/lib.rs

```
impl Messenger for MockMessenger {
    fn send(&self, message: &str) {
        let mut one_borrow = self.sent_messages.borrow_mut();
        let mut two_borrow = self.sent_messages.borrow_mut();

        one_borrow.push(String::from(message));
        two_borrow.push(String::from(message));
    }
}
```



範例 15-23：在同個作用域建立兩個可變參考並觀察到 `RefCell<T>` 會恐慌

我們從 `borrow_mut` 回傳的 `RefMut<T>` 智慧指標來建立變數 `one_borrow`。然後我們再以相同方式建立另一個變數 `two_borrow`。這在同個作用域下產生了兩個可變參考，而這是不允許的。我們執行函式庫的測試時，範例 15-23 可以編譯通過，但是執行測試會失敗：

```

$ cargo test
   Compiling limit-tracker v0.1.0 (file:///projects/limit-tracker)
   Finished test [unoptimized + debuginfo] target(s) in 0.91s
   Running unittests src/lib.rs (target/debug/deps/limit_tracker-
e599811fa246dbde)

running 1 test
test tests::it_sends_an_over_75_percent_warning_message ... FAILED

failures:

---- tests::it_sends_an_over_75_percent_warning_message stdout ----
thread 'main' panicked at 'already borrowed: BorrowMutError', src/lib.rs:60:53
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
    tests::it_sends_an_over_75_percent_warning_message

test result: FAILED. 0 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass `--lib`

```

注意到程式碼恐慌時的訊息 `already borrowed: BorrowMutError`。這就是 `RefCell<T>` 如何在執行時處理違反借用規則的情況。

像我們在這裡選擇在執行時獲取借用錯誤而不是在編譯時，代表你會在開發過程之後才找到程式碼錯誤，甚至有可能一直到程式碼部署到正式環境後才查覺。而且你的程式碼也會多了一些小小的執行時效能開銷，作為在執行時而非編譯時檢查的代價。不過使用 `RefCell<T>` 讓你能够在只允許有不可變數值的環境中寫出能夠變更內部追蹤訊息的模擬物件。這是想獲得 `RefCell<T>` 帶來的功能時，要與一般參考之間作出的取捨。

組合 `Rc<T>` 與 `RefCell<T>` 來擁有多個可變資料的擁有者

`RefCell<T>` 的常見使用方法是搭配 `Rc<T>`。回想一下 `Rc<T>` 讓你可以對數個擁有者共享相同資料，但是它只能用於不可變資料。如果你有一個 `Rc<T>` 並存有 `RefCell<T>` 的話，你就可以取得一個有數個擁有者而且可變的數值！

舉例來說，回憶一下範例 15-18 `cons list` 的範例我們使用了 `Rc<T>` 來讓數個列表可以共享另一個列表的所有權。因為 `Rc<T>` 只能有不可變數值，我們一旦建立它們後就無法變更列表中的任何數值。讓我們加上 `RefCell<T>` 來獲得能改變列表數值的能力吧。範例 15-24 顯示了在 `Cons` 定義中使用 `RefCell<T>`，這樣一來我們就可以變更儲存在列表中的所有數值：

檔案名稱：src/main.rs

```

#[derive(Debug)]
enum List {
    Cons(Rc<RefCell<i32>>, Rc<List>),
    Nil,
}

use crate::List::{Cons, Nil};
use std::cell::RefCell;
use std::rc::Rc;

fn main() {
    let value = Rc::new(RefCell::new(5));

    let a = Rc::new(Cons(Rc::clone(&value), Rc::new(Nil)));

    let b = Cons(Rc::new(RefCell::new(3)), Rc::clone(&a));
    let c = Cons(Rc::new(RefCell::new(4)), Rc::clone(&a));

    *value.borrow_mut() += 10;

    println!("a 之後 = {:?}", a);
    println!("b 之後 = {:?}", b);
    println!("c 之後 = {:?}", c);
}

```

範例 15-24：使用 `Rc<RefCell<i32>>` 建立一個可變的 `List`

我們建立了一個 `Rc<RefCell<i32>>` 實例數值並將其存入變數 `value` 好讓我們之後可以直接取得。然後我們在 `a` 用持有 `value` 的 `Cons` 變體來建立 `List`。我們需要克隆 `value`，這樣 `a` 和 `value` 才能都有內部數值 5 的所有權，而不是從 `value` 轉移所有權給 `a`，或是讓 `a` 借用 `value`。

我們用 `Rc<T>` 封裝列表 `a`，所以當我們建立列表 `b` 和 `c` 時，它們都可以參考 `a`，就像範例 15-18 一樣。

在我們建立完列表 `a`、`b` 和 `c` 之後，我們想對 `value` 的數值加上 10。我們對 `value` 呼叫 `borrow_mut`，其中使用到了我們在第五章討論過的自動解參考功能（請查閱「[-> 運算子跑去哪了？](#)」的段落）來解參考 `Rc<T>` 成內部的 `RefCell<T>` 數值。`borrow_mut` 方法會回傳 `RefMut<T>` 智慧指標，而我們使用解參考運算子並改變其內部數值。

當我們印出 `a`、`b` 和 `c` 時，我們可以看到它們的數值都改成了 15 而非 5：

```
$ cargo run
  Compiling cons-list v0.1.0 (file:///projects/cons-list)
    Finished dev [unoptimized + debuginfo] target(s) in 0.63s
    Running `target/debug/cons-list`
a 之後 = Cons(RefCell { value: 15 }, Nil)
b 之後 = Cons(RefCell { value: 3 }, Cons(RefCell { value: 15 }, Nil))
c 之後 = Cons(RefCell { value: 4 }, Cons(RefCell { value: 15 }, Nil))
```

此技巧是不是很厲害！透過使用 `RefCell<T>`，我們可以得到一個外部是不可變的 `List` 數值，但是我們可以使用 `RefCell<T>` 提供的方法來取得其內部可變性，讓我們可以在我們想要時改變我們的資料。執行時的借用規則檢查能防止資料競爭，並在某些場合犧牲一點速度來換取資料結構的彈性。注意到 `RefCell<T>` 無法用在多執行緒的程式碼！`Mutex<T>` 才是執行緒安全版的 `RefCell<T>`，我們會在第十六章再討論 `Mutex<T>`。

參考循環會導致記憶體洩漏

意外情況下，執行程式時可能會產生永遠不會被清除的記憶體（通稱為**記憶體洩漏** / **memory leak**）。Rust 的記憶體安全性雖然可以保證令這種情況難以發生，但並非絕不可能。雖然 Rust 在編譯時可以保證做到禁止資料競爭（**data races**），但它無法保證完全避免記憶體洩漏，這是因為對 Rust 來說，記憶體洩漏是屬於安全範疇內的（**memory safe**）。透過使用 `Rc<T>` 和 `RefCell<T>`，我們能觀察到 Rust 允許使用者自行產生記憶體洩漏：因為使用者可以產生兩個參考並互相參照，造成一個循環。這種情況下會導致記憶體洩漏，因為循環中的參考計數永遠不會變成 0，所以數值永遠不會被釋放。

產生參考循環

讓我們看看參考循環是怎麼發生的，以及如何避免它。我們從範例 15-25 的 `List` 列舉定義與一個 `tail` 方法開始：

檔案名稱：src/main.rs

```
use crate::List::{Cons, Nil};
use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug)]
enum List {
    Cons(i32, RefCell<Rc<List>>),
    Nil,
}

impl List {
    fn tail(&self) -> Option<&RefCell<Rc<List>>> {
        match self {
            Cons(_, item) => Some(item),
            Nil => None,
        }
    }
}

fn main() {}
```

範例 15-25：一個 cons list 定義並持有 `RefCell<T>`，所以我們可以修改 `Cons` 變體參考的值

我們用的是範例 15-5 中 `List` 的另一種定義寫法。`Cons` 變體的第二個元素現在是 `RefCell<Rc<List>>`，代表不同於範例 15-24 那樣能修改 `i32` 數值，我們想要能修改 `Cons` 變

體指向的 `List` 數值。我們也加了一個 `tail` 方法讓我們如果有 `Cons` 變體的話，能方便取得第二個項目。

在範例 15-26 我們要加入 `main` 函式並使用範例 15-25 的定義。此程式碼建立了列表 `a` 與指向列表 `a` 的列表 `b`。然後它修改了列表 `a` 來指向 `b`，因而產生循環參考。在程序過程中 `println!` 陳述式會顯示不同位置時的參考計數。

檔案名稱：src/main.rs

```
fn main() {
    let a = Rc::new(Cons(5, RefCell::new(Rc::new(Nil))));

    println!("a 初始參考計數 = {}", Rc::strong_count(&a));
    println!("a 下個項目 = {:?}", a.tail());

    let b = Rc::new(Cons(10, RefCell::new(Rc::clone(&a))));

    println!("a 在 b 建立後的參考計數 = {}", Rc::strong_count(&a));
    println!("b 初始參考計數 = {}", Rc::strong_count(&b));
    println!("b 下個項目 = {:?}", b.tail());

    if let Some(link) = a.tail() {
        *link.borrow_mut() = Rc::clone(&b);
    }

    println!("b 在變更 a 後的參考計數 = {}", Rc::strong_count(&b));
    println!("a 在變更 a 後的參考計數 = {}", Rc::strong_count(&a));

    // 取消下一行的註解可以看到循環產生
    // 這會讓堆疊溢位
    // println!("a 下個項目 = {:?}", a.tail());
}
```

範例 15-26：透過兩個彼此指向對方的 `List` 數值來產生參考循環

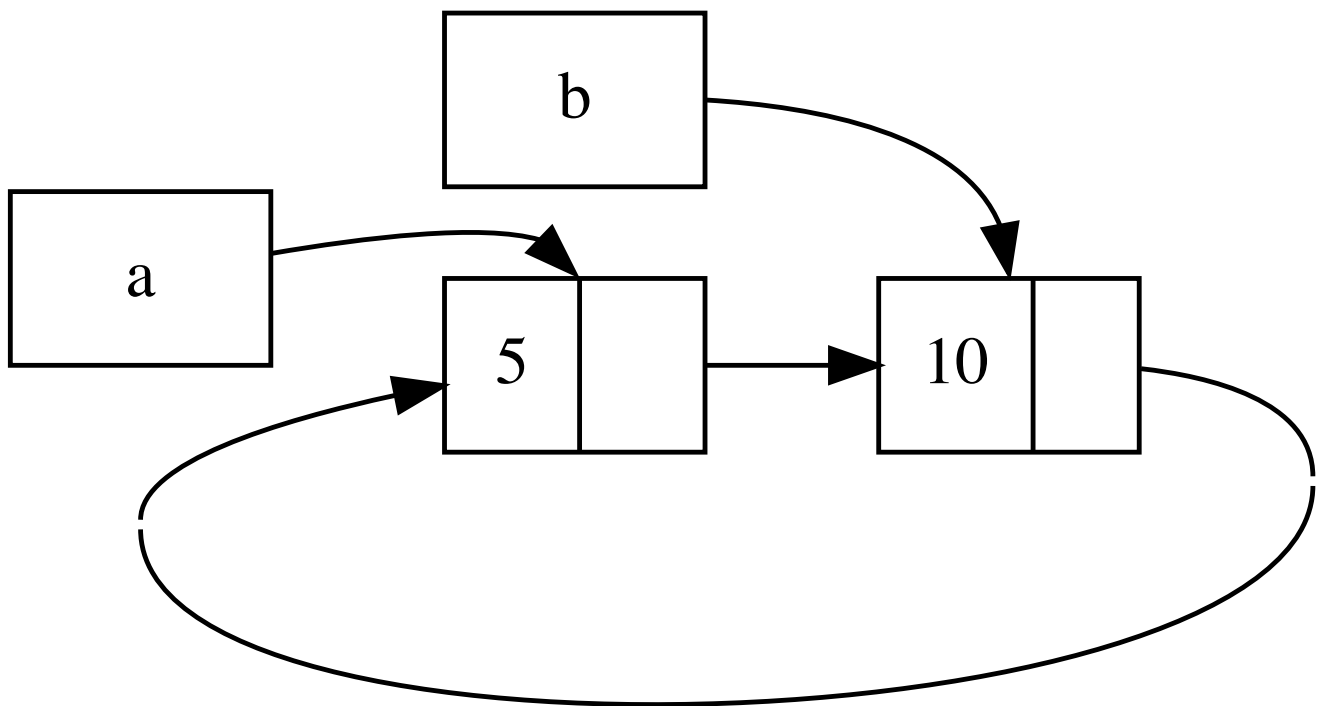
我們在變數 `a` 建立了一個 `Rc<List>` 實例的 `List` 數值並持有 `5`, `Nil` 初始列表的。我們然後在變數 `b` 建立另一個 `Rc<List>` 實例的 `List` 數值並持有數值 `10` 與指向的列表 `a`。

我們將 `a` 修改為指向 `b` 而非 `Nil` 來產生循環。我們透過使用 `tail` 方法來取得 `a` 的 `RefCell<Rc<List>>` 參考，並放入變數 `link` 中。然後我們對 `RefCell<Rc<List>>` 使用 `borrow_mut` 方法來改變 `Rc<List>` 的值，從數值 `Nil` 改成 `b` 的 `Rc<List>`。

當我們執行此程式並維持將最後一行的 `println!` 註解掉的話，我們會得到以下輸出：


```
$ cargo run
Compiling cons-list v0.1.0 (file:///projects/cons-list)
Finished dev [unoptimized + debuginfo] target(s) in 0.53s
Running `target/debug/cons-list`
a 初始參考計數 = 1
a 下個項目 = Some(RefCell { value: Nil })
a 在 b 建立後的參考計數 = 2
b 初始參考計數 = 1
b 下個項目 = Some(RefCell { value: Cons(5, RefCell { value: Nil }) })
b 在變更 a 後的參考計數 = 2
a 在變更 a 後的參考計數 = 2
```

在我們變更列表 `a` 來指向 `b` 後，`a` 和 `b` 的 `Rc<List>` 實例參考計數都是 2。在 `main` 結束後，Rust 會釋放 `b`，讓 `b` 的 `Rc<List>` 實例計數從 2 減到 1。此時堆積上 `Rc<List>` 的記憶體還不會被釋放，因為參考計數還有 1，而非 0。然後 Rust 釋放 `a`，讓 `a` 的 `Rc<List>` 實例也從 2 減到 1。此實例的記憶體也不會被釋放，因為另一個 `Rc<List>` 的實例仍然參考著它。列表配置的記憶體會永遠不被釋放。為了視覺化參考循環，我們用圖示 15-4 表示。



圖示 15-4：列表 `a` 與 `b` 彼此指向對方的參考循環

如果你解除最後一個 `println!` 的註解並執行程式的話，Rust 會嘗試印出此循環，因為 `a` 會指向 `b` 會指向 `a` 以此循環下去，直到堆疊溢位（stack overflow）。

比起真實世界的程式，此循環造成的影響並不嚴重。因為當我們建立完循環參考，程式就結束了。不過要是有個更複雜的程式配置了大量的記憶體而產生循環，並維持很長一段時間的話，程式會用到比原本預期還多的記憶體，並可能壓垮系統，導致它將記憶體用光。

要產生循環參考並不是件容易的事，但也不是絕對不可能。如果你有包含 `Rc<T>` 數值的 `RefCell<T>` 數值，或是有類似具內部可變性與參考計數巢狀組合的話，你必須確保不會產生循環

參考，你無法依靠 Rust 來檢查它們。產生循環參考是程式中的邏輯錯誤，你需要使用自動化測試、程式碼審查以及其他軟體開發技巧來最小化問題。

另一個避免參考循環的解決辦法是重新組織你的資料結構，確定哪些參考要有所有權，哪些參考不用。這樣一來，循環會由一些有所有權的關係與沒有所有權的關係所組成，而只有所有權關係能影響數值是否能被釋放。在範例 15-25 中。我們永遠會希望 `Cons` 變體擁有它們的列表，所以重新組織資料結構是不可能的。讓我們看看一個由父節點與子節點組成的圖形結構，來看看無所有權的關係何時適合用來避免循環參考。

避免參考循環：將 `Rc<T>` 轉換成 `Weak<T>`

目前，我們解釋過呼叫 `Rc::clone` 會增加 `Rc<T>` 實例的 `strong_count`，而 `Rc<T>` 只會在 `strong_count` 為 0 時被清除。你也可以對 `Rc<T>` 實例呼叫 `Rc::downgrade` 並傳入 `Rc<T>` 的參考來建立弱參考 (**weak reference**)。強參考是你分享 `Rc<T>` 實例的方式。弱參考不會表達所有權關係，它們的計數與 `Rc<T>` 的清除無關。它們不會造成參考循環，因為弱參考的循環會在其強參考計數歸零時解除。

當你呼叫 `Rc::downgrade` 時，你會得到一個型別為 `Weak<T>` 的智慧指標。不同於對 `Rc<T>` 實例的 `strong_count` 增加 1，呼叫 `Rc::downgrade` 會對 `weak_count` 增加 1。`Rc<T>` 型別使用 `weak_count` 來追蹤有多少 `Weak<T>` 的參考存在，這類似於 `strong_count`。不同的地方在於 `weak_count` 不需要歸零才能將 `Rc<T>` 清除。

由於 `Weak<T>` 的參考數值可能會被釋放，要對 `Weak<T>` 指向的數值做任何事情時，你都必須確保該數值還存在。你可以透過對 `Weak<T>` 實例呼叫 `upgrade` 方法，這會回傳 `Option<Rc<T>>`。如果 `Rc<T>` 數值還沒被釋放的話，你就會得到 `Some`；而如果 `Rc<T>` 數值已經被釋放的話，就會得到 `None`。因為 `upgrade` 回傳 `Option<Rc<T>>`，Rust 會確保 `Some` 與 `None` 的分支都有處理好，所以不會取得無效指標。

為了做示範，與其使用知道下一項的列表的例子，我們會建立一個樹狀結構，每一個項目會知道它們的子項目以及它們的父項目。

建立樹狀資料結構：帶有子節點的 `Node`

首先我們建立一個帶有節點的樹，每個節點知道它們的子節點。我們會定義一個結構體 `Node` 來存有它自己的 `i32` 數值以及其子數值 `Node` 的參考：

檔案名稱：src/main.rs

```

use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug)]
struct Node {
    value: i32,
    children: RefCell<Vec<Rc<Node>>>,
}

```

我們想要 `Node` 擁有自己的子節點，而且我們想要透過變數分享所有權，讓我們可以在樹中取得每個 `Node`。為此我們定義 `Vec<T>` 項目作為型別 `Rc<Node>` 的數值。我們還想要能夠修改哪些節點才是該項目的子節點，所以我們將 `children` 中的 `Vec<Rc<Node>>` 加進 `RefCell<T>`。

接著，我們使用我們定義的結構體來建立一個 `Node` 實例叫做 `leaf`，其數值為 3 且沒有子節點；我們再建立另一個實例叫做 `branch`，其數值為 5 且有個子節點 `leaf`。如範例 15-27 所示：

檔案名稱：src/main.rs

```

fn main() {
    let leaf = Rc::new(Node {
        value: 3,
        children: RefCell::new(vec![]),
    });

    let branch = Rc::new(Node {
        value: 5,
        children: RefCell::new(vec![Rc::clone(&leaf)]),
    });
}

```

範例 15-27：建立一個沒有子節點的 `leaf` 節點與一個有 `leaf` 作為子節點的 `branch` 節點

我們克隆 `leaf` 的 `Rc<Node>` 並存入 `branch`，代表 `leaf` 的 `Node` 現在有兩個擁有者：`leaf` 和 `branch`。我們可以透過 `branch.children` 從 `branch` 取得 `leaf`，但是從 `leaf` 無法取得 `branch`。原因是因為 `leaf` 沒有 `branch` 的參考且不知道它們之間是有關聯的。我們想要 `leaf` 能知道 `branch` 是它的父節點。這就是我們接下來要做的事。

新增從子節點到父節點的參考

要讓子節點意識到它的父節點，我們需要在我們的 `Node` 結構體定義中加個 `parent` 欄位。問題在於 `parent` 應該要是什麼型別。我們知道它不能包含 `Rc<T>`，因為那就會造成參考循環，`leaf.parent` 就會指向 `branch` 且 `branch.children` 就會指向 `leaf`，導致同名的 `strong_count` 數值無法歸零。

讓我們換種方式思考此關係，父節點必須擁有它的子節點，如果父節點釋放的話，它的子節點也應該要被釋放。但子節點不應該擁有它的父節點，如果我們釋放子節點的話，父節點應該要還存在。

這就是弱參考的使用時機！

所以與其使用 `Rc<T>`，我們使用 `Weak<T>` 來建立 `parent` 的型別，更明確的話就是 `RefCell<Weak<Node>>`。現在我們的 `Node` 結構體定義看起來會像這樣：

檔案名稱：src/main.rs

```
use std::cell::RefCell;
use std::rc::{Rc, Weak};

#[derive(Debug)]
struct Node {
    value: i32,
    parent: RefCell<Weak<Node>>,
    children: RefCell<Vec<Rc<Node>>>,
}
```

節點能夠參考其父節點但不會擁有它。在範例 15-28 中我們更新了 `main` 來使用新的定義，讓 `leaf` 節點有辦法參考它的父節點 `branch`：

檔案名稱：src/main.rs

```
fn main() {
    let leaf = Rc::new(Node {
        value: 3,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![]),
    });

    println!("leaf 的父節點 {:?}", leaf.parent.borrow().upgrade());

    let branch = Rc::new(Node {
        value: 5,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![Rc::clone(&leaf)]),
    });

    *leaf.parent.borrow_mut() = Rc::downgrade(&branch);

    println!("leaf 的父節點 {:?}", leaf.parent.borrow().upgrade());
}
```

範例 15-28：`leaf` 節點有其父節點 `branch` 的弱參考

建立 `leaf` 節點與範例 15-27 類似，只是要多加個 `parent` 欄位：`leaf` 一開始沒有任何父節點，所以我們建立一個空的 `Weak<Node>` 參考實例。

此時當我們透過 `upgrade` 方法嘗試取得 `leaf` 的父節點參考的話，我們會取得 `None` 數值。我們能在輸出結果的第一個 `println!` 陳述式看到：

```
leaf 的父節點 None
```

當我們建立 `branch` 節點，它的 `parent` 欄位也會有個新的 `Weak<Node>` 參考，因為 `branch` 沒有父節點。我們仍然有 `leaf` 作為 `branch` 其中一個子節點。一旦我們有了 `branch` 的 `Node` 實例，我們可以修改 `leaf` 使其擁有父節點的 `Weak<Node>` 參考。我們對 `leaf` 中 `parent` 欄位的 `RefCell<Weak<Node>>` 使用 `borrow_mut` 方法，然後我們使用 `Rc::downgrade` 函式來從 `branch` 的 `Rc<Node>` 建立一個 `branch` 的 `Weak<Node>` 參考。

當我們再次印出 `leaf` 的父節點，這次我們就會取得 `Some` 變體其內就是 `branch`，現在 `leaf` 可以取得它的父節點了！當我們印出 `leaf`，我們也能避免產生像範例 15-26 那樣最終導致堆疊溢位（stack overflow）的循環，`Weak<Node>` 會印成 `(Weak)`：

```
leaf 的父節點 Some(Node { value: 5, parent: RefCell { value: (Weak) },
children: RefCell { value: [Node { value: 3, parent: RefCell { value: (Weak) },
children: RefCell { value: [] } } ] } })
```

沒有無限的輸出代表此程式碼沒有產生參考循環。我們也能透過呼叫 `Rc::strong_count` 與 `Rc::weak_count` 的數值看出。

視覺化 `strong_count` 與 `weak_count` 的變化

讓我們看看 `Rc<Node>` 實例中 `strong_count` 與 `weak_count` 的數值如何變化，我們建立一個新的內部作用域，並將 `branch` 的產生移入作用域中。這樣我們就能看到 `branch` 建立與離開作用域而釋放時發生了什麼事。如範例 15-29 所示：

檔案名稱：src/main.rs

```

fn main() {
    let leaf = Rc::new(Node {
        value: 3,
        parent: RefCell::new(Weak::new()),
        children: RefCell::new(vec![]),
    });

    println!(
        "leaf 的強參考 = {}、弱參考 = {}",
        Rc::strong_count(&leaf),
        Rc::weak_count(&leaf),
    );

    {
        let branch = Rc::new(Node {
            value: 5,
            parent: RefCell::new(Weak::new()),
            children: RefCell::new(vec![Rc::clone(&leaf)]),
        });

        *leaf.parent.borrow_mut() = Rc::downgrade(&branch);

        println!(
            "branch 的強參考 = {}、弱參考 = {}",
            Rc::strong_count(&branch),
            Rc::weak_count(&branch),
        );

        println!(
            "leaf 的強參考 = {}、弱參考 = {}",
            Rc::strong_count(&leaf),
            Rc::weak_count(&leaf),
        );
    }

    println!("leaf 的父節點 {:?}", leaf.parent.borrow().upgrade());
    println!(
        "leaf 的強參考 = {}、弱參考 = {}",
        Rc::strong_count(&leaf),
        Rc::weak_count(&leaf),
    );
}

```

範例 15-29：在內部作用域建立 `branch` 並觀察強與弱參考計數

在 `leaf` 建立後，它的 `Rc<Node>` 有強計數為 1 與弱計數為 0。在內部作用域中，我們建立了 `branch` 並與 `leaf` 做連結，此時當我們印出計數時，`branch` 的 `Rc<Node>` 會有強計數為 1 與弱計數為 1（因為 `leaf.parent` 透過 `Weak<Node>` 指向 `branch`）。當我們印出 `leaf` 的計數

時，我們會看到它會有強計數為 2，因為 `branch` 現在有個 `leaf` 的 `Rc<Node>` 克隆儲存在 `branch.children`，但弱計數仍為 0。

當內部作用域結束時，`branch` 會離開作用域且 `Rc<Node>` 的強計數會歸零，所以它的 `Node` 就會被釋放。`leaf.parent` 的弱計數 1 與 `Node` 是否被釋放無關，所以我們沒有產生任何記憶體泄漏！

如果我們嘗試在作用域結束後取得 `leaf` 的父節點，我們會再次獲得 `None`。在程式的最後，`leaf` 的 `Rc<Node>` 強計數為 1 且弱計數為 0，因為變數 `leaf` 現在是 `Rc<Node>` 唯一的參考。

所有管理計數與數值釋放都已經實作在 `Rc<T>` 與 `Weak<T>`，它們都有 `Drop` 特徵的實作。在 `Node` 的定義中指定子節點對父節點的關係應為 `Weak<T>` 參考，讓你能夠將父節點與子節點彼此關聯，且不必擔心產生參考循環與記憶體泄漏。

總結

本章節涵蓋了如何使用智慧指標來得到一些不同於 Rust 預設參考所帶來的保障以及取捨。

`Box<T>` 型別有已知大小並能將資料配置到堆積上。`Rc<T>` 型別會追蹤堆積上資料的參考數量，讓該資料能有數個擁有者。`RefCell<T>` 型別具有內部可變性，提供一個外部不可變的型別，但有方法可以改變內部數值，它會在執行時強制檢測借用規則，而非編譯時。

我們也討論了 `Deref` 與 `Drop` 特徵，這些對智慧指標提供了許多功能。我們探討了參考循環可能會導致記憶體泄漏以及如何使用 `Weak<T>` 避免它們。

如果本章節引起你的興趣，讓你想要實作你自己的智慧指標的話，歡迎查閱「[The Rustonomicon](#)」來學習更多實用資訊。

接下來，我們將討論 Rust 的並行性。你還會再學到一些新的智慧指標。

無懼並行

能夠安全高效處理並行程式設計是 Rust 的另一項主要目標。**並行程式設計**（Concurrent programming）會讓程式的不同部分獨立執行，而**平行程式設計**（parallel programming）則是程式的不同部分同時執行。這些隨著電腦越能善用多處理器時也越顯得重要。歷史上，這種程式設計是很困難且容易出錯的，Rust 希望能改善這點。

起初 Rust 團隊認為確保記憶體安全與預防並行問題是兩個分別的問題，要用不同的解決方案。隨著時間過去，團隊發現所有權與型別系統同時是管理記憶體安全**以及**並行問題的強大工具！透過藉助所有權與型別檢查，許多並行錯誤在 Rust 中都是編譯時錯誤而非執行時錯誤。因此，你不用花大量時間嘗試重現編譯時並行錯誤出現時的特定情況，不正確的程式碼會在編譯時就被拒絕，並顯示錯誤解釋問題原因。這樣一來，你就可以在開發時就修正問題，而不用等到可能都部署到生產環境了才發現問題。我們稱呼這個 Rust 的特色為**無懼並行（fearless concurrency）**。無懼並行可以避免你寫出有微妙錯誤的程式碼，並能輕鬆重構，不用擔心產生新的程式錯誤。

注意：出於簡潔考量，我們將把許多問題歸類為**並行**，而不是精確地區分是**並行與 / 或平行**。如果本書是本專注在並行與/或平行的書，我們才會更在意用詞。至於本章節，當我們使用**並行**的詞彙時，請記得這代表 **並行與 / 或平行**。

許多語言對於處理並行問題所提供的解決方案都很有特色。舉例來說，Erlang 有非常優雅的訊息傳遞並行功能，但跨執行緒共享狀態就只有比較隱晦的方法。只提供支援可能解決方案的子集對於高階語言來說是合理的策略，因為高階語言所承諾的效益來自於犧牲一些掌控以換取大量的抽象層面。然而，低階語言則預期會提供在任何給定場合中能有最佳效能的解決方案，而且對硬體的抽象較少。因此 Rust 提供了多種工具來針對適合你的場合與需求將問題定義出來。

本章節中我們會涵蓋這些主題：

- 如何建立執行緒（threads）來同時執行多段程式碼
- **訊息傳遞**（Message-passing）並行提供通道（channels）在執行緒間傳遞訊息
- **共享狀態**（Shared-state）並行提供多執行緒可以存取同一位置的資料
- **Sync** 與 **Send** 特徵擴展 Rust 的並行保障至使用者定義的型別與標準函式庫的型別中

使用執行緒同時執行程式碼

在大部分的現代作業系統中，被執行的程式碼會在**程序（process）**中執行，作業系統會負責同時處理數個程序。在你的程式中，你也可以將各自獨立的部分同時執行。執行這些獨立部分的功能就叫做**執行緒（threads）**。舉例來說，一個網路伺服器可以有數個執行緒來同時回應一個以上的請求。

將程式中的運算拆成數個執行緒可以提升效能，不過這也同時增加了複雜度。因為執行緒可以同時執行，所以無法保證不同執行緒的程式碼執行的順序。這會導致以下問題：

- 競爭條件（Race conditions）：數個執行緒以不一致的順序取得資料或資源
- 死結（Deadlocks）：兩個執行緒彼此都在等待對方，因而讓執行緒無法繼續執行
- 只在特定情形會發生的程式錯誤，並難以重現與穩定修復

Rust 嘗試降低使用執行緒所帶來的負面效果，不過對於多執行緒程式設計還是得格外小心，其所要求的程式結構也與單一執行緒的程式有所不同。

不同程式語言會以不同的方式實作執行緒，許多作業系統都有提供 API 來建立新的執行緒。Rust 標準函式庫使用的是 1:1 的執行緒實作模型，也就是每一個語言產生的執行緒就是一個作業系統的執行緒。有其他 crate 會實作其他種執行緒模型，讓我們能與 1:1 模型之間做取捨。

透過 `spawn` 建立新的執行緒

要建立一個新的執行緒，我們呼叫函式 `thread::spawn` 並傳入一個閉包（我們在第十三章談過閉包），其包含我們想在新執行緒執行的程式碼。範例 16-1 會在主執行緒印出一些文字，並在新執行緒印出其他文字：

檔案名稱：src/main.rs

```

use std::thread;
use std::time::Duration;

fn main() {
    thread::spawn(|| {
        for i in 1..10 {
            println!("數字 {} 出現在產生的執行緒中！", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("數字 {} 出現在主執行緒中！", i);
        thread::sleep(Duration::from_millis(1));
    }
}

```

範例 16-1：建立一個會印出一些字的新執行緒，而主執行緒會印出其他字

注意到當 Rust 程式的主執行緒完成的話，所有執行緒也會被停止，無論它有沒有完成任務。此程式的輸出結果每次可能都會有點不相同，但它會類似以下這樣：

```

數字 1 出現在主執行緒中！
數字 1 出現在產生的執行緒中！
數字 2 出現在主執行緒中！
數字 2 出現在產生的執行緒中！
數字 3 出現在主執行緒中！
數字 3 出現在產生的執行緒中！
數字 4 出現在主執行緒中！
數字 4 出現在產生的執行緒中！
數字 5 出現在產生的執行緒中！

```

`thread::sleep` 的呼叫強制執行緒短時間內停止運作，讓不同的執行緒可以執行。執行緒可能會輪流執行，但並不保證絕對如此，這會依據你的作業系統如何安排執行緒而有所不同。在這一輪中，主執行緒會先顯示，就算程式中是先寫新執行緒的 `println!` 陳述式。而且雖然我們是寫說新執行緒印出 `i` 一直到 9，但它在主執行緒結束前只印到 5。

如果當你執行此程式時只看到主執行緒的結果，或者沒有看到任何交錯的話，你可以嘗試增加數字範圍來增加作業系統切換執行緒的機會。

使用 `join` 等待所有執行緒完成

範例 16-1 的程式碼在主執行緒結束時不只會在大多數的時候提早結束新產生的執行緒，還不能保證執行緒運行的順序，我們甚至無法保證產生的執行緒真的會執行！

透過儲存 `thread::spawn` 回傳的數值為變數，我們可以修正產生的執行緒完全沒有執行或沒有執行完成的問題。`thread::spawn` 的回傳型別為 `JoinHandle`。`JoinHandle` 是個有所有權的數值，當我們對它呼叫 `join` 方法時，它就會等待它的執行緒完成。範例 16-2 顯示了如何使用我們在範例 16-1 中執行緒的 `JoinHandle` 並呼叫 `join` 來確保產生的執行緒會在 `main` 離開之前完成：

檔案名稱：src/main.rs

```
use std::thread;
use std::time::Duration;

fn main() {
    let handle = thread::spawn(|| {
        for i in 1..10 {
            println!("數字 {} 出現在產生的執行緒中！", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("數字 {} 出現在主執行緒中！", i);
        thread::sleep(Duration::from_millis(1));
    }

    handle.join().unwrap();
}
```

範例 16-2：從 `thread::spawn` 儲存 `JoinHandle` 以保障執行緒能執行完成

對其呼叫 `join` 會阻擋當前正在執行的執行緒中直到 `JoinHandle` 的執行緒結束為止。**阻擋**

(Blocking) 一條執行緒代表該執行緒不會繼續運作或離開。因為我們在主執行緒的 `for` 迴圈之後加上了 `join` 的呼叫，範例 16-2 應該會產生類似以下的輸出：

```
數字 1 出現在主執行緒中！
數字 2 出現在主執行緒中！
數字 1 出現在產生的執行緒中！
數字 3 出現在主執行緒中！
數字 2 出現在產生的執行緒中！
數字 4 出現在主執行緒中！
數字 3 出現在產生的執行緒中！
數字 4 出現在產生的執行緒中！
數字 5 出現在產生的執行緒中！
數字 6 出現在產生的執行緒中！
數字 7 出現在產生的執行緒中！
數字 8 出現在產生的執行緒中！
數字 9 出現在產生的執行緒中！
```

兩條執行緒會互相交錯，但是主執行緒這次會因為 `handle.join()` 而等待，直到產生的執行緒完成前都不會結束。

那如果我們如以下這樣將 `handle.join()` 移到 `main` 中的 `for` 迴圈前會發生什麼事呢：

檔案名稱：src/main.rs

```
use std::thread;
use std::time::Duration;

fn main() {
    let handle = thread::spawn(|| {
        for i in 1..10 {
            println!("數字 {} 出現在產生的執行緒中！", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    handle.join().unwrap();

    for i in 1..5 {
        println!("數字 {} 出現在主執行緒中！", i);
        thread::sleep(Duration::from_millis(1));
    }
}
```

主執行緒會等待產生的執行緒完成才會執行它的 `for` 迴圈，所以輸出結果就不會彼此交錯，如下所示：

```
數字 1 出現在產生的執行緒中！
數字 2 出現在產生的執行緒中！
數字 3 出現在產生的執行緒中！
數字 4 出現在產生的執行緒中！
數字 5 出現在產生的執行緒中！
數字 6 出現在產生的執行緒中！
數字 7 出現在產生的執行緒中！
數字 8 出現在產生的執行緒中！
數字 9 出現在產生的執行緒中！
數字 1 出現在主執行緒中！
數字 2 出現在主執行緒中！
數字 3 出現在主執行緒中！
數字 4 出現在主執行緒中！
```

像這樣將 `join` 呼叫置於何處的小細節，會影響你的執行緒會不會同時運行。

透過執行緒使用 `move` 閉包

我們通常會使用 `thread::spawn` 時都會搭配有 `move` 關鍵字的閉包，因為該閉包能獲取周圍環境的數值，轉移那些數值的所有權到另一個執行緒中。在第十三章的「[獲取參考或移動所有權](#)」段落我們討論過閉包如何運用 `move`。現在我們會來專注在 `move` 與 `thread::spawn` 之間如何互動。

在第十三章中，我們提到我們可以在閉包參數列表前使用 `move` 關鍵字來強制閉包取得其從環境獲取數值的所有權。此技巧在建立新的執行緒特別有用，讓我們可以從一個執行緒轉移數值所有權到另一個執行緒。

注意到範例 16-1 中我們傳入 `thread::spawn` 的閉包沒有任何引數，我們在產生的執行緒程式碼內沒有使用主執行緒的任何資料。要在產生的執行緒中使用主執行緒的資料的話，產生的執行緒閉包必須獲取它所需的資料。範例 16-3 嘗試在主執行緒建立一個向量並在產生的執行緒使用它。不過這目前無法執行，你會在稍後知道原因。

檔案名稱：src/main.rs

```
use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(|| {
        println!("這是個向量：{:?}", v);
    });

    handle.join().unwrap();
}
```



範例 16-3：嘗試在其他執行緒使用主執行緒建立的向量

閉包想使用 `v`，所以它得獲取 `v` 並使其成為閉包環境的一部分。因為 `thread::spawn` 會在新的執行緒執行此閉包，我們要能在新的執行緒內存取 `v`。但當我們編譯此範例時，我們會得到以下錯誤：

```

$ cargo run
   Compiling threads v0.1.0 (file:///projects/threads)
error[E0373]: closure may outlive the current function, but it borrows `v`,
which is owned by the current function
  --> src/main.rs:6:32
   |
6 |         let handle = thread::spawn(|| {
   |                                   ^^ may outlive borrowed value `v`
7 |             println!("這是個向量：{:?}", v);
   |                                   - `v` is borrowed here
   |
note: function requires argument type to outlive `'static`
  --> src/main.rs:6:18
   |
6 |         let handle = thread::spawn(|| {
   |         -----^
7 | |             println!("這是個向量：{:?}", v);
8 | |         });
   | |         ^
help: to force the closure to take ownership of `v` (and any other referenced
variables), use the `move` keyword
   |
6 |         let handle = thread::spawn(move || {
   |                                   ++++

```

For more information about this error, try `rustc --explain E0373`.

error: could not compile `threads` due to previous error

Rust 會推斷如何獲取 `v` 而且因為 `println!` 只需要 `v` 的參考，閉包得借用 `v`。不過這會有個問題，Rust 無法知道產生的執行緒會執行多久，所以它無法確定 `v` 的參考是不是永遠有效。

範例 16-4 提供了一個情境讓 `v` 很有可能不再有效：

檔案名稱：src/main.rs

```

use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(|| {
        println!("這是個向量：{:?}", v);
    });

    drop(v); // 喔不！

    handle.join().unwrap();
}

```



範例 16-4：執行緒的閉包嘗試獲取 `v` 的參考，但主執行緒會釋放 `v`

如果 Rust 允許執行此程式碼，產生的執行緒是有可能會置於背景而沒有馬上執行。產生的執行緒內部有 `v` 的參考，但主執行緒會立即釋放 `v`，使用我們在第十五章討論過的 `drop` 函式。然後當產生的執行緒開始執行時，`v` 就不再有效了，所以它的參考也是無效的了。喔不！

要修正範例 16-3 的編譯錯誤，我們可以使用錯誤訊息的建議：

```
help: to force the closure to take ownership of `v` (and any other referenced
variables), use the `move` keyword
6 |         let handle = thread::spawn(move || {
    |                                     +++++
```

透過在閉包前面加上 `move` 關鍵字，我們強制讓閉包取得它所使用數值的所有權，而非任由 Rust 去推斷它是否該借用數值。範例 16-5 修改了範例 16-3 並能夠如期編譯與執行：

檔案名稱：src/main.rs

```
use std::thread;

fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(move || {
        println!("這是個向量：{:?}", v);
    });

    handle.join().unwrap();
}
```

範例 16-5：使用 `move` 關鍵字強制閉包取得它所使用數值的所有權

我們可能會想嘗試用範例 16-4 做的事來修正程式碼，使用 `move` 閉包的同時在主執行緒呼叫 `drop`。但這樣的修正沒有用，因為範例 16-4 想做的事情會因為不同原因而不被允許。如果我們對閉包加上了 `move`，我們將會把 `v` 移入閉包環境，而在主執行緒將無法再對它呼叫 `drop` 了。我們會得到另一個編譯錯誤：

```

$ cargo run
   Compiling threads v0.1.0 (file:///projects/threads)
error[E0382]: use of moved value: `v`
  --> src/main.rs:10:10
   |
4  |     let v = vec![1, 2, 3];
   |         - move occurs because `v` has type `Vec<i32>`, which does not
   |         implement the `Copy` trait
5  |
6  |     let handle = thread::spawn(move || {
   |                                ----- value moved into closure here
7  |         println!("這是個向量：{:?}", v);
   |                                - variable moved due to use in closure
...
10 |     drop(v); // 喔不！
   |         ^ value used here after move

For more information about this error, try `rustc --explain E0382`.
error: could not compile `threads` due to previous error

```

Rust 的所有權規則再次拯救了我們！我們在範例 16-3 會得到錯誤是因為 Rust 是保守的，所以只會為執行緒借用 `v`，這代表主執行緒理論上可能會使產生的執行緒的參考無效化。透過告訴 Rust 將 `v` 的所有權移入產生的執行緒中，我們向 Rust 保證不會在主執行緒用到 `v`。如果我們用相同方式修改範例 16-4 的話，當我們嘗試在主執行緒使用 `v` 的話，我們就違反了所有權規則。`move` 關鍵字會覆蓋 Rust 保守的預設借用行為，且也不允許我們違反所有權規則。

有了對執行緒與執行緒 API 的基本瞭解，讓我們看看我們可以透過執行緒做些什麼。

使用訊息傳遞在執行緒間傳送資料

有一種確保安全並行且漸漸流行起來的方式是**訊息傳遞 (message passing)**，執行緒或 actors 透過傳遞包含資料的訊息給彼此來溝通。此理念源自於 [Go 語言技術文件](#) 中的口號：「別透過共享記憶體來溝通，而是透過溝通來共享記憶體。」

對於訊息傳遞的並行，Rust 的標準函式庫有提供**通道 (channel)** 的實作。通道是一種程式設計的概念，會把資料從一個執行緒傳送到另一個。

你可以把程式設計的通道想像成水流的通道，像是河流或小溪。如果你將橡皮小鴨或船隻放入河流中，它會順流而下到下游。

一個通道會包含兩個部分：發送者 (transmitter) 與接收者 (receiver)。發送者正是你會放置橡皮小鴨到河流中的上游，而接收者則是橡皮小鴨最後漂流到的下游。你程式碼中的一部分會呼叫發送者的方法來傳送你想要傳遞的資料，然後另一部分的程式碼會檢查接收者收到的訊息。當發送者或接收者有一方被釋放掉時，該通道就會被關閉。

我們在此將寫一支程式，它會在一個執行緒中產生數值，傳送給通道，然後另一個執行緒會接收到數值並印出來。我們會使用通道在執行緒間傳送簡單的數值來作為這個功能的解說。一旦你熟悉此技巧後，你可以使用通道讓執行緒間可以互相溝通。像是實作個聊天系統，或是一個利用數個執行緒進行運算，然後將結果傳入一個執行緒統整結果的分散式系統。

首先在範例 16-6，我們會建立個通道但還不會做任何事。注意這樣不會編譯通過因為 Rust 無法知道我們想對通道傳入的數值型別為何。

檔案名稱：src/main.rs

```
use std::sync::mpsc;

fn main() {
    let (tx, rx) = mpsc::channel();
}
```



範例 16-6：建立通道並賦值分別兩個部分給 `tx` 與 `rx`

我們使用 `mpsc::channel` 函式來建立新的通道，`mpsc` 指的是**多重生產者、唯一消費者 (multiple producer, single consumer)**。簡單來說，Rust 標準函式庫實作通道的方式讓通道可以有多個發送端來產生數值，不過只有一個接收端能消耗這些數值。想像有數個溪流匯聚成一條大河流，任何溪流傳送的任何東西最終都會流向河流的下游。我們會先從單一生產者開始，等這個範例能夠執行後我們再來增加數個生產者。

`mpsc::channel` 函式會回傳一個元組，第一個元素是發送者然後第二個元素是接收者。`tx` 與 `rx` 通常分別作為**發送者 (transmitter)** 與**接收者 (receiver)** 的縮寫，所以我們以此作為我們的

變數名稱。我們的 `let` 陳述式使用到了能解構元組的模式我們會在第十八章討論 `let` 陳述式的模式與解構方式。用這樣的方式使用 `let` 能輕鬆取出 `mpsc::channel` 回傳的元組每個部分。

讓我們將發送端移進一個新產生的執行緒並讓它傳送一條字串，這樣產生的執行緒就可以與主執行緒溝通了，如範例 16-7 所示。這就像是在河流上游放了一隻橡皮小鴨，或是從一條執行緒傳送一條聊天訊息給別條執行緒一樣。

檔案名稱：src/main.rs

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let val = String::from("嗨");
        tx.send(val).unwrap();
    });
}
```

範例 16-7：將 `tx` 移入產生的執行緒並傳送「hi」

我們再次使用 `thread::spawn` 來建立新的執行緒並使用 `move` 將 `tx` 移入閉包，讓產生的執行緒擁有 `tx`。產生的執行緒必須要擁有發送者才能夠傳送訊息至通道。發送端有個 `send` 方法可以接受我們想傳遞的數值。`send` 方法會回傳 `Result<T, E>` 型別，所以如果接收端已經被釋放因而沒有任何地方可以傳遞數值的話，傳送的動作就會回傳錯誤。在此例中，我們呼叫 `unwrap` 所以有錯誤時就會直接恐慌。但在實際的應用程式中，我們會更妥善地處理它，你可以回顧第九章來複習如何適當地處理錯誤。

在範例 16-8 我們會在主執行緒中從接收者取得數值。這就像在河流下游取回順流而下的橡皮小鴨，或是像取得一條聊天訊息一樣。

檔案名稱：src/main.rs

```

use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let val = String::from("嗨");
        tx.send(val).unwrap();
    });

    let received = rx.recv().unwrap();
    println!("取得: {}", received);
}

```

範例 16-8：在主執行緒取得數值「hi」並顯示出來

接收者有兩個實用的方法：`recv` 與 `try_recv`。我們使用 `recv` 作為接收（receive）的縮寫，這會阻擋主執行緒的運行並等待直到通道有訊息傳入。一旦有數值傳遞，`recv` 會就以此回傳 `Result<T, E>`。當發送者關閉時，`recv` 會回傳錯誤來通知不會再有任何數值出現了。

`try_recv` 方法則不會阻擋，而是會立即回傳 `Result<T, E>`。如果有數值的話，就會是存有訊息的 `Ok` 數值，如果尚未有任何數值的話，就會是 `Err` 數值。`try_recv` 適用於如果此執行緒在等待訊息的同時有其他事要做的情形。我們可以寫個迴圈來時不時呼叫 `try_recv`，當有數值時處理訊息，不然的話就先做點其他事直到再次檢查為止。

我們出於方便考量在此例使用 `recv`，我們的主執行緒除了等待訊息以外沒有其他事好做，所以阻擋主執行緒是合理的。

當我們執行範例 16-8 的程式碼，我們會看到主執行緒印出的數值：

```
取得：嗨
```

太棒了！

通道與所有權轉移

所有權規則在訊息傳遞中扮演了重要的角色，因為它們可以幫助你寫出安全的並行程式碼。在 Rust 程式中考慮所有權的其中一項好處就是你能在並行程式設計避免錯誤發生。讓我們做個實驗來看通道與所有權如何一起合作來避免問題發生，我們會在 `val` 數值傳送給通道之後嘗試使用其值。請嘗試編譯範例 16-9 的程式碼並看看為何此程式碼不被允許：

檔案名稱：src/main.rs

```

use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let val = String::from("嗨");
        tx.send(val).unwrap();
        println!("val 為 {}", val);
    });

    let received = rx.recv().unwrap();
    println!("取得: {}", received);
}

```



範例 16-9：在我們將 `val` 傳入通道後嘗試使用其值

我們在這裡透過 `tx.send` 將 `val` 傳入通道之後嘗試印出其值。允許這麼做的話會是個壞主意，一旦數值被傳至其他執行緒，該執行緒就可以在我們嘗試再次使用該值之前修改或釋放其值。其他執行緒的修改有機會因為不一致或不存在的資料而導致錯誤或意料之外的結果。不過如果我試著編譯範例 16-9 的程式碼的話，Rust 會給我們一個錯誤：

```

$ cargo run
   Compiling message-passing v0.1.0 (file:///projects/message-passing)
error[E0382]: borrow of moved value: `val`
  --> src/main.rs:10:31
   |
8  |         let val = String::from("嗨");
   |         --- move occurs because `val` has type `String`, which does
not implement the `Copy` trait
9  |         tx.send(val).unwrap();
   |         --- value moved here
10 |         println!("val 為 {}", val);
   |                                ^^^ value borrowed here after move

= note: this error originates in the macro `$crate::format_args_nl` which
comes from the expansion of the macro `println` (in Nightly builds, run with -Z
macro-backtrace for more info)

For more information about this error, try `rustc --explain E0382`.
error: could not compile `message-passing` due to previous error

```

我們的並行錯誤產生了一個編譯時錯誤。`send` 函式會取走其參數的所有權，並當數值移動時，接收端會再取得其所有權。這能阻止我們在傳送數值過後不小心再次使用其值，所有權系統會檢查一切是否符合規則。

傳送多重數值並觀察接收者等待

範例 16-8 的程式碼可以編譯通過並執行，但它並沒有清楚表達出兩個不同的執行緒正透過通道彼此溝通。在範例 16-10 中我們做了些修改來證明範例 16-8 的程式有正確執行，產生的執行緒現在會傳送數個訊息，並在每個訊息間暫停個一秒鐘。

檔案名稱：src/main.rs

```
use std::sync::mpsc;
use std::thread;
use std::time::Duration;

fn main() {
    let (tx, rx) = mpsc::channel();

    thread::spawn(move || {
        let vals = vec![
            String::from("執行緒"),
            String::from("傳來"),
            String::from("的"),
            String::from("嗨"),
        ];

        for val in vals {
            tx.send(val).unwrap();
            thread::sleep(Duration::from_secs(1));
        }
    });

    for received in rx {
        println!("取得：{}", received);
    }
}
```

範例 16-10：傳送數個訊息並在之間暫停片刻

這次產生的執行緒有個字串向量，我們希望能傳送它們到主執行緒中。我們遍歷它們，單獨傳送每個值，然後透過 `Duration` 數值呼叫 `thread::sleep` 來暫停一秒。

在主執行緒中，我們不再顯式呼叫 `recv` 函式，我們改將 `rx` 作為疊代器使用。對每個接收到的數值，我們印出它。當通道關閉時，疊代器就會結束。

當執行範例 16-10 的程式碼，你應該會看到以下輸出，每一行會間隔一秒鐘：

```
取得：執行緒
取得：傳來
取得：的
取得：嗨
```

因為我們在主執行緒中的 `for` 迴圈內沒有任何會暫停或延遲的程式碼，所以我們可以看出主執行緒是在等待產生的執行緒傳送的數值。

透過克隆發送者來建立多重生產者

稍早之前我們提過 `mpsc` 是多重生產者、唯一消費者（multiple producer, single consumer）的縮寫。讓我們來使用 `mpsc` 並擴展範例 16-10 的程式碼來建立數個執行緒，它們都將傳遞數值給同個接收者。為此我們可以克隆發送者，如範例 16-11 所示：

檔案名稱：src/main.rs

```
// --省略--

let (tx, rx) = mpsc::channel();

let tx1 = tx.clone();
thread::spawn(move || {
    let vals = vec![
        String::from("執行緒"),
        String::from("傳來"),
        String::from("的"),
        String::from("嗨"),
    ];

    for val in vals {
        tx1.send(val).unwrap();
        thread::sleep(Duration::from_secs(1));
    }
});

thread::spawn(move || {
    let vals = vec![
        String::from("更多"),
        String::from("給你"),
        String::from("的"),
        String::from("訊息"),
    ];

    for val in vals {
        tx.send(val).unwrap();
        thread::sleep(Duration::from_secs(1));
    }
});

for received in rx {
    println!("取得: {}", received);
}

// --省略--
```

範例 16-11：從多重生產者傳遞數個訊息

這次在我們建立第一個產生的執行緒前，我們會對發送者呼叫 `clone`。這能給我們一個新的發送者，讓我們可以移入第一個產生的執行緒。接著我們將原本的通道發送端移入第二個產生的執行緒中。這樣我們就有了兩條執行緒，每條都能傳送不同的訊息給接收者。

當你執行程式碼時，你的輸出應該會類似以下結果：

取得：執行緒
取得：更多
取得：傳來
取得：給你
取得：的
取得：的
取得：嗨
取得：訊息

依據你的系統你可能會看到數值以不同順序排序。這正是並行程式設計既有趣卻又困難的地方。如果你加上 `thread::sleep` 來實驗，並在不同執行緒給予不同數值的話，就會發現每一輪都會更不確定，每次都會產生不同的輸出結果。

現在我們已經看完通道如何運作，接著讓我們來看看並行的不同方法吧。

共享狀態並行

雖然我們之前說過 Go 語言技術文件中的口號：「別透過共享記憶體來溝通。」而訊息傳遞是個很好的並行處理方式，但它不是唯一的選項。另一種方式就是在多重執行緒間共享資料。

透過共享記憶體來溝通會是什麼樣子呢？除此之外，為何訊息傳遞愛好者不喜歡這種共享記憶體的方式呢？

任何程式語言的通道某方面來說類似於單一所有權，因為一旦你轉移數值給通道，你就不該使用該數值。共享記憶體並行則像多重所有權，數個執行緒可以同時存取同個記憶體位置。如同你在第十五章所見到的，智慧指標讓多重所有權成為可能，但多重所有權會增加複雜度，因為我們會需要管理這些不同的擁有者。Rust 的型別系統與所有權規則大幅地協助了正確管理這些所有權。作為範例就讓我們看看互斥鎖（mutexes），這是共享記憶體中常見的並行原始元件之一。

使用互斥鎖在同時間只允許一條執行緒存取資料

互斥鎖（Mutex）是 *mutual exclusion* 的縮寫，顧名思義互斥鎖在任意時刻只允許一條執行緒可以存取一些資料。要取得互斥鎖中的資料，執行緒必須先透過獲取互斥鎖的鎖（lock）來表示它想要進行存取。鎖是互斥鎖其中一部分的資料結構，用來追蹤當前誰擁有資料的獨佔存取權。因此互斥鎖被描述為會透過鎖定系統守護（guarding）其所持有的資料。

互斥鎖以難以使用著名，因為你必須記住兩個規則：

- 你必須在使用資料前獲取鎖。
- 當你用完互斥鎖守護的資料，你必須解鎖資料，所以其他的執行緒才能獲取鎖。

要用真實世界來比喻互斥鎖的話，想像在會議中有個座談會只有一支麥克風。如果有講者想要發言時，他們需要請求或示意他們想要使用麥克風。當他們取得麥克風時，他們想講多久都沒問題，直到將麥克風遞給下個要求發言的講者。如果講者講完後忘記將麥克風遞給其他人的話，就沒有人有辦法發言。如果麥克風的分享出狀況的話，座談會就無法如期進行！

互斥鎖的管理要正確處理是極為困難的，這也是為何這麼多人傾向於使用通道。然而有了 Rust 的型別系統與所有權規則，你就不會在鎖定與解鎖之間出錯了。

Mutex<T> 的 API

作為使用互斥鎖的範例，讓我們先在單執行緒使用互斥鎖，如範例 16-12 所示：

檔案名稱：src/main.rs

```

use std::sync::Mutex;

fn main() {
    let m = Mutex::new(5);

    {
        let mut num = m.lock().unwrap();
        *num = 6;
    }

    println!("m = {:?}", m);
}

```

範例 16-12：基於簡便考量先用單一執行緒探討 `Mutex<T>` 的 API

就像許多型別一樣，我們使用關聯函式 `new` 建立 `Mutex<T>`。要取得互斥鎖內的資料，我們使用 `lock` 方法來獲取鎖。此呼叫會阻擋當前執行緒做任何事，直到輪到它取得鎖。

如果其他持有鎖的執行緒恐慌的話 `lock` 的呼叫就會失敗。在這樣的情況下，就沒有任何人可以獲得鎖，因此當我們遇到這種情況時，我們選擇 `unwrap` 並讓此執行緒恐慌。

在我們獲取鎖之後，我們在此例可以將回傳的數值取作 `num`，作為內部資料的可變參考。型別系統能確保我們在使用數值 `m` 之前有獲取鎖，`Mutex<i32>` 並不是 `i32`，所以我們必須取得鎖才能使用 `i32` 數值。我們不可能會忘記這麼做，不然型別系統不會讓我們存取內部的 `i32`。

如同你所想像的，`Mutex<T>` 就是個智慧指標。更精確的來說，`lock` 的呼叫會回傳一個智慧指標叫做 `MutexGuard`，這是我們從 `LockResult` 呼叫 `unwrap` 取得的型別。`MutexGuard` 智慧指標有實作 `Deref` 特徵來指向我們的內部資料。此智慧指標也有 `Drop` 的實作，這會在 `MutexGuard` 離開作用域時自動釋放鎖，也就是在內部作用域結尾就會執行此動作。這樣一來，我們就不會忘記釋放鎖，怕互斥鎖會阻擋其他執行緒，因為鎖會自動被釋放。

在釋放鎖之後，我們就能印出互斥鎖的數值並觀察到我們能夠變更內部的 `i32` 為 6。

在數個執行緒間共享 `Mutex<T>`

現在讓我們來透過 `Mutex<T>` 來在數個執行緒間分享數值。我們會建立 10 個執行緒並讓它們都會對一個計數增加 1，讓計數能從 0 加到 10。作為下個例子的範例 16-13 會出現一個編譯錯誤，我們會用此錯誤瞭解如何使用 `Mutex<T>` 以及 Rust 如何協助我們來正確使用它。

檔案名稱：src/main.rs

```

use std::sync::Mutex;
use std::thread;

fn main() {
    let counter = Mutex::new(0);
    let mut handles = vec![];

    for _ in 0..10 {
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();

            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("結果：{}", *counter.lock().unwrap());
}

```



範例 16-13：十個執行緒都會對 `Mutex<T>` 守護的計數增加 1

我們建立個變數 `counter` 並在 `Mutex<T>` 內存有 `i32`，就像我們在範例 16-12 所做的一樣。接著我們透過指定的範圍建立 10 個執行緒。我們使用 `thread::spawn` 讓所有的執行緒都有相同的閉包，此閉包會將計數移入執行緒、呼叫 `lock` 以獲取 `Mutex<T>` 的鎖，然後將互斥鎖內的數值加 1。當有執行緒執行完它的閉包時，`num` 會離開作用域並釋放鎖，讓其他的執行緒可以獲取它。

在主執行緒中，我們要收集所有的執行緒。然後如同我們在範例 16-2 所做的，我們呼叫每個執行緒的 `join` 來確保所有執行緒都有完成。在這時候，主執行緒就能獲取鎖並印出此程式的結果。

我們曾暗示範例不會編譯過，讓我們看看是為何吧！

```

$ cargo run
   Compiling shared-state v0.1.0 (file:///projects/shared-state)
error[E0382]: use of moved value: `counter`
  --> src/main.rs:9:36
   |
5  |         let counter = Mutex::new(0);
   |         ----- move occurs because `counter` has type `Mutex<i32>`, which
does not implement the `Copy` trait
...
9  |         let handle = thread::spawn(move || {
   |                                     ^^^^^^^^ value moved into closure here,
in previous iteration of loop
10 |             let mut num = counter.lock().unwrap();
   |                               ----- use occurs due to use in closure

For more information about this error, try `rustc --explain E0382`.
error: could not compile `shared-state` due to previous error

```

錯誤訊息表示 `counter` 數值在之前的迴圈循環中被移動了，所以 Rust 告訴我們我們無法將 `counter` 鎖的所有權移至數個執行緒中。讓我們用第十五章提到的多重所有權方法來修正此編譯錯誤吧。

多重執行緒中的多重所有權

在第十五章中，我們透過智慧指標 `Rc<T>` 來建立參考計數數值讓該資料可以擁有數個擁有者。讓我們在此也做同樣的動作來看看會發生什麼事。我們會在範例 16-14 將 `Mutex<T>` 封裝進 `Rc<T>` 並在將所有權移至執行緒前克隆 `Rc<T>`。

檔案名稱：src/main.rs

```

use std::rc::Rc;
use std::sync::Mutex;
use std::thread;

fn main() {
    let counter = Rc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Rc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();

            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("結果: {}", *counter.lock().unwrap());
}

```



範例 16-14：嘗試使用 `Rc<T>` 來允許數個執行緒擁有 `Mutex<T>`

再編譯一次的話我們會得到... 不同的錯誤！編譯器真的是教了我們很多事。

```

$ cargo run
   Compiling shared-state v0.1.0 (file:///projects/shared-state)
error[E0277]: `Rc<Mutex<i32>>` cannot be sent between threads safely
  --> src/main.rs:11:36
   |
11 |         let handle = thread::spawn(move || {
   |                                     ^-----
   |                                     |
   |                                     |
   | _____|_____within this
`[closure@src/main.rs:11:36: 11:43]`
   | |
   | |                                     required by a bound introduced by this call
12 | |         let mut num = counter.lock().unwrap();
13 | |
14 | |         *num += 1;
15 | |     });
   | |_____^ `Rc<Mutex<i32>>` cannot be sent between threads safely
   |
   = help: within `[closure@src/main.rs:11:36: 11:43]`, the trait `Send` is not
implemented for `Rc<Mutex<i32>>`
note: required because it's used within this closure
  --> src/main.rs:11:36
   |
11 |         let handle = thread::spawn(move || {
   |                                     ^^^^^^^
note: required by a bound in `spawn`

For more information about this error, try `rustc --explain E0277`.
error: couldnot compile `shared-state` due to previous error

```

哇，這個錯誤訊息的內容真多！這是我們需要注意到的部分：``Rc<Mutex<i32>>` cannot be sent between threads safely`。編譯器也告訴了我們原因：`the trait `Send` is not implemented for `Rc<Mutex<i32>>``。我們會在下一個段落討論 `Send`，這是其中一種確保我們在執行緒中所使用的型別可以用於並行場合的特徵。

不幸的是 `Rc<T>` 無法安全地跨執行緒分享。當 `Rc<T>` 管理參考計數時，它會在每個 `clone` 的呼叫增加計數，並在每個克隆釋放時減少計數。但是它沒有使用任何並行原始元件來確保計數的改變不會被其他執行緒中斷。這樣的計數可能會導致微妙的程式錯誤，像是記憶體洩漏或是在數值釋放時嘗試使用其值。我們需要一個型別和 `Rc<T>` 一模一樣，但是其參考計數在執行緒間是安全的。

原子參考計數 Arc<T>

幸運的是 `Arc<T>` 正是一個類似 `Rc<T>` 且能安全用在並行場合的型別。字母 *A* 指的是原子性 (atomic) 代表這是個原子性參考的計數型別。原子型別是另一種我們不會在此討論的並行原始元件，你可以查閱標準函式庫的 `std::sync::atomic` 技術文件以瞭解更多詳情。在此你只需要知道原子型別和原始型別類似，但它們可以安全在執行緒間共享。

你可能會好奇為何原始型別不是原子性的，以及為何標準函式庫的型別預設不使用 `Arc<T>` 來實作。原因是因為執行緒安全意味著效能開銷，你會希望在你真的需要時才買單。如果你只是在單一執行緒對數值做運算的話，你的程式碼就不必強制具有原子性的保障並能執行地更快。

讓我們回到我們的範例：`Arc<T>` 與 `Rc<T>` 具有相同的 API，所以我們透過更改 `use` 這行、`new` 的呼叫以及 `clone` 的呼叫來修正我們程式，。範例 16-15 的程式碼最終將能夠編譯並執行：

檔案名稱：src/main.rs

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();

            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("結果：{}", *counter.lock().unwrap());
}
```

範例 16-15：使用 `Arc<T>` 封裝 `Mutex<T>` 來在數個執行緒間分享所有權

此程式碼會印出以下結果：

```
結果：10
```

我們辦到了！我們從 0 數到了 10，雖然看起來不是很令人印象深刻，但這的確教會了我們很多有關 `Mutex<T>` 與執行緒安全的知識。你也可以使用此程式結構來做更多複雜的運算，而不只是數數而已。使用此策略，你可以將運算拆成數個獨立部分，將它們分配給執行緒，然後使用 `Mutex<T>` 來讓每個執行緒更新該部分的結果。

如果你只是想做單純的數值運算，其實標準函式庫提供的 `std::sync::atomic` 模組會比 `Mutex<T>` 型別來得簡單。這些型別對原生型別提供安全、並行且原子性的順取。我們在此例對原

生型別使用 `Mutex<T>`，是因為我們想解釋 `Mutex<T>` 是如何運作的。

`RefCell<T>/Rc<T>` 與 `Mutex<T>/Arc<T>` 之間的相似度

你可能已經注意到 `counter` 是不可變的，但我們卻可以取得數值其內部的可變參考，這代表 `Mutex<T>` 有提供內部可變性，就像 `Cell` 家族一樣。我們在第十五章也以相同的方式使用 `RefCell<T>` 來讓我們能改變 `Rc<T>` 內部的數值，而在此我們使用 `Mutex<T>` 改變 `Arc<T>` 內部的內容。

另一個值得注意的細節是當你使用 `Mutex<T>` 時，Rust 無法避免所有種類的邏輯錯誤。回憶一下第十五章使用 `Rc<T>` 時會有可能產生參考循環的風險，兩個 `Rc<T>` 數值可能會彼此參考，造成記憶體洩漏。同樣地，`Mutex<T>` 有產生死結（deadlocks）的風險。這會發生在當有個動作需要鎖定兩個資源，而有兩個執行緒分別擁有其中一個鎖，導致它們永遠都在等待彼此。如果你對此有興趣的話，歡迎嘗試建立一個有死結的 Rust 程式，然後研究看看任何語言中避免的互斥鎖死結的策略，並嘗試實作它們在 Rust 中。標準函式庫中 `Mutex<T>` 與 `MutexGuard` 的 API 技術文件可以提供些實用資訊。

接下來在本章結尾我們會來討論 `Send` 與 `Sync` 特徵以及我們如何在自訂型別中使用它們。

可延展的並行與 `Sync` 及 `Send` 特徵

有一個有趣的點是 Rust 語言提供的並行功能並沒有很多。本章節討論到的並行功能幾乎都來自於標準函式庫，並不是語言本身。你能處理並行的選項並不限於語言或標準函式庫，你可以寫出你自己的並行功能或使用其他人提供的。

然而，還有有兩個並行概念深植於語言中，那就是 `std::marker` 中的 `Sync` 與 `Send` 特徵。

透過 `Send` 來允許所有權能在執行緒間轉移

`Send` 標記特徵 (marker traits) 指定有實作 `Send` 特徵的型別才能將其數值的所有權在執行緒間轉移。幾乎所有的 Rust 型別都有 `Send`，但有些例外。這包含 `Rc<T>`，此型別沒有 `Send` 是因為如果你克隆了 `Rc<T>` 數值並嘗試轉移克隆的所有權到其他執行緒，會有兩條執行緒可能同時更新參考計數。基於此原因，`Rc<T>` 是用於當你不需要付出執行緒安全效能開銷時而在單一執行緒使用的情況。

因此 Rust 的型別系統與特徵界限確保你無法意外不安全地傳送 `Rc<T>` 數值到其他執行緒。當我們嘗試範例 16-14 時，我們就會得到錯誤 `the trait Send is not implemented for Rc<Mutex<i32>>`。當我們切換成有實作 `Send` 的 `Arc<T>` 的話，程式碼就能編譯通過。

任何由具有 `Send` 的型別所組成的型別也都會自動標記為 `Send`。幾乎所有原始型別都是 `Send`，除了我們將在第十九章提及的裸指標 (raw pointers)。

透過 `Sync` 來允許多重執行緒存取

`Sync` 標記特徵指定有實作 `Sync` 的型別都能安全從多個執行緒來參考。換句話說，對於任何型別 `T`，如果 `&T` (對 `T` 的不可變參考) 有 `Send` 的話，`T` 就是 `Sync` 的，這代表參考可以安全地傳給其他執行緒。與 `Send` 類似，原始型別都是 `Sync`，所以由具有 `Sync` 的型別所組成的型別也都有 `Sync`。

智慧指標 `Rc<T>` 沒有 `Sync` 的原因和沒有 `Send` 的原因一樣。`RefCell<T>` 型別 (我們在第十五章提過) 與其 `Cell<T>` 也都沒有 `Sync`。`RefCell<T>` 在執行時的借用檢查實作沒有執行緒安全。智慧指標 `Mutex<T>` 才有 `Sync` 並能像你在「[在數個執行緒間共享 `Mutex<T>`](#)」段落看到的那樣用來在多個執行緒間分享存取。

手動實作 `Send` 與 `Sync` 是不安全的

因為由具有 `Send` 與 `Sync` 的型別組成的型別自動就會有 `Send` 與 `Sync`，我們不需要親自實作這些特徵。至於標記特徵，它們甚至沒有任何方法需要實作。它們只是用於強制確保並行相關的不

變性。

要手動實作這些特徵會需要實作不安全（unsafe）的 Rust 程式碼。我們會在第十九章討論如何使用不安全的 Rust 程式碼，現在最重要的資訊是要從不具有 `Send` 與 `Sync` 的元件來組成新的並行型別需要格外小心才能確保其安全保障。「[The Rustonomicon](#)」有更多關於這些保障與如何維持它們的資訊。

總結

這不會是你在本書中最後一次看到並行程式碼，第二十章的專案將會在更實際的場合中使用本章節的概念，而非這裡討論的簡單範例。

如之前提過的，因為 Rust 語言本身很少處理並行的部分，許多並行解決方案都實作成 crate。這些 crate 通常發展的比標準函式庫還快，所以別忘了到線上尋找目前最先進的 crate 來在多執行緒場合中使用喔。

Rust 標準函式庫提供訊息傳遞的通道與智慧指標，像是 `Mutex<T>` 與 `Arc<T>`，能夠在並行環境中安全使用。型別系統與借用檢查器中確保使用這些解決方案的程式碼不會發生資料競爭或是無效參考。一旦你讓你的程式碼能編譯通過後，你可以放心地認定它會開開心心地在多執行緒中執行，並且不會發生任何在其他語言中常見且難以追蹤的程式錯誤。並行程式設計就不再是個令人害怕的概念，無畏無懼地開發並行程式吧！

接下來，我們要討論當你的 Rust 程式成長時，定義出問題並組織解決辦法的慣用方案。除此之外，我們也將討論 Rust 有哪些與物件導向程式設計（object-oriented programming）類似的概念。

Rust 的物件導向程式設計特色

物件導向程式設計（Object-oriented programming，OOP）是一種模組化程式的方式，物件這種程式設計的概念始於 1960 年的 Simula。這些物件影響了 Alan Kay 的程式設計架構中物件彼此之間訊息的傳遞。他在 1967 年提出了**物件導向程式設計**來描述此架構。對於 OOP 的定義有許多種描述，有些定義會將 Rust 歸類為屬於物件導向的，而有些則不會。在本章節中，我們會探討特定常視為是物件導向的特色並看看這些特色如何轉換成慣用的 Rust 程式碼。然後我們會向你展示如何在 Rust 中實作物件導向設計模式，並討論這麼做與利用 Rust 自身的優勢實現的版本有何取捨差別。

物件導向語言的特色

對於一個被視為物件導向的語言該有哪些功能，在程式設計語言社群中並沒有達成共識。Rust 受到許多程式設計理念影響，這當然包括 OOP。舉例來說，我們在第十三章探討了源自於函式語言的特性。同樣地，OOP 語言有一些特定常見特色，諸如物件、封裝（encapsulation）與繼承（inheritance）。讓我們看看這些特色分別是什麼意思以及 Rust 有沒有支援。

物件包含資料與行為

由 Erich Gamma、Richard Helm、Ralph Johnson 與 John Vlissides（Addison-Wesley Professional，1994）所寫的書《Design Patterns: Elements of Reusable Object-Oriented Software》俗稱為「The Gang of Four」，這是本物件導向設計模式的目錄。它是這樣定義 OOP 的：

物件導向程式由物件所組成。**物件**會包裝資料以及運作在資料上的行為。此行為常稱為**方法（methods）**或**操作（operations）**。

在此定義下，Rust 是物件導向的，結構體與列舉擁有資料，而 `impl` 區塊對結構體與列舉提供方法。就算有方法的結構體與列舉不會被稱為**物件**，依據 Gang of Four 對物件的定義，它們還是有提供相同的功能。

隱藏實作細節的封裝

另外一個常和 OOP 相關的概念就是**封裝（encapsulation）**，這指的是物件的實作細節不會讓使用物件的程式碼取得。因此要與該物件互動的方式是透過它的公開 API，使用物件的程式碼不該有辦法觸及物件內部並直接變更資料的行為。這讓程式設計師能變更並重構物件內部，無需擔心要變更使用物件的程式碼。

我們在第七章討論過如何控制封裝，我們可以使用 `pub` 關鍵字來決定程式中的哪些模組、型別、函式與方法要公開出來，且預設一切都是私有的。舉例來說，我們可以定義個結構體 `AveragedCollection` 並有個欄位包含一個 `i32` 數值的向量。此結構體還有個欄位包含向量數值的平均值，讓我們不必在每次呼叫時都得重新計算平均值。換句話說，`AveragedCollection` 會為我們快取計算出的平均值。範例 17-1 展示了結構體 `AveragedCollection` 的定義：

檔案名稱：`src/lib.rs`

```
pub struct AveragedCollection {
    list: Vec<i32>,
    average: f64,
}
```

範例 17-1：結構體 `AveragedCollection` 有個整數列表與集合中的項目平均值 `collection`

此結構體有 `pub` 標記所以其他程式碼可以使用它，但結構體內部的欄位是私有的。這在此例中是很重要的，因為我們希望在有數值加入或移出列表時，平均值也能更新。我們會實作結構體的 `add`、`remove` 與 `average` 方法來達成，如範例 17-2 所示：

檔案名稱：src/lib.rs

```
impl AveragedCollection {
    pub fn add(&mut self, value: i32) {
        self.list.push(value);
        self.update_average();
    }

    pub fn remove(&mut self) -> Option<i32> {
        let result = self.list.pop();
        match result {
            Some(value) => {
                self.update_average();
                Some(value)
            }
            None => None,
        }
    }

    pub fn average(&self) -> f64 {
        self.average
    }

    fn update_average(&mut self) {
        let total: i32 = self.list.iter().sum();
        self.average = total as f64 / self.list.len() as f64;
    }
}
```

範例 17-2：對 `AveragedCollection` 實作公開方法 `add`、`remove` 與 `average`

公開的方法 `add`、`remove` 與 `average` 是存取或修改 `AveragedCollection` 實例資料的唯一方法。當有個項目透過 `add` 方法加入或透過 `remove` 方法移出 `list` 中時，每個方法會同時呼叫 `update_average` 方法來更新 `average` 欄位。

我們讓 `list` 與 `average` 欄位維持私有，所以外部的程式碼不可能直接新增或移除 `list` 欄位的項目。不然的話，`average` 欄位可能就無法與變更的 `list` 同步了。`average` 方法會回傳

`average` 欄位的數值，讓外部程式碼能夠讀取 `average` 但不會修改它。

由於我們封裝了 `AveragedCollection` 結構體的實作細節，我們可以在未來輕鬆變更像是資料結構等內部細節。舉例來說，我們可以用 `HashSet<i32>` 來替換 `list` 欄位的 `Vec<i32>`。只要 `add`、`remove` 與 `average` 的公開方法簽名維持一樣，使用到 `AveragedCollection` 的程式碼就不需要改變。如果我們讓 `list` 公開的話，情況可能就不相同了，`HashSet<i32>` 與 `Vec<i32>` 有不同的方法來新增和移除項目，所以外部的程式碼如果會直接修改 `list` 的話，可能會需要做些改變。

如果封裝是物件導向的必備條件的話，Rust 也符合此條件。對程式碼中不同部分使用 `pub` 可以封裝實作細節。

作為型別系統與程式碼共享來繼承

繼承 (Inheritance) 是指一個物件可以繼承其他物件定義的機制，使其可以獲取繼承物件的資料與行為，不必再定義一次。

如果一個語言一定要有繼承才算物件導向語言的話，那麼 Rust 就不是。在定義結構體時我們無法繼承父結構體欄位的方法實作，除非使用巨集。

然而如果你在程式設計時常常用到繼承的話，依據你想使用繼承的原因，Rust 還是有提供其他方案。

你想選擇繼承通常會有兩個主要原因。第一個是想能重複使用程式碼，你可以定義一個型別的特定行為，然後繼承讓你可以在不同的型別重複使用該實作。為此你可以使用預設的特徵方法實作來分享 Rust 程式碼，你在範例 10-14 就有看到我們在 `Summary` 特徵加上的預設 `summarize` 方法實作。任何有實作 `Summary` 特徵的型別都不必加上更多程式碼就能有 `summarize` 可以呼叫。這就類似於父類型 (class) 實作的方法可以在繼承的子類型擁有該方法實作。我們也可以在實作 `Summary` 特徵時，覆寫 `summarize` 方法的預設實作，這就類似於子類型覆寫父類型的方法實作。

另一個想使用繼承的原因與型別系統有關，讓子類型可以視為父類型來使用。這也稱為**多型** (**polymorphism**)，代表要是數個物件有共享特定特性的話，你可以在執行時彼此替換使用。

多型

對許多人來說，多型就是繼承的代名詞。不過這其實是個更通用的概念，用來指程式碼可適用於多種型別資料。而對繼承來說，這些型別通常都是子類型。

Rust 則是使用泛型來抽象化不同可能的型別，並以特徵界限來加強約束這些型別必須提供的內容。這有時會稱為**限定的參數多型** (**bounded parametric polymorphism**)。

近年來像繼承這種程式設計的解決方案在許多程式設計語言中都漸漸失寵了，因為這經常有分享不必要程式碼的風險。子類型不應該永遠分享其父類型的所有特性，但繼承會這樣做。這會讓程式的設計較不具有彈性。這還可能產生不具意義或導致錯誤的子類型方法呼叫，因為該方法不適用於子類型。除此之外，有些語言只會允許單一繼承，也就是一個子類型只能繼承一個類別，進一步限制了程式設計的彈性。

基於這些原因，Rust 採取了不同的方案，使用特徵物件（trait objects）而非繼承。讓我們看看 Rust 的特徵物件如何達成多型。

允許不同型別數值的特徵物件

在第八章中，我們提及向量其中一項限制是它儲存元素只能有一種型別。我們在範例 8-9 提出一個替代方案，那就是我們定義 `SpreadsheetCell` 列舉且其變體能存有整數、浮點數與文字。這讓我們可以對每個元素儲存不同的型別，且向量仍能代表元素的集合。當我們的可變換的項目有固定的型別集合，而且我們在編譯程式碼時就知道的話，這的確是完美的解決方案。

然而，有時我們會希望函式庫的使用者能夠在特定的情形下擴展型別的集合。為了展示我們如何達成，我們來建立個圖形使用者介面（graphical user interface，GUI）工具範例來遍歷一個項目列表，呼叫其內每個項目的 `draw` 方法將其顯示在螢幕上，這是 GUI 工具常見的技巧。我們會建立個函式庫 crate 叫做 `gui`，這會包含 GUI 函式庫的結構體。此 crate 可能會包含一些人們會使用到的型別，像是 `Button` 或 `TextField`。除此之外，`gui` 使用者也能夠建立他們自己的型別來顯示出來。舉例來說，有些開發者可以加上 `Image` 而其他人可能會加上 `SelectBox`。

我們在此例中不會實作出整個 GUI 函式庫，但會展示各個元件是怎麼組合起來的。在寫函式庫時，我們無法知道並定義開發者想建立出來的所有型別。但我們知道 `gui` 需要追蹤許多不同型別的數值，且它需要能夠對這些不同的型別數值呼叫 `draw` 方法。它不需要知道當我們呼叫 `draw` 方法時實際發生了什麼事，只需要知道該數值有我們可以呼叫的方法。

在有繼承的語言中，我們可能會定義一個類型（class）叫做 `Component` 且其有個方法叫做 `draw`。其他的類型像是 `Button`、`Image` 和 `SelectBox` 等等，可以繼承 `Component` 以取得 `draw` 方法。它們可以覆寫 `draw` 方法來定義它們自己的自訂行為，但是整個框架能將所有型別視為像是 `Component` 實例來對待，並對它們呼叫 `draw`。但由於 Rust 並沒有繼承，我們需要其他方式來組織 `gui` 函式庫，好讓使用者可以透過新的型別來擴展它。

定義共同行為的特徵

要定義我們希望 `gui` 能擁有的行為，我們定義一個特徵叫做 `Draw` 並有個方法叫做 `draw`。然後我們可以定義一個接收特徵物件（trait object）的向量。一個特徵物件會指向有實作指定特徵的型別以及一個在執行時尋找該型別方法的尋找表（look up table）。要建立特徵物件，我們指定一些指標，像是參考 `&` 或者智慧指標 `Box<T>`，然後加上 `dyn` 關鍵字與指定的相關特徵。（我們會在第十九章的「動態大小型別與 `Sized` 特徵」段落討論特徵物件必須使用指標的原因）我們可以對泛型或實際型別使用特徵物件。當我們使用特徵物件時，Rust 的型別系統會確保在編譯時該段落使用到的任何數值都有實作特徵物件的特徵。於是我們就不必在編譯時知道所有可能的型別。

我們提到在 Rust 中，我們避免將結構體和列舉稱為「物件」，來與其他語言的物件做區別。在結構體或列舉中，結構體欄位中的資料與 `impl` 區塊的行為是分開來的。在其他語言中，資料與行為會組合成一個概念，也就是所謂的物件。然而特徵物件才比較像是其他語言中的物件，因為這才會將資料與行為組合起來。但特徵物件與傳統物件不同的地方在於，我們無法向特徵物件新增資料。特徵物件不像其他語言的物件那麼通用，它們是特別用於對共同行為產生的抽象概念。

範例 17-3 定義了一個特徵叫做 `Draw` 以及一個方法叫做 `draw`：

檔案名稱：src/lib.rs

```
pub trait Draw {  
    fn draw(&self);  
}
```

範例 17-3：Draw 特徵的定義

此語法和我們在第十章介紹過的特徵定義方式相同。接下來才是新語法用到的地方，範例 17-4 定義了一個結構體叫做 `Screen` 並持有個向量叫做 `components`。此向量的型別為 `Box<dyn Draw>`，這是一個特徵物件，這代表 `Box` 內的任何型別都得有實作 `Draw` 特徵。

檔案名稱：src/lib.rs

```
pub struct Screen {  
    pub components: Vec<Box<dyn Draw>>,  
}
```

範例 17-4：定義結構體 `Screen` 且有個 `components` 欄位來持有一個實作 `Draw` 特徵的特徵物件向量

在 `Screen` 結構體中，我們定義了一個方法叫做 `run` 來對其 `components` 呼叫 `draw` 方法，如範例 17-5 所示：

檔案名稱：src/lib.rs

```
impl Screen {  
    pub fn run(&self) {  
        for component in self.components.iter() {  
            component.draw();  
        }  
    }  
}
```

範例 17-5：Screen 的方法 `run` 會呼叫每個 `component` 的 `draw` 方法

這與定義一個結構體並使用附有特徵界限的泛型型別參數的方式不相同。泛型型別參數一次只能替換成一個實際型別，特徵物件則是在執行時允許數個實際型別能填入特徵物件中。舉例來說，我們可以使用泛型型別與特徵界限來定義 `Screen`，如範例 17-6 所示：

檔案名稱：src/lib.rs

```

pub struct Screen<T: Draw> {
    pub components: Vec<T>,
}

impl<T> Screen<T>
where
    T: Draw,
{
    pub fn run(&self) {
        for component in self.components.iter() {
            component.draw();
        }
    }
}

```

範例 17-6：Screen 結構體的另種實作方式，它的方法 run 則使用泛型與特徵界限

這樣我們會限制 Screen 實例必須擁有一串全是 Button 型別或全是 TextField 型別的列表。如果你只會有同型別的集合，使用泛型與特徵界限的確是比較合適的，因為其定義就會在編譯時單型化為使用實際型別。

另一方面，透過使用特徵物件的方法，Screen 實例就能有個同時包含 Box<Button> 與 Box<TextField> 的 Vec<T>。讓我們看看這如何辦到的，然後我們會討論其對執行時效能的影響。

實作特徵

現在我們來加上一些有實作 Draw 特徵的型別。我們會提供 Button 型別。再次重申 GUI 函式庫的實際實作超出了本書的範疇，所以 draw 的本體不會有任何有意義的實作。為了想像該實作會像是什麼，Button 型別可能會有欄位 width、height 與 label，如範例 17-7 所示：

檔案名稱：src/lib.rs

```

pub struct Button {
    pub width: u32,
    pub height: u32,
    pub label: String,
}

impl Draw for Button {
    fn draw(&self) {
        // 實際畫出按鈕的程式碼
    }
}

```

範例 17-7：結構體 Button 實作了 Draw 特徵

在 `Button` 中的 `width`、`height` 與 `label` 欄位會與其他元件不同，像是 `TextField` 可能就會有前面所有的欄位在加上 `placeholder` 欄位。每個我們想在螢幕上顯示的型別都會實作 `Draw` 特徵，但在 `draw` 方法會使用不同程式碼來定義如何印出該特定型別，像是這裡的 `Button` 型別（不包含實際 GUI 程式碼）。舉例來說，`Button` 可能會有額外的 `impl` 區塊來包含使用者點擊按鈕時該如何反應的方法。這種方法就不適用於 `TextField`。

如果有人想用我們的函式庫來實作個 `SelectBox` 結構體並擁有 `width`、`height` 與 `options` 欄位的話，他們也可以對 `SelectBox` 實作 `Draw` 特徵，如範例 17-8 所示：

檔案名稱：src/main.rs

```
use gui::Draw;

struct SelectBox {
    width: u32,
    height: u32,
    options: Vec<String>,
}

impl Draw for SelectBox {
    fn draw(&self) {
        // 實際畫出選擇框的程式碼
    }
}
```

範例 17-8：別的 crate 使用 `gui` 來對 `SelectBox` 結構體實作 `Draw` 特徵

我們的函式庫使用者現在可以在他們的 `main` 建立個 `Screen` 實例。在 `Screen` 實例中，他們可以透過將 `SelectBox` 和 `Button` 放入 `Box<T>` 來成為特徵物件並加入元件中。他們接著就可以對 `Screen` 實例呼叫 `run` 方法，這會呼叫每個元件的 `draw` 方法。如範例 17-9 所示：

檔案名稱：src/main.rs

```

use gui::{Button, Screen};

fn main() {
    let screen = Screen {
        components: vec![
            Box::new(SelectBox {
                width: 75,
                height: 10,
                options: vec![
                    String::from("Yes"),
                    String::from("Maybe"),
                    String::from("No"),
                ],
            }),
            Box::new(Button {
                width: 50,
                height: 10,
                label: String::from("OK"),
            }),
        ],
    };

    screen.run();
}

```

範例 17-9：使用特徵物件來儲存實作相同特徵的不同型別數值

我們在寫函式庫時，我們並不知道有人會想要新增 `SelectBox` 型別，但我們的 `Screen` 實作能夠運用新的型別並顯示出來，因為 `SelectBox` 有實作 `Draw` 特徵，這代表它就有實作 `draw` 方法。

這種只在意數值回應的訊息而非數值實際型別的概念，類似於動態型別語言中**鴨子型別**（duck typing）的概念。如果它走起來像隻鴨子、叫起來像隻鴨子，那它必定是隻鴨子！在範例 17-5 中 `Screen` 的 `run` 實作不需要知道每個元件的實際型別為何。它不會檢查一個元件是 `Button` 還是 `SelectBox` 實例，它只會呼叫元件的 `draw` 方法。透過指定 `Box<dyn Draw>` 來作為 `components` 向量中的數值型別，我們定義 `Screen` 需要我們能夠呼叫 `draw` 方法的數值。

我們使用特徵物件與 Rust 型別系統能寫出類似鴨子型別的程式碼，這樣的優勢在於我們在執行時永遠不必檢查一個數值有沒有實作特定方法，或擔心我們會不會呼叫了一個沒有實作該方法的數值而產生錯誤。如果數值沒有實作特徵物件要求的特徵的話，Rust 不會編譯通過我們的程式碼。

舉例來說，範例 17-10 展示了要是我們嘗試使用 `String` 作為元件來建立 `Screen` 的話會發生什麼事：

檔案名稱：src/main.rs

```
use gui::Screen;

fn main() {
    let screen = Screen {
        components: vec![Box::new(String::from("嗨"))],
    };

    screen.run();
}
```



範例 17-10：嘗試使用沒有實作特徵物件的特徵的型別

我們會因為 `String` 沒有實作 `Draw` 特徵而得到錯誤：

```
$ cargo run
   Compiling gui v0.1.0 (file:///projects/gui)
error[E0277]: the trait bound `String: Draw` is not satisfied
--> src/main.rs:5:26
   |
5  |         components: vec![Box::new(String::from("嗨"))],
   |                                     ^^^^^^^^^^^^^^^^^^^^^ the trait `Draw` is
not implemented for `String`
   |
   = help: the trait `Draw` is implemented for `Button`
   = note: required for the cast from `String` to the object type `dyn Draw`

For more information about this error, try `rustc --explain E0277`.
error: could not compile `gui` due to previous error
```

此錯誤讓我們知道要麼我們傳遞了不希望傳給 `Screen` 的型別所以應該要傳遞其他型別，要麼我們應該要對 `String` 實作 `Draw`，這樣 `Screen` 才能對其呼叫 `draw`。

特徵物件執行動態調度

回想一下第十章的「[使用泛型的程式碼效能](#)」段落我們討論過，當我們對泛型使用閉包時，編譯器會執行單型化（monomorphization）的過程。編譯器會在我們對每個用泛型型別參數取代的實際型別產生非泛型的函式與方法實作。單型化產生程式碼的動作會稱為「靜態調度（static dispatch）」，這代表編譯器在編譯時知道我們呼叫的方法為何。與其相反的則是**動態調度（dynamic dispatch）**，這種方式時編譯器在編譯時無法知道你呼叫的方法為何。在動態調度的情況下，編譯器會產生在執行時能夠確定會呼叫何種方法的程式碼。

當我們使用特徵物件時，Rust 必須使用動態調度。編譯器無法知道使用特徵物件的程式碼會使用到的所有型別為何，所以它會不知道該呼叫哪個型別的哪個實作方法。取而代之的是，Rust 在執行時會使用特徵物件內部的指標來知道該呼叫哪個方法。這樣尋找的動作會產生靜態調度所沒有的執行時開銷。動態調度也讓編譯器無法選擇內聯（inline）方法的程式碼，這樣會因而阻止一些優

化。不過我們的確對範例 17-5 的程式碼增加了額外的彈性，並能夠支援範例 17-9，所以這是個權衡取捨。

實作物件導向設計模式

狀態模式 (state pattern) 是種物件導向設計模式。此模式的關鍵在於我們會定義一個數值擁有些內部狀態，以**狀態物件** (state objects) 呈現，然後數值的行為會依據內部狀態而有所改變。我們將用一個部落格文章結構體作為範例，讓它持有個狀態能在「草稿」、「審核」、「發佈」間轉換，成為狀態物件。

狀態物件會分享功能，當然在 Rust 中我們使用結構體與特徵，而不是使用物件與繼承。每個狀態物件負責本身的行為並監測何時要改變成其他狀態。持有狀態物件的數值不會知道狀態中不同的行為，或是何時要轉換狀態。

使用狀態模式的優勢在於當程式的業務需求改變時，我們不需要改變持有狀態的數值或使用其數值的程式碼。我們只需要變更其中一個狀態物件的程式碼來改變其規則，或者新增更多狀態物件。

我們先用比較傳統的物件導向方式實作狀態模式，然後我們會再來改成比較符合 Rust 的形式。讓我們來用狀態模式一步步實作部落格文章流程吧：

部落格最終的功能會長得像這樣：

1. 部落格文章從空白草稿開始。
2. 當草稿完成時，請求審核文章。
3. 當文章通過時，它就會被發佈。
4. 只有發佈的部落格文章內容會顯示出來，所以沒被通過的文章不會被意外顯示出來。

其他任何對文章的修改不會有任何影響。舉例來說，如果我們嘗試在請求審核一個文章前，通過其他部落格文章草稿的話，該文章應維持未發佈的狀態。

此範例顯示了此工作流程的程式碼形式，這是個會用到我們等等會實作的函式庫 `crate blog` API 的範例。這目前還無法編譯，因為我們還沒有實作 `blog`。

檔案名稱：`src/main.rs`

```
use blog::Post;

fn main() {
    let mut post = Post::new();

    post.add_text("我今天午餐吃了沙拉");
    assert_eq!("", post.content());

    post.request_review();
    assert_eq!("", post.content());

    post.approve();
    assert_eq!("我今天午餐吃了沙拉", post.content());
}
```



範例 17-11：展示我們所希望的 `blog` crate 預期行為的程式碼

我們想要讓使用者能透過 `Post::new` 來建立新的部落格文章草稿，然後我們希望能對部落格文章加入文字。如果我們想在通過前立即取得文章內容的話，我們什麼都不會看到，因為該文章還只是個草稿。我們加入的 `assert_eq!` 在此只是作為解釋目的。更好地做法是寫個判定部落格文章草稿是否會從 `content` 方法回傳空字串的單元測試，不過我們在此例不會寫任何測試。

接著，我們想要請求文章審核，且我們希望在等待審核時 `content` 仍是回傳空字串。當文章通過時，它就會被發佈，代表當 `content` 呼叫時，文章中的文字就會回傳。

注意到我們要使用此 crate 時只會接觸到一個型別 `Post`。此型別會使用狀態模式，並持有個數值能包含三種狀態物件其中之一，來代表文章狀態可以是擬稿中、等待審核或已發佈。變更狀態由 `Post` 型別內部管理。狀態依據函式庫使用者對 `Post` 實例呼叫的方法而改變，但他們不用手動管理狀態的變更。而且使用者也不可能會在狀態中出錯，像是在審核前就發佈文章。

定義 `Post` 並在草稿階段建立新實例

讓我們開始實作出函式庫吧！我們知道我們需要一個公開的結構體 `Post` 來存有些內容，所以我們先從結構體的定義開始，它會有個公開的關聯函式 `new` 來建立 `Post` 的實例，如範例 17-12 所示。我們還會再定義一個私有的特徵 `State`，定義 `Post` 所有狀態物件該有的行為。

然後 `Post` 會有個私有欄位 `state` 來擁有 `Option<T>` 且其內會存有一個特徵物件 `Box<dyn State>`。你會在之後瞭解為何 `Option<T>` 在此是必要的。

檔案名稱：`src/lib.rs`


```

pub struct Post {
    state: Option<Box<dyn State>>,
    content: String,
}

impl Post {
    pub fn new() -> Post {
        Post {
            state: Some(Box::new(Draft {})),
            content: String::new(),
        }
    }
}

trait State {}

struct Draft {}

impl State for Draft {}

```

範例 17-12：Post 結構體、能建立新的 Post 實例的 new 函式、State 特徵與 Draft 結構體的定義

State 定義了不同文章狀態共享的行為。Draft、PendingReview 與 Published 這些狀態物件都會實作 State 特徵。目前特徵還沒有任何方法，而且我們也只先定義 Draft 狀態，因為這是文章的初始狀態。

當我們建立新的 Post，我們對其 state 欄位給予存有 Box 的 Some 數值。此 Box 會指向一個新的 Draft 結構體實例。這確保每當我們建立 Post 的新實例時，它會從草稿起始。因為 Post 的 state 欄位是私有的，我們沒有任何方法可以建立處於其他狀態的 Post！在 Post::new 函式中，我們設置 content 欄位為一個新的空 String。

儲存文章內容的文字

在範例 17-11 我們想要能夠呼叫一個叫做 add_text 的函式並傳入 &str 來對部落格文章增加文字內容。我們實作此方法，而不是將 content 欄位透過 pub 公開出去，這樣我們之後就可以實作個方法來控制 content 欄位資料該怎麼讀取。add_text 方法非常直觀，所以讓我們在 impl Post 區塊中加上範例 17-13 的實作吧：

檔案名稱：src/lib.rs

```
impl Post {
    // --省略--
    pub fn add_text(&mut self, text: &str) {
        self.content.push_str(text);
    }
}
```

範例 17-13：實作 `add_text` 方法來將文字加入文章的 `content` 中

`add_text` 方法接收 `self` 的可變參考，因為我們在呼叫 `add_text` 時會改變 `Post` 實例。然後我們對 `content` 中的 `String` 呼叫 `push_str`，並傳入 `text` 引數來存到 `content` 之中。此行為與文章的狀態無關，所以它沒有被包含在狀態模式中。`add_text` 方法不會與 `state` 欄位有關係，但它是我們想支援的部分行為之一。

確保文章草稿的內容為空

儘管我們已經能透過 `add_text` 來為我們的文章加些內容，但我們還是希望 `content` 方法會回傳空字串切片，因為文章還在草稿階段中，如範例 17-11 的第七行所示。現在先讓我們用能滿足需求最簡單的方式來實作 `content` 方法，也就是永遠回傳空字串切片。之後一旦我們實作出能改變文章狀態為已發佈的能力，我們會回來修改這部分。目前文章只能處於草稿階段，所以文章內容應該要永遠為空。範例 17-14 顯示了此暫時的實作方式：

檔案名稱：src/lib.rs

```
impl Post {
    // --省略--
    pub fn content(&self) -> &str {
        ""
    }
}
```

範例 17-14：Post 暫時實作的 `content` 方法，這會永遠回傳一個空字串切片

透過此 `content` 方法，範例 17-11 的程式碼到第七行都能如期執行。

請求文章審核來變更它的狀態

接下來，我們需要增加請求文章審核的功能，這會將其狀態從 `Draft` 變更為 `PendingReview`。如範例 14-15 所示：

檔案名稱：src/lib.rs

```

impl Post {
    // --省略--
    pub fn request_review(&mut self) {
        if let Some(s) = self.state.take() {
            self.state = Some(s.request_review())
        }
    }
}

trait State {
    fn request_review(self: Box<Self>) -> Box<dyn State>;
}

struct Draft {}

impl State for Draft {
    fn request_review(self: Box<Self>) -> Box<dyn State> {
        Box::new(PendingReview {})
    }
}

struct PendingReview {}

impl State for PendingReview {
    fn request_review(self: Box<Self>) -> Box<dyn State> {
        self
    }
}

```

範例 17-15：對 `Post` 與 `State` 特徵實作的 `request_review` 方法

`Post` 現在有個公開方法叫做 `request_review`，這會接收 `self` 的可變參考。然後我們對 `Post` 目前的狀態呼叫其內部的 `request_review` 方法，然後此 `request_review` 方法會消耗目前的狀態並回傳新的狀態。

我們對 `State` 特徵也加上了 `request_review` 方法，所有有實作此特徵的型別現在都需要實作 `request_review` 方法。注意到不同於擁有 `self`、`&self` 或 `&mut self` 來作為方法的第一個參數，我們用的是 `self: Box<Self>`。此語法代表對持有型別的 `Box` 呼叫方法才有效。此語法取得 `Box<Self>` 的所有權，將舊的狀態無效化，讓 `Post` 的狀態數值可以轉換成新的狀態。

要消耗掉舊的狀態，`request_review` 方法需要取得狀態數值的所有權。這正是 `Post` 的 `state` 欄位中使用 `Option` 的用途，我們呼叫 `take` 方法來取得 `state` 欄位中 `Some` 的數值，並留下 `None`，因為 Rust 不允許結構體的欄位為空。這讓我們將 `Post` 的 `state` 移出來，而不只是借用。然後我們會將文章 `state` 數值設為此運算的結果。

我們需要暫時將 `state` 設為 `None`，而非只是像這樣 `self.state = self.state.request_review();` 直接設置來取得 `state` 的數值。這確保 `Post` 不會在我們轉換

到新狀態時，使用到舊的 `state` 數值。

`Draft` 的 `request_review` 方法需要回傳一個新的結構體 `PendingReview` box 實例，這代表文章正在等待審核的狀態。`PendingReview` 結構體也實作了 `request_review` 方法但沒有做任何轉換。反之，它只會回傳自己，因為當我們向已經處於 `PendingReview` 狀態的文章請求審核的話，它應該會維持 `PendingReview` 的狀態。

現在我們可以開始看出狀態模式的優勢了，`Post` 的 `request_review` 方法不管其 `state` 數值為何都是一樣的。每個狀態負責自己的規則。

我們維持 `Post` 的方法 `content` 不變，依然回傳一個空字串切片。我們現在的 `Post` 可以處於 `PendingReview` 狀態與 `Draft` 狀態，但我們想要 `PendingReview` 狀態也有相同的行為。現在範例 17-11 可以運行到第十行了！

透過 `approve` 改變 `content` 的行為

`approve` 方法會類似於 `request_review` 方法，它會設置 `state` 的數值為目前狀態審核通過時該處於的狀態，如範例 17-16 所示：

檔案名稱：src/lib.rs

```

impl Post {
    // --省略--
    pub fn approve(&mut self) {
        if let Some(s) = self.state.take() {
            self.state = Some(s.approve())
        }
    }
}

trait State {
    fn request_review(self: Box<Self>) -> Box<dyn State>;
    fn approve(self: Box<Self>) -> Box<dyn State>;
}

struct Draft {}

impl State for Draft {
    // --省略--
    fn approve(self: Box<Self>) -> Box<dyn State> {
        self
    }
}

struct PendingReview {}

impl State for PendingReview {
    // --省略--
    fn approve(self: Box<Self>) -> Box<dyn State> {
        Box::new(Published {})
    }
}

struct Published {}

impl State for Published {
    fn request_review(self: Box<Self>) -> Box<dyn State> {
        self
    }

    fn approve(self: Box<Self>) -> Box<dyn State> {
        self
    }
}

```

範例 17-16：對 `Post` 與 `State` 特徵實作 `approve` 方法

我們在 `State` 特徵加上 `approve` 方法，並新增一個也有實作 `State` 的新結構體 `Published` 特徵。

和 `PendingReview` 的 `request_review` 類似，如果我們對 `Draft` 呼叫 `approve` 方法，它不會有任何效果，因為它會回傳 `self`。當我們對 `PendingReview` 呼叫 `approve`，它會回傳一個新的結構體 `Published` `Box` 實例。`Published` 也有實作 `State` 特徵，對於 `request_review` 方法與 `approve` 方法，它只會回傳自己，因為文章在這些情況下都應該維持 `Published` 狀態。

現在我們需要更新 `Post` 的 `content` 方法。我們想依據 `Post` 當前的狀態回傳 `content`，所以我們將用 `Post` 來回傳它自己 `state` 所定義的 `content` 方法，如範例 17-17 所示：

檔案名稱：src/lib.rs

```
impl Post {  
    // --省略--  
    pub fn content(&self) -> &str {  
        self.state.as_ref().unwrap().content(self)  
    }  
    // --省略--  
}
```



範例 17-17：更新 `Post` 的 `content` 方法來呼叫 `State` 的 `content` 方法

因為目標是將這些所有規則維持在實作 `State` 的結構體內，我們對 `state` 呼叫 `content` 方法並傳遞文章實例（也就是 `self`）來作為引數。然後我們的回傳值就是對 `state` 數值使用 `content` 的回傳值。

我們對 `Option` 呼叫 `as_ref` 方法，因為我們希望取得 `Option` 內的數值參考，而不是該值的所有權。因為 `state` 的型別是 `Option<Box<dyn State>>`，當我們呼叫 `as_ref` 時會回傳 `Option<&Box<dyn State>>`。如果我們沒有呼叫 `as_ref` 的話，我們會得到錯誤，因為我們無法從借用的函式參數 `&self` 移動 `state`。

然後我們呼叫 `unwrap` 方法，我們知道這絕對不會恐慌，因為我們知道當 `Post` 的方法完成執行時，它們會確保 `state` 永遠包含一個 `Some` 數值。這是我們在第九章的「[當你知道的比編譯器還多的時候](#)」段落介紹過的其中一種情況。雖然編譯器不能理解，但我們知道永遠不可能會有 `None` 數值。

此時當我們呼叫 `&Box<dyn State>` 的 `content`，強制解參考（deref coercion）對 `&` 與 `Box` 產生影響，讓 `content` 方法最終對有實作 `State` 特徵的型別呼叫。這代表我需要在 `State` 特徵定義加上 `content`，而這正是我們要填入依據狀態為何來回傳何種內容的地方，如範例 17-18 所示：

檔案名稱：src/lib.rs

```

trait State {
    // --省略--
    fn content<'a>(&self, post: &'a Post) -> &'a str {
        ""
    }
}

// --省略--
struct Published {}

impl State for Published {
    // --省略--
    fn content<'a>(&self, post: &'a Post) -> &'a str {
        &post.content
    }
}

```

範例 17-18：在 `State` 特徵加上 `content` 方法

我們對 `content` 方法加上預設實作來回傳一個空字串切片。這代表我們不需要在 `Draft` 與 `PendingReview` 結構體實作 `content`。 `Published` 結構體會覆寫 `content` 方法並回傳 `post.content` 的數值。

注意到我們在此方法需要生命週期詮釋，如我們在第十章所討論到的。我們取得 `post` 的參考作為引數並回傳 `post` 的部分參考，所以回傳參考的生命週期與 `post` 引數的生命週期有關聯。

這樣就完成了！範例 17-11 可以成功執行！我們實作了部落格文章工作流程規則的狀態模式。規則邏輯會位於狀態物件中，而不會分散在 `Post` 中。

為何不用列舉？

你可能會好奇為何我們不使用 `enum` 的變體來表達不同的文章狀態。這的確是個可能的解法，歡迎嘗試親自實作看看，然後比較出你傾向於何者！使用列舉的缺點會是當你每次想要檢查列舉數值時，就需要使用 `match` 表達式或類似的方式才能處理每種變體。這可能會比特徵物件的做法還麻煩。

狀態模式的權衡取捨

我們展示了 Rust 能夠實作出物件導向狀態模式，來封裝文章每個狀態之間不同的行為。`Post` 的方法不會知道這些不同的行為。我們組織程式碼的方式，讓我們可以只看一個地方就能知道已發佈文章會擁有的各種行為，也就是實作 `State` 特徵的 `Published` 結構體。

如果我們要建立個不使用狀態模式的替代實作，我們可能會在 `Post` 或甚至在 `main` 程式碼中使用 `match` 表達式檢查文章狀態並變更行為。這意味著我們得查看許多地方才能知道已發佈文章狀態的含義！而且當我們增加的狀態越多，每個 `match` 表達式就需要更多分支。

透過狀態模式，`Post` 方法以及我們使用 `Post` 的地方就不需要 `match` 表達式，而且要加入新的狀態的話，我們只需要新增一個結構體並對其實作特徵方法。

使用狀態模式的實作能非常容易地擴展功能。為了觀察維護使用狀態模式的程式碼有多簡單，你可以嘗試以下一些建議：

- 新增一個 `reject` 方法讓文章狀態從 `PendingReview` 變回 `Draft`。
- 要求要呼叫兩次 `approve` 狀態才會變成 `Published`。
- 只允許使用者在 `Draft` 狀態才能新增文字內容。提示：讓狀態物件負責內容會發生什麼改變，但不負責修改 `Post`。

不過狀態模式有個劣勢，由於狀態實作狀態的轉換，有些狀態之間是彼此耦合的。如果我們在 `PendingReview` 與 `Published` 之間再加上另一個狀態像是 `Scheduled` 的話，我們就需要變更 `PendingReview` 的程式碼改轉換成 `Scheduled`。如果 `PendingReview` 不需要因為新狀態的加入做改變的話，我們可以少寫些程式碼，但這就意味著切換成其他種設計模式。

另一項劣勢是我們重複了一些邏輯。要消除掉一些重複的部分，我們可以試著對 `State` 的 `request_review` 和 `approve` 方法提供回傳 `self` 的預設實作。但是這樣就違反物件安全了，因為特徵不知道 `self` 的實際型別為何。我們想要能將 `State` 用在特徵物件中，所以它的方法必須是物件安全的。

另一個重複的部分包含 `Post` 的 `request_review` 與 `approve` 方法都以類似的方式實作。兩者均呼叫 `state` 欄位中 `Option` 內數值對應的相同方法。如果 `Post` 有很多方法都遵循這樣的模式的話，我們可以考慮定義巨集（macro）來消除重複的部分（請查閱第十九章的「[巨集](#)」段落）。

如其他物件導向語言所定義的來實作狀態模式，我們並沒有完全發揮出 Rust 的所有潛力。讓我們看看我們能對 `blog crate` 做些什麼改善，讓無效的狀態與轉換會產生編譯時錯誤。

定義狀態與行為成型別

我們會向你展示如何重新思考狀態模式，來達到不同的取舍效果。與其完全封裝狀態與轉換，讓外部程式碼完全看不到它們，我們會將狀態定義成不同的型別。這樣一來，Rust 的型別檢查系統就能避免在只能使用已發佈文章的地方使用到了文章草稿，並在編譯時就回傳錯誤。

讓我們先想一下範例 17-11 中 `main` 的第一個部分：

檔案名稱：`src/main.rs`


```
fn main() {
    let mut post = Post::new();

    post.add_text("我今天午餐吃了沙拉");
    assert_eq!("", post.content());
}
```

我們仍使用 `Post::new` 來建立新文章的草稿狀態以及能對文章內容新增文字的能力。但不同於在文章草稿的 `content` 方法中回傳空字串，我們這次選擇文章草稿不會實作 `content` 方法。這樣如果我們嘗試取得文章草稿內容時，我們會得到編譯錯誤告訴我們該方法不存在。如此一來，我們就不可能在生產環境意外顯示出文章草稿內容了，因為程式碼根本不會編譯過。範例 17-19 顯示了 `Post` 結構體與 `DraftPost` 結構體的定義，以及它們個別的方法：

檔案名稱：src/lib.rs

```
pub struct Post {
    content: String,
}

pub struct DraftPost {
    content: String,
}

impl Post {
    pub fn new() -> DraftPost {
        DraftPost {
            content: String::new(),
        }
    }

    pub fn content(&self) -> &str {
        &self.content
    }
}

impl DraftPost {
    pub fn add_text(&mut self, text: &str) {
        self.content.push_str(text);
    }
}
```

範例 17-19： `Post` 有 `content` 方法，而 `DraftPost` 則沒有 `content` 方法

`Post` 與 `DraftPost` 結構體都有個私有欄位 `content` 來儲存部落格文章文字。結構體不再有 `state` 欄位，因為我們將狀態的定義移到了結構體的类型中。`Post` 結構體就代表已發佈的文章，且其有個 `content` 方法來回傳 `content`。

我們仍然有 `Post::new` 函式，但是它沒有回傳 `Post` 實例，而是回傳了 `DraftPost` 的實例。因為 `content` 是私有的，而且沒有任何函式回傳 `Post`，所以目前沒有任何辦法能建立 `Post` 的實例。

`DraftPost` 結構體有個 `add_text` 方法，所以我們可以像之前一樣為 `content` 新增文字，但注意到 `DraftPost` 沒有定義 `content` 方法！所以現在程式確保所有文章都以草稿為起始，而且文章草稿不會提供顯示其內容的方法。任何想嘗試繞過此約束的方式都會產生編譯錯誤。

透過不同型別的轉移來實作狀態轉換

所以我們該怎麼取得發佈的文章呢？我們想要遵守執行的規則，文章草稿在審核並通過後才能夠發佈。在審核中的文章狀態應保持不顯示任何內容。讓我們新增另一個結構體 `PendingReviewPost` 來遵守這些約束吧。在 `DraftPost` 中訂一個會回傳 `PendingReviewPost` 的 `request_review` 方法，再對 `PendingReviewPost` 定義 `approve` 方法來回傳 `Post`，如範例 17-20 所示：

檔案名稱：`src/lib.rs`

```
impl DraftPost {
    // --省略--
    pub fn request_review(self) -> PendingReviewPost {
        PendingReviewPost {
            content: self.content,
        }
    }
}

pub struct PendingReviewPost {
    content: String,
}

impl PendingReviewPost {
    pub fn approve(self) -> Post {
        Post {
            content: self.content,
        }
    }
}
```

範例 17-20：呼叫 `DraftPost` 的 `request_review` 來建立 `PendingReviewPost`，且其有個 `approve` 方法能將 `PendingReviewPost` 轉換成已發佈的 `Post`

`request_review` 與 `approve` 方法都會取得 `self` 的所有權，因此會消耗 `DraftPost` 與 `PendingReviewPost` 實例，並分別轉換成 `PendingReviewPost` 與已發佈的 `Post`。這樣在我們呼叫 `request_review` 時，就不會有殘留的 `DraftPost` 實例，以此類推。`PendingReviewPost` 結構體也沒有定義 `content` 方法，所以嘗試讀取其內容會導致編譯錯誤，就如同 `DraftPost`。由於唯一能取得有 `content` 方法定義的已發佈 `Post` 是透過呼叫 `PendingReviewPost` 的

`approve` 方法，而唯一能取得 `PendingReviewPost` 的方法是呼叫 `DraftPost` 的 `request_review` 方法，我們現在將部落格文章工作流程寫進了型別系統中。

但我們也得對 `main` 做些小修改。`request_review` 與 `approve` 方法會回傳新的實例，而不是修改它們所呼叫的結構體，所以我們需要加些 `let post =` 來遮蔽賦值來儲存回傳的實例。我們也無法判定草稿與審核中的文章內容是否是空字串，不過我們其實也不需要它們，我們不再能編譯嘗試讀取這些狀態文章內容的程式碼了。

檔案名稱：`src/main.rs`

```
use blog::Post;

fn main() {
    let mut post = Post::new();

    post.add_text("我今天午餐吃了沙拉");

    let post = post.request_review();

    let post = post.approve();

    assert_eq!("我今天午餐吃了沙拉", post.content());
}
```

範例 17-21：修改 `main` 來使用新的部落格文章工作流程實作

我們修改 `main` 來重新賦值 `post` 意味著此實作不再遵循物件導向狀態模式了，狀態的轉換不再完全封裝在 `Post` 實作內部。然而我們得到的好處是的是無效狀態是不可能發生的了，這都多虧了型別系統與編譯時型別檢查！這確保了特定程式錯誤會在進入生產環境前就被察覺，像是顯示尚未發佈的文章內容。

你可以試試看在範例 17-20 之後，對 `blog crate` 實作稍早提及的額外需求任務建議，來看看你覺得此版本的程式碼設計如何。注意有些任務很可能在此設計就已經實作完成了。

我們看到儘管 Rust 能夠實作物件導向設計模式、其他像是將狀態寫入型別系統中的模式在 Rust 中也是可行的。這些模式有不同的取舍。雖然你可能非常熟悉物件導向模式，但重新思考問題，並善用 Rust 的特色可以帶來不少優勢，像是在編譯時就避免錯誤發生。物件導向模式在 Rust 中不會永遠是最好的解決方案，因為 Rust 有像是所有權這樣物件導向語言所沒有的特定功能。

總結

無論你讀完此章後，認為 Rust 是否屬於物件導向語言，你都知道在 Rust 中你可以使用特徵物件來取得一些物件導向的特色。動態分配能給予你的程式碼更多的彈性，但會犧牲一點執行時效能。你可以使用此彈性來實作物件導向模式，幫助提升程式碼可維護性。Rust 還有其他像是所有權等

物件導向語言所沒有的功能。物件導向模式不會永遠是善用 Rust 潛能的最佳方案，不過仍是個不錯的選項。

接下來，我們要看看模式（patterns），這是 Rust 另一個可以提供大量彈性的功能。我們在書中一路下來簡單看過它們好幾次了，不過我們還沒見識到它們全部的本事。讓我們來探索吧！

模式與配對

模式 (Patterns) 是 Rust 中的特殊語法，能用來配對複雜與簡單的类型結構。搭配 `match` 表達式與其他結構來使用模式的話，可以給予你對程式控制流更多的掌控權。模式與以下元件組合而成：

- 字面值 (Literals)
- 解構的陣列、列舉結構體或元組
- 變數
- 萬用字元 (Wildcards)
- 佔位符 (Placeholders)

模式的範例包含 `x`、`(a, 3)` 以及 `Some(Color::Red)`。當我們提到哪些模式是有效的時候，這些元件描述了我們要處理的資料形狀。我們的程式與其數值配對來決定我們的程式是否有取得正確的資料來繼續執行特定部分的程式碼。

要使用模式，我們將其與一些數值做比較。如果模式配對到數值的話，我們就能在程式碼中使用該數值部分。回憶一下第六章中使用模式的 `match` 表達式，像是硬幣分類機器的範例。如果有數值符合模式的形狀，我們就可以使用這些命名的部分。如果沒有的話，配對相關的程式碼就不會執行。

本章節會涵蓋所有與模式相關的內容。我們會討論能使用模式的地方，可反駁 (refutable) 與不可反駁 (irrefutable) 模式間的差別，以及不同種類的模式語法。在本章結束後，你便能知道如何使用模式來清楚表達許多概念。

所有能使用模式的地方

模式常出現於 Rust 中數個位置，而你已經不經意使用了很多模式了！此段落會介紹所有模式能有效出現的地方。

match 分支

如同第六章所討論過的，我們可以在 `match` 表達式中的分支使用模式。正式來說，`match` 表達式的定義為 `match` 關鍵字加上一個要配對的數值，然後會有一或數個包含模式的分支，以及如果數值配對到該分支模式之後要執行的表達式，如以下所示：

```
match VALUE {  
    PATTERN => EXPRESSION,  
    PATTERN => EXPRESSION,  
    PATTERN => EXPRESSION,  
}
```

舉例來說，以下是範例 6-5 的變數 `x` 對 `Option<i32>` 進行配對：

```
match x {  
    None => None,  
    Some(i) => Some(i + 1),  
}
```

在此 `match` 表達式中的模式有 `None` 與 `Some(i)`，顯示在每個箭頭的左方。

`match` 表達式有個要求就是它們必須是**徹底的 (exhaustive)**，所有 `match` 表達式數值可能的結果都必須涵蓋到。其中一個確保你有考慮到所有可能性的方式是在最後一個分支使用捕獲模式，命名一個能配對任何數值的變數就絕不會失敗，因此可以涵蓋剩餘的情況。

還有一個特定模式 `_` 可以獲取任意可能情況，但它不會綁定到變數中，所以它也很常用在最後的配對分支。舉例來說，`_` 模式就很適合用來忽略任何沒指明的數值。我們會在本章之後的「[忽略模式中的數值](#)」段落談到更多 `_` 的細節。

if let 條件表達式

在第六章中我們介紹了如何使用 `if let` 表達式，它等同於只配對一種情況的 `match` 表達式，主要作為更簡潔的語法。此外，`if let` 可以再加上 `else` 來包含如果 `if let` 模式不符的話能執行的程式碼。

範例 18-1 展示了我們能夠混合並配對 `if let`、`else if` 與 `else if let` 表達式。這樣做可以比 `match` 表達式還來得有彈性，因為 `match` 只能有一個數值與模式們配對。另外，Rust 也不需要 `if let`、`else if` 與 `else if let` 分支之間的條件彼此要有關聯。

範例 18-1 的程式碼依據一系列的條件檢查來決定背景顏色該為何。在此例中，我們建立一個寫死的變數數值，在實際程式中應該會由使用者輸入。

檔案名稱：src/main.rs

```
fn main() {
    let favorite_color: Option<&str> = None;
    let is_tuesday = false;
    let age: Result<u8, _> = "34".parse();

    if let Some(color) = favorite_color {
        println!("使用你最喜歡的顏色{color}作為背景");
    } else if is_tuesday {
        println!("星期二就用綠色！");
    } else if let Ok(age) = age {
        if age > 30 {
            println!("使用紫色作為背景顏色");
        } else {
            println!("使用橘色作為背景顏色");
        }
    } else {
        println!("使用藍色作為背景顏色");
    }
}
```

範例 18-1：混合 `if let`、`else if`、`else if let` 與 `else`

如果使用者指定的最喜歡的顏色，該顏色就是背景顏色。如果沒有喜歡的顏色且今天是星期二的話，背景顏色就是綠色。如果使用者用字串指定他們的年齡且可以成功轉換成數字的話，背景顏色依據數字結果就是紫色或橘色。如果以上條件都不符合的話，背景顏色就是藍色。

這樣的條件結構讓我們可以支援複雜的需求。透過我們在此寫死的數值，此例會印出 使用紫色作為背景顏色。

你可以看到 `if let` 也能如同 `match` 的分支一樣遮蔽變數，`if let Ok(age) = age` 這行就產生了新的遮蔽變數 `age` 來包含 `Ok` 變體內的數值。這意味著我們需要將 `if age > 30` 的條件放在區塊內，我們不能組合這兩個條件成 `if let Ok(age) = age && age > 30`。遮蔽的 `age` 在大括號開始之後的新作用域才有效，此時才能與 30 做比較。

使用 `if let` 表達式的缺點是編譯器不會徹底檢查，而 `match` 表達式則會。如果我們省略最後一個 `else` 區塊而因此忘了處理一些情況，編譯器不會警告我們這種可能的邏輯錯誤。

while let 條件迴圈

與 `if let` 的結構類似，`while let` 條件迴圈允許 `while` 迴圈只要在模式持續配對符合的情況下一直執行。在範例 18-2 的例子我們寫了一個 `while let` 迴圈使用向量做為堆疊，並以數值被插入向量時相反的順序印出它們。

```
let mut stack = Vec::new();

stack.push(1);
stack.push(2);
stack.push(3);

while let Some(top) = stack.pop() {
    println!("{}", top);
}
```

範例 18-2：使用 `while let` 迴圈，且只要 `stack.pop()` 有回傳 `Some` 就持續印出數值

此範例會依序顯示 3、2 然後是 1。 `pop` 方法會取得向量最後一個數值並回傳 `Some(value)`。如果向量是空的，`pop` 就回傳 `None`。只要 `pop` 有回傳 `Some`，`while` 迴圈就會持續執行其區塊中的程式碼。當 `pop` 回傳 `None` 時，迴圈就會結束。我們可以使用 `while let` 來取得堆疊彈出的每個數值。

for 迴圈

在 `for` 迴圈中，`for` 關鍵字之後的數值就是模式。舉例來說，在 `for x in y` 中 `x` 就是模式範例 18-3 展示了如何在 `for` 迴圈使用模式來解構或拆開一個 `for` 迴圈中的元組。

```
let v = vec!['a', 'b', 'c'];

for (index, value) in v.iter().enumerate() {
    println!("{}", value, index);
}
```

範例 18-3：使用模式來解構 `for` 迴圈中的元組

範例 18-3 的程式碼會顯示以下結果：

```
$ cargo run
Compiling patterns v0.1.0 (file:///projects/patterns)
Finished dev [unoptimized + debuginfo] target(s) in 0.52s
Running `target/debug/patterns`
a 位於索引 0
b 位於索引 1
c 位於索引 2
```


我們使用 `enumerate` 方法來配接一個疊代器來產生一個數值與該數值在疊代器中的索引，並放入元組中。第一個產生的數值為元組 `(0, 'a')`。當此數值配對到 `(index, value)` 模式時，`index` 會是 `0` 而 `value` 會是 `'a'`，並印出第一行的輸出。

let 陳述式

在本章節之前，我們只有告訴你模式能用在 `match` 和 `if let`，但事實上我們在其他地方也早就使用過模式了，這包含 `let` 陳述式。舉例來說，請看看以下這個使用 `let` 賦值變數的直白例子：

```
let x = 5;
```

每次當你像這樣使用 `let` 陳述式時，你就已經在使用模式了，儘管你還沒有察覺到！所以更正式地來說，`let` 陳述式是這樣定義的：

```
let PATTERN = EXPRESSION;
```

像 `let x = 5;` 這樣的陳述式中變數名稱會位於 `PATTERN` 的位置，變數名稱恰好是種特別簡單的模式。Rust 會將表達式與模式做比較，並賦值給它找到的任何名稱。所以在 `let x = 5;` 的範例中，`x` 是個模式並表示「將配對到的數值綁定給變數 `x`」。因為名稱 `x` 就是整個模式，此模式實際上等同於「將任何數值綁定給變數 `x`，無論該數值為何」。

為了更清楚理解 `let` 怎麼使用模式配對，請參考範例 18-4，這對 `let` 使用模式來解構一個元組。

```
let (x, y, z) = (1, 2, 3);
```

範例 18-4：使用模式來解構元組，並同時建立三個變數

我們在此用一個元組來配對一個模式。Rust 會將數值 `(1, 2, 3)` 與模式 `(x, y, z)` 做比較，並看出數值能配對到模式中，所以 Rust 將 `1` 綁定給 `x`、`2` 給 `y` 然後 `3` 給 `z`。你可以把此元組模式想成是三個獨立的變數模式組合在一起。

如果模式中的元素個數與元組中的元素個數不符合的話，整體型別就無法配對，所以我們會得到編譯錯誤。舉例來說，範例 18-5 嘗試將有三個元素的元組解構到兩個變數中，這樣就無法成功。

```
let (x, y) = (1, 2, 3);
```

範例 18-5：錯誤的模式結構，因為變數數量與元組元素個數不符

嘗試編譯此程式碼的話，會得到此型別錯誤：

```
$ cargo run
   Compiling patterns v0.1.0 (file:///projects/patterns)
error[E0308]: mismatched types
  --> src/main.rs:2:9
   |
2  |         let (x, y) = (1, 2, 3);
   |         ^^^^^^^   ----- this expression has type `({integer}, {integer}, {integer})`
   |         |
   |         expected a tuple with 3 elements, found one with 2 elements
   |
   = note: expected tuple `({integer}, {integer}, {integer})`
           found tuple `(_, _)`

For more information about this error, try `rustc --explain E0308`.
error: could not compile `patterns` due to previous error
```

要修正錯誤的話，我們可以使用 `_` 或 `..` 來忽略元組中的一或數個數值，你會在「[忽略模式中的數值](#)」段落中瞭解更多詳情。如果問題出在於我們模式中有太多變數的話，解決辦法就是移除些變數使變數數量等同於元組元素個數，讓型別可以配對。

函式參數

函式參數也可以是模式。範例 18-6 的程式碼宣告了一個函式叫做 `foo` 來接收一個參數叫做 `x` 其型別為 `i32`，現在這看起來你應該都還很熟悉。

```
fn foo(x: i32) {
    // 內部的程式碼
}
```

範例 18-6：在參數中使用模式的函式簽名

`x` 的部分就是模式！就如同我們在 `let` 所做的一樣，我們可以在函式引數中使用模式來配對元組，範例 18-7 將傳遞給函式的元組拆為不同數值。

檔案名稱：src/main.rs

```
fn print_coordinates(&(x, y): &(i32, i32)) {
    println!("目前位置: ({}, {})", x, y);
}

fn main() {
    let point = (3, 5);
    print_coordinates(&point);
}
```

範例 18-7：函式透過參數來解構元組

此程式碼會顯示 目前位置：(3, 5)。數值 `&(3, 5)` 能配對到模式 `&(x, y)`，所以 `x` 會是數值 3 而 `y` 會是數值 5。

我們還可以在閉包參數列表中像函式參數列表這樣使用模式，因為第十三章就提過閉包類似於函式。

到目前為止，你已經見過許多使用模式的方式，但模式在我們能使用的地方並不都會有相同的行為。在某些地方，模式必須是不可反駁的（irrefutable），而在其他場合它們則是可反駁的（refutable）。接下來我們會來討論這兩個概念。

可反駁性：何時模式可能會配對失敗

模式有兩種形式：可反駁的（refutable）與不可反駁的（irrefutable）。可以配對任何可能數值的模式屬於**不可反駁的（irrefutable）**。其中一個例子就是陳述式 `let x = 5;` 中的 `x`，因為 `x` 可以配對任何數值，因此不可能會配對失敗。而可能會對某些數值配對失敗的模式則屬於**可反駁的（refutable）**。其中一個例子是表達式 `if let Some(x) = a_value` 中的 `Some(x)`，因為如果 `a_value` 變數中的數值為 `None` 而非 `Some` 的話，`Some(x)` 模式就會配對失敗。

函式參數、`let` 陳述式與 `for` 迴圈只能接受不可反駁的模式，當數值無法配對時，程式無法作出任何有意義的事。`if let` 與 `while let` 表達式接受可反駁與不可反駁的模式，但是編譯器會警告不可反駁的模式，因為定義上來說它們用來處理可能會失敗的場合，條件表達式的功能就是依據成功或失敗來執行不同動作。

大致上來說，你通常不需要擔心可反駁與不可反駁模式之間的區別，不過你會需要熟悉可反駁性這樣的概念，所以當你看到錯誤訊息時，能及時反應理解。在這樣的場合，你需要依據程式碼的預期行為來改變模式或是使用模式的結構。

讓我們看看當我們嘗試在 Rust 要求不可反駁模式的地方使用可反駁模式的範例與其反例。範例 18-8 顯示了一個 `let` 陳述式，但是我們指定的模式是 `Some(x)`，這是可反駁模式。如我們所預期的，此程式碼無法編譯。

```
let Some(x) = some_option_value;
```

範例 18-8：嘗試在 `let` 使用可反駁模式

如果 `some_option_value` 是數值 `None`，它會無法與模式 `Some(x)` 做配對，這意味著此模式是可反駁的。但是 `let` 陳述式只能接受不可反駁的模式，因為程式碼對 `None` 數值就無法作出任何有效的動作。在編譯時 Rust 就會抱怨我們嘗試在需要不可反駁模式的地方使用了可反駁模式：

```

$ cargo run
   Compiling patterns v0.1.0 (file:///projects/patterns)
error[E0005]: refutable pattern in local binding: `None` not covered
  --> src/main.rs:3:9
   |
3  |     let Some(x) = some_option_value;
   |         ^^^^^^^ pattern `None` not covered
   |
   = note: `let` bindings require an "irrefutable pattern", like a `struct` or
an `enum` with only one variant
   = note: for more information, visit https://doc.rust-lang.org/book/ch18-02-
refutability.html
note: `Option<i32>` defined here
   = note: the matched value is of type `Option<i32>`
help: you might want to use `if let` to ignore the variant that isn't matched
   |
3  |     let x = if let Some(x) = some_option_value { x } else { todo!() };
   |     ++++++                                     ++++++

```

For more information about this error, try `rustc --explain E0005`.

error: could not compile `patterns` due to previous error

因為 `Some(x)` 模式沒有涵蓋（且也涵蓋不了！）所有有效的數值，Rust 合理地產生了編譯錯誤。

如果我們在需要不可反駁模式的地方使用可反駁模式的錯誤，我們可以變更程式碼使用模式的方式來修正：與其使用 `let`，我們可以改用 `if let`。這樣如果模式不符的話，程式碼就會跳過大括號中的程式碼，讓我們可以繼續有效執行下去。範例 18-9 顯示了如何修正範例 18-8 的程式碼。

```

if let Some(x) = some_option_value {
    println!("{}", x);
}

```

範例 18-9：使用 `if let` 而非 `let` 來使用可反駁模式

我們給了程式碼出路！此程式碼可以完美執行，雖然這也代表我們使用不可反駁模式的話會得到一些警告。如果我們給予 `if let` 一個像是 `x` 這樣永遠能配對的模式的話，編譯器會出現警告，如範例 18-10 所示。

```

if let x = 5 {
    println!("{}", x);
};

```

範例 18-10：嘗試在 `if let` 使用不可反駁的模式

Rust 會抱怨說在 `if let` 使用不可反駁的模式沒有任何意義：

```

$ cargo run
   Compiling patterns v0.1.0 (file:///projects/patterns)
warning: irrefutable `if let` pattern
  --> src/main.rs:2:8
   |
2  |         if let x = 5 {
   |             ^^^^^^^
   |
= note: `[warn(irrefutable_let_patterns)]` on by default
= note: this pattern will always match, so the `if let` is useless
= help: consider replacing the `if let` with a `let`

warning: `patterns` (bin "patterns") generated 1 warning
   Finished dev [unoptimized + debuginfo] target(s) in 0.39s
   Running `target/debug/patterns`
5

```

基於此原因，`match` 的分支必須是可反駁模式。除了最後一個分支因為要配對任何剩餘數值，所以會是不可反駁模式。Rust 允許我們在 `match` 只使用一個不可反駁模式的分支，不過這樣做並不是很實用，且可以直接用簡單的 `let` 陳述式取代。

現在你知道哪裡能使用模式，以及可反駁與不可反駁模式的不同了。讓我們來涵蓋模式建立時可以使用的所有語法吧。

模式語法

在此段落中，我們會收集所有模式中的有效語法，並討論你會怎麼使用它們。

配對字面值

如同你在第六章所見的，你可以直接使用字面值來配對模式，以下程式碼展示了一些範例：

```
let x = 1;

match x {
  1 => println!("一"),
  2 => println!("二"),
  3 => println!("三"),
  _ => println!("任意數字"),
}
```

此程式碼會顯示「一」因為 `x` 的數值為 1。此語法適用於當你想要程式碼取得一個特定數值時，就馬上採取行動的情況。

配對變數名稱

變數名稱是能配對任何數值的不可反駁模式，而且我們在本書中已經使用非常多次。不過當你在 `match` 表達式中使用變數名稱時會複雜一點。因為 `match` 會初始一個新的作用域，作為 `match` 表達式部分模式的宣告變數會遮蔽 `match` 結構外同名的變數，和所有變數一樣。在範例 18-11 中，我宣告了一個變數叫做 `x` 其有數值 `Some(5)` 和一個變數 `y` 其有數值 `10`。然後我們建立一個數值 `x` 的 `match` 表達式。檢查配對分之中的模式並在最後用 `println!` 顯示出來，並嘗試在程式碼執行或進一步閱讀之前推測其會顯示的結果會為何。

檔案名稱：src/main.rs

```
let x = Some(5);
let y = 10;

match x {
  Some(50) => println!("取得 50"),
  Some(y) => println!("配對成功，y = {y}"),
  _ => println!("預設情形，x = {:?}" , x),
}

println!("最後結果：x = {:?}, y = {y}" , x);
```

範例 18-11：`match` 表達式其中一個分支宣告了遮蔽的變數 `y`

讓我們跑一遍看看當 `match` 執行時發生了什麼事。第一個配對分支並不符合 `x` 定義的數值，所以程式繼續執行下去。

第二個配對分支宣告了一個新的變數叫做 `y` 來配對 `Some` 內的任何數值。因為我們位於 `match` 表達式內的新作用域，此新的 `y` 變數並不是我們一開始宣告有數值 10 的 `y`。這個新的 `y` 會配對 `Some` 內的任何數值，也就是 `x` 擁有的數值。因此，這個新的 `y` 會綁定 `x` 中 `Some` 的內部數值。該數值是 5，所以該分支的表達式就會執行並印出 配對成功，`y = 5`。

如果 `x` 是 `None` 數值而非 `Some(5)` 的話，前兩個分支的模式都不會配對到，所以數值會配對到底線的分支。我們沒有在底線分支的模式中宣告 `x` 變數，所以表達式中的 `x` 仍然是外部沒有被遮蔽的 `x`。在這樣的假設狀況下，`match` 會印出 預設情形，`x = None`。

當 `match` 完成時，其作用域就會結束，所以作用域內的內部 `y` 也會結束。最後一個 `println!` 會顯示 最後結果：`x = Some(5)`，`y = 10`。

要建立個能對外部 `x` 與 `y` 數值做比較的 `match` 表達式而非遮蔽變數的話，我們需要改用條件配對守護。我們會在之後的「[提供額外條件的配對守護](#)」段落討論配對守護。

多重模式

在 `match` 表達式中，你可以使用 `|` 語法來配對數個模式，這是 **OR（或）** 運算子模式。舉例來說，以下程式碼會配對 `x` 的數值到配對分支，第一個分支有個 `OR` 的選項，代表如果 `x` 的數值配對的到分支中任一數值的話，該分支的程式碼就會執行：

```
let x = 1;

match x {
  1 | 2 => println!("一或二"),
  3 => println!("三"),
  _ => println!("任意數字"),
}
```

此程式碼會印出 一或二。

透過 `..=` 配對數值範圍

`..=` 語法讓我們可以配對一個範圍內包含的數值。在以下程式碼中，當模式配對的到範圍內的任何數值時，該分支就會執行：


```
let x = 5;

match x {
    1..=5 => println!("一到五"),
    _ => println!("其他"),
}
```

如果 `x` 是 1、2、3、4 或 5 的話，第一個分支就能配對到。在配對多重數值時，此語法比使用 `|` 運算子來表達相同概念還輕鬆得多。如果我們使用 `|` 的話，就得指明 `1 | 2 | 3 | 4 | 5` 而非 `1..=5`。指定範圍相對就簡短許多，尤其是如果我們得配對像是數字 1 到 1,000 的話！

編譯器會在編譯時檢查範圍是否為空，然後因為 `char` 與數字數值是 Rust 中唯一能判斷範圍是否為空的型別，所以範圍只允許使用數字或 `char` 數值。

以下是使用 `char` 數值作為範圍的範例：

```
let x = 'c';

match x {
    'a'..'j' => println!("前半部 ASCII 字母"),
    'k'..'z' => println!("後半部 ASCII 字母"),
    _ => println!("其他"),
}
```

Rust 可以知道 `'c'` 有包含在第一個模式的範圍內，所以印出 前半部 ASCII 字母。

解構拆開數值

我們可以使用模式來解構結構體、列舉與元組，以便使用這些數值的不同部分。讓我們依序來看看。

解構結構體

範例 18-12 有個結構體 `Point` 其有兩個欄位 `x` 與 `y`，我們可以在 `let` 陳述式使用模式來拆開它。

檔案名稱：src/main.rs

```

struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 0, y: 7 };

    let Point { x: a, y: b } = p;
    assert_eq!(0, a);
    assert_eq!(7, b);
}

```

範例 18-12：解構結構體欄位成獨立的變數

此程式碼建立了變數 `a` 與 `b` 來配對 `p` 結構體中 `x` 與 `y` 的欄位。此範例顯示出模式中的變數名稱不必與結構體中的欄位名稱一樣。不過通常還是建議變數名稱與欄位名稱一樣，以便記得哪些變數來自於哪個欄位。因為用變數名稱來配對欄位是十分常見的，而且因為 `let Point { x: x, y: y } = p;` 會包含許多重複部分，所以配對結構體欄位的模式有另一種簡寫方式，你只需要列出結構體欄位的名稱，這樣從結構體建立的變數名稱就會有相同名稱。範例 18-13 顯示的程式碼行為與範例 18-12 一樣，但是在 `let` 模式建立的變數是 `x` 與 `y` 而非 `a` 與 `b`。

檔案名稱：src/main.rs

```

struct Point {
    x: i32,
    y: i32,
}

fn main() {
    let p = Point { x: 0, y: 7 };

    let Point { x, y } = p;
    assert_eq!(0, x);
    assert_eq!(7, y);
}

```

範例 18-13：使用結構體欄位簡寫來解構結構體欄位

此程式碼建立了變數 `x` 與 `y` 並配對到變數 `p` 的 `x` 與 `y` 欄位。結果就是變數 `x` 與 `y` 會包含 `p` 結構體中的數值。

我們也可以將字面值數值作為結構體模式中的一部分，而不用建立所有欄位的變數。這樣做我們可以在解構一些欄位成變數時，測試其他欄位是否有特定數值。

在範例 18-14 的 `match` 表達式將 `Point` 的數值分成三種情況：位於 `x` 軸的點（也就是 `y = 0`）、位於 `y` 軸的點（`x = 0`）或不在任何軸的點。

檔案名稱：src/main.rs

```
fn main() {  
    let p = Point { x: 0, y: 7 };  
  
    match p {  
        Point { x, y: 0 } => println!("位於 x 軸的 {x}"),  
        Point { x: 0, y } => println!("位於 y 軸的 {y}"),  
        Point { x, y } => println!("不在任一軸：({x}, {y})"),  
    }  
}
```

範例 18-14：解構並配對模式中的字面值數值

第一個分支透過指定 `y` 欄位配對字面值為 `0` 來配對任何在 `x` 軸上的點。此模式仍然會建立變數 `x` 能讓我們在此分支的程式碼中使用。

同樣地，第二個分支透過指定 `x` 欄位配對字面值為 `0` 來配對任何在 `y` 軸上的點，並建立擁有 `y` 欄位數值的變數 `y`。第三個分支沒有指定任何字面值，所以它能配對任何其他 `Point` 並建立 `x` 與 `y` 欄位對應的變數。

在此例中，數值 `p` 會配對到第二個分支，因為其 `x` 為 `0`，所以此程式碼會印出 位於 `y` 軸的 `7`。

回想一下 `match` 表達式在找到第一個符合的配對模式之後就會停止檢查分支，所以就算 `Point { x: 0, y: 0 }` 真的在 `x` 軸與 `y` 軸，此程式碼也只會印出 位於 `x` 軸的 `0`。

解構列舉

我們已經在本書中之前的章節就解構過列舉（比如第六章的範例 6-5）。但我們還沒談到的細節是解構列舉的模式必須與列舉定義中其所儲存的資料相符。作為示範，我們在範例 18-15 中使用範例 6-2 的 `Message` 列舉並寫一個 `match` 來提供會解構每個內部數值的模式。

檔案名稱：src/main.rs

```

enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(i32, i32, i32),
}

fn main() {
    let msg = Message::ChangeColor(0, 160, 255);

    match msg {
        Message::Quit => {
            println!("Quit 變體沒有資料能解構。");
        }
        Message::Move { x, y } => {
            println!("Move 往 x 的方向為 {x} 且往 y 的方向為 {y}");
        }
        Message::Write(text) => println!("文字訊息：{text}"),
        Message::ChangeColor(r, g, b) => {
            println!("變更顏色為紅色 {r}、綠色 {g} 與藍色 {b}")
        }
    }
}

```

範例 18-15：解構持有不同種數值的列舉變體

此程式碼會印出 變更顏色為紅色 0、綠色 160 與藍色 255。請嘗試變更 `msg` 的數值來看看其他分支的程式碼會執行出什麼。

對於像是 `Message::Quit` 這種沒有任何資料的列舉，我們無法進一步解構出任何資料。我們只能配對其本身的數值 `Message::Quit`，所以在該模式中沒有任何變數。

對於像是 `Message::Move` 這種類結構體列舉變體，我們可以使用類似於指定配對結構體的模式。在變體名稱之後，我們加上大括號以及列出欄位名稱作為變數，讓我們能拆成不同部分並在此分支的程式碼中使用。我們在此使用範例 18-13 一樣的簡寫形式。

對於像是 `Message::Write` 這種持有一個元素，以及 `Message::ChangeColor` 這種持有三個元素的類元組列舉變體，我們可以使用類似於配對元組的模式。模式中的變數數量必須與我們要配對的變體中元素個數相符。

解構巢狀結構體與列舉

到目前為止，我們所有的結構體或列舉配對範例的深度都只有一層，但配對也可以用於巢狀項目中！舉例來說，我們可以重構範例 18-15 的程式碼，在 `ChangeColor` 中支援 RGB 與 HSV 顏色，如範例 18-16 所示。

```

enum Color {
    Rgb(i32, i32, i32),
    Hsv(i32, i32, i32),
}

enum Message {
    Quit,
    Move { x: i32, y: i32 },
    Write(String),
    ChangeColor(Color),
}

fn main() {
    let msg = Message::ChangeColor(Color::Hsv(0, 160, 255));

    match msg {
        Message::ChangeColor(Color::Rgb(r, g, b)) => {
            println!("變更顏色為紅色 {r}、綠色 {g} 與藍色 {b}");
        }
        Message::ChangeColor(Color::Hsv(h, s, v)) => {
            println!("變更顏色為色相 {h}、飽和度 {s} 與明度 {v}");
        }
        _ => (),
    }
}

```

範例 18-16：配對巢狀列舉

`match` 表達式的第一個分支模式會配對包含 `Color::Rgb` 變體的 `Message::ChangeColor` 列舉變體，然後該模式會綁定內部三個 `i32` 數值。第二個分支也是配對到 `Message::ChangeColor` 列舉變體，但是內部列舉會改配對 `Color::Hsv`。我們可以在一個 `match` 表達式指定這些複雜條件，即使有兩個列舉參與其中。

解構結構體與元組

我們甚至可以用更複雜的方式來混合、配對並巢狀解構模式。以下範例展示了一個複雜的結構模式，其將一個結構體與一個元組置於另一個元組中，並將所有的原始數值全部解構出來：

```
let ((feet, inches), Point { x, y }) = ((3, 10), Point { x: 3, y: -10 });
```

此程式碼讓我們將複雜的型別拆成部分元件，讓我們可以分別使用我們有興趣的數值。

解構模式是個能方便使用部分數值的方式，比如結構體每個欄位分別獨立的數值。

忽略模式中的數值

你已經看過有時候在模式中忽略數值是很實用的，像是在 `match` 中的最後一個分支能捕獲所有剩餘用不到的可能數值。模式有一些方式可以忽略所有或部分數值：使用（你已經看過的）`_` 模式、在其他模式使用 `_` 模式、在名稱前加上底線，或是使用 `..` 來忽略剩餘部分的數值。讓我們來探討如何與為何要使用這些模式吧。

透過 `_` 忽略整個數值

我們使用底線在來作為萬用字元（wildcard）模式，這會配對任何數值，但不會綁定其值。雖然底線 `_` 模式特別適合作為 `match` 表達式最後一個分支，但我們可以將它用在任何模式中，包含函式參數，如範例 18-17 所示。

檔案名稱：src/main.rs

```
fn foo(_: i32, y: i32) {
    println!("此程式碼只使用了參數 y: {}", y);
}

fn main() {
    foo(3, 4);
}
```

範例 18-17：在函式簽名中使用 `_`

此程式碼會完全忽略第一個引數傳入的數值 `3`，並會印出 `此程式碼只使用了參數 y: 4`。

在大多數情況中如果當你不再需要特定函式參數的話，你會直接變更簽名讓它不會包含沒有使用到的參數。但忽略函式參數在某些場合會很有用。舉例來說，當你實作的特徵有個特定的型別簽名，但是你實作的函式本體不需要其中某個參數。這樣你就不會被編譯器警告沒有使用到的函式參數，會當做你有使用參數名稱一樣。

透過巢狀 `_` 忽略部分數值

我們也可以在其他模式中使用 `_` 來忽略部分數值。舉例來說，當我們只想測試部分數值，但不會用到執行的程式碼中其他部分數值的情況。範例 18-18 的程式碼負責管理設定值的數值。業務要求使用者不能覆寫已經存在的自訂數訂值，但可以取消設定值，也可以在目前未設定時提供數值。

```

let mut setting_value = Some(5);
let new_setting_value = Some(10);

match (setting_value, new_setting_value) {
    (Some(_), Some(_)) => {
        println!("無法覆寫已經存在的自訂數值");
    }
    _ => {
        setting_value = new_setting_value;
    }
}

println!("設定為 {:?}", setting_value);

```

範例 18-18：當我們不需要使用 `Some` 內部數值時，在模式中使用底線來配對 `Some` 變體

此程式碼會印出 無法覆寫已經存在的自訂數值 接著印出 設定為 `Some(5)`。在第一個配對分支中，我們不需要去配對或使用任一 `Some` 變體內的數值，但我們的確需要檢測 `setting_value` 與 `new_setting_value` 是否都為 `Some` 變體的情況。在此情況下，我們印出為何不能變更 `setting_value`，且不讓它被改變。

在其他所有情況下（無論是 `setting_value` 還是 `new_setting_value` 為 `None`），我們用第二個分支的 `_` 模式來配對，我們讓 `new_setting_value` 變成 `setting_value`。

我們也可以在同個模式中的多重位置使用底線來忽略特定數值。範例 18-19 忽略了有五個元素的元組中第二個與第四個數值。

```

let numbers = (2, 4, 8, 16, 32);

match numbers {
    (first, _, third, _, fifth) => {
        println!("一些數字：{first}、{third}、{fifth}")
    }
}

```

範例 18-19：忽略元組中數個位置

此程式碼會印出 一些數字：2、8、32，然後數值 4 與 16 會被忽略。

在名稱前加上 `␣` 來忽略未使用的變數

如果你建立了一個變數但沒有在任何地方使用到它，Rust 通常會提出警告，因為未使用的變數可能會是個錯誤。但有時候先建立個你還沒有使用的變數是很有用的，像是你還在寫原型或是才剛開個專案而已。在這種場合，你可以在尚未使用的變數名稱前加上底線，來告訴 Rust 不用提出警告。在範例 18-20 中，我們建立了兩個未使用的變數，但當我們編譯此程式碼時，我們應該會只收到其中一個的警告而已。

檔案名稱：src/main.rs

```
fn main() {  
    let _x = 5;  
    let y = 10;  
}
```

範例 18-20：在變數名稱前加上底線來避免未使用變數的警告

我們在此收到沒有使用變數 `y` 的警告，但是我們沒有收到 `_x` 的警告。

注意到只使用 `_` 與在名稱前加上底線之間是有些差別的。`_x` 仍會綁定數值到變數中，但 `_` 不會做任何綁定。為了展示這樣的區別是有差的，我們用範例 18-21 來展示一個錯誤。

```
let s = Some(String::from("哈囉！"));  
  
if let Some(_s) = s {  
    println!("發現字串");  
}  
  
println!("{:?}", s);
```



範例 18-21：以底線開頭的未使用變數仍會綁定數值，因而造成數值所有權被移動

我們會收到錯誤，因為 `s` 的數值仍會被移至 `_s`，讓我們無法再使用 `s`。不過只使用底線的話就不會綁定數值。範例 18-22 就能夠編譯不會產生任何錯誤，因為 `s` 沒有移至 `_`。

```
let s = Some(String::from("哈囉！"));  
  
if let Some(_) = s {  
    println!("發現字串");  
}  
  
println!("{:?}", s);
```

範例 18-22：使用底線不會綁定數值

此程式碼就能執行，因為我們沒有將 `s` 綁定給誰，它沒被移動。

透過 `..` 忽略剩餘部分數值

對於有許多部分的數值，我們可以用 `..` 語法來使用特定部分，然後忽略剩餘部分，來避免需要對每個要忽略的數值都得加上底線。`..` 模式會忽略模式中剩餘尚未配對的任何部分數值。在範例 18-23 中，我們有個 `Point` 結構體存有三維空間中的座標。而在 `match` 表達式中，我們想要只處理 `x` 座標並忽略 `y` 與 `z` 欄位的數值。


```

struct Point {
    x: i32,
    y: i32,
    z: i32,
}

let origin = Point { x: 0, y: 0, z: 0 };

match origin {
    Point { x, .. } => println!("x 為 {}", x),
}

```

範例 18-23：透過使用 `..` 來忽略 `Point` 中除了 `x` 以外的所有數值

我們列出 `x` 數值接著只包含 `..` 模式。這比需要列出 `y: _` 和 `z: _` 還要快，尤其是當我們要處理有許多欄位的結構體，但只需要用到一或兩個欄位的狀況下。

`..` 語法會擴展其所有所需得數值。範例 18-24 展示如何在元組使用 `..`。

檔案名稱：src/main.rs

```

fn main() {
    let numbers = (2, 4, 8, 16, 32);

    match numbers {
        (first, .., last) => {
            println!("一些數字：{first}、{last}");
        }
    }
}

```

範例 18-24：只配對元組中的第一個與最後一個數值，並忽略其他所有數值

在此程式碼中，第一個與最後一個數值會配對到 `first` 與 `last`。`..` 會配對並忽略中間所有數值。

然而，使用 `..` 必須是明確的。如果 Rust 無法確定是哪些數值要配對，而哪些是要忽略的話，它會回傳錯誤給我們。範例 18-25 含糊地使用了 `..`，所以它無法編譯。

檔案名稱：src/main.rs

```
fn main() {
    let numbers = (2, 4, 8, 16, 32);

    match numbers {
        (.., second, ..) => {
            println!("一些數字: {}", second)
        },
    }
}
```



範例 18-25：嘗試含糊地使用 ..

當我們編譯此範例時，我們會得到此錯誤：

```
$ cargo run
   Compiling patterns v0.1.0 (file:///projects/patterns)
error: `..` can only be used once per tuple pattern
--> src/main.rs:5:22
   |
5  |         (.., second, ..) => {
   |         --             ^^ can only be used once per tuple pattern
   |         |
   |         previously used here

error: could not compile `patterns` due to previous error
```

Rust 不可能知道在配對 `second` 之前要忽略多少元組中的數值，以及在之後得再忽略多少數值。此程式碼可以代表我們想要忽略 2、綁定 `second` 到 4 然後忽略 8、16 和 32；或者我們想要忽略 2 和 4、綁定 `second` 到 8 然後忽略 16 和 32，以及更多可能。變數名稱 `second` 對 Rust 沒有任何特別意義，所以我們得到編譯錯誤，因為像這樣在兩個地方使用 `..` 是含糊不清的。

提供額外條件的配對守護

配對守護 (match guard) 是個在 `match` 分支之後額外指定的 `if` 條件，此條件也必須符合配對才能選擇該分支。配對守護適用於比單獨模式所能表達的還複雜的情況。

該條件能使用配對建立的變數。範例 18-26 展示 `match` 的第一個分支有個模式 `Some(x)` 並使用配對守護 `if x % 2 == 0` (如果數字為偶數就會成立)。

```
let num = Some(4);

match num {
    Some(x) if x % 2 == 0 => println!("數字 {x} 是偶數"),
    Some(x) => println!("數字 {x} 是奇數"),
    None => (),
}
```

範例 18-26：對模式加上配對守護

此範例會印出 `數字 4 是偶數`。當 `num` 與第一個分支做比較時，它會配對到，因為 `Some(4)` 能與 `Some(x)` 做配對。然後配對守護會檢查數值 `x` 除以 2 的餘數是否為 0，然後因為的確如此，所以就選擇了第一個分支。

如果 `num` 為 `Some(5)` 的話，第一個分支的配對守護則會是否，因為 5 除以 2 的餘數為 1，並不等於 0。Rust 就會接著檢查第二條分支，然後因為第二條分支沒有任何配對守護所以能配對到任何 `Some` 變體。

在模式中沒有任何方式能夠表達 `if x % 2 == 0`，所以配對守護讓我們能夠有能力表達此邏輯。這種額外的表達能力的缺點是，編譯器對有配對守護的分支就不會窮舉檢查。

在範例 18-11 中，我們提到我們可以使用模式配對來解決我們的模式遮蔽問題。回想一下 `match` 表達式中使用的是模式內建立的新變數，而不是使用 `match` 外部的變數。該新變數會讓我們無法測試外部變數的數值。範例 18-27 展示我們如何使用配對守護來修正此問題。

檔案名稱：src/main.rs

```
fn main() {
    let x = Some(5);
    let y = 10;

    match x {
        Some(50) => println!("取得 50"),
        Some(n) if n == y => println!("配對成功，n = {n}"),
        _ => println!("預設情形，x = {:?}", x),
    }

    println!("最後結果：x = {:?}", y = {y}", x);
}
```

範例 18-27：使用配對守護來測試與外部變數是否相等

此程式碼現在會印出 `預設情形，x = Some(5)`。第二個模式中沒有宣告新的變數 `y` 來遮蔽外部的 `y`，意味著我們可以在配對守護中使用外部的 `y`。我們不再指定模式為 `Some(y)`，因為這樣會遮蔽外部的 `y`，我們改指定成 `Some(n)`。這樣建立了一個新的變數 `n` 且不會遮蔽任何事物，因為 `match` 外部沒有任何變數 `n`。

配對守護 `if n == y` 不屬於模式，因此不會宣告任何新變數。此 `y` 就是外部的 `y` 而非新遮蔽的 `y`，而且我們可以透過將 `n` 與 `y` 做比較來檢查數值是否與外部 `y` 的數值相等。

你也可以在配對守護中使用 `OR` 運算子 `|` 來指定多重模式，配對守護的條件會套用在所有的模式中。範例 18-28 顯示了結合配對守護與使用 `|` 模式之間的優先層級（precedence）。此例中的重點部分在於 `if y` 配對守護能套用在 `4`、`5` 與 `6`，而不是只有 `6` 會用到 `if y`。

```
let x = 4;
let y = false;

match x {
  4 | 5 | 6 if y => println!("是"),
  _ => println!("否"),
}
```

範例 18-28：結合數個模式與配對守護

此配對條件表示該分支只有在數值 `x` 等於 `4`、`5` 或 `6`，以及如果 `y` 為 `true` 時才算配對到。當此程式碼執行時，第一個分支的模式有配對到，因為 `x` 為 `4`，但是配對守護 `if y` 為否，所以不會選擇第一個分支。程式碼會移動到第二個分支，然後程式會配對到並印出 `no`。原因在於 `if` 條件會套用到整個模式 `4 | 5 | 6`，而不是只有最後一個數值 `6`。換句話說，配對守護與模式之間的優先層級會像是這樣：

```
(4 | 5 | 6) if y => ...
```

而不是這樣：

```
4 | 5 | (6 if y) => ...
```

在執行此程式碼之後，優先層級的行為就很明顯了，如果配對守護只會用在由 `|` 運算子指定數值列表中最後一個數值的話，該分支應該要能配對到並讓程式印出 `是`。

@ 綁定

`At` 運算子（`@`）能讓我們在測試某個數值是否配對的到一個模式的同時，建立出一個變數來持有該數值。在範例 18-29 我們想要測試 `Message::Hello` 的 `id` 欄位是否位於 `3..=7` 的範圍中。我們也想要將該數值綁定到變數 `id_variable` 之中，讓我們可以在該分支對應的程式碼中使用它。我們可以將此變數命名為與欄位同名的 `id`，但在此例中我們會使用不同名稱。

```

enum Message {
    Hello { id: i32 },
}

let msg = Message::Hello { id: 5 };

match msg {
    Message::Hello {
        id: id_variable @ 3..=7,
    } => println!("id 在此範圍中: {}", id_variable),
    Message::Hello { id: 10..=12 } => {
        println!("id 在其他範圍中")
    }
    Message::Hello { id } => println!("找到其他 id: {}", id),
}

```

範例 18-29：使用 `@` 來在測試時綁定模式中的數值

此範例會印出 `id 在此範圍中: 5`。透過在範圍 `3..=7` 之前指定 `id_variable @`，我們能獲取要配對到範圍的數值，並同時測試該數值是否有配對到範圍模式。

在第二個分支中，我們只有在模式中指定範圍，該分支對應的程式碼就沒有變數能包含 `id` 欄位的實際數值。`id` 欄位數值可能是 10、11 或 12，但此模式的程式碼不會知道其值為何。該模式程式碼無法使用 `id` 欄位的數值，因為我們沒有將 `id` 數值存為變數。

在最後一個分支中，我們指定沒有限制範圍的變數，我們有能在分支程式碼中使用的有效變數 `id`。原因是因為我們使用了結構體欄位簡寫語法。不過我們在此分支沒有像前兩個分支進行任何對 `id` 欄位的測試，任何數值都會配對到此模式。

使用 `@` 讓我們能在一個模式中測試數值並將其儲存至變數。

總結

Rust 的模式對於分辨不同種資料來說非常實用。當在 `match` 表達式中使用時，Rust 確保你的模式有涵蓋所有可能數值，不然你的程式就不會編譯通過。在 `let` 陳述式與函式參數中的模式使它們的結構更實用，在能夠解構數值成更小部分的同時賦值給變數。我們能夠建立簡單或複雜的模式來滿足我們的需求。

接下來，在本書的倒數第二章中，我們要來看 Rust 眾多特色中的一些進階部分。

進階特色

現在，你已經了解 Rust 程式語言最常用的部分。在開始做第二十章的另一個專案之前，先來了解你可能偶爾會遇到，但不一定天天用到的語言種種面向。當你碰到一些未知情況時，可以將本章作為技術文件查閱。本章涵蓋之特色僅在特定情況下有實用性。雖然可能不會經常碰到這些，但我們希望確保你能掌握 Rust 提供的所有特色。

本章涵蓋：

- 不安全的 Rust：如何選擇捨棄部分 Rust 提供的保證，並自行負責遵守這些保證
- 進階特徵：關聯型別 (associated type)、預設型別參數 (default type parameter)，完全限定語法 (fully qualified syntax)，超特徵 (supertrait)，以及跟特徵相關的新型別模式 (newtype pattern)
- 進階型別：更多有關新型別模式、型別別名 (type alias)，永不型別 (never type)，以及動態大小型別 (dynamically sized type)
- 進階函式與閉包：函式指標與回傳閉包
- 巨集 (macro)：一種定義「在編譯期定義程式碼的程式碼」之方法

這些 Rust 全功能特色適合所有人。一起來深究吧！

不安全的 Rust

到目前為止，我們討論的所有程式碼都在編譯期強制加上 Rust 記憶體安全保證。然而，Rust 內部其實隱藏了第二種語言，並不強制加上這些記憶體安全保證：這語言叫做**不安全 (unsafe)** 的 **Rust**，可和常規 Rust 一樣正常執行，同時賦予我們極強的能力。

不安全的 Rust 之所以存在，是由於靜態分析本質上過於保守。當編譯器嘗試確認程式碼是否遵守這些安全保證時，比起接受一些非法的程式，更寧願拒絕部分有效的程式。儘管有些程式碼看起來正確，但 Rust 無法獲取足夠資訊保證的話，它就是會擋下來。在這些案例中，你可以寫不安全程式碼並告訴編譯器：「相信我，我知道我在幹麻。」從反面來看這也有缺點，你必須自行承擔風險：若誤用不安全程式碼，可能會造成記憶體不安全，例如發生對空指標 (null pointer) 解參考。

Rust 擁有另一個不安全的自我的另一理由是電腦硬體本質上就不安全。如果 Rust 不允許這些不安全操作，就無法完成特定任務。Rust 必須允許你做這些底層系統程式設計，例如直接與作業系統互動，甚至撰寫自己的作業系統。系統程式設計是這個語言的目標之一，一起探索我們可以用不安全的 Rust 做什麼和如何使用吧。

不安全的超能力

欲切換成不安全的 Rust，可使用 `unsafe` 關鍵字開啟一個新程式碼區塊，並封裝這些不安全程式碼。在不安全的 Rust，你可使用在安全的 Rust 之下無法使用的五種功能，我們稱之為**不安全的超能力**。這些超能力包含：

- 對裸指標 (raw pointer) 解參考
- 呼叫不安全函式或方法
- 存取或修改可變的靜態變數 (static variable)
- 實作不安全特徵 (trait)
- 存取聯合體 (union) 的欄位

需要謹記在心的是，`unsafe` 並不會關閉借用檢查器 (borrow checker) 或是停用其他 Rust 的安全檢查：在不安全程式碼中操作一個參考仍然會經過檢查。`unsafe` 關鍵字只提供上述不經由編譯器檢查記憶體安全的五項功能，在不安全區塊內你依然保有一定程度的安全性。

此外，`unsafe` 並不意味在此區塊內的程式碼一定有風險或有記憶體安全問題：其目的是作為一個程式設計師，你必須確保在 `unsafe` 區塊內的程式碼透過有效途徑存取記憶體。

錯誤因人類不可靠而發生。不過，將五種不安全操作標記在 `unsafe` 區塊內，讓你得知任何記憶體安全相關的錯誤一定在某個 `unsafe` 內。請將 `unsafe` 區塊保持夠小，當你在調查一個記憶體錯誤時，會慶幸當初有這麼做。

為了盡可能隔離不安全程式碼，最佳作法是將之封裝在安全的抽象並提供安全的 API，本章在後面的探討不安全函式和方法一併討論之。部分的標準函式庫同樣是在審核過的不安全程式碼上提供安

全抽象。透過安全抽象封裝不安全程式碼，可防止你或你的使用者使用以 `unsafe` 實作的功能，不會將實際的 `unsafe` 使用洩漏到四散各地，因為安全抽象就是安全的 Rust。

接下來將依序探討這五個不安全的超能力。也會看看一些替不安全程式碼提供安全介面的抽象。

對裸指標解參考

在第四章「[迷途參考](#)」一節，我們提及編譯器確保參考一定是有效的。不安全的 Rust 有兩種新型別叫**裸指標**，和參考非常相似。和參考一樣，裸指標能是不可變或可變，分別寫做 `*const T` 和 `*mut T`。星號不是解參考運算子，它就是型別名稱的一部分。在裸指標的脈絡下，**不可變**代表指標不能在解參考之後直接賦值。

和參考與智慧指標（smart pointer）不同，裸指標是：

- 允許忽略借用規則，同時可存在指向相同位置的可變和不可變的指標，或是多個可變指標
- 不能保證一定指向有效記憶體
- 可以為空（null）
- 並無實作任何自動清理機制

在停用 Rust 的保證之後，你能透過放棄這些安全性保證換得更高的效能，或是介接其他語言與硬體等無法套用 Rust 安全保證的場景。

範例 19-1 展示了如何從參考分別建立不可變和可變的裸指標。

```
let mut num = 5;

let r1 = &num as *const i32;
let r2 = &mut num as *mut i32;
```

範例 19-1：從參考建立裸指標

注意，這段程式碼並無使用 `unsafe` 關鍵字。我們可以在安全程式碼中建立裸指標，我們只是不能在不安全區塊外對其解參考，你很快就會看到。

我們透過 `as` 將不可變與可變參考轉型成個別對應的裸指標。由於這些裸指標是從保證有效的參考而來，就能得知這些裸指標同樣有效，但我們無法推導所有裸指標都有效。

為了展示上述情形，接下來，我們將建立無法確認有效性的裸指標，範例 19-2 展示了如何從任意記憶體的位置建立裸指標。嘗試使用任意的記憶體行為並未定義，該位址上可能有也可能沒資料，且編譯器可能會最佳化該程式，所以該處可能不會存取記憶體，或是程式因區段錯誤導致崩潰。一般情況下，雖然這種程式碼能寫得出來，但不會有任何好理由寫出它。

```
let address = 0x012345usize;
let r = address as *const i32;
```

範例 19-2：從任意記憶體位址建立裸指標

回想一下，我們可以在安全的程式碼下建立裸指標，但我們不能對裸指標解參考並讀取它指向的資料。範例 19-3 我們對裸指標使用參考運算子 `*` 需要封裝在 `unsafe` 區塊內。

```
let mut num = 5;

let r1 = &num as *const i32;
let r2 = &mut num as *mut i32;

unsafe {
    println!("r1 為:{}", *r1);
    println!("r2 為:{}", *r2);
}
```

範例 19-3：在 `unsafe` 區塊對裸指標解參考

建立一個指標沒有危險性，只有當我們嘗試存取它指向的值時，才可能需要處理非法的值。

請注意，範例 19-1 與 19-3，我們建立了 `*const i32` 與 `*mut i32` 兩個裸指標，皆指向相同儲存 `num` 的記憶體位置。若我們走正常程序建立指向 `num` 的不可變與可變參考，程式碼將因為 Rust 所有權規則不允許同時存在一個可變參考與多個不可變參考，進而無法編譯。有了裸指標，即可建立指向同個位置的可變指標和不可變指標，並透過可變指標改變其資料，但可能帶來資料競爭 (data races)，請小心！

既然有這些危險，為什麼你還要用裸指標呢？一個主要使用案例是與 C 程式碼介接，你將會在下一節「[呼叫不安全函式或方法](#)」讀到。另一個用例是在借用檢查器不理解之處建立一層安全抽象。我們將會介紹不安全函式，再探討一個使用到不安全程式碼的安全抽象範例。

呼叫不安全函式或方法

第二種需要不安全區塊的操作是呼叫不安全函式。不安全函式與方法外觀看起來與正常函式及方法並無二致，僅在整個函式定義前多了額外的 `unsafe`。 `unsafe` 關鍵字在此脈絡下是指此函式在呼叫時必須遵守某些要求，因為 Rust 無法保證我們能達成這項要求。當我們在一個 `unsafe` 區塊內呼叫一個 `unsafe` 函式，意味著我們已閱讀此函式的文件，而且有責任遵守此函式的使用條款。

這裡有個不安全函式叫做 `dangerous`，函式本體內無任何東西：

```
unsafe fn dangerous() {}

unsafe {
    dangerous();
}
```

我們必須在單獨的 `unsafe` 區塊中呼叫 `dangerous` 函式，若不在 `unsafe` 區塊中呼叫，會得到一個錯誤：

```

$ cargo run
   Compiling unsafe-example v0.1.0 (file:///projects/unsafe-example)
error[E0133]: call to unsafe function is unsafe and requires unsafe function or block
  --> src/main.rs:4:5
   |
4  |     dangerous();
   |     ^^^^^^^^^^^ call to unsafe function
   |
   = note: consult the function's documentation for information on how to avoid undefined behavior

For more information about this error, try `rustc --explain E0133`.
error: could not compile `unsafe-example` due to previous error

```

藉由一個 `unsafe` 區塊，我們可以對 Rust 聲明我們閱讀過該函式的文件，理解如何合理使用它，並且驗證過我們已履行該函式的使用條款。

不安全函式本體與 `unsafe` 區塊等效，所以可以在該不安全函式執行其他不安全操作，不需再加 `unsafe` 區塊。

在不安全程式碼上建立安全的抽象

一個函式有不安全程式碼並不代表我們必須將整個函式標註為不安全。事實上，將不安全程式碼封裝在安全函式中一直是常見的抽象。我們來研讀標準函式庫的 `split_at_mut` 函式作為範例，它需要一些不安全程式碼。我們將探索如何實作之。這個安全方法定義在可變的切片上：它將一個切片在給定的索引引數（argument）上一分為二。範例 19-4 展示了如何使用 `split_at_mut`。

```

let mut v = vec![1, 2, 3, 4, 5, 6];

let r = &mut v[..];

let (a, b) = r.split_at_mut(3);

assert_eq!(a, &mut [1, 2, 3]);
assert_eq!(b, &mut [4, 5, 6]);

```

範例 19-4：使用一個安全的 `split_at_mut` 函式

我們不可能在 safe Rust 下實作這個函式。一個嘗試可能會像範例 19-5 無法編譯。為了簡化，我們將 `split_at_mut` 實作為一個函式而非方法，並且以 `i32` 取代泛型型別 `T`。

```
fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();

    assert!(mid <= len);

    (&mut values[..mid], &mut values[mid..])
}
```



範例 19-5：嘗試僅用安全的 Rust 實作 `split_at_mut`

這個函式先取得該切片的總長度，再來檢查從參數而來的索引小於等於該長度。這項檢查代表若我們傳入欲分割的索引位置大於該長度，這個函式會在嘗試使用該索引前就恐慌（panic）。

之後，我們回傳一個元組，其內包含兩個可變切片：一個從原始切片的起頭到 `mid` 索引位置，另一個則從 `mid` 到尾端。

當我們嘗試編譯範例 19-5 的程式碼，會得到一個錯誤。

```
$ cargo run
   Compiling unsafe-example v0.1.0 (file:///projects/unsafe-example)
error[E0499]: cannot borrow `*values` as mutable more than once at a time
--> src/main.rs:6:31
   |
1  | fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32])
   | {
   |                                     - let's call the lifetime of this reference `1`
...
6  |     (&mut values[..mid], &mut values[mid..])
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |     |         |                                     |
   |     |         |                                     second mutable borrow occurs here
   |     |         first mutable borrow occurs here
   |     returning this value requires that `*values` is borrowed for `1`
```

For more information about this error, try `rustc --explain E0499`.
error: could not compile `unsafe-example` due to previous error

Rust 的借用檢查器（borrow checker）不能理解我們同時借用一個切片的不同部分，它只認知到我們借用同一個切片兩次。借用同一個切片的不同部分基本上沒什麼問題，因為兩個切片不會重疊，但 Rust 不夠聰明以致無法理解這件事。當我們知道程式碼沒問題，但 Rust 並不知道，就是時候搞一點 不安全程式碼了。

範例 19-6 展示了如何使用一個 `unsafe` 區塊、一個裸指標，以及呼叫一些不安全函式來實作可成功執行的 `split_at_mut`

```

use std::slice;

fn split_at_mut(values: &mut [i32], mid: usize) -> (&mut [i32], &mut [i32]) {
    let len = values.len();
    let ptr = values.as_mut_ptr();

    assert!(mid <= len);

    unsafe {
        (
            slice::from_raw_parts_mut(ptr, mid),
            slice::from_raw_parts_mut(ptr.add(mid), len - mid),
        )
    }
}

```

範例 19-6：在 `split_at_mut` 函式實作中使用不安全程式碼

回憶一下第四章「切片型別」一節中，提及切片會儲存指向某些資料的指標以及該切片長度。我們可使用 `len` 方法取得切片的長度，並用 `as_mut_ptr` 取得指向切片的裸指標。在此範例中，由於我們擁有指向某些 `i32` 值的可變切片，`as_mut_ptr` 會回傳一個型別為 `*mut i32` 的裸指標，即是儲存在 `ptr` 變數中的值。

我們判定 `mid` 索引在該切片內。此後我們進入不安全程式碼：`slice::from_raw_parts_mut` 函式需要一個裸指標與一個長度，並建立一個切片。我們使用這個函式來建立一個從 `ptr` 開始長度為 `mid` 的切片。而後，我們以 `mid` 作為引數，對 `ptr` 呼叫 `add` 方法，以取得從 `mid` 開始的裸指標，再來用此指標與從 `mid` 開始剩下的元素個數作為長度，建立另一個切片。

`slice::from_raw_parts_mut` 之所以為不安全函式，是因為它需要裸指標，且必須相信這個指標有效。`add` 是不安全方法是由於它必須相信偏移後的位址是有效指標。因此，我們需要在呼叫 `slice::from_raw_parts_mut` 和 `add` 外包一層 `unsafe` 函式。透過閱讀程式碼與加上對 `mid` 一定等於或比 `len` 小的斷言，我們可以宣稱所有在 `unsafe` 區塊的裸指標都是指向原始切片內的有效指標。這是一個可接受且合理的 `unsafe` 使用情境。

注意，我們不需替 `split_at_mut` 函式輸出結果做上 `unsafe` 的記號，而且我們可以在安全的 Rust 呼叫它。我們藉由安全的方式使用 `unsafe` 函式，完成了對不安全程式碼建立一層安全抽象，這個抽象只會從該函式能夠存取的資料內建立有效指標。

對比之下，範例 19-7 中使用 `slice::from_raw_parts_mut` 則極有可能會在該切片被使用時崩潰。這段程式碼從任意的記憶體位置建立了一個 10,000 元素長的切片。

```
use std::slice;

let address = 0x01234usize;
let r = address as *mut i32;

let values: &[i32] = unsafe { slice::from_raw_parts_mut(r, 10000) };
```

範例 19-7：從任意記憶體位址建立切片

我們不擁有此位址之下的記憶體，且並不保證這段程式碼建立的切片一定包含有效的 `i32` 值。嘗試將 `values` 當作有效的切片來使用，會導致為未定義行為（undefined behavior）。

使用 `extern` 函式呼叫外部程式碼

有些時候，你的 Rust 程式碼可能需要與其他語言撰寫的程式碼互動。這種情況 Rust 提供 `extern` 關鍵字，予以協助建立與使用外部函式介面（Foreign Function Interface，FFI）。FFI 的功能是給在一門程式語言定義函式，使得另一門（外部）程式語言可以呼叫這些函式。

範例 19-8 展示了如何建立整合一個 C 標準函式庫的 `abs` 函式。由於其他語言並無強制遵守 Rust 的規則和保證，而且 Rust 也無法檢查之，因此在 Rust 程式碼中呼叫在 `extern` 區塊內宣告的函式一定是不安全的操作，所以確保安全的重責大任就會落在程式設計師身上。

檔案名稱：src/main.rs

```
extern "C" {
    fn abs(input: i32) -> i32;
}

fn main() {
    unsafe {
        println!("依據 C 所判斷 -3 的絕對值為：{}", abs(-3));
    }
}
```

範例 19-8：宣告並呼叫一個用其他語言定義的 `extern` 函式

在 `extern "C"` 區塊內，我們列出我們想要呼叫的，從其他語言而來的外部函式名稱與簽名。`"C"` 的部分定義了外部函式使用了哪個應用程式二進位制介面（ABI）：ABI 定義了在組合語言層級該如何呼叫此函式。`"C"` ABI 最為通用且遵循 C 程式語言的 ABI 規範。

從其他語言呼叫 Rust 函式

我們也可透過 `extern` 定義一個介面，允許其他語言呼叫 Rust 的函式。有別於建立整個 `extern` 區塊，我們會在 `fn` 關鍵字前加上 `extern` 關鍵字並指明應用程式二進位制介面

(ABI)。我們甚至可加上 `#[no_mangle]` 註記來告訴編譯器不要重整 (mangle) 該函式名稱。**重整**是一個編譯器透過改變我們賦予函式的名稱，成為帶有更多資訊的名稱進而提供給編譯過程使用，但人類就相對難以閱讀。每個程式語言編譯器重整名稱的作法有些許不同，因此必須關閉 Rust 編譯器的名稱重整功能。

接下來的範例，我們寫了 `call_from_c` 函式，可以在編譯成共享函式庫 (shared library) 且連結至 C 後，由 C 程式碼存取：

```
#[no_mangle]
pub extern "C" fn call_from_c() {
    println!("從 C 呼叫了一個 Rust 函式!");
}
```

這類的 `extern` 用途不需要 `unsafe`。

存取或修改可變的靜態變數

在本書中，我們還沒聊到**全域變數** (global variable)，這個 Rust 支援但會被 Rust 的所有權規則搞得七葷八素的功能。試想有兩個執行緒同時存取同一個可變全域變數，豈不導致資料競爭。

Rust 的全域變數稱做**靜態變數**。範例 19-9 展示了宣告並使用一個儲存字串切片的靜態變數。

檔案名稱：src/main.rs

```
static HELLO_WORLD: &str = "Hello, world!";

fn main() {
    println!("name 為: {}", HELLO_WORLD);
}
```

範例 19-9：定義並使用一個不可變的靜態變數

靜態變數 (static variable) 與我們在第三章「[變數與常數的差異](#)」一節討論的常數相似。慣例上靜態變數會用尖叫蛇式命名 (`SCREAMING_SNAKE_CASE`)。由於靜態變數只能儲存 `static` 生命週期的參考，代表 Rust 編譯器可推導出它的生命週期，不需要我們顯式標註。存取一個不可變的靜態變數是安全的。

常數和不可變靜態變數看似相同，實則有些許隱晦差異：靜態變數之值有固定的記憶體位址，使用該值永遠會存取相同的資料。反之，常數在使用上則可複製它們儲存的資料。

另一個常數與靜態變數的差異是，靜態變數可能是可變的。存取並修改可變的靜態變數並「**不安全**」。範例 19-10 展示了如何宣告、存取、修改一個可變的靜態變數 `COUNTER`。

檔案名稱：src/main.rs

```
static mut COUNTER: u32 = 0;

fn add_to_count(inc: u32) {
    unsafe {
        COUNTER += inc;
    }
}

fn main() {
    add_to_count(3);

    unsafe {
        println!("COUNTER: {}", COUNTER);
    }
}
```

範例 19-10：讀取與寫入可變的靜態變數為不安全的操作

與普通變數一樣，我們透過 `mut` 關鍵字指明可變性。任何讀寫 `COUNTER` 的程式碼皆必須在 `unsafe` 區塊中。這個程式碼會編譯並打印出我們預期中的 `COUNTER: 3` 是因為他在單執行緒執行，若在多執行緒存取 `COUNTER` 則可能導致資料競爭。

當能從全域存取可變資料時，確保沒有資料競爭就不容易了，這就是為什麼 Rust 將可變的靜態變數視為不安全。若是可能的話，我們推薦使用第十六章討論的並行技術與執行緒安全（thread-safe）的智慧指標（smart pointer），如此一來編譯器就能檢查從不同執行緒存取資料是安全的。

實作不安全特徵

我們可以用 `unsafe` 實作不安全特徵。當一個特徵是有至少一個方法包含編譯器無法驗證的不變條件（invariant），就稱該特徵不安全。我們透過在 `trait` 前加上 `unsafe` 關鍵字來宣告一個特徵為 `unsafe`，這也讓實作該特徵會變成 `unsafe`，如 19-11 所示。

```
unsafe trait Foo {
    // 內部的方法
}

unsafe impl Foo for i32 {
    // 內部實作的方法
}

fn main() {}
```

範例 19-11：定義並實作一個不安全特徵

透過 `unsafe impl`，我們承諾我們將會遵守這些編譯器無法驗證的不變條件（invariant）。

回想第十六章「[可延展的並行與 Sync 及 Send 特徵](#)」一節的兩個記號特徵（marker trait）

`Sync` 與 `Send`：若我們的型別是由 `Send` 與 `Sync` 組合而成，編譯器會自動實作這些特徵。若我們的型別包含一些非 `Send` 或 `Sync` 的型別，例如裸指標，但我們希望替型別做上 `Send` 或 `Sync` 的記號，就必須使用 `unsafe`。Rust 無法驗證我們的型別有遵守可以在多執行緒中傳遞或存取的保證。因而，我們需要自己手動檢查，並指明這是 `unsafe`。

存取聯合體的欄位

最後一個可以使用 `unsafe` 的地方是存取 `union` 的欄位。`union` 與 `struct` 十分相似，差異是在一個聯合體實例中僅儲存其中一個宣告的欄位。聯合體主要用在與 C 程式碼的聯合體介接。存取聯合體的欄位並不安全，由於 Rust 無法保證當前儲存在聯合體實例中的資料是什麼型別，因此存取聯合體的欄位並不安全。你可以從 [Rust 參考手冊](#) 了解更多關於聯合體的資訊。

何時該用不安全程式碼

透過 `unsafe` 使用上述五種功能（超能力）並沒有錯，更並非不能接受，但由於編譯期無法協助遵守記憶體安全，這讓 `unsafe` 程式碼要正確無誤略顯棘手。當你因故需要使用 `unsafe` 程式碼，就去用吧，並且記得替 `unsafe` 撰寫明確的註釋，讓有問題發生時更容易追蹤查找源頭。

進階特徵

我們在第十章「[特徵：定義共享行為](#)」一節首次提及特徵 (trait)，但對其進階細節並無著墨。現在你已熟稔 Rust，了解箇中真諦的時機已至。

利用關聯型別在特徵定義中指定佔位符型別

關聯型別 (associated types) 連結了一個型別佔位符 (placeholder) 與一個特徵，可以將這些佔位符型別使用在這些特徵所定義的方法簽名上。對特定實作來說，特徵的實作者將指明一個具體型別來代替型別佔位符。如此一來，我們可以定義一個使用了某個型別的特徵，但直到特徵被實作之前，都不需知道實際上的型別。

多數在本章提及的進階特色都較少使用，而關聯型別則是介於其中：他們比書中其他內容來得少用，但比本章介紹的其他特色來得更常見。

一個具有關聯型別的特徵之範例是標準函式庫提供的 `Iterator` 特徵。這例子中的關聯型別叫做 `Item`，表示一型別實作 `Iterator` 特徵時，會被疊代的那些值的型別。範例 19-12 展示了 `Iterator` 特徵的定義：

```
pub trait Iterator {  
    type Item;  
  
    fn next(&mut self) -> Option<Self::Item>;  
}
```

範例 19-12：`Iterator` 特徵自帶一個關聯型別

`Item` 型別是個佔位符，`next` 方法的定義顯示它會回傳型別為 `Option<Self::Item>` 之值。`Iterator` 特徵的實作者會指定 `Item` 的具體型別，而 `next` 方法則會回傳一個包含該具體型別的值的一個 `Option`。

關聯型別可能看起來和泛型的概念非常相似，而後者允許定義函式而不需指定該函式可以處理何種型別。為了檢視以下兩者概念上的差異，我們來看這個替型別 `Counter` 上實作 `Iterator` 特徵，且 `Counter` 指定的 `Item` 的型別為 `u32`：

檔案名稱：`src/lib.rs`

```
impl Iterator for Counter {  
    type Item = u32;  
  
    fn next(&mut self) -> Option<Self::Item> {  
        // --snip--  
    }
```

語法似乎和泛型很像，所以為什麼我們不使用泛型定義 `Iterator` 特徵，如範例 19-13 所示？

```
pub trait Iterator<T> {  
    fn next(&mut self) -> Option<T>;  
}
```

範例 19-13: 假設使用泛型來定義 `Iterator` 特徵

差別在於使用泛型時，如範例 19-13 所示，由於我們可以實作出 `Iterator<String> for Counter` 或以任意多個其他泛型型別來替 `Counter` 實作 `Iterator`，因此，必須在每個實作都標註該型別。換言之，當一特徵擁有泛型參數，一型別即可透過改變泛型型別參數（generic type parameter）的具體型別，進而實作該特徵多次。於是，當我們使用 `next` 方法時，必須提供型別標註，指名要用哪個 `Iterator` 的實作。

有了關聯型別，同個型別就不能實作同個特徵多次，所以我們不需要標註型別。範例 19-12 中的定義用上了關聯型別，因為只能擁有一個 `impl Iterator for Counter`，於是只能替 `Item` 選擇唯一一個型別。在任何地方呼叫 `Counter` 的 `next` 方法就不必再明確指定我們想要 `u32` 疊代器了。

關聯型別也會成為該特徵的條款：該特徵的實作者必須替關聯型別佔位符提供一個型別。關聯型別通常該有個能解釋自己如何被用的名字，而且替關聯型別加上 API 技術文件會是個不錯的實踐。

預設泛型型別參數與運算子重載

我們可以透過泛型型別參數（generic type parameter）指定該泛型型別預設的具體型別。在預設型別可運作的情形下，這可省去實作者需要指定具體型別的勞動。替泛型型別指定預設型別的語法是在宣告泛型型別時寫成 `<PlaceholderType=ConcreteType>`。

運算子重載（operator overloading）就是一個使用這個技術的好例子。你可以在特定情況下自訂運算子（如 `+`）的行為。

Rust 不允許建立你自己的運算子或重載任意的運算子，但你可以透過實作 `std::ops` 表列出的特徵與相關的運算子，來重載特定運算與相應特徵。在範例 19-14 我們重載了 `+` 運算子，讓兩個 `Point` 實例可相加。這個功能是透過對 `Point` 結構體實作 `Add` 特徵來達成：

檔案名稱：src/main.rs

```

use std::ops::Add;

#[derive(Debug, Copy, Clone, PartialEq)]
struct Point {
    x: i32,
    y: i32,
}

impl Add for Point {
    type Output = Point;

    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}

fn main() {
    assert_eq!(
        Point { x: 1, y: 0 } + Point { x: 2, y: 3 },
        Point { x: 3, y: 3 }
    );
}

```

範例 19-14：藉由實作 `Add` 特徵，重載 `Point` 實例的 `+` 運算子

`add` 方法將兩個 `Point` 實例的 `x` 值相加，兩個 `y` 值相加，並建立新的 `Point` 實例。`Add` 特徵有個關聯型別 `Output` 可以決定 `add` 方法回傳的型別。

這段程式碼的預設泛型型別寫在 `Add` 特徵中，定義如下：

```

trait Add<Rhs=Self> {
    type Output;

    fn add(self, rhs: Rhs) -> Self::Output;
}

```

這段程式碼大體上看起來很眼熟：具有一個方法與一個關聯型別的特徵。新朋友是 `Rhs=Self`，這部分叫做**預設型別參數 (default type parameter)**。`Rhs` 泛型參數（「右運算元 right hand side」的縮寫）定義了 `add` 方法中 `rhs` 參數的型別。若我們未在實作 `Add` 特徵時指定 `Rhs` 的具體型別，這個 `Rhs` 的型別預設會是 `Self`，也就是我們正在實作 `Add` 的型別。

當我們對 `Point` 實作 `Add`，因為我們想要將兩個 `Point` 實例相加，所以用到預設的 `Rhs`。讓我們看一個實作 `Add` 的範例，如何不用預設值，轉而自訂 `Rhs`。

我們有兩個結構體，`Millimeters` 與 `Meters`，分別儲存不同單位的值。這樣在現有的型別簡單地封裝進另一個結構體的模式叫做**新型別模式 (newtype pattern)**，我們會在「[使用新型別模式替外部型別實作外部特徵](#)」段落做詳細介紹。我們想將公釐透過 `Add` 做好正確單位轉換來加至公尺，這可透過對 `Millimeters` 實作 `Add` 並將 `Rhs` 設為 `Meters` 達成，如範例 19-15。

檔案名稱：`src/lib.rs`

```
use std::ops::Add;

struct Millimeters(u32);
struct Meters(u32);

impl Add<Meters> for Millimeters {
    type Output = Millimeters;

    fn add(self, other: Meters) -> Millimeters {
        Millimeters(self.0 + (other.0 * 1000))
    }
}
```

範例 19-15：藉由替 `Millimeters` 實作 `Add` 特徵，使 `Millimeters` 可與 `Meters` 相加

欲相加 `Millimeters` 與 `Meters`，先將 `Rhs` 型別參數指定為 `impl Add<Meters>`，替代預設的 `Self`。

你會在下列兩種情況下使用預設型別參數：

- 擴充一個型別但不破壞既有程式碼
- 提供大多數使用者不會需要的特殊狀況之自訂空間

標準函式庫是第二種情況的範例：通常你會將兩個相同的型別相加，但 `Add` 特徵提供超乎預設的自訂能力。`Add` 特徵定義中的預設型別參數讓我們大多數時候不需要指定額外的參數。換句話說，不用再寫部分重複的樣板，讓該特徵更易用。

第一種情況和第二種類似，但概念相反：若你想替既有特徵加上新的型別參數，可以給它一個預設值，允許擴充該特徵的功能，而不破壞既有的程式實作。

消除歧義的完全限定語法：呼叫同名的方法

Rust 並沒有限制不同特徵之間不能有同名的方法，也沒有阻止你對同一個型別實作這兩個特徵。有可能實作一個型別，其擁有多個從多個特徵而來的同名方法的型別。

當呼叫這些同名方法，你必須告訴 Rust 你想呼叫誰。試想範例 19-16 的程式碼，我們定義了兩個特徵 `Pilot` 與 `Wizard`，兩者都有 `fly` 方法。當我們對一個已經擁有 `fly` 方法的 `Human` 型別分別實作這兩個特徵時，每個 `fly` 方法的行為皆不同。

檔案名稱：`src/main.rs`

```

trait Pilot {
    fn fly(&self);
}

trait Wizard {
    fn fly(&self);
}

struct Human;

impl Pilot for Human {
    fn fly(&self) {
        println!("這裡是艦長發言。");
    }
}

impl Wizard for Human {
    fn fly(&self) {
        println!("起!");
    }
}

impl Human {
    fn fly(&self) {
        println!("*狂揮雙臂*");
    }
}

```

範例 19-16：Human 分別實作了兩個特徵的 `fly` 方法，且 `Human` 自己實作了一個 `fly` 方法

當我們對一個 `Human` 實例呼叫 `fly`，編譯器預設會呼叫直接在該型別上實作的方法，如範例 19-17 所示：

檔案名稱：src/main.rs

```

fn main() {
    let person = Human;
    person.fly();
}

```

範例 19-17：對 `Human` 實例呼叫 `fly`

執行這段程式碼會印出 `*狂揮雙臂*`，表示 Rust 呼叫直接在 `Human` 上實作的 `fly` 方法。

欲呼叫在 `Pilot` 或 `Wizard` 特徵上的 `fly` 方法，我們要用更明確的語法指定我們想要的 `fly` 方法。範例 19-18 展示了這個語法。

檔案名稱：src/main.rs

```
fn main() {  
    let person = Human;  
    Pilot::fly(&person);  
    Wizard::fly(&person);  
    person.fly();  
}
```

範例 19-18：指定想要呼叫哪個特徵的 `fly` 方法

在你要呼叫的方法名前指定特徵名稱，可以讓 Rust 清楚得知我們要呼叫哪個實作 `fly`。我們也可以寫成 `Human::fly(&person)`，同義於在範例 19-18 的 `person.fly()`，只是為了消歧義而寫得長一點罷了。

執行這段程式碼會印出：

```
$ cargo run  
    Compiling traits-example v0.1.0 (file:///projects/traits-example)  
    Finished dev [unoptimized + debuginfo] target(s) in 0.46s  
    Running `target/debug/traits-example`  
這裡是艦長發言。  
起！  
*狂揮雙臂*
```

因為 `fly` 方法有個 `self` 參數，所以我們若有兩個**型別**都實作了同個特徵，Rust 可以透過 `self` 的型別理出該用哪個特徵的實作。

然而不屬於方法的關聯函式（associated function）則沒有 `self` 參數。當同個作用域下的多個型別或特徵皆定義了非方法且同名的函式時，除非使用**完全限定語法**（fully qualified syntax），否則 Rust 無法推斷你指涉哪個型別。舉例來說，範例 19-19 中，我們替動物庇護所建立了一個特徵，會將所有小狗狗命名為**小不點**。我們建立 `Animal` 特徵並帶有一個關聯非方法的函式 `baby_name`。 `Animal` 特徵有個在 `Dog` 上的實作，也提供了關聯非方法的函式 `baby_name`。

檔案名稱：src/main.rs

```

trait Animal {
    fn baby_name() -> String;
}

struct Dog;

impl Dog {
    fn baby_name() -> String {
        String::from("小不點")
    }
}

impl Animal for Dog {
    fn baby_name() -> String {
        String::from("小狗狗")
    }
}

fn main() {
    println!("幼犬被稱作{}", Dog::baby_name());
}

```

範例 19-19: 一個特徵和一個型別分別擁有同名關聯函式，並且該型別實作了該特徵

我們在 `Dog` 中的 `baby_name` 關聯函式實作了一段程式碼，將所有小狗狗命名為小不點。這個 `Dog` 型別同時實作了 `Animal` 特徵，`Animal` 特徵則描述了所有動物都有的習性。在實作了 `Animal` 特徵的 `Dog` 上，透過與 `Animal` 特徵關聯的 `baby_name` 函式中，表達了幼犬被稱作小狗狗這一概念。

在 `main` 中我們呼叫 `Dog::baby_name` 函式，最終會直接呼叫 `Dog` 上的關聯函式。這段程式碼會印出：

```

$ cargo run
Compiling traits-example v0.1.0 (file:///projects/traits-example)
Finished dev [unoptimized + debuginfo] target(s) in 0.54s
Running `target/debug/traits-example`
幼犬被稱作小不點

```

這個輸出結果不符合我們預期。我們想呼叫的 `baby_name` 函式應該是我們在 `Dog` 上實作的 `Animal` 特徵，所以程式碼應該印出 `幼犬被稱作小狗狗`。我們在範例 19-18 所使用的指明特徵的技巧不適用於此，如果我們更改 `main` 成範例 19-20，會得到一個編譯錯誤：

檔案名稱：src/main.rs

```

fn main() {
    println!("幼犬被稱為{}", Animal::baby_name());
}

```

範例 19-20：嘗試呼叫 `Animal` 特徵上的 `baby_name` 函式，但 Rust 不知道該用哪個實作

因為 `Animal::baby_name` 沒有 `self` 參數，且其他型別也可能有 `Animal` 的實作，所以 Rust 無法推斷出我們想要哪個 `Animal::baby_name` 實作。我們會得到這個編譯錯誤：

```
$ cargo run
   Compiling traits-example v0.1.0 (file:///projects/traits-example)
error[E0790]: cannot call associated function on trait without specifying the
corresponding `impl` type
  --> src/main.rs:20:43
   |
2  |     fn baby_name() -> String;
   |     ----- `Animal::baby_name` defined here
...
20 |     println!("A baby dog is called a {}", Animal::baby_name());
   |                                           ^^^^^^^^^^^^^^^^^^^^^^^^^^^ cannot call
associated function of trait
   |
help: use the fully-qualified path to the only available implementation
   |
20 |     println!("A baby dog is called a {}", <::Dog as Animal>::baby_name());
   |                                           ++++++++ +

For more information about this error, try `rustc --explain E0790`.
error: could not compile `traits-example` due to previous error
```

欲消除歧義，告訴 Rust 我們想用 `Dog` 上的 `Animal` 實作，而不是其他型別的 `Animal` 實作，我們必須使用完全限定語法。範例 19-21 展示了如何使用完全限定語法。

檔案名稱：src/main.rs

```
fn main() {
    println!("幼犬被稱作{}", <Dog as Animal>::baby_name());
}
```

範例 19-21：使用完全限定語法指定呼叫實作了 `Animal` 的 `Dog` 上的 `baby_name` 函式

我們提供一個用角括號包住的型別詮釋（type annotation），這個詮釋透過將此函式呼叫的 `Dog` 型別視為 `Animal`，來指明我們想要呼叫有實作 `Animal` 特徵的 `Dog` 上的 `baby_name` 方法。這段程式碼現在會印出我們所要的：

```
$ cargo run
   Compiling traits-example v0.1.0 (file:///projects/traits-example)
   Finished dev [unoptimized + debuginfo] target(s) in 0.48s
   Running `target/debug/traits-example`
幼犬被稱作小狗狗
```


普遍來說，完全限定語法定義如下：

```
<Type as Trait>::function(receiver_if_method, next_arg, ...);
```

對於不是方法的關聯函式來說，不會有 `receiver`，只會有其他引數的列表。你可以在任何呼叫函式或方法之處使用完全限定語法。然而，你亦可在 Rust 能透過程式其他資訊推斷出的地方省略這個語法。只需要在有多個同名實作且需要協助 Rust 指定呼叫哪個實作時，才需要使用這囉嗦冗長的語法。

使用超特徵要求在一個特徵內有另一特徵的功能

有些時候，你會定義一個依賴到其他特徵的特徵：對於已實作第一個特徵的型別，你要求其同時實作第二個特徵。當你做了這件事，你的特徵定義就可以利用第二個特徵的關聯項目。你定義的特徵依賴的特徵就稱為**超特徵** (supertrait)。

假設我們想要建立一個 `OutlinePrint` 特徵，它有一個 `outline_print` 方法會印出一個被星號包圍的值。換句話說，給定一個實作 `Display` 而會產生 `(x, y)` 的 `Point` 結構體，當我們對 `x` 為 1，`y` 為 3 的 `Point` 實例呼叫 `outline_print`，它印出如下：

```
*****
*           *
* (1, 3) *
*           *
*****
```

在這個 `outline_print` 實作中，我們想要使用到 `Display` 特徵的功能。因此，我們需要指明 `OutlinePrint` 特徵只會在型別同時實作 `Display` 且提供 `OutlinePrint` 所需功能時才會成功。這件事可以在特徵定義中做到，透過指明 `OutlinePrint: Display`。這項技巧很類似特徵上的特徵約束 (trait bound)。範例 19-22 展示了 `OutlinePrint` 特徵的實作。

檔案名稱：src/main.rs

```
use std::fmt;

trait OutlinePrint: fmt::Display {
    fn outline_print(&self) {
        let output = self.to_string();
        let len = output.len();
        println!("{}", "*".repeat(len + 4));
        println!("*{}*", " ".repeat(len + 2));
        println!("* {} *", output);
        println!("*{}*", " ".repeat(len + 2));
        println!("{}", "*".repeat(len + 4));
    }
}
```

範例 19-22: 實作要求 `Display` 功能的 `OutlinePrint` 特徵

因為我們已指明 `OutlinePrint` 需要 `Display` 特徵，且只要有實作 `Display` 的型別都會自動實作 `to_string` 這個函式，所以我們可以使用 `to_string`。若我們嘗試使用 `to_string` 但並沒有在該特徵後加上冒號並指明 `Display`，會得到一個錯誤，告訴我們在當前作用域下的 `&Self` 型別找不到名為 `to_string` 函數。

我們嘗試看看在一個沒有實作 `Display` 的型別上實作 `OutlinePrint`（如 `Point` 結構體）會發生什麼事：

檔案名稱：src/main.rs

```
struct Point {  
    x: i32,  
    y: i32,  
}  
  
impl OutlinePrint for Point {}
```



於是得到 `Display` 為必須但沒實作的錯誤：

```
$ cargo run  
Compiling traits-example v0.1.0 (file:///projects/traits-example)  
error[E0277]: `Point` doesn't implement `std::fmt::Display`  
--> src/main.rs:20:6  
|  
20 | impl OutlinePrint for Point {}  
|      ^^^^^^^^^^^^^^^^^ `Point` cannot be formatted with the default formatter  
|  
= help: the trait `std::fmt::Display` is not implemented for `Point`  
= note: in format strings you may be able to use `{:?}` (or `{:#?}` for  
pretty-print) instead  
note: required by a bound in `OutlinePrint`  
--> src/main.rs:3:21  
|  
3 | trait OutlinePrint: fmt::Display {  
|      ^^^^^^^^^^^^^^^^^ required by this bound in `OutlinePrint`  
  
For more information about this error, try `rustc --explain E0277`.  
error: could not compile `traits-example` due to previous error
```

我們可以透過對 `Point` 實作 `Display` 並滿足 `OutlinePrint` 要求的約束 (constraint)，如下：

檔案名稱：src/main.rs

```
use std::fmt;

impl fmt::Display for Point {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "({}, {})", self.x, self.y)
    }
}
```

於是對 `Point` 實作 `OutlinePrint` 特徵就成功編譯，我們即可以對 `Point` 實例呼叫 `outline_print` 來顯示一個星號外框框住它。

使用新型別模式替外部型別實作外部特徵

在第十章「為型別實作特徵」一節中，我們提及孤兒規則 (orphan rule)，這個規則指出只要型別或特徵其一是在本地的 crate 中定義，就允許我們對該型別實作該特徵。使用**新型別模式**

(**newtype pattern**)，即可繞過這項規則，此模式涉及建立一個元組結構體 (tuple struct) 型別（我們在「使用無名稱欄位的元組結構體來建立不同型別」說明了元組結構體）。元組結構體包含一個欄位，在我們想要實作該特徵的型別外作一層薄薄的封裝。這封裝型別對 crate 來說算作在本地定義，因此可以對該封裝實作該特徵。**新型別**是一個源自 Haskell 程式語言的術語。使用此模式不會有任何執行時效能的耗損，這個封裝型別會在編譯期刪略。

舉個例子，我們想要對 `Vec<T>` 實作 `Display`，但孤兒規則限制我們不能這樣做，因為 `Display` 特徵與 `Vec<T>` 都是在我們的 crate 之外定義。我們可以建立一個 `Wrapper` 結構體，帶有一個 `Vec<T>` 實例，接下來再對 `Wrapper` 實作 `Display` 並使用 `Vec<T>` 之值，如範例 19-23 所示。

檔案名稱：src/main.rs

```
use std::fmt;

struct Wrapper(Vec<String>);

impl fmt::Display for Wrapper {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}

fn main() {
    let w = Wrapper(vec![String::from("hello"), String::from("world")]);
    println!("w = {}", w);
}
```

範例 19-23：建立一個 `Wrapper` 型別封裝 `Vec<String>` 以實作 `Display`

因為 `Wrapper` 是一個元組結構體而 `Vec<T>` 是該元組在索引 0 上的項目，所以該 `Display` 的實作使用 `self.0` 存取內部的 `Vec<T>`。我們就可以在 `Wrapper` 上使用 `Display` 的功能了。

使用這個技術的缺點是 `Wrapper` 是個新型別，並無它封裝的值所擁有的方法。我們不得不在 `Wrapper` 上實作所有 `Vec<T>` 的方法，委派這些方法給 `self.0`，讓我們可以將 `Wrapper` 作為 `Vec<T>` 一樣對待。如果我們想要新型別得到所有內部型別擁有的所有方法，一個解法是透過對 `Wrapper` 實作 `Deref` 特徵（在第十五章「[透過 Deref 特徵將智慧指標視為一般參考](#)」一節有相應討論）並回傳內部型別。如果我們不想要 `Wrapper` 擁有所有內部型別的方法，例如限制 `Wrapper` 型別之行為，就僅須實作那些我們想要的方法。

現在，你知道如何將新型別模式與特徵相關聯，縱使不涉及特徵，新型別模式仍非常實用。接下來我們將目光轉移到其他與 Rust 型別系統互動的方法吧。

進階型別

至此，我們提及 Rust 型別系統的諸多特色，不過尚未深入討論。本章將從一般角度切入討論新型別（newtype）並檢驗為何作為型別來說，新型別非常好用。再來，接續看看型別別名（type alias）這個類似新型別但語意上不盡相同的特色。我們也會探討 [！ 型別與動態大小型別](#)（dynamically sized type）。

透過新型別模式達成型別安全與抽象

注意：接下來一節假定你已閱讀前面的章節 [「使用新型別模式替外部型別實作外部特徵」](#)。

目前為止，我們討論過的任務中，新型別模式皆游刃有餘，包括靜態強制不讓值被混淆，同時能表示該值的單位。在範例 19-15 可以見到如何善用新型別表示該值的單位：回憶一下，`Millimeters` 與 `Meters` 將 `u32` 的值封裝在新型別內，若我們寫了一個函式需要型別為 `Millimeters` 的參數，我們不可能編譯出一支可以誤傳 `Meters` 型別或 `u32` 來呼叫這個函式的程式。

我們也可以善用新型別是替一個型別的實作細節建立抽象層：如果我們直接將新型別作為限制可用功能的手段，新型別就可以公開有別於私有內部型別的 API。

新型別也可以隱藏內部實作。例如，我們可以提供 `People` 型別，封裝用來儲存人們的 ID 與姓名之間的關聯的 `HashMap<i32, String>`。使用 `People` 的程式碼僅能與我們提供的公開 API 互動，例如透過一個方法替 `People` 集合添加名字字串，這段程式碼就不需知道內部會將 `i32` 作為 ID 並映射到姓名上。我們在第十七章的 [「隱藏實作細節的封裝」](#) 一節也曾提及，利用新型別模式來達到封裝與隱藏實作細節，不失為一種輕量的方法。

透過型別別名建立型別同義詞

Rust 提供了替一個既有型別宣告型別別名的方式。對此我們會使用 `type` 關鍵字，例如我們可以建立 `i32` 的別名 `Kilometers`，如範例所示：

```
type Kilometers = i32;
```

現在，`Kilometers` 別名就是 `i32` 的同義詞。不像我們在範例 19-15 建立的 `Millimeters` 與 `Meters` 型別，`Kilometers` 並非獨立的新型別。型別為 `Kilometers` 的值會被當作型別是 `i32` 的值。

```

type Kilometers = i32;

let x: i32 = 5;
let y: Kilometers = 5;

println!("x + y = {}", x + y);

```

由於 `Kilometers` 與 `i32` 實際上是同個型別，所以兩者可以相加，我們也可以將 `Kilometers` 值傳入需要 `i32` 參數的函式。然而，這種作法並不像前面討論的新型別模式一樣有益於型別檢查。也就是說，如果我們在某處混用 `Kilometers` 和 `i32`，編譯器不會給予任何錯誤。

型別同義詞的主要使用情境在於減少重複。例如我們有一個又臭又長的型別：

```
Box<dyn Fn() + Send + 'static>
```

到處在函式簽名與型別註解寫這個型別既累人又容易失誤。想像你有一個專案的程式碼都長得像範例 19-24。

```

let f: Box<dyn Fn() + Send + 'static> = Box::new(|| println!("嗨"));

fn takes_long_type(f: Box<dyn Fn() + Send + 'static>) {
    // --省略--
}

fn returns_long_type() -> Box<dyn Fn() + Send + 'static> {
    // --省略--
}

```

範例 19-24：在多處使用很長的型別

使用型別別名減少重複，讓程式碼更可控。範例 19-25，我們替落落長的型別導入一個 `Thunk` 別名，所有用到該型別之處都能用短小的 `Thunk` 替代。

```

type Thunk = Box<dyn Fn() + Send + 'static>;

let f: Thunk = Box::new(|| println!("嗨"));

fn takes_long_type(f: Thunk) {
    // --省略--
}

fn returns_long_type() -> Thunk {
    // --省略--
}

```

範例 19-25：導入型別別名 `Thunk` 來減少重複

這段程式碼更容易讀寫了！選擇有意義的型別別名也有助於溝通傳達你的意圖（*think* 是一個表示會在未來對此程式碼求值，所以很適用表達儲存起來的閉包）。

型別別名同樣十分常用在 `Result<T, E>` 來減少重複。試想標準函式庫的 `std::io` 模組，輸入輸出（I/O）操作通常會藉由回傳 `Result<T, E>` 來處理失敗的操作。標準函式庫有個 `std::io::Error` 結構體來表示所有可能的 I/O 錯誤。許多在 `std::io` 內的函式會回傳 `E` 為 `std::io::Error` 的 `Result<T, E>`，例如這些 `Write` 特徵下的函式：

```
use std::fmt;
use std::io::Error;

pub trait Write {
    fn write(&mut self, buf: &[u8]) -> Result<usize, Error>;
    fn flush(&mut self) -> Result<(), Error>;

    fn write_all(&mut self, buf: &[u8]) -> Result<(), Error>;
    fn write_fmt(&mut self, fmt: fmt::Arguments) -> Result<(), Error>;
}
```

這些 `Result<..., Error>` 不斷重複，有鑑於此，`std::io` 宣告了這個型別的別名：

```
type Result<T> = std::result::Result<T, std::io::Error>;
```

由於這個宣告是在 `std::io` 模組內，因此我們可直接使用完全限定的別名

`std::io::Result<T>`，實際上就是 `E` 預先填入 `std::io::Error` 的 `Result<T, E>`。最終，`Write` 特徵的函式簽名就會長得這樣：

```
pub trait Write {
    fn write(&mut self, buf: &[u8]) -> Result<usize>;
    fn flush(&mut self) -> Result<()>;

    fn write_all(&mut self, buf: &[u8]) -> Result<()>;
    fn write_fmt(&mut self, fmt: fmt::Arguments) -> Result<()>;
}
```

型別別名有助於兩個面向：讓程式碼更容易撰寫，且對所有 `std::io` 提供一致的介面。因為它僅僅是別名，所以就是一個 `Result<T, E>` 而已，這意味著我們能使用任何可與 `Result<T, E>` 互動的方法，以及使用類似 `?` 運算子這種特殊語法。

永不回傳的永不型別

Rust 有一個特殊的型別叫做 `!`，由於它沒有任何值，在型別理論的行話中又稱為空型別（empty type）。不過我們更喜歡稱之為永不型別（never type），因為當一個函式永遠不會回傳，永不型別將會替代原本的回傳型別。這裡來個範例：

```
fn bar() -> ! {
    // --省略--
}
```

這段程式碼可讀作「函式 `bar` 永不回傳」。永不回傳的函數稱為**發散函式**（diverging function），我們無法建立 `!` 型別，所以 `bar` 永遠無法回傳。

不過，若永遠無法替這個型別建立值，那要這個型別幹嘛呢？回想一下，範例 2-5 的猜數字程式碼，在我們的範例 19-26 又重現了。

```
let guess: u32 = match guess.trim().parse() {
    Ok(num) => num,
    Err(_) => continue,
};
```

範例 19-26：match 其中一個分支結束在 `continue`

當時我們跳過了這段程式碼的一些細節。在第六章「`match` 控制流運算子」一節，我們探討了每個 `match` 分支必須回傳相同的型別，所以，例如以下程式碼就不能執行：

```
let guess = match guess.trim().parse() {
    Ok(_) => 5,
    Err(_) => "hello",
};
```



這段程式碼中 `guess` 的型別必須是**同時是整數與字串**，並且 Rust 要求 `guess` 只能是一種型別。那 `continue` 回傳了什麼？範例 19-26 中，為什麼允許一個分支回傳 `u32` 但同時有另一分支結束在 `continue`？

如你所猜，`continue` 具有 `!` 值。意即當 Rust 根據兩個分支來推算 `guess` 型別時，會觀察到前者會是 `u32`，而後者是 `!`。因為 `!` 永遠不會有值，Rust 於是決定 `guess` 的型別為 `u32`。

描述這種行為的正確方式是：`!` 型別的表達式能夠轉型為任意其他型別。我們允許 `match` 分支結束在 `continue` 就是因為 `continue` 不會回傳任何值，相反地，它將控制流移至迴圈的最上面，所以在 `Err` 的情況，我們不會對 `guess` 賦值。

永不型別在使用 `panic!` 巨集很實用。回想一下我們對 `Option<T>` 呼叫 `unwrap` 函式，會產生一個值或是恐慌，這是它的定義：


```
impl<T> Option<T> {
    pub fn unwrap(self) -> T {
        match self {
            Some(val) => val,
            None => panic!("called `Option::unwrap()` on a `None` value"),
        }
    }
}
```

和範例 19-26 `match` 相同的情況，在這段程式碼再度上演：Rust 看到 `val` 的型別是 `T` 且 `panic` 是 `!` 型別，所以 `match` 表達式的總體結果是 `T`。這段程式碼可執行是因為 `panic!` 會結束程式而不會產生值。當遇上 `None` 的情形，我們不會從 `unwrap` 回傳任何值，所以這段程式碼有效。

最後一個具有 `!` 型別的表達式是 `loop`：

```
print!("永永");

loop {
    print!("遠遠");
}
```

這裡迴圈永不結束，所以 `!` 就是迴圈表達式的值。但當我們有一個 `break` 時，這就不成立了，因為迴圈會在抵達 `break` 時終止。

動態大小型別與 `Sized` 特徵

Rust 需要了解其型別的特定細節，例如需替特定型別之值配置多少空間。這導致型別系統有令人困惑的小地方：即是**動態大小型別**（*dynamically sized type*）的概念。有時稱為 *DST* 或**不定大小（unsized）型別**，這些型別賦予我們寫出僅能在執行期（*runtime*）得知值的大小之程式碼。

讓我們深入研究一個貫穿全書到處使用的動態大小型別 `str` 的細節。你沒看錯，不是 `&str` 而是 `str` 本身就是 *DST*。在執行期前我們無從得知字串多長，也就表示無法建立一個型別為 `str` 的變數，更不能將 `str` 型別作為引數。試想以下不能執行的程式碼：

```
let s1: str = "Hello there!";
let s2: str = "How's it going?";
```

Rust 必須知道該配置多少記憶體給特定型別之值，且所有該型別之值都會使用相同的記憶體量。若 Rust 允許我們寫出這種程式碼，代表這兩個 `str` 值會用相同的空間大小，但它們長度不同：`s1` 需要 12 位元組來儲存，而 `s2` 需要 15 位元組。這就是為什麼不可能建立一個持有動態大小型別的變數。

那我們該如何是好？這種情況下，你其實已經知道答案：將 `s1` 與 `s2` 的型別從 `str` 改成 `&str`。回憶一下，第四章「字串切片」一節我們說了，切片資料結構會儲存該切片的開始位置與長度。所以雖然 `&T` 是單一的值，儲存了 `T` 所在的記憶體位址，`&str` 卻儲存兩個值：`str` 的位址與它的長度。如此一來，無論 `&str` 指向的字串有多長，我們都可以在編譯期得知 `&str` 的大小。一般來說，這就是動態大小型別在 Rust 中的使用方式，通常具有額外的資料紀錄動態資訊的大小。動態大小型別的黃金法則即是我們必將動態大小型別的值放在指向某種指標之後。

我們將各種指標與 `str` 結合，例如 `Box<str>` 或 `Rc<str>`。事實上，你早已看過此類作法，不過是在其他動態大小型別上看過，那個型別就是特徵（trait）。每個特徵都是一個動態大小型別，我們可以透過使用特徵的名字來指涉它。在第十七章的「允許不同型別數值的特徵物件」部分，我們提及欲將特徵做為特徵物件來使用，必須將特徵放在指標之後，例如 `&dyn Trait` 或 `Box<dyn Trait>`（`Rc<dyn Trait>` 也行）。

為了使用 DST，Rust 提供一個 `Sized` 特徵，來決定一個型別的大小可否在編譯期就確定下來。對於能在編譯期得知大小的所有東西，都會自動實作這個特徵。此外 Rust 自動替所有泛型函式隱含加上 `Sized` 的約束。也就是說若一泛型函數定義如下：

```
fn generic<T>(t: T) {  
    // --省略--  
}
```

實際上就如同寫成這樣：

```
fn generic<T: Sized>(t: T) {  
    // --省略--  
}
```

預設情形下，泛型函式只能在編譯器得知大小的型別上使用。然而，你可以加上以下這個特殊語言來放寬這個限制：

```
fn generic<T: ?Sized>(t: &T) {  
    // --省略--  
}
```

`?Sized` 特徵界限代表「`T` 可能是或不是 `Sized`」，而此詮釋會覆蓋原本預設泛型型別必須在編譯期就已知大小。`?Trait` 的語法與語義只能用在 `Sized`，不適用於其他特徵。

也請注意，我們將參數 `t` 的型別由 `T` 轉為 `&T`，是因為這個型別可能不是 `Sized`，所以我們需要將它放在指標之後才能使用之，而在這例子中，我們選擇將它放在參考之後。接下來，我們會聊聊函式和閉包！

進階函式與閉包

本節探索函式與閉包相關的進階特色，包括函式指標和回傳閉包。

函式指標

我們已探討過如何將閉包傳遞給函式，其實你還可以將一般的函式傳給函式！當你想要傳遞已經定義好的函式，而不是新的閉包時，就會凸顯這個技巧好用之處。函式將會轉型為 `fn` 型別（小寫的 `f`），別和閉包特徵的 `Fn` 搞混了。這個 `fn` 型別就稱為**函式指標**（function pointer）。你可以將函數作為函式指標傳遞，當作其他函式的引數。

將函式指標當作參數傳遞的語法，和閉包相當相似，如範例 19-27 所示。我們定義了函式 `add_one`，替其參數加一。函數 `do_twice` 接受兩個參數：一個函式指標，指向任何輸入 `i32` 且回傳 `i32` 和 `i32 value` 的函式。 `do_twice` 函式呼叫函式 `f` 兩次，並傳遞 `arg` 的數值，再將兩個函式呼叫的回傳值加總。 `main` 函式以引數 `add_one` 和 `5` 呼叫 `do_twice`。

檔案名稱：src/main.rs

```
fn add_one(x: i32) -> i32 {
    x + 1
}

fn do_twice(f: fn(i32) -> i32, arg: i32) -> i32 {
    f(arg) + f(arg)
}

fn main() {
    let answer = do_twice(add_one, 5);

    println!("答案是：{}", answer);
}
```

範例 19-27：藉由 `fn` 型別接收函式指標引數

這段程式碼會印出 答案是：12。我們可以指定 `do_twice` 的參數 `f` 是一個需要一個 `i32` 當參數的 `fn`，並會回傳 `i32`。接下來我們在 `do_twice` 內呼叫 `f`。在 `main` 中，我們就可將 `add_one` 函式作為 `do_twice` 第一個引數。

和閉包不同的是，`fn` 不是特徵而是一個型別，所以我們可以直接將 `fn` 作為參數型別，而不需要宣告一個以 `Fn` 特徵作為特徵限制的泛型型別參數。

函式指標將三個閉包特徵（`Fn`、`FnMut`、`FnOnce`）通通實作了，意思是在預期要傳入閉包之處，你一定可以將函式指標作為引數傳進去。最佳的做法是寫一個同時使用泛型型別和其中一個閉

包特徵的函式，這樣無論是函式還是閉包，你的函式全都可以接收。

也就是說，有個你只會想接收 `fn` 但不要閉包的例子，就是當你在與外部那些沒有閉包的程式碼打交道的時候，比如 `C` 可以接收函式作為引數，但 `C` 並沒有閉包。

讓我們來看一下標準函式庫中 `Iterator` 特徵提供的 `map` 的用法，`map` 就是可以用行內閉包（closure defined inline）或一個命名函式（named function）的例子。欲將數字的向量轉換成字串的向量，我們可以使用閉包，例如：

```
let list_of_numbers = vec![1, 2, 3];
let list_of_strings: Vec<String> =
    list_of_numbers.iter().map(|i| i.to_string()).collect();
```

或者，我們也可以將一個函式作為引數，代替閉包傳入 `map`：

```
let list_of_numbers = vec![1, 2, 3];
let list_of_strings: Vec<String> =
    list_of_numbers.iter().map(ToString::to_string).collect();
```

請注意，因為有多個可用的函式都叫做 `to_string`，所以我們必須使用先前在「[進階特徵](#)」一節提及的完全限定語法。這裡，我們使用了在 `ToString` 特徵中定義的 `to_string` 函式，只要有實作 `Display` 的型別，標準函式庫都會提供 `ToString` 的實作。

回想一下第六章「[列舉數值](#)」一節提及，我們定義的每個列舉變體的名字，也是一個初始化函式，我們可以將這些初始化函式當作實作了閉包特徵的函式指標，這就代表我們可以指定初始化函式作為引數，傳給需要閉包的方法，例如：

```
enum Status {
    Value(u32),
    Stop,
}

let list_of_statuses: Vec<Status> =
    (0u32..20).map(Status::Value).collect();
```

這裡，我們對一個範圍呼叫 `map`，並用每個 `u32` 值，透過 `Status::Value` 的初始化函式來建立 `Status::Value` 的實例。有些人更喜歡上述的作法，但有人偏好閉包。這兩者的編譯結果相同，所以選一個你覺得清晰的風格吧。

回傳閉包

閉包是用特徵來表示，言下之意是你不能直接回傳一個閉包。大多數的情況，當你想回傳一個特徵時，可以改回傳有實作該特徵的具體型別。但你並無法對閉包這樣做，因為它們根本沒有可供回傳的具體型別，比方說不允許你使用 `Fn` 特徵作為回傳型別。

接下來的程式碼嘗試直接回傳一個閉包，但它無法編譯：

```
fn returns_closure() -> dyn Fn(i32) -> i32 {  
    |x| x + 1  
}
```

編譯錯誤如下：

```
$ cargo build  
   Compiling functions-example v0.1.0 (file:///projects/functions-example)  
error[E0746]: return type cannot have an unboxed trait object  
--> src/lib.rs:1:25  
  |  
1 | fn returns_closure() -> dyn Fn(i32) -> i32 {  
  |                               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ doesn't have a size known at  
compile-time  
  |  
  = note: for information on `impl Trait`, see <https://doc.rust-lang.org/book/ch10-02-traits.html#returning-types-that-implement-traits>  
help: use `impl Fn(i32) -> i32` as the return type, as all return paths are of  
type `[closure@src/lib.rs:2:5: 2:8]`, which implements `Fn(i32) -> i32`  
  |  
1 | fn returns_closure() -> impl Fn(i32) -> i32 {  
  |                               ~~~~~  
  
For more information about this error, try `rustc --explain E0746`.  
error: could not compile `functions-example` due to previous error
```

這個錯誤再度指出 `Sized` 特徵！Rust 不知道我們需要多少空間儲存這個閉包，我們之前看過這類問題的解法。可以使用特徵物件：

```
fn returns_closure() -> Box<dyn Fn(i32) -> i32> {  
    Box::new(|x| x + 1)  
}
```

這段程式碼恰巧能通過編譯。欲知更多特徵物件相關資訊，請查閱第十七章「[允許不同型別數值的特徵物件](#)」部分。

接下來，讓我們來探討巨集吧！

巨集

在本書中，我們到處使用像 `println!` 這類的巨集 (macro)，但尚未完全探索巨集究竟是何物，以及該如何駕馭。巨集指的是一整家族的 Rust 功能集合：使用 `macro_rules!` 的宣告式 (declarative) 巨集，以及另外三種程序式 (procedural) 巨集：

- 自訂 `#[derive]` 巨集，可以將指定的程式碼加在使用 `derive` 屬性的結構體和列舉
- 類屬性巨集，定義可以用在任何項目的自訂屬性
- 類函式巨集，看起來像是函式的呼叫但實際上將標記 (token) 當作引數來處理

我們將會按照順序聊聊每種巨集，但首先，來看看為什麼我們已經有了函式，仍需要巨集呢？

巨集與函式的差異

基本上，巨集是一種透過寫程式碼來產生其他程式碼的手段，又稱作超程式設計 (metaprogramming)。像是在附錄 C，我們探討的 `derive` 屬性，這個屬性會替你產生多種特徵的實作。還有在整本書中到處使用 `println!` 和 `vec!` 兩巨集。以上這些巨集都會展開來，產生比你自已手寫的還要多的程式碼。

超程式設計對減少撰寫和維護的程式碼量非常有幫助，這和函式扮演的角色相同，然而，巨集具有函式沒有的特殊本事。

一個函式簽名必須宣告該函式需要的參數型別與數量。反觀巨集可以接收變動數量的參數：我們可以用一個參數呼叫 `println!("hello")`，也可以是兩個參數的 `println!("hello {}", name)`。另外，巨集會在編譯器開始翻譯程式碼的意義之前展開。例如可以使用巨集實作一個特徵。這種事函式便無法做到，因為函式會在執行期呼叫，而特徵需要在編譯期就實作。

選擇實作巨集而不用函式也有缺點，巨集的定義比函式更加複雜，因為你是在寫 Rust 的程式碼來生成 Rust 的程式碼。就是因為這種間接迂迴的關係，一般情況下，相較於函式來說巨集的定義都更加難以閱讀、理解與維護。

另一個巨集和函式之間的重要差異，在一個檔案中想呼叫巨集，必須在作用域 (scope) 內定義或是將巨集帶到這個作用域，而反過來函式可以在任何地方定義與呼叫。

使用 `macro_rules!` 宣告式巨集做普通的超程式設計

Rust 中最廣泛使用的巨集形式非宣告式巨集莫屬。這種巨集有時也稱為「巨集為例 (macros by example)」，「`macro_rules!`」，或是直白的「巨集」。宣告式巨集的核心就是賦予你寫些類似 Rust `match` 表達式的東西。在第六章我們聊了 `match` 表達式是一種流程控制結構，會拿一個表達式，將其結果值與其他模式作比較，並執行匹配模式對應的程式碼。巨集同樣會拿一個值，與模式相比較，而這個模式又與特定程式碼相關聯：這種情況會是，傳入巨集的值就是一字一字刻出來

Rust 原始碼，而所謂模式則是比較原始碼的結構，當原始碼與模式相匹配，就會帶入與模式關聯的這段特定程式碼，取代原先傳入巨集的原始碼。這些都發生在編譯的期間。

你可以透過 `macro_rules!` 定義一個巨集。讓我們藉著閱讀 `vec!` 的定義來探索如何使用 `macro_rules!`。第八章我們介紹了如何使用 `vec!` 巨集來建立含有特定值的向量。例如，下面的巨集會建立帶著三個整數的新向量：

```
let v: Vec<u32> = vec![1, 2, 3];
```

我們也可利用 `vec!` 巨集產生兩個整數的向量或是五個字串的切片。因為不能預先得知這些值的數量，所以我們無法透過函式做到這件事。

範例 19-28 展示了稍微簡化過的 `vec!` 巨集定義。

檔案名稱：src/lib.rs

```
#[macro_export]
macro_rules! vec {
    ( $( $x:expr ),* ) => {
        {
            let mut temp_vec = Vec::new();
            $(
                temp_vec.push($x);
            )*
            temp_vec
        }
    };
}
```

範例 19-28： `vec!` 巨集定義簡化版

注意：在標準函式庫中真實的 `vec!` 巨集定義有預先配置正確記憶體用量的程式碼，因為這段程式碼是一種最佳化手段，為了簡化範例，並無將之包含其中。

這個 `#[macro_export]` 標註（annotation）用來指明只要這個 crate 有在程式碼可見作用域中，就可以使用該巨集。若沒有這個標註，巨集就不能帶入該作用域內。

我們的巨集定義從 `macro_rules!` 和我們欲定義的巨集名稱**去除驚嘆號**開始。這個名稱，在我們例子裡是 `vec`，的後面接著花括號表示巨集定義的本體。

這個 `vec!` 本體的結構和 `match` 表達式的結構相似。這裡我們有一個 `match` 分支，帶著模式（`$( $x:expr ),*`），並接著 `=>` 後面與該模式相關聯的程式碼區塊。這個分支是此巨集唯一一個模式，所以只有一個有效匹配方式；任何其他模式都會產生錯誤。更複雜的巨集會有多於一個分支。

有效的巨集定義模式語法和在第十八章的模式語法並不相同，巨集的模式並不跟值比較，而是與 Rust 程式碼的結構相互匹配。在範例 19-28 我們會走過一次這些模式的意義，至於完整的巨集模式語法，請閱讀 [Rust 參考手冊](#)。

首先，我們用一對括號包圍整個模式。我們使用錢字號（`$`）在巨集系統定義一個變數，該變數將包含與模式匹配的 Rust 程式碼。採用錢字號清楚展現它並非尋常的 Rust 變數，而是巨集變數。再來就是一對括號，用以捕獲與括號內的模式匹配之值，以替換為程式碼。在 `$()` 內的 `$x:expr` 會匹配任意 Rust 表達式，並賦予表達式一個 `$x` 名稱。

在 `$()` 後的逗號代表字面上的逗號分隔，可以選擇性地在匹配 `$()` 內的程式碼後出現。而 `*` 這指明，這個模式可以匹配零至多個在 `*` 之前的東西。

當我們以 `vec![1, 2, 3]` 呼叫這個巨集，`$x` 模式會匹配到三次，分別為 `1`、`2` 和 `3` 三個表達式。

現在來看看這個模式分支的本體程式碼：在 `$()*` 內的 `temp_vec.push()` 會根據 `$()` 模式匹配了幾次而產生幾次。這個 `$x` 會被每個匹配的表達式取代。當我們使用 `vec![1, 2, 3]` 呼叫巨集時，這個取代巨集呼叫而產生出來的程式碼會是：

```
{
    let mut temp_vec = Vec::new();
    temp_vec.push(1);
    temp_vec.push(2);
    temp_vec.push(3);
    temp_vec
}
```

我們定義了一個巨集，接收任意數量任意型別的引數，並產生建立一個包含指定元素的向量的程式碼。

想理解更多有關撰寫巨集之事，可查閱線上文件或其他資源，例如原作者 Daniel Keep 與後繼維護者 Lukas Wirth 所寫的「[The Little Book of Rust Macros](#)」。

使用程序式巨集從屬性產生程式碼

第二種巨集形式是**程序式巨集**，其行為更像是函式（也是一種程序）。程序式巨集接受一些程式碼作為輸入，操作這些程式碼，然後輸出一些程式碼。和宣告式巨集去匹配模式和取代程式碼的方式不同。三種程序式巨集（自訂 `derive`，類屬性、類函式）都有著相近的工作方式。

當建立一個程序式巨集時，該巨集必須放置在自己特殊的一種 `crate` 中。會這樣是因為一些複雜的技術問題，我們希望在未來消弭這個情況。範例 19-29 我們展示了如何定義程序式巨集，其中 `some_attribute` 是一個用來代表特定巨集的佔位符。

檔案名稱：`src/lib.rs`


```
use proc_macro;

#[some_attribute]
pub fn some_name(input: TokenStream) -> TokenStream {
}
```

範例 19-29：定義一個程序式巨集

這個函式定義一個程序式巨集，接受輸入 `TokenStream`，並輸出 `TokenStream`。 `TokenStream` 型別定義在 `proc_macro` crate 中，這個 crate 包含在 Rust 中，可以表示一連串的標記，這就是巨集的核心：巨集替來自輸入的 `TokenStream` 搽脂抹粉，而巨集產生的程式碼就是輸出的 `TokenStream`。上面例子中這個函式附加了一個屬性，指定我們要產生哪個程序式巨集。在同一個 crate 中我們可以使用多個不同的程序式巨集。

我們來看不同的程序式巨集吧。就從自訂 `derive` 巨集開始，逐步介紹它與其他種類巨集的細部差異。

如何撰寫自訂的 `derive` 巨集

我們建立一個 `hello_macro` crate，並定義 `HelloMacro` 特徵與它的 `hello_macro` 關聯函式。我們提供一個程序式巨集，讓 crate 的使用者透過 `#[derive(HelloMacro)]` 標註它們的型別，來獲得預設的 `hello_macro` 函式的實作，而不需要使用者替每個型別手動實作 `HelloMacro` 特徵。這個預設的函式實作會印出 `你好，巨集！我叫做型別名稱！`，其中 `型別名稱` 是實作特徵那個型別的名字。換句話說，就是我們會寫出一個 crate，讓其他程式設計師用我們的 crate，以範例 19-30 的方式來寫程式。

檔案名稱：`src/main.rs`

```
use hello_macro::HelloMacro;
use hello_macro_derive::HelloMacro;

#[derive(HelloMacro)]
struct Pancakes;

fn main() {
    Pancakes::hello_macro();
}
```



範例 19-30：使用者使用我們的程序式巨集時，能夠寫出的程式碼

當我們完成後，這段程式碼會印出 `你好，巨集！我叫做鬆餅！`。第一步，先建立一個新的函式庫 crate：

```
$ cargo new hello_macro --lib
```

接下來，我們會定義 `HelloMacro` 特徵與它的關聯函式：

檔案名稱：`src/lib.rs`

```
pub trait HelloMacro {  
    fn hello_macro();  
}
```

我們有個特徵及其函式。至此，我們的 crate 使用者可以實作此特徵來達成他們想要的功能，例如：

```
use hello_macro::HelloMacro;  
  
struct Pancakes;  
  
impl HelloMacro for Pancakes {  
    fn hello_macro() {  
        println!("你好，巨集！我叫做鬆餅！");  
    }  
}  
  
fn main() {  
    Pancakes::hello_macro();  
}
```

然而，使用者必須自行替每個想使用 `hello_macro` 的型別分別撰寫實作區塊，我們想節約這些重複工作。

另外，我們尚未提供 `hello_macro` 函式的預設實作，這個預設實作將會印出實作該特徵的型別名稱，但 Rust 並沒有反射（reflection）這種功能，所以無法在執行期檢查型別，因此我們需要一個巨集在編譯期產生程式碼。

下一步是定義程序式巨集。在我們寫此章時，程序式巨集必須在自己的 crate 中定義，最終這個限制會解除。組織安排 crate 和巨集 crate 的慣例如下：有一個 crate `foo` 和一個自訂 derive 程序式巨集 crate `foo_derive`，讓我們在 `hello_macro` 專案中建立一個新的 crate

`hello_macro_derive`：

```
$ cargo new hello_macro_derive --lib
```

由於我們的兩個 crate 高度關聯，所以會在 `hello_macro` crate 的目錄中建立一個程序式巨集 crate。若我們改變 `hello_macro` 中定義的特徵，就必須同時改變 `hello_macro_derive` 中的程序式巨集。這兩個 crate 必須各自發佈，且若程式設計師想要使用這些 crate，則必須將兩者都加入為依賴（dependency），並將之引入作用域。當然，我們也可以讓 `hello_macro` 將 `hello_macro_derive` 作為一個依賴並重新導出（re-export）該程序式巨集。然而，我們這樣組織專案的方式就是想提供當程式設計師不想要 `derive` 功能時，也可以直接使用 `hello_macro`。

我們必須宣告 `hello_macro_derive` 為一個程序式巨集 crate。我們同時需要等會兒就會遇到的 `syn` 和 `quote` 這些 crate 的功能，所以先將他們加至依賴。至此，`hello_macro_derive` 的 `Cargo.toml` 會加入以下的程式碼：

檔案名稱：`hello_macro_derive/Cargo.toml`

```
[lib]
proc-macro = true

[dependencies]
syn = "1.0"
quote = "1.0"
```

欲開始定義程序式巨集，請將範例 19-31 的程式碼放入你的 `hello_macro_derive` crate 的 `src/lib.rs` 檔案中。注意，在我們定義 `impl_hello_macro` 函式之前，這段程式碼都無法編譯。

檔案名稱：`hello_macro_derive/src/lib.rs`

```
use proc_macro::TokenStream;
use quote::quote;
use syn;

#[proc_macro_derive(HelloMacro)]
pub fn hello_macro_derive(input: TokenStream) -> TokenStream {
    // 建構 Rust 程式碼的語法樹呈現
    // 讓我們可以進行操作
    let ast = syn::parse(input).unwrap();

    // 建構特徵實作
    impl_hello_macro(&ast)
}
```



範例 19-31：若要產生 Rust 程式碼，絕大部分程序式巨集 crate 都必須包含這段程式碼

留意到了嗎，我們將程式碼函式拆分，其中 `hello_macro_derive` 函式負責解析 `TokenStream`，而 `impl_hello_macro` 函式則用來轉換語法樹（syntax tree），這讓撰寫程序式巨集更為方便。外面這個函式的程式碼（在這例子是 `hello_macro_derive`）在每個你遇見或建立的程序式巨集裡看起來都幾乎一模一樣。而在裡面的函式（在這個例子是 `impl_hello_macro`）的本體則根據不同程序式巨集的目的而有所不同。

我們導入了三個新 crate：`proc_macro`，`syn` 和 `quote`。`proc_macro` 包含在 Rust 裡面，所以我們不需要將之加入 `Cargo.toml`。`proc_macro` crate 就是編譯器的 API，提供從我們的程式碼讀取和操作 Rust 程式碼。

`syn` crate 負責從字串解析 Rust 程式碼，轉成我們可以操作的資料結構。而 `quote` crate 則將 `syn` 的資料結構轉回 Rust 程式碼。撰寫完整的 Rust 程式碼解析器並不是容易的工作，而這些 crate 讓解析任何 Rust 程式碼更為簡便。

當使用者在一個型別上指定 `#[derive(HelloMacro)]`，`hello_macro_derive` 函式就會被呼叫，這是由於我們使用 `proc_macro_derive` 和指定的 `HelloMacro` 名稱來標註 `hello_macro_derive` 函式，而其中的 `HelloMacro` 是我們的特徵名稱。以上就是大多數程序式巨集遵守的慣例。

`hello_macro_derive` 函式會先將輸入 `input` 的 `TokenStream` 轉換成一個我們可以翻譯並執行操作的資料結構，這就是 `syn` 參與的部分，`syn` 的 `parse` 函式需要一個 `TokenStream` 並回傳一個 `DeriveInput` 結構體，代表解析過後的 Rust 程式碼。範例 19-32 展示了解析完 `struct Pancakes` 字串後所得的 `DeriveInput` 的部分：

```
DeriveInput {  
    // --省略--  
  
    ident: Ident {  
        ident: "Pancakes",  
        span: #0 bytes(95..103)  
    },  
    data: Struct(  
        DataStruct {  
            struct_token: Struct,  
            fields: Unit,  
            semi_token: Some(  
                Semi  
            )  
        }  
    )  
}
```

範例 19-32：這是在範例 19-30 解析具有 `macro` 屬性的程式碼時所得的 `DeriveInput` 實例

這些結構體的欄位展示了解析過後的 Rust 程式碼是一個結構體，帶著 `ident`（識別字 identifier）。這裡其他結構體的欄位都在描述 Rust 程式碼，更多資訊請參考 [syn 有關 `DeriveInput` 的文件](#)。

我們很快就進入定義 `impl_hello_macro` 函式的環節，這個函式協助打造我們想要的新 Rust 程式碼。再動手做之前，注意我們的 `derive` 巨集輸出也是一個 `TokenStream`。回傳的 `TokenStream` 會添加到我們的 crate 使用者撰寫的程式碼中，因此，當他們編譯他們的 crate 時，會從我們提供的修編過的 `TokenStream` 中取得額外功能。

也許你注意到我們對 `hello_macro_derive` 呼叫 `unwrap` 讓 `sync::parse` 函式失敗時恐慌。由於我們需要符合 `proc_macro_derive` 程序式巨集的 API 定義，回傳一個 `TokenStream` 而非 `Result`，所以我們的程序式巨集必須在錯誤時恐慌。這裡使用 `unwrap` 是為了簡化範例，在正式環境程式碼中，你應該透過 `panic!` 或 `expect` 提供更特定的錯誤訊息，告知什麼出錯了。

現在，被標註的 Rust 程式碼已經從一個 `TokenStream` 轉換成 `DeriveInput` 實例，現在來替被標註的型別產生實作 `HelloMacro` 特徵的程式碼，如範例 19-33。

檔案名稱：hello_macro_derive/src/lib.rs

```
fn impl_hello_macro(ast: &syn::DeriveInput) -> TokenStream {
    let name = &ast.ident;
    let gen = quote! {
        impl HelloMacro for #name {
            fn hello_macro() {
                println!("你好，巨集！我叫做{}！", stringify!(#name));
            }
        }
    };
    gen.into()
}
```

範例 19-33：利用解析過的 Rust 程式碼來實作 `HelloMacro` 特徵

我們從 `ast.ident` 取得 `Ident` 結構體實例，這個實例中帶有被標註的型別之名稱（識別字）。當我們執行在範例 19-30 程式碼中的 `impl_hello_macro` 函式，會獲得一個 `ident`，帶有一個值為 `"Pancakes"` 的 `ident` 欄位，就如同範例 19-30 所示。因此，在範例 19-33 的 `name` 變數會包含一個 `Ident` 結構體實例，當我們印之，會出現字串 `"Pancakes"`，也就是該結構體在範例 19-30 所示的名字。

`quote!` 巨集提供我們定義想要回傳的 Rust 程式碼。編譯器期望接收到不同於 `quote!` 巨集執行後直接輸出的結果，所以我們需要將結果轉換為一個 `TokenStream`。我們透過呼叫 `into` 方法達成，這個方法會消耗中介碼（intermediate representation）並回傳一個型別為 `TokenStream` 之值。

`quote!` 巨集也提供非常炫的模板機制：我們可以輸入 `#name`，而 `quote!` 會以變數 `name` 值取而代之。我們甚至可以做一些類似普通巨集的重複工作。閱讀 [quote crate 的文件](#) 以獲得完整的介紹。

我們想要我們的程序式巨集對使用者標註的型別產生 `HelloMacro` 特徵的實作，這個標註的型別名稱可以從 `#name` 取得。這個特徵的實作有一個函式 `hello_macro`，函式本體包含我們想要的功能：印出 `你好，巨集！我叫做` 再加上被標註的型別的名稱。

`stringify!` 巨集是 Rust 內建的，會將一個 Rust 表達式，例如 `1 + 2`，在編譯期轉換成字串字面值（string literal），例如 `"1 + 2"`。這和 `format!` 或 `println!` 巨集會對表達式求值並將結果轉為 `String` 不同。因為輸入的 `#name` 可能是一個表達式，但要直接照字面印出來，所以我們選擇使用 `stringify!`。使用 `stringify!` 也可以節省在編譯器因為轉換 `#name` 成為字串字面量所需的空間配置。

至此，`cargo build` 應該可以成功在 `hello_macro` 和 `hello_macro_derive` 完成。我們在範例 19-30 來玩玩這些 crate 看看他們如何實際作用！先在你的專案目錄下，透過 `cargo new pancakes` 建立一個新的執行檔專案。我們必須將 `hello_macro` 和 `hello_macro_derive` 加入 `pancakes` 的 `Cargo.toml` 作為依賴。若你已經發佈自己的 `hello_macro` 和

`hello_macro_derive` 的版本到 crates.io，他們就是普通的依賴；若無，你可以指定他們為 `path` 的依賴，如下：

```
hello_macro = { path = "../hello_macro" }  
hello_macro_derive = { path = "../hello_macro/hello_macro_derive" }
```

將這段程式碼放到範例 19-30 的 `src/main.rs` 並執行 `cargo run`，他應該會印出 你好，巨集！我叫做鬆餅！。這個由程序式巨集實作的 `HelloMacro` 特徵，不需要 `pancakes` 自行手動實作，而是透過 `#[derive>HelloMacro)]` 將特徵的實作加上。

接著，一起來探索其他種類的程序式巨集和自訂 `derive` 巨集有何不同。

類屬性巨集

類屬性巨集和自訂 `derive` 巨集相似，但並非只能透過 `derive` 屬性產生程式碼，類屬性巨集讓你可以建立新的屬性。它們更靈活：`derive` 只能用在結構體和列舉，而屬性可以用在其他項目之上，例如函式。這裡有個類屬性巨集例子，是在使用一個網頁應用程式框架時，透過你的 `route` 屬性來標註一個函式：

```
#[route(GET, "/")]  
fn index() {
```

這個 `#[route]` 屬性在該框架以程序式巨集定義之，其巨集定義函式的簽名如下：

```
#[proc_macro_attribute]  
pub fn route(attr: TokenStream, item: TokenStream) -> TokenStream {
```

這裡，我們有兩個 `TokenStream` 型別的參數，第一個是屬性的內容，也就是 `Get, "/"` 這部分。第二部分則是該屬性附著的項目本體：在這個例子就是 `fn index() {}` 及其函式本體。

除此之外，類屬性巨集的工作方式和自訂 `derive` 巨集一樣：透過 `proc-macro crate` 建立一個 `crate`，並實作一個函式替你產生程式碼！

類函式巨集

類函式巨集可以定義和函式呼叫很類似的巨集。和 `macro_rules!` 一樣，類函式巨集比函式更有靈活，例如可以接收未知長度的引數。然而，`macro_rules!` 巨集只能使用像 `match` 一樣的語法，如同早前在「使用 `macro_rules!` 宣告式巨集做普通的超程式設計」一節所述。而類函式巨集則可以拿 `TokenStream` 參數及其定義來操作 Rust 程式碼，和另外兩個程序式巨集所做的一模一樣。

舉個例子，一個 `sql!` 類函式巨集可能會被這樣呼叫：

```
let sql = sql!(SELECT * FROM posts WHERE id=1);
```

這個巨集會解析他內部的 SQL 陳述句 (statement)，並檢查語法是否正確，這個過程比 `macro_rules!` 能做到的複雜太多。這個 `sql!` 巨集定義如下：

```
#[proc_macro]
pub fn sql(input: TokenStream) -> TokenStream {
```

這個定義和自訂 `derive` 巨集簽名相似：我們接受在括號內的標記，並回傳想要產生的程式碼。

總結

呼！現在你的工具箱多了一些 Rust 特色功能，雖然不常用，但在特定情況下你會知道它們存在。我們介紹了許多複雜的主題，所以當你在錯誤訊息或是其他人的程式碼與它們相遇，你會有辦法辨認這些概念和語法。你可以將這章作為能引導找到解法的參考書。

接下來，我們會動手做另一個專案，實際運用本書所講的一切。

最終專案：建立多執行緒網頁伺服器

這真是趟漫長的旅途，但我們已經抵達本書的最終章了。在本章中，我們會在建構另一個專案來解釋最後幾章提到的概念，並複習一些之前更早的章節。

在我們的最終專案，我們將建立一個可以回覆「hello」的網頁伺服器，如圖示 20-1 的網頁瀏覽器所示。



Hello!

Hi from Rust

圖示 20-1：我們的最終專案

以下是我們建構網頁伺服器的計劃：

1. 學習一些 TCP 與 HTTP。
2. 在插座 (socket) 上監聽 TCP 連線。
3. 解析一些 HTTP 請求。
4. 建立合適的回應。
5. 透過執行緒池 (thread pool) 改善伺服器的吞吐量。

不過在我們開始之前，我們需要提醒一件事，我們使用的方法不會是在 Rust 中建立網頁伺服器的最佳方案。crates.io 上有不少已經能用在正式環境的 crate，它們都有提供比我們所建立的還更完善的網頁伺服器與執行緒池。然而我們在本章節的目的是要幫助你學習，而不是走捷徑。因為 Rust 是個系統程式設計語言，我們可以選擇我們想運用的抽象層級，而且可以比其他語言更可能且實際地抵達最底層。我們會親自寫出基本的 HTTP 伺服器與執行緒池，來幫助你瞭解往後你可能会用到的 crate 背後的基本概念與技術。

建立單一執行緒的網頁伺服器

我們會先從建立一個可運作的單一執行緒網頁伺服器作為起始。在我們開始之前，讓我們快速瞭解一下建構網頁伺服器會涉及到的協定。這些協定的細節超出本書的範疇，不過以下簡短的概覽可以提供你所需要知道的訊息。

網頁伺服器會涉及到的兩大協定為**超文本傳輸協定（Hypertext Transfer Protocol，HTTP）**與**傳輸控制協定（Transmission Control Protocol，TCP）**。這兩種協定都是**請求-回應（request-response）**協定，這意味著客戶端會初始一個請求，然後伺服器接聽請求並提供回應給客戶端。協定會定義這些請求與回應的內容。

TCP 是個較底層的協定並描述資訊如何從一個伺服器傳送到另一個伺服器的細節，但是它並不指定資訊內容為何。HTTP 建立在 TCP 之上並定義請求與回應的內容。技術上來說，HTTP 是可以與其他協定組合的，但是對大多數場合中，HTTP 主要還是透過 TCP 來傳送資訊。我們會處理 TCP 與 HTTP 中請求與回應的原始位元組（raw bytes）。

監聽 TCP 連線

我們的網頁伺服器需要監聽一個 TCP 連線，所以這就是我們要處理的第一個步驟。標準函式庫有提供 `std::net` 模組能讓我們使用。讓我們如往常一樣建立一個新的專案：

```
$ cargo new hello
    Created binary (application) `hello` project
$ cd hello
```

現在輸入範例 20-1 的程式碼到 `src/main.rs`。此程式碼會監聽 `127.0.0.1:7878` 本地位址（address）傳來的 TCP 流（Stream）。當其收到傳入的資料流時，它就會印出 `連線建立！`。

檔案名稱：`src/main.rs`

```
use std::net::TcpListener;

fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();

    for stream in listener.incoming() {
        let stream = stream.unwrap();

        println!("連線建立!");
    }
}
```

範例 20-1：監聽傳入的流並在收到流時顯示訊息

透過 `TcpListener`，我們可以監聽 `127.0.0.1:7878` 位址上的 TCP 連線。在位址中，分號之前指的是代表你電腦自己的 IP 位址（每部電腦都一樣，這並不只是用於本書作者電腦的例子），然後 `7878` 是通訊埠（port）。我們選擇此通訊埠的原因有兩個：HTTP 通常不會佔用此通訊埠，所以我們的伺服器不太可能會與你機器上執行的其他網路伺服器衝突，而且在傳統電話的九宮格上輸入 7878 的話就是「rust」。

在此情境中的 `bind` 函式與常見的 `new` 函式行為類似，這會回傳一個新的 `TcpListener` 實例。此函式會叫做 `bind` 的原因是因為在網際網路中，連接一個通訊埠並監聽就稱為「綁定（bind）通訊埠」。

`bind` 函式會回傳 `Result<T, E>`，也就是說綁定可能會失敗。舉例來說，連接通訊埠 80 需要管理員權限（非管理員使用者可以連接 1023 以上的通訊埠），所以如果我們不是管理員卻嘗試連接通訊埠 80 的話，綁定就不會成功。另一個例子是，如果執行同個程式碼兩次產生兩個實例，造成這兩個程式同時監聽同個通訊埠的話，綁定也不會成功。由於我們只會寫個用於學習目的的基本伺服器，我們不太需要擔心如何處理這些種類的錯誤。所以我如果遇到錯誤的話，我們採用 `unwrap` 來停止程式。

`TcpListener` 的 `incoming` 方法會回傳一個疊代器，給予我們一連串的流（更準確的來說是 `TcpStream` 型別的流）。一個流（Stream）代表的是客戶端與伺服器之間的開啟的連線。而連線（connection）指的是整個請求與回應的過程，這之中客戶端會連線至伺服器、伺服器會產生回應，然後伺服器會關閉連線。這樣一來，我們就能讀取 `TcpStream` 來查看客戶端傳送了什麼，然後將我們的回應寫入流中，將資料傳回給客戶端。整體來說，此 `for` 迴圈會依序遍歷每個連線，然後產生一系列的流讓我們能夠加以處理。

目前我們處理流的方式包含呼叫 `unwrap`，這當流有任何錯誤時，就會結束我們的程式。如果沒有任何錯誤的話，程式就會顯示訊息。我們會在下一個範例在成功的情況下加入更多功能。當客戶端連接伺服器時，我們可能會從 `incoming` 方法取得錯誤的原因是因為我們實際上不是在遍歷每個連線。反之，我們是在遍歷連線嘗試。連線不成功可能有很多原因，而其中許多都與作業系統有關。舉例來說，許多作業系統都會限制它們能支援的同時連線開啟次數，當新的連線超出此範圍時就會產生錯誤，直到有些連線被關閉為止。

讓我們跑跑看此程式碼吧！在終端機呼叫 `cargo run` 然後在網頁瀏覽器載入 `127.0.0.1:7878`。瀏覽器應該會顯示一個像是「Connection reset」之類的錯誤訊息，因為伺服器目前還不會回傳任何資料。但是當你觀察終端機，在瀏覽器連接伺服器時，你應該會看到一些訊息顯示出來！

```
Running `target/debug/hello`
```

```
連線建立！  
連線建立！  
連線建立！
```

有時候你可能從一次瀏覽器請求看到數個訊息顯示出來，原因很可能是因為瀏覽器除了請求頁面內容以外，也嘗試請求其他資源，像是出現在瀏覽器分頁上的 `favicon.ico` 圖示。

而瀏覽器嘗試多次連線至伺服器的原因還有可能是因為伺服器沒有回傳任何資料。當 `stream` 離開作用域並在迴圈結尾被釋放時，連線會在 `drop` 的實作中被關閉。瀏覽器有時處理被關閉的連線的方式是在重試幾次，因為發生的問題可能是暫時的。不管如何，現在最重要的是我們成功取得 TCP 的連線了！

當你執行完特定版本的程式碼後，記得按下 `ctrl-c` 來停止程式。然後在你變更一些程式碼後重新呼叫 `cargo run` 來確保你有執行到最新的程式碼。

讀取請求

讓我們來實作讀取瀏覽器請求的功能吧！為了能分開取得連線的方式與處理連線的方式，我們會建立另一個新的函式來處理連線。在此 `handle_connection` 新的函式中，我們會讀取從 TCP 流取得的資料並顯示出來，讓我們可以觀察瀏覽器傳送的資料。請修改程式碼成範例 20-2。

檔案名稱：src/main.rs

```
use std::{
    io::{prelude::*, BufReader},
    net::{TcpListener, TcpStream},
};

fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();

    for stream in listener.incoming() {
        let stream = stream.unwrap();

        handle_connection(stream);
    }
}

fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&mut stream);
    let http_request: Vec<_> = buf_reader
        .lines()
        .map(|result| result.unwrap())
        .take_while(|line| !line.is_empty())
        .collect();

    println!("請求：{:#?}", http_request);
}
```

範例 20-2：讀取 `TcpStream` 並顯示資料

我們將 `std::io::prelude` 和 `std::io::BufReader` 引入作用域來取得特定的特徵，讓我們可以讀取並寫入流之中。在 `main` 函式的 `for` 迴圈中，不同於印出說我們取得連線的訊息，我們現在

會呼叫新的 `handle_connection` 函式並將 `stream` 傳入。

在 `handle_connection` 函式中，我們建立 `BufReader` 的實例並取得 `stream` 的可變參考。`BufReader` 提供了緩衝區 (buffering)，幫助我們管理 `std::io::Read` 方法的呼叫。

我們建立了一個變數 `http_request` 來收集瀏覽器傳送到伺服器的每行請求。我們加上 `Vec<_>` 型別詮釋來指示我們想要將每行收集成向量。

`BufReader` 實作的 `std::io::BufRead` 特徵有提供個 `lines` 方法。該方法會回傳個 `Result<String, std::io::Error>` 的疊代器，這會在每次看到換行 (newline) 位元組時，將資料流分開。我們用 `map` 對每個 `Result` 呼叫 `unwrap` 來取得 `String`。如果資料不是有效的 UTF-8 或是讀取流時發生問題的話，`Result` 可能會產生錯誤。在正式環境的程式應該要適當地處理這些錯誤，但為了簡潔我們在這裡選擇直接在遇到錯誤時就終止程式。

瀏覽器會傳送兩次換行字元來表達 HTTP 的請求結束了，所以要確定我們從流中取得一個請求的話，我們就重複取得行數直到有一行是空字串為止。一旦我們將行數收集到向量中，我們就使用好看的除錯格式印出來，讓我們可以觀察瀏覽器傳送了哪些指令給我們的伺服器。

讓我們嘗試看看此程式碼！開啟程式並再次從網頁瀏覽器發送請求。注意到我們仍然會在瀏覽器中取得錯誤頁面，但是我們程式在終端機的輸出應該會類似以下結果：

```
$ cargo run
Compiling hello v0.1.0 (file:///projects/hello)
Finished dev [unoptimized + debuginfo] target(s) in 0.42s
Running `target/debug/hello`
Request: [
  "GET / HTTP/1.1",
  "Host: 127.0.0.1:7878",
  "User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:99.0)
Gecko/20100101 Firefox/99.0",
  "Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*
;q=0.8",
  "Accept-Language: en-US,en;q=0.5",
  "Accept-Encoding: gzip, deflate, br",
  "DNT: 1",
  "Connection: keep-alive",
  "Upgrade-Insecure-Requests: 1",
  "Sec-Fetch-Dest: document",
  "Sec-Fetch-Mode: navigate",
  "Sec-Fetch-Site: none",
  "Sec-Fetch-User: ?1",
  "Cache-Control: max-age=0",
]
```

依據你的瀏覽器，你可能會看到一些不同的輸出結果。現在我們顯示了請求的資料，我們可以觀察為何瀏覽器會產生多次請求，我們可以看看 `Request: GET` 之後的路徑。如果重複的連線都在請求 `/` 的話，我們就能知道瀏覽器在重複嘗試取得 `/`，因為它沒有從我們的程式取得回應。

讓我們拆開此請求資料來理解瀏覽器在向我們的程式請求什麼。

仔細觀察 HTTP 請求

HTTP 是基於文字的協定，而請求格式如下：

```
Method Request-URI HTTP-Version CRLF
headers CRLF
message-body
```

第一行是請求行 (**request line**) 並持有客戶端想請求什麼的資訊。請求行的第一個部分代表著想使用的方法 (**method**)，像是 `GET` 或 `POST`，這描述了客戶端如何產生此請求。在此例中，我們的客戶端使用的是 `GET` 請求。

請求行的下一個部分是 `/`，這代表客戶端請求的**統一資源標誌符 (Uniform Resource Identifier, URI)**，URI 絕大多數（但不是絕對）就等於**統一資源定位符 (Uniform Resource Locator, URL)**。URI 與 URL 的差別對於本章節的學習目的來說並不重要，但是 HTTP 規格使用的術語是 URI，所以我們這裡將 URL 替換為 URI。

最後一個部分是客戶端使用的 HTTP 版本，然後請求行最後以 **CRLF 序列** 做結尾，CRLF 指的是**輸入 (carriage return) 與換行 (line feed)**，這是打字機時代的術語！CRLF 序列也可以寫成 `\r\n`，`\r` 指的是輸入，而 `\n` 指的是換行。CRLF 序列將請求行與剩餘的請求資料區隔開來。注意到當 CRLF 印出時，我們會看到的是新的一行而不是 `\r\n`。

觀察我們目前從程式中取得的請求行資料，我們看到它使用 `GET` 方法，`/` 為請求 URI，然後版本為 `HTTP/1.1`。

在請求行之後，剩餘從 `Host:` 開始的行數都是標頭 (header)。`GET` 請求不會有本體 (body)。

你可以嘗試看看從不同的瀏覽器或尋問不同的位址，像是 `127.0.0.1:7878/test`，來看看請求資料有什麼改變。

現在我們知道瀏覽器在請求什麼了，讓我們回傳一些資料吧！

寫入回應

現在我們要實作傳送資料來回應客戶端的請求。回應格式如下：

```
HTTP-Version Status-Code Reason-Phrase CRLF
headers CRLF
message-body
```

第一行為**狀態行 (status line)**，這包含回應使用的 HTTP 版本、用來總結請求結果的狀態碼，以及狀態碼的文字來描述原因。在 CRLF 序列後，會接著任何標頭、另一個 CRLF 序列，然後是回應

的本體。

以下是個使用 HTTP 版本 1.1 的回應範例，其狀態碼為 200、文字描述為 OK，沒有標頭與本體：

```
HTTP/1.1 200 OK\r\n\r\n
```

狀態碼 200 是標準的成功回應。這段文字就是小小的 HTTP 成功回應。讓我們將此寫入流中作為我們對成功請求的回應吧！在 `handle_connection` 函式中，移除原先印出請求資料的 `println!`，然後換成範例 20-3 的程式碼。

檔案名稱：src/main.rs

```
fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&mut stream);
    let http_request: Vec<_> = buf_reader
        .lines()
        .map(|result| result.unwrap())
        .take_while(|line| !line.is_empty())
        .collect();

    let response = "HTTP/1.1 200 OK\r\n\r\n";

    stream.write_all(response.as_bytes()).unwrap();
}
```

範例 20-3：寫入一個小小的成功 HTTP 回應至流中

新的第一行定義了變數 `response` 會持有成功訊息的資料。然後我們對我們的 `response` 呼叫 `as_bytes` 來將字串轉換成位元組。`stream` 中的 `write_all` 方法接收 `&[u8]` 然後將這些位元組直接傳到連線中。由於 `write_all` 操作可能會失敗，我們如前面一樣對任何錯誤使用 `unwrap`。同樣地，在實際的應用程式中你應該要在此加上錯誤處理。

有了這些改變，讓我們執行程式碼然後下達請求。我們不再顯示任何資料到終端機上了，所以我們不會看到任何輸出，只會有 Cargo 執行的訊息。當你在網頁瀏覽器讀取 `127.0.0.1:7878` 時，你應該會得到一個空白頁面，而不是錯誤了。你剛剛手寫了一個 HTTP 請求與回應！

回傳真正的 HTML

讓我們實作不止是回傳空白頁面的功能。首先在專案根目錄建立一個檔案 `hello.html`，而不是在 `src` 目錄內。你可以輸入任何你想要的 HTML，範例 20-4 示範了其中一種可能的範本。

檔案名稱：hello.html

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Hello!</title>
  </head>
  <body>
    <h1>Hello!</h1>
    <p>Hi from Rust</p>
  </body>
</html>

```

範例 20-4：用於回應的 HTML 檔案範本

這是最小化的 HTML 文件，其附有一個標頭與一些文字。為了要在收到請求後從伺服器回傳此檔案，我們要修改範例 20-5 的 `handle_connection` 來讀取 HTML 檔案、加進回應本體中然後傳送出去。

檔案名稱：src/main.rs

```

use std::fs;
// --省略--

fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&mut stream);
    let http_request: Vec<_> = buf_reader
        .lines()
        .map(|result| result.unwrap())
        .take_while(|line| !line.is_empty())
        .collect();

    let status_line = "HTTP/1.1 200 OK";
    let contents = fs::read_to_string("hello.html").unwrap();
    let length = contents.len();

    let response =
        format!("{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}");

    stream.write_all(response.as_bytes()).unwrap();
}

```

範例 20-5：將 *hello.html* 內容作為回應本體傳送出去

我們在 `use` 新增了 `fs` 來將標準函式庫中的檔案系統模組引入作用域。讀取檔案內容至字串的程式碼看起來會很熟悉，因為這在第十二章範例 12-4 當我們想在 I/O 專案中讀取檔案內容時就用過了。

接下來，我們使用 `format!` 來加入檔案內容來作為成功回應的本體。為了確保這是有效的 HTTP 回應，我們加上 `Content-Length` 標頭並設置為回應本體的大小，在此例中就是 `hello.html` 的大小。

透過 `cargo run` 執行此程式碼並在你的瀏覽器讀取 `127.0.0.1:7878`，你應該就會看到 HTML 的顯示結果了！

目前我們忽略了 `http_request` 中的請求資料，並毫無條件地回傳 HTML 檔案內容。這意味著如果你嘗試在瀏覽器中請求 `127.0.0.1:7878/something-else`，你還是會得到相同的 HTML 回應。這樣我們的伺服器是很受限的，而且這也不是大多數網頁伺服器會做的行為。我們想要依據請求自訂我們的回應，並只對格式良好的 `/` 請求回傳 HTML 檔案。

驗證請求並選擇性地回應

目前我們的網頁伺服器不管客戶端的請求為何，都會回傳 HTML 檔案。讓我們加個功能來在回傳 HTML 檔案前檢查瀏覽器請求是否為 `/`，如果瀏覽器請求的是其他的話就回傳錯誤。為此我們得修改 `handle_connection` 成像是範例 20-6 這樣。此新的程式碼會檢查收到的請求，比較是否符合 `/` 的前半部分，並增加了 `if` 與 `else` 區塊來處理不同請求。

檔案名稱：`src/main.rs`

```
// --省略--

fn handle_connection(mut stream: TcpStream) {
    let buf_reader = BufReader::new(&mut stream);
    let request_line = buf_reader.lines().next().unwrap().unwrap();

    if request_line == "GET / HTTP/1.1" {
        let status_line = "HTTP/1.1 200 OK";
        let contents = fs::read_to_string("hello.html").unwrap();
        let length = contents.len();

        let response = format!(
            "{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}"
        );

        stream.write_all(response.as_bytes()).unwrap();
    } else {
        // some other request
    }
}
```

範例 20-6：分開處理 `/` 與其他請求

我們只會查看 HTTP 請求的第一行，所以與其讀取整個請求到向量中，我們不如呼叫 `next` 來取得疊代器的第一個項目就好。第一個 `unwrap` 會處理 `Option`，如果疊代器沒有任何項目的話程

式就會停止。第二個 `unwrap` 處理的則是 `Result`，它和範例 20-2 `map` 裡的 `unwrap` 有相同的效果。

接著，我們檢查 `request_line` 是否等同於以 / 路徑形式寫成的 GET 請求。如果是的話，那麼 `if` 區塊就回傳 HTML 檔案的內容。

如果 `request_line` 並沒有符合在 / 路徑形式下的 GET 請求的話，代表我們收到的是其他請求。我們稍後會在 `else` 區塊加上回應其他所有請求的程式碼。

執行此程式碼並請求 `127.0.0.1:7878` 的話，你應該會收到 `hello.html` 的 HTML。如果你下達其他任何請求，像是 `127.0.0.1:7878/something-else` 的話，你會和執行範例 20-1 與 20-2 的程式碼時獲得一樣的錯誤。

現在讓我們將範例 20-7 的程式碼加入 `else` 區塊來回傳狀態碼為 404 的回應，這代表請求的內容無法找到。我們也會回傳一個 HTML 頁面讓瀏覽器能顯示並作為終端使用者的回應。

檔案名稱：src/main.rs

```
// --省略--
} else {
    let status_line = "HTTP/1.1 404 NOT FOUND";
    let contents = fs::read_to_string("404.html").unwrap();
    let length = contents.len();

    let response = format!(
        "{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}"
    );

    stream.write_all(response.as_bytes()).unwrap();
}
```

範例 20-7：如果不是 / 的請求就回應狀態碼 404 與一個錯誤頁面

我們在此的狀態行有狀態碼 404 與原因描述 `NOT FOUND`。回應的本體會是 `404.html` 檔案內的 HTML。你會需要在 `hello.html` 旁建立一個 `404.html` 檔案來作為錯誤頁面。同樣地，你可以使用任何你想使用的 HTML 或者使用範例 20-8 的 HTML 範本。

檔案名稱：404.html

```

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Hello!</title>
  </head>
  <body>
    <h1>Oops!</h1>
    <p>Sorry, I don't know what you're asking for.</p>
  </body>
</html>

```

範例 20-8：回傳任何 404 回應的頁面內容範本

有了這些改變後，再次執行你的伺服器。請求 `127.0.0.1:7878` 的話就應該會回傳 `hello.html` 的內容，而任何其他請求，像是 `127.0.0.1:7878/foo` 就應該回傳 `404.html` 的錯誤頁面。

再做一些重構

目前 `if` 與 `else` 區塊有很多重複的地方，它們都會讀取檔案並將檔案內容寫入流中。唯一不同的地方在於狀態行與檔案名稱。讓我們將程式碼變得更簡潔，將不同之處分配給 `if` 與 `else`，它們會分別將相對應的狀態行與檔案名稱賦值給變數。我們就能使用這些變數無條件地讀取檔案並寫入回應。範例 20-9 顯示了替換大段 `if` 與 `else` 區塊後的程式碼。

檔案名稱：`src/main.rs`

```

// --省略--

fn handle_connection(mut stream: TcpStream) {
    // --省略--

    let (status_line, filename) = if request_line == "GET / HTTP/1.1" {
        ("HTTP/1.1 200 OK", "hello.html")
    } else {
        ("HTTP/1.1 404 NOT FOUND", "404.html")
    };

    let contents = fs::read_to_string(filename).unwrap();
    let length = contents.len();

    let response =
        format!("{status_line}\r\nContent-Length: {length}\r\n\r\n{contents}");

    stream.write_all(response.as_bytes()).unwrap();
}

```

範例 20-9：重構 `if` 與 `else` 區塊來只包含兩個條件彼此不同的地方

現在 `if` 與 `else` 區塊只回傳狀態行與檔案名稱的數值至一個元組，我們可以在 `let` 陳述式使用模式來解構並將分別兩個數值賦值給 `status_line` 與 `filename`，如第十八章所提及的。

之前重複的程式碼現在位於 `if` 與 `else` 區塊之外並使用變數 `status_line` 與 `filename`。這讓我們更容易觀察兩種條件不同的地方，且也意味著如果我們想要變更讀取檔案與寫入回應的行為的話，我們只需要更新其中一段程式碼就好。範例 20-9 與範例 20-8 的程式碼行為一模一樣。

太棒了！我們現在有個用約莫 40 行 Rust 程式碼寫出的簡單網頁瀏覽器，可以對一種請求回應內容頁面，然後對其他所有請求回應 404 錯誤。

目前我們的伺服器只跑在單一執行緒，這意味著它一次只能處理一個請求。讓我們模擬些緩慢的請求來探討這為何會成為問題。然後我們會加以修正讓我們伺服器可以同時處理數個請求。

將單一執行緒伺服器轉換為多執行緒伺服器

現在的伺服器會依序處理請求，代表它處理完第一個連線之前，都無法處理第二個連線。如果伺服器收到越來越多請求，這樣的連續處理方式會變得越來越沒效率。如果伺服器收到一個會花很久時間才能處理完成的請求，之後的請求都得等待這個長時間的請求完成才行，就算新的請求能很快處理完成也是如此。我們需要修正此問題，但首先讓我們先觀察此問題怎麼發生的。

對目前伺服器實作模擬緩慢的請求

我們來觀察看看處理緩慢的請求如何影響我們目前伺服器實作中的其他請求。範例 20-10 實作了處理 `/sleep` 的請求，其在回應前讓伺服器沉睡 5 秒鐘來模擬緩慢的回應。

檔案名稱：src/main.rs

```
use std::{
    fs,
    io::{prelude::*, BufReader},
    net::{TcpListener, TcpStream},
    thread,
    time::Duration,
};
// --省略--

fn handle_connection(mut stream: TcpStream) {
    // --省略--

    let (status_line, filename) = match &request_line[..] {
        "GET / HTTP/1.1" => ("HTTP/1.1 200 OK", "hello.html"),
        "GET /sleep HTTP/1.1" => {
            thread::sleep(Duration::from_secs(5));
            ("HTTP/1.1 200 OK", "hello.html")
        }
        _ => ("HTTP/1.1 404 NOT FOUND", "404.html"),
    };
    // --省略--
}
```

範例 20-10：沉睡 5 秒鐘來模擬緩慢的請求

由於我們現在有三種情況了，我們將從 `if` 改成 `match`。我們需要用字串字面值數值來配對 `request_line`。 `match` 不會像相等方法那樣自動參考和解參考。

第一個分支和範例 20-9 的 `if` 區塊相同。第二個分支配對的請求是 `/sleep`。當收到請求時，伺服器會在成功顯示 HTML 頁面之前沈睡 5 秒。第三個和範例 20-9 的 `else` 區塊相同。

你可以看出我們的伺服器有多基本：真實的函式庫會以較不冗長的方式來識別處理數種請求！

使用 `cargo run` 來啟動伺服器，然後開啟兩個瀏覽器視窗：一個請求 `http://127.0.0.1:7878/` 然後另一個請求 `http://127.0.0.1:7878/sleep`。如果你輸入好幾次 `/` URI 的話，你會如之前一樣迅速地收到回應。但如果你先輸入 `/sleep` 在讀取 `/` 的話，你會看到 `/` 得等待 `sleep` 沉睡整整 5 秒鐘後才能讀取。

我們有好幾種方式能避免緩慢請求造成的請求堆積。其中一種就是我們要實作的執行緒池 (thread pool)。

透過執行緒池改善吞吐量

執行緒池 (thread pool) 會產生一群執行緒來等待並隨時準備好處理任務。當程式收到新任務時，它會將此任務分配給執行緒池其中一條執行緒，然後該執行緒就會處理該任務。池中剩餘的執行緒在第一條執行緒處理任務時，仍能隨時處理任何其他來臨的任務。當第一條執行緒處理完成時，他會回到閒置執行緒池之中，等待處理新的任務。執行緒池讓你能並行處理連線，增加伺服器的吞吐量。

我們會限制執行緒池的數量為少量的數量就好，以避免我們遭受阻斷服務 (Denial of Service, DOS) 攻擊。如果我們的程式每次遇到新的請求時就產生新的執行緒，某個人就可以產生一千萬個請求至我們的伺服器，來破壞並用光我們伺服器的資源，並導致所有請求的處理都被擱置。

所以與其產生無限制的執行緒，我們會有個固定數量的執行緒在池中等待。當有請求來臨時，它們會被送至池中處理。此池會維護一個接收請求的佇列 (queue)。每個執行緒會從此佇列彈出一個請求、處理該請求然後再繼續向佇列索取下一個請求。有了此設計，我們就可以同時處理 `N` 個請求，其中 `N` 就是執行緒的數量。如果每個執行緒都負責到需要長時間處理的請求，隨後的請求還是會阻塞佇列，但是我們至少增加了能夠同時處理長時間請求的數量。

此技巧只是其中一種改善網頁伺服器吞吐量的方式而已。其他你可能會探索到的選項還有 **fork/join 模型**、**單執行緒非同步模型**或**多執行緒非同步模型**。如果你對此議題有興趣，你可以閱讀其他解決方案，並嘗試實作到 Rust 中。像 Rust 這種低階語言，這些所有選項都是可能的。

在我們開始實作執行緒池之前，讓我們討論一下使用該池會是什麼樣子。當你嘗試設計程式碼時，先寫出使用者的介面能協助引導你的設計。寫出程式碼的 API，使其能以你所期望的方式呼叫，然後在該結構內實作功能，而不是先實作功能再設計公開 API。

類似於第十二章的專案所用到的測試驅動開發 (test-driven development)，我們會在此使用編譯器驅動開發方式。我們會先寫出呼叫所預期函式的程式碼，然後觀察編譯器的錯誤來決定接下來該改變什麼，才能讓程式碼成功運行。不過在那之前，讓我們先觀察一些我們最後不會使用的方式作為起始點。

對每個請求都產生執行緒

首先，讓我們先探討如果我們的程式碼都對每次連線建立新的執行緒會怎樣。如之前提及的，這不會是我們最終的計劃，因為這有可能會產生無限條執行緒的問題，但對於討論多執行緒伺服器來說，這是個很好的起始點。我們接下來會加入執行緒池來改善，然後比較兩者誰比較簡單。範例 20-11 在 `main` 的 `for` 迴圈中，對每個流都產生一條新的執行緒。

檔案名稱：src/main.rs

```
fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();

    for stream in listener.incoming() {
        let stream = stream.unwrap();

        thread::spawn(|| {
            handle_connection(stream);
        });
    }
}
```

範例 20-11：對每個流都產生新的一條執行緒

如你在第十六章所學到的，`thread::spawn` 會建立一條執行緒並在新的執行緒執行閉包的程式碼。如果你執行此程式碼，並在瀏覽器中讀取 `/sleep`，然後在開兩個瀏覽器分頁來讀取 `/` 的話，你的確就能看到 `/` 請求不必等待 `/sleep` 完成。但如我們所提的，這最終可能會拖累系統，因為你可以無限制地產生新的執行緒。

建立數量有限的執行緒

我們想要我們的執行緒池能以類似的方式運作，這樣從執行緒切換成執行緒池時，使用我們 API 的程式碼就不必作出大量修改。範例 20-12 顯示一個我們想使用的假想 `ThreadPool` 結構體，而非使用 `thread::spawn`。

檔案名稱：src/main.rs

```
fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();
    let pool = ThreadPool::new(4);

    for stream in listener.incoming() {
        let stream = stream.unwrap();

        pool.execute(|| {
            handle_connection(stream);
        });
    }
}
```



範例 20-12：我們理想的 `ThreadPool` 介面

我們使用 `ThreadPool::new` 來建立新的執行緒池且有個可設置的執行緒數量參數，在此例中設為四。然後在 `for` 迴圈中，`pool.execute` 的介面類似於 `thread::spawn`，其會接收一個執行緒池執行在每個流中的閉包。我們需要實作 `pool.execute`，使其能接收閉包並傳給池中的執行緒來執行。此程式碼還不能編譯，但是我們接下來能試著讓編譯器引導我們如何修正。

透過編譯器驅動開發建立 `ThreadPool`

將範例 20-12 的變更寫入 `src/main.rs`，然後讓我們從 `cargo check` 產生的編譯器錯誤來引導我們的開發吧。以下是我們第一個收到的錯誤：

```
$ cargo check
    Checking hello v0.1.0 (file:///projects/hello)
error[E0433]: failed to resolve: use of undeclared type `ThreadPool`
  --> src/main.rs:11:16
   |
11 |     let pool = ThreadPool::new(4);
   |                  ^^^^^^^^^^^^^ use of undeclared type `ThreadPool`

For more information about this error, try `rustc --explain E0433`.
error: could not compile `hello` due to previous error
```

很好！此錯誤告訴我們需要一個 `ThreadPool` 型別或模組，所以現在就讓我們來建立一個。我們的 `ThreadPool` 實作會與網頁伺服器相互獨立，所以讓我們將 `hello` crate 從執行檔 crate 轉換成函式庫 crate 來存放我們的 `ThreadPool` 實作。這樣在我們切換成函式庫 crate 之後，我們就能夠將分出來的執行緒池函式庫用在其他我們想使用執行緒池的地方，而不僅僅是作為網頁請求所用。

建立一個包含以下內容的 `src/lib.rs`，這是我們現在所能寫出最簡單的 `ThreadPool` 結構體定義了：

檔案名稱：`src/lib.rs`

```
pub struct ThreadPool;
```

然後編輯 `main.rs` 檔案將 `ThreadPool` 從函式庫 `crate` 引入作用域，請將以下程式碼寫入 `src/main.rs` 最上方：

檔案名稱：`src/main.rs`

```
use hello::ThreadPool;
```

此程式碼仍然無法執行，讓我們再次檢查並取得下一個要解決的錯誤：

```
$ cargo check
    Checking hello v0.1.0 (file:///projects/hello)
error[E0599]: no function or associated item named `new` found for struct
`ThreadPool` in the current scope
--> src/main.rs:12:28
   |
12 |     let pool = ThreadPool::new(4);
   |                                ^^^ function or associated item not found in
`ThreadPool`

For more information about this error, try `rustc --explain E0599`.
error: could not compile `hello` due to previous error
```

此錯誤指示我們需要對 `ThreadPool` 建立個關聯函式叫做 `new`。我們還知道 `new` 需要有個參數來接受作為引數的 `4`，並需要回傳 `ThreadPool` 實例。讓我們來實作擁有這些特性的最簡單 `new` 函式：

檔案名稱：`src/lib.rs`

```
pub struct ThreadPool;

impl ThreadPool {
    pub fn new(size: usize) -> ThreadPool {
        ThreadPool
    }
}
```

我們選擇 `usize` 作為參數 `size` 的型別，因為我們知道負數對執行緒數量來說沒有任何意義。我們也知道 `4` 會作為執行緒集合的元素個數，這正是使用 `usize` 型別的原因，如同第三章「[整數型別](#)」段落所講的。

讓我們再檢查程式碼一次：


```
$ cargo check
    Checking hello v0.1.0 (file:///projects/hello)
error[E0599]: no method named `execute` found for type `ThreadPool` in the
current scope
  --> src/main.rs:17:14
   |
17 |         pool.execute(|| {
   |                   ^^^^^^^ method not found in `ThreadPool`

For more information about this error, try `rustc --explain E0599`.
error: could not compile `hello` due to previous error
```

現在錯誤的原因是因為我們的 `ThreadPool` 沒有 `execute` 方法。回想一下「[建立數量有限的執行緒](#)」段落中，我們決定我們的執行緒池要有類似於 `thread::spawn` 的介面。除此之外，我們會實作 `execute` 函式使其接收給予的閉包並傳至執行緒池中閒置的執行緒來執行。

我們定義 `ThreadPool` 的 `execute` 方法接收一個閉包來作為參數。回憶一下第十三章的「[Fn 特徵以及將獲取的數值移出閉包](#)」段落中，我們可以透過三種不同的特徵來接受閉包：`Fn`、`FnMut` 與 `FnOnce`。我們需要決定這裡該使用何種閉包。我們知道我們的行為會類似於標準函式庫中 `thread::spawn` 的實作，所以讓我們看看 `thread::spawn` 簽名中的參數有哪些界限吧。技術文件會顯示以下結果給我們：

```
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
    where
        F: FnOnce() -> T,
        F: Send + 'static,
        T: Send + 'static,
```

`F` 型別參數正是我們所在意的，`T` 型別則是與回傳型別有關，而我們目前並不在意。我們可以看到 `spawn` 使用 `FnOnce` 作為 `F` 的界限。這大概就是我們也想要的，因為我們最終會將 `execute` 的引數傳遞給 `spawn`。我們現在更確信 `FnOnce` 就是我們想使用的特徵，因為執行請求的執行緒只會執行該請求閉包一次，這正符合 `FnOnce` 中 `Once` 的意思。

`F` 型別參數還有個特徵界限 `Send` 與生命週期界限 `'static`，這在我們的場合中也很實用，我們需要 `Send` 來將閉包從一個執行緒轉移到另一個，而會需要 `'static` 是因為我們不知道執行緒會處理多久。讓我們對 `ThreadPool` 建立 `execute` 方法，並採用泛型參數型別 `F` 與其界限：

檔案名稱：src/lib.rs

```
impl ThreadPool {
    // --省略--
    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
    }
}
```

我們在 `FnOnce` 之後仍然使用 `()`，因為此 `FnOnce` 代表閉包沒有任何參數且回傳值為單元型別 `()`。與函式定義一樣，回傳型別可以在簽名中省略，但是儘管我們沒有任何參數，我們還是得加上括號。

同樣地，這是 `execute` 方法最簡單的實作，它不會做任何事情，但是我們指示要先讓我們的程式碼能夠編譯通過。讓我們再次檢查：

```
$ cargo check
    Checking hello v0.1.0 (file:///projects/hello)
    Finished dev [unoptimized + debuginfo] target(s) in 0.24s
```

編譯通過了！但值得注意的是如果你嘗試 `cargo run` 並在瀏覽器下請求的話，你會像本章開頭一樣在瀏覽器看到錯誤。我們的函式庫還沒有實際呼叫傳至 `execute` 的閉包！

注意：你可能聽過對於像是 Haskell 和 Rust 這種嚴格編譯器的語言，會號稱「如果程式碼能編譯，它就能正確執行。」但這不全然是正確的。我們的專案能編譯，但是它沒有做任何事！如果我們在寫的是實際的完整專案，這是個寫單元測試的好時機，這能檢查程式碼能編譯而且有我們的預期行為。

在 `new` 驗證執行緒數量

我們對 `new` 與 `execute` 的參數沒有做任何事情。讓我們對這些函式本體實作出我們所預期的行為吧。我們先從 `new` 開始。稍早我們選擇非帶號型別作為 `size` 的參數，因為負數對於執行緒數量並沒有任何意義。然而，零條執行緒的池一樣也沒有任何意義，但零卻可以是完全合理的 `usize`。我們要在回傳 `ThreadPool` 前，加上程式碼來檢查 `size` 有大於零，並透過 `assert!` 來判定。如果為零的話就會恐慌，如範例 20-13 所示。

檔案名稱：`src/lib.rs`

```
impl ThreadPool {
    /// Create a new ThreadPool.
    ///
    /// The size is the number of threads in the pool.
    ///
    /// # Panics
    ///
    /// The `new` function will panic if the size is zero.
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        ThreadPool
    }

    // --省略--
}
```

範例 20-13：實作 `ThreadPool::new` 且如果 `size` 為零時就會恐慌

我們透過技術文件註解來對 `ThreadPool` 加上技術文件說明。注意到我們有加上一個段落說明何種情況呼叫函式會恐慌，這樣我們就有遵守良好的技術文件典範，如同第十四章所討論過的。嘗試執行 `cargo doc --open` 然後點擊 `ThreadPool` 結構體來看看 `new` 產生出的技術文件長什麼樣子！

除了像我們這樣使用 `assert!` 巨集之外，我們也可以將 `new` 改成 `build` 來回傳 `Result`，就像範例 12-9 我們對 I/O 專案的 `Config::build` 所做的一樣。但是我們決定在此情況中，嘗試建立零條執行緒的池應該要是不可回復的錯誤。如果你有信心的話，你可以試著寫出有以下簽名的 `build` 版本，並比較與 `new` 函式之間的區別：

```
pub fn build(size: usize) -> Result<ThreadPool, PoolCreationError> {
```

建立執行緒的儲存空間

現在我們有一個有效的執行緒數量能儲存至池中，我們可以在回傳實例前，建立這些執行緒並儲存至 `ThreadPool` 結構體中。但我們要怎麼「儲存」執行緒呢？讓我們再看一次 `thread::spawn` 的簽名：

```
pub fn spawn<F, T>(f: F) -> JoinHandle<T>
where
    F: FnOnce() -> T,
    F: Send + 'static,
    T: Send + 'static,
```

`spawn` 函式會回傳 `JoinHandle<T>`，而 `T` 為閉包回傳的型別。讓我們也試著使用 `JoinHandle` 來看看會發生什麼事。在我們的情況中，我們傳遞至執行緒池的閉包會處理連線但不會回傳任何

值，所以 `T` 就會是單元型別 `()`。

範例 20-14 的程式碼可以編譯，但還不會產生任何執行緒。我們變更了 `ThreadPool` 的定義來儲存一個有 `thread::JoinHandle<()>` 實例的向量，用 `size` 來初始化向量的容量，設置一個會執行些程式碼來建立執行緒的 `for` 迴圈，然後回傳包含它們的 `ThreadPool` 實例。

檔案名稱：`src/lib.rs`

```
use std::thread;

pub struct ThreadPool {
    threads: Vec<thread::JoinHandle<()>>,
}

impl ThreadPool {
    // --省略--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let mut threads = Vec::with_capacity(size);

        for _ in 0..size {
            // 將產生些執行緒並儲存至向量
        }

        ThreadPool { threads }
    }

    // --省略--
}
```



範例 20-14：在 `ThreadPool` 中建立向量來儲存執行緒

我們將 `std::thread` 引入函式庫 `crate` 中的作用域，因為我們使用 `thread::JoinHandle` 作為 `ThreadPool` 中向量的項目型別。

一旦有收到有效大小，`ThreadPool` 就會建立一個可以儲存 `size` 個項目的新向量。

`with_capacity` 函式會與 `Vec::new` 做同樣的事，但是有一個關鍵差別：它會預先配置空間給向量。由於我們知道要儲存 `size` 個元素至向量中，這樣的配置方式會比 `Vec::new` 還要略為有效一點，因為後者只會在元素插入時才重新配置自身大小。

當你再次執行 `cargo check`，這次就能成功編譯。

結構體 `Worker` 負責從 `ThreadPool` 傳遞程式碼給一條執行緒

我們在範例 20-14 的 `for` 迴圈中留下一個關於建立執行緒的註解。我們在此將看看我們該如何實際建立執行緒。標準函式庫提供 `thread::spawn` 作為建立執行緒的方式，然後 `thread::spawn`

預期在執行緒建立時就會獲得一些程式碼讓執行緒能夠執行。但在我們的場合中，我們希望建立執行緒，並讓它們等待我們之後會傳送的程式碼。標準函式庫的執行緒實作並不包含這種方式，我們得自己實作。

我們實作此行為的方法是在 `ThreadPool` 與執行緒間建立一個新的資料結構，這用來管理此新的行為。我們將此資料結構稱為 `Worker`，這在池實作中是很常見的術語。`Worker` 拿取要執行的程式碼然後在自己的執行緒跑這段程式碼。想像一下這是有一群人在餐廳廚房內工作：工作者（worker）會等待顧客的訂單，然後他們負責接受這些訂單並完成它們。

所以與其在執行緒池中儲存 `JoinHandle<()>` 實例的向量，我們可以儲存 `Worker` 結構體的實例。每個 `Worker` 會儲存一個 `JoinHandle<()>` 實例。然後對 `Worker` 實作一個方法來取得閉包要執行的程式碼，並傳入已經在執行的執行緒來處理。我們也會給每個 `Worker` 一個 `id`，好讓我們在記錄日誌或除錯時，分辨池中不同的工作者。

當我們建立 `ThreadPool` 時會發生以下事情。我們會用以下方式在設置完 `Worker` 後，實作將閉包傳遞給執行緒的程式碼：

1. 定義 `Worker` 結構體存有 `id` 與 `JoinHandle<()>`。
2. 變更 `ThreadPool` 改儲存 `Worker` 實例的向量。
3. 定義 `Worker::new` 函式來接收 `id` 數字並回傳一個 `Worker` 實例，其包含該 `id` 與一條具有空閉包的執行緒。
4. 在 `ThreadPool::new` 中，使用 `for` 迴圈計數來產生 `id`，以此建立對應 `id` 的新 `Worker`，並將其儲存至向量中。

如果你想要挑戰看看的話，你可以試著先自己實作這些改變，再來查看範例 20-15 的程式碼。

準備好了嗎？以下是範例 20-15 作出修改的方式。

檔案名稱：`src/lib.rs`

```

use std::thread;

pub struct ThreadPool {
    workers: Vec<Worker>,
}

impl ThreadPool {
    // --省略--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id));
        }

        ThreadPool { workers }
    }
    // --省略--
}

struct Worker {
    id: usize,
    thread: thread::JoinHandle<()>,
}

impl Worker {
    fn new(id: usize) -> Worker {
        let thread = thread::spawn(|| {});

        Worker { id, thread }
    }
}

```

範例 20-15：變更 `ThreadPool` 來儲存 `Worker` 實例，而非直接儲存執行緒

我們將 `ThreadPool` 中欄位的名稱從 `threads` 改為 `workers`，因為它現在儲存的是 `Worker` 實例而非 `JoinHandle<()>` 實例。我們使用 `for` 迴圈的計數作為 `Worker::new` 的引數，然後我們將每個新的 `Worker` 儲存到名為 `workers` 的向量中。

外部的程式碼（像是我們在 `src/main.rs` 的伺服器）不需要知道 `ThreadPool` 內部實作細節已經改為使用 `Worker` 結構體，所以我們讓 `Worker` 結構體與其 `new` 函式維持私有。 `Worker::new` 函式會使用我們給予的 `id` 並儲存一個 `JoinHandle<()>` 實例，這是用空閉包產生的新執行緒所建立的。

注意：如果作業系統因為系統資源不足，而無法建立執行緒的話，`thread::spawn` 會恐慌。這會使我們的伺服器恐慌，就算有些執行緒能成功建立。基於簡潔原則，這段程式碼還算能接受。但如果是正式環境的執行緒實作，你可能會想使用 `std::thread::Builder` 與其 `spawn` 方法來回傳 `Result`。

此程式碼會編譯通過並透過 `ThreadPool::new` 的指定引數儲存一定數量的 `Worker` 實例。但我們仍然沒有處理 `execute` 中取得的閉包。讓我們看看接下來怎麼做。

透過通道傳遞請求給執行緒

接下來我們要來處理的問題是 `thread::spawn` 中的閉包不會做任何事情。目前我們透過 `execute` 取得我們想執行的閉包。但是我們當在 `ThreadPool` 的產生中建立每個 `Worker` 時，我們會需要給 `thread::spawn` 一個閉包來執行。

我們想要我們建立的 `Worker` 結構體能夠從 `ThreadPool` 中的佇列提取程式碼來執行，並將該程式碼傳至自身的執行緒來執行。

我們在第十六章中學過的**通道**（channels）是個能在兩個執行緒間溝通的好辦法，這對我們的專案來說可說是絕佳解法。我們會用通道來作為任務佇列，然後 `execute` 來傳送從 `ThreadPool` 一份任務至 `Worker` 實例，其就會傳遞該任務給自身的執行緒。以下是我們的計劃：

1. `ThreadPool` 會建立通道並儲存發送者。
2. 每個 `Worker` 會持有接收者。
3. 我們會建立一個新的結構體 `Job` 來儲存我們想傳入通道的閉包。
4. `execute` 方法將會傳送其想執行的 `Job` 至發送者。
5. 在其執行緒中，`Worker` 會持續遍歷接收者並執行它所收到的任何任務閉包。

讓我們先在 `ThreadPool::new` 建立通道並讓 `ThreadPool` 實例儲存發送者，如範例 20-16 所示。現在結構體 `Job` 還不會儲存任何東西，但是它最終會是我們傳送給通道的型別。

檔案名稱：src/lib.rs

```
// --省略--
use std::{sync::mpsc, thread};

pub struct ThreadPool {
    workers: Vec<Worker>,
    sender: mpsc::Sender<Job>,
}

struct Job;

impl ThreadPool {
    // --省略--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id));
        }

        ThreadPool { workers, sender }
    }
    // --省略--
}
```

範例 20-16：變更 `ThreadPool` 來儲存發送者以傳送 `Job` 實例

在 `ThreadPool::new` 中，我們建立了一個新的通道並讓執行緒池儲存發送者。這能成功編譯，但還是會有些警告。

讓我們嘗試在執行緒池建立通道時，將接收者傳給每個 `Worker`。我們知道我們想在 `Worker` 產生的執行緒中使用接收者，所以我們得在閉包中參考 `receiver` 參數。不過範例 20-17 的程式碼還不能編譯過。

檔案名稱：src/lib.rs


```
impl ThreadPool {
    // --省略--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id, receiver));
        }

        ThreadPool { workers, sender }
    }
    // --省略--
}

// --省略--

impl Worker {
    fn new(id: usize, receiver: mpsc::Receiver<Job>) -> Worker {
        let thread = thread::spawn(|| {
            receiver;
        });

        Worker { id, thread }
    }
}
```



範例 20-17：傳遞接收者給每個工作者

我們做了一些小小卻直觀的改變：我們將接收者傳給 `Worker::new`，然後我們在閉包中使用它。

當我們檢查此程式碼時，我們會得到以下錯誤：

```

$ cargo check
    Checking hello v0.1.0 (file:///projects/hello)
error[E0382]: use of moved value: `receiver`
  --> src/lib.rs:26:42
   |
21 |         let (sender, receiver) = mpsc::channel();
   |                                ----- move occurs because `receiver` has type
   |                                `std::sync::mpsc::Receiver<Job>`, which does not implement the `Copy` trait
...
26 |         workers.push(Worker::new(id, receiver));
   |                                ^^^^^^^^^^^ value moved here, in
previous iteration of loop

For more information about this error, try `rustc --explain E0382`.
error: could not compile `hello` due to previous error

```

程式碼嘗試將 `receiver` 傳給數個 `Worker` 實例。回憶第十六章的話，你就知道這不會成功：Rust 提供的通道實作是多重生產者、單一消費者。這意味著我們不能只是克隆接收者來修正此程式碼。我們也不想重複傳送一個訊息給多重消費者，我們想要的是由數個工作者建立的訊息列表，然後每個訊息只會被處理一次。

除此之外，從通道佇列取得任務會需要可變的 `receiver`，所以執行緒需要有個安全的方式來共享並修改 `receiver`。不然的話，我們可能會遇到競爭條件（如第十六章所提及的）。

回想一下第十六章討論到的執行緒安全智慧指標：要在多重執行緒共享所有權並允許執行緒改變數值的話，我們需要使用 `Arc<Mutex<T>>`。Arc 型別能讓數個工作者能擁有接收端，而 `Mutex` 能確保同時間只有一個工作者能獲取任務。範例 20-18 顯示了我們需要作出的改變：

檔案名稱：src/lib.rs

```

use std::{
    sync::{mpsc, Arc, Mutex},
    thread,
};
// --省略--

impl ThreadPool {
    // --省略--
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let receiver = Arc::new(Mutex::new(receiver));

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id, Arc::clone(&receiver)));
        }

        ThreadPool { workers, sender }
    }

    // --省略--
}

// --省略--

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        // --省略--
    }
}

```

範例 20-18：透過 `Arc` 與 `Mutex` 來在工作者間共享接收者

在 `ThreadPool::new` 中，我們將接收者放入 `Arc` 與 `Mutex` 之中。對於每個新的工作者，我們會克隆 `Arc` 來增加參考計數，讓工作者可以共享接收者的所有權。

有了這些改變，程式碼就能編譯了！我們就快完成了！

實作 `execute` 方法

最後讓我們來對 `ThreadPool` 實作 `execute` 方法吧。我們還會將 `Job` 的型別從結構體改為特徵物件的型別別名，這會儲存 `execute` 收到的閉包型別。如同在第十九章的「[透過型別別名建立型](#)

別同義詞」段落所介紹的，型別別名讓我們能將很長的型別變短一些以便使用，如範例 20-19 所示。

檔案名稱：src/lib.rs

```
// --省略--

type Job = Box<dyn FnOnce() + Send + 'static>;

impl ThreadPool {
    // --省略--

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);

        self.sender.send(job).unwrap();
    }
}

// --省略--
```

範例 20-19：建立一個對 `Box` 的型別別名 `Job`，其存有每個閉包並傳送至通道

在使用 `execute` 收到的閉包來建立新的 `Job` 實例之後，我們將該任務傳送至發送者。我們對 `send` 呼叫 `unwrap` 來處理發送失敗的情況。舉例來說，這可能會發生在當我們停止所有執行緒時，這意味著接收端不再接收新的訊息。不過目前我們還無法讓我們的執行緒停止執行，只要執行緒池還在我們的執行緒就會繼續執行。我們使用 `unwrap` 的原因是因為我們知道失敗不可能發生，但編譯器並不知情。

不過我們還沒結束呢！在工作者中，傳給 `thread::spawn` 的閉包仍然只有參考接收者。我們需要讓閉包一直循環，向接收者請求任務，並在取得任務時執行它。讓我們對 `Worker::new` 加上範例 20-20 的程式碼。

檔案名稱：src/lib.rs

```
// --省略--

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        let thread = thread::spawn(move || loop {
            let job = receiver.lock().unwrap().recv().unwrap();

            println!("Worker {id} got a job; executing.");

            job();
        });

        Worker { id, thread }
    }
}
```

範例 20-20：在工作者的執行緒中接收並執行任務

我們在此首先對 `receiver` 呼叫 `lock` 以取得互斥鎖，然後我們呼叫 `unwrap` 讓任何錯誤都會恐慌。如果互斥鎖處於污染（poisoned）狀態的話，該鎖可能就會失敗，這在其他執行緒持有鎖時，卻發生恐慌而沒有釋放鎖的話就可能發生。在這種情形，呼叫 `unwrap` 來讓此執行緒恐慌是正確的選擇。你也可以將 `unwrap` 改成 `expect` 來加上一些對你更有幫助的錯誤訊息。

如果我們得到互斥鎖的話，我們呼叫 `recv` 來從通道中取得 `Job`。最後的 `unwrap` 也繞過了任何錯誤，這在持有發送者的執行緒被關閉時就可能發生；就和如果接收端關閉時 `send` 方法就會回傳 `Err` 的情況類似。

`recv` 的呼叫會阻擋執行緒，所以如果沒有任何任務的話，當前執行緒將等待直到下一個任務出現為止。`Mutex<T>` 確保同時間只會有一個 `Worker` 執行緒嘗試取得任務。

我們的執行緒池終於可以運作了！賞它個 `cargo run` 然後下達一些請求吧：

```

$ cargo run
   Compiling hello v0.1.0 (file:///projects/hello)
warning: field is never read: `workers`
--> src/lib.rs:7:5
|
7 |     workers: Vec<Worker>,
|     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= note: `[warn(dead_code)]` on by default

warning: field is never read: `id`
--> src/lib.rs:48:5
|
48 |     id: usize,
|     ^^^^^^^^^
|

warning: field is never read: `thread`
--> src/lib.rs:49:5
|
49 |     thread: thread::JoinHandle<()>,
|     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|

warning: `hello` (lib) generated 3 warnings
    Finished dev [unoptimized + debuginfo] target(s) in 1.40s
    Running `target/debug/hello`
Worker 0 got a job; executing.
Worker 2 got a job; executing.
Worker 1 got a job; executing.
Worker 3 got a job; executing.
Worker 0 got a job; executing.
Worker 2 got a job; executing.
Worker 1 got a job; executing.
Worker 3 got a job; executing.
Worker 0 got a job; executing.
Worker 2 got a job; executing.

```

成功了！我們現在有個執行緒池能非同步地處理連線。我們產生的執行緒不超過四條，所以如果伺服器收到大量請求時，我們的系統就不會超載。如果我們下達 `/sleep` 的請求，伺服器會有其他執行緒來處理其他請求並執行它們。

注意：如果你在數個瀏覽器視窗同時打開 `/sleep`，它們可能會彼此間隔 5 秒鐘來讀取。這是因為有些網頁瀏覽器會對多個相同請求的實例做快取。這項限制不是網頁伺服器造成的。

在學習過第十八章的 `while let` 迴圈後，你可能會好奇為何我們不像範例 20-21 這樣來寫工作者執行緒的程式碼。

檔案名稱：src/lib.rs

```
// --省略--
```

```
impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        let thread = thread::spawn(move || {
            while let Ok(job) = receiver.lock().unwrap().recv() {
                println!("Worker {id} got a job; executing.");

                job();
            }
        });

        Worker { id, thread }
    }
}
```



範例 20-21：使用 `while let` 來實作 `Worker::new` 的替代方案

此程式碼能編譯並執行，但不會是有我們預期的執行緒行為：緩慢的請求仍然會卡住其他請求。發生的原因有點微妙，`Mutex` 結構體沒有公開的 `unlock` 方法，這是因為鎖的所有權是依據 `lock` 方法所回傳的 `LockResult<MutexGuard<T>>` 中 `MutexGuard<T>` 的生命週期。在編譯時借用檢查器可以以此確保沒有持有鎖的話，我們就無法取得 `Mutex` 守護的資源。不過沒有仔細思考 `MutexGuard<T>` 的生命週期的話，此實作可能就會導致持有鎖的時間比預期的更久。

在範例 20-20 程式碼中的 `let job = receiver.lock().unwrap().recv().unwrap();` 可以這樣寫的原因是因為用的是 `let`，等號右方任何表達式中的暫時數值都會在 `let` 陳述式結束時釋放。然而 `while let`（還有 `if let` 和 `match`）是不會釋放暫時數值的，直到其區塊結束為止。在範例 20-21 中，在呼叫 `job` 的這段期間內，鎖都會持續鎖著，代表其他工作者無法取得工作。

正常關機與清理

範例 20-20 的程式碼能如我們所預期地使用執行緒池來同時回應多重請求。我們有看到些警告說 `workers`、`id` 與 `thread` 欄位沒有被直接使用，這提醒我們尚未清理所有內容。當我們使用比較不優雅的 `ctrl-c` 方式來中斷主執行緒時，所有其他執行緒也會立即停止，不管它們是否正在處理請求。

接著我們要實作 `Drop` 特徵來對池中的每個執行緒呼叫 `join`，讓它們能在關閉前把任務處理完畢。然後我們要實作個方式來告訴執行緒它們該停止接收新的請求並關閉。為了觀察此程式碼的實際運作，我們會修改伺服器讓它在正常關機（graceful shutdown）前，只接收兩個請求。

對 `ThreadPool` 實作 `Drop` 特徵

讓我們先對執行緒池實作 `Drop`。當池被釋放時，我們的執行緒都該加入（`join`）回來以確保它們有完成它們的工作。範例 20-22 為實作 `Drop` 的第一次嘗試，不過此程式碼還無法執行。

檔案名稱：src/lib.rs

```
impl Drop for ThreadPool {  
    fn drop(&mut self) {  
        for worker in &mut self.workers {  
            println!("Shutting down worker {}", worker.id);  
  
            worker.thread.join().unwrap();  
        }  
    }  
}
```



範例 20-22：在執行緒池離開作用域將每個執行緒加入回來

首先，我們遍歷執行緒池中的每個 `workers`。我們對此使用 `&mut` 因為 `self` 是個可變參考，而且我們也需要能夠改變 `worker`。我們對每個工作者印出訊息來說明此工作者正要關閉，然後我們對工作者的執行緒呼叫 `join`。如果 `join` 的呼叫失敗的話，我們使用 `unwrap` 來讓 Rust 恐慌使其變成較不正常的關機方式。

以下是當我們編譯此程式碼時產生的錯誤：


```
$ cargo check
    Checking hello v0.1.0 (file:///projects/hello)
error[E0507]: cannot move out of `worker.thread` which is behind a mutable
reference
    --> src/lib.rs:52:13
    |
52  |         worker.thread.join().unwrap();
    |         ^^^^^^^^^^^^^^^^^^^^^ ----- `worker.thread` moved due to this method
call
    |
    |         move occurs because `worker.thread` has type `JoinHandle<()>`,
which does not implement the `Copy` trait
    |
note: this function takes ownership of the receiver `self`, which moves
`worker.thread`

For more information about this error, try `rustc --explain E0507`.
error: could not compile `hello` due to previous error
```

錯誤告訴我們無法呼叫 `join`，因為我們只有每個 `worker` 的可變借用，而 `join` 會取走其引數的所有權。要解決此問題，我們需要將 `thread` 中的執行緒移出 `Worker` 實例，讓 `join` 可以消耗該執行緒。我們在範例 17-15 做過這樣的事，如果 `Worker` 改持有

`Option<thread::JoinHandle<()>>` 的話，我們可以對 `Option` 呼叫 `take` 方法來移動 `Some` 變體中的數值，並在原處留下 `None` 變體。換句話說，`thread` 中有 `Some` 變體的話就代表 `Worker` 正在執行，而當我們清理 `Worker` 時，我們會將 `Some` 換成 `None` 來讓 `Worker` 沒有任何執行緒可以執行。

所以我們想要更新 `Worker` 的定義如下所示：

檔案名稱：src/lib.rs

```
struct Worker {
    id: usize,
    thread: Option<thread::JoinHandle<()>>,
}
```



現在讓我們再看看編譯器的結果中還有哪些地方需要修改。檢查此程式碼，我們會得到兩個錯誤：

```

$ cargo check
    Checking hello v0.1.0 (file:///projects/hello)
error[E0599]: no method named `join` found for enum `Option` in the current
scope
  --> src/lib.rs:52:27
   |
52 |         worker.thread.join().unwrap();
   |                        ^^^^^ method not found in
`Option<JoinHandle<()>>`
   |
note: the method `join` exists on the type `JoinHandle<()>`
help: consider using `Option::expect` to unwrap the `JoinHandle<()>` value,
panicking if the value is an `Option::None`
   |
52 |         worker.thread.expect("REASON").join().unwrap();
   |                        ++++++

error[E0308]: mismatched types
  --> src/lib.rs:72:22
   |
72 |         Worker { id, thread }
   |                   ^^^^^^^^ expected enum `Option`, found struct
`JoinHandle`
   |
   = note: expected enum `Option<JoinHandle<()>>`
             found struct `JoinHandle<_>`
help: try wrapping the expression in `Some`
   |
72 |         Worker { id, thread: Some(thread) }
   |                   ++++++      +

error: aborting due to 2 previous errors

Some errors have detailed explanations: E0308, E0599.
For more information about an error, try `rustc --explain E0308`.
error: could not compile `hello` due to 2 previous errors

```

讓我們來修復第二個錯誤，這指向程式碼中 `Worker::new` 的結尾。當我們建立新的 `Worker`，我們需要將 `thread` 的數值封裝到 `Some`。請作出以下改變來修正程式碼：

檔案名稱：src/lib.rs

```
impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        // --省略--

        Worker {
            id,
            thread: Some(thread),
        }
    }
}
```



而第一個錯誤則位在 `Drop` 的實作中。我們剛剛有提到我們打算對 `Option` 呼叫 `take` 來將 `thread` 移出 `worker`。所以以下改變就能修正：

檔案名稱：src/lib.rs

```
impl Drop for ThreadPool {
    fn drop(&mut self) {
        for worker in &mut self.workers {
            println!("Shutting down worker {}", worker.id);

            if let Some(thread) = worker.thread.take() {
                thread.join().unwrap();
            }
        }
    }
}
```



如同第十七章所討論的，`Option` 的 `take` 方法會取走 `Some` 變體的數值並在原地留下 `None`。我們使用 `if let` 來解構 `Some` 並取得執行緒，然後我們對執行緒呼叫 `join`。如果工作者的執行緒已經是 `None`，我們就知道該該工作者已經清理其執行緒了，所以沒有必要再處理。

對執行緒發送停止接收任務的信號

有了以上的改變，我們的程式碼就能成功編譯且沒有任何警告。但壞消息是此程式碼並沒有如我們所預期地運作。關鍵邏輯位於 `Worker` 實例中執行緒執行的閉包，現在雖然我們有呼叫 `join`，但這無法關閉執行緒，因為它們會一直 `loop` 來尋找任務執行。如果我們嘗試以目前的 `drop` 實作釋放 `ThreadPool` 的話，主執行緒會被阻擋，一直等待第一個執行緒處理完成。

要修正此問題，我們要修改 `ThreadPool drop` 的實作以及 `Worker` 內的一些程式碼。

首先我們先將 `ThreadPool drop` 的實作改成在執行緒完成前就顯式釋放 `sender`。範例 20-23 展示了 `ThreadPool` 顯式釋放 `sender`。我們使用處理執行緒時一樣的 `Option` 與 `take` 技巧來將 `sender` 移出 `ThreadPool`：

檔案名稱：src/lib.rs



```
pub struct ThreadPool {
    workers: Vec<Worker>,
    sender: Option<mpsc::Sender<Job>>,
}
// --省略--
impl ThreadPool {
    pub fn new(size: usize) -> ThreadPool {
        // --省略--

        ThreadPool {
            workers,
            sender: Some(sender),
        }
    }

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);

        self.sender.as_ref().unwrap().send(job).unwrap();
    }
}

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.sender.take());

        for worker in &mut self.workers {
            println!("Shutting down worker {}", worker.id);

            if let Some(thread) = worker.thread.take() {
                thread.join().unwrap();
            }
        }
    }
}
```

範例 20-23：在工作者執行緒加入回來前顯式釋放 `sender`

釋放 `sender` 會關閉通道，也就代表沒有任何訊息會再被傳送。工作者在無限迴圈呼叫的 `recv` 會回傳錯誤。在範例 20-24 中，我們改變 `Worker` 的迴圈來處理該狀況並正常退出迴圈，也就是說 `ThreadPool` `drop` 的實作呼叫 `join` 時，執行緒就會工作完成。

檔案名稱：src/lib.rs

```

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        let thread = thread::spawn(move || loop {
            let message = receiver.lock().unwrap().recv();

            match message {
                Ok(job) => {
                    println!("Worker {id} got a job; executing.");

                    job();
                }
                Err(_) => {
                    println!("Worker {id} disconnected; shutting down.");
                    break;
                }
            }
        });

        Worker {
            id,
            thread: Some(thread),
        }
    }
}

```

範例 20-24：當 `recv` 回傳錯誤時就顯式退出迴圈

要實際看到此程式碼的運作情形，讓我們修改 `main` 來在正常關閉伺服器前，只接收兩個請求，如範例 20-25 所示。

檔案名稱：src/bin/main.rs

```

fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();
    let pool = ThreadPool::new(4);

    for stream in listener.incoming().take(2) {
        let stream = stream.unwrap();

        pool.execute(|| {
            handle_connection(stream);
        });
    }

    println!("Shutting down.");
}

```

範例 20-25：在處理兩個請求後，離開迴圈並關閉伺服器

在真實世界中的網頁伺服器當然不會只處理兩個請求就關機。此程式碼只是用來說明正常關機與清理的運作流程。

`take` 方法是由 `Iterator` 特徵所定義且我們限制該疊代最多只會取得前兩項。`ThreadPool` 會在 `main` 結束時離開作用域，然後 `drop` 的實作就會執行。

使用 `cargo run` 開啟伺服器，並下達三個請求。第三個請求應該會出現錯誤，而在你的終端機中你應該會看到類似以下的輸出：

```
$ cargo run
  Compiling hello v0.1.0 (file:///projects/hello)
    Finished dev [unoptimized + debuginfo] target(s) in 1.0s
    Running `target/debug/hello`
Worker 0 got a job; executing.
Shutting down.
Shutting down worker 0
Worker 3 got a job; executing.
Worker 1 disconnected; shutting down.
Worker 2 disconnected; shutting down.
Worker 3 disconnected; shutting down.
Worker 0 disconnected; shutting down.
Shutting down worker 1
Shutting down worker 2
Shutting down worker 3
```

你可能會看到不同順序的工作者與訊息輸出。我們可以從訊息中看到此程式碼如何執行的，工作者 0 與 3 獲得前兩個請求。在第二個請求之後，伺服器會停止接受連線。然後在工作者 3 開始工作之前，`ThreadPool` 的 `Drop` 實作就會執行。釋放 `sender` 會將所有工作者斷線並告訴它們關閉。每個工作者在斷線時都印出訊息，然後執行緒池會呼叫 `join` 來等待每個工作者的執行緒完成。

此特定執行方式中有個有趣的地方值得注意：在 `ThreadPool` 釋放 `sender` 然後任何工作者收到錯誤之前，我們嘗試將工作者 0 加入回來。工作者 0 尚未從 `recv` 收到錯誤，所以主執行緒會被擋住並等待工作者 0 完成。同一時間，工作者 3 收到一份工作但所有執行緒都收到錯誤。當工作者 0 完成時，主執行緒會等待剩下的工作者完成任務。屆時，它們都會退出它們的迴圈並能夠關閉。

恭喜！我們的專案完成了，我們有個基礎的網頁瀏覽器，其使用執行緒池來做非同步回應。我們能夠對伺服器正常關機，並清理池中所有的執行緒。

以下是完整的程式碼參考：

檔案名稱：`src/main.rs`

```

use hello::ThreadPool;
use std::fs;
use std::io::prelude::*;
use std::net::TcpListener;
use std::net::TcpStream;
use std::thread;
use std::time::Duration;

fn main() {
    let listener = TcpListener::bind("127.0.0.1:7878").unwrap();
    let pool = ThreadPool::new(4);

    for stream in listener.incoming().take(2) {
        let stream = stream.unwrap();

        pool.execute(|| {
            handle_connection(stream);
        });
    }

    println!("Shutting down.");
}

fn handle_connection(mut stream: TcpStream) {
    let mut buffer = [0; 1024];
    stream.read(&mut buffer).unwrap();

    let get = b"GET / HTTP/1.1\r\n";
    let sleep = b"GET /sleep HTTP/1.1\r\n";

    let (status_line, filename) = if buffer.starts_with(get) {
        ("HTTP/1.1 200 OK", "hello.html")
    } else if buffer.starts_with(sleep) {
        thread::sleep(Duration::from_secs(5));
        ("HTTP/1.1 200 OK", "hello.html")
    } else {
        ("HTTP/1.1 404 NOT FOUND", "404.html")
    };

    let contents = fs::read_to_string(filename).unwrap();

    let response = format!(
        "{}\r\nContent-Length: {}\r\n\r\n{}",
        status_line,
        contents.len(),
        contents
    );

    stream.write_all(response.as_bytes()).unwrap();
}

```

```
stream.flush().unwrap();  
}
```

檔案名稱：src/lib.rs


```

use std::{
    sync::{mpsc, Arc, Mutex},
    thread,
};

pub struct ThreadPool {
    workers: Vec<Worker>,
    sender: Option<mpsc::Sender<Job>>,
}

type Job = Box<dyn FnOnce() + Send + 'static>;

impl ThreadPool {
    /// Create a new ThreadPool.
    ///
    /// The size is the number of threads in the pool.
    ///
    /// # Panics
    ///
    /// The `new` function will panic if the size is zero.
    pub fn new(size: usize) -> ThreadPool {
        assert!(size > 0);

        let (sender, receiver) = mpsc::channel();

        let receiver = Arc::new(Mutex::new(receiver));

        let mut workers = Vec::with_capacity(size);

        for id in 0..size {
            workers.push(Worker::new(id, Arc::clone(&receiver)));
        }

        ThreadPool {
            workers,
            sender: Some(sender),
        }
    }

    pub fn execute<F>(&self, f: F)
    where
        F: FnOnce() + Send + 'static,
    {
        let job = Box::new(f);

        self.sender.as_ref().unwrap().send(job).unwrap();
    }
}

```

```

impl Drop for ThreadPool {
    fn drop(&mut self) {
        drop(self.sender.take());

        for worker in &mut self.workers {
            println!("Shutting down worker {}", worker.id);

            if let Some(thread) = worker.thread.take() {
                thread.join().unwrap();
            }
        }
    }
}

struct Worker {
    id: usize,
    thread: Option<thread::JoinHandle<()>>,
}

impl Worker {
    fn new(id: usize, receiver: Arc<Mutex<mpsc::Receiver<Job>>>) -> Worker {
        let thread = thread::spawn(move || loop {
            let message = receiver.lock().unwrap().recv();

            match message {
                Ok(job) => {
                    println!("Worker {} got a job; executing.", id);

                    job();
                }
                Err(_) => {
                    println!("Worker {} disconnected; shutting down.", id);
                    break;
                }
            }
        });

        Worker {
            id,
            thread: Some(thread),
        }
    }
}

```

我們還可以做更多事！如果你想繼續改善此專案的話，以下是些不錯的點子：

- 對 `ThreadPool` 與其公開方法加上技術文件。
- 對函式庫功能加上測試。
- 將 `unwrap` 的呼叫改成更完善的錯誤處理。

- 使用 `ThreadPool` 來處理其他種類的任務，而不只是網頁請求。
- 在 crates.io 找到一個執行緒池 crate，並使用該 crate 實作類似的網頁伺服器。然後比較該 crate 與我們實作的執行緒池之間的 API 與穩固程度。

總結

做得好！你已經讀完整本書了！我們由衷感謝你一同加入 Rust 的旅途。現在你已經準備好實作你自己的 Rust 專案並協助其他人的專案。別忘了我們有個友善的社群，其他 Rustaceans 會很樂意幫助你一同面對 Rust 旅途中的任何挑戰。

附錄

以下段落包含你可能會在你的 Rust 旅途中覺得有用的參考資源。

附錄 A：關鍵字

以下列表包含 Rust 目前或未來會使用到而保留起來的關鍵字。這意味著它們不能作為標識符使用（不過等等會提到的「[原始標識符](#)」除外），標識符包含函式、變數、參數、結構體欄位、模組、crates、常數、巨集、靜態數值、屬性、型別、特徵與生命週期的名稱。

目前有在使用的關鍵字

以下為目前的關鍵字列表與其對應的功能描述。

- `as` - 進行原始型別轉換、消除包含項目的特定特徵之歧異，或重新命名 `use` 陳述式內的項目
- `async` - 回傳 `Future` 而非阻擋目前執行緒
- `await` - 暫停執行直到 `Future` 的結果已經準備好
- `break` - 立即離開迴圈
- `const` - 定義常數項目或常數裸指標
- `continue` - 繼續進入下一次迴圈疊代
- `crate` - 在模組路徑中，指的是 `crate` 的源頭
- `dyn` - 對特徵物件的動態配置
- `else` - `if` 與 `if let` 控制流結構的例外選項
- `enum` - 定義列舉
- `extern` - 連結外部函式或變數
- `false` - 布林字面值 `false`
- `fn` - 定義函式或函式指標型別
- `for` - 從疊代器遍歷項目、實作特徵，或指定高階生命週期（higher-ranked lifetime）
- `if` - 依據條件表達式的分支
- `impl` - 實作本身或特徵的功能
- `in` - `for` 迴圈語法的其中一部分
- `let` - 綁定變數
- `loop` - 無條件的迴圈
- `match` - 將數值配對到模式
- `mod` - 定義模組
- `move` - 讓閉包取得其所有捕獲的所有權
- `mut` - 表示參考、裸指標或模式綁定具有可變性
- `pub` - 表示結構欄位、`impl` 區塊或模組對外公開
- `ref` - 綁定參考
- `return` - 函式的回傳
- `Self` - 我們正在定義或實作型別的类型別名
- `self` - 方法本體或當前模組
- `static` - 全域變數或存在於整個程式執行期間的生命週期

- `struct` - 定義結構體
- `super` - 當前模組的上層模組
- `trait` - 定義特徵
- `true` - 布林字面值 `true`
- `type` - 定義型別別名或關聯型別
- `union` - 定義聯集 `union`; 只作為宣告聯集時的關鍵字
- `unsafe` - 表示不安全的程式碼、函式、特徵或實作
- `use` - 將符號引入作用域
- `where` - 表示約束該型別用的子句
- `while` - 依據表達式結果的條件迴圈

未來可能會使用而保留的關鍵字

以下關鍵字沒有任何功能但是 Rust 可能會在未來使用到所以作為保留：

- `abstract`
- `become`
- `box`
- `do`
- `final`
- `macro`
- `override`
- `priv`
- `try`
- `typeof`
- `unsized`
- `virtual`
- `yield`

原始標識符

*原始標識 (Raw identifiers) *是個能讓你使用正常情況下不允許使用的關鍵字的語法。你能夠過加上 `r#` 關鍵字前綴來使用原始標識符。

舉例來說，`match` 是個關鍵字。如果你嘗試編譯以下使用 `match` 作為名稱的函式的話：

檔案名稱：src/main.rs

```
fn match(needle: &str, haystack: &str) -> bool {  
    haystack.contains(needle)  
}
```

你會獲得此錯誤：

```
error: expected identifier, found keyword `match`
--> src/main.rs:4:4
  |
4 | fn match(needle: &str, haystack: &str) -> bool {
  |      ^^^^^ expected identifier, found keyword
```

錯誤表示你不能使用關鍵字 `match` 作為函式標識符。要使用 `match` 作為函式名稱的話，你可以使用原始標識符語法，如以下所示：

檔案名稱：src/main.rs

```
fn r#match(needle: &str, haystack: &str) -> bool {
    haystack.contains(needle)
}

fn main() {
    assert!(r#match("foo", "foobar"));
}
```

此程式碼就能夠編譯並沒有任何錯誤。注意到 `r#` 前綴會用於函式定義的名稱以及在 `main` 呼叫該函式的地方。

原始標識符能讓你使用任何字作為標識符，就算該字剛好是保留的關鍵字。這給了我們更多選擇標識符名稱的自由，以及讓我們與以這些單詞不是關鍵詞的語言編寫的程式進行整合。除此之外，原始標識符讓你可以使用與你 crate 的 Rust 版號（edition）不相同的函式庫。舉例來說 `try` 在 2015 版號還不是關鍵字，但到 2018 版號才加入。如果你依賴一個使用 2015 版號的函式庫，且其中有個 `try` 函式，你就需要使用原始標識符語法。在此例中就是在你 2018 版號的程式碼用 `r#try` 來呼叫該函式。請查閱[附錄 E](#)以瞭解關於版號的更多資訊。

附錄 B：運算子與符號

此附錄包含 Rust 語法的詞彙表，包含運算子以及其他符號，這些符號會單獨出現或出現在路徑、泛型、特徵界限、巨集、屬性、註解、元組與大括號中。

運算子

表 B-1 包含 Rust 中的運算子、運算子如何出現的範例、簡單解釋以及該運算子是否能超載 (overloadable)。如果一個運算子可以超載，用來超載該運算子對應的特徵會列出來。

表 B-1：運算子

運算子	範例	解釋	能否超載？
!	ident!(...), ident!{...}, ident![...]	巨集表達式	
!	!expr	位元運算 (Bitwise) 或邏輯運算補數 (logical complement)	Not
!=	expr != expr	不相等比較	PartialEq
%	expr % expr	算數餘數	Rem
%=	var %= expr	算數餘數並賦值	RemAssign
&	&expr, &mut expr	借用	
&	&type, &mut type, &'a type, &'a mut type	借用指標型別	
&	expr & expr	位元運算 AND	BitAnd
&=	var &= expr	位元運算 AND 並賦值	BitAndAssign
&&	expr && expr	邏輯運算 AND	
*	expr * expr	算數乘法	Mul
*=	var *= expr	算數乘法並賦值	MulAssign
*	*expr	解參考	Deref
*	*const type, *mut type	裸指標	
+	trait + trait, 'a + trait	複合型別約束	

運算子	範例	解釋	能否超載？
+	<code>expr + expr</code>	算數加法	Add
+=	<code>var += expr</code>	算數加法並賦值	AddAssign
,	<code>expr, expr</code>	引數與元素分隔符	
-	<code>- expr</code>	算數負數	Neg
-	<code>expr - expr</code>	算數減法	Sub
--	<code>var -= expr</code>	算數減法並賦值	SubAssign
->	<code>fn(...) -> type,</code> <code> ... -> type</code>	函式與閉包回傳型別	
.	<code>expr.ident</code>	成員存取	
..	<code>..., expr..., ..expr,</code> <code>expr..expr</code>	右排除範圍	PartialOrd
..=	<code>..=expr,</code> <code>expr..=expr</code>	右包含範圍	PartialOrd
..	<code>..expr</code>	結構體更新語法	
..	<code>variant(x, ..),</code> <code>struct_type { x, ..</code> <code>}</code>	「與剩餘部分」模式綁定	
...	<code>expr...expr</code>	(已棄用，請改用 <code>..=</code>) 模式：包含範圍模式	
/	<code>expr / expr</code>	算數除法	Div
/=	<code>var /= expr</code>	算數除法並賦值	DivAssign
:	<code>pat: type, ident:</code> <code>type</code>	約束	
:	<code>ident: expr</code>	結構體欄位初始化	
:	<code>'a: loop {...}</code>	迴圈標籤	
;	<code>expr;</code>	陳述式與項目結束符	
;	<code>[...; len]</code>	固定大小陣列語法的其中一部分	
<<	<code>expr << expr</code>	左移	Shl
<<=	<code>var <<= expr</code>	左移並賦值	ShlAssign
<	<code>expr < expr</code>	小於比較	PartialOrd
<=	<code>expr <= expr</code>	小於等於比較	PartialOrd
=	<code>var = expr, ident =</code> <code>type</code>	賦值 / 等值	

運算子	範例	解釋	能否超載？
<code>==</code>	<code>expr == expr</code>	等於比較	<code>PartialEq</code>
<code>=></code>	<code>pat => expr</code>	配對分支語法的其中一部分	
<code>></code>	<code>expr > expr</code>	大於比較	<code>PartialOrd</code>
<code>>=</code>	<code>expr >= expr</code>	大於等於比較	<code>PartialOrd</code>
<code>>></code>	<code>expr >> expr</code>	右移	<code>Shr</code>
<code>>>=</code>	<code>var >>= expr</code>	右移並賦值	<code>ShrAssign</code>
<code>@</code>	<code>ident @ pat</code>	模式綁定	
<code>^</code>	<code>expr ^ expr</code>	位元運算互斥（exclusive）OR	<code>BitXor</code>
<code>^=</code>	<code>var ^= expr</code>	位元運算互斥 OR 並賦值	<code>BitXorAssign</code>
<code> </code>	<code>pat pat</code>	模式 OR	
<code> </code>	<code>expr expr</code>	位元運算 OR	<code>BitOr</code>
<code> =</code>	<code>var = expr</code>	位元運算 OR 並賦值	<code>BitOrAssign</code>
<code> </code>	<code>expr expr</code>	邏輯運算 OR	
<code>?</code>	<code>expr?</code>	錯誤傳遞	

非運算子符號

以下列表包含所有不作為運算子的符號；也就是說，它們的行為並不像是在呼叫函式或方法。

表 B-2 顯示了出現在各處單獨出現且有效的符號。

表 B-2：獨立語法

符號	解釋
<code>'ident</code>	有名稱的生命週期或迴圈標籤
<code>...u8, ...i32, ...f64, ...usize, etc.</code>	指定型別的數值字面值
<code>"..."</code>	字串字面值
<code>r"...", r#"..."#, r##"..."##, etc.</code>	原始字串字面值，不會處理跳脫字元
<code>b"..."</code>	位元組字串字面值，其會組織一個位元組陣列（ <code>[u8]</code> ）而非字串
<code>br"...", br#"..."#, br##"..."##, etc.</code>	原始位元組字串字面值，結合原始與位元組字串的字面值

符號	解釋
<code>'...'</code>	字元字面值
<code>b'...'</code>	ASCII 位元組字面值
<code> ... expr</code>	閉包
<code>!</code>	發散函式（diverging functions）的永遠為空的型別
<code>_</code>	「忽略」模式綁定，也用於整數字面值的可讀性

表 B-3 顯示了出現在模組架構中到一個項目的路徑的符號。

表 B-3：路徑相關語法

符號	解釋
<code>ident::<code>ident</code></code>	命名空間路徑
<code>::<code>path</code></code>	與 <code>crate</code> 源頭相對應的路徑（如顯式絕對路徑）
<code>self::<code>path</code></code>	與目前模組相對應的路徑（如顯式相對路徑）
<code>super::<code>path</code></code>	與上層模組相對應的路徑
<code>type::<code>ident</code>, <code><type as trait>::<code>ident</code></code></code>	關聯常數、函式與型別
<code><type>::...</code>	無法直接命名的型別的關聯項目（如 <code><&T>::...</code> 、 <code><[T]>::...</code> 等等）
<code>trait::<code>method</code>(...)</code>	透過命名其定義的特徵來消除方法呼叫的歧義
<code>type::<code>method</code>(...)</code>	透過命名其定義的型別來消除方法呼叫的歧義
<code><type as trait>::<code>method</code>(...)</code>	透過命名特徵與型別來消除方法呼叫的歧義

Table B-4 顯示出現在泛型型別參數的符號。

表 B-4：泛型

型別	解釋
<code>path<...></code>	指定參數給型別中的泛型型別（如 <code>Vec<u8></code> ）
<code>path::<code><...></code>, <code>method::<code><...></code></code></code>	指定參數給表達式中的泛型型別、函式或方法，通常被稱之為 <code>turbofish</code> （如 <code>"42".parse::<i>i32</i>>()</code> ）
<code>fn ident<...> ...</code>	定義泛型函式
<code>struct ident<...> ...</code>	定義泛型結構體
<code>enum ident<...> ...</code>	定義列舉結構體
<code>impl<...> ...</code>	定義泛型實作

型別	解釋
<code>for<...> type</code>	高階生命週期界限
<code>type<ident=type></code>	其一或數個關聯型別有特定賦值的泛型型別（如 <code>Iterator<Item=T></code> ）

表 B-5 顯式出現在透過特徵界限約束泛型型別參數的符號。

表 B-5：特徵界限約束

符號	解釋
<code>T: U</code>	泛型參數 <code>T</code> 約束於實作 <code>U</code> 的型別
<code>T: 'a</code>	泛型參數 <code>T</code> 的生命週期必須比 <code>'a</code> 還長（代表該型別無法傳遞包含任何聲明週期短於 <code>'a</code> 的參考）
<code>T: 'static</code>	泛型型別 <code>T</code> 不包含 <code>'static</code> 以外的借用參考
<code>'b: 'a</code>	泛型生命週期 <code>'b</code> 必須長於 <code>'a</code>
<code>T: ?Sized</code>	允許泛型型別參數為動態大小型別
<code>'a + trait, trait + trait</code>	複合型別約束

表 B-6 顯示出現在呼叫或定義巨集與指定項目屬性的符號。

表 B-6：巨集與屬性

符號	解釋
<code>#[meta]</code>	外部屬性
<code>#![meta]</code>	內部屬性
<code>\$ident</code>	巨集替代
<code>\$ident:kind</code>	巨集捕獲
<code>\$(...)...</code>	巨集重複
<code>ident!(...), ident!{...}, ident![...]</code>	巨集調用

表 B-7 顯示建立註解的符號。

表 B-7：註解

符號	解釋
<code>//</code>	行註解
<code>///!</code>	內部行技術文件註解
<code>///</code>	外部行技術文件註解
<code>/*...*/</code>	區塊註解

符號	解釋
<code>/*!...*/</code>	內部區塊技術文件註解
<code>/**...*/</code>	外部區塊技術文件註解

表 B-8 顯示出現在元組中的符號。

表 B-8：元組

符號	解釋
<code>()</code>	空元組（也稱為單元），同時是字面值與型別
<code>(expr)</code>	括號表達式
<code>(expr,)</code>	單一元素元組表達式
<code>(type,)</code>	單一元素元組型別
<code>(expr, ...)</code>	元組表達式
<code>(type, ...)</code>	元組型別
<code>expr(expr, ...)</code>	函式呼叫表達式，也用來初始化元組 <code>struct</code> 與元組 <code>enum</code> 變體
<code>expr.0</code> , <code>expr.1</code> , etc.	元組索引

表 B-9 顯示大括號使用到的地方。

表 B-9：大括號

符號	解釋
<code>{...}</code>	區塊表達式
<code>Type {...}</code>	<code>struct</code> 字面值

表 B-10 顯示中括號使用到的地方。

表 B-10：中括號

符號	解釋
<code>[...]</code>	陣列字面值
<code>[expr; len]</code>	包含 <code>len</code> 個 <code>expr</code> 的陣列字面值
<code>[type; len]</code>	包含 <code>len</code> 個 <code>type</code> 的陣列字面值
<code>expr[expr]</code>	集合索引，可超載（ <code>Index</code> 、 <code>IndexMut</code> ）
<code>expr[..]</code> , <code>expr[a..]</code> , <code>expr[..b]</code> , <code>expr[a..b]</code>	使用 <code>Range</code> 、 <code>RangeFrom</code> 、 <code>RangeTo</code> 或 <code>RangeFull</code> 作為「索引」來替代集合 <code>slice</code> 的集合索引

附錄 C：可推導的特徵

在本書中的許多地方，我們都有遇到 `derive` 屬性，這能用在結構體或列舉定義中。`derive` 屬性會透過 `derive` 語法來對被標記的型別產生對應的預設特徵實作。

在此附錄中，我們提供了所有在標準函式庫中你可以透過 `derive` 來使用的特徵。每個段落會包含：

- 該特徵會推導出哪些運算子與方法
- `derive` 提供的特徵實作會做什麼事情
- 該特徵實作對型別有何影響
- 你能夠或不能夠實作特徵的條件
- 需要特徵來做運算的範例

如果你想要不同於 `derive` 屬性提供的行為，請查閱每個特徵的[標準函式庫技術文件](#)來瞭解如何手動實作它們。

以下列出的是唯一能在標準函式庫中使用 `derive` 來對你的型別實作的特徵。其他在標準函式庫的特徵不太常有理想的預設行為，所以會由你來決定最合理的方式來實作它們。

其中一個無法推導的特徵範例就是 `Display`，這用來顯示格式化資訊給終端使用者。這應該永遠由你來決定顯示型別給終端使用者的最佳方式。型別的哪些部分應該給使用者看到？哪些部分他們會覺得是有關聯的？什麼樣的資料格式對他們最相關？Rust 編譯器並不具這樣的眼光能判斷，所以它無法為你提供適合的預設行為。

此附錄提供的可推導的特徵列表並不是就是所有能用的特徵：函式庫也可以為他們自己的特徵實作 `derive`，所以你可以使用 `derive` 的特徵是沒有極限的。實作 `derive` 會需要用到過程式巨集 (procedural macro)，這在第十九章的「[巨集](#)」段落有提到。

用於開發時輸出的 `Debug`

`Debug` 特徵用於啟用除錯格式資訊的格式化字串，讓你可以在 `{}` 佔位符加上 `:?` 來顯示。

`Debug` 特徵讓你可以印出型別實例的除錯資訊，好讓你以及其他程式設計師在使用你的型別時，能在程式執行的特定時間點觀察實例。

舉例來說，要使用 `assert_eq!` 巨集的話就必須要有 `Debug` 特徵。如果相等判定失敗的話，此巨集會印出作為引數的實例數值，讓程式設計師可以看到為何兩個實例不相等。

用於比較相等的 `PartialEq` 與 `Eq`

`PartialEq` 特徵讓你可以比較型別實例來檢查是否相等，並可因此使用 `==` 與 `!=` 運算子。

推導 `PartialEq` 會實作 `eq` 方法。當推導結構體的 `PartialEq` 時，兩個實例之間必須所有欄位都相等才算相等，所以要是有任何欄位不相等實例就不算相等。而在列舉推導時，每個變體會與自己相等，且與其他變體不相等。

舉例來說，使用 `assert_eq!` 巨集的話就必須要有 `PartialEq` 特徵，因為這要用來比較兩個型別實例是否相等。

`Eq` 特徵沒有任何方法，它用來表示指定型別的每個數值都與自己本身相等。`Eq` 只能用於也有實作 `PartialEq` 的型別，然而並非所有實作 `PartialEq` 的型別都能實作 `Eq`。其中一個例子就是浮點數型別，浮點數的實作就指明兩個非數（not-a-number, `NaN`）實例數值彼此並不相等。

而 `Eq` 會用到的地方則有像是 `HashMap<K, V>` 中的鍵，這樣 `HashMap<K, V>` 才能知道兩個鍵之間是否相等。

用於比較順序的 `PartialOrd` 與 `Ord`

`PartialOrd` 特徵讓你比較型別實例排序的順序。有實作 `PartialOrd` 的型別能夠使用 `<`、`>`、`<=` 與 `>=` 運算子。你只能在有實作 `PartialEq` 的型別實作 `PartialOrd` 特徵。

推導 `PartialOrd` 會實作 `partial_cmp` 方法，這會回傳一個 `Option<Ordering>`，當給予的數值無法產生任何順序的話就會是 `None`。其中一個儘管該型別大多數數值都能比較時，但仍有機會無法產生順序的範例就是非數（not-a-number, `NaN`）浮點數數值。對任意浮點數與 `NaN` 浮點數數值呼叫 `partial_cmp` 的話就會回傳 `None`。

當在結構體推導時，`PartialOrd` `compare` 會比較兩個實例，並從結構體定義的欄位順序來依序比較每個欄位的數值。而在列舉推導時，在列舉定義中較早宣告的列舉變體會比之後的變體還小。

舉例來說，`rand` crate 中的 `gen_range` 方法就必須要有 `PartialOrd` 特徵，該方法會在指定的範圍內產生隨機數值。

`Ord` 特徵能让你知道指定型別的任意兩個數值存在著順序。`Ord` 特徵實作了 `cmp` 方法，這會回傳 `Ordering` 而不只是 `Option<Ordering>`，因為其永遠會有有效的順序。你只能在有實作 `PartialOrd` 與 `Eq`（而 `Eq` 需要 `PartialEq`）的特徵實作 `Ord` 特徵。在結構體與列舉推導時，`cmp` 的行為會與 `PartialOrd` 推導的 `partial_cmp` 行為一致。

其中一個需要 `Ord` 的範例就是當 `BTreeSet<T>` 要儲存數值的時候，這是一個依據數值排序順序儲存資料的資料結構。

用於複製數值的 `Clone` 與 `Copy`

`Clone` 特徵讓你能顯式建立一個數值的深拷貝，而且在複製的過程中可能會包含執行其他程式碼並拷貝堆積的資料。你可以在第四章的「[變數與資料互動的方式：克隆（Clone）](#)」段落瞭解更多關於 `Clone` 的資訊。

推導 `Clone` 會實作 `clone` 方法，這在整個型別實作時，會呼叫每個型別部分的 `clone`。這意味著該型別的所有欄位或數值都必須有實作 `Clone` 才能推導 `Clone`。

會需要用到 `Clone` 的其中一個例子是對 `slice` 呼叫 `to_vec` 方法。`slice` 不擁有其所包含的型別實例，但是 `to_vec` 回傳的向量會需要擁有其實例，所以 `to_vec` 會對每個項目呼叫 `clone`。因此，儲存在 `slice` 內的型別必須實作 `Clone`。

`Copy` 特徵讓你能只拷貝堆疊上的資料來複製數值，而且不需要額外的程式碼。你可以在第四章的「[只在堆疊上的資料：拷貝 \(Copy\)](#)」段落瞭解更多關於 `Copy` 的資訊。

`Copy` 特徵沒有定義任何方法，以避免開發者超載這些方法並違反不執行任何程式碼的假設。這樣所有的程式設計師才都能預定拷貝數值是很迅速的。

你可以對任何內部所有部分有實作 `Copy` 的型別推導 `Copy`。實作 `Copy` 的型別必須也實作 `Clone`，因為其所做的事會與 `Copy` 一樣。

`Copy` 通常不是必要的，實作 `Copy` 的型別會能進行優化，代表你不需要呼叫 `clone` 並讓程式碼更簡潔。

所有 `Copy` 能辦到的事你也能用 `Clone` 來達成，但是程式碼會變得比較慢或是需要使用 `clone`。

用於映射數值至固定大小數值的 `Hash`

`Hash` 特徵讓你能取得任意大小的型別實例，並使用在雜湊函式映射 (`map`) 到固定大小的數值實例。推導 `Hash` 會實作 `hash` 方法。推導出的 `hash` 方法實作會組合該型別每個部分的 `hash` 呼叫結果，這代表所有的欄位或數值也必須實作 `Hash` 才能推導 `Hash`。

會需要 `Hash` 的其中一個範例是在 `HashMap<K, V>` 儲存鍵，這樣才能有效率地儲存資料。

用於預設數值的 `Default`

`Default` 特徵讓你能建立一個型別的預設數值。推導 `Default` 會實作 `default` 函式。推導出的 `default` 函式實作會呼叫該型別每個部分的 `default` 函式，這代表該型別的所有欄位或數值都得實作 `Default` 才能推導 `Default`。

`Default::default` 函式常用於結合結構體更新語法，如果我們在第五章的「[使用結構體更新語法從其他結構體建立實例](#)」段落所提及的。你可以自訂結構體中的一些欄位，然後使用 `..Default::default()` 將剩餘的欄位設為預設數值。

舉例來說，當你在 `Option<T>` 實例中使用 `unwrap_or_default` 方法就會需要 `Default` 特徵。如果 `Option<T>` 為 `None`，`unwrap_or_default` 方法就會回傳儲存在 `Option<T>` 的 `T` 的 `Default::default` 結果。

附錄 D - 實用開發工具

在此附錄中，我們會討論些 Rust 專案提供的實用開發工具。我們會介紹自動格式化工具、修正警告最快速的方式、linter 以及 IDE 的整合工具。

透過 `rustfmt` 自動格式化

`rustfmt` 工具會依據社群程式碼風格來重新格式化你的程式碼。許多協作專案都會使用 `rustfmt` 來避免 Rust 風格的歧義，每個人都能用此工具格式化他們的程式碼。

欲安裝 `rustfmt`，請輸入以下命令：

```
$ rustup component add rustfmt
```

此命令會給你 `rustfmt` 與 `cargo-fmt`，就像 Rust 會提供你 `rustc` 與 `cargo` 一樣。要格式化任何 Cargo 專案的話，請輸入：

```
$ cargo fmt
```

執行此命令會重新格式化目前 crate 中所有的 Rust 程式碼。不過這只會變更程式碼風格，並不會影響程式碼語義。想瞭解更多 `rustfmt` 的資訊，歡迎查閱[它的技術文件](#)。

透過 `rustfix` 修正你的程式碼

`rustfix` 工具包含在 Rust 的安裝中，它可以自動修復編譯器警告，這可能是你想要糾正問題的明確方法。你以前可能看過編譯器警告。舉例來說，請參考以下程式碼：

檔案名稱：src/main.rs

```
fn do_something() {}

fn main() {
    for i in 0..100 {
        do_something();
    }
}
```

我們在此呼叫 `do_something` 函式 100 次，但是我們在 `for` 迴圈中完全沒用到變數 `i`。Rust 會警告我們：

```
$ cargo build
   Compiling myprogram v0.1.0 (file:///projects/myprogram)
warning: unused variable: `i`
--> src/main.rs:4:9
  |
4 |     for i in 0..100 {
  |           ^ help: consider using `_i` instead
  |
= note: #[warn(unused_variables)] on by default

Finished dev [unoptimized + debuginfo] target(s) in 0.50s
```

警告訊息建議我們改使用 `_i` 來作為名稱，底線指的是我們認定此變數不會被使用。我們可以透過執行 `cargo fix` 來使用 `rustfix` 工具以自動採用這些建議：

```
$ cargo fix
   Checking myprogram v0.1.0 (file:///projects/myprogram)
   Fixing src/main.rs (1 fix)
Finished dev [unoptimized + debuginfo] target(s) in 0.59s
```

當我們再次檢查 `src/main.rs`，我們會看到 `cargo fix` 已經將程式碼修正了：

檔案名稱：`src/main.rs`

```
fn do_something() {}

fn main() {
    for _i in 0..100 {
        do_something();
    }
}
```

`for` 迴圈變數現在改名為 `_i`，而警告也不再出現了。

你也可以使用 `cargo fix` 命令來在不同的 Rust 版號之間做轉換程式碼。版號會在附錄 E 做介紹。

透過 Clippy 運用更多功能

Clippy 工具是一系列的 lint 集合，用來分析程式碼以獲取常見錯誤並改善你的 Rust 程式碼。

要安裝 Clippy 的話，輸入以下命令：

```
$ rustup component add clippy
```

要在任何 Cargo 專案執行 Clippy，輸入以下命令：

```
$ cargo clippy
```

舉例來說，假設你在寫程式時使用到如 pi 這種數學常數的近似值，如以下所示：

檔案名稱：src/main.rs

```
fn main() {  
    let x = 3.1415;  
    let r = 8.0;  
    println!("the area of the circle is {}", x * r * r);  
}
```

對此專案執行 `cargo clippy` 會顯示以下錯誤：

```
error: approximate value of `f{32, 64}::consts::PI` found  
--> src/main.rs:2:13  
|  
2 |     let x = 3.1415;  
|               ^^^^^^  
|  
= note: `[deny(clippy::approx_constant)]` on by default  
= help: consider using the constant directly  
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#approx_constant
```

此錯誤告訴你 Rust 已經有個更精準的 `PI` 常數定義，如果改使用此定義的話，你的程式會更準確。你可以將你的程式碼改使用 `PI` 常數。以下程式碼就不會透過 Clippy 獲得任何錯誤或警告：

檔案名稱：src/main.rs

```
fn main() {  
    let x = std::f64::consts::PI;  
    let r = 8.0;  
    println!("the area of the circle is {}", x * r * r);  
}
```

關於更多 Clippy 的資訊，請查閱[它的技術文件](#)。

使用 `rust-analyzer` 整合 IDE

為了協助 IDE 的整合，Rust 社群推薦使用 `rust-analyzer`。此工具會與 [Language Server Protocol](#) 溝通，來提供許多與編譯器相關的協助，這是 IDE 與程式語言彼此溝通的協定規格。`rust-analyzer` 可用於各種不同的客戶端，像是 [Visual Studio Code](#) 的 [Rust analyzer 外掛](#)。

前往 `rust-analyzer` 專案的[首頁](#)可以了解更多安裝方法，讓你所使用的 IDE 也能獲得 Language Server 的支援。這樣你的 IDE 就能獲得許多功能像是：自動補全、跳至定義與顯示錯誤等等。

附錄 E - 版號

在第一章中，你會看到 `cargo new` 會在 *Cargo.toml* 檔案中加上一些關於版號 (edition) 的詮釋資料。此附錄會講解其意義！

Rust 語言與編譯器有一個為其六週的發佈循環，這意味著使用者可能定期獲得一些新功能。其他程式設計語言可能會發佈較大的更新但就會比較不頻繁。Rust 傾向於較頻繁地發佈小更新。過一段時間後，這些所有變更會漸漸累積起來。不過隨著一次次的發佈，回過頭來看可能會發覺「哇！Rust 1.10 與 Rust 1.31 之間的變化真大！」

所以每個兩到三年，Rust 團隊會產生新的 Rust **版號 (edition)**。每個版本會整合已推出的功能成一個附有完整技術文件更新與工具的套件。然後新的版號就會包含在每六週循環過程的發佈之中。

版號對不同客群提供不同功能：

- 對於活躍的 Rust 使用者來說，新的版號將累積的變更整合成容易理解的單一套件。
- 對非使用者來說，新的版號意味著有一些新的重大進展，讓 Rust 可能值得再看一次。
- 對於開發 Rust 的人來說，新的版號提供整個專案的一個集結點。

在本書撰寫時，Rust 已經有三個版號：Rust 2015、Rust 2018 與 Rust 2021。本書使用的是 Rust 2021 版號的慣用寫法。

Cargo.toml 中的 `edition` 指的是編譯器該對你的程式碼使用何種版號。如果沒有指定的話，Rust 會以向下相容作為考量而是使用 `2015`。

每個專案都能選擇一種版號而不只是使用預設的 2015 版號。版號會包含無法相容的變更，像是包含新的關鍵字使得程式碼中的標識符衝突。然而，除非你親自改變版號，不然就算你更新 Rust 編譯器的版本，你的程式碼依然能夠編譯通過。

所有的 Rust 編譯器版本會支援在其編譯器發佈之前的任何版號，而且可以連結任何支援版號的 crate。版號變更只會影響編譯器初始解析程式碼的方式而已。因此，如果你使用 Rust 2015 但你其中一個依賴使用 Rust 2018 的話，你的專案仍然能編譯並使用該依賴函式庫。相對地，當你的專案使用 Rust 2018 而有依賴使用 Rust 2015 的話依然是如此。

這裡要澄清一點：大多數功能在所有版號中都能使用。開發者使用任何 Rust 版號都能繼續獲得新的穩定版本帶來的改善。然而有些情況下，主要是增加新的關鍵字的時候，有一些新功能可能就只有後期的版號才能使用。你想使用利用新功能的話，就得切換版號。

想瞭解更多資訊的話，請查閱 [Edition Guide](#)，這是本涵蓋所有版號之間不同的書籍，並會解釋如何使用 `cargo fix` 來自動升級你的程式碼至新的版號。

附錄 F：本書的翻譯本

對於英文以外的語言資源，大部分都還在翻譯中，請查看 [Translations label](#) 來幫助或讓我們知道有哪些新的翻譯！

- [Português \(BR\)](#)
- [Português \(PT\)](#)
- 简体中文
- 正體中文
- [Українська](#)
- [Español, alternate](#)
- Italiano
- Русский
- 한국어
- 日本語
- Français
- Polski
- Cebuano
- Tagalog
- Esperanto
- ελληνική
- Svenska
- Farsi
- Deutsch
- हिंदी
- ไทย
- Danske

附錄 G - Rust 的開發流程與「每夜版 Rust」

本附錄會介紹 Rust 是如何開發的，以及這對身為 Rust 開發者的你會有何影響。

無停滯穩定

身為一門語言，Rust 十分注重程式碼穩定性。我們希望 Rust 成為你在開發中的穩固基石，如果經常在更新的話，這樣的願望就很難達成了。同時，如果我們不能實驗新功能的話，直到它們發佈之前，我們可能就無法找出重大瑕疵，而且發佈後我們就很難再加以更改了。

我們對此問題的解決方案為「無停滯穩定 (stability without stagnation)」，而我們的指導原則為：你永遠不該害怕升級最新的 Rust 穩定版。每次都該是無痛升級，但同時也該提供新功能、修正錯誤並加快編譯時間。

嘟嘟火車出發！發佈通道與時刻表

Rust 開發團隊有個發佈時刻表 (**train schedule**)。而所有的開發工作都在 Rust repository 的 `master` 分支上。發佈採用軟體發佈火車模型 (software release train model)，這也被用於 Cisco IOS 與其他的軟體專案。Rust 有三個發佈通道 (**release channels**)：

- 每夜版 (Nightly)
- 測試版 (Beta)
- 穩定版 (Stable)

大多數的 Rust 開發者主要都使用穩定版的通道，但是想要實驗新功能的人可以嘗試使用每夜版或測試版。

以下是開發與發佈過程是如何進行的範例：讓我們假設 Rust 團隊正在準備發佈 Rust 1.5。這版本在 2015 年十二月就發佈，做此假設只是為我們提供較真實的版本數字。其中有一個新功能會加入 Rust 並在 `master` 分支新增一個 commit。每個晚上 Rust 都會產生新的每夜版版本，然後每到天亮就會發佈，而這些發佈均由發佈基礎設施自動產生。所以隨著時間過去，我們每晚都會有一個發佈像這樣進行：

```
nightly: * - - * - - *
```

接著每隔六週就是時候發佈新的發佈版本了！Rust repository 的 `beta` 分支會從每夜版使用的 `master` 分支產生。現在我們就有了兩種發佈：

```

nightly: * - - * - - *
          |
beta:    *

```

大多數的 Rust 使用者並不會直接使用測試版發佈，而會用他們的 CI 系統來檢查測試版來幫助 Rust 發現可能的迴歸錯誤 (regressions)。同時，每夜版仍然會每晚產生新的發佈：

```

nightly: * - - * - - * - - * - - *
          |
beta:    *

```

假設有迴歸錯誤被發現了，那這對我們來說就是個好消息。因為在錯誤潛入穩定版發佈之前，我們還有些時間能檢測測試版的發佈！修正會加到 `master`，所以每夜版就能被修正。然後該修正也會合併到 `beta`，所以新的測試版發佈就會跟著產生：

```

nightly: * - - * - - * - - * - - * - - *
          |
beta:    * - - - - - - - - *

```

再經過第一個測試版產生的六週後，就是時候發佈穩定版了！`stable` 分支會從 `beta` 分支中產生：

```

nightly: * - - * - - * - - * - - * - - * - * - *
          |
beta:    * - - - - - - - - *
          |
stable:  *

```

太好了！Rust 1.5 終於釋出了！不過我們不能忘記一件事，由於六週過去了，我們也必須為下一個 Rust 1.6 版本準備新的測試版。所以在 `stable` 從 `beta` 產生後，下個版本的 `beta` 分支會再次從 `nightly` 產生：

```

nightly: * - - * - - * - - * - - * - - * - * - *
          |                                     |
beta:    * - - - - - - - - *                   *
          |
stable:  *

```

這就叫「火車模型 (train model)」，因為每隔六週就會有個發佈「駛離車站」，但其仍需要穿梭過測試版通道，才能抵達穩定版發佈。

Rust 會像發條裝置一樣每隔六週定時發佈。如果你已經知道一個 Rust 發佈的日期，你就能知道下一個的發佈日期，也就是六週之後。這樣每隔六週發佈的時程表有個好處是下一輛火車很快也會接著抵達。如果某個特定版本遺漏某項功能的話，不用擔心，因為下一版很快就會來臨了！這能降低發佈截止日期前，不得不偷偷釋出尚未完善的功能的壓力。

幸虧有此流程，你永遠都可以看到 Rust 的下一個版本並驗證你是否能輕鬆升級。如果測試版不如你所預期，你可以回報給團隊並在穩定版發佈前修正完成！在測試版出現重大缺陷是很少見的，但 `rustc` 本身仍是個軟體，總避免不了些錯誤發生。

不穩定功能

此發佈模型還有一項重點，那就是不穩定 (unstable) 功能。Rust 使用一個叫做「功能標記 (feature flags)」的技術來決定一個發佈能啟用哪些功能。如果有個新功能正在積極開發中，它可以加進 `master` 因而出現在每夜版中，但是會有個**功能標記**。如果身為使用者的你想要嘗試看看仍在開發中的功能的話，你是可以使用的。但是你必須透過 Rust 每夜版並在你的程式碼指明對應的功能標記才行。

如果你使用的是測試版或穩定版 Rust，你就無法使用任何功能標記。這是讓我們在宣佈新功能已經永遠穩定前，能夠確實測試它們的關鍵。這滿足了想使用前沿技術的人，同時也確保維持在穩定版的人有穩固的基石，不會讓他們的程式碼被破壞。這就是所謂的無停滯穩定。

本書只涵蓋了穩定版的功能資訊，因為開發中的功能可能隨時會改變。當其納入穩定版時肯定會與此書撰寫的時候而有所不同。你可以在線上找到每夜版功能的技術文件。

Rustup 與 Rust 每夜版的職責

Rustup 能夠輕鬆切換不同的 Rust 發佈通道，在全域或是每個專案的範圍都行。而預設情況下，你會安裝穩定版 Rust。要安裝每夜版的話，請輸入以下命令：

```
$ rustup toolchain install nightly
```

你還可以看到你透過 `rustup` 安裝的所有**工具鏈 (toolchains)** (Rust 的發佈與相關元件)。以下是本書其中一位作者 Windows 電腦中的範例：

```
> rustup toolchain list
stable-x86_64-pc-windows-msvc (default)
beta-x86_64-pc-windows-msvc
nightly-x86_64-pc-windows-msvc
```

如你所見，穩定版工具鏈是預設選項。大多數 Rust 使用者在大部分時間都會使用穩定版。你可以平時在大部分時間使用穩定版，並在需要使用前沿技術功能的特定專案下使用每夜版。為此，你可以在該專案目錄下使用 `rustup override` 來設置 `rustup` 在該目錄下需要使用每夜版工具鏈：

```
$ cd ~/projects/needs-nightly
$ rustup override set nightly
```

現在你每次在 `~/projects/needs-nightly` 底下呼叫 `rustc` 或 `cargo` 的話，`rustup` 會確保你使用的是每夜版 Rust，而不是預設的穩定版 Rust。這在當你有一堆 Rust 專案時會非常好用！

RFC 流程與團隊

所以你該怎麼學習這些新功能呢？Rust 的開發模型遵循的是**請求意見稿（Request For Comments, RFC）**流程。如果你想要改善 Rust，你可以寫篇 RFC 提案。

任何人都可以寫篇 RFC 來改善 Rust，然後該提案會經由 Rust 團隊審核並討論，而團隊有許多子主題團隊所組成。在 [Rust 官網上](#) 有完整的團隊列表，包含每個專案領域的團隊，像是語言設計、編譯器實作、基礎設施、技術文件以及更多等等。相對應的團隊會閱讀提案並留言、寫些他們的想法，並在最後達成共識，決定要接受或拒絕該功能。

如果功能被接受了，Rust repository 便會開啟對應 issue，然後每個人就都能嘗試實作它。實作該功能的人很可能與當初提案的人不相同！當實作準備好後，它便會加入 `master` 分支並有個功能標記，如同我們在「[不穩定功能](#)」段落所提及的。

經過一段時間後，一旦使用每夜版發佈的 Rust 開發者嘗試過新功能後，團隊成員會討論此功能，其在每夜版運行的如何，並決定它是否該加到穩定版。如果決定進一步加入的話，功能標記就會被移除，然後該功能就會是穩定功能了！它就像搭乘火車班抵達最新的 Rust 穩定版發佈。

中英術語對照表

以下為本書所使用到的常用術語：

English 英文	Traditional Chinese 正體中文	Note 備註
Abstract Syntax Tree	抽象語法樹	參考： 維基百科
ahead-of-time compiled	預先編譯	
annotations	詮釋	
argument	引數	
arity	元數	所需運算元的數量。參考： 維基百科
array	陣列	參考： 維基百科
assignment	賦值	
associated function	關聯函式	
benchmarking	基準化分析法	
best-practice	最佳做法	
bit	位元	參考： 維基百科
block	區塊	
boolean	布林	參考： 維基百科
bounds-check	邊界檢查	參考： 維基百科
borrowing	借用	
borrow checker	借用檢查器	
bug	程式錯誤	
Builder Pattern	建造者模式	參考： 維基百科
byte	位元組	參考： 維基百科
camel case	駝峰式大小寫	參考： 維基百科
clone	克隆	
closures	閉包	
coerce	強制	
collection	集合	參考： 維基百科
command line	命令列	參考： 維基百科
commit	提交	
concurrency	並行	參考： 維基百科

English 英文	Traditional Chinese 正體中文	Note 備註
conditional	條件運算	參考： 維基百科
configuration	配置	
constant	常數	參考： 維基百科
constructor	建構子	
copy	拷貝	
crash	崩潰	亦譯作「當機」，為了避免混淆使用「崩潰」統一指稱，參考： 樂詞網
dangling pointer	迷途指標	參考： 維基百科
data race	資料競爭	
declaration statements	宣告陳述式	
dependencies	依賴	
deque	雙向佇列	Double-ended queue
dereference	取值	即 <code>*</code> 運算子
dispatch	分派	參考： 維基百科
diverging functions	發散函式	不回傳值的函式
edition	版號	
enumerate	列舉	參考： 維基百科
equality	等式	
ergonomics	人因工程	參考： RFC 0198 PR
executable	執行檔	
expression	表達式	
expression-oriented	表達式導向	
expression statements	表達陳述式	
filename extension	副檔名	
handle	控制代碼	參考： 維基百科 、 MSDN
heap	堆積	參考： 維基百科
fault	錯誤	
formalization	正規化	
function	函式	參考： 維基百科
generics	泛型	參考： 維基百科

English 英文	Traditional Chinese 正體中文	Note 備註
hash	雜湊	
hash map	雜湊映射	
identifier	標識符	參考： 維基百科
import	匯入	
index	索引	參考： 維基百科
instance	實體	
iterative	疊代	參考： 維基百科
iterator	疊代器	參考： 維基百科
immutable	不可變	參考： 維基百科
inheritance	繼承	
join	會合	意旨 threads 的會合，參考： RFC 3151 PR
keyword	關鍵字	
language feature	語言特徵	參考： 中華民國資訊學會
library	函式庫	參考： 維基百科
lifetimes	生命週期	
linker	連結器	參考： 維基百科
literal	字面值	
loop	迴圈、循環	參考： 維基百科
macro	巨集	參考： 維基百科
main function	主函式	參考： 維基百科
metadata	詮釋資料	
metaprogramming	超程式設計	參考： 維基百科
method	方法	參考： 維基百科
module	模組	參考： 維基百科
monomorphism	單型	
mutable	可變	參考： 維基百科
mutability	可變性	
mutation	可變數	
namespace	命名空間	參考： 維基百科
nested	巢狀	
Nightly	每夜版	

English 英文	Traditional Chinese 正體中文	Note 備註
operators	運算子	參考： 維基百科
overloading	重載	參考： 維基百科
ownership	所有權	
package	套件	參考： 維基百科
panic	恐慌	
parse	分析、分析語法	
parser	語法分析器	參考： 維基百科
pattern	模式	參考： 維基百科
pattern matching	模式配對	參考： 中華民國資訊學會
placeholder	佔位符	
plugins	外掛	
pointer	指標	參考： 維基百科
polymorphism	多型	參考： 維基百科
primitive type	基本型別	參考： 維基百科
profile	設定檔	
reference	參照、參考	參考： 維基百科
regression	迴歸錯誤	
round bracket	圓括號	參考： 維基百科
runtime	執行時	
scalar	純量	
scope	有效範圍	
section	段落	
semantics	語意	
segment	區段	參考： 維基百科
segmentation fault	記憶體區段錯誤	參考： 維基百科
shadowing	遮蔽	
sibling	同輩	
signed integer	帶號整數	參考： 維基百科
slices	切片	其他資料結構的參考
square bracket	方括號	參考： 維基百科
stack	堆疊	參考： 維基百科
statements	陳述式	

English 英文	Traditional Chinese 正體中文	Note 備註
string	字串	參考： 維基百科
string interpolation	字串插值	參考： MSDN
struct	結構體	參考： 維基百科 、 維基百科
subscript	下標	指的是 <code>a[1]</code> 中的 <code>[1]</code>
symbols	符號	
syntax	語法	
tabs	分頁	
thread	執行緒	參考： 維基百科
trait	特徵	參考： 維基百科
tuple	元組	
two's complement	二補數	參考： 維基百科
type	型別	參考： 維基百科
type inference	型別推斷	參考： 維基百科
unsigned integer	非帶號整數	參考： 維基百科
values	值、數值	
variables	變數	
variant	變體	
vector	向量	
view	視圖	參考： 維基百科
wildcard	萬用字元	
workspaces	工作空間	
zero-cost abstractions	無成本抽象化	

未翻譯

English 英文	Traditional Chinese 正體中文	Note 備註
backtrace		
build		名詞，例：create a "build"
crates		
map		

English 英文	Traditional Chinese 正體中文	Note 備註
master		git branch
prelude		預先載入的函式庫
repository		
Rustaceans		Rust 開發者稱呼自己的常用稱號
shell		